

CS336 学习笔记

Language Modeling from Scratch · Stanford 2026 春
非官方个人笔记 · 根据公开课视频字幕整理 · 图为自绘示意

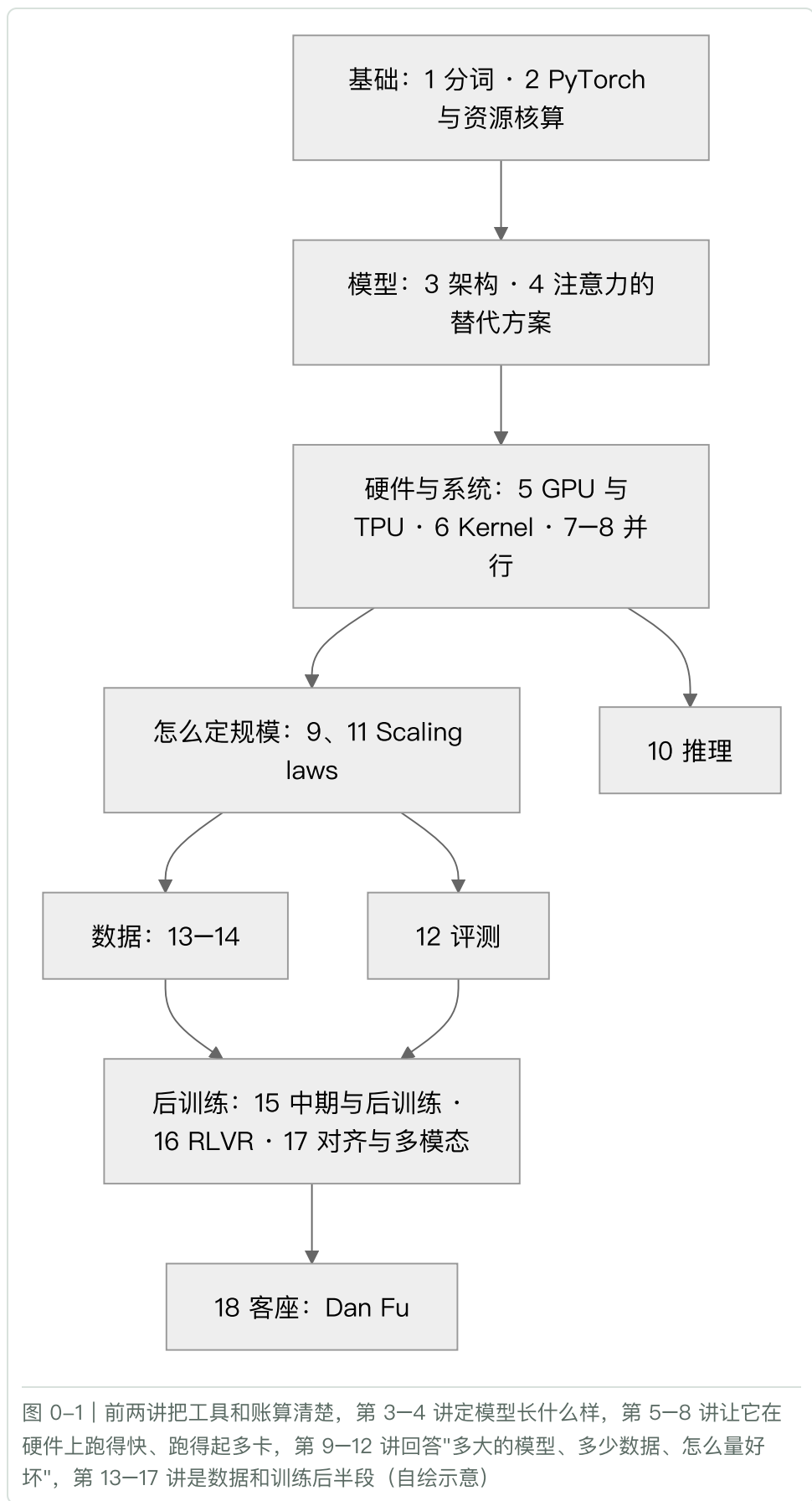
2026-09-24

CS336 学习笔记

Stanford CS336: Language Modeling from Scratch (2026 春) · 讲者 Percy Liang、Tatsunori Hashimoto, 最后一讲客座 Dan Fu 非官方个人笔记, 根据 YouTube 公开课视频的字幕整理 · [播放列表](#) · [课程主页](#)

一句话：这门课把一个语言模型从零造出来，每一步都算清代价——分词 → 用 PyTorch 写模型并核算算力和显存 → 架构与注意力的替代方案 → GPU / TPU 与 kernel → 多卡并行 → scaling laws → 推理 → 评测 → 数据 → 中期与后训练（RLVR）→ 对齐与多模态。CME295 第 1–4 讲讲过的 Transformer、训练、并行、FlashAttention，这里会算到每一个 FLOP 和每一个字节；CS224R 第 9–10 讲和 CS329A 第 6 讲讲过的 RLHF / RLVR，这里讲怎么在自己训的模型上跑起来。

课程地图



各讲索引

讲	主题	时长	视频
第 1 讲	概览与分词：为什么要从头造、效率是组织原则、五个单元的地图、BPE	1:19:22	YouTube
第 2 讲	PyTorch 与资源核算：张量与精度、einops、FLOPs 与 MFU、roofline、6ND、显存账、checkpointing	1:17:25	YouTube
第 3 讲	架构：pre-norm 与 RMSNorm、SwiGLU、RoPE、超参经验值、稳定性三件套、MQA / GQA	1:29:14	YouTube
第 4 讲	注意力的替代方案与 MoE：线性注意力、Mamba 2 / Gated DeltaNet、DSA、MoE 路由与训练、DeepSeek V3 与 MLA	1:26:20	YouTube
第 5 讲	GPU 与 TPU：SM 与内存金字塔、SIMT、TPU、roofline、六个提速技巧、FlashAttention	1:18:39	YouTube
第 6 讲	Kernel 与 Triton：benchmark 与 profiling、torch.compile、Triton、PTX、归约、分块 matmul（标题里的 XLA 未讲）	1:26:41	YouTube
第 7 讲	并行（上）：集合通信原语、NVLink 与 InfiniBand、torch.distributed、数据 / 张量 / 流水线三种切法	1:21:02	YouTube
第 8 讲	并行（下）：ZeRO 三级与 FSDP、pipeline bubble、TP 的前后向对偶、激活显存、EP 与 CP、组合决策	1:20:10	YouTube
第 9 讲	Scaling laws（上）：幂律、数据与模型的 scaling law、batch 与学习率、Chinchilla 对 Kaplan、IsoFLOP	1:17:57	YouTube
第 10 讲	推理：KV cache、瘦矩阵乘的算术强度、GQA / MLA / CLA、量化、剪枝与蒸馏、speculative decoding、continuous batching	1:25:30	YouTube
第 11 讲	Scaling laws（下）：MiniCPM、DeepSeek、StepFun 的超参 scaling law、Muon、 μ P 推导与压力测试	1:17:03	YouTube
第 12 讲	评测：困惑度、六个评测家族、成对比较与裁判、智能体评测、ARC-AGI、训练-测试重叠	1:18:34	YouTube
第 13 讲	数据（上）：网络是什么、版权、Common Crawl、数据集谱系、FineWeb 与 Dolma、DCLM、The Stack、Common Pile	1:22:01	YouTube
第 14 讲	数据（下）：过滤骨架与 fastText 打分、阈值、去重（MinHash / LSH）、配比、后训练的合成数据	1:24:46	YouTube
第 15 讲	中期训练与后训练：SFT 数据谱系、安全 SFT、mid-training、RLHF 偏好数据、PPO 与 DPO 推导、RLHF 三个坑	1:19:54	YouTube
第 16 讲	后训练 RLVR：系统组成、GRPO、DeepSeek R1、Kimi K1.5、数据与 reward、RL infra、Qwen 3、reward hacking	1:15:50	YouTube
第 17 讲	多模态（题目里的「对齐」未讲）：CLIP、SigLIP、LLaVA 与 LLaVA-OneVision、Qwen-VL 系列、Chameleon	1:17:39	YouTube
第 18 讲	客座 Dan Fu：推理引擎、prefill 与 decode、continuous batching、PD 分离、megakernel、Parcae 循环 Transformer	1:11:40	YouTube

和前三门课怎么对照

这门课	前面课里对应的内容	关系
第 1—3 讲 分词、资源核算、架构	CME295 第 1—2 讲	那边讲"是什么", 这边讲"怎么造、多少钱"
第 5—8 讲 GPU、kernel、并行	CME295 第 4 讲的显存与并行	那边一页带过, 这边逐字节、逐 FLOP 算
第 9、11 讲 scaling laws	CME295 第 4 讲 Chinchilla	这边讲怎么拟合、怎么用它定预算
第 10 讲 推理	CME295 第 3 讲的 KV cache 与解码	这边讲吞吐与延迟的账
第 12 讲 评测	CME295 第 8 讲、CS329A 第 8 讲	这边是造模型的人怎么用评测
第 15—16 讲 后训练与 RLVR	CME295 第 5—6 讲、CS224R 第 9—10 讲、CS329A 第 6 讲	那边讲公式和前沿, 这边讲在自己模型上跑通

怎么用这份笔记

- **每讲的结构是固定的**：一句话 → 时间轴 → 核心内容（带图、公式和短伪代码）→ 关键图表速查 → 提到的工作 → 术语对照 → 字幕勘误 → 带走的问题。赶时间可以只读"一句话"、图和"带走的问题"。
- **这门课数字和代码多**：讲者的课件是可执行的 Python，笔记里用自己写的短伪代码概括算法，不照抄屏幕；算力、显存、带宽这些数字按课上的口径照实记，关键公式单独成块。
- **时间戳都能点**，直接跳到视频里对应的位置。图下面黄色的「▶ 看原幻灯片」会跳到老师讲那一页的时刻。
- **图是重新画的示意图**，用来解释机制，不是课件截图。
- "小注" 是补充的课外事实或对口误的更正；"应为 / 应出自" 表示这是根据内容推断的，老师没有明说。
- 想对照原文：在播放器「设置 → 字幕 → 自动翻译 → 中文（简体）」可以开中文字幕。

CS336 第 1 讲 | 概览与分词 (Overview, Tokenization)

Stanford CS336: Language Modeling from Scratch (2026 春, 第三次开课; Percy Liang 与 Tatsunori Hashimoto 轮流授课) · 第 1 讲 视频: <https://www.youtube.com/watch?v=JuoVZkPBiKk> (1:19:22; 英文字幕为自动生成, 人名、项目名错得不少, 见文末勘误) 讲者: Percy Liang (全程) · 开场两分钟由 Tatsunori Hashimoto 与三位 CA (Marcel、Herman、Steven) 各自介绍 课程主页: <https://stanford-cs336.github.io/> (课上给的地址是 cs336.stanford.edu) · 分词部分的源头: Sennrich et al., 2016 (BPE 进入 NLP) · Radford et al., 2019 (GPT-2, 第一个用 BPE 的语言模型)

一句话: 这门课赌的是"只有亲手造过才算真懂", 可前沿模型已经工业化 (一次训练一亿到十亿美元、细节不公开), 而小规模实验并不代表大规模——MLP 的 FLOPs 占比从 44% 涨到 80%、涌现能力过了临界规模才出现——所以课程只承诺教能跨规模迁移的两样东西: 机制 (mechanics) 和心法 (mindset: 效率是第一原则, $\text{accuracy} = \text{efficiency} \times \text{resources}$), 分 basics、systems、scaling laws、data、alignment 五个单元各配一次作业; 第一个单元从分词起步: 字符级、字节级、词级各有致命伤, BPE 靠"反复合并最频繁的相邻对"从数据里学出词表, 把 20 个字节压成 8 个 token (压缩率 2.5), 而任何想取代 tokenizer 的端到端方案都得保住"抽象"和"自适应计算"这两条性质。

时间轴

时间	内容
0:05	教学团队自我介绍; 第三次开课; from scratch 哲学不变, 今年多讲 MoE 与长上下文
3:09	为什么开这门课: 研究者与底层技术脱节; 抽象会漏; 理解靠亲手造
4:42	工业化: GPT-4 一亿美元、如今十亿量级、细节不公开; 小模型不代表前沿 (MLP 占比 44% → 80%、涌现)
7:18	三种知识: mechanics、mindset、intuitions——哪些能跨规模迁移; SwiGLU 论文的"神的恩泽"
8:50	bitter lesson 的正确读法; $\text{accuracy} = \text{efficiency} \times \text{resources}$; ImageNet 算法效率 44 倍
11:21	简史: Shannon、n-gram、神经网络谱系、GPT 系与 scaling、复现潮、开放权重生态与 Marin

时间	内容
17:31	"语言模型是什么"在变：微调 → prompt → 对话 → agent；基础没变；可执行讲义
20:04	课务：5 学分、5 次作业、该不该选、单元测试代替脚手架、AI 使用政策、Modal 算力
27:17	五个单元总览；单元一 basics：分词的效率视角、架构改进清单、训练决策清单；作业 1
34:31	表达力 · 稳定性 · 效率的三角
36:01	单元二 systems：6ND、B200 的 2.25 PFLOP/s 与 8 TB/s、kernel 与算力融合、多卡分片、推理两阶段；作业 2
45:15	单元三 scaling laws：scaling recipe、可预测性、Chinchilla 的 20 倍、Marin 预注册；作业 3
53:28	单元四 data：内部 / 外部评测、数据来源与法律、处理流水线、三个去处；作业 4
1:00:07	单元五 alignment：弱监督、PPO / GRPO / DPO、RL 上规模的系统难题；效率视角串起五个单元
1:04:47	分词入门：往返一致、空格与数字的怪癖、压缩率 $20 / 8 = 2.5$ 、词表 100k–200k
1:08:19	三个朴素方案：字符级 150k 词表、字节级压缩率 1、词级无界词表
1:11:51	BPE：来历、训练算法逐步演示 (the cat in the hat)、编码与解码
1:15:58	作业里怎么做快：只看相关 merge、特殊 token、预分词；小结；tokenizer-free 的两个条件；下讲预告

核心内容

1. 为什么还要"从头造"：三种知识，哪些能跨规模迁移

- **课程的来历**：两年前开课时的判断是，研究者正在与底层技术脱节。十年前每个 AI 研究者都自己实现、自己训模型；八年前是下载 BERT 这类预训练模型再微调；今天多数人靠 prompt 就能干活。prompt 没有错，抽象层级上移也是好事，但**抽象会漏** (leaky)：总会撞上模型做不到、又无从下手的情形。做基础研究时只在 prompt 这一层活动，等于把设计空间砍掉一大截，得能撕开整个栈。所以课程立场是：完整理解 = 亲手造出来。
- **一个麻烦：语言模型已经工业化**。GPT-4 三年前据传训练一次花 1 亿美元，现在的前沿模型大概是 10 亿量级（讲者自称是猜测）；各大实验室囤的 GPU 数量惊人；而且细节不公开——2023 年的 GPT-4 报告明说，出于竞争与安全考虑，不披露模型是怎么建的。

- **我们能造的是小模型，但小模型未必能代表前沿模型。**讲者给了两个证据：
 - 算力去哪了随规模而变：2021 年的一张图里，小模型的 FLOPs 只有约 44% 花在 MLP 层，到 175B 参数时涨到约 80%。在小规模上给 attention 做的优化，到大规模未必有同样收益。
 - 涌现 (emergence)：早年的 zero-shot / few-shot 曲线在小模型上几乎贴地，过了某个临界规模才突然上升。小规模看不到某些现象。
- **那学什么才能迁移？**讲者把知识分三类：
 - mechanics (机制)：Transformer 是什么、模型并行怎么工作。课上能教好，而且跨规模成立——并行和 kernel 为什么能提速，是"按构造"就能看出来的。
 - mindset (心法)：怎么榨干硬件、认真对待 scaling，凡事 profile 和 benchmark。课上能教好，也能迁移。
 - intuitions (直觉)：哪些建模和数据决策有效。未必跨规模迁移，得到有算力的地方去练。
- **关于直觉的一个注脚：**很多设计决策没有解释，纯靠实验。Shazeer 引入 SwiGLU 的那篇论文 (第 3 讲会讲)，结论里坦白地写道不提供任何解释，把这些架构的成功归于"神的恩泽"。

2. 效率是组织原则：bitter lesson 的正确读法

- **常见误读：**规模就是一切，算法不重要。**正确读法：**能随规模放大的算法才重要。讲者给的粗略等式：

$$\text{accuracy} = \text{efficiency} \times \text{resources}, \quad \text{efficiency} = \frac{\text{output}}{\text{input}}$$

accuracy 是模型最后有多好，resources 是投入的算力和数据，efficiency 是单位投入换来多少产出；两者相乘，缺一不可。

- **效率在大规模上反而更关键：**小实验跑慢一倍，等一等就好；上亿美元的训练里，5% 的改进都是大事。
- **实证：**OpenAI 2020 年的论文测得 ImageNet 上 2012—2019 年间算法效率提升 44 倍。硬件同期也大幅进步，两者相乘才是我们看到的飞跃。
- **于是全课的提问方式固定下来：**给定数据和算力预算，能造出的最好模型是什么？预训练里主要盯算力预算——假设数据比算力多；如果你数据受限、或者囤了一屋子 B200，那就是另一套算计。讲者在

1:04:14 补了一句：明天我们可能就数据受限了，具体权衡会变，"想清楚你的方法效率如何"这个心法不变。

3. 语言模型简史，以及"语言模型是什么"在怎么变

- **前神经网络时代**：Shannon 在 50 年代就用语言模型测英语的熵；n-gram 模型长期是机器翻译和语音识别系统里保证输出流畅的一个部件。
- **神经网络谱系**（讲者点名的节点）：90 年代的 LSTM；Bengio 2003 年的第一个神经语言模型（其实是看短上下文的前馈网络）；seq2seq——大胆宣称整句可以压成一个向量；Adam；为机器翻译发明的 attention；同样为机器翻译发明的 Transformer；然后是 MoE、模型并行等 2010 年代的架构、系统与优化器工作。
- **2010 年代末**：ELMo、BERT 在大量文本上预训练、再在问答等下游任务上微调，效果大幅提升；Google 的一篇论文（应为 T5）预示了"prompt 进、回答出"的形态。
- **真正打开闸门的是 OpenAI 拥抱 scaling**：GPT（2018 年前后）→ GPT-2 → 靠 scaling laws 支撑训出比当时大 10 倍以上的 GPT-3，出现 in-context learning 这类涌现行为。Google 随后训了一个巨型模型（应为 PaLM），事后看训练不足；当时还未并入 Google 的 DeepMind 则搞清了 compute-optimal 的 scaling law（Chinchilla）。
- **复现潮**：GPT-3 是很多人的警钟。草根组织 EleutherAI 做了开放数据集和模型（算力少，模型不大）；Meta 的第一个 LLM 一看就是复现——也是 175B（应为 OPT），效果不好，硬件问题不断；还有 Hugging Face 的 BigScience（应为 BLOOM）。这一批都不算强。
- **最近三年生态巨变**：Meta 的 Llama 1 / 2 / 3 领头，Mistral 入局，然后是一整批中国模型——DeepSeek、Qwen，还有 ByteDance、Tencent 的（讲者说记不全）。开放权重模型已经逼近闭源模型，工业界大量使用。
- **更进一步的是"不只开权重"**：AI2、NVIDIA，以及讲者参与的 Marlin 项目，连论文、代码、数据一起放出。讲者强调开放生态是这门课的前提：正是那些讲大 MoE、讲 RL 系统的论文，让我们能拼凑出前沿模型的做法——哪怕 Qwen 这类论文也缺关键细节（尤其数据配比），聊胜于无。
- **"语言模型是什么"十年里换了四次**：拿来微调的东西 → 拿来 prompt 的东西 → 能对话的东西（ChatGPT 时代）→ agent。讲者本想现场

点开一条巨长的 agent trace (没网): 给一页文字, 它就能完成相当复杂的编码任务, 十年前没人敢想。

- **但基础没怎么变:** 还是 GPU 和 kernel, 还是 (随机) 梯度法, 还是 Transformer 和 attention。变的是规格——上下文更长, 推理效率更重要。所以这门课只改了两节, 加进最新的中国模型架构。

4. 课务 (几条)

- **讲义是"可执行的" (19:32):** 每一讲就是一个 Python 程序渲染出的页面, 讲到哪就执行到哪, 可以单步看代码; 讲义的层级就是函数调用的层级。
- **5 学分, 5 次作业, 强度出名:** 某条课评说第一次作业相当于 CS224N 的全部五次作业——讲者觉得夸张, 但建议按保守估计安排时间。
- **该不该选:** 想弄清东西怎么工作、想练研究和工程肌肉, 选; 这个季度要出研究成果、想学最热的新技术 (本课不讲多模态、不深入 agent)、或者只想在某个应用上拿结果 (先 prompt, 再微调, 最后才考虑预训练), 别选。
- **作业哲学:** 不给脚手架代码, 但给单元测试——避免"交上去要么全对要么全错"的稀疏奖励。大部分能在笔记本上做完并验证正确性; 集群 (今年由 Modal 提供算力额度, 走 API 而不是 SSH) 用来跑真正的训练和 kernel benchmark。多数作业有 leaderboard: 给定预算把 perplexity 压到最低。
- **AI 使用政策 (25:13):** coding agent 已经能整份做完作业, 但那样什么也学不到。课程提供一份 agents.md (等价于一段 prompt), 要求把 AI 当"有教学意识的辅导": 答疑、讲代码可以, 替你写 Transformer 不行。今年第一次试行, 欢迎反馈。
- 讲课材料上网, SCPD 录像后发 YouTube; 在家跟读的人得自己找动力做作业。

5. 课程地图: 五个单元、五次作业

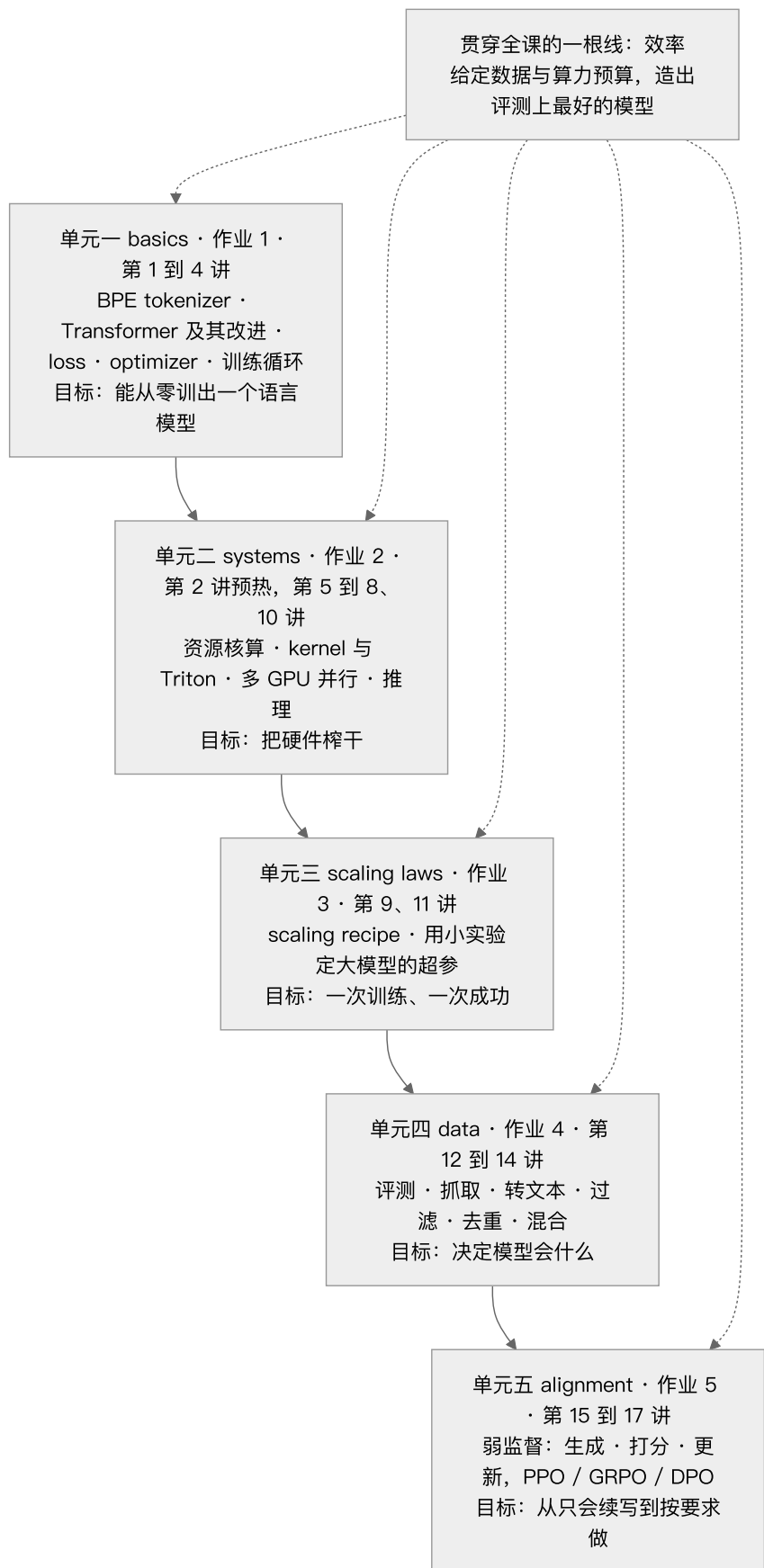


图 1-1 | 五个单元、五次作业，以及对应的讲次（自绘示意）· [▶ 看原幻灯片 27:17](#)

- 五个单元镜像五次作业，顺序是：先能训出来，再让它快，再让它大，再决定喂什么，最后让它听话。讲者用一个词把它们串起来：效率（第 11 节的表）。下面按单元记要点，每个单元后面的讲次会展开。

6. 单元一 basics：把一个语言模型从零训出来

- **目标**：头两周结束时能自己训出一个 LM。三件事：分词、定义架构、实现优化器和训练循环。
- **分词的效率视角**（第 12 节起细讲）：tokenizer 在原始字节和整数序列之间来回转换；BPE 把输入切成高频块。好处一是序列变短；更微妙也更重要的是**自适应计算**——一大段字节可能就该压成一个 token，而稀有、有信息量的部分该留成多个 token。讲者每年都盼着不用再教分词：端到端直接吃字节的方案（最近的 H-Net 看着有希望）还没在前沿规模上验证，前沿模型仍在用 tokenizer，所以还得教。
- **架构**：默认你在 CS224N 见过 Transformer，此后的改进才是重点（第 3 讲由 Tatsu 讲）：激活函数、位置编码、归一化放在哪才不炸、降低 n^2 的 attention 开销（ n 是序列长度；第 4 讲）、状态空间模型 / 线性 attention（Mamba、Gated DeltaNet）以及它们与 attention 的混合、MLP 换成 MoE（如今做算力高效 Transformer 的主流范式，训练方法也得跟着变）、以及看似无聊的形状问题——几层、几个头、隐层多宽、几个 expert——放到 scaling 语境下影响巨大。
- **训练**：loss（默认预测下一个 token；预测多个 token 被发现有帮助）、优化器（Adam 之外，Muon 越来越多，Kimi K2 等最新开放模型在用）、初始化（决定大模型训练稳不稳）、学习率 schedule、正则化、batch size、MoE 特有的设定。讲者的话：这些不是"多试几组"的超参，按原则设定它们，是"跑炸了"与"达到 SOTA"的分界，讲 scaling laws 时会回到这一点。
- **作业 1**：BPE tokenizer、Transformer、loss、优化器、整套训练栈；资源核算（FLOPs 花在哪）；在 TinyStories 和 OpenWebText 上训练；leaderboard 像 NanoGPT speedrun——给定预算，perplexity 压得越低越好。
- **讲者的总括**：tokenizer、模型、训练看似三块，其实都在平衡同一个三角——**表达力**（表示数据的复杂性）、**稳定性**（参数范数和梯度范数保持在"金发姑娘区"，不爆不消失；训 LM 很大一部分工作就是保

稳)、效率（在硬件上跑得快；比如把某处投影到低维能提速，但效果还一样吗？做这种取舍就是这门课的日常）。

7. 单元二 systems：把硬件榨干

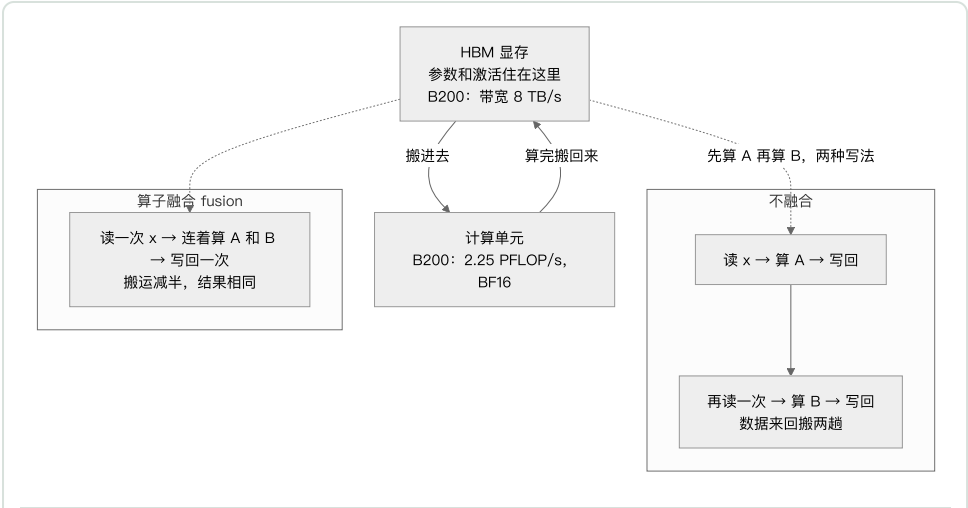


图 1-2 | 内存不在计算单元旁边：数据搬运常是瓶颈，算子融合省的就是这个（自绘示意）· [▶ 看原幻灯片 37:02](#)

- 资源核算（第 2 讲开始）：追踪每一个 FLOP、每一个字节去了哪。会遇到这条公式：

$$\text{FLOPs}_{\text{train}} \approx 6ND$$

N 是参数量，D 是训练 token 数；讲者举的例子是"7B 模型训 1T token 要多少 FLOPs"，第 2 讲推导那个 6 从哪来。

小注：代进去是 $6 \times 7 \times 10^9 \times 10^{12} \approx 4.2 \times 10^{22}$ FLOPs；按下面 B200 的 2.25 PFLOP/s、利用率 100% 算，单卡要 1.9×10^7 秒 ≈ 216 天。这一步课上没算；第 2、5 讲会告诉你实际利用率为什么远低于 100%。

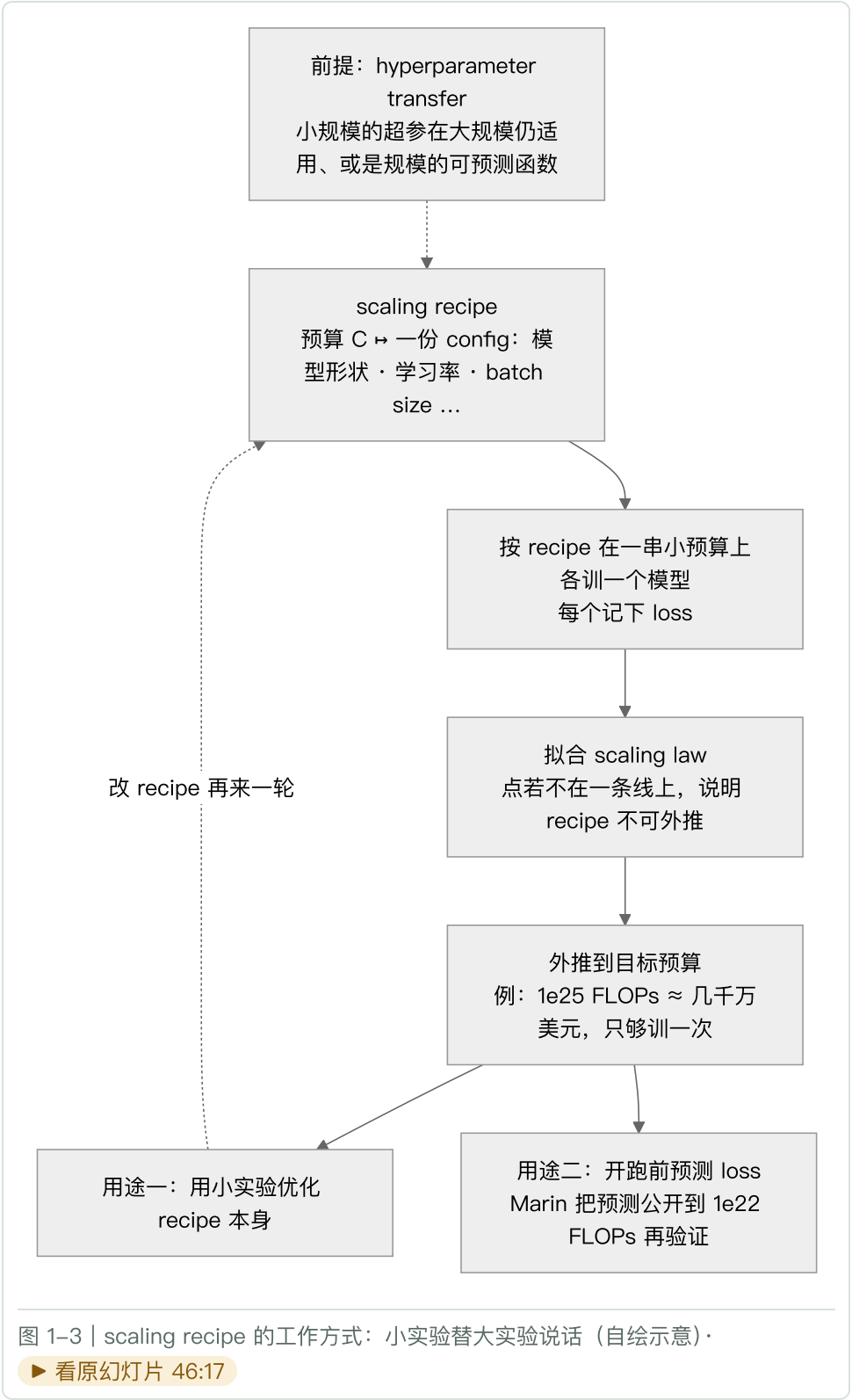
- 硬件的卡通图（图 1-2）：内存不在计算单元旁边。参数或激活得从内存搬到计算单元、算完再搬回去，这一搬常常就是瓶颈。讲者报的 B200 数字：BF16 算力 2.25 PFLOP/s，内存带宽 8 TB/s。第 2 讲会用这两个数估各类算法要跑多久，并讲 roofline 分析——判断一段计算受限于算力还是内存（通常是内存），以及 benchmark 和 profile 的方法。
 - 术语：FLOPs 是浮点运算次数（计算量），FLOP/s 是每秒能多少次（硬件速度）；HBM（high bandwidth memory）是 GPU 上那块大而相对慢的显存；带宽是每秒能搬多少字节。

小注：roofline 的核心量是 arithmetic intensity（算术强度）= 每搬 1 字节能做多少次浮点运算。按讲者给的两个数，B200 的分界点是 $2.25 \times 10^{15} \div 8 \times 10^{12} \approx 281 \text{ FLOP/byte}$ ：一段计算每字节做不到这么多运算，就是内存受限。

- **拓扑**：一台 DGX B200 里 8 张 GPU 由 NVLink 相连；上千张 GPU 就是许多台这样的机器，用 InfiniBand 或以太网连起来。
- **kernel**：在 GPU 上跑的函数。你写的每个 PyTorch 算子背后都在启动某个内置 kernel——不管知不知道，你一直在用。对特定计算写自定义 kernel 能更快，原则只有一条：**组织计算以最小化数据搬运**。例子：先算 A 再算 B，朴素做法是读 HBM → 算 A → 写回 → 再读 → 算 B → 写回，数据来回搬两趟；**算子融合**（fusion）读一次、连着算完、写一次。tiling（分块）是同一思想的更精细版本——CME295 第 4 讲的 FlashAttention 就是它；本课第 5—6 讲会真的写 kernel。GPU 本身也越来越复杂，课上至少让你见识它的“怪脾气”。
- **上千张 GPU**（第 7—8 讲）：原则不变，只是跨 GPU 搬数据更贵。语言是经典的**集合通信**（collective operations）：gather、reduce、all-reduce 等。棋局是：参数、激活、梯度、优化器状态要分片（shard，切开分到多张卡），然后把对的数据送到对的节点去算、再写回。分片方式有好几种——切数据、切模型、切层、切序列、切 expert——各有取舍。
- **推理**（第 10 讲）：用模型的过程，不只是聊天——RL 的 rollout、test-time compute、合成数据、评测全靠它。两个阶段：**prefill** 把整段 prompt 一次前向、建好 key-value 对（很像训练）；**decode** 一次生成一个 token，很快就变成内存受限——这就是推理难的原因。提速手段：剪枝出更小的模型、量化、蒸馏、speculative decoding（小模型先往前猜一串 token，大模型并行验证，猜对就整串接受）、推理专用 kernel。服务场景还有调度问题：请求随机到达要凑 batch，训练里 batch 是预先定好的。讲者说这部分课时不够，是否让大家从零写推理引擎还在讨论。
 - 术语：prefill 阶段算出的每个 token 的 key、value 存下来供后续步骤复用，就是 KV cache；decode 每生成一个 token 都要把全部参数和 KV cache 从显存读一遍，而算的量很小，所以受限于带宽而不是算力。
- **作业 2**：用 Triton 写 kernel、做并行训练；细节可能因 CA 重做系统部分而变。推荐读物（44:12）：Google 的《How to Scale Your

Model》——roofline、Transformer 算术；以 TPU 为主，新加了 GPU 一章。

8. 单元三 scaling laws：一次训练、一次成功



- **情境**：手里有 $1e25$ FLOPs（几千万美元），训什么模型？只够训一个，没法像小规模那样调超参，搞砸就是真金白银打水漂。这是训大模型独有的难题，微调或小实验里没有。
- **概念转换**：别想“训一个模型”，想 **scaling recipe**——从算力预算到一套超参（一份 config 文件）的映射。有了 recipe，就在小预算上跑一串实验、拟合 scaling law、外推到目标预算：

$$\text{recipe} : C \mapsto \theta(C), \quad \hat{L}(C_{\text{target}}) \leftarrow \text{fit}(\{(C_i, L_i)\}_{C_i \ll C_{\text{target}}})$$

C 是 FLOPs 预算， $\theta(C)$ 是 recipe 在该预算下给出的整套超参， (C_i, L_i) 是各个小预算实验测到的 loss， $\hat{L}$ 是对目标预算的预测——recipe 定了，小实验就能替大实验说话。

- **两个用途**：用小实验优化面向大规模的 recipe；开跑前预测能到的 loss（讲者的说法：拿着小实验的曲线去融资）。
- **第二个常见误解**：scaling law 不是自然律，不会自动出现，得靠精心构造 recipe 把它“逼出来”。recipe 必须能外推：规模增大时学习率是常数还是下降、batch size 涨多少，都得定。前提是 **hyperparameter transfer**——小规模用的超参在大规模仍适用、或是规模的可预测函数；如果学习率一会儿 $1e-5$ 一会儿 $1e-4$ ，到了大规模就只能瞎猜。
- **由此的思维转换**：可预测性至少和最优性同样重要。
- **经典结果**：compute-optimal scaling (Kaplan et al. 与 Chinchilla；CME295 第 4 讲见过，本课第 9、11 讲深入) ——对每个预算（课上的图从 $6e18$ 到 $3e21$ FLOPs）扫模型大小、取最优，拟合“预算 $\rightarrow$ 参数量”的曲线；点若落在一条线上才有底气外推。经验法则：

$$D_{\text{opt}} \approx 20 N$$

N 是参数量， D 是训练 token 数：70B 参数配约 1.4T token。数据和架构不同，这个数会变；而且它不算推理成本——现在很多模型故意做小、用远超 compute-optimal 的 token 数训，就是为了推理便宜。

- **Marin 项目的玩法**：**预注册**——先在若干预算上拟合，公开预测到 $1e22$ FLOPs 的 loss，再真训一个从没训过的规模看是否命中。课上那个正在跑，讲者说周三报结果。
- **作业 3**：课程提供一个“训练 API”——你交 config，它返回 loss（后台是离线训好的一批模型做缓存，看上去像在训练）；你拟合 scaling

law、外推，最后给你一个预算，看你的模型落得准不准。模拟"手握 1 亿美元怎么花"的高压场景，只是没有真的赌注。

9. 单元四 data 与评测：决定模型会什么

- 训到这里还差一件事：训在什么上。数据质量基本决定模型多好；数据就是"你想让模型会什么"的具体化——多语言、对话、长程 agentic coding，都得先在数据里。
- **先讲评测**（第 12 讲），因为它定义你要的能力。讲者强调两类指标常被混为一谈，其实各司其职：
 - 内部指标，用于研发：要跨规模平滑（还是为了可预测），看相对值就够——perplexity 1.2 的绝对值没有意义。perplexity 至今仍是衡量模型内在质量的好办法，不怕 benchmaxing（冲榜）。评测数据最好是互联网上没有的，避免污染（contamination）。
 - 外部指标，报给客户或审稿人：讲究生态效度（ecological validity，测的东西像不像真实使用）。
 - 语言模型号称通用，评测就得多样；可以平均成一个数，但平均会把很多不同的东西搅在一起。
- **数据不会从天上掉下来**（第 13–14 讲）：网页要抓、书（如今有争议）、arXiv 论文、GitHub 代码……课上放的是 2021 年 The Pile 的组成图。法律问题真实存在：用版权数据训练算不算合理使用，要不要买授权；大量 GitHub 代码没有 license，是当作宽松许可还是保守处理？
- **数据甚至不是文本**：HTML、PDF、代码目录，得先处理。流水线：转换（非文本 → 文本）、过滤（Common Crawl 里随机一篇往往极差）、去重、多源混合，以及近来火热的合成数据（把真实数据改写成更像下游任务、或更像 Wikipedia 的样子）。
- **数据的三个去处**：预训练；mid-training——预训练末尾放的高质量数据，包括长上下文数据（大代码库、整本书）；post-training——对话、带工具调用的 agentic trace（做 agent 的人应该盯紧这一条）。
- **作业 4**：从原始网页抓取开始，过滤、去重、清洗。讲者称之为"脏活"，但从零造模型就得有全部体验。

10. 单元五 alignment：从全监督到弱监督

- 到此为止全是全监督（预测下一个或后几个 token），模型应该已经不错。再往上要靠**弱监督**：批评比生成容易——不一定能写出"这个

prompt 的正确回答", 但可以说清"什么样算好"。

- **模板**：从模型采样回答 → 打分（人、verifier、LM judge）→ 更新模型使其偏好更好的回答。实例化为各种 RL 算法（PPO、GRPO），或者偏好数据上更简单的 DPO（CS224R 推导过；本课第 15—17 讲）。
- **难处**：RL 不稳、难调（讲者：在座有人应该有切身体会）。他个人偏好是能全监督就全监督，实在不行再上 RL。
- **今年希望讲到的**：RL 上规模是系统问题——推理服务器负责生成 rollout（尤其是对着要执行代码的环境），训练服务器负责更新；worker 一慢就滑向 off-policy，要在 on-policy 程度和吞吐之间不停折中。讲者称之为"一团美妙的乱麻"。
- **作业 5 未定**：去年是实现 DPO 和 GRPO 并在数学 benchmark 上跑通，今年想往更真实的方向推。

11. 效率视角：五个单元各自省的是什么

讲者在 1:02:39 把资源列成一张清单——数据、算力核心、内存、通信带宽——问题只有一个：给定这些，按某个评测造出最好的模型。效率可以是数据效率，也可以是算力效率，五个单元的设计决策都能从这个角度看：

单元	效率对准的是什么	展开的讲次
systems	直接就是算力效率：kernel、并行、推理	第 5—8、10 讲
tokenization	算力效率：以今天的架构，直接吃原始字节太长、太贵	本讲
architecture	多数改动是为了少用显存或少算 FLOPs，很多受"推理要快"驱动	第 3—4 讲
data filtering	算力预算固定时，花在冗余坏数据上的梯度更新，就是从好数据那里抢来的	第 13—14 讲
scaling laws	本质是在小得多的模型上做有效的超参调优	第 9、11 讲

12. 分词：问题定义、怪癖与压缩率

- **起点**：原始文本是 Unicode 字符串；语言模型是在 token 序列（整数索引）上放的分布。tokenizer 就是一对过程：encode 把字符串变成 token，decode 变回去，而且必须往返一致：

$$\text{decode}(\text{encode}(s)) = s$$

s 是任意输入字符串；实现的 tokenizer 若不满足这条，就是有 bug。

- CME295 第 1 讲已经见过 BPE 的概念；这里往下走一层：算法本身、压缩率怎么算、工程上怎么做快。讲者推荐 Karpathy 的分词视频。
- **现成 tokenizer 的两个怪癖**（讲者说这就是大家想除掉 tokenizer 的原因）：
 - 词和"空格 + 词"是两个毫不相干的 token：hello 和带前导空格的 hello 索引完全不同。很多 token 都是带前导空格的词——能用，但别扭。
 - 数字：有的 tokenizer 每几位一个 token，切法有时可预测有时不；有的每一位一个 token，可预测但 token 数暴涨。取舍。
- **压缩率的定义**：

$$\text{compression ratio} = \frac{\# \text{ bytes}}{\# \text{ tokens}}$$

分子是字符串的 UTF-8 字节数，分母是 token 数，单位是"字节每 token"；课上用 GPT-5 的 tokenizer 编码一个 20 字节的字符串得到 8 个 token，压缩率 $20 / 8 = 2.5$ 。

- 压缩率越大，序列越短——attention 是二次的，序列短就是钱。加大词表能提高压缩率，但词表里每个元素彼此独立，词表越大越稀疏（每个 token 见到的次数越少）。现在的 tokenizer，尤其多语言的，词表在 100k–200k 之间。

13. 三个朴素方案，各有致命伤

方案	词表大小	压缩率（字节 / token）	讲者指出的问题
字符级：一个 Unicode 字符一个 token	约 150k	一般	大量字符极少出现，词表用得 not 划算
字节级：UTF-8 的一个字节一个 token	256	恰好 1	序列太长
词级：按空格或正则切块	训练集里不同块的数量，实际无界	好	没见过的词只能给 UNK；搞乱 perplexity
BPE：合并高频相邻对	自选，今天常见 100k–200k	课上例子 2.5	见第 14–15 节的工程问题

- **字符级**：Unicode 字符串本来就是字符序列，每个字符对应一个整数（Python 的 ord），直接当 token。约 150k 个 Unicode 字符，词表就是 150k——不算离谱，但大量字符极少出现，词表用得很不划算，压缩率也一般。

- **字节级**：Unicode 有 UTF-8 编码——a 是 1 个字节，有的字符是好几个字节。序列变长，但每个值都在 0—255 之间，词表只有 256。压缩率恰好是 1：序列太长。

小注：UTF-8 是变长编码：ASCII 1 字节，多数汉字 3 字节，emoji 4 字节；"字符" (code point) 与"字节"不是一回事，所以字节级序列比字符级更长。Unicode 16.0 定义了约 15.5 万个字符，和课上说的 150k 对得上。

- **词级**：NLP 的老做法——按空格或正则切块，每块一个 token。好处是每个 token 有稳定语义（词是人发明的）；坏处是词表 = 训练数据里不同块的数量，很大，而且实际上无界：测试时来一个没见过的词只能给 UNK，很丑，还会搞乱 perplexity 的计算。
- 两个很糟、一个半糟。BPE 要同时做到：词表大小可控、压缩率好、任何输入都能编码。

14. BPE：从数据里学出词表

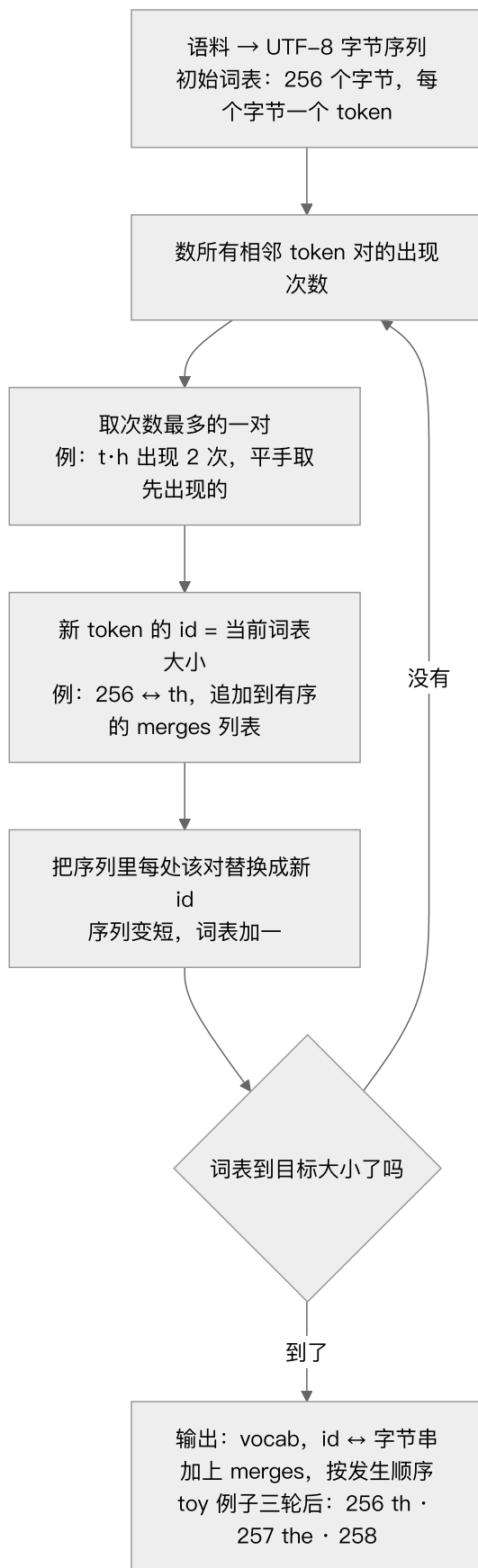


图 1-4 | BPE 训练循环：每轮合并最频繁的相邻对（自绘示意）·

► 看原幻灯片 1:13:23 · 出处：Sennrich et al., 2016

- **来历：**BPE 很早就是数据压缩算法（应为 Gage, 1994），远早于语言模型；Sennrich 等人 2016 年把它引入 NLP 做神经机器翻译；第一个用 BPE 的语言模型是 GPT-2。
- **思想：**在原始文本上训练 tokenizer，构造一份为这批数据量身定做的词表；同时保证任何东西都能被编码——稀有的就拆成更小的单元，而不是 UNK。常见序列一个 token，稀有序列多个 token。
- **算法（图 1-4）：**语料视为一条长序列 → 转成字节，每个字节初始是一个 token（词表 256）→ 反复：数相邻 token 对的频次，取最频繁的一对，合并成一个新 token（id 从 256 起），把序列里所有该对替换掉。序列越来越短，词表越来越大，停在目标词表大小。
- **课上的 toy 例子** "the cat in the hat"：字节序列里 (116, 104) 即 t·h 出现 2 次（有并列，取第一个）→ 新 token 256 代表 th，两处替换；下一轮 (256, 101) 即 th·e 合并成 257；再合并一次得到 258。三轮之后压缩率 1.5。

```
def train_bpe(text: bytes, num_merges: int):
    ids = list(text)                                # 每个字节一个
    token, id 0-255
    vocab = {i: bytes([i]) for i in range(256)}
    merges = []                                     # 有序：编码时按同
    样顺序套用
    for _ in range(num_merges):
        counts = Counter(zip(ids, ids[1:]))         # 相邻对计数
        pair = max(counts, key=counts.get)          # 最频繁的一对（平
    手取先出现的）
        new_id = len(vocab)                         # 256, 257, ...
        vocab[new_id] = vocab[pair[0]] + vocab[pair[1]]
        merges.append((pair, new_id))
        ids = replace_pair(ids, pair, new_id)       # 全序列替换，序列
    变短
    return vocab, merges
```

训练循环的骨架：每轮一次全序列计数、一次全序列替换，这也正是它慢的原因（作业要优化的就是这两步）。

- merges 列表是有序的，编码时要按同样顺序套用；merge 的次数就是词表大小减去 256。

小注：GPT-2 的词表 50,257 = 256 个字节 + 50,000 次 merge + 1 个特殊 token，正好对上这个算式。

15. 编码、解码，与作业里怎么把它做快

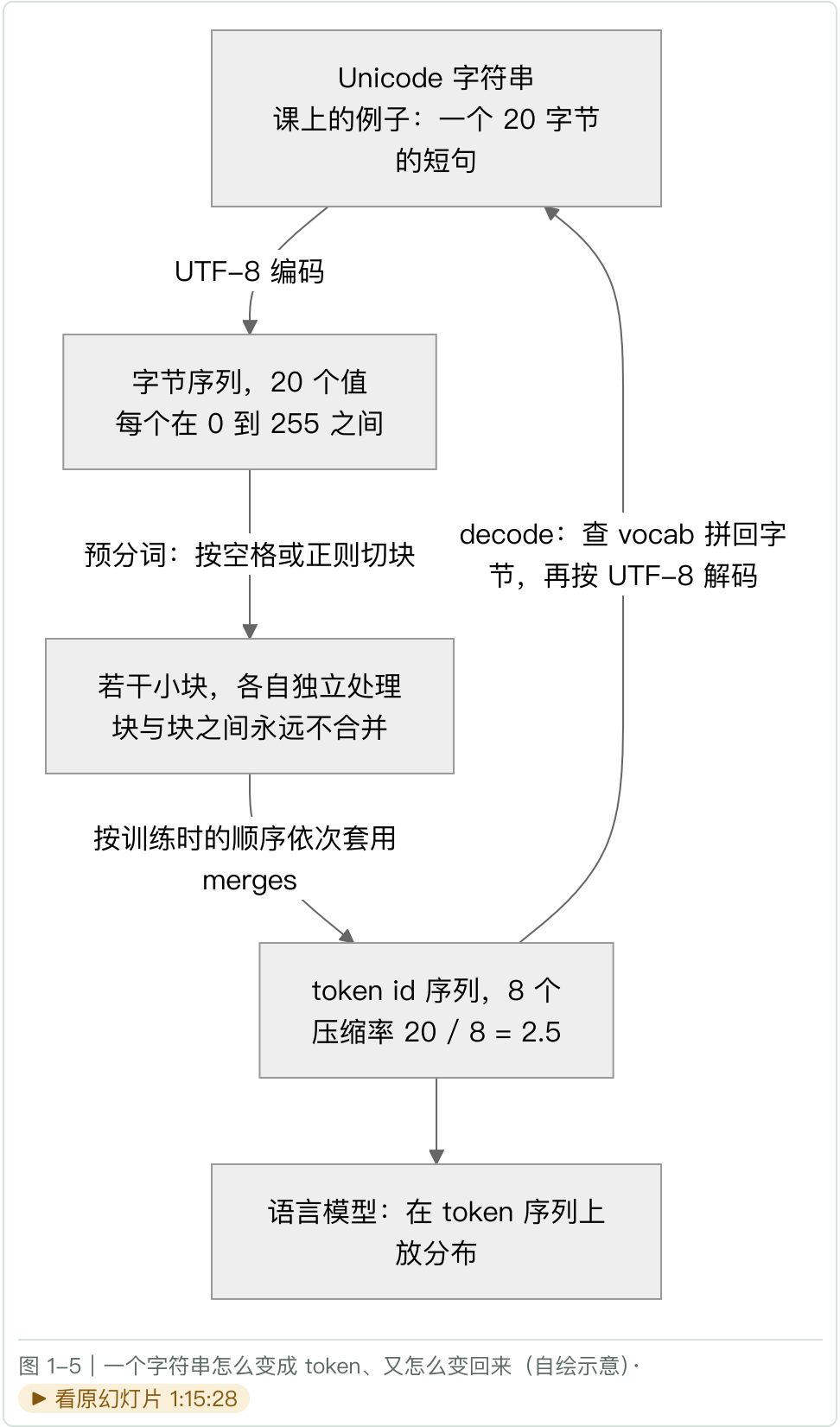


图 1-5 | 一个字符串怎么变成 token、又怎么变回来（自绘示意）·

► 看原幻灯片 1:15:28

- **编码新文本：**概念上就是把训练时记下的 merges 按顺序在新字符串上过一遍；解码把每个 id 查回字节串、拼起来、按 UTF-8 解出字符串。课上 "the quick brown fox" 往返无损。

```
def encode(s: str, merges, pretokenize) -> list[int]:
    out = []
    for chunk in pretokenize(s):
        # 先切块：块内才做合并
        ids = list(chunk.encode("utf-8"))
        for pair, new_id in merges:
            # 严格按训练顺序
            ids = replace_pair(ids, pair, new_id)
        out += ids
    return out

def decode(ids: list[int], vocab) -> str:
    return b"".join(vocab[i] for i in ids).decode("utf-8",
errors="replace")
```

朴素的 encode 对每一块都遍历全部 merges，这正是讲者说的慢点；decode 只是查表拼接。

- 课上这份实现是完整的 BPE，但极慢。作业 1 的任务就是加速，讲者给的方向：
 - encode 目前遍历全部 merges（约等于词表大小减 256），绝大多数用不上；只碰“可能出现在这段文本里”的 merge，得建索引。
 - 特殊 token（如文档分隔符）：概念不深，但现代 tokenizer 必须处理。
 - 预分词（pre-tokenization）：课上为简化把整个字符串当一段来处理；实际是先把文本切成块，对每块独立分词，快得多。
 - 到某一步你会发现 Python 本身就慢——想用 Rust 或 C 重写，随意。

小注：预分词也是建模假设：块与块之间永远不会合并，所以词表里不会出现跨空格的 token；GPT-2 用一条正则把文本切成“空格 + 字母串”、数字串、标点等块。这就是第 12 节“带前导空格的词是独立 token”的来源。

16. 小结：tokenizer-free 的方案要接班，得满足什么

- **总结**：tokenizer 在字符串和 token 之间转换；字符级、字节级、词级各以各的方式次优；BPE 是数据驱动的有效启发式。
- **讲者的预告**：也许明年就不用教了。但无论什么方案取代 tokenizer，都得保留两个性质：
 1. **抽象**：Transformer 得在序列的某种抽象之上工作。看视频或 DNA 序列最明显——单个字节 / 单元信噪比极低，必须先“抬”到一个能建模的层次。

2. 自适应计算：块的大小必须可变，不是所有字节都值得同样的算力；做不到这一点就是次优。

- 下一讲（周三）：资源核算，讲者称之为"婴儿版系统课"；之后回到架构。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
MLP 的 FLOPs 占比随规模变化	横轴模型大小；MLP 份额从约 44% 升到 175B 时的约 80%，attention 份额相应缩小	6:00	—（讲者说是 2021 年的图）
涌现曲线	zero / few-shot 准确率在临界规模之前贴地，之后陡升	6:30	Wei et al., 2022（应为）
SwiGLU 论文的结论段	最后一句：不提供解释，归于神的恩泽	8:50	Shazeer, 2020
ImageNet 算法效率	2012–2019 达到同一精度所需算力降 44 倍	10:21	Hernandez & Brown, 2020
开放模型生态	Llama、Mistral、DeepSeek、Qwen — 栏；再看 AI2 / NVIDIA / Marin"连数据一起开"	15:27	—
可执行讲义	讲义就是 Python 程序：单步执行、函数层级	19:32	—
硬件卡通图	内存与计算单元分离；B200 的 2.25 PFLOP/s 与 8 TB/s 两个数	37:02	—
算子融合示意	不融合来回搬两趟 vs 融合读一次写一次	39:05	—
prefill / decode	两个阶段；decode 一步一个 token、内存受限	42:38	—
Chinchilla 曲线	每种预算一条曲线取最低点；最低点连成线才可外推；预算 6e18 到 3e21	49:54	Hoffmann et al., 2022
Marin 预注册	多预算拟合后外推到 1e22 FLOPs 的预测点	50:57	Marin
The Pile 数据组成	2021 年语言模型数据集的多样性：网页、书、论文、代码	57:03	Gao et al., 2021
tokenizer 示例	20 字节、8 个 token、压缩率 2.5；hello 与带空格的 hello 索引不同	1:06:49	—
BPE 逐步演示	116·104 计数为 2 → 新 token 256 → 序列替换 → 257、258	1:13:23	—

提到的工作

名称	在本讲里的作用
GPT-4 技术报告 (OpenAI, 2023)	"出于竞争与安全考虑不披露细节"的出处；训练成本据传 1 亿美元
2021 年的 MLP / attention FLOPs 占比图	小规模 44%、175B 时 80%：小模型不代表大模型的证据一（讲者未报出处）
Emergent Abilities (Wei et al., 2022)	涌现曲线：证据二（应为这篇）
GLU Variants Improve Transformer (Shazeer, 2020)	SwiGLU 的来源；结论里"归于神的恩泽"，说明直觉靠实验
The Bitter Lesson (Sutton, 2019)	常被误读；正解是"能 scale 的算法才重要"
Measuring the Algorithmic Efficiency of Neural Networks (Hernandez & Brown, 2020)	ImageNet 2012–2019 算法效率 44 倍
Shannon (1950 年代) · n-gram 模型	前神经网络时代：测英语的熵；MT 与语音识别的部件
Bengio et al., 2003	第一个神经语言模型（前馈网络、短上下文）
LSTM · seq2seq · Adam · attention · Transformer · MoE	神经语言模型谱系上的节点（讲者逐个点名）
ELMo · BERT	"预训练 + 微调"时代
T5 (推断)	Google 那篇"预示 prompt 进、回答出"的论文
GPT · GPT-2 · GPT-3	OpenAI 拥抱 scaling：GPT-3 比当时大 10 倍以上，涌现出 in-context learning
PaLM (应为) · Chinchilla	Google 训练不足的巨型模型；DeepMind 的 compute-optimal scaling law
EleutherAI · OPT (应为) · BLOOM (应为)	GPT-3 之后的早期复现：草根开放数据集与模型；Meta 的 175B；BigScience
Llama 1 / 2 / 3 · Mistral · DeepSeek · Qwen · ByteDance · Tencent	近三年的开放权重生态，逼近闭源模型
OLMo (AI2) · NVIDIA · Marin	连论文、代码、数据一起开放；Marin 是讲者参与的项目，还做 scaling law 预注册
H-Net (Hwang, Wang & Gu, 2025)	端到端吃字节、动态分块的 tokenizer-free 尝试，尚未在前沿规模验证
Mamba · Gated DeltaNet	状态空间模型 / 线性 attention；常与 attention 混合使用
多 token 预测 (应为 Gloeckle et al., 2024)	比预测下一个 token 更有帮助的 loss 变体

名称	在本讲里的作用
Muon · Kimi K2	Adam 之外越来越常用的优化器；Kimi K2 等最新开放模型在用
TinyStories · OpenWebText · NanoGPT speedrun	作业 1 的训练集与 leaderboard 的原型
NVIDIA B200 · DGX B200 · NVLink · InfiniBand	2.25 PFLOP/s、8 TB/s；8 卡一机；千卡互联
Triton	作业 2 写 kernel 的语言
How to Scale Your Model (Google, 2025)	推荐读物：roofline、Transformer 算术；TPU 为主，新增 GPU 章
Kaplan et al., 2020	compute-optimal scaling 的经典之一
The Pile (Gao et al., 2021)	数据来源多样性的示例图
Common Crawl	随机一篇往往极差：过滤的必要性
PPO · GRPO · DPO	弱监督模板的三种实例化
Modal	今年提供算力额度的平台，走 API 不走 SSH
CS224N	假定你在那里见过 Transformer；一条课评说作业 1 相当于它的五次作业
Karpathy 的分词视频	推荐观看
GPT-5 的 tokenizer	压缩率示例：20 字节 → 8 个 token
BPE (应为 Gage, 1994)	原本是数据压缩算法
Sennrich et al., 2016	BPE 进入 NLP：神经机器翻译
GPT-2 (Radford et al., 2019)	第一个用 BPE 的语言模型

术语对照

English	中文
from scratch	从零开始：自己实现每一个部件
leaky abstraction	会漏的抽象：上层接口挡不住底层细节
mechanics / mindset / intuitions	机制 / 心法 / 直觉：三类知识，前两类跨规模迁移
emergence	涌现：过了临界规模才出现的能力
the bitter lesson	苦涩的教训：正解是"能 scale 的算法才重要"
efficiency	效率：产出除以投入
executable lecture	可执行讲义：讲义本身是一个 Python 程序
scaffolding code	脚手架代码：作业不提供
sparse reward	稀疏奖励：这里指作业只有全对或全错的反馈

English	中文
leaderboard	排行榜：给定预算比 perplexity
tokenizer / token / vocabulary	分词器 / 词元 / 词表
adaptive computation	自适应计算：不同片段分到不同数量的 token
resource accounting	资源核算：追踪每个 FLOP 和字节去向
FLOPs / FLOP/s	浮点运算次数 / 每秒浮点运算次数
HBM (high bandwidth memory)	高带宽显存：GPU 上大而相对慢的那块内存
memory bandwidth	内存带宽：每秒能搬多少字节
roofline analysis	屋顶线分析：判断计算受限于算力还是内存
arithmetic intensity	算术强度：每搬 1 字节做多少次浮点运算
compute-bound / memory-bound	算力受限 / 内存受限
kernel	核函数：在 GPU 上运行的函数
operator fusion	算子融合：读一次、连着算、写一次
tiling	分块
collective operations (gather / reduce / all-reduce)	集合通信：多卡之间收集 / 归约 / 全归约
sharding	分片：把参数、激活、梯度、优化器状态切开分到多卡
data / model / pipeline / sequence / expert parallelism	切数据 / 切模型 / 切层 / 切序列 / 切 expert 的并行
prefill / decode	预填充（整段 prompt 一次前向） / 解码（一次一个 token）
KV cache	键值缓存：prefill 算出的 key、value 存下来复用
speculative decoding	投机解码：小模型先猜，大模型并行验证
pruning / quantization / distillation	剪枝 / 量化 / 蒸馏
batching	凑批：把随机到达的请求合并处理
scaling recipe	规模配方：从算力预算到整套超参的映射
hyperparameter transfer	超参迁移：小规模超参在大规模仍适用或可预测
compute-optimal	算力最优：预算固定时参数量与 token 数的最佳配比
pre-registration	预注册：先公开预测，再跑实验验证
perplexity	困惑度：内部评测的主力指标
benchmaxing	冲榜：专为 benchmark 分数优化
contamination	污染：评测数据混进了训练数据
ecological validity	生态效度：评测像不像真实使用

English	中文
deduplication / data mixture	去重 / 数据配比
synthetic data	合成数据
mid-training / post-training	中期训练 / 后训练
weak supervision	弱监督：不给正确答案，只给"什么算好"的信号
verifier / LM judge	验证器 / 模型裁判
rollout / on-policy / off-policy	采样轨迹 / 同策略 / 异策略
Unicode / code point	统一码 / 码位：每个字符对应的整数
UTF-8	Unicode 的变长字节编码：一个字符 1-4 字节
byte	字节：0-255 的整数
compression ratio	压缩率：字节数除以 token 数
round trip	往返一致：编码再解码得回原串
UNK token	未知词标记
BPE (byte pair encoding)	字节对编码
merge	合并：把一对相邻 token 换成一个新 token；也指记录下来那条规则
pre-tokenization	预分词：先把文本切成块，块内再做 BPE
special tokens	特殊 token：如文档分隔符
Goldilocks zone	金发姑娘区：参数与梯度范数不爆不消失的区间

字幕勘误

"CS 33336" → CS336；"Tatsus" → Tatsu（Tatsunori Hashimoto）；"Luther" → EleutherAI；"Moraine / Marine project" → Marin；"Kimmy K2" → Kimi K2；"H net" → H-Net；"Joshua Bengio" → Yoshua Bengio；"seek-to-see" → seq2seq；"cgoe" → SCPD；"Deep Seek" → DeepSeek；"MOEs" → MoEs；"NV link" → NVLink；"beta sets" → datasets；"computer optimal" → compute-optimal；"the computer is being spent" → compute；"genetic traces" → agentic traces；"verify or LM judge" → verifier or LM judge；"archive papers" → arXiv papers；"called from the internet" → crawled；"ecologically validity" → ecological validity；"bench maxing" → benchmaxing；"ORD" → ord；"Claude code" → Claude Code；"modal" → Modal；"unk token" → UNK

token; "conglomerate with its preceding space" → concatenated with its preceding space; "sent sequence" → sequence。

带走的问题

1. 用讲者的三分法给自己的经验分类：你在小规模上验证过的哪条结论属于 intuition（可能不迁移）、哪条属于 mechanics？MLP 的 FLOPs 占比从 44% 到 80%，意味着小规模上对 attention 的优化到大规模会怎样？
2. 按 6ND 算 $7B \times 1T$ 的 FLOPs，再除以 B200 的 2.25 PFLOP/s，得到单卡多少天？这假设了 100% 利用率——第 2、5 讲的 roofline 会告诉你，为什么真实利用率取决于每字节能做多少次运算。
3. 压缩率越大越好吗？把词表从 200k 加到 1M，哪些成本上升（embedding 矩阵、输出 softmax、低频 token 训练不足）？"每位数字一个 token"和"每几位一个 token"各自对算术能力意味着什么？
4. 预分词让块与块之间永远不合并——这为什么既是加速手段又是建模假设？对没有空格的中文、对靠缩进表达结构的代码，会带来什么后果？
5. tokenizer-free 的两个必要条件（抽象、自适应计算），H-Net 这类动态分块方案是怎么满足的？如果模型直接看字节，序列长了几倍，attention 的 n^2 代价怎么办（联系第 4 讲的 attention 替代方案）？

CS336 第 2 讲 | PyTorch 与资源核算 (PyTorch, einops)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 2 讲 视频: <https://www.youtube.com/watch?v=kuYAsz7zspQ> (1:17:25; 英文字幕是人工上传的 CC 轨, 不是自动生成, 专名基本准确, 少数数字和口误见文末勘误) 讲者: Percy Liang (全程一人; 开场顺带报告了 Marin 项目 1e23 FLOPs 训练的结果, 结尾预告下一讲由 Tatsunori Hashimoto 讲架构) 课程主页: <https://stanford-cs336.github.io/> · 本讲不围绕某篇论文, 主线是 einops 和 NVIDIA H100 规格表上的几个数; 幻灯片是在讲者没有 GPU 的笔记本上执行过的 Python 代码, 涉及 GPU 的部分只展示不运行

一句话: 这一讲不讲 ML 技巧, 只教一套记账法——一切都是张量, 显存 = 元素数 $\times$ 每元素字节数 (参数、激活、梯度用 bf16, 优化器状态留 fp32, Adam 训练时每参数常驻 $2 + 2 + 4 + 4 = 12$ 字节, 8 张 H100 的 640 GB 最多装 53B); 计算量按 FLOPs 数, 一次矩阵乘法 $2 \cdot B \cdot D \cdot K$, backward 是 forward 的两倍, 合起来就是 6ND (70B 模型、15T token、1024 张 H100 要 143 天); 硬件的 FLOP/s 要读规格表脚注 (H100 的 1979 TFLOP/s 带稀疏, 稠密只有 989), 再打一个 MFU 约 0.5 的折扣; 算子快不快看算术强度——每搬一个字节做多少 FLOPs——H100 的门槛约 295, 只有大矩阵乘法能过线; 最后用 einops 把矩阵乘写得不出错, 用 gradient accumulation 和 activation checkpointing 省显存换更大的 batch。

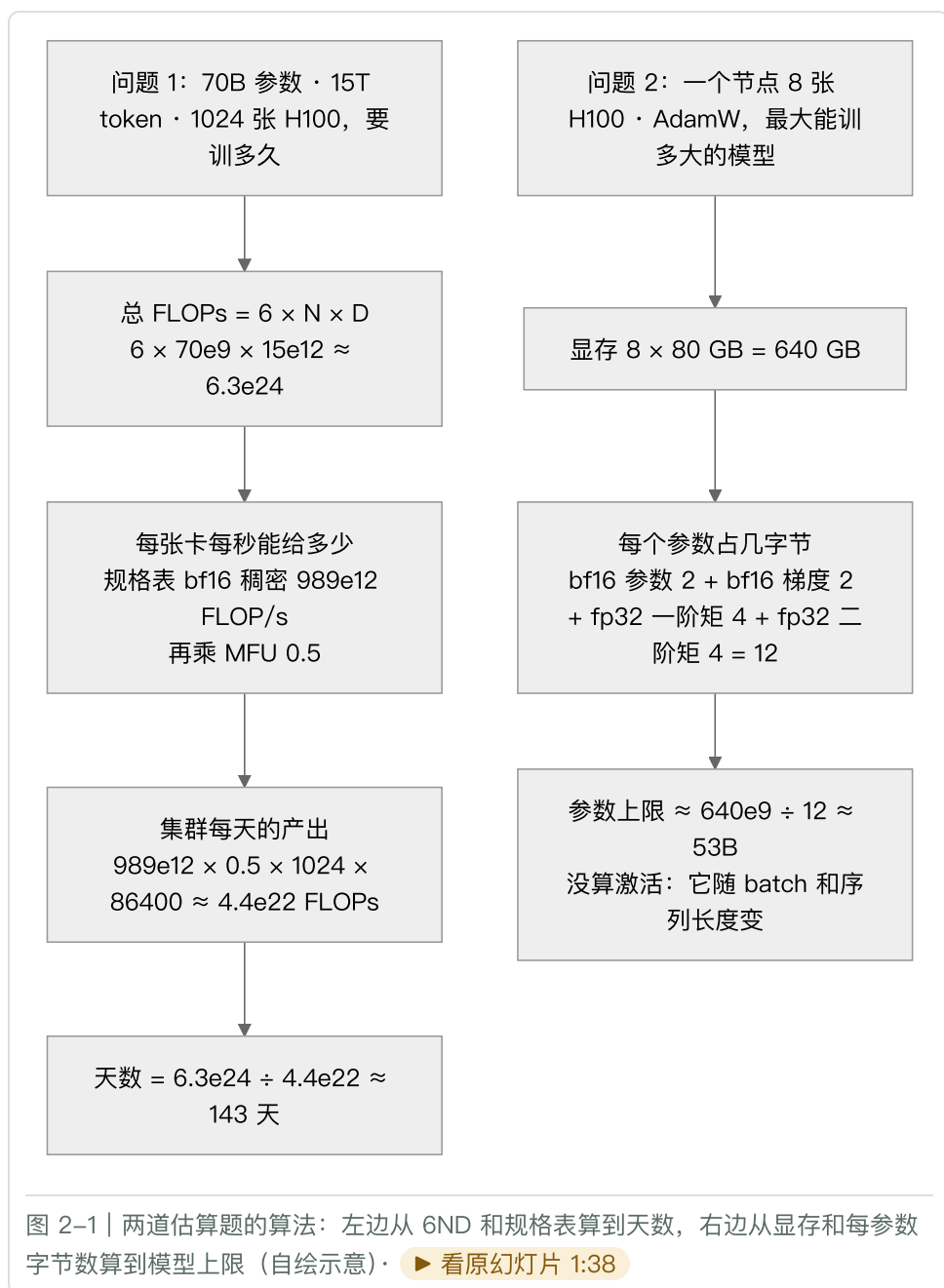
时间轴

时间	内容
0:05	开场: Marin 的 1e23 FLOPs 训练跑完了, loss 与 scaling law 的预测差不到 0.05
1:08	本讲主题: 资源核算; 两道估算题——70B 模型在 1024 张 H100 上训 15T token 要 143 天; 8 张 H100 用 AdamW 最多训 53B
3:47	三种收获: mechanics (PyTorch 怎么工作, 没有魔法)、mindset (每写一行代码想它的性能)、intuitions
4:49	一切都是张量; float32 的 1 / 8 / 23 位分配; 4×8 矩阵 128 字节, GPT-3 一个 FFN 矩阵 2.3 GB
8:30	float16 的动态范围不够 (1e-8 变 0, 训练出 NaN); bfloat16 把位数挪给指数; 混合精度与 AMP
13:16	fp8 有两种变体; nvfp4 只有 -6 到 6 的 16 个值, 靠按块缩放; Nemotron 3 Super 用 fp4 训练; 问答: 1-bit 是推理量化的事

时间	内容
17:52	einops: 下标换成名字; einsum 就是带记账的广义矩阵乘, 批量维用 ... 省略, transpose 消失
22:37	reduce 与 rearrange: 拆维、乘、再合回去; 问答: 没有加速, 只是语法糖; 行优先按写的顺序
27:20	FLOPs 与 FLOP/s; H100 的 1979 TFLOP/s 是带稀疏的, 稠密要除以 2; 8 张卡跑一周约 $5e21$ FLOPs
30:26	矩阵乘法 $x \cdot w$ 的 FLOPs = $2 \cdot B \cdot D \cdot K$; 逐元素运算只有 $m \cdot n$; 问答: 次立方算法在实践里不重要
34:37	实测要 <code>cuda.synchronize</code> ; MFU = 实际 $\div$ 承诺; 0.5 就该满意, 0.8 只有纯矩阵乘能到, 0.1 说明出问题
40:28	硬件卡通图: HBM 与计算核心之间 3.3 TB/s; ReLU 的搬运时间 $1e-6$ s、计算 $1e-9$ s; 内存受限与算力受限
45:45	算术强度: H100 每字节约 295 FLOPs; ReLU 0.25、GELU 5、点积 0.5、矩阵乘向量 1、矩阵乘法 $n/3$
52:36	Transformer 靠大矩阵乘法过线; 推理逐 token 生成是矩阵乘向量, 天然内存受限; roofline 图; 问答: 为什么加速器算力过剩
57:17	运行例子: L 层 $D \times D$ 的深度网络; autograd 的 <code>.grad</code> ; 用 einsum 写出一层的 forward 与两个 backward
1:05:52	forward $2 \cdot B \cdot P$ 、backward $4 \cdot B \cdot P$, 合计 $6 \cdot B \cdot P$ ——6ND 的来历; 上下文太长要加平方项; AdaGrad 的优化器状态
1:09:28	显存核算: 参数 2、梯度 2、优化器状态 4 (AdaGrad) 或 8 (Adam) 字节每参数; 激活 $2 \cdot B \cdot D \cdot L$; 优化器状态为什么必须 fp32
1:12:06	省显存的两招: gradient accumulation 与 activation checkpointing (每 $\sqrt{L}$ 层留一个 checkpoint); 总结

核心内容

1. 开场: Marin 的预测兑现了, 以及这门课要教你算的两道题



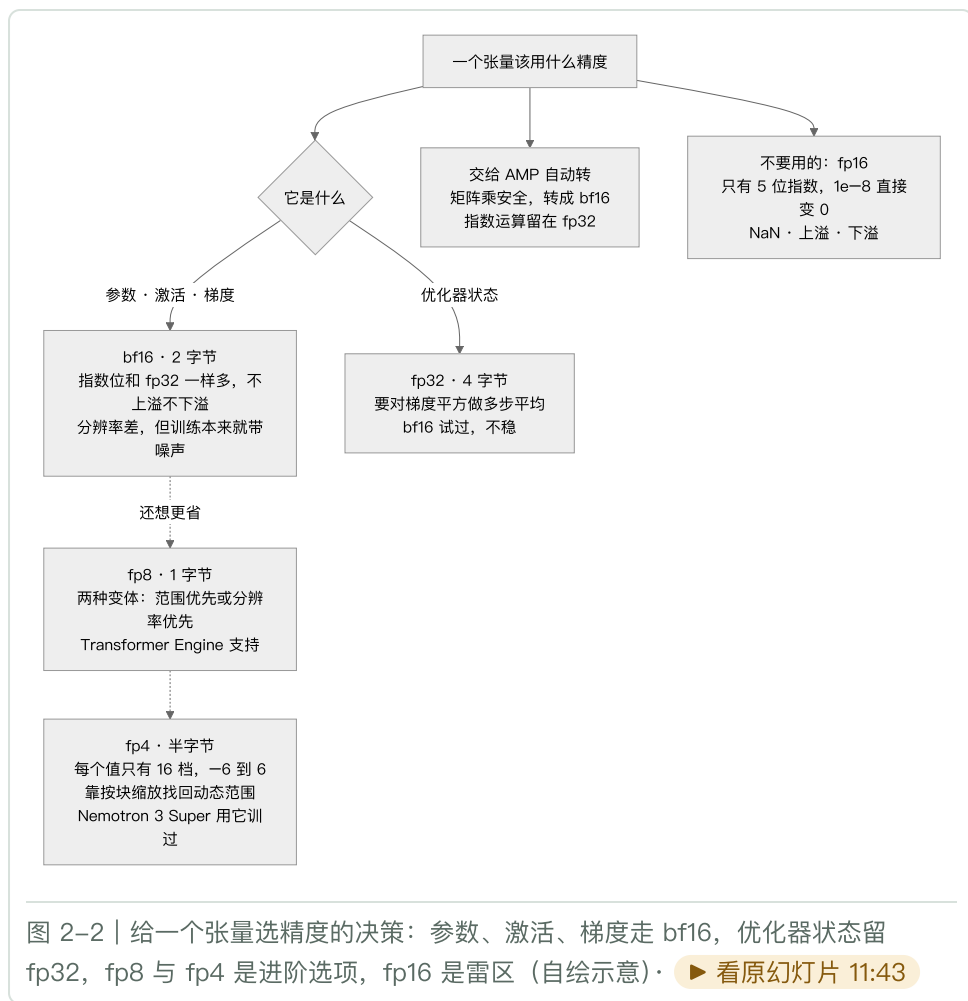
- **Marin 的消息:** 上一讲提到 Marin 项目在跑一个 $1e23$ FLOPs 的训练, 现在跑完了, 和预报对上了。做法是第 9 讲的那一套: 用一批小模型跑出 IsoFLOP 曲线 (同一算力预算下换不同的模型大小), 找到 compute-optimal 的点, 拟合 scaling law, 预测大模型的 loss——实际与预测差不到 0.05。沿同一条线外推, 还能读出 GPT-5 级别的表现对应什么 loss; 讲者提醒: 外推准不准, 全看 scaling law 本身靠不靠得住。

小注: Marin 是 Percy Liang 主持的开放式基础模型实验室 (<https://marin.community/>), 训练过程与实验记录全部公开。

- **本讲的位置**：第 1 讲是总览加 tokenization（作业 1 的内容），今天转到系统这一侧。整门课的目标只有一句：资源有限——算力、显存、有时是数据（这门课里数据不是瓶颈）——训出最好的模型，也就是把训练的计算效率做到最高；而要优化效率，先得会度量效率，这就要懂一段计算的算力特性和内存特性。
- **两道题**（图 2-1，算术都在图里），课程结束时你应该能在餐巾纸上算出来：
 1. 70B 参数的模型在 1024 张 H100 上训 15T token 要多久？总 FLOPs 按 $6 \times N \times D$ 算（第 7 节推导），每张卡每秒给多少查规格表，再乘 MFU 0.5（第 5 节），一除——143 天。
 2. 用 AdamW，一个节点的 H100 最多能训多大的模型？每张 80 GB，每个参数占 $2 + 2 + 4 + 4 = 12$ 字节（第 9 节），一除——约 53B。没算激活，激活随 batch size 和序列长度变。
- 讲者反复强调：这些都是粗糙的 back-of-the-envelope 估算，目的不是精确到每一个数，而是抓住量级和形状。

小注：第 2 题讲者没说卡数；按 $53B \times 12 \text{ 字节} \approx 640 \text{ GB} = 8 \times 80 \text{ GB}$ 反推，应为一个节点的 8 张 H100。
- **三种收获**（沿用第 1 讲的分法）：mechanics——PyTorch 和张量怎么工作，没有魔法；mindset——每写一行代码就想它的性能特征；intuitions——资源到底花在哪。今天没有 ML 上的花招，那是下一讲 Tatsu 的事。

2. 张量与它的字节：从 float32 一路降到 fp4



- **一切都是张量**：参数、梯度、优化器状态、数据、激活，底层全是张量——打开 DeepSeek V3.2 的模型文件，看到的就是一大堆带形状和精度的张量。显存很好算：元素个数 × 每元素字节数。
- **float32**：32 位里 1 位符号、8 位指数、23 位尾数——指数决定动态范围（能表示多大多小的数），尾数决定分辨率（相邻两个数隔多远）。科学计算里这叫 single precision，是浮点数的默认底线，想更准就上 float64；深度学习反着走——32 位都嫌多。CME295 第 4 讲列过各格式的位数表，这里只补新加的：每种精度用在哪儿，以及 fp8、fp4。
 - 例子：一个 4×8 的矩阵，PyTorch 默认建成 float32，占 $4 \times 8 \times 4 = 128$ 字节。GPT-3 这种老模型，feedforward 层里的一个矩阵就是约 2.3 GB，而且这还不是能想象的最大的张量。

小注：2.3 GB 的来历——GPT-3 175B 的 $d_{\text{model}} = 12,288$ ，FFN 内层是 4 倍即 49,152，一个权重矩阵 $12,288 \times 49,152 \approx 6.0e8$ 个元素，float32 下约 $2.4e9$ 字节 ≈ 2.25 GiB。

- **降精度赚两头**：位数减半，显存减半；16 位的运算也更快，粗略说快一倍，但不总是。而且显存变小本身就能让计算变快（第 6 节的算术强度），这一点不那么显然。
- **float16 的坑**：1 位符号、5 位指数、10 位尾数。指数只有 5 位，动态范围很差——建一个 $1e-8$ 的张量，存进去直接变成 0。早年用 fp16 训练的人都碰过：上溢、下溢、NaN。
- **bfloat16**（2018 年出现）：还是 16 位，把尾数的位挪给指数——指数位和 float32 一样是 8 位，动态范围与 float32 相同，代价是分辨率变差（没有免费午餐）。深度学习里这笔交易很值：不上溢不下溢是刚需，而训练本来就又粗又随机，分辨率没那么要紧。
- **训练的结论**：float32 能训，小模型不想操心就用它，只是每个数 4 字节；float16 太冒险；bf16 是甜点，但也不是绝对安全。于是通行做法是**混合精度训练**（mixed precision training）：参数、激活、梯度用 bf16，优化器状态用 fp32（第 9 节说为什么）。PyTorch 的 AMP 库（torch.autocast）把代码包一层，在安全的地方自动转成 bf16——矩阵乘法一般安全，指数运算就留在 fp32。这门课大概到 bf16 为止。
- **再往下**（有胆量的话）：
 - fp8：四年前提出，已经标准化，有两种变体，一种偏动态范围，一种偏分辨率；NVIDIA 的 Transformer Engine 支持。
 - fp4：去年 NVIDIA 推出 nvfp4，每个值只有 4 位——所有取值在 -6 到 6 之间，一行就能写完。光靠这 16 个值当然训不了，所以有个作弊：值按块分组，每块带一个缩放因子，块内 4 位自由变化，块整体可以放大缩小——单个值拿到的动态范围超过 4 位，只是相邻两个值不能一个在天上一个在地下。今年发布的 Nemotron 3 Super 就是用 fp4 训练的。这些不是你建张量时能指定的类型，全在 NVIDIA 的软件栈底下完成。

小注：fp8 的两种变体应为 E4M3（4 位指数、3 位尾数，偏分辨率）与 E5M2（偏范围），出自 [Micikevicius et al., 2022](#)；nvfp4 的元素格式是 E2M1（正负 0、0.5、1、1.5、2、3、4、6，正好是 -6 到 6），每 16 个元素共用一个 fp8 缩放因子。

- **问答**：能到 1 bit 吗？要分训练和推理——很多低比特工作是用 bf16 训好，再量化到 1、2 bit 做推理（第 10 讲会讲量化），这比直接训一个 1-bit 语言模型容易得多，后者讲者不知道有谁做出过像样的结果。

- 张量住在哪：PyTorch 默认建在 CPU 上，要快就得搬到 GPU，别忘了做。

格式	位数	符号 / 指数 / 尾数	每元素字节	讲者的定位
float32 (fp32, single)	32	1 / 8 / 23	4	小模型可用；优化器状态必须用它
float16 (fp16, half)	16	1 / 5 / 10	2	动态范围不够，训练会炸，别用
bfloat16 (bf16)	16	1 / 8 / 7	2	参数、激活、梯度的甜点；这门课的终点
fp8 (两种变体)	8	偏范围或偏分辨率	1	Transformer Engine 支持，课上不展开
nvfp4	4	16 个取值加按块缩放	0.5	Nemotron 3 Super 用它训过；由 NVIDIA 软件栈接管

3. einops：用名字而不是下标操作张量

- 动机：像 `x @ y.transpose(-2, -1)` 这种代码，看到 `-2` 和 `-1` 就得停下来想是哪一维，很容易搞错——讲者说他自己也常被 `transpose` 弄糊涂。einops 给维度起名字，用名字而不是位置来操作，灵感来自 Einstein summation notation。在场的同学约三分之二用过。
- einsum：带记账的广义矩阵乘法。规则只有一条：模式串里出现的每个名字都枚举一遍取值，把各输入按名字索引出来的元素相乘，累加到输出里；没有出现在箭头右边的名字就被求和掉。
 - `3 × 4` 乘 `4 × 3`：把 `x` 的两维叫 `seq1`、`hidden`，`y` 的两维叫 `hidden`、`seq2`，输出 `seq1`、`seq2`，`hidden` 被求和——就是矩阵乘法。
 - `2 × 3 × 4` 的两个张量做批量矩阵乘：老写法要 `transpose` 最后两维、再依赖 `@` 对前面的维度隐式批处理，得在脑子里推一遍；einsum 里写成 `batch`、`seq1`、`hidden` 和 `batch`、`seq2`、`hidden` 到 `batch`、`seq1`、`seq2`，**transpose 消失了**——命名本身就完成了转置。
 - 批量维可以用 `...` 代替：语言模型里常同时有 `batch`、`sequence`、`head` 好几个顺带的维度，写 `...` 就不必枚举，代码也不依赖进来的张量几维。
- reduce：`sum`、`mean`、`max`、`min` 的统一写法：右边没写的维度就是被归约的，归约方式作为参数传入，取代 `dim=-1` 这种要数位置的写

法。

- 问答：有加速吗？没有，它最后落到同样的原语上，只是语法糖。
- **rearrange**：作业里会用到。常见场景是一个维度其实是两个维度压扁的（比如 hidden 其实是 heads × head_dim），你想只对其中一个做运算：把 3 × 8 矩阵里的 8 拆成 2 × 4（括号里写 (heads hidden1)，指定 heads = 2，另一半自动算出），对拆出的那一维做矩阵乘，再用同样的括号语法合回去。
 - 问答：二维压成一维有行优先、列优先两种，用哪种？由你在括号里写名字的顺序决定。

小注：einops 的括号按行优先展开——括号里写在后面的名字变化最快，(heads d) 表示同一个 head 的 d 个元素连续排放。

- **代价与回报**：上手要花点时间，但值得——有了 einsum 之后思考方式会变，transpose、reduction 不再是零散的原语，而是名字之间的对应关系。第 7 节的梯度推导就靠它绕开“哪个矩阵要转置”。

```
from einops import einsum, reduce, rearrange
# 批量矩阵乘：右边没写的 hidden 被求和，前面的 ... 原样保留，不用 transpose
scores = einsum(q, k, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
# 归约：右边少了 seq2，就按 mean 把它收掉
row_mean = reduce(scores, "... seq1 seq2 -> ... seq1", "mean")
# 拆维·乘·合回：把压扁的 hidden 拆成 heads 个 d，对每个 head 各乘一个 d×d2 的矩阵
x = rearrange(x, "... (heads d) -> ... heads d", heads=2)
y = einsum(x, w, "... heads d, heads d d2 -> ... heads d2")
y = rearrange(y, "... heads d2 -> ... (heads d2)")
```

einops 三件套各写一次：没有一处 transpose，也没有一处 dim=-1；被求和、被归约、被拆合的维度全靠名字对上。

小注：往年公开的课程代码里，einops 还搭配 jaxtyping 把命名形状写进函数签名（形如 Float[Tensor, "batch seq hidden"]）；本讲口头没有提到 jaxtyping（推断，据往年课程代码）。

4. 算力核算 I：FLOPs、FLOP/s，和一次矩阵乘法的价钱

- **FLOP** = floating-point operation，一次浮点运算，这里只算加法和乘法这类基本运算——GPU 还能做别的，但矩阵乘法里的乘加是主食，吃掉绝大部分时间。

- **两个记号**（讲者的 pet peeve）：FLOPs 是做了多少运算，衡量计算量；FLOP/s 是每秒能做多少，衡量硬件速度——后者常被写成大写 S 的 FLOPS，容易混，讲者一律写 /s。"H100 有 989 teraFLOP/s"是后者，"GPT-3 花了 $3e23$ FLOPs"是前者。 $1e22$ 、 $1e23$ 、 $1e25$ 这些数在这门课里默认指训练算力，也就是模型的规模。

小注：GPT-3 175B 的训练算力在原论文里是 $3.14e23$ FLOPs，字幕里的 "325" 应是这个数的口误；用 6ND 验算： $6 \times 175e9 \times 300e9 \approx 3.15e23$ ，对得上。

- **规格表要读脚注**：H100 的表上 bf16 写着 1979 teraFLOP/s。你去跑一遍 benchmark 发现远远不到，翻到脚注——with sparsity，这是按稀疏矩阵算的，稠密矩阵要除以 2，也就是 989。所以这门课里凡是看到"除以 2"，就是这个原因。
- **量级感**：一个节点 8 张 H100 跑一周，能做 $8 \times 604,800 \text{ 秒} \times 989e12 \approx 5e21$ FLOPs。纯餐巾纸算术，但要把"某种硬件一周给多少"和"某种模型需要多少"两头都装进脑子。
- **线性层的 FLOPs**：B 个数据点，每个 D 维，映射到 K 维输出—— x 是 $B \times D$ ， w 是 $D \times K$ 。矩阵乘法的 FLOPs 是三个维度之积的 2 倍：每个 (b, d, k) 三元组做一次乘法、一次加法（严格说加法是 D - 1 次，忽略）。这是本讲全部 FLOPs 核算的核心，因为 Transformer 里贵的东西本质上都是线性层。

$$\text{FLOPs}(xw) = 2 \cdot B \cdot D \cdot K, \quad x \in \mathbb{R}^{B \times D}, w \in \mathbb{R}^{D \times K}$$

B 是数据点 (token) 数，D 是输入维度，K 是输出维度；2 来自每个乘积项配一次乘法和一次累加。逐元素运算（加法、ReLU 之类）只要矩阵大小 $m \times n$ 次运算，矩阵够大时没有任何别的运算能贵过矩阵乘法。

- **换一种读法**：B 是 token 数， $D \times K$ 正好是这一层的参数个数，所以 forward 的 FLOPs = $2 \times \text{token 数} \times \text{参数数}$ 。这条读法可以推广到 Transformer，你已经能看到 6ND 的雏形——第 7 节把 backward 补上就齐了。
- **问答**：有次立方的矩阵乘法算法（Strassen 一类），这里只算朴素算法吗？是——实践中矩阵乘法的优化是和系统协同设计（第 6 讲的 kernel），不是渐近意义上更快的算法。加法不是该比乘法便宜？硬件把两者做成一样的代价。

5. 算力核算 II：实测、MFU，以及为什么只能拿到一半

- **FLOPs 和硬件无关**，它只是模型需要做的运算数；真正要问的是在这块硬件上跑多久。最直接的办法是计时——GPU 异步执行，调用一发出就返回，所以计时前后都要 `torch.cuda.synchronize()` 设同步点，否则你会惊喜地发现怎么这么快，那只是非阻塞调用返回了。好习惯是多跑几次取平均；后面的讲次会专门讲 benchmarking。
- **实际 FLOP/s = 做了的 FLOPs ÷ 实测时间**；**承诺的 FLOP/s = 规格表上的数 (989e12)**。两者几乎总有差距，衡量这个差距的量叫 MFU。

$$\text{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}} = \frac{\text{FLOPs}/t_{\text{measured}}}{\text{peak FLOP/s}}$$

分子是按模型逻辑 FLOPs 数和实测墙钟时间算出的实际吞吐，分母是规格表的稠密峰值 (H100 bf16 取 989e12)；这个定义忽略了通信和其他开销，只问“承诺的算力你用上了几成”。

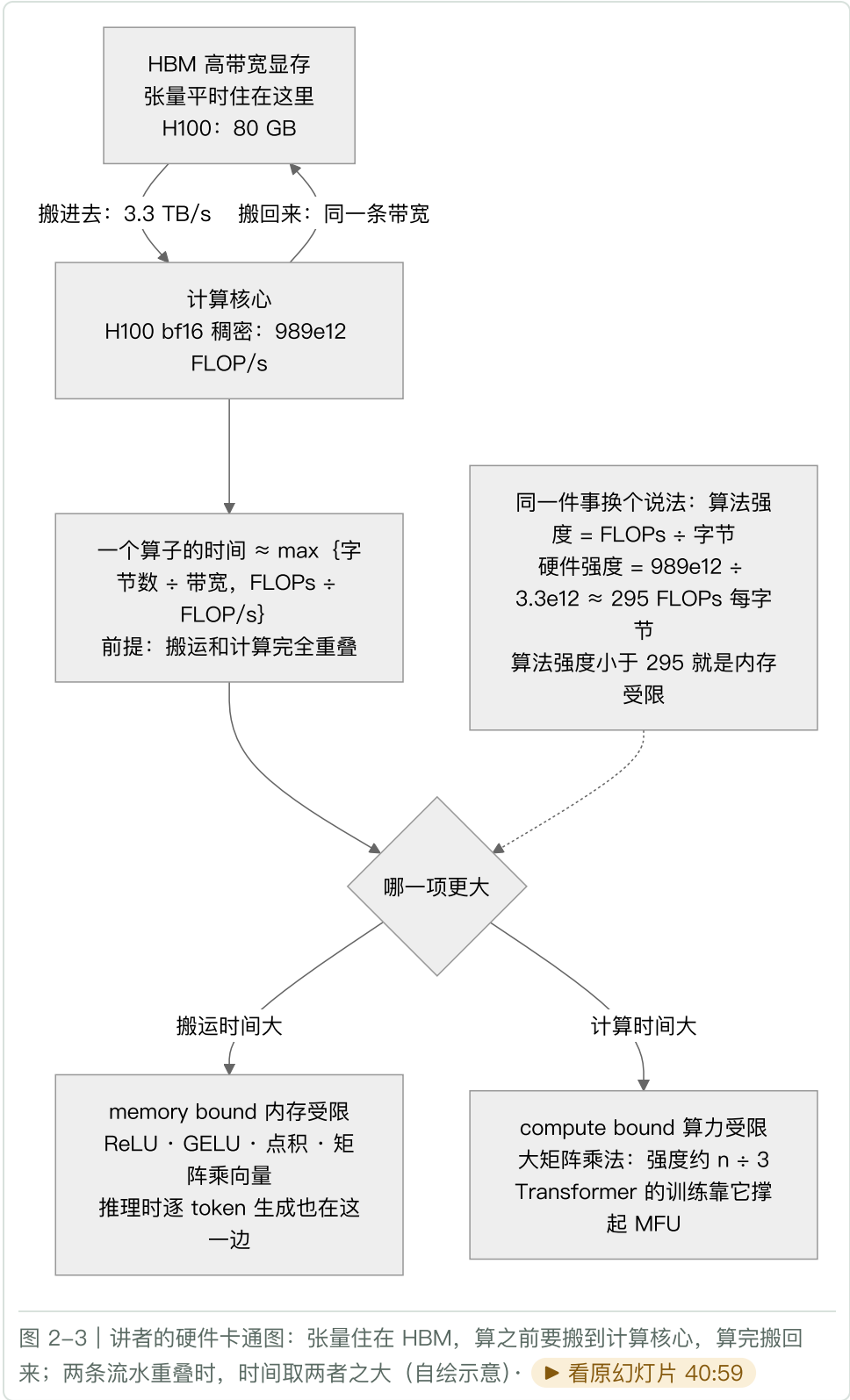
- **量级**：永远拿不到超过 1；现代模型能到 0.5 就该对自己满意；纯矩阵乘法也许能到 0.8；只有 0.1 就说明哪里出了问题，该去修。
- **问答**：承诺值是不是已经除过 2 了？是——989 已经是除以 2 之后的数，在此之上通常再只能拿到一半。为什么只有一半？等讲完内存瓶颈（第 6 节）再回答。
- **小结**：矩阵乘法主导计算，这是设计使然；FLOP/s 取决于硬件，也取决于数据类型——现在的 GPU 不再为 float32 优化，用它会非常慢，bf16、fp8 才是快车道；MFU 自己算：数出逻辑 FLOPs，看墙钟时间，一除。

小注：MFU 这个名字出自 PaLM 论文 (Chowdhery et al., 2022)：只按模型的理论 FLOPs 算，不把 activation checkpointing 的重算算作有效工作。

```
def measured_mfu(fn, flops, peak_flop_per_s, trials=5):
    torch.cuda.synchronize()                # GPU 是异步的：先把队列排空
    t0 = time.perf_counter()
    for _ in range(trials):
        fn()                                # 要计时的算子或一步训练
    torch.cuda.synchronize()                # 不等它算完，时间会短得离谱
    dt = (time.perf_counter() - t0) / trials
    actual = flops / dt                      # 实际 FLOP/s
    return actual / peak_flop_per_s          # MFU: 拿到的 ÷ 承诺的
```

计时的两处同步点和"多跑几次取平均"是讲者点名的三件事；flops 用第 4 节的公式数出来，peak 用规格表的稠密值。

6. 算术强度与 roofline：一个算子是内存受限还是算力受限



- **为什么只有一半**：MFU 的定义假定你只是在做矩阵乘法；硬件不是这样工作的。讲者的卡通版硬件（图 2-3）只有两块：**HBM** (high bandwidth memory, 高带宽显存, 张量平时住的地方, H100 有 80 GB) 和计算核心。要算就得把输入从 HBM 搬到核心, 算完再搬回去, 于是一个算子的时间取决于两个硬件参数：核心的速度 (FLOP/s) 和**内存带宽** (每秒能搬多少字节) ——H100 规格表： $1979\text{e}12 \div 2 \text{ FLOP/s}, 3.3 \text{ TB/s}$ 。
- **算 ReLU 要多久**：一个一百万维的 bf16 向量, 做 ReLU (逐元素取 $\max(x, 0)$)。
 - 搬运：读 x 进来 $2n$ 字节 (bf16 每个 2 字节), 写 y 回去 $2n$ 字节, 共 $4n = 4\text{e}6$ 字节; $\div 3.3\text{e}12 \approx 1\text{e}-6$ 秒。
 - 计算：每个元素和 0 比一次, $n = 1\text{e}6$ 次 FLOPs; $\div 989\text{e}12 \approx 1\text{e}-9$ 秒。
 - 假设搬运和计算**重叠** (数据一到就开算, 第 5 讲会细说; 实际不会完美重叠)：总时间 = 两者取大 = $1\text{e}-6$ 秒。

$$T \approx \max\left(\frac{\text{bytes moved}}{\text{bandwidth}}, \frac{\text{FLOPs}}{\text{FLOP/s}}\right)$$

第一项是把输入输出在 HBM 和计算核心之间搬一遍的时间, 第二项是纯计算时间; 取大是"完美重叠"假设下的总时间。搬运项更大叫 **memory bound** (内存受限, 大部分时间在等数据到), 计算项更大叫 **compute bound** (算力受限, 瓶颈真的是算)。ReLU 显然是前者——算比搬少了三个数量级。

- **强度** (intensity)：同一件事换个说法。硬件的**加速器强度** = $\text{FLOP/s} \div \text{字节/s}$, 即每搬一个字节硬件能做多少运算——H100 是 $989\text{e}12 \div 3.3\text{e}12 \approx 295$, 脑子里记"约 300"; 算法的**算术强度** = 这个负载做的 FLOPs $\div$ 搬的字节。两个分数交叉相乘, 就是等价的判据：

$$I_{\text{acc}} = \frac{\text{FLOP/s}}{\text{bytes/s}} \approx \frac{989 \times 10^{12}}{3.3 \times 10^{12}} \approx 295, \quad I_{\text{alg}} = \frac{\text{FLOPs}}{\text{bytes}}, \quad I_{\text{alg}} < I_{\text{acc}} \iff \text{memory bound}$$

I_{acc} 是硬件每字节配多少 FLOPs, I_{alg} 是算法每字节做多少 FLOPs; 前者就是分界线。ReLU 的 $I_{\text{alg}} = n \div 4n = 0.25$ ——讲者：谁报出 0.25 这个数, 你该说这太糟了。

小注：295 = $989 \div 3.35$, 讲者代码里用的应是规格表上 H100 SXM 的 3.35 TB/s, 口头说成了 3.3; 按 3.3 算是 300。

- **一路算上去** (下表)：GELU 比 ReLU 复杂得多 (每个元素约 20 次运算), 搬的字节一样是 $4n$, 强度约 5——仍远小于 295, 还是内存受

限：GELU 看着贵，墙钟时间却和 ReLU 一模一样，因为瓶颈不在计算。点积、矩阵乘向量也一样，强度在 0.5 到 1 之间。直到矩阵乘法：搬 n^2 量级的东西，算 n^3 量级的东西，强度约 $n \div 3$ ，随矩阵变大而变大——这就是大家要大 batch、大矩阵的全部原因：低于硬件强度时把东西做小并不会更快，高于它才算把 GPU 喂饱。

算子 (bf16)	搬运字节	FLOPs	算术强度	判定
ReLU, n 维向量	$2n + 2n$	n	0.25	内存受限
GELU, n 维向量	$2n + 2n$	约 $20n$	约 5	内存受限
点积, 两个 n 维向量	$2n + 2n + 2$	$2n - 1$	约 0.5	内存受限
矩阵乘向量, $n \times n$ 乘 n	$2n + 2n^2 + 2n$	n 个点积, 约 $2n^2$	约 1	内存受限
矩阵乘法, $n \times n$ 乘 $n \times n$	$2n^2 + 2n^2 + 2n^2$	n^2 个点积, 约 $2n^3$	$n \div 3$ (讲者的例子约 340)	算力受限

$$I_{\text{matmul}} = \frac{2n^3}{2n^2 + 2n^2 + 2n^2} = \frac{n}{3}$$

分子是 n^2 个长度为 n 的点积（各约 $2n$ 次运算），分母是读两个输入矩阵、写一个输出矩阵的字节数（bf16 每元素 2 字节）；强度随 n 线性增长，所以只有足够大的矩阵才能越过硬件强度。

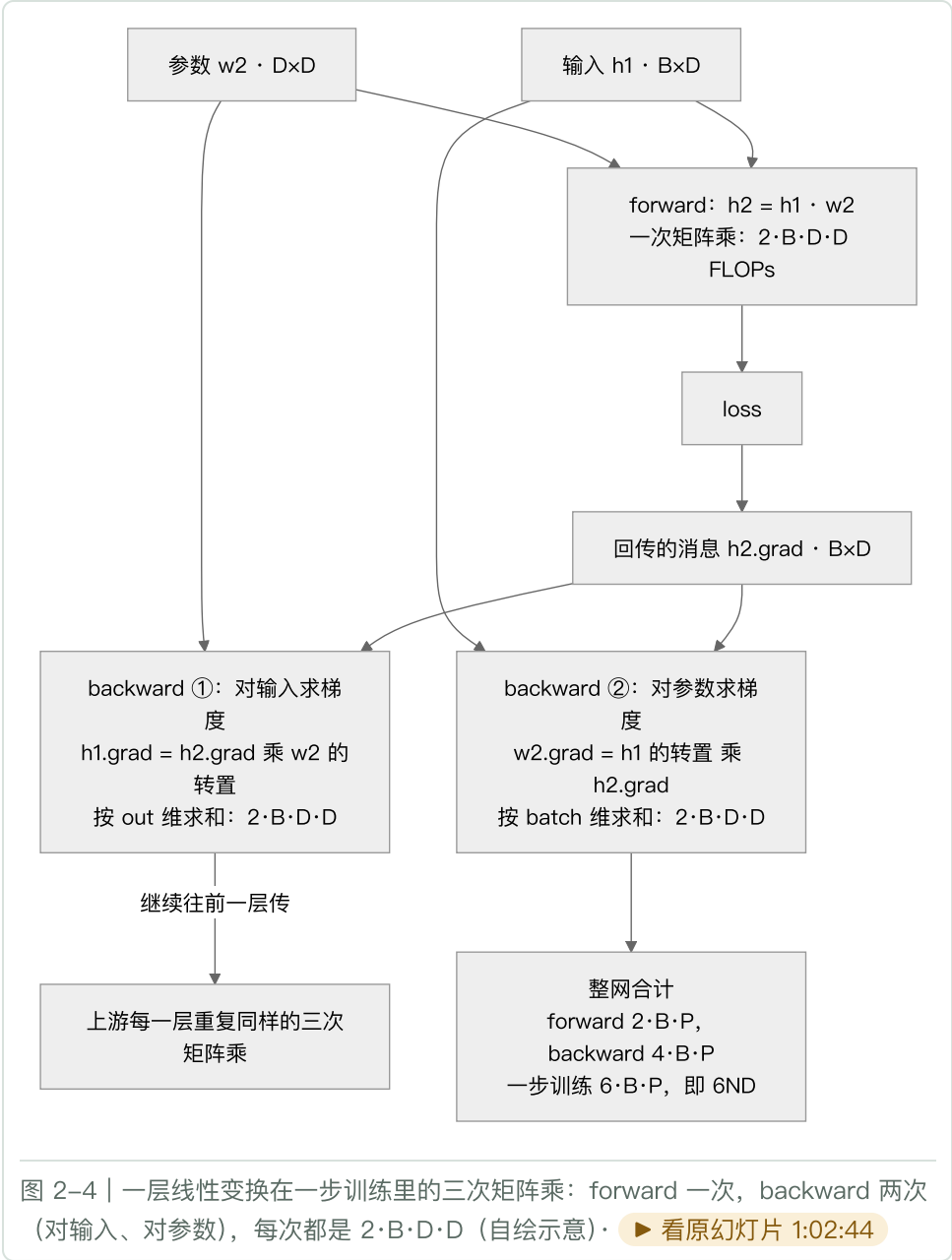
小注： $n \div 3 = 340$ 对应 $n = 1024$ ，应为讲者演示代码里的矩阵边长（推断）。

- **落到 Transformer 上**：作业 1 和下一讲都会看到，Transformer 本质上是一堆大矩阵乘法，中间撒了些别的算子——高算术强度是设计出来的。但推理不一样（第 10 讲的预告）：逐 token 生成时每一步是一个向量去乘权重矩阵，即矩阵乘向量，内存受限；训练时整条序列一起进来，才是矩阵乘矩阵。强度还和精度有关，这里默认 bf16。
- **回答 MFU 为什么只有 0.5**：承诺值假设没有内存瓶颈；真实模型里有大量内存受限的环节，吞吐自然上不去。
- **问答**：既然大部分时候内存受限，加速器为什么造得算力过剩、宁可以让核心闲着等数据？讲者留到第 5 讲——如果你有答案，去告诉 Jensen。
- **roofline 图**：横轴算术强度，纵轴实际达到的 FLOP/s，每条折线一种加速器（H100、B200……）。低强度的算子（ReLU、点积）落在左边

的斜坡上，达不到峰值；强度越高越接近饱和，越过拐点就是算力受限，封顶在峰值。

小注：roofline 模型出自 Williams, Waterman & Patterson, 2009；斜坡的斜率是内存带宽，平顶是峰值 FLOP/s，拐点横坐标就是 L_{acc} 。

7. 反向传播要多少 FLOPs：6ND 从哪来



- 运行例子：一个深度网络——输入 $B \times D$ ， L 层，每层一个 $D \times D$ 的矩阵乘再接逐元素 ReLU，输出与输入同形，参数量 $D^2 \times L$ ；PyTorch 里就是 L 个 block，每个 block 一个权重矩阵。
- 数 backward 的 FLOPs：把模型再简化—— $x (B \times D)$ 依次乘 w_1 、 w_2 （都是 $D \times D$ ），先忽略 ReLU；用 einsum 写 forward，给中间结

果 `retain_grad()` 便于调试，然后 `loss.backward()`。这一步 backward 做了多少浮点运算？盯住第二层：

- forward 是 $h_2 = h_1 \cdot w_2$ ，一次矩阵乘， $2 \times B \times D \times D$ 。
- backward 按链式法则要算**两样东西**：一是往回传的消息——loss 对这一层输入的梯度 `h1.grad`；二是 loss 对这一层参数的梯度 `w2.grad`。
- $h_1.grad = h_2.grad$ 乘 w_2 ，按 out 维求和； $w_2.grad = h_1$ 乘 $h_2.grad$ ，按 batch 维求和。微积分课上总有一个转置不知该放哪边，用 `einsum` 就不必想：从标量情形就知道 `h1.grad` 长得像 $h_2.grad \times w_2$ ，剩下的只是看形状对名字。算出来和 `autograd` 填进 `.grad` 的一模一样。
- 关键观察：`einsum` 的 FLOPs 就是所有维度之积的 2 倍，**不管哪个维度在批处理、哪个在求和**——枚举 i, j, k 再聚合，聚合方式不同，运算数相同。所以两个 backward 矩阵乘各是 $2 \times B \times D \times D$ ，backward 恰好是 forward 的两倍。

```
# 一层线性变换在一步训练里的三次矩阵乘，FLOPs 都是  $2 \cdot B \cdot D_{in} \cdot D_{out}$ ，只是求和的名字不同
h2      = einsum(h1, w2,      "batch d_in, d_in d_out -> batch d_out") # forward
h1_grad = einsum(h2_grad, w2, "batch d_out, d_in d_out -> batch d_in") # 对输入：按 d_out 求和
w2_grad = einsum(h1, h2_grad, "batch d_in, batch d_out -> d_in d_out") # 对参数：按 batch 求和
# forward 一次 + backward 两次 =  $3 \times 2 \cdot B \cdot P$ ，也就是  $6 \cdot B \cdot P$ 
```

三行 `einsum` 里没有一个转置：哪个维度被求和，由它只出现在箭头左边决定；三次的运算量完全相同。

- **整网合计**：对每个参数都这么算一遍，加起来——

$$FLOPs_{fwd} = 2BP, \quad FLOPs_{bwd} = 4BP, \quad FLOPs_{step} = 6BP \quad (\text{即 } 6ND)$$

B 是这一步的 token 数， P 是参数个数；forward 每参数每 token 2 次运算，backward 对输入和对参数各算一次梯度所以是 4 次，合计 6——各处引用的 $6ND$ (N 参数、 D 总 token 数) 就是把 B 换成整个训练的 token 数。

- **推广到 Transformer**：这里只对深度网络推了一遍，但对 Transformer 是很好的近似——只要上下文长度不太长。上下文一

长，attention 里就多出一项随上下文长度平方增长的 FLOPs，不在这个账里。作业 1 会让你对 Transformer 仔细数一遍。

小注：Kaplan et al., 2020 的记账是每 token 的 forward FLOPs $\approx 2N + 2 \times n_{\text{layer}} \times n_{\text{ctx}} \times d_{\text{attn}}$ ，后一项就是讲者说的上下文平方项， d_{model} 远大于 n_{ctx} 时可忽略。CME295 第 4 讲把 6ND 当作已知公式引用，这里补上了推导。

8. 优化器与它的状态：拿 AdaGrad 当例子

- 为什么讲 AdaGrad：作业 1 要你实现 Adam，讲者不想直接演答案，就用 2011 年的 AdaGrad——它处在 SGD 和 Adam 之间：momentum 看梯度的一阶矩，AdaGrad 看二阶矩（梯度平方的累积），Adam 两者都用。细节不在本讲范围，重点是用法。
- 一步优化的骨架：优化器按 param group 遍历参数（这里就是 w_1 、 w_2 ）；每个参数在 `optimizer.state` 里有一块自己的存储。AdaGrad 存的是梯度平方的累加和 g_2 ：每步先用当前梯度更新 g_2 存回去，再按 $w \leftarrow w - \text{lr} \cdot g \div \sqrt{g_2}$ 更新参数。这块随参数走的存储就是优化器状态（optimizer state），下一节给它记账。

小注：AdaGrad 出自 Duchi, Hazan & Singer, 2011；Adam (Kingma & Ba, 2014) 的两个状态就是 CME295 第 4 讲写过的 m_t 和 v_t ，每个参数各一份。

9. 显存核算：一步训练里字节都去了哪

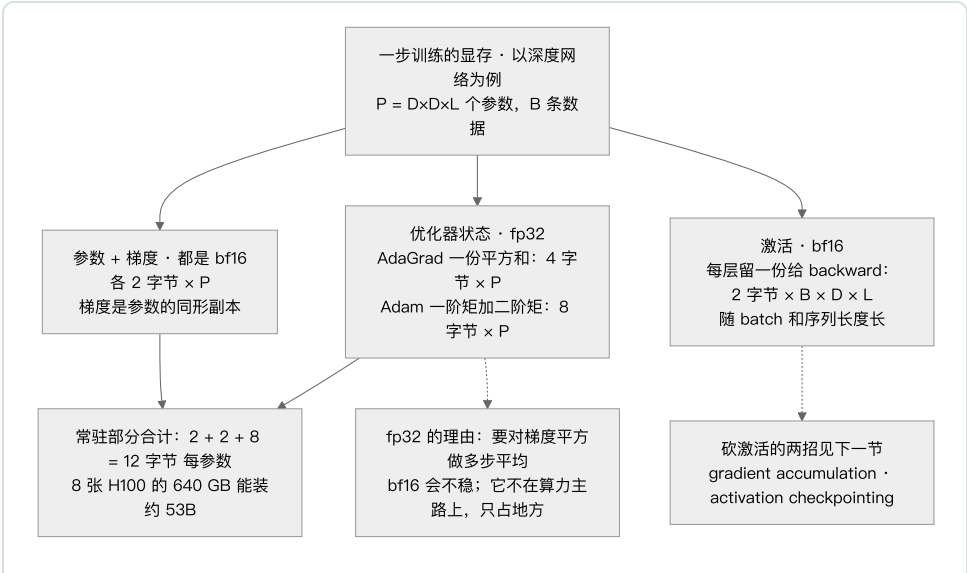


图 2-5 | 训练一步的显存去向：三项常驻（参数、梯度、优化器状态）按每参数字节数记，激活按 batch $\times$ 宽度 $\times$ 层数记（自绘示意）· [▶ 看原幻灯片 1:09:28](#)

- **四本账**（深度网络， $P = D^2 \times L$ 个参数，batch 为 B，参数、激活、梯度按 bf16）：
 - 参数：2 字节 $\times P$ ；
 - 激活：每层留一份给 backward 用，2 字节 $\times B \times D \times L$ ；
 - 梯度：参数的同形副本，2 字节 $\times P$ ；
 - 优化器状态：按惯例用 fp32——AdaGrad 一份平方和，4 字节 $\times P$ ；Adam 一阶矩加二阶矩，8 字节 $\times P$ （幻灯片这一行写成了参数显存的倍数，讲者当场说是笔误，应为参数个数）。
- **为什么优化器状态要 fp32**：它要对梯度平方取值、再跨很多步做平均，bf16 的分辨率撑不住——人试过，不稳。所以优化器状态往往是显存里最大的一块。
- **显存的两种用途**：一是得放得下（HBM 的容量），二是要搬到计算核心（第 6 节的带宽）。优化器状态不在计算的主路上，它的大小几乎不影响速度，影响的是能不能装下更大的模型。

$$\text{bytes/param} = \underbrace{2}_{\text{bf16 参数}} + \underbrace{2}_{\text{bf16 梯度}} + \underbrace{4+4}_{\text{fp32 的 } m, v} = 12, \quad N_{\max} \approx \frac{8 \times 80 \text{ GB}}{12 \text{ B}} \approx 53\text{B}$$

这就是开场第 2 题的账：Adam（或 AdamW）训练时每个参数常驻 12 字节；一个节点 640 GB 显存除以 12，约 53B 参数——不含激活，激活随 batch 和序列长度变。

小注：这套 12 字节的账没有保留 fp32 的权重主副本（master weights）。ZeRo 论文和 CME295 第 4 讲用的是 16 字节（多 4 字节 fp32 权重副本，用来累积很小的更新量），按 16 字节算同一个节点只能装 40B。

- **合起来**：参数量 $D \times D \times L$ ，一步训练的 FLOPs 是 $6 \times \text{token 数} \times \text{参数量}$ ；Transformer 的账要复杂一些，作业 1 里仔细做。训练循环讲者说是通用复习，直接跳过；参数初始化和数据加载这次课上没有讲到，这份笔记不替他补。

10. 少存一点：gradient accumulation 与 activation checkpointing

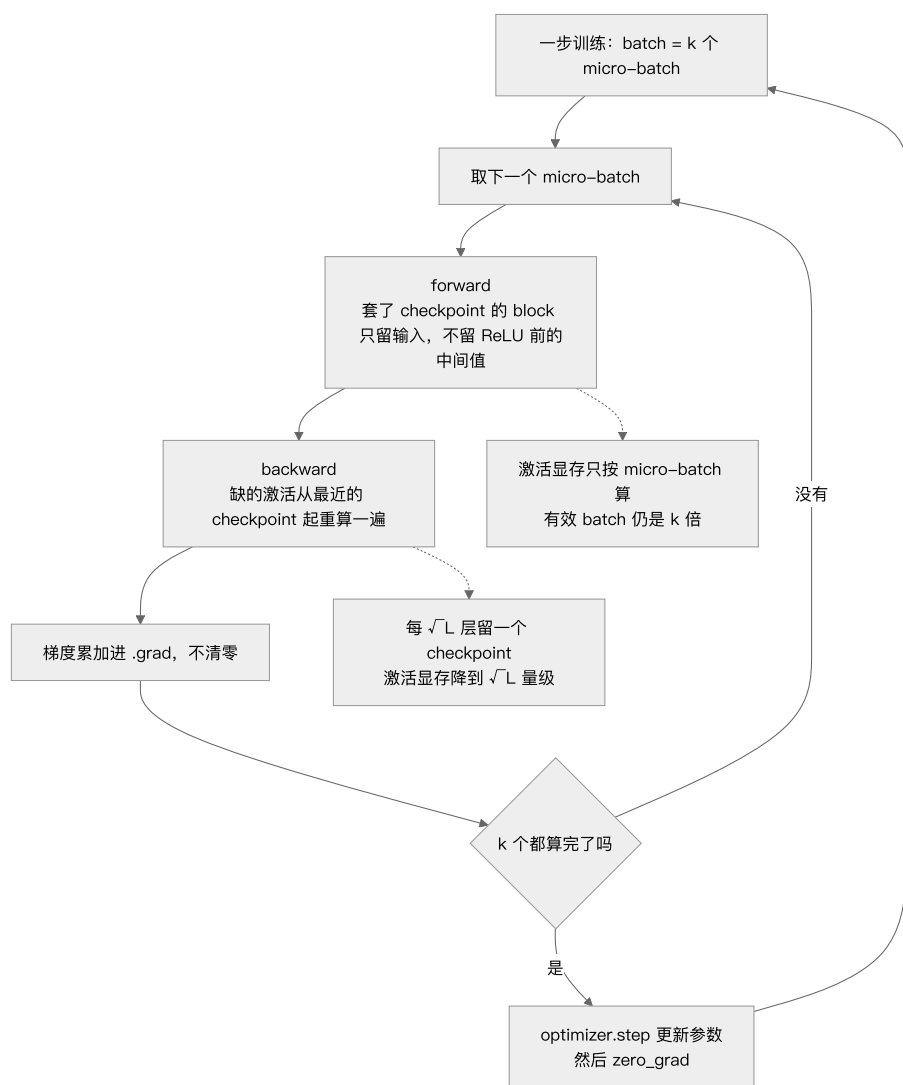


图 2-6 | 带 micro-batch 累积和 activation checkpointing 的一步训练: 激活按 micro-batch 计, 缺的中间值在 backward 时重算 (自绘示意)。

► 看原幻灯片 1:12:37 · 出处: [Chen et al., 2016](#)

- **Gradient accumulation** (梯度累积): 训练希望 batch 大一些以求稳定——大到 critical batch size 为止 (Tatsu 后面会讲), 但激活显存和 batch 成正比, 太大就 OOM。办法: 把 batch 切成若干 micro-batch, 各做 forward 和 backward, 梯度累加进 `.grad` 不清零; 累够 $\text{batch} \div \text{micro-batch}$ 个之后才 `optimizer.step`, 然后清零。代码改动极小 (讲者口误说省算力, 省的是显存)。

小注: critical batch size 的经验模型应出自 [McCandlish et al., 2018](#), 第 9 / 11 讲的 scaling laws 会回到它。

- **Activation checkpointing** (也叫 gradient checkpointing、rematerialization): 训练默认把所有层的激活都留着 (推理不算梯度, 只需当前层的), 显存 $B \times D \times 2 \times L$ 。想法: forward 只保留一

部分层的激活 (checkpoint), backward 缺什么就从最近的 checkpoint 起重算——用计算换显存, 系统里到处是这种权衡。

- PyTorch 里就是给一层套上 `torch.utils.checkpoint`: 这段计算照做, 但不存中间结果。一个 block 是线性加 ReLU, 默认要存 ReLU 前后两份; 套上 checkpoint 就不存 ReLU 前那份, 显存省一半——backward 需要时从上一层的输出重算即可, 很便宜。
- 极端情况: 一层都不存, 显存最省, 但计算变成 L^2 量级——每一层都要从头算起。折中: 每 $\sqrt{L}$ 层留一个 checkpoint, 激活显存和重算开销都是 $\sqrt{L}$ 量级。

$$M_{\text{act}} = 2 \cdot B \cdot D \cdot L \text{ bytes} \xrightarrow{\text{每 } \sqrt{L} \text{ 层一个 checkpoint}} M_{\text{act}} \propto \sqrt{L}$$

左边是深度网络默认要留的激活 (bf16、每层一份 $B \times D$); 每隔 $\sqrt{L}$ 层留一个 checkpoint 后, 常驻的只有 $\sqrt{L}$ 个 checkpoint 加上正在重算的一段 (也是 $\sqrt{L}$ 层)。

小注: 这就是 [Chen et al., 2016](#) 的 sublinear memory 方案: $O(\sqrt{n})$ 显存, 代价是整个 backward 总共多做相当于一遍 forward 的计算——讲者说的“重算开销 $\sqrt{L}$ ”指每段重算 $\sqrt{L}$ 层, 加总仍是一遍 forward (一步约多 1/3 的 FLOPs)。CME295 第 4 讲 FlashAttention 的重算是同一思路用在单个算子上。

```
for step in range(num_steps):
    for _ in range(k):                # 有效 batch = k ×
micro-batch                          # 数据加载: 本讲没展开
        x = next(loader)              # forward; block 内可用
        loss = model(x) / k           # torch.utils.checkpoint 包住
        loss.backward()               # 梯度累加进 .grad, 不清
零                                   # optimizer.step()
        optimizer.step()              # k 个 micro-batch 之
后更新一次                          # optimizer.zero_grad(set_to_none=True)
        optimizer.zero_grad(set_to_none=True)  # 这时才清零
```

一步训练的骨架: 内层循环是讲者说的 micro-batch 累积, 外层一次 step 一次清零; 把 loss 除以 k 是为了让累加后的梯度等于整个 batch 的平均梯度。

- 讲者的总结:** 一切都是张量; 用 einops 思考张量运算; $6 \times \text{batch} \times$ 参数量是一步训练的 FLOPs; 算术强度和 roofline 判断内存受限还是算力受限——矩阵乘法算力受限, 其余基本都内存受限; gradient

accumulation 和 activation checkpointing 省显存，换更大的 batch。下周 Tatsu 讲架构。

11. 数字速查：本讲说过的每一个数

数	是什么	讲者的用法
1e23 FLOPs · 0.05	Marin 那次训练的算力；实际 loss 与预测之差	scaling law 预测能兑现的证据
143 天	70B × 15T token × 1024 张 H100 × MFU 0.5	第 1 题
12 字节 · 53B	每参数 2 + 2 + 4 + 4；640 GB ÷ 12	第 2 题
1 / 8 / 23 · 1 / 5 / 10	fp32 与 fp16 的符号 / 指数 / 尾数位	指数决定范围，尾数决定分辨率
128 字节 · 2.3 GB	4 × 8 的 fp32 矩阵；GPT-3 一个 FFN 矩阵	张量显存 = 元素数 × 字节数
1e−8 → 0	fp16 下的下溢	为什么不用 fp16
−6 到 6 · 16 个值	nvfp4 的全部取值	4 位有多少
1979 → 989 TFLOP/s	H100 bf16 带稀疏 / 稠密	规格表要除以 2
3.3 TB/s · 80 GB	H100 的 HBM 带宽与容量	算术强度的分母；显存上限
约 5e21 FLOPs	8 张 H100 跑一周	硬件一周给多少
3e23 FLOPs	GPT-3 的训练算力（口头约数）	模型规模的量级
0.5 · 0.8 · 0.1	MFU：该满意 / 纯矩阵乘的上限 / 出问题了	实际 ÷ 承诺
295 (约 300)	H100 的加速器强度，FLOPs 每字节	内存受限的分界线
0.25 · 5 · 0.5 · 1 · n/3	ReLU · GELU · 点积 · 矩阵乘向量 · 矩阵乘法的算术强度	只有矩阵乘法过线
1e−6 s · 1e−9 s	百万维 ReLU 的搬运时间与计算时间	差三个数量级
约 20	GELU 每元素的 FLOPs（粗估）	强度 5 的来历
2 · 4 · 6	forward / backward / 一步训练的 FLOPs，每参数每 token	6ND
4 · 8 字节	AdaGrad / Adam 每参数的 fp32 状态	优化器状态是大头
一半 · √L · L²	checkpoint 一个 block 省的激活；每 √L 层一个 checkpoint 的显存；全不存的重算量	activation checkpointing 的三个档位
2011 · 2018 · 四年前 · 去年	AdaGrad · bfloat16 · fp8 · nvfp4 的年份	精度格式的时间线

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Marin 的 IsoFLOP 预测图	每条曲线一个算力预算，最低点连成 scaling law；1e23 那个点上预测与实测的 loss 只差 0.05	0:05	Marin
两道估算题	第一题的链条：6ND → 规格表 → MFU → 每天产出 → 143 天；第二题：80 GB × 8 ÷ 12 字节 → 53B	1:38	—
浮点格式的位分配	fp32 的 1 / 8 / 23；fp16 只有 5 位指数；bf16 把尾数的位挪给指数	5:51	—
fp4 的取值表	一行写完的 16 个值，−6 到 6；配合按块缩放才可用	13:47	—
einsum 与 rearrange 的例子	3 × 4 乘 4 × 3 的命名写法；批量版本里 transpose 消失；24:10 的括号语法把 8 拆成 2 × 4	19:27	einops
H100 规格表脚注	bf16 写 1979 TFLOP/s，脚注 with sparsity；稠密除以 2 得 989	29:24	NVIDIA H100
矩阵乘法的 FLOPs	B × D 乘 D × K：每个三元组一乘一加，2·B·D·K	31:31	—
计时代码与 MFU 定义	前后两处 cuda.synchronize；MFU = 实际 ÷ 承诺，0.5 就好	35:08	PaLM (MFU 一词)
硬件卡通图	HBM 在下，计算核心在上；两个参数：989 TFLOP/s 与 3.3 TB/s	40:59	—
算术强度逐算子表	ReLU 0.25 → GELU 5 → 点积 0.5 → 矩阵乘向量 1 → 矩阵乘法 n/3；只有最后一个过 295	42:36	—
roofline 图	横轴强度、纵轴实际 FLOP/s；斜坡是内存受限，平顶是算力受限，每条线一种加速器	55:42	Williams et al., 2009
backward 的两个 einsum	h1.grad 按 out 求和，w2.grad 按 batch 求和；和 autograd 的结果逐项相同	1:02:44	—
显存核算表	参数 2、激活 2·B·D·L、梯度 2、优化器 4 或 8；讲者当场指出的笔误	1:09:28	—
activation checkpointing 代码	给 block 套 torch.utils.checkpoint；不存 ReLU 前的中间值，省一半	1:14:45	Chen et al., 2016

提到的工作

名称	在本讲里的作用
Marin (Stanford)	开场消息：1e23 FLOPs 训练跑完，loss 与 scaling law 预测差 0.05 以内
DeepSeek V3.2	模型就是一堆带形状和精度的张量的例子
GPT-3 (Brown et al., 2020)	一个 FFN 矩阵约 2.3 GB；训练算力约 3e23 FLOPs 的量级例子
bfloat16 (Google Brain, 2018)	把 fp16 的尾数位挪给指数，动态范围与 fp32 相同
PyTorch AMP (torch.autocast)	混合精度的自动实现：矩阵乘转 bf16，指数运算留 fp32
FP8 Formats for Deep Learning (Micikevicius et al., 2022)	四年前提出、已标准化、两种变体的 fp8，应为这篇
NVIDIA Transformer Engine	fp8 训练的软件支持
NVFP4 (NVIDIA, 2025)	4 位格式：16 个取值加按块缩放
Nemotron 3 Super (NVIDIA, 2026)	用 fp4 训练出来的模型
einops (Rogozhnikov)	einsum / reduce / rearrange，命名维度；本讲的工具主线
Einstein summation notation	einops 的记号来源
NVIDIA H100 规格表	80 GB、bf16 1979 / 989 TFLOP/s、3.3 TB/s；所有硬件数字的来源
B200	roofline 图里的另一条线
Strassen 一类次立方矩阵乘法	问答：实践里不重要，优化靠系统协同设计
PaLM (Chowdhery et al., 2022)	MFU 这个指标的出处（小注）
Roofline model (Williams et al., 2009)	算术强度对实际 FLOP/s 的图
Scaling Laws for Neural Language Models (Kaplan et al., 2020)	6ND 与上下文平方项的原始记账（小注）
AdaGrad (Duchi et al., 2011)	优化器状态的演示例子：梯度平方的累加和
Adam · AdamW	作业 1 要实现；两个 fp32 矩，每参数 8 字节
critical batch size (McCandlish et al., 2018)	gradient accumulation 的动机：batch 大到这里为止（应为此文）
torch.utils.checkpoint	activation checkpointing 的 PyTorch 入口
Training Deep Nets with Sublinear Memory Cost (Chen et al., 2016)	每 $\sqrt{L}$ 层一个 checkpoint 的方案（小注）

术语对照

English	中文
resource accounting	资源核算：把 FLOPs、字节、时间一项项数清楚
back-of-the-envelope / napkin math	餐巾纸估算：只求量级
tensor	张量：参数、梯度、激活、优化器状态、数据的统一容器
precision / dtype	精度 / 数据类型
sign / exponent / mantissa	符号位 / 指数位 / 尾数位
dynamic range	动态范围：能表示的最大最小数
resolution	分辨率：相邻可表示数之间的间隔
single / double precision	单精度 fp32 / 双精度 fp64
underflow / overflow / NaN	下溢 / 上溢 / 非数
mixed precision training	混合精度训练：不同计算用不同精度
AMP (automatic mixed precision)	自动混合精度：PyTorch 的 torch.autocast
block scaling	按块缩放：一块值共用一个缩放因子（fp4 的做法）
quantization	量化：训好之后再降精度，与低精度训练是两回事
named dimensions	命名维度：einops 用名字代替下标
einsum / reduce / rearrange	广义矩阵乘 / 归约 / 重排（拆维、合维）
batch dimensions	批处理维：原样保留、不参与运算的维度，模式里写成 ...
FLOP / FLOPs / FLOP/s (FLOPS)	一次浮点运算 / 运算次数（计算量） / 每秒运算次数（硬件速度）
spec sheet	规格表
sparsity (2:4 structured)	结构化稀疏：规格表大数的前提；稠密要除以 2
MFU (model FLOPs utilization)	模型算力利用率：实际 FLOP/s ÷ 承诺 FLOP/s
synchronize / non-blocking	同步点 / 非阻塞调用：GPU 计时前后必须同步
HBM (high bandwidth memory)	高带宽显存：张量常驻的地方
memory bandwidth	内存带宽：每秒能搬多少字节
accelerator	加速器：GPU / TPU 做计算的那一侧
overlap (communication and computation)	搬运与计算重叠：时间取两者之大
memory bound / compute bound	内存受限 / 算力受限
arithmetic intensity	算术强度：算法每字节做多少 FLOPs
accelerator intensity	加速器强度：硬件每字节配多少 FLOPs（H100 约 295）
roofline	屋顶线模型：强度对实际 FLOP/s 的图

English	中文
forward / backward pass	前向 / 反向传播
backward message	回传消息：loss 对某层输入的梯度
retain_grad	让非叶子张量保留梯度，便于调试
optimizer state	优化器状态：随参数走的额外存储
param group	参数组：优化器遍历参数的单位
first / second moment	一阶矩 / 二阶矩：梯度的均值 / 梯度平方的均值
gradient accumulation	梯度累积：多个 micro-batch 的梯度加起来再更新
micro-batch	微批：一次装得下显存的那一小份
critical batch size	临界 batch 大小：再大也不更稳的那个点
activation checkpointing / rematerialization	激活检查点 / 重物化：少存激活，backward 时重算
IsoFLOP curve	等算力曲线：同一算力预算下不同模型大小的 loss
compute-optimal	算力最优
GELU	高斯误差线性单元：比 ReLU 复杂的激活函数，每元素约 20 FLOPs

字幕勘误

字幕是人工 CC 轨，错得不多。"scaling loss" → scaling laws；"800, 989 teraflops" → 989 TFLOP/s；GPT-3 的 "325 ... FLOPs" → 应为 3.14×10^{23} ；"each VM memory" → HBM memory；"eisum" → einsum；"BID" → $B \times D$ ；"as in buy out" / "batch buy out" → in, out / batch, out；"batch by Alt matrix" → batch $\times$ out；"80.8" → 0.8 (MFU)；"1 minus 9" → 10^{-9} ；"arithmetic density" → arithmetic intensity；"for each 100" → for H100；"5e 21" → 5×10^{21} ；"Torch utils at checkpoint" → torch.utils.checkpoint；"first order bit" → first-order moment。讲者自己的口误（字幕没错）："D times number of tokens" 应为 $6 \times$ ；"save on compute" 应为省显存；"everything we're doing here is fp16" 应为 bf16；"two weeks" 当场改为一周。

带走的问题

- 把第 1 题换成你自己的配置：8B 参数、15T token、256 张 H100、MFU 0.4，要几天？如果换成 B200，你需要从规格表上读哪两个数、

各要注意什么脚注？

2. GELU 比 ReLU 多做 20 倍运算，墙钟时间却一样——一个逐元素算子每个元素要做多少 FLOPs 才会在 H100 上变成算力受限？（提示：每元素搬 4 字节，门槛 295 FLOPs 每字节。）这个数说明了什么？
3. 讲者的 12 字节账和 ZeRO 的 16 字节账差在一份 fp32 权重主副本。如果权重只保留 bf16，学习率乘梯度得到的很小的更新量会发生什么？这和"优化器状态必须 fp32"是同一个问题吗？
4. 推理时逐 token 生成是矩阵乘向量，内存受限。把多个请求 batch 在一起能否把它推回算力受限？KV cache 的读取又会把强度往哪边拉（第 10 讲）？
5. 每 $\sqrt{L}$ 层放一个 checkpoint，请自己算：backward 总共多做了多少次 forward 的计算？Transformer 里 attention 的中间结果随序列长度平方增长，若显存只够 checkpoint 一部分，应该优先不存哪些激活？

CS336 第 3 讲 | 架构 (Architectures)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 3 讲 视频: <https://www.youtube.com/watch?v=IVynu4bo1rY> (1:29:14; 英文字幕为自动生成, 模型名、人名错得多, 见文末勘误) 讲者: Tatsunori Hashimoto (全程; 多处以第三人称提到 Percy——"上一讲 Percy 讲过 arithmetic intensity""推理的细节 Percy 之后讲") 课程主页: <https://stanford-cs336.github.io/> · 本讲反复引用的证据: Shazeer 2020 (GLU 变体) · Narang et al. 2021 (架构改动能否迁移) · RoFormer · Kaplan et al. 2020 · Shazeer 2019 (MQA) · Ainslie et al. 2023 (GQA)

一句话: 把 2017 年的原版 Transformer 和近三年几十个开源模型摆在一起看, dense 架构上真正被所有人改掉的只有几件小事——归一化挪出残差流并换成 RMSNorm、线性层去掉 bias、FFN 换成带门控的 SwiGLU / GeGLU、位置编码换成 RoPE; 超参数 (d_{ff} 约为 d_{model} 的 2.67 到 4 倍、 h 乘 d_{head} 等于 d_{model} 、 d_{model} 除以层数约 100、词表 30k 或 100k 以上) 都落在一个很宽的"盆地"里, 怎么选都差不多; 真正还在变的是两件事: 怎么让训练不炸 (z -loss、QK-norm、logit soft-capping, 口诀是"哪里不稳就往哪里加一个 norm") 和怎么便宜地处理长上下文 (GQA 减 KV 头、滑窗与全注意力 3:1 交替、甚至混入 SSM 层) ——而这一切几乎都不是从理论推出来的, 是大家一次次训练试出来的。

时间轴

时间	内容
0:06	开场: 架构"不可捉摸", 用 survey 视角学别人的经验; A1 和原版的三处差异
3:08	样本量: 去年 19 个新 dense 模型, 今年 Qwen 3、Gemma 4、OLMo 3、Marin 8B; 多数新模型是 MoE (下一讲)
5:42	架构是三重折衷: 要泛化、要在 GPU 上高效、要不炸; 三段历史: 百花齐放 → 人人复刻 LLaMA 2 → 稳定性 → 长上下文
7:42	归一化放哪: 原版唯一被公认做错的地方; pre-norm 的两条证据; OPT-350M 例外; Grok / Gemma 2 / OLMo 2 的残差流外 post-norm
13:50	LayerNorm → RMSNorm: 0.17% 的 FLOPs 却占到 25% 的运行时间; 去掉 bias
19:59	激活函数动物园: ReLU → GELU → GLU 家族; 门控的定义; 2/3 缩放; Shazeer 与 Narang 的证据
27:09	串行 vs 并行 block: GPT-J、PaLM 的想法为何退潮; 着色总表看趋势

时间	内容
30:44	位置编码四代：sine · 绝对 · 相对偏置 · RoPE；RoPE 的相对性要求与旋转直觉、分对旋转、实现
38:56	问答：高维旋转、怎么学这些知识、并行层到底掉多少、p-RoPE、相对偏置为何不算"嵌入"
43:31	超参数一：d_ff 与 d_model 的比例——4、2.67、Llama 2 的 3.5、T5 的 64；Kaplan 的盆地
49:37	超参数二、三：h 乘 d_head 等于 d_model；aspect ratio 约 100，深了难做 pipeline 并行
55:13	超参数四：词表——单语 30k 与多语 100k 以上；问答：多模态词表、跨 tokenizer 比 bits per byte
59:22	正则化的反直觉：单轮训练不过拟合，weight decay 却仍流行——它是优化手段；问答：diffusion LM、训练中改什么
1:05:02	稳定性：loss 尖峰的代价；两个 softmax 是危险区；输出端的 z-loss
1:09:38	attention 端：QK-norm（来自多模态模型）；logit soft-capping (Gemma)；NVIDIA 的对比
1:13:47	推理成本：训练 / prefill 与逐 token 解码的算术强度；KV cache 为什么让强度塌掉
1:19:19	MQA 共享 K、V；GQA 的折衷与 T5 时代的证据；问答：还搜不搜超参、MQA 是训练时定的
1:25:26	长上下文：GPT-3 就有的滑窗交替；Command A 的 3:1；NoPE；Llama 4 / Gemma 4 / OLMo 3；Qwen 3.5 混入 Gated DeltaNet；结语

核心内容

1. 这一讲怎么讲：survey 视角，以及"架构是一组折衷"

- **方法论：**讲者坦言架构对他也一直"不可捉摸"——理想世界里记住 VC dimension 这类干净理论就够了，现实不是。最好的学法是自己训模型换架构试（这门课的哲学），但谁也没有算力跑遍设计空间；退而求其次是看别人做了什么：把现代模型摆在一起，找出哪些选择在一切有效架构里固定不变、哪些随便变也不影响效果。
- **从 A1 说起：**作业里让实现的和 Vaswani 原版有三处不同——LayerNorm 挪到每个 block 的前面（pre-norm）、位置用 RoPE、激活用 SwiGLU 而不是 ReLU。为什么选这三样？很大程度是从 LLaMA 抄的，"但大家都这么抄"；真要自己训模型，马上会撞上"这么多选项选哪个"的问题。
- **样本量够大：**去年备课时数了一下，新出的 dense 模型有 19 个（Qwen 2、Gemma 3、InternLM2、Nemotron 4.....）；今年以为总

该少了，结果还是一堆：Qwen 3、上周四刚发的 Gemma 4、OLMo 3，还有 Percy 用 Marin 训的 8B。只是今年多数新模型是 MoE，留到下一讲。模型多反而好——趋势看得清。

- **贯穿全讲的观点：**架构是一组复杂的折衷。一个架构要同时做到三件事——从数据里学到东西（泛化）、在 GPU 上高效训练、训练中途不炸（loss 一路下降然后突然飞掉，就前功尽弃）。三个要求全被"烤"进了架构里，所以架构显得杂乱、不优雅，这是它本来的样子。
- **三段历史：**从 Transformer 到 GPT-3 是百花齐放期，没有金标准；LLaMA 2 一出，人人复刻 LLaMA 2 再加一点小改；去年的主题是让训练更稳的架构改动；今年的主题是支持更长上下文的改动。
- **本讲范围：**只讲 dense 的、全对全的 attention。SSM、线性 attention 等替代方案是第 4 讲的事。

2. 一个 block 上有哪些旋钮

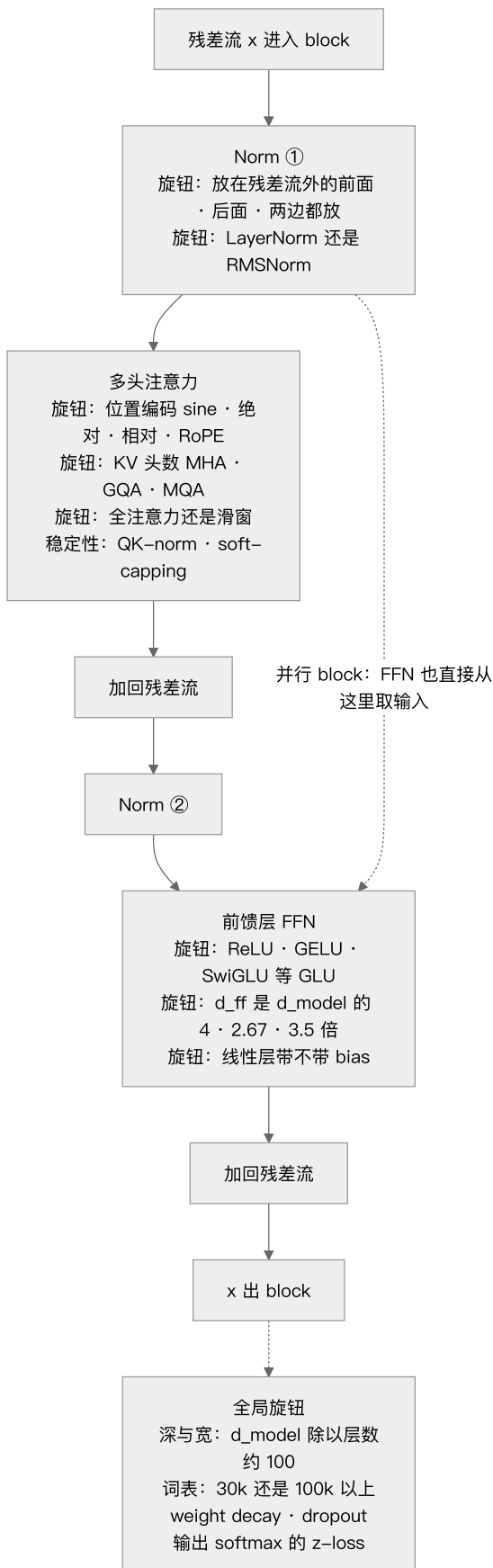


图 3-1 | 一个 Transformer block 的解剖：本讲逐个讨论的旋钮标在它们所在的位置（自绘示意）· [▶ 看原幻灯片 4:09](#)

- 图 3-1 是这一讲的地图，三个层次自上而下：block 内部的 building blocks（第 3 到 7 节）、再往下一层的超参数与正则化（第 8、9 节，图中右下角）、最底层的稳定性技巧（第 10 节）——后者放进架构课讲，因为它和架构变体绑在一起；attention 框里的"KV 头数、滑窗"是第 11、12 节。
- 读者在 CME295 第 2 讲见过 RoPE、RMSNorm、GQA 的定义，这里不重复，重点放在"为什么选它"的证据、每个选择省下或付出的是什么，以及谁用了什么——讲者开场的那张总表（29:43 有着色版）在第 13 节按讲课内容重建。

3. 归一化放哪：唯一被公认"原版做错了"的地方

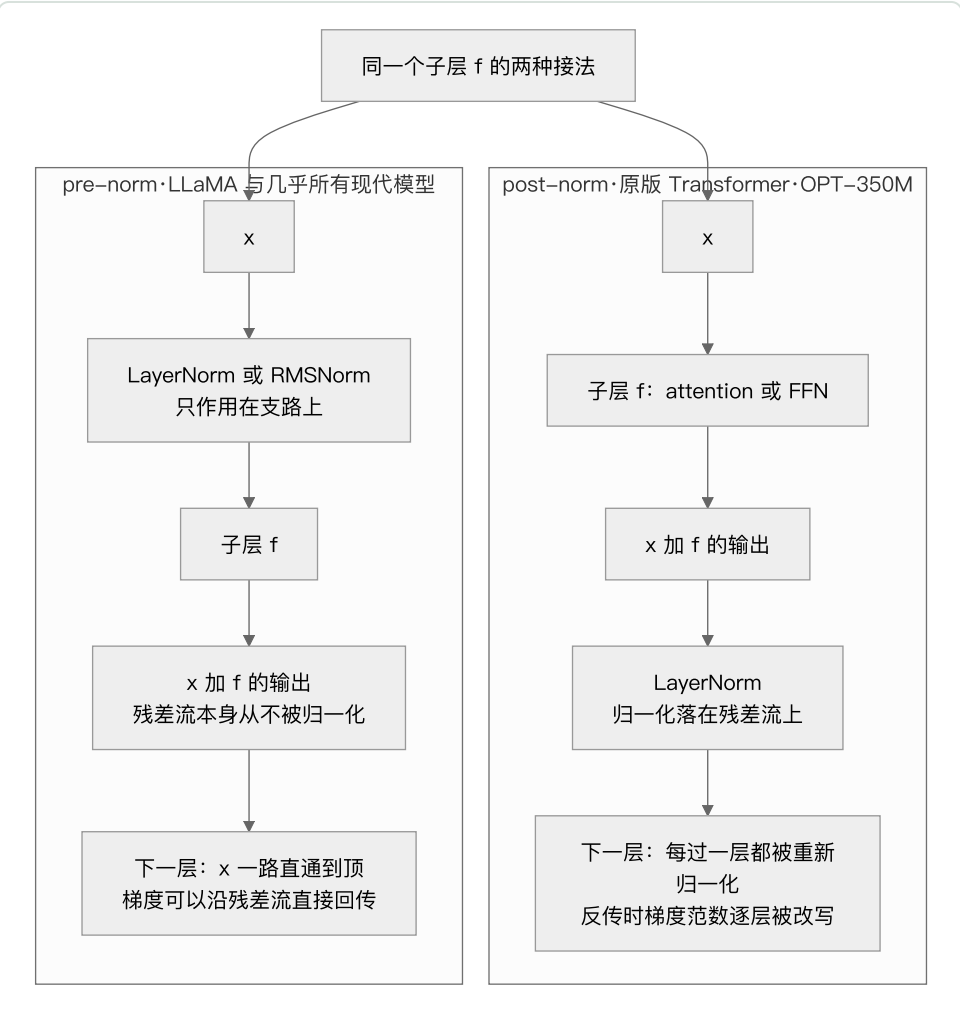


图 3-2 | post-norm 与 pre-norm 的数据流：区别只在 norm 落不落在残差流上（自绘示意） · [▶ 看原幻灯片 7:42](#) · 出处: Xiong et al., 2020

- **大家唯一的共识：**Transformer 原论文几乎什么都做对了，只有一处——LayerNorm 放在哪——被公认做错了。原版把 norm 放在残差路径上（每个子层的输出加回残差流之后再归一化）；替代方案是把

norm 挪到残差流之外、放在每个子层计算之前，称 pre-norm（讲者建议把原版叫"residual norm"，免得术语混淆）。基本上所有现代模型都把 norm 推出了残差流。唯一有趣的例外是 OPT-350M：OPT 本来就是个乱糟糟的模型系列，而只有 350M 这一档用了残差流内的 post-norm，没人知道为什么。

- **证据一：去 warm-up 的初衷。**早期研究这个问题的动机是想去掉学习率 warm-up（现代训练仍然要 warm-up）。结果发现 post-norm 一去 warm-up 就收敛得很差（图里那条紫色虚线），而 pre-norm 即使没有 warm-up 也收敛得很好。
- **证据二：梯度衰减。**这才是它留下来的原因。做架构设计的人常说一句话：keep your residual stream clean。pre-norm 下，x 从最底层一路直通到最顶层，反传时梯度也沿这条直线原路回去；初始化时各层的梯度大小基本一致。post-norm 下每过一个 block 就被归一化一次，梯度范数在反传过程中逐层被改写，效应复杂。实验上，pre-norm 的梯度尖峰更小也更少（Salazar 与 Nguyen 的图）。稳定、能做深，这两点对大模型都太重要，于是人人采用。

小注：去 warm-up 那条曲线应出自 Xiong et al., 2020 《On Layer Normalization in the Transformer Architecture》；梯度尖峰的图出自 Salazar & Nguyen, 2019 《Transformers without Tears》——讲者只报了作者名。

- **变体：**既然要点是"别碰残差流"，norm 放在子层计算之后（仍在残差流外）逻辑上同样成立——Grok、Gemma 2、OLMo 2 就是这么做的；还有模型两头都放（double norm）。由此得到一条听上去荒唐却屡试不爽的经验：训练不稳，就往各处撒 LayerNorm，"往 attention 里也扔一个"照样管用（第 10 节的 QK-norm）。
- **一个诚实的备注：**pre-norm 之所以是主流，可能有一部分只是因为 LLaMA 2 这么做了。

```
def block_post_norm(x):                # 原版：norm 落在残差流上
    x = norm(x + attn(x))
    return norm(x + ffn(x))

def block_pre_norm(x):                 # 现代：残差流干净，norm 只在支路上
    x = x + attn(norm1(x))
    return x + ffn(norm2(x))

def block_parallel(x):                 # GPT-J / PaLM：两条支路同时从同一个
```

norm 出发

```
h = norm(x)
return x + attn(h) + ffn(h)
```

三种 block 的接法只差一行的位置：post-norm 让残差流本身经过 norm，pre-norm 让 x 直通，并行 block（第 6 节）则把 attention 和 FFN 并排接在同一个归一化后的输入上。

4. LayerNorm → RMSNorm、去掉 bias：一笔算术强度的账

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}}$$

x 是一个 token 的 d 维激活， μ 、 σ^2 是它各维的均值和方差， γ 、 β 是可学的缩放和偏移；RMSNorm 去掉了减均值和 β ，只剩“按均方根缩小、再按 γ 放大”。

- **表达力**：LayerNorm 严格比 RMSNorm 更有表达力，表示上没有任何理由非用 RMSNorm；但实践中换过去没有损失——建模能力一样。
- **为什么还要换**：这里开始出现系统与架构的协同设计。上一讲 Percy 讲了 **arithmetic intensity**（算术强度：每从内存搬一个字节，能做多少次浮点运算）：GPU 要靠矩阵乘这类“算得多、搬得少”的操作才能跑满，搬一小块内存、算一点点的操作是浪费。LayerNorm 正是后者——只要减均值和加偏移不带来什么，就该删掉。
- **数字**：LayerNorm 只占总 FLOPs 的 **0.17%**，但视工作负载可以占到 **25% 的运行时间**——FLOPs 不是运行时间。矩阵乘（tensor contraction）的大头是乘法，统计归一化的大头是内存搬运，而激活很大。25% 是很小的模型上的极端数字，但道理一样；柱状图里白色是算术强度、黑色是 FLOPs，LayerNorm 最低。
- **实验证据**：Google 的一篇架构对比论文（Narang et al., 讲者说“2020”）在约 200M 参数的模型上把 LayerNorm 换成 RMSNorm：每秒的训练步数更多（表里的第三列），而且效果还更好——后者没人能保证，算意外之喜。一个免费的系统收益，于是所有人都换了。

小注：这篇应为 Narang et al., 2021 《Do Transformer Modifications Transfer Across Implementations and Applications?》(EMNLP 2021)，用的是 T5 架构而非自回归模型；第 5 节的 GLU 对比也出自它。

- **bias 项**：同一逻辑。原版每个线性层都带 bias，大多数实现全部去掉——同样是算术强度低、内存开销相对高的操作；顺带一提，bias 有

时还会引发稳定性问题。去掉它的首要理由是系统上更简单。

- **这一节的认识论：**norm 这块做法高度标准化，理解虽不算深刻理论但够用；让人不满足的是，“去掉 bias 行不行”无法事先推理，只能靠实验积累的集体知识——对典型语言建模负载，线性层和 RMSNorm 都不要 bias，没问题。

5. 激活函数：一整个动物园，只有“门控”这一条轴重要

$$\text{FFN}_{\text{ReLU}}(x) = \max(0, xW_1)W_2, \quad \text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW_1) \odot xV)W_2, \quad \text{Swish}(z) = z\sigma(z)$$

W_1 把 d_{model} 升到 d_{ff} ， W_2 降回来；GLU 变体多了一个同样大小的门矩阵 V ，用 xV 逐元素地调制主支路的输出。三个矩阵而不是两个，所以按 2/3 缩小 d_{ff} 才能和原 FFN 参数量相同——通行的经验规则，不是铁律，后面的超参数一节还会用到。

- **动物园：**ReLU、GELU、Swish、ELU、GeGLU、SeLU、SwiGLU、LiGLU……讲者曾发誓一辈子不学 SwiGLU 是什么，现在不得不学——要紧的是弄清名字里哪一部分真的影响效果。
- **朴素激活也能训：**只用 ReLU 就能训出像样的语言模型（那一组里 Chinchilla 大概是最好的）；GELU 只是在零附近多了一个小凹陷，改变的是零点附近的梯度，GPT-3 就是 GELU——按今天的标准不算先进，但完全能用。
- **真正的动作在 GLU：**和 norm 一样，几乎所有靠谱的现代模型都用某种 gated linear unit。做法（22:02）就是把架构设计里那句“门控通常有帮助”套到 FFN 上：再来一个线性投影 xV ，逐元素乘到激活的输出上，然后照常降维。命名规则是“激活名 + GLU”：ReLU、GeGLU、SwiGLU。
- **谁用什么：**Google 系用 GeGLU（Gemma、T5）；LLaMA 后裔和 PaLM 用 SwiGLU。SwiGLU 占多数，但讲者的判断是：在门控单元之间，选哪个并不重要。
- **证据：**Shazeer 提出 GLU 变体的论文里改进幅度小但方向一致，他一贯训多个副本给出误差条，对比按 2/3 调整做到参数量匹配——GLU 变体几乎总是更好。Narang et al. 的系统对比（T5 架构）里，SwiGLU、GeGLU 在 loss 和下游指标上都显著更好。结论：门控这条轴重要，且几乎不增加计算量。
- **不是必需：**GPT-3 没用；Nemotron-4 340B 用了 squared ReLU——讲者称之为“疯狂的选择”，但也训成了。只是今天很少见到不用 GLU 的模型。

```

class SwiGLU(nn.Module):
    def __init__(self, d_model, d_ff):          # d_ff 取约 8/3 倍
d_model, 参数量与 4 倍的 ReLU FFN 相同
        self.w1 = Linear(d_model, d_ff, bias=False)  # 主支路
        self.v = Linear(d_model, d_ff, bias=False)  # 门
        self.w2 = Linear(d_ff, d_model, bias=False)  # 降回 d_model
    def forward(self, x):
        return self.w2(F.silu(self.w1(x)) * self.v(x))  # silu 即
Swish; 三处都不带 bias

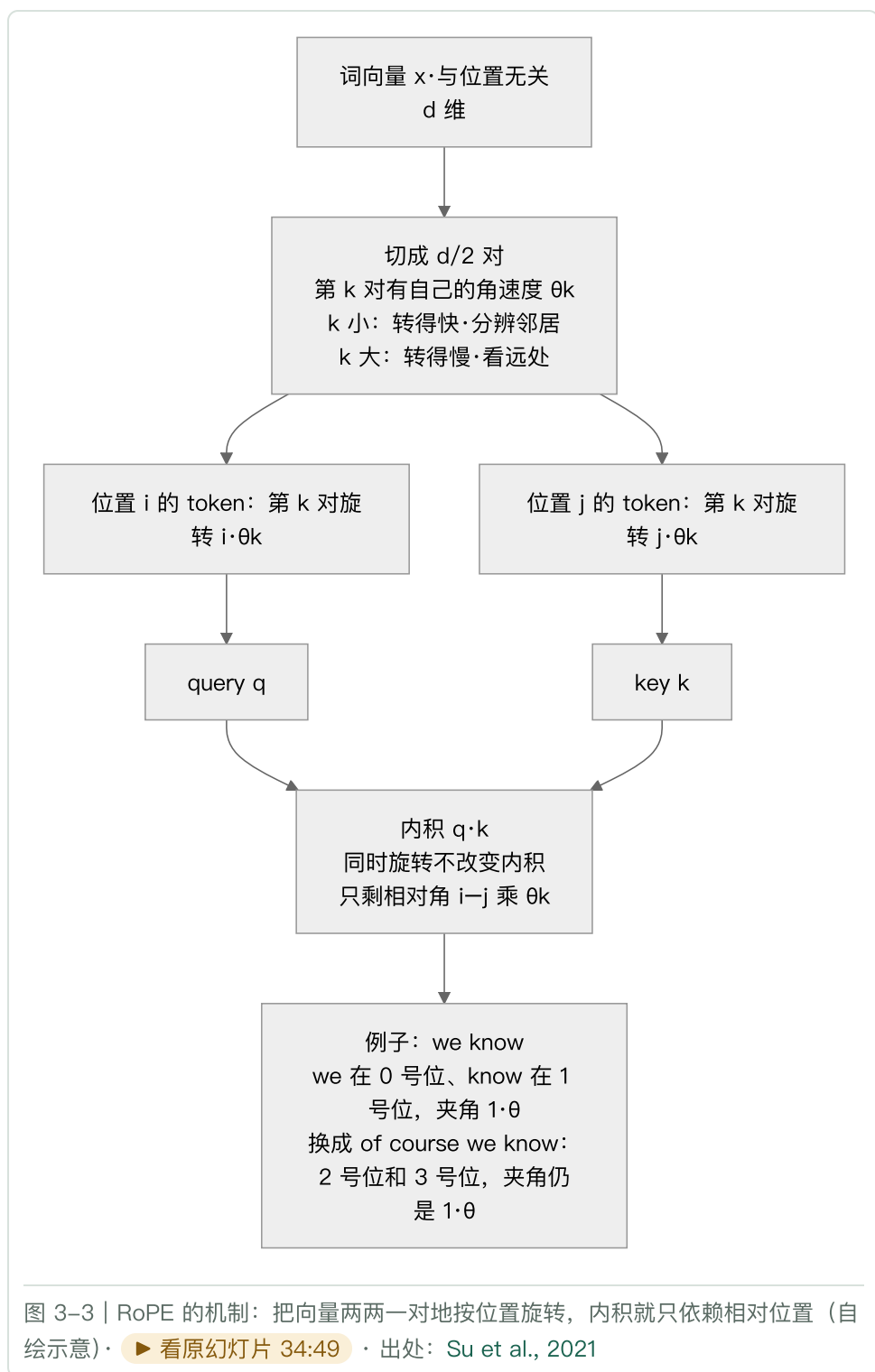
```

一个 SwiGLU 前馈层的骨架：比 ReLU FFN 只多一个门矩阵和一次逐元素乘法，bias 全部去掉。

6. 串行 vs 并行 block：一个退潮的想法

- **想法**：正常的 block 是串行的——先算 attention，再算 MLP。系统思维的人会说这是个瓶颈：后者要等前者。并行 block 把两条支路接在同一个归一化后的输入上，两者的输出一起加回残差流（见第 3 节的 `block_parallel`）。实现得当可以共享 LayerNorm、融合矩阵乘，换来额外的系统优化空间。
- **出处**：最早是 GPT-J（GPT-3 的开源复刻，影响出人意料），PaLM 的报告写得最清楚；受 Google 影响的 Cohere（由一位 Transformer 作者创办）也跟进了，除此之外用的人不多。
- **为什么退潮**：过去两年基本没人再用：串行形式的系统优化已经够好，并行的收益抵不上它对表达力的小幅损伤——直观上相当于丢掉一半的深度。
- **问答 (40:27)**：两者精度差多少？证据混杂。PaLM 论文很自信：效果不降、系统利用率提升 15%；但后来的 Google 模型都不再用，可当作隐含信号——可能有损失。没人做过干净的受控消融。
- **小结**：架构这一部分讲得这么短，本身就说明原版 Transformer 经受住了时间考验——在 dense attention 的范围内，真正改的只有 norm 的位置、bias、门控，都是小改动。

7. 位置编码：RoPE 为什么赢



- **为什么需要:** attention 是一堆内积, 把 token 打乱顺序结果不变——位置信息必须另外注入。这是各家实现差异最大、仍在变动的地方。
- **四代做法:** (1) 原版的 sine / cosine, 直觉像傅里叶变换, 总能从中恢复出位置; (2) 紧随其后的一批大模型用可学的绝对位置嵌入; (3) Google 系 (T5、Chinchilla) 用相对位置偏置: 不动词向量, 按两个位置的距离往 attention 分数矩阵上加偏移; (4) RoPE: 2024 年之

后的模型几乎都用它，而它"凭空出现"——最初是一位中国作者的博客加论文，被 GPT-J 采用后传开。

- **RoPE 想要的性质 (33:18)** ——一个有立场的假设：词的绝对位置不该被关心，"an apple"出现在开头和结尾应该得到同样的结果：

$$\langle f(x_i, i), f(x_j, j) \rangle = g(x_i, x_j, i - j)$$

f 把词向量 x 和它的绝对位置 i 编码成一个向量，要求任意两个这样的向量做内积时，结果只通过 $i-j$ 依赖位置。此前的方案都不满足：sine 嵌入的内积里有绝对位置的交叉项；绝对嵌入顾名思义不满足；相对偏置虽然"相对"，但它不是嵌入——没有内积结构可言。

- **几何直觉**：内积对旋转不变。把与位置无关的语义向量按它所在的位置旋转：位置 0 不转，位置 1 转 θ ，位置 2 转 2θ"we know"里 we 和 know 夹角 θ ；换成"of course we know"，两个词到了 2 号位和 3 号位，夹角还是 θ (图 3-3)。
- **高维怎么转**：二维只有顺时针、逆时针；高维有无穷多种旋转。做法是最简单的那种，而且管用——把 d 维切成 $d/2$ 对，每一对做一次二维旋转，各对角速度 θ_k 不同：转得慢的对捕捉长程关系，转得快的对分辨"是不是邻居"。论文用复数写了一套很绕的动机，讲者认为直觉就是"分对旋转"。

$$f(x, m)_{[2k, 2k+1]} = \begin{pmatrix} \cos m\theta_k & -\sin m\theta_k \\ \sin m\theta_k & \cos m\theta_k \end{pmatrix} \begin{pmatrix} x_{2k} \\ x_{2k+1} \end{pmatrix}, \quad \theta_k = \Theta^{-2k/d}$$

m 是 token 的绝对位置，第 k 对坐标 (x_{2k}, x_{2k+1}) 被旋转 $m \cdot \theta_k$ ； θ_k 随 k 按几何级数变小，所以前面的对转得快、后面的对转得慢。

小注： θ_k 的具体形式课上没写，这里按 RoFormer (Su et al., 2021) 补出，论文默认基数 $\Theta = 10000$ ；作者 Jianlin Su 的博客先于论文。

- **实现 (38:24)**：按位置 id 生成 cos 和 sin，作用到 query 和 key 上 (不动 value)，写成稀疏矩阵乘或手工按对旋转都行。关键有两点：一是乘上正弦余弦而不是加——没有交叉项，内积里榨不出绝对位置；二是在每一层 attention 里都做，而不是只在最底层加一次——每次算 attention 都重新强制相对性。
- **Gemma 4 的 p-RoPE (上周四发布)**：只旋转一部分坐标对。问答里讲者解释差别只是"转哪些坐标"：角速度最低的那些对本来就几乎不转，缺空间时可以省掉——这是给隐藏维度很小的小模型准备的优化。

小注：p-RoPE 应出自 Barbero et al., 2024 《Round and Round We Go! What makes Rotary Positional Encodings useful?》，做法是截掉最低频的那部分频率（推断）。

- **问答：**超出二维块的高维旋转？讲者没见过，理论上任何闭合的一维流形都行。ALiBi 一类直接往 attention 矩阵上加偏置的方法效果也不错，问题是"审美"上的：不能分解成 $f(x_i, i)$ 与 $f(x_j, j)$ 的内积；接受了"我要相对性"这个前提，就会自然走到 RoPE。怎么学这些知识？看得足够广以找出模式（这一讲在做的事），加上自己在小规模上试出直觉——今天没有哪篇论文会给出一个模型的全部细节。

```
def rope(x, pos, theta):
    # x: [..., seq, d];
    theta: [d/2] 各对的角速度
    x1, x2 = x[..., 0::2], x[..., 1::2]    # 第 k 对 = 第 2k 和 2k+1
    维
    ang = pos[:, None] * theta[None, :]    # [seq, d/2]: 位置 m 乘
    角
    c, s = ang.cos(), ang.sin()
    return interleave(x1 * c - x2 * s, x1 * s + x2 * c)  # 每一对做二
    维旋转

q, k = rope(q, pos, theta), rope(k, pos, theta)  # 只旋转 q 和 k, v 不
    动
scores = q @ k.transpose(-1, -2) / d_head ** 0.5  # 内积只依赖相对位置
```

RoPE 的实现就是"生成角度、分对旋转、只作用于 q 和 k"三步；每一层 attention 里都重做一遍。

8. 超参数：四条经验值，以及它们的"盆地"

- **什么时候会遇到：**一旦要实例化模型，就得回答 d_ff 多大、几个头、词表多大、要不要 weight decay 或 dropout、做深还是做宽。从零搜这个高维空间很吓人，但人们实际尝试过的范围很小。

超参数	经验值	例外与证据
d_ff 与 d_model 之比	4 (非 GLU)；GLU 取 2/3 后约 2.67，很多模型在 2.5 到 2.67	Llama 2 因为 GQA 省了 attention 的算力，"随手"再乘 1.33 得约 3.5；T5 用了 64 倍，T5 1.1 又退回约 2.5；Kaplan 的扫描显示 1 到 10 之间是平坦盆地
头数与头维度	h 乘 d_head 等于 d_model，即"和单头时总维度相同"	T5、LaMDA 是例外；盆地也很宽，比值在 1 附近都行

超参数	经验值	例外与证据
aspect ratio: d_model 除以层数	约 100 (GPT-3、LLaMA 都是)	深了要做 pipeline 并行，痛苦；宽了做 tensor 并行，简单；Kaplan：最优值不随规模变，约 100 或略低；Tay et al.：起决定作用的是 FLOPs
词表大小	单语（英语）模型约 30k；多语 / 生产系统 100k 到 200k 以上	Google 系词表远大；LLaMA 后裔约 100k；scaling law 表明模型越大能用的词表越大；没人再训大的单语模型了

- **d_ff 的比例**：4 倍这个经验值好得出奇。GLU 变体按第 5 节的 2/3 缩放，得到 2.5 到 2.67；Llama 2 团队觉得有了 MQA / GQA（第 11 节）attention 变便宜了，就把比例再乘 1.33 得约 3.5——把预算往 MLP 挪了一点。翻遍论文，GLU 模型不是 2.6 左右就是 3.5，非 GLU 是 4。
- **T5 的传奇**：多数技术报告在架构上很无聊——“做了 LLaMA，改了一处”；Google 有时很大胆，T5 直接用了 64 倍，理由也是系统式的：矩阵越大，硬件越好跑满。Gemma 2 也往高里推了些。Kaplan et al. 2020 顺手扫过这个比例（很小的模型）：从约 1 到约 10 是一片平坦盆地，离最优 loss 差得很少；超过 10 到 100，loss 二次方地飙升。所以 2.6 到 4 都在盆地里；T5 不是坏模型，激进选择“技术上可行”，只是算力效率差——收尾是 T5 1.1 一声不响地退回了约 2.5。
- **头**：讲 224N 时他就觉得奇怪——大家总让 h 乘 d_head 刚好等于 d_model，仿佛必须和单头时总维度一样。这没有必然性，但几乎所有模型（包括最新的）都遵守，也确实好用；T5 和 LaMDA 是例外。同样宽容。
- **深与宽**：放大模型时通常固定 aspect ratio 整体变大，所以这个比值控制了整条 scaling 路线上的深宽取舍。它的差异比别的超参数大得多，但仍有清楚的甜点：约 100。取舍是表达力对硬件：极深的模型只能沿层切（pipeline 并行，第 7、8 讲，“多数人真的不想碰”）；宽模型沿宽度切就行（tensor 并行，简单得多）。表达力想深、系统想宽，落在 100 附近。Kaplan 的另一张扫描图显示最优 aspect ratio 不随模型大小变；Tay et al. 扫过各种深宽组合，结论是主导变量是 FLOPs，形状次要。讲者的总结：有一条宽容的超参数带，选在带内之后主要操心系统利用率，而不是难以推理的表达力。

小注：“Tay et al.” 在字幕里是 “ETA”，应为 Tay et al., 2021 《Scale Efficiently》；那篇论文同时指出模型形状对下游微调有影响，并提出“深而窄”更省算力——讲者引用的是它上面那幅“按 FLOPs 看各种形状”的图。

- **词表**：早期开源模型多是只求英语好的单语模型，词表 3 万左右；LLaMA 之后大家关心多语和生产系统（包括 GPT-4 这类闭源模型），词表升到 10 万到 20 万以上。多语确实需要更大的词表覆盖；右边那一列的模型本身也更大——scaling law 研究表明模型越大、能驾驭的词表越大。第 1 讲的分词与这里相接。
- **问答 (56:49)**：多模态呢？如果图像也切成 token，就需要一个自己的、通常很大的图像词表。不同 tokenizer 之间比 bits per byte 合法吗？合法，只要 tokenizer 是“完备”的（能建模任意字节串，现代子词分词器都能，早年丢词的分词器不行），且归一化用的是同一个数——字节数；困惑度与 bits per byte 互为对偶。

9. 正则化的反直觉：weight decay 是优化手段，不是防过拟合

- **ML 101 的推理**：语言建模的数据比算力多（互联网上的数据比 FLOPs 多，也许 Google 也不例外），所以语料只过一遍；单轮 SGD 基本不会记住数据；于是在算力受限的语言建模里过拟合几乎从不发生——有人因此只看训练 loss。那还要 dropout 或 weight decay 吗？
- **现实**：最近的技术报告很少写这么底层的细节，但翻一翻会发现很多模型两者都用；weight decay 在现代高性能模型里尤其普遍（dropout 倒是退潮了）。这很费解。
- **解释**：有论文给出了漂亮的证据——weight decay 在这里根本不是正则化：不同 weight decay 下，训练 loss 对验证 loss 全落在 x 等于 y 的对角线上，本来就没有过拟合可消。但和学习率衰减放在一起看，强 weight decay 的那几条（图中底部的蓝色虚线）起步慢、最终却收敛到明显更好的极小值；常数学习率下没有这个效应。所以 weight decay 是在和优化器、学习率调度交互——是优化干预。

小注：应为 D'Angelo、Andriushchenko 等，2023 《Why Do We Need Weight Decay in Modern Deep Learning?》：对接近单轮的 LLM 训练，weight decay 通过平衡随机优化里的偏差一方差，降低训练 loss 并改善训练稳定性。

- **这就是课程要你动手的原因**：这些东西以奇怪的方式相互纠缠，无法从零推理。问答里的补充：dropout 基本没人用了，和优化配合不好；weight decay 把权重往零收缩，可能让你用更高的学习率或衰减得更快。训练途中会改的超参数几乎只有 weight decay（和学习率一起改，是个好用的启发式）；架构类的超参数改了训练就不兼容。

- 问答：diffusion 语言模型的架构有没有不同？讲者没深究；训大 diffusion 的人少，而且多数是拿 LLaMA 式架构改装的，从零训的最优架构他不知道。

10. 稳定性三件套：z-loss、QK-norm、logit soft-capping

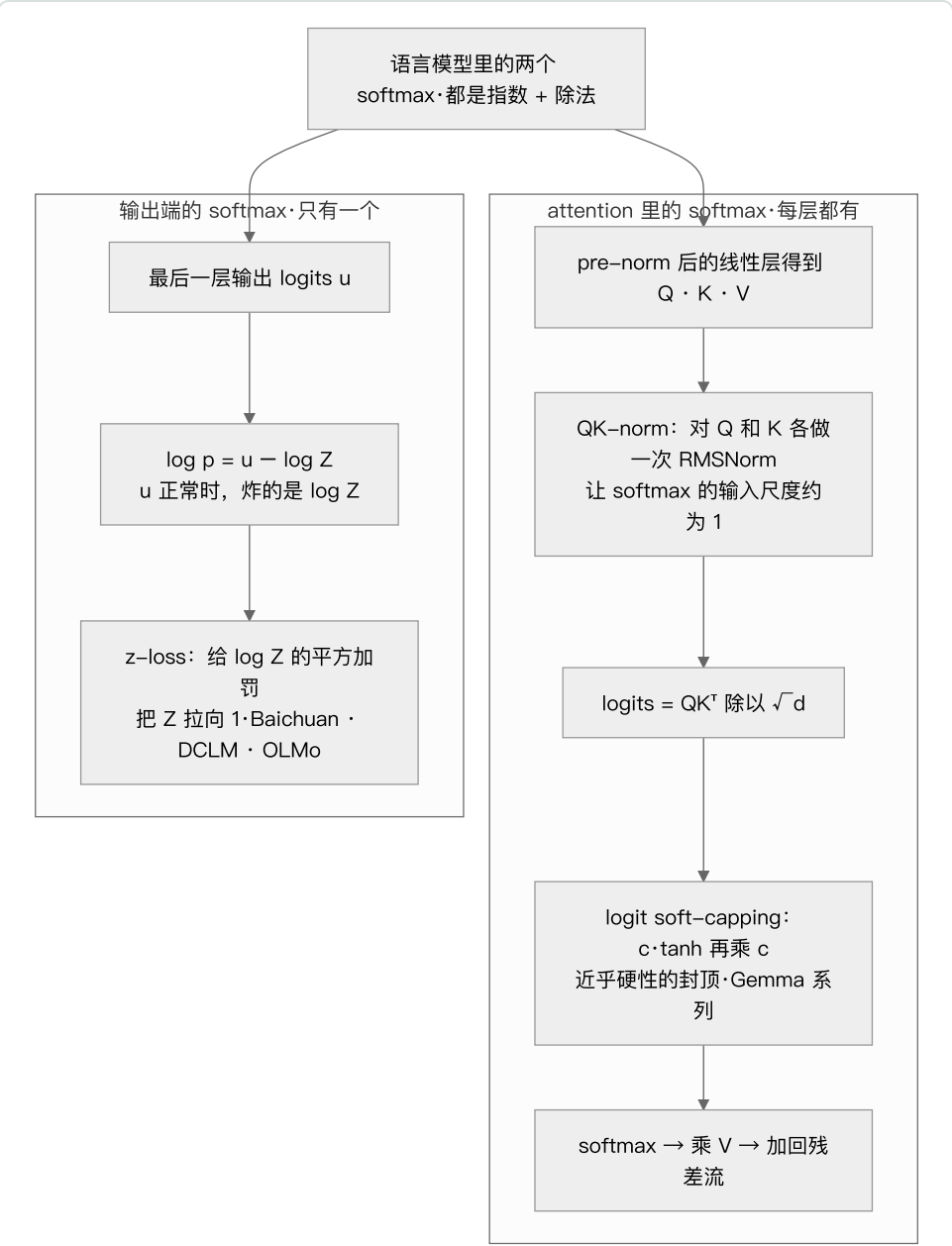


图 3-4 | 两个 softmax 就是两个危险区：三种稳定性干预各卡在哪一步（自绘示意）
· ▶ 看原幻灯片 1:10:09 · 出处：Rybakov et al., 2024

- 为什么稳定性成了主题：前面的选择在效果上都很宽容，怎么选差别不大；但模型越贵，“训到一半炸掉”越不可接受——花了几百万美元，模型要么质量受损，要么再也训不动。不想要的就是那条到处是尖峰、梯度范数乱跳的蓝色曲线。

- **通常的嫌疑犯**：softmax。它有两样对稳定性都很坏的东西——指数（很快爆掉）和除法（同样危险）。语言模型里有两个 softmax：输出端的概率分布，和 attention 里的归一化，后者尤其危险。

$$\log p(y) = u_y - \log Z, \quad Z = \sum_v e^{u_v}, \quad \mathcal{L}_{z\text{-loss}} = -\log p(y) + \alpha (\log Z)^2$$

u 是模型输出的 logits（残差流最后的输出）， Z 是配分函数；只要模型本身正常， u 就正常，会出问题的是 $\log Z$ —— Z 是指数和，可能极大，也可能趋近 0，两个方向都会炸。 $z\text{-loss}$ 把 $(\log Z)^2$ 作为惩罚项加进 loss，把 Z 拉向 1、 $\log Z$ 拉向 0。

- **输出端为什么可以这样做**：softmax 是过参数化的——给所有 u 加同一个常数，概率不变而 Z 会变，这个自由度正好拿来约束 Z 。这一招来自 Jacob Devlin 2014 年的论文（讲者口误说成 2024 又自己改正），近年被开源模型重新捡起：Baichuan 是讲者记得的第一个，之后 DCLM、OLMo 都用它稳住输出 softmax，出奇地有效。

小注：Devlin et al., 2014 《Fast and Robust Neural Network Joint Models for Statistical Machine Translation》，原目的是让网络"自归一化"以省掉推理时算 Z ；PaLM 也用了 $z\text{-loss}$ ，系数 10^{-4} 。

- **attention 端：QK-norm**。设计哲学就是那句"不稳就扔一个 norm 进去"：在 Q 和 K 相乘之前各做一次 RMSNorm，softmax 的输入尺度就总在 1 左右。它来自多模态那边——Idefics、Chameleon 用它证明了有效，之后开源语言模型发现同样适用；现在绝大多数大模型都做，不伤效果，但确实能挡住 attention 的退化。数一数 norm 的位置：pre-norm、子层之后、再加 Q 和 K 上——这就是这几年稳定性技巧的走向。

小注：QK-norm 更早见于 Henry et al., 2020 和 ViT-22B (Dehghani et al., 2023)；Chameleon 论文明确写了沿用 ViT-22B 的做法。

- **logit soft-capping**：更硬的干预，基本是 Google 专属。QK-norm 控制 softmax 的输入、指望输出正常；soft-capping 直接对进入 softmax 的 logits 封顶——tanh 有界，再大的 logit 也不会超过上限。讲者说 Gemma 2、3、4 都对 attention 的 logits 这么做。

$$\text{logits} \leftarrow c \cdot \tanh\left(\frac{\text{logits}}{c}\right)$$

c 是封顶值：logit 远小于 c 时几乎不变，接近或超过 c 时被压到 c 以内——一个"软"的硬约束。

- **代价：**NVIDIA 的一组系统对比里，基线之上加 QK-norm 略好——学习率可以再调高一点；只用 soft-capping 反而掉点：干预太强，模型没法在 softmax 里表达超过上限的高置信度。安全，但不免费。

小注：Gemma 2 的封顶值是 attention logits 50、最终输出 logits 30；Gemma 3 的报告称把 attention 的 soft-capping 换成了 QK-norm（应为）。NVIDIA 那篇应为 Rybakov et al., 2024 《Methods of improving LLM training stability》，用 830M 模型故意调高学习率制造发散来比较各种干预。

11. 推理成本与 MQA / GQA

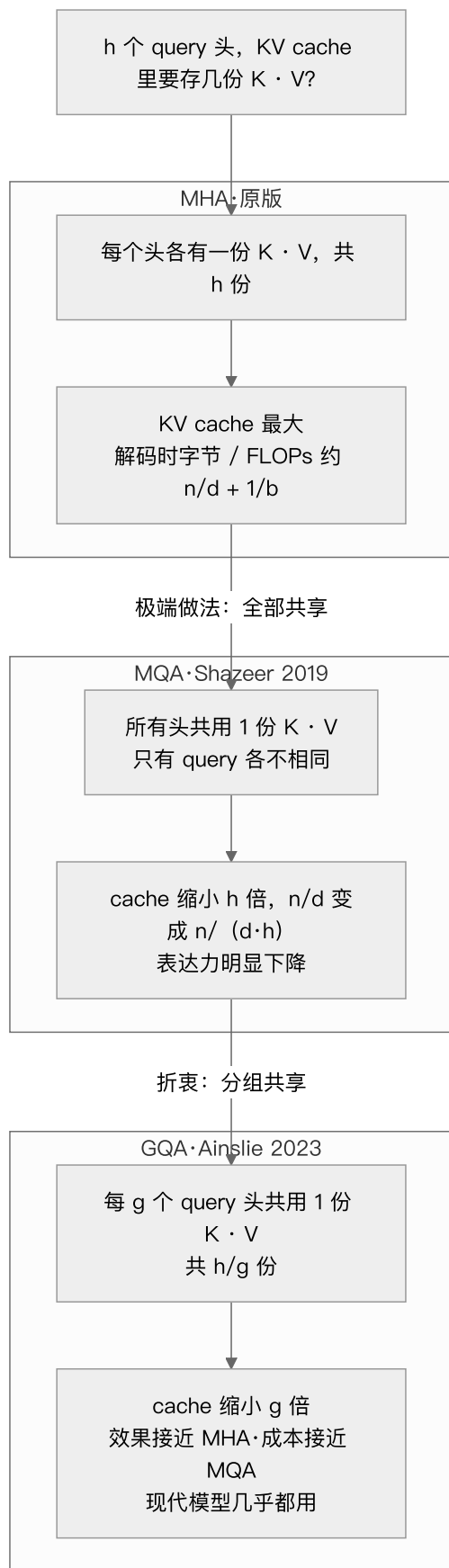


图 3-5 | MHA、MQA、GQA 只差一件事: 多少个 query 头共用一份 K 、 V (自绘示意) · [▶ 看原幻灯片 1:20:50](#) · 出处: [Ainslie et al., 2023](#)

- **换个视角：部署。**训完的大模型要服务大量用户，成本抽象成两种资源：FLOPs（做的计算）和内存访问（搬的字节）——后者同样决定延迟和利用率，两者都要小。
- **训练 / prefill 的账 (1:15:48)：**

$$\text{FLOPs} \sim b n d^2, \quad \text{bytes} \sim b n d + b h n^2 + d^2, \quad \frac{\text{bytes}}{\text{FLOPs}} \sim \frac{1}{k} + \frac{1}{b n}$$

b 是 batch 大小， n 是序列长度， d 是隐藏维度， h 是头数， $k = d/h$ 是每个头的维度；三项字节分别是激活、attention 分数矩阵（softmax 那一项， n^2 量级）和权重。比值告诉你：只要头维度 k 够大、 b 乘 n 够大，搬运相对计算就很少，GPU 能跑满。

- **逐 token 解码的账：**生成没法并行——生成一个 token、接在后面、再生成下一个，这是自回归的“诅咒”。高效的做法是把过去所有位置的 K 和 V 存在 **KV cache** 里（讲者口误说成“keys and queries”），每步只算新 query 那一行，算过的子矩阵全部复用，FLOPs 大省。但算术强度塌了：每生成一个 token 都要把参数从头读一遍。

$$\text{decode: bytes} \sim b n^2 d + n d^2 \Rightarrow \frac{\text{bytes}}{\text{FLOPs}} \sim \frac{n}{d} + \frac{1}{b}, \quad \text{MQA: } \sim \frac{n}{d h} + \frac{1}{b}$$

总 FLOPs 和一次算完时相同（同样的矩阵，只是分步乘），但权重那一项从 d^2 变成了 $n \cdot d^2$ ——每步重读一次；比值里多出的 n/d 项要求“batch 大、序列短，或者 d 巨大”，小模型尤其吃亏，而且用增量计算就绕不开它。MQA 让所有头共用一份 K 、 V ，这一项被头数 h 除掉。

- **MQA：**只让 query 按头不同， K 和 V 全部共享。KV cache 与内存访问骤减，算术强度显著上升；代价是表达力明显下降。
- **GQA：**在系统效率和表达力之间找甜点——保留全部 query 头，只把 K 、 V 的头数减到一个可调的数目，这个比例就是旋钮。DeepSeek-V2 的 multi-head latent attention 是另一种分解，取舍不同，下一讲略提。
- **证据**（T5 时代的图，1:21:50）：横轴每样本耗时、纵轴下游效果。MHA 最好但最慢；MQA 快得多但差很多；为赶速度把模型做小，更差；GQA 几乎和 MHA 一样好、成本接近 MQA——KV 头数只要稍减一点，大部分效果就保住了。所以今天几乎所有模型都用 GQA。推理的机制 Percy 在第 10 讲展开。
- **问答：**MQA / GQA 不是推理时才做的技巧，训练时就定好了 KV 头数。还搜不搜超参？两者兼有：每次训练都有几条“什么可以变”的假设，报告里超参数很少动、架构一次改一处；Google 是少数敢大改

的，Gemma 4 甚至给每一层配了独立的 embedding 来换取内存与 FLOPs 之间的取舍。

小注：GQA 论文的另一卖点是可以把已训好的 MHA checkpoint 用约 5% 的原训练算力"改造"成 GQA，而不必从头训。

```
# h 个 query 头、h_kv 个 K/V 头；每 g = h // h_kv 个 query 头共用一份 K、V
q = wq(x).view(b, n, h, d_head)
k = wk(x).view(b, n, h_kv, d_head)      # h_kv = h → MHA; h_kv = 1
→ MQA
v = wv(x).view(b, n, h_kv, d_head)
k = k.repeat_interleave(h // h_kv, dim=2)  # 算 attention 时把 K、V 复制
给每个 query 头
v = v.repeat_interleave(h // h_kv, dim=2)  # 但 KV cache 里只存 h_kv
份
out = attention(q, k, v)
```

MHA、MQA、GQA 在代码上只是 K、V 投影的头数不同：存的时候是 h_kv 份，算的时候再复制。

12. 长上下文：全注意力与滑窗交替

- **老想法**：GPT-3 的论文就写着交替使用全注意力（看过去所有位置）和带状的局部注意力（只看固定窗口）；OpenAI 早年就有关于 attention 模式的工作。过去一年它重新流行：全局层和局部层交替，是长上下文效果与推理成本之间的甜点，不必上 SSM 之类的"异域"方案。
- **Command A 的结构**（讲者见到的第一个开源例子）：每 4 层一个全注意力层，中间 3 层是只看局部的滑窗注意力；往上走，局部信息逐层汇聚成全局信息。还有一项创新是把全局层的 RoPE 去掉——没有任何位置编码（NoPE），全局层几乎在看一个"词袋"，位置信息由短程层提供。
- **跟进者**：Llama 4、Gemma 4、OLMo 3 都是滑窗加全注意力的组合（用完整的 RoPE 而不是 NoPE）。Qwen 3.5 结构相同但"便宜层"换了：每 4 层一个全注意力，中间是一种叫 Gated DeltaNet 的状态空间模型——第 4 讲讲它是什么。
- **今年的主题**：混合模型——既不全是全局 attention，也不全是便宜层。长上下文的效果与成本怎么平衡，是架构上仍在活跃变动、变化最多的地方。

- **结语：**把这些模型放在一起，能看到大量共性（前面各节），也能看到差异集中在上下文处理、位置编码、甚至分词上。带着这些直觉去做作业、刷排行榜。

13. 总表与决策图：谁用什么

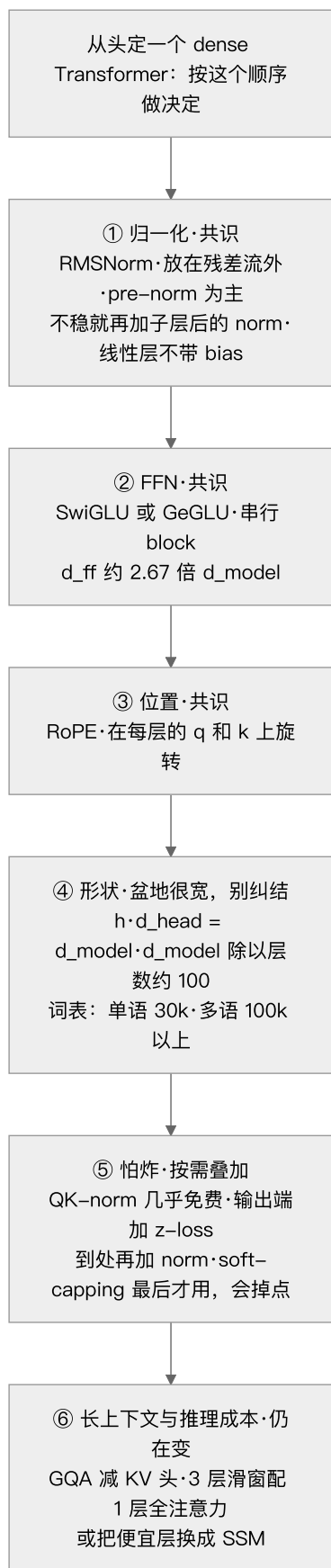


图 3-6 | 按本讲证据排出的决策顺序: 三条共识、一条宽容带、两块仍在变动的区域 (自绘示意) · [▶ 看原幻灯片 1:03:30](#)

模型	norm 位置 / 类型	激活	位置编码	block	课上点到的其他特点
Transfo rmer 原 版	残差流内 post- norm / LayerNorm	ReLU	sine / cosine	串行	线性层带 bias；h 乘 d_head 等于 d_model 的规则 由此而来
GPT-3	—	GELU	绝对 ("紧随 原版的一批 大模型", 推 断含 GPT- 3)	串行	论文里已交替使 用全注意力与带 状局部注意力； 非 GLU 模型的 d_ff 取 4 倍
GPT-J	—	—	RoPE (首 个采用)	并行 (首创)	GPT-3 的开源复 刻, 影响出人意 料
PaLM	—	SwiGL U	—	并行	报告称并行块效 果不降、利用率 升 15%
T5	—	GeGL U (T5 1.1)	相对偏置	串行	d_ff 64 倍 d_model；头维 度不守规则；T5 1.1 退回约 2.5 倍
Chinchil la	—	ReLU 一组里 最好的	相对偏置	串行	—
OPT- 350M	残差流内 post- norm	—	—	串行	讲者唯一找到的 "残差流内 post-norm"现代 例外
LLaMA 2	pre-norm / RMSNorm (A1 从 它抄来的那一套)	SwiGL U	RoPE	串行	因 MQA / GQA 省了算力而把 d_ff 提到约 3.5 倍；"人人复 刻"的模板
Grok · Gemma 2 · OLMo 2	子层之后、残差流 之外的 post- norm；总表里标 post 的不少其实前 后都放	Gemm a 用 GeGL U	—	串行	Gemma 2 到 4 用 logit soft- capping； Gemma 2 的 d_ff 比例也偏高
Falcon	—	非 GLU (GEL U, 推 断)	—	—	讲者点名的非 GLU 例外
Nemotr on-4 340B	—	squar ed ReLU	—	—	"疯狂但能训"

模型	norm	位置 / 类型	激活	位置编码	block	课上点到的其他特点
Cohere Command A	—		—	局部层 RoPE、全局层 NoPE	并行（讲者说 Cohere 沿用 Google 风格的并行块）	每 4 层 1 层全注意力、3 层滑窗
Baichuan · DCLM · OLMo	—		—	—	—	输出端 z-loss
Idefics · Chamelion	—		—	—	—	QK-norm 的来源
Llama 4 · Gemma 4 · OLMo 3	—		—	完整 RoPE；Gemma 4 另有 p-RoPE	—	滑窗与全注意力交替；Gemma 4 每层独立 embedding
Qwen 3.5	—		—	—	混合	每 4 层 1 层全注意力，其余是 Gated DeltaNet

小注：表中"—"表示课上没说。几个课外的词表数字供对照：LLaMA 1 与 T5 约 32k，GPT-4 约 100k，Llama 3 约 128k，Gemma 约 256k——与讲者"单语 30k、多语 100k 到 200k 以上、Google 最大"的分组一致。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
模型总表（开场版）	列：词表、norm、位置编码、pre / post、串行 / 并行、激活；先扫一眼有多少行	4:09	—
pre-norm 与 post-norm 结构图	左边 norm 落在残差路径上，右边 norm 在支路上、x 直通	7:42	Xiong et al., 2020
去掉 warm-up 的收敛曲线	紫色虚线（post-norm）不收敛，pre-norm 没有 warm-up 也收敛	10:15	同上
梯度尖峰对比	pre-norm 下尖峰更小、更少	12:19	Salazar & Nguyen, 2019
算术强度柱状图	白色是算术强度、黑色是 FLOPs；LayerNorm 一栏最低；旁边的 0.17% 与 25%	16:23	—

图	看什么	跳转	出处
RMSNorm 对比表	第三列 steps per second 更高，效果列也更好	17:24	Narang et al., 2021
GLU 变体对比表	参数量匹配下 GLU 行几乎全部更好；误差条来自多次重复	25:06	Shazeer, 2020
并行 block 的式子	串行是嵌套，并行是两项相加；能共享的 norm 和可融合的矩阵乘	28:12	PaLM
着色总表	蓝色是 RMSNorm、并行块；右侧几乎全是 GLU；post-norm 标记的多为前后都放	30:14	—
RoPE 二维旋转例子	we know 与 of course we know：绝对角变了，夹角不变	34:49	Su et al., 2021
d_ff 比例与 aspect ratio 的扫描	比例 1 到 10 是平坦盆地，往上二次方飙升；aspect ratio 的最优不随规模变、约 100	48:06 · 53:43	Kaplan et al., 2020
weight decay 图	左：训练 loss 对验证 loss 全在对角线上；右：强 weight decay 加学习率衰减最后最低	1:01:58	D'Angelo et al., 2023
稳定性干预对比	QK-norm 略好于基线（学习率能调高），soft-capping 单独用掉点	1:13:17	Rybakov et al., 2024
GQA 效果—耗时图	MHA 右上、MQA 左下、GQA 左上；小模型那一点更差	1:21:50	Ainslie et al., 2023
Command A 的层结构图	每 4 层一个全注意力，其余滑窗；图是倒着画的	1:25:56	Command A

提到的工作

名称	在本讲里的作用
Transformer (Vaswani et al., 2017)	基线：sine 位置编码、ReLU、残差流内 post-norm、带 bias；"几乎全对，只错了 norm 的位置"
On Layer Normalization in the Transformer Architecture (Xiong et al., 2020)	pre-norm 可以不用 warm-up 的那条曲线（应为）
Transformers without Tears (Salazar & Nguyen, 2019)	pre-norm 下梯度尖峰更小更少
RMSNorm (Zhang & Sennrich, 2019)	去掉减均值和 bias 的归一化
Do Transformer Modifications Transfer (Narang et al., 2021)	Google 的系统对比：RMSNorm 更快更好；GLU 显著更好；字幕里的 "Noam et al. 2020"
GLU Variants Improve Transformer (Shazeer, 2020)	ReGLU / GeGLU / SwiGLU 的提出；2/3 缩放；带误差条的参数匹配对比

名称	在本讲里的作用
GPT-J (Wang & Komatsuzaki, 2021)	并行 block 与 RoPE 的推广者
PaLM (Chowdhery et al., 2022)	并行 block 的正式描述; SwiGLU
OPT (Zhang et al., 2022)	350M 那一档是残差流内 post-norm 的唯一例外
GPT-3 (Brown et al., 2020)	GELU 的代表; 已交替使用全注意力与带状局部注意力
Chinchilla (Hoffmann et al., 2022)	朴素激活一组里最好的; 相对位置偏置
Nemotron-4 340B	squared ReLU 也能训
RoFormer (Su et al., 2021)	RoPE 的出处
ALiBi (Press et al., 2021)	问答里提到的"往 attention 矩阵上加偏置"一类方法
Round and Round We Go (Barbero et al., 2024)	p-RoPE 应出自这里 (推断)
T5 (Raffel et al., 2020)	d_ff 64 倍、头维度不守规则的大胆设定; 相对位置偏置; T5 1.1 退回 2.5 倍
Scaling Laws for Neural Language Models (Kaplan et al., 2020)	d_ff 比例和 aspect ratio 的扫描: 宽盆地、最优约 100
Scale Efficiently (Tay et al., 2021)	深宽扫描: FLOPs 是主导变量 (字幕 "ETA", 应为)
LaMDA	头维度规则的另一个例外
Llama 2 (Touvron et al., 2023)	d_ff 乘 1.33 得 3.5; GQA; "人人复刻"的模板
Why Do We Need Weight Decay in Modern Deep Learning? (D'Angelo et al., 2023)	weight decay 是优化干预: 训练 / 验证 loss 在对角线上, 配合学习率衰减才见效 (应为)
Devlin et al., 2014	z-loss 的最初来源
Baichuan 2 · DCLM · OLMo 2	用 z-loss 稳住输出 softmax; OLMo 2 也是残差流外 post-norm 的例子
Idefics / OBELICS · Chameleon	QK-norm 在多模态模型里被证明有效
Gemma 2 · Gemma 3 · Gemma 4	logit soft-capping; 前后都放 norm; Gemma 4 的 p-RoPE、每层独立 embedding、滑窗交替
Methods of improving LLM training stability (Rybakov et al., 2024)	NVIDIA 的稳定性干预对比
Fast Transformer Decoding (Shazeer, 2019)	MQA; 算术强度的推导
GQA (Ainslie et al., 2023)	分组共享 K、V; 效果一耗时图
DeepSeek-V2	multi-head latent attention, 另一种分解, 下一讲

名称	在本讲里的作用
Sparse Transformer (Child et al., 2019)	OpenAI 早年的 attention 模式工作 (应为)
Command A (Cohere, 2025)	3 层滑窗配 1 层全注意力；全局层 NoPE；并行 block
Llama 4 · Gemma 4 · OLMo 3	滑窗与全注意力交替的跟进者
Qwen 3.5 · Gated DeltaNet	便宜层换成状态空间模型的混合结构
Marin	Percy 用它训的 8B 模型出现在总表里

术语对照

English	中文
residual stream	残差流：贯穿整个网络、每层往上加一个增量的那条 x
pre-norm / post-norm	前置 / 后置归一化；本讲的关键区分是 norm 在不在残差流里
LayerNorm / RMSNorm	层归一化 / 均方根归一化（不减均值、无 β ）
bias term	偏置项
warm-up	学习率预热
gradient attenuation / gradient spike	梯度衰减 / 梯度尖峰
arithmetic intensity	算术强度：每搬一个字节做多少次浮点运算
tensor contraction	张量收缩：这里指矩阵乘
gated linear unit (GLU) / gating	门控线性单元 / 门控：用另一条线性支路逐元素调制主支路
ReGLU / GeGLU / SwiGLU	用 ReLU / GELU / Swish 作激活的 GLU
Swish (SiLU)	x 乘 $\text{sigmoid}(x)$
squared ReLU	ReLU 的平方
parallel layers / serial layers	并行块 / 串行块
positional embedding: sinusoidal / absolute / relative	正弦 / 绝对 / 相对位置编码
RoPE (rotary position embedding)	旋转位置编码
p-RoPE	只旋转部分坐标对的 RoPE 变体
NoPE	不加任何位置编码
feed-forward ratio	d_{ff} 与 d_{model} 之比
head dimension	每个注意力头的维度 d_{head}
aspect ratio	深宽比： d_{model} 除以层数

English	中文
vocabulary size	词表大小
bits per byte (BPB)	每字节比特数：按字节归一化的 loss，可跨 tokenizer 比较
basin	盆地：超参数在一段范围内 loss 几乎平坦
ablation	消融实验：一次只改一处的对照
weight decay	权重衰减：把权重往零收缩
dropout	随机失活
learning rate decay	学习率衰减
single-pass / single epoch	语料只过一遍
loss spike	loss 尖峰
partition function Z / log-normalizer	配分函数 / 对数归一化项
z-loss	对 $(\log Z)^2$ 的惩罚，把 Z 拉向 1
QK-norm	对 query 和 key 各做一次归一化再相乘
logit soft-capping	用 tanh 把 logits 软性封顶
prefill / decoding	预填充（处理 prompt，可并行） / 逐 token 解码
KV cache	缓存过去所有位置的 key 和 value
MHA / MQA / GQA	多头 / 多 query（共享一份 K、V） / 分组 query 注意力
multi-head latent attention (MLA)	多头潜在注意力：DeepSeek-V2 的低秩 KV 分解
sliding window attention	滑窗注意力：只看固定窗口内的位置
banded attention	带状注意力：滑窗在矩阵上的形状
hybrid model	混合模型：全局层与便宜层交替
state space model (SSM) / Gated DeltaNet	状态空间模型 / 其一种，第 4 讲
pipeline parallel / tensor parallel	流水线并行（沿层切） / 张量并行（沿宽度切），第 7、8 讲
mixture of experts (MoE)	混合专家，下一讲

字幕勘误

"Salazar and UN" → Salazar and Nguyen；"Get All in 2020"、"Noam et al. in 2020" → Narang et al.（2021）；"Marine" → Marin；"Almost 3"、"Omo 3" → OLMo 3；"Almo" → OLMo；"NeMo Tron" → Nemotron；"E to fix" → Idefics（推断）；"ETA and others" → Tay et

al. (推断); "Eagle and RoPE" → p-RoPE and RoPE (推断); "Lambda" → LaMDA; "MQ A" → MQA; "rope" → RoPE; "nope" → NoPE; "the Virgin 1 T5" → the version 1 T5; "Falcon, which use a gated linear unit" → Falcon 用的不是 GLU (应为 GELU, 推断); "Jacob Devlin's paper 2024" → 2014 (讲者随即自我更正); "past keys and queries ... KV" → keys and values (讲者口误); "one over K, this is the number of heads" → K 是每个头的维度 (讲者随即更正为 head dims)。

带走的问题

1. LayerNorm 只占 0.17% 的 FLOPs 却能占 25% 的运行时间——这个比例在大 batch 训练和端侧小模型逐 token 推理下各会怎么变? 第 6 讲的 kernel 融合把这部分搬运省掉之后, RMSNorm 相对 LayerNorm 的优势还剩什么?
2. 2/3 规则保持的是参数量; FFN 的 FLOPs 和激活内存跟着一起守恒吗? Llama 2 把 GQA 省下的 attention 预算挪给 MLP (乘 1.33), 这种"预算转移"的逻辑推到 MoE 上会变成什么?
3. RoPE 只旋转 q 和 k、不动 v——如果也旋转 v, 输出会带上什么? NoPE 的全局层没有任何位置编码, 它靠什么区分顺序 (因果 mask 会泄露位置吗)? 这 and 第 12 节"局部层提供位置信息"的说法怎么对上?
4. 如果 weight decay 是优化手段, 那 warm-up、学习率衰减、weight decay 三者应当怎样一起调? "训练中途和学习率一起改 weight decay"的启发式, 在不衰减学习率的调度下还成立吗?
5. 解码算术强度里的 n/d 项, GQA 除以 g、滑窗把 n 封顶、MLA 压缩每份 K、V 的宽度、KV cache 量化压缩每个数的字节——四者各改的是式子里的哪个量? 对一个 1B 参数的端侧模型, 哪一项最先卡住?

CS336 第 4 讲 | 注意力的替代方案 (Attention Alternatives)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 4 讲 视频: https://www.youtube.com/watch?v=cKSwj_qZ8Jg (1:26:20; 英文字幕为自动生成, 模型名和论文名错得多, 见文末勘误) 讲者: Tatsunori Hashimoto (全程; 开头说"上一讲讲的是基础 Transformer"、结尾说"推理部分 Percy 之后讲", 可以确认) 课程主页: <https://stanford-cs336.github.io/> · 本讲围绕的论文: Mamba 2 · Gated DeltaNet · DeepSeek-V3.2 (DSA) · Switch Transformer · DeepSeekMoE · OLMoE · DeepSeek-V3 标题只写了 attention, 实际前 34 分钟讲 attention 的替代方案, 后 52 分钟讲 MoE——讲者把两者看成同一件事: 一个改 attention 块, 一个改 MLP 块。

一句话: 上下文一长, attention 的 N^2 就压过 FFN 的 N , 本讲给出两条压成本的路和一套"稀疏化 MLP"的做法——把 softmax 去掉, attention 就靠矩阵乘法的结合律变成 $O(N \cdot d_k \cdot d_v)$, 还能改写成状态大小固定的 RNN (训练用并行形式、推理用递推形式), 再加只依赖输入的门就得到 Mamba 2 和 Gated DeltaNet, 但没有人在大规模上证明过纯线性方案, 落地的全是 7:1、3:1 的混合; 另一条路是 DeepSeek V3.2 的 DSA, 用便宜的 indexer 先选 top-k 个 token 再做全注意力, 仍是二次但常数小; 下半场的 MoE 把一个 FFN 换成"路由器 + 多个专家", 同样的 FLOPs 装进几倍的参数, 代价是 top-k 路由不可微、专家会坍缩、系统会堵——靠负载均衡 loss、路由器用 FP32 加 z-loss、dropless 内核这一套启发式才驯服, DeepSeek V1 到 V3 就是这套做法的标准答案。

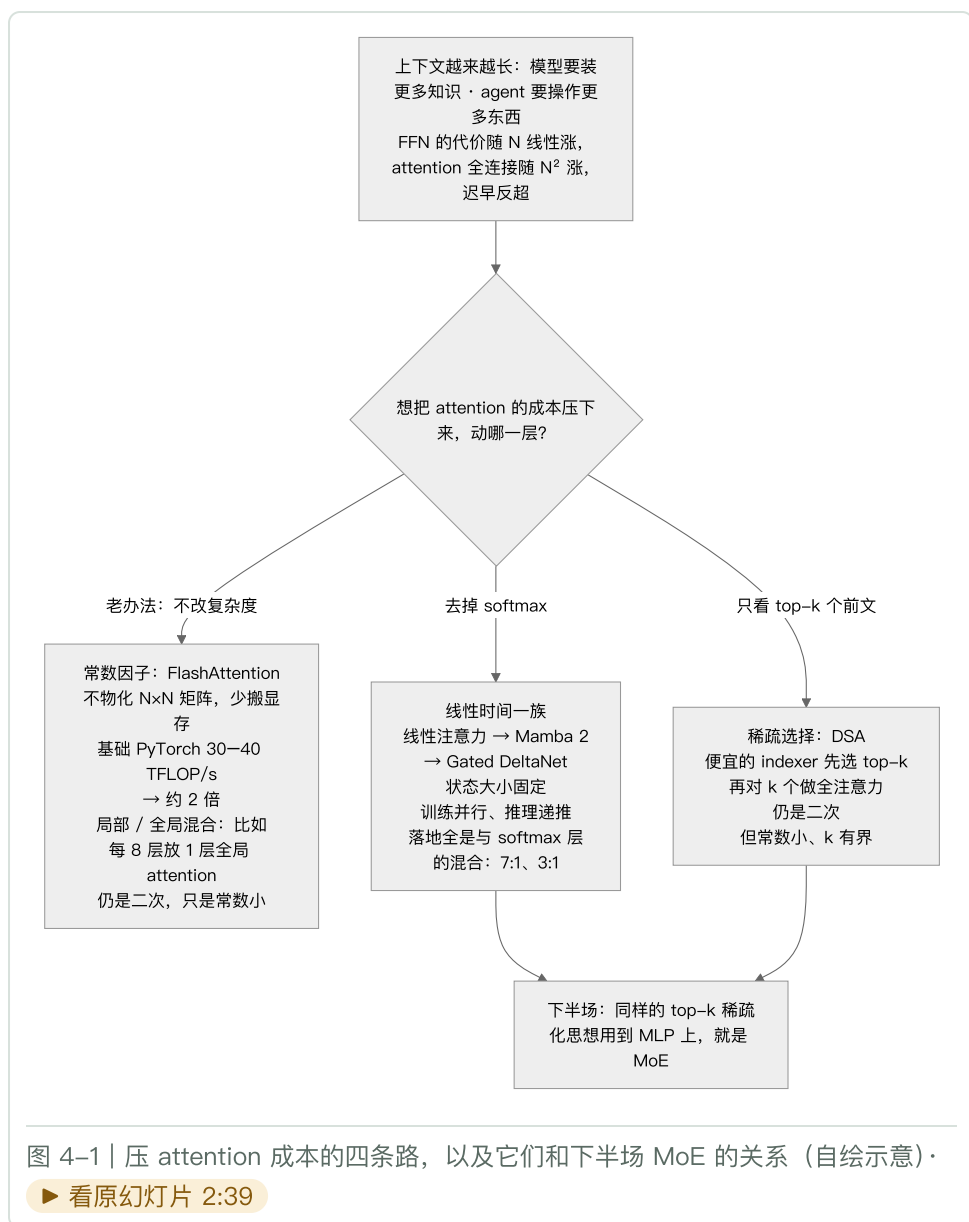
时间轴

时间	内容
0:05	开场: 本讲两件事——改 attention (对序列长度线性) 与改 MLP (MoE)
1:37	上下文长度竞赛; FFN 与 attention 的算力占比随序列长度反转; 工具箱: 局部/全局混合、FlashAttention 这类常数因子
5:41	线性注意力: 去掉 softmax, 用结合律把 N^2 换成 $N \cdot d_k \cdot d_v$
8:14	线性注意力的 RNN 形式: 固定大小的状态 S , 训练并行、推理递推; MiniMax M1 的 7:1 混合
11:17	Mamba 2: 只依赖输入的遗忘门 γ_t ; Nemotron 3 用它做轻量层

时间	内容
14:20	经验法则：门只看输入就保住对偶性；Gated DeltaNet 的写入门 β_t 与"先擦后写"；Qwen3-Next / Qwen 3.5 的 3:1
18:57	混合比例的对照研究：RNN 层占比越高、长上下文越差；问答：哪一步有损、输出里的 $D \cdot v_t$ 直通项
22:59	DeepSeek V3.2 的 DSA：轻量 indexer 选 top-k 再做全注意力；长上下文扩展阶段再接入；GLM 5 的消融
27:04	问答：indexer 仍是二次但常数小；短上下文预训练 $\rightarrow$ 长上下文扩展 $\rightarrow$ 后训练；FP4 attention；SSM 的代价是表达力
34:16	MoE 开讲：一个更高效的 MLP；同算力下参数越多越好（Switch、OLMoE、DeepSeek V2、Qwen1.5-MoE）
39:54	expert parallelism 是新的并行轴；谁在做 MoE；问答：通信代价、训练时也稀疏、按 token 路由
45:27	为什么 MoE 火得慢；设计三轴；token choice 还是 expert choice；四种路由函数
53:04	top-k 路由公式；DeepSeekMoE 的细粒度专家与 shared experts；OLMoE 的不同结论
58:40	训练 MoE：非可微与反事实两个难题；RL、noisy top-k、Switch 的乘性扰动都不是主流
1:03:19	专家坍塌；Switch 的负载均衡 loss 及其梯度含义；DeepSeek 的按设备均衡与 V3 的免辅助 loss；OLMoE 去掉均衡 loss 的消融
1:11:30	系统：expert parallelism、块对角稀疏矩阵乘、Nemotron 3 先降维再通信、token dropping 与 dropless
1:16:40	稳定性：路由 softmax 用 FP32 与 z-loss；MoE 微调容易过拟合；upcycling
1:22:15	DeepSeek V1 $\rightarrow$ V2 $\rightarrow$ V3 的演进；MLA 与 MTP；收尾

核心内容

1. 为什么要动 attention： N^2 迟早压过 FFN 的 N



- **需求侧：**讲者先放一张各家模型上下文窗口随时间的图（纵轴对数），头部厂商都在抢着把窗口做大——要往上下文里塞更多知识，agent 要操作更多东西。
- **成本侧：**同一页右边是 FFN 与 attention 的算力之比随序列长度 N 的变化。FFN 是逐 token 的运算，代价随 N 线性；attention 是全体位置两两交互，随 N^2 涨。序列短时大模型的成本大头在 FFN，序列一长 attention 就反超，而且越来越离谱。

小注：按第 2 讲的记账方式可以算出反超点。一层里 attention 的二次部分（ QK^T 和 PV 两次矩阵乘）约 $4 \cdot N^2 \cdot d$ FLOPs，FFN 约 $16 \cdot N \cdot d^2$ （两次 d 到 $4d$ 的矩阵乘；SwiGLU 三个矩阵配 $8/3 \cdot d$ 的中间维也是这个数），比值是 $N/(4d)$ ： $d = 4096$ 时 N 过 16k 二次项就开始占大头， $d = 8192$ 时是 32k——还没算 attention 里 QKV 和输出投影那 $8 \cdot N \cdot d^2$ 的线性部分。

- 已有的工具箱（图 4-1）

- 局部 / 全局混合：多数层只看邻近窗口，每隔几层放一层全局 attention（讲者的例子是每 8 层一层），成本就大体可控。
- 系统工程，常数因子：讲者强调这一点被严重低估。理论出身的人习惯只看大 O，但对上下文成本影响最大的单项工作其实是 FlashAttention——它没改 attention 的数学，只是把计算重排得对内存友好、少搬数据。课上的吞吐图：基础 PyTorch 在较短序列上约 30–40 TFLOP/s，FlashAttention 直接翻倍；有些长度下基础实现根本放不下 $N \times N$ 的打分矩阵，FlashAttention 不物化它，还能继续跑，只是慢。第 6 讲和作业 2 会亲手写它（CME295 第 4 讲讲过 tiling 的思路，不重复）。

- 但常数因子救不了 500 万、1000 万 token。要到那个量级，得有对 N 线性的方案。多年来试过很多、失败过很多，最近两年才出现一批在大规模、在生产里验证过的配方——讲者说这是他第一年在课上讲线性时间 attention，因为它终于被证明真的能用。

2. 线性注意力：去掉 softmax，用结合律把 N^2 换掉

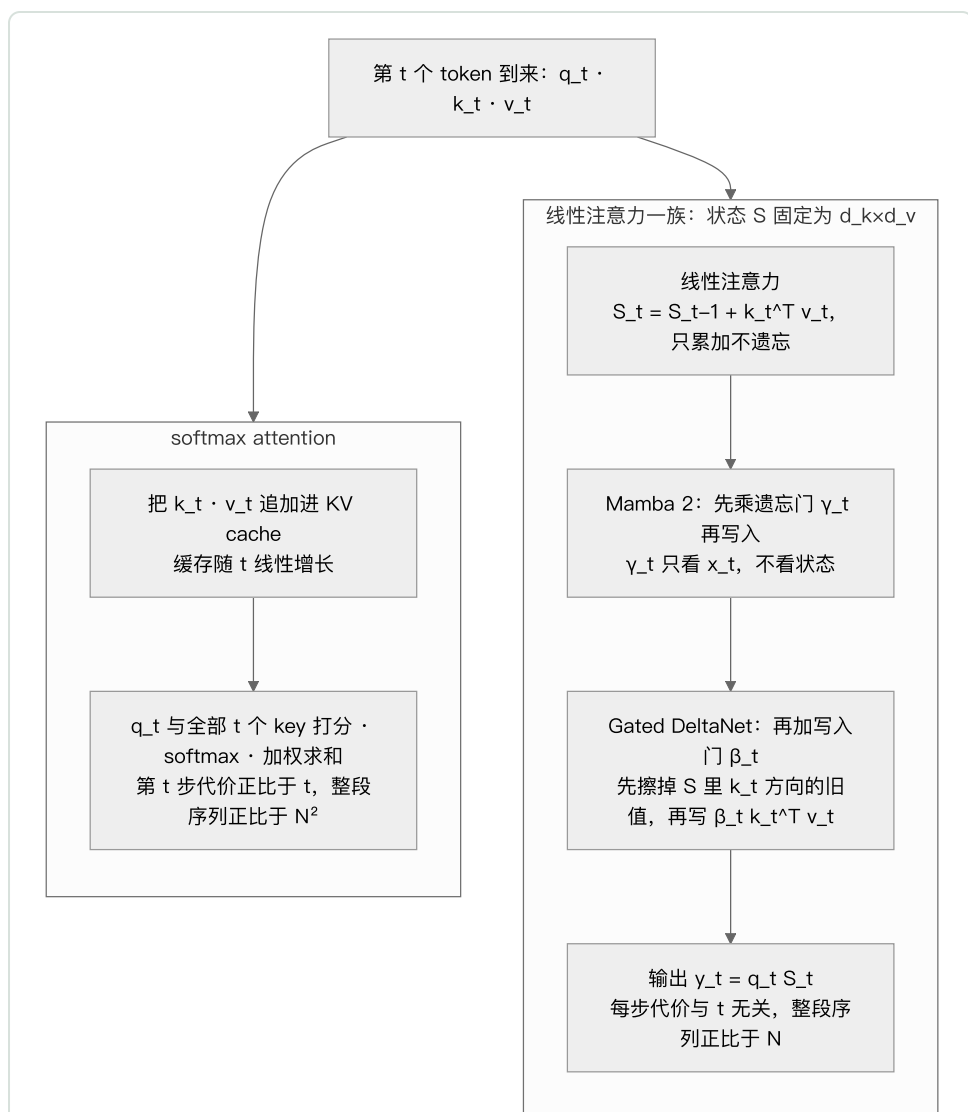


图 4-2 | 每来一个 token，softmax attention 与线性注意力一族各自做什么、带着多大的状态（自绘示意）· [▶ 看原幻灯片 8:14](#) · 出处: Katharopoulos et al., 2020（应出自）

- 只需要一个核心想法：矩阵乘法的结合律。讲者把 attention 写成 $\rho(QK^T)V$ ， ρ 是逐行 softmax；先假装 ρ 不存在——

$$\text{Attn}(Q, K, V) = \rho(QK^T)V \xrightarrow{\rho=\text{id}} (QK^T)V = Q(K^TV)$$

Q 、 K 、 V 各有 N 行（ N 是序列长度），列数分别是 d_k 、 d_k 、 d_v 。左边先算 QK^T ，得到 $N \times N$ 矩阵，代价 $N^2 \cdot d_k$ ；右边先算 K^TV ，得到 $d_k \times d_v$ 的小矩阵，代价 $N \cdot d_k \cdot d_v$ ，再乘 Q 也是 $N \cdot d_k \cdot d_v$ 。

- 换掉的是什么：二次项从 N^2 变成 $d_k \cdot d_v$ 。 N 是上下文长度，可以是百万级； d 是隐层维度，通常几千到几万，没人把隐层做到一百万。所以对 N 的依赖从二次降到一次。

- 它还是一个 RNN：把 $Q(K^T V)$ 按位置从左到右展开，就是一个递推

$$S_t = S_{t-1} + k_t^\top v_t, \quad y_t = q_t S_t, \quad S_t \in \mathbb{R}^{d_k \times d_v}$$

k_t 、 v_t 、 q_t 是第 t 个 token 的 key、value、query（行向量）， S_t 是前 t 个 token 的 $k^T v$ 累加起来的**状态**，大小固定为 $d_k \times d_v$ ，与 t 无关；输出只要拿 q_t 乘一下当前状态。

- 两种形式，各取所需

- 稠密形式（矩阵式）：一次大矩阵乘，可并行，训练用；
- 递推形式：状态大小固定，一路往前带，推理用。传统 RNN 推理好、训练难；线性注意力两头都占。
- 对照 softmax attention：推理时得把全部前文的 K 、 V 存着——这就是 **KV cache**（为免重算而缓存的历史 key / value，随上下文线性增长，第 10 讲的主角），每生成一个 token 都要对全部前文打分；线性注意力的“KV cache”只是一个 $d_k \times d_v$ 的 S （图 4-2）。

- 代价：讲者一句话点破——可惜它是线性的。去掉 softmax 这一步有损（后面问答再确认），表达力掉了一截。

```
# 同一个算子的两种算法，结果相同（省略归一化与因果掩码）
def dense_form(Q, K, V):                # 训练：一次大矩阵乘，可并行
    return Q @ (K.T @ V)                # (N,dk)@(dk,dv): 代价 N·dk·dv, N×N
    从不出现

def recurrent_form(Q, K, V):            # 推理：状态 S 固定为 dk×dv
    S = zeros(dk, dv); ys = []
    for q, k, v in zip(Q, K, V):        # 逐 token 扫过
        S = S + outer(k, v)             # 只累加，不遗忘
        ys.append(q @ S)
    return stack(ys)
```

这段展示的是对偶性本身：稠密形式里 $N \times N$ 矩阵从不出现，递推形式里状态大小与 t 无关。

小注：课上省掉了两件事。一是真正的线性注意力（Katharopoulos et al., 2020, "Transformers are RNNs"）不是简单丢掉 softmax，而是把 $\exp(q \cdot k)$ 换成特征映射 $\phi(q) \cdot \phi(k)$ ，再除以一个同样可递推的归一化项；二是带因果掩码时稠密形式不能直接写成 $Q(K^T V)$ ，训练实际用的是分块算法——块内当 attention 算、块间传状态，Mamba 2 论文的 SSD 算法就是这个。

- **落地例子：MiniMax M1。**一个规模不小、性能不错的中国开源模型，用 7:1 的混合——七层线性注意力配一层完整的 softmax attention，和 OpenAI o3、DeepSeek R1 等比也拿得出手；因为还有 softmax 层，对上下文长度的依赖不是纯线性，但温和得多。讲者的结论：至今没有人在大规模上验证过纯线性的 attention，接下来讲的全是混合。

小注：MiniMax-M1 论文摘要：456B 总参数、每 token 激活 45.9B，原生 100 万 token 上下文（DeepSeek R1 的 8 倍），RL 阶段用 512 张 H800 训了三周，租金 53.47 万美元。

3. 加门：Mamba 2 与 Gated DeltaNet，门只许看输入

- **Mamba 2：**想要比线性注意力更“像神经网络”的更新，看它的 RNN 形式就行。线性注意力的毛病是状态永远往前传，LSTM 时代就知道要学会什么时候传、什么时候忘，Mamba 2 就是给状态加一个遗忘门：

$$S_t = \gamma_t S_{t-1} + k_t^\top v_t, \quad y_t = q_t S_t + D v_t, \quad \gamma_t = \gamma(x_t)$$

γ_t 是由当前输入 x_t 算出的门，控制旧状态保留多少； $D \cdot v_t$ 是讲者为求完整才写上的项——当前 token 的 value 像残差一样直通到输出， D 控制直通量，与状态更新无关（问答里有人问了它）。

- **关键约束，也是往下推广的经验法则：**门只由输入决定、不含状态，状态相关的项只有 S ，就仍能展开成一次大矩阵乘（训练）或按步递推（推理），对偶性保住；照这个法则可以继续往递推里加东西。Mamba 系列（Albert Gu、Tri Dao 等人的 state space model 一族，已出到 Mamba 3）原本从状态空间理论推出，但实际做的事就是线性注意力加一个门。

小注：Mamba 2 论文（Dao & Gu, 2024）把这种“递推形式与矩阵形式等价”的关系起名 state space duality (SSD)，标题就叫 Transformers are SSMs。

- **落地：**Nemotron 3（第 3 讲提过）用 Mamba 2 做轻量层，隔一段放一层 softmax attention，靠交替方式平衡推理成本与表达力；和 Qwen 3 thinking、GPT-OSS 比性能不差，长上下文下吞吐更好——讲者的定位是“小号前沿模型”。
- **Gated DeltaNet：**讲者认为是目前用得最广的 SSM，Qwen 3.5 做了大规模验证。相对 Mamba 2 加了两样：

$$S_t = \gamma_t (I - \beta_t k_t^\top k_t) S_{t-1} + \beta_t k_t^\top v_t$$

β_t 是第二个门，由输入算出， $\beta_t = 0$ 表示"这个 token 什么都不写进状态"（LSTM 的输入门）；括号里的 $I - \beta_t k_t^\top k_t$ 是 DeltaNet 的"delta"部分——写入之前先把状态里沿 k_t 方向的旧内容投影掉，等于"先擦掉这个 key 下存的旧值，再写新值"，而不是简单按 $1 - \beta_t$ 衰减。讲者提醒 k_t 没有单位化，"投影"只是直觉。

小注：把括号展开就看出名字的来历： $S_t = \gamma_t \cdot [S_{t-1}] + \beta_t \cdot k_t^\top \cdot (v_t - k_t \cdot S_{t-1})$ —— $k_t \cdot S_{t-1}$ 是状态对这个 key 当前"记得"的 value，更新量正比于新旧 value 之差（delta rule）。

- 这个投影式更新被独立发明过多次：解某类 meta-learning 的最小二乘问题会得到它，fast weight programming、test-time training 从完全不同的原则出发也落到同一个解。

```
# 三种递推只差在"写入前对 S 做什么"； $\gamma$ 、 $\beta$  都只由  $x_t$  算出，不看 S
def step(S, q, k, v, gamma=1.0, beta=1.0, delta=False):
    if delta:
        # Gated DeltaNet: 先擦掉 k 方向
        # 向上的旧值
        S = S - beta * outer(k, k @ S) #  $(I - \beta k k^\top) S$ 
        S = gamma * S + beta * outer(k, v) # Mamba 2:  $\gamma < 1$  表示遗忘；线性
        # 注意力:  $\gamma = \beta = 1$ 
    return S, q @ S
```

一步递推就够看出三者的差别：线性注意力只累加，Mamba 2 先衰减，Gated DeltaNet 先擦后写。

- 落地**：Qwen3-Next 和之后的 Qwen 3.5（讲者称是今天最好的开源模型之一）用 3:1 的 Gated DeltaNet 与 attention 混合。课上的三联图：解码吞吐随上下文变长比 Qwen 3 高得多；和闭源与上一代模型比，混合架构几乎不损性能。
- 混合比例的对照研究**：讲者说这类受控研究很少，能找到的一篇（ByteDance Seed 与 UC Santa Cruz）比较了 Mamba 2、Gated DeltaNet 等架构在不同混合比例下的表现：虚线是全 attention 的水平，横轴向右是 RNN 层越来越多。结果有点乱，要点清楚——最好的几个架构（Gated DeltaNet 及变体）低比例时几乎没有损失，过了某个点长上下文性能明显下滑，纯 RNN 时全都明显变差。单 key 检索各家都专门优化过，看 QA 更诚实：RNN 层占比越高，性能稳定往下掉。

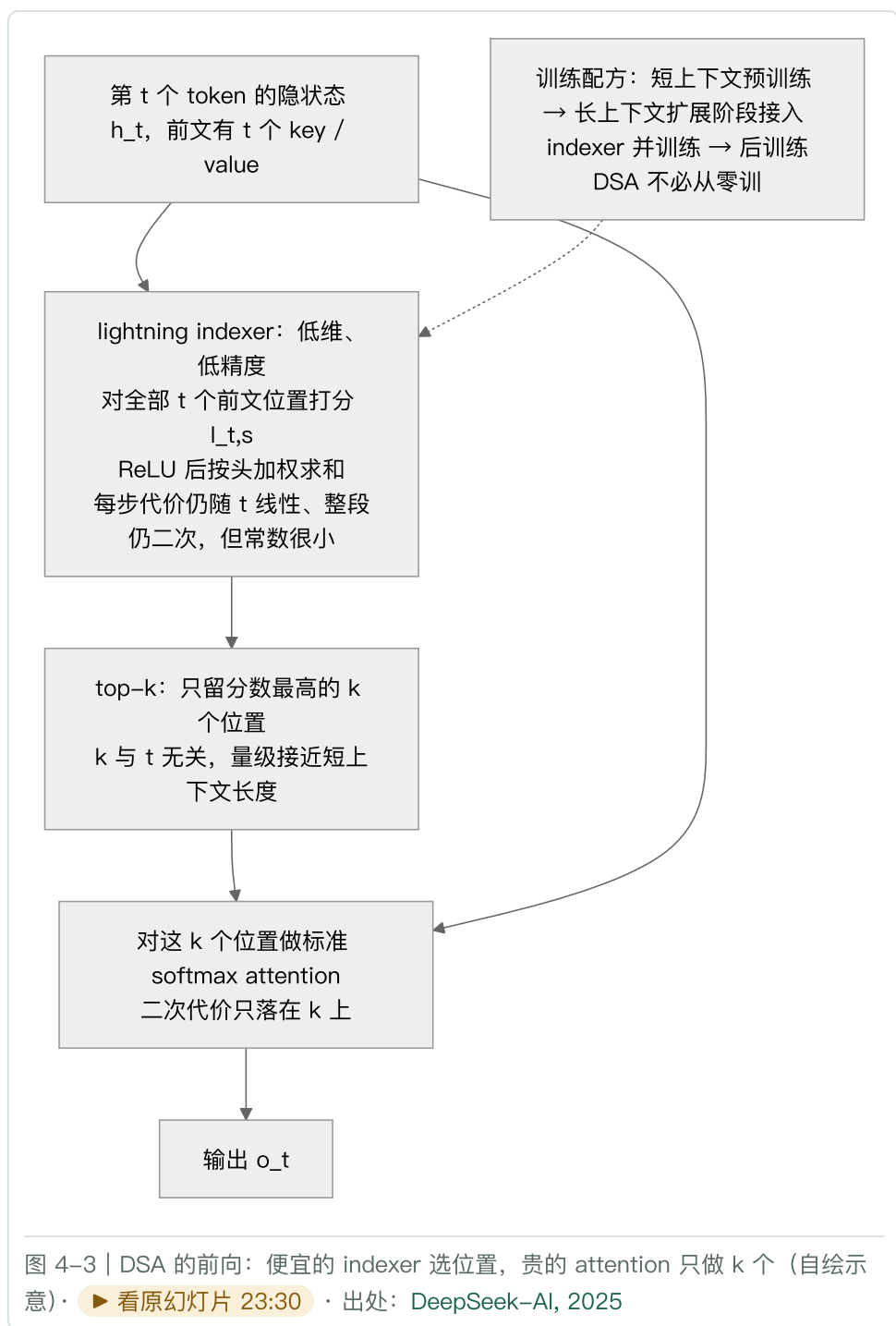
小注：应为 Wang et al., 2025 《A Systematic Analysis of Hybrid Linear Attention》（作者含 ByteDance Seed 与 UCSC）：72 个模型、六种线性注意力乘五种混合比例；单独强的线性模型不一定适合做混合，召回在 3:1 以下随全 attention 层数明显提升，推荐 Gated DeltaNet 或 HGRN-2 配 3:1 到 6:1。

- **问答**

- 并行形式和递推形式不是等价吗，怎么会掉性能？讲者拆成两步：softmax attention 到线性形式（丢掉 ρ ）这一步有损；线性形式到递推形式是精确等价。
- 这些方法虽然多，但绝大多数最后都收敛到了非常像 LSTM 的东西：线性注意力式的递推加几个门。
- 去掉 softmax 会不会有训练稳定性问题？讲者没见过文献说有；softmax 本身反倒常是不稳定的来源，去掉应该更稳。

- **SSM 到底放弃了什么（问答）**：表达力。softmax attention 的全连接极其强大，也容易训。过去 attention 相对 LSTM 还有硬件效率的优势——能并行训练——现在被对偶性抹平了；剩下的取舍是有限的状态必须装下整段上下文：状态和上下文一样大当然行，但那就回到大代价；想要小状态，就得压缩大上下文的信息，目前没有免费午餐。

4. 另一条路：DSA——先选 top-k 个 token，再做全注意力



- **思路:** DeepSeek V3.2 的 DSA (DeepSeek Sparse Attention) 不对全部 token 做 attention, 先用一个轻量的 indexer 从长上下文里挑出远小于全长的一个子集, 再只对这个子集做完整的 attention。前向的机制很简单:

$$I_{t,s} = \sum_{j=1}^{H_I} w_{t,j} \text{ReLU}(q_{t,j}^I \cdot k_s^I), \quad \mathcal{S}_t = \text{TopK}_s(I_{t,s}), \quad o_t = \text{Attn}(q_t, \{k_s, v_s\}_{s \in \mathcal{S}_t})$$

$l_{t,s}$ 是位置 t 对前文位置 s 的索引分数: indexer 有自己的一组小 query q^I 和 key k^I (H_I 个头), 内积过 ReLU 后按头加权 w 求和; 对

每个 t 取分数最高的 k 个位置组成集合 S_t ，真正的 attention 只在这 k 个位置的 k_s 、 v_s 上算。

```
# DSA 前向：先用便宜的 indexer 给每个前文位置打分，只对 top-k 做真正的
attention
def dsa(h, cache, k=2048):
    qI, wI = index_query(h)                                # 低维、低精度的索引
    引用 q 与各头权重
    scores = (wI[:, None] * relu(qI @ cache.kI.T)).sum(0)   # 看全部前
    文：0(t)，但常数小
    idx = topk(scores, k)                                    # 只留 k 个位置，k
    与 t 无关
    return attention(h, cache.K[idx], cache.V[idx]) # 二次代价只在 k 上
```

要看的是两级成本：打分这一步仍要扫全部前文，只是每个位置便宜得多；昂贵的 attention 被限制在 k 个位置上。

- **不用从头训**：带着 indexer 从零预训练又烦又复杂；DSA 和 GLM 都表明可以先训普通 Transformer，到长上下文扩展阶段再接入 indexer、训模型适应它——这一步本来就要做，顺手把省成本的东西装上，算力开销不大。讲者说这个不可微的 top-k 看着吓人，居然能事后接上；下半场会看到它没那么可怕。
- **效果**：V3.2 与当时的前沿模型（讲者举的是 Claude 4.5 Sonnet、Gemini 3）打平；prefill（一次性处理输入提示）和 decoding（逐 token 生成）的成本随上下文长度的曲线都远好于没有稀疏 attention 的上一代 DeepSeek。GLM 5（讲者认为是目前最好的开源模型之一）独立采用了 DSA，论文里有干净的消融——带 warm-up 的 DSA、直接 DSA、不用 DSA：完整 DSA 训练相对全 attention 损失很小，连 RNN 类架构最怕的长上下文检索也是。

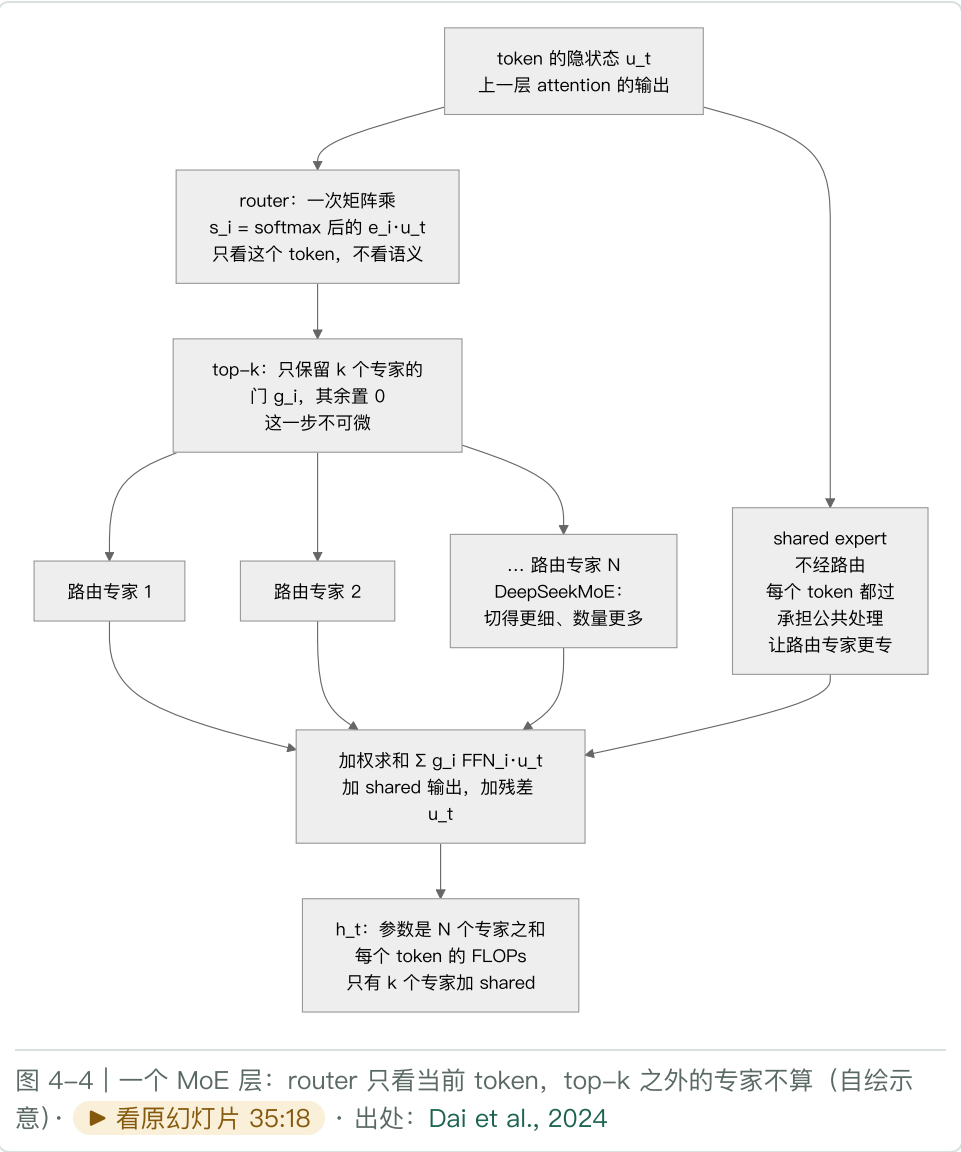
小注：据 DeepSeek-V3.2-Exp 技术报告（2025 年 9 月）： $k = 2048$ ；indexer 用 FP8 算；接入分两步——先冻结主模型、让 indexer 学着模仿全 attention 的分布（dense warm-up），再切到稀疏训练。V3.2 正式版（2025 年 12 月）沿用了 DSA。

- **它不是线性时间**：indexer 仍要算 t 对全部前文的内积，整段序列还是二次。但 indexer 可以做得很便宜——降到低维、权重矩阵做小、用更低精度；第二级虽是二次，却是 k 上的二次， k 有界、不随输入长度涨，取值接近短上下文长度。讲者的总结：别太纠结二次还是线性，常数因子经常才是关键。

• 问答

- FP4 attention 可行吗？indexer 的设计部分就是冲着低精度去的：softmax 的上溢下溢让低精度很难受，"低精度选位置、全精度算 attention"是合理分工；全 attention 直接做到 FP4 或更低，讲者不知道有好办法。
- 未来的 attention 长什么样？讲者不押注单一架构，估计是把所有有效的招都堆进去；更上面还有一层——用后训练让模型自己管理上下文（compaction、retrieval），架构层和这一层的整合会是接下来的工作。做 agent 的人应该对这句话有感觉。

5. MoE 是什么、为什么大家都在用



- 定义：概念上 MoE 不改变游戏规则——它就是一个更高效的 MLP。把 Transformer 块里的 FFN 换成若干个 FFN（比如 4 个，每个和原来一样大），再有一个机制告诉你每个输入该走哪一个：FFN 的参数变

成 4 倍，但每次前向或反向只付一个 FFN 的代价。早期论文的动机就是这么"参数中心"的：相信参数多是好事，又不想为参数付算力。

CME295 第 3 讲对 MoE 只是概览，这里往下挖。

- **为什么每个人都在用：**Hugging Face 上超过某个规模的模型如今清一色是 MoE，因为经验上，总算力不变、只增加稀疏参数，模型就是会变好。
 - Switch Transformer (Fedus et al., 2022)：激活参数固定，专家数越多，语言模型的测试 loss 越低；横轴换成训练算力，同样算力下专家越多越好。训练和推理都是白赚。
 - OLMoE (AI2 的开源 MoE 研究)：同样的图，训练 loss、验证 loss、下游指标都是 MoE 领先，讲者的说法是训练快大约 2 倍。
 - DeepSeek V2 和更早的 DeepSeek MoE 刚出来时，激活参数远少于别家的稠密模型，MMLU 不输甚至更好；Qwen1.5-MoE 只激活 2.7B，赢了当时不少 7B 模型。中国团队 (Qwen、DeepSeek、MiniCPM) 是最早训练并推广 MoE 的一批，西方的大 MoE 是 Llama 4 和 GPT-OSS，之后西方的开源发布基本停了。
- **第三个理由：多一个并行轴。**LLM 太大，单卡既训不了也推不了，要有多种切法 (第 7、8 讲)。每个专家天然是一块，可以放到不同设备上，把激活路由过去——这叫 expert parallelism，多了一个可优化的维度。
- **问答**
 - 专家分在不同设备上不会有通信瓶颈吗？会。你用通信换更多的总算力和更少的显存，是否划算取决于拓扑。后面会展示一个减通信的技巧。
 - 训练时也稀疏吗？是，这正是 MoE 难的根源：只有 k 个专家被激活，其余专家发生了什么你看不到，却得学会路由——带 bandit 味道的问题，但业界不用 RL 也不用 bandit，用启发式加深度学习的"魔法"。
 - 路由的粒度？token 级，每个 token 选自己的专家。router 极其朴素，一次矩阵乘，不会判断"这是医学问题"，更像"这个 token 看着像日语，送去 7 号专家"。
 - 并行有上限吗？有，切得越多通信越炸；作业会让你在给定网络拓扑下决定怎么切。
- **为什么火得慢：**Google 2022 年就在猛推，2024 年以后才真正流行；做研究的人至今也多用稠密模型。原因是麻烦：基础设施复杂，

专家难以高效并行，参数太多单卡放不下，训练还会炸（后面会看到怎么炸）。也有人把 attention 块做成 MoE，少见且不好驯，大模型都只 MoE 化 FFN。

6. 设计轴一、二：路由函数与专家粒度

- **三个设计轴**：路由函数、专家大小（预算固定时，多而小还是少而大）、训练。前提都一样：只激活一部分专家，所以总要想 top-k。
- **谁选谁**：token 选专家 (token choice)，或专家选 token (expert choice，每个专家挑自己最喜欢的 token)，或一个全局路由器统筹分配。几乎所有 MoE 都是 token choice top-k。OLMoE 的对照：token choice 验证 loss 更低、下游更高；expert choice 也能训，只是更难调（讲者记得某个未发布的 Llama 4 用过 expert choice，但不算背书）。
- **路由函数的四种做法**
 - *内积路由*：每个专家一个向量，和输入做内积、取最近的——Switch Transformer、GShard、GLaM、Grok、Mistral、DBRX、DeepSeek 都是，k 各家不同。
 - *哈希路由*：不学，直接把输入哈希到不同 FFN。讲者一直觉得神奇——这居然也有收益，只是不如 top-k；论文里常做 baseline，生产里不用。
 - *RL 学路由*：把路由器当策略，用强化学习学（Bengio 等人 2013 年的工作）。从学习理论看这是最自然的建模，但 RL 的开销和随机性太大，而启发式配方已经把简单 top-k 调通了，没理由复杂化。
 - *线性分配*：先算所有 token 与专家两两的匹配分，再解线性分配问题求全局最优——讲者作为“喜欢讲道理的人”很喜欢它，但太贵，没在大规模上出现过。
- **top-k 路由的公式**——认出这个模式很重要，DSA 里刚见过，H-Net 里也会见到：

$$s_{t,i} = \text{softmax}_i(e_i^\top u_t), \quad g_{t,i} = \begin{cases} s_{t,i} & i \in \text{TopK}(s_t, k) \\ 0 & \text{otherwise} \end{cases}, \quad h_t = u_t + \sum_{i=1}^N g_{t,i} \text{FFN}_i(u_t)$$

u_t 是这个 token 进入 MoE 层的向量，e_i 是第 i 个专家的路由向量，s_{t,i} 是 softmax 后的分数；只有分数前 k 名的专家门 g 非零，输出是残差加上这 k 个专家输出的加权和。门的学习只靠 e_i 与 u_t 的内积，非常轻。

- **DeepSeekMoE 的两个改动**（图 4-4）

- *shared experts*: 有些处理是所有 token 都需要的，与其让每个路由专家各自重复学，不如设几个"常开"专家绕过 router、每个 token 都过，让其余专家更专。
- 细粒度专家：把大专家切成更小的块，数量更多。
- DeepSeek 的消融：0 个 shared 加 16 个大路由专家（GShard 的老设计）对比细粒度专家加 1 个 shared，两项改动都有明显收益，shared expert 在 TriviaQA、Natural Questions 上尤其大。OLMoE 的受控消融则认为 shared expert 帮助不大、细粒度多专家有用——两家在 shared 上意见不一。
- **谱系**：最早三个 MoE 都来自 Google；Mistral、DBRX、Grok 是西方早期的尝试；DeepSeek V1 提出细粒度加 shared 的设计后，大家都跟了——像稠密模型都抄 Llama 一样，MoE 都抄 DeepSeekMoE 和 DeepSeek V3，Qwen 3.5、GLM 里两样都在。
- **问答**：有 shared expert 时怎么并行？它没有并行收益，每个激活都得过；可以把 shared expert 复制到每张卡上，用显存换通信。

7. 设计轴三：训练——不可微的 top-k 怎么学

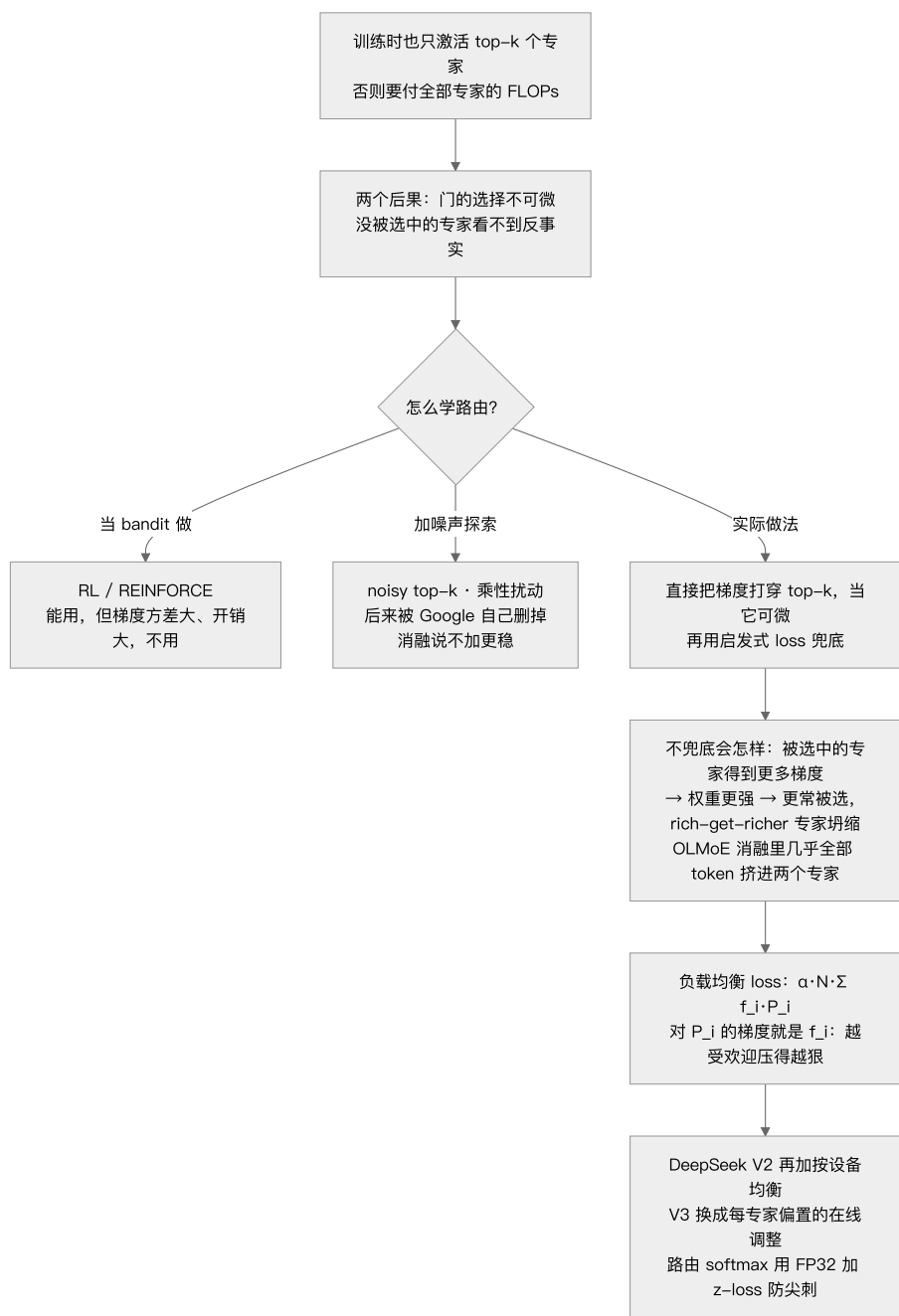


图 4-5 | 训练 MoE 的两个难题、三条路，以及实际用的那条上的两股相反的力（自绘示意）。 ▶ 看原幻灯片 1:03:19 · 出处：Fedus et al., 2022

- **难在哪：**训练时不能激活全部专家，否则付全额 FLOPs。可一旦稀疏，门的决策就不可微，没选中的专家发生了什么也看不到。讲者说自己最初觉得这东西不可能训好，结果一堆技巧叠起来居然稳定有效。
- **三条路：**RL 学门策略；随机扰动（探索—利用那一套）；用启发式平衡专家。实际用的是第三条，前两条讲者也过了一遍：
 - *RL*：Clark 等人的论文里 REINFORCE 学路由（图上绿线）能用，但梯度方差大、复杂，被同一篇论文提出的其他方法轻松超过。

小注：应为 Clark et al., 2022 《Unified Scaling Laws for Routed Language Models》，字幕里成了"Clark 2020"；论文比较了 REINFORCE 路由、哈希路由和基于 Sinkhorn 的 S-BASE。

- *noisy top-k* (Shazeer 等人最早的 MoE 论文)：训练时不做硬决策——路由内积上加一个尺度随输入变化的噪声，再取 top-k、做 softmax：两个专家分数接近时随机选一个，反传把有用的专家推高，等于打破平局、多探索；softmax 还让模型学会给 k 个专家排序。
- *乘性扰动* (Switch Transformer)：均匀的乘性噪声，为了让专家更鲁棒；后来 Google 自己的路由论文删掉了它，消融显示不加任何随机技巧，稳定性和最终质量反而更好。
- **启发式的核心：平衡。**不管探索、直接做梯度下降会怎样？top-k 里最强的专家拿到更多信号，权重更强，更常被选——rich-get-richer，少数专家包揽一切，其余饿死。专家坍塌是启发式训练要解决的核心问题，解法是在建模 loss 之外加一个平衡 loss，Switch Transformer 的版本：

$$\mathcal{L}_{\text{aux}} = \alpha N \sum_{i=1}^N f_i P_i, \quad \frac{\partial \mathcal{L}_{\text{aux}}}{\partial P_i} = \alpha N f_i$$

N 是专家数， f_i 是一个 batch 里被派给专家 i 的 token 比例（硬计数）， P_i 是 router 分给专家 i 的平均概率质量（软版本）， α 是系数。讲者说这式子不是从第一性原理来的，看懂它的办法是求导：对 P_i 的梯度恰好是 f_i ——一个专家拿到的 token 越多，它的概率被压得越狠，作用是在梯度空间里惩罚热门专家。

小注：Switch Transformer 论文默认 $\alpha = 0.01$ 。 f_i 里的硬计数不可导，梯度只经 P_i 流回 router，这正是"对 P_i 求导"就能看懂它的原因。

```
# top-k 路由 + Switch 式负载均衡 loss (一层、一个 batch 的 T 个 token)
def moe_layer(u, experts, We, k, alpha):
    p = softmax(u @ We.T) # (T, N): 每个 token
    对 N 个专家的概率
    g = keep_topk(p, k) # 未选中的门置 0: 稀疏,
    只算 k 个专家
    y = u + sum(g[:, i:i+1] * E(u[g[:, i] > 0])) for i, E in
    enumerate(experts))
    f = (g > 0).float().mean(0) # 各专家实际分到的
    token 比例 (硬计数)
```

```

P = p.mean(0) # 各专家拿到的平均概率质量 (软)
aux = alpha * len(experts) * (f * P).sum() # 越热门的专家, P 上被压得越狠
return y, aux

```

要点是两条相反的力在同一个前向里：top-k 让好专家越用越好，aux loss 把过热的专家往下压。

- **DeepSeek 的做法 (V1、V2)：**梯度直接打穿专家，不管不可微和探索；加与 Switch 相同的每专家平衡 loss；再加一个**按设备平衡**的 loss——一台机器放 4 个专家、另一台放另外 4 个，两台都要满负荷，公式一样，只是把专家换成设备，纯为利用率。V3 改成每个专家一个偏置项、在线调平，去掉了大部分辅助 loss，但防极端失衡仍留了一点——目前没有完全不要辅助 loss 的方案。

小注：V3 的偏置只加在 top-k 选择用的分数上，不进入门的加权；每步看各专家负载，过载减 γ 、欠载加 γ (Wang et al., 2024, Auxiliary-Loss-Free Load Balancing)。

- **不平衡会怎样：**OLMoE 的消融——去掉平衡 loss，训练 loss 明显变差；更直观的是专家占用图：没有平衡时几乎所有 token 都进了两个专家（黄色和粉色那两个），有平衡时所有专家都在被均匀使用。没有它，等于扔掉了大部分参数。
- **为什么这么简单就行：**有用的专家被正反馈强化，平衡 loss 把它拉回来，两股力抵消。同一个技巧 DSA 的 top-k 和 H-Net（去 tokenizer 的尝试）里都用了；"top-k 选择加辅助 loss 兜住不可微"会是未来架构设计的常见成分。
- **问答**
 - 专家真的是"专家"吗？不是。router 太简单，不会分出医学专家、法律专家；看得到的是标点、符号、非英语字符集各去某个专家，没有语义。
 - 有了每专家平衡，为什么还要每设备平衡？前者若完全强制、专家又均匀分布在设备上，后者自动成立；但每专家 loss 不能开太大，完全均匀对训练动力学有害；设备均衡对利用率太重要，值得单独多加一点 loss。

8. 系统与稳定性：并行、稀疏矩阵乘、掉 token、路由器的 softmax、微调与 upcycling

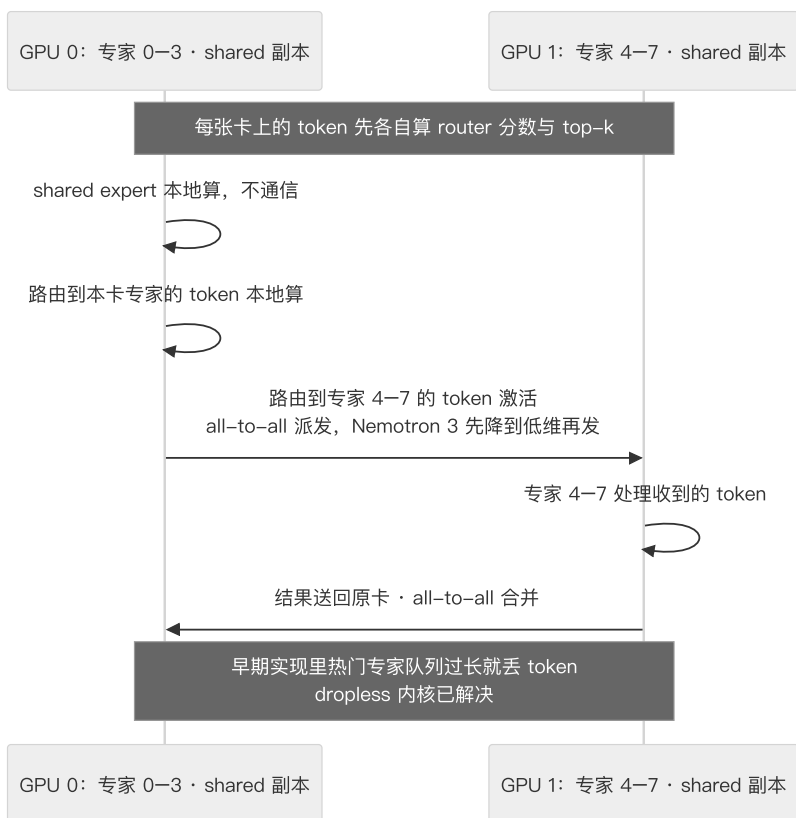


图 4-6 | expert parallelism 里谁把什么发给谁：跨卡的只有被路由走的激活（自绘示意）· [▶ 看原幻灯片 1:14:04](#)

- **并行的第三个轴**：数据并行到了 batch size 上限就没法再切；模型并行只能在模型天然的切点上切，切点用完也到头；expert parallelism 又给了一个轴（详见第 7、8 讲）。
- **块对角稀疏矩阵乘**：一张卡上放多个专家，朴素做法是多次小矩阵乘，但硬件喜欢能复用缓存的大矩阵乘；MoE 的计算模式天然对应块对角乃至更复杂的结构化稀疏，而这些是硬件原生支持的——讲者说这是 MoE 与硬件的协同设计。
- **减通信**（Nemotron 3 的新招）：expert parallelism 要把激活跨卡送到专家所在的设备，通信量大。shared expert 不用通信，可以保持大维度；路由专家的激活先把残差流降到低维再做集合通信（all-to-all：每张卡把各自的一份发给所有其他卡、同时收齐别人发来的），大幅省通信，又不承担整体缩小隐层维度的坏处（图 4-6）。
- **掉 token**（已解决的历史问题）：早期 MoE 基础设施里，热门专家的 token 队列越堆越长，长到一定程度就直接丢弃——返回零、假装没事。于是别的用户的请求撞上你在用的专家，会把你挤出队列，结果随负载随机变化。dropleless 架构（MegaBlocks 等常见开源框架）已经没有这个问题。

- 稳定性**：第 3 讲说过指数和除法是危险区，softmax 是稳定性事故的高发地——MoE 又在路由里加了一个 softmax。Google 早期就有一整篇论文研究 MoE 的稳定性（Barrett Zoph 等），常见解法是路由器单独用 FP32，再给路由的 softmax 加 z-loss（第 3 讲讲过）。OLMoE 的消融：不加路由 z-loss，训练 loss 曲线尖刺明显。

小注：Zoph 等人的稳定性论文应为 ST-MoE（2022）；路由器 FP32 加 router z-loss 都出自那里。

- 微调容易过拟合**：参数太多，直接微调专家会严重过拟合——同一张图上稠密模型的 train / val 很接近，稀疏模型差距极大（一个 GLUE 任务）。对策：只微调非 MoE 的 FFN，或只微调 attention（讲者最近常见）；或者"苦涩教训"版——用 140 万条样本，多到能把 MoE 整体重训，泛化差距就小了。
- Upcycling**：想要 MoE 又不想从零训？把训好的稠密模型整个复制，MLP 复制成若干份专家，router 随机初始化，接着训——输入被随机分到不同专家，专家自然分化。最早的论文显示 upcycling 比继续训同一个稠密模型好；MiniCPM 把 2.4B 升成 13.4B 的 MoE，几乎白赚；Qwen 用 1.8B 升成 Qwen1.5-MoE-A2.7B，是较早的大规模成功案例。但今年讲者一个 upcycling 的模型都没见到：主力模型一开始就训 MoE，没必要先训稠密再转换。

9. DeepSeek V1 → V2 → V3，外加 MLA 与 MTP

版本	MoE 设计	讲者的点评
DeepSeekMoE (V1)	细粒度专家 + shared experts；标准 top-k 路由；辅助 loss 平衡	现代 MoE 的原型，"柏拉图式的理想 MoE"
DeepSeek V2	同样的设计放大：2 个 shared experts、多得多的细粒度专家；加按设备路由平衡与通信平衡的辅助 loss	用辅助 loss 去优化系统——训好大模型不只是深度学习，还得尊重你的系统
DeepSeek V3	仍是 shared 加细粒度；改用免辅助 loss 的偏置调平；门的打分从 softmax 换成 sigmoid（讲者原话是 sigmoid 加 softmax）	大体相同；另外带了 MLA 和 MTP 两个推理侧的好东西

小注：V3 论文的写法：用 sigmoid 算 token 对各专家的亲和度，只在选中的 k 个专家之间归一化成门值。

- MLA (multi-head latent attention)**：不直接从隐状态 h_t 生成 Q、K、V，先生成一个低维的潜变量 c_t ，再从 c_t 生成 K 和 V：

$$c_t = W_c h_t \in \mathbb{R}^{d_c}, \quad k_t = W_k c_t, \quad v_t = W_v c_t, \quad d_c \ll n_h d_h$$

h_t 是 token 的隐状态, c_t 是维度 d_c 的潜变量 (latent), K 、 V 由它线性生成; $n_h \cdot d_h$ 是全部头的 K 或 V 的总维度。KV cache 只需存 c_t 而不是 K 、 V , 缓存按 d_c 与 $n_h \cdot d_h$ 之比缩小——这就是"latent"名字的来历。麻烦是它和 RoPE 冲突 (RoPE 要逐维旋转 K , 而 K 是缓存后才从 c_t 生成的), 解法是留几个不经潜变量的维度专门编码位置, 讲者没展开。CME295 第 2 讲讲过 GQA 靠多个 query 头共用 K 、 V 省缓存, MLA 是另一条路: 压缩。

小注: DeepSeek-V2 论文摘要: MLA 把 KV cache 缩小 93.3%, 最大生成吞吐提高 5.76 倍 (相对 DeepSeek 67B)。

- **MTP (multi-token prediction)**: 不只预测下一个 token, 一次预测多个未来 token。统计上的理由是也许能更好地预测更远的未来; 系统上的理由更实在——模型自带了一个投机解码器 (speculative decoding: 便宜的草稿先猜几个 token, 主模型一次验证), 能加速生成, Percy 在第 10 讲会展开。讲者说这个想法很酷但没怎么流行开。

小注: DeepSeek-V3 的 MTP 是串行地多预测一个 token (深度 1), 论文报告用它做投机解码时第二个 token 的接受率约 85% 到 90%。

- **收尾**: MoE 是利用稀疏性"拥有比你付费更多的参数"的聪明办法; 路由问题看着难, 但非常简单的做法在大规模上就能用; MoE 会长期存在, 值得搞懂。

10. 放在一起看: 每个方案对 N 的代价、要带着什么、放弃了什么

方案	对序列长度 N 的代价	推理时要带着什么	放弃了什么	课上的落地例子
softmax attention (含 FlashAttention)	二次; FlashAttention 只改常数	全部 K 、 V (KV cache 随 N 长)	无损, 但 N 一大付不起	所有主流模型
局部 / 全局混合	多数层线性、少数层二次	全局层的 KV cache	多数层看不到远处	每 8 层一层全局
线性注意力	线性	固定的 $d_k \times d_v$ 状态	softmax (这一步有损); 有限状态压不下长上下文	MiniMax M1 的 7:1
Mamba 2 / Gated DeltaNet	线性	同上, 多几个门	同上; 表达力仍不及全连接	Nemotron 3; Qwen3-Next 的 3:1
DSA	二次, 但 indexer 常数极	全部 K 、 V 仍要缓存	每个 token 只看 k 个前文; top- k 不可微	DeepSeek V3.2、GLM 5

方案	对序列长度 N 的 代价	推理时要带着什 么	放弃了什么	课上的落地例子
	小, attention 只 在 k 上	(indexer 要 看全部)		
MoE (改 MLP)	与 N 无关; 每 token 的 FLOPs 不变	全部专家的参 数都得驻留	显存、通信、路 由训练的复杂度	DeepSeek、 Qwen、GLM、 Llama 4、GPT- OSS

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
上下文窗口随时 间 + FFN/attention 算 力比	左图纵轴是对数；右图随 N 增大曲线反转， attention 反超 FFN	1:37	—
FlashAttention 吞吐对比	蓝色基础 PyTorch 约 30–40 TFLOP/s； FlashAttention 约 2 倍；长序列下基础版直 接缺数据（放不下）	4:10	Dao et al., 2022
MiniMax M1 性能 与算力曲线	7:1 混合；和 o3、R1 的对比；算力随上下文 的曲线远缓于纯 softmax	9:45	MiniMax– M1
Mamba 2 更新式 + Nemotron 3 吞 吐	公式里 y_t 只依赖 x_t ；右图长上下文下吞 吐优势	13:17	Dao & Gu, 2024
Gated DeltaNet 对照 Mamba 2	蓝框是 $I - \beta k^T k$ 的投影项； β 是写入门	15:20	Yang et al., 2024
Qwen3–Next 三 联图	右图解码吞吐随上下文拉开；中图混合架构 不损性能	18:26	—
混合比例对照研 究	虚线是全 attention；横轴右移是 RNN 层增 多；黄橙蓝三条低比例几乎不掉	19:28	Wang et al., 2025 (应为)
DSA 结构图与 V3.2 成本曲线	indexer $\rightarrow$ top-k $\rightarrow$ attention；25:32 处 prefill、decoding 成本随长度的曲线	23:30	DeepSeek– V3.2
Switch Transformer 两 张图	左：激活参数固定，专家数增多 loss 下降； 右：同训练算力下专家多更好	37:21	Fedus et al., 2022
OLMoE 对稠密的 训练曲线	同算力 MoE 领先，约 2 倍训练速度	38:23	OLMoE
DeepSeekMoE 消融表	0 shared + 16 大专家 (GShard) 对比细粒 度 + 1 shared；TriviaQA、NQ 的蓝黄差距	55:35	DeepSeek MoE
OLMoE 去掉平衡 loss	上：粉线有平衡、loss 正常；下：无平衡时 token 挤进两个专家	1:07:56	OLMoE

图	看什么	跳转	出处
OLMoE 路由 z-loss 消融	不加 z-loss 的 loss 曲线全是尖刺	1:17:41	OLMoE
微调过拟合	稠密 train / val 贴近；稀疏差距极大	1:18:11	ST-MoE (应为)

提到的工作

名称	在本讲里的作用
FlashAttention (Dao et al., 2022)	"常数因子也很重要"的证据：不改数学、只改内存访问，吞吐翻倍
Transformers are RNNs (Katharopoulos et al., 2020)	线性注意力及其 RNN 形式的出处（应出自，讲者没报论文名）
MiniMax-M1 (2025)	7:1 线性 / softmax 混合的大规模例子
Mamba 2 (Dao & Gu, 2024) · Mamba	线性注意力加只依赖输入的遗忘门；SSM 一族
Nemotron 3 (NVIDIA)	Mamba 2 与 softmax 层交替的混合模型；先降维再通信的 expert parallelism 技巧
Gated DeltaNet (Yang et al., 2024)	再加写入门与"先擦后写"的 delta 更新；讲者眼中最常用的 SSM
Qwen3-Next · Qwen 3.5	3:1 Gated DeltaNet 混合；解码吞吐随上下文拉开
A Systematic Analysis of Hybrid Linear Attention (Wang et al., 2025)	混合比例的受控研究（应为这篇）
Fast weight programmers · Test-time training	独立发明出同一种投影式更新的领域（推断为这两篇代表作）
DeepSeek-V3.2 (2025)	DSA: lightning indexer 选 top-k 再全注意力；长上下文扩展阶段接入
GLM 5	独立采用 DSA，带 warm-up 消融
Switch Transformer (Fedus et al., 2022)	专家数 vs loss 的两张图；负载均衡 loss 的原型；乘性扰动
GShard · GLaM	Google 早期 MoE；内积路由；"0 shared 加 16 大专家"是 GShard 的设计
Outrageously Large Neural Networks (Shazeer et al., 2017)	最早的 MoE 论文；noisy top-k 门
OLMoE (AI2, 2024)	西方最干净的受控 MoE 研究：token vs expert choice、shared expert、平衡 loss、z-loss 的消融
DeepSeekMoE (Dai et al., 2024)	细粒度专家 + shared experts；哈希 / switch / 稠密的消融

名称	在本讲里的作用
DeepSeek-V2 · DeepSeek-V3	V2：按设备与通信平衡；V3：免辅助 loss 平衡、sigmoid 门、MLA、MTP
Qwen1.5-MoE	2.7B 激活赢 7B 稠密；从 1.8B 模型 upcycling 而来
MiniCPM	2.4B 升成 13.4B MoE 的 upcycling 案例；讲者喜欢它的受控消融
Llama 4 · GPT-OSS	西方的大 MoE 开源模型
Mixtral · DBRX · Grok	西方早期 MoE
Hash Layers (Roller et al., 2021)	哈希路由（应为这篇）
Bengio et al., 2013	用 RL 学路由的早期工作（应为这篇）
BASE Layers (Lewis et al., 2021)	线性分配路由（应为这篇）
Expert Choice (Zhou et al., 2022)	专家选 token 的路由（应为这篇）
Unified Scaling Laws for Routed LMs (Clark et al., 2022)	REINFORCE 路由的对照（应为这篇）
ST-MoE (Zoph et al., 2022)	MoE 稳定性：路由器 FP32、router z-loss；只微调非 MoE 层（应为这篇）
Auxiliary-Loss-Free Load Balancing (Wang et al., 2024)	V3 的偏置调平（小注引用）
H-Net	去 tokenizer 的尝试，同样用 top-k 加辅助 loss
MegaBlocks	dropless MoE 框架
Sparse Upcycling (Komatsuzaki et al., 2022)	upcycling 的最早论文（应为这篇）

术语对照

English	中文
sequence length / context length	序列长度 / 上下文长度 (N)
quadratic / linear (in N)	对 N 二次 / 线性
constant factor	常数因子：不改复杂度、只改系数的优化
associativity	结合律： $(QK^T)V = Q(K^T V)$
linear attention	线性注意力：去掉 softmax 后的 attention
recurrent form / dense (parallel) form	递推形式 / 稠密（并行）形式
duality	对偶性：同一算子既能并行算、也能按步递推
state	状态：递推里一路带着的 $d_k \times d_v$ 矩阵
state space model (SSM)	状态空间模型：Mamba 一族
gate (forget / input)	门（遗忘门 γ / 写入门 β ）

English	中文
delta rule	增量规则：按新旧 value 之差更新
hybrid ratio	混合比例：线性层与 softmax 层之比，如 7:1、3:1
KV cache	键值缓存：推理时存下的历史 K、V
decoding throughput	解码吞吐：单位时间生成的 token 数
prefill / decoding	预填充（处理输入提示） / 解码（逐 token 生成）
sparse attention	稀疏注意力：每个 token 只看一部分前文
indexer	索引器：给前文位置打分、决定看谁的轻量模块
top-k selection	取前 k：不可微的硬选择
long-context extension	长上下文扩展：短上下文预训练后的续训阶段
expressive power	表达力
mixture of experts (MoE)	混合专家：多个 FFN 加一个路由器
expert / router	专家 / 路由器
active (activated) parameters	激活参数：每个 token 真正用到的参数量
sparse parameters	稀疏参数：总参数，其中大部分每次不用
token choice / expert choice	token 选专家 / 专家选 token
shared expert	共享专家：不经路由、常开
fine-grained experts	细粒度专家：切小、切多
hash routing	哈希路由
linear assignment	线性分配：全局最优的 token—专家匹配
bandit	老虎机问题：只观察到被选动作的反馈
REINFORCE	策略梯度的基本算法
load balancing	负载均衡
expert collapse / starvation	专家坍缩 / 饿死
rich-get-richer	强者愈强的正反馈
auxiliary loss	辅助 loss：加在建模 loss 之外的启发式项
aux-loss-free balancing	免辅助 loss 的平衡：用偏置在线调平
expert parallelism	专家并行：专家分放不同设备
all-to-all	全交换：每张卡向所有卡各发一份、同时收齐
block-diagonal / structured sparsity	块对角 / 结构化稀疏
token dropping / dropless	丢 token / 不丢 token
z-loss	惩罚 softmax 的 log 配分函数偏离 0
upcycling	升级改造：把稠密模型复制成 MoE 再训

English	中文
multi-head latent attention (MLA)	多头潜变量注意力：只缓存低维 c_t
multi-token prediction (MTP)	多 token 预测
speculative decoding	投机解码：草稿模型先猜、主模型验证
continued pre-training	继续预训练

字幕勘误

"row"（反复出现）→ ρ （幻灯片里代表 softmax 归一化的记号，应为）；"Open as 03" → OpenAI o3；"NeMo Tron / Neumitron" → Nemotron；"Qwen Next" → Qwen3-Next；"Elmo" → OLMoE；"AIQ folks" → AI2；"MOE" → MoE；"Fit Fedus" → Fedus；"Plan"（路由器例子里）→ GLaM（应为）；"Clark 2020" → Clark et al., 2022（应为）；"Deep Seek" → DeepSeek；"H-Nets" → H-Net；"lower-dimensional latency" → latent；"Qwen 1.5A 2.7B" → Qwen1.5-MoE-A2.7B；"DSA, not do sparse attention" → DeepSeek Sparse Attention；"lightweight indexer" 与 "lightning indexer" 混用，DeepSeek 的正式名是 lightning indexer；"coms" → comms；"Minimax" → MiniMax；"top K" → top-k。

带走的问题

- 用第 1 节小注的记账算一算： $d = 4096$ 、上下文 128k 时，一层 attention 的二次部分是 FFN 的几倍？换成 3:1 的 Gated DeltaNet 混合，每 token 的 FLOPs 和推理时要带的状态各降到多少？对 agent 的长会话，算力和 KV cache 显存哪个先要命？
- 线性注意力到递推形式是精确等价，softmax 到线性那一步才有损。既然如此，MiniMax M1、Qwen3-Next 为什么都留了 softmax 层，而且比例收敛到 7:1、3:1 而不是 0？对照第 3 节的对照研究，你会怎么设计一个实验来找自己任务的最优比例？
- DSA 的 indexer 仍是二次，讲者却说别纠结二次。给 $t = 1\text{M}$ 的请求估一下：indexer 的内积次数和 $k = 2048$ 上全 attention 的内积次数各是多少？哪一步会先撞上显存带宽而不是算力（第 5、6 讲）？
- 负载均衡 loss 对 P_i 的梯度是 f_i 。如果把 α 开得很大、专家被强制完全均匀，会失去什么？DeepSeek V3 的偏置调平只改选择、不改加权，为什么这一点很重要？

5. 讲者说未来一层是"用后训练让模型自己管理上下文" (compaction、retrieval)。对你在做的 agent, 哪些上下文该靠架构 (线性层、DSA) 便宜地带着, 哪些该靠模型主动压缩或检索? 两层的失败模式分别是什么?

CS336 第 5 讲 | GPU 与 TPU (GPUs, TPUs)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 5 讲, 系统部分的第一讲 (第 5—8 讲: GPU → kernel → 并行) 视频:
<https://www.youtube.com/watch?v=izZba4UA7iY> (1:18:39; 英文字幕为自动生成, 专名和数字错得不少, 见文末勘误) 讲者: Tatsunori Hashimoto (全程; 开场自嘲"不是搞系统的", 讲义大量取材于下面三份资源) 课程主页: <https://stanford-cs336.github.io/> · 收尾围绕 FlashAttention (Dao et al., 2022) · 讲者推荐的读物: Google 的 [How to Scale Your Model](#) (课上叫"TPU 书", 现在也有 GPU 章节, 附练习, 作业里有类似的题)、Horace He 的 [Making Deep Learning Go Brrrr](#)、GPU MODE 社区 (原名 CUDA MODE)

一句话: GPU 是拿几百个 SM 堆吞吐的机器, 而它的算力增速远快于内存带宽和互联带宽, 所以今天写快代码的核心不是"多算"而是"少搬"——衡量标准是运算强度 (每搬一个字节做多少 FLOP), 落到 roofline 的平顶上才算用满硬件; 讲者给了六个抓手 (避免分支、低精度、算子融合、重计算、合并访存、分块), 用它们解释了一张诡异的 matmul 吞吐图 (可整除性、burst 对齐、108 个 SM 的 wave quantization), 最后证明 FlashAttention 不过是"分块 matmul + online softmax + backward 重算"的组合; TPU 是同一套思路的趋同演化, 芯片里只差在"少而大"的 MXU, 真正的差别在网络。

时间轴

时间	内容
0:05	开场: 系统部分开始; 本讲三段 (硬件模型 → 六个技巧 → FlashAttention); "讲完你能解释这张 matmul 吞吐图"
2:07	推荐资源: Horace He 的博客、GPU MODE、Google 的 TPU / GPU 书; 算力是硬通货
4:41	Dennard scaling 在 2000 年代失效 → 横向并行; GPU 的 FLOP/s 逐代超指数增长, V100 引入 Tensor Core
7:15	CPU 重延迟、GPU 重吞吐; SM 是基本单元; 内存层级与延迟表; 为什么不全做成 SRAM (Groq)
13:22	编程模型: thread / block / warp 与 SIMT; 从寄存器到 host memory 的内存模型
17:00	TPU 两页: 趋同演化; Tensor Core 与 MXU 都是 systolic array; 132 个 SM 对 2 个 TensorCore; 命名混乱

时间	内容
22:39	GPU 为什么成功；2001 年用着色器算 matmul；V100 之后 matmul 成了唯一的特权算子
25:13	算力、内存带宽、互联三条曲线增速不同 → 一切优化都是内存优化；问答：prefill / decode 分离、Step-3
30:16	第二部分：谜题图与 roofline；六个技巧的共同目标是提高运算强度
32:47	技巧 1 控制分歧：if 的两支串行执行；ReLU 用乘 mask
34:49	技巧 2 低精度：每 FLOP 的字节减半；Tensor Core 内部先降精度再高精度累加；FP8 的 E4M3 / E5M2
38:52	MXFP8：每 32 个元素一个 E8M0 缩放因子；转置要重新量化 → 存两份；FP8 训练省 20—30%；MXFP4 只有 -6 到 6
44:31	问答：只量化 matmul 划算；缩放因子不训练；结构化稀疏；先训大再量化
47:35	技巧 3 算子融合（工厂与传送带，torch.compile / XLA）；技巧 4 重计算（8 次访存变 5 次）
52:42	技巧 5 合并访存：DRAM 按 burst 出货；warp 内线程落在同一 burst；行主序矩阵按列读的代价
57:51	技巧 6 分块：tile 搬进 shared memory 反复用，全局访存少 T 倍；tile 大小、对齐与 padding；max-autotune；Karpathy 的词表 padding
1:06:28	解谜：颜色是可整除性；1792 → 1793 的断崖 = 98 对 120 个 tile 装进 108 个 SM（wave quantization）；问答：SRAM 为何贵
1:12:03	第三部分 FlashAttention：分块 matmul + online softmax + backward 重算；总结

核心内容

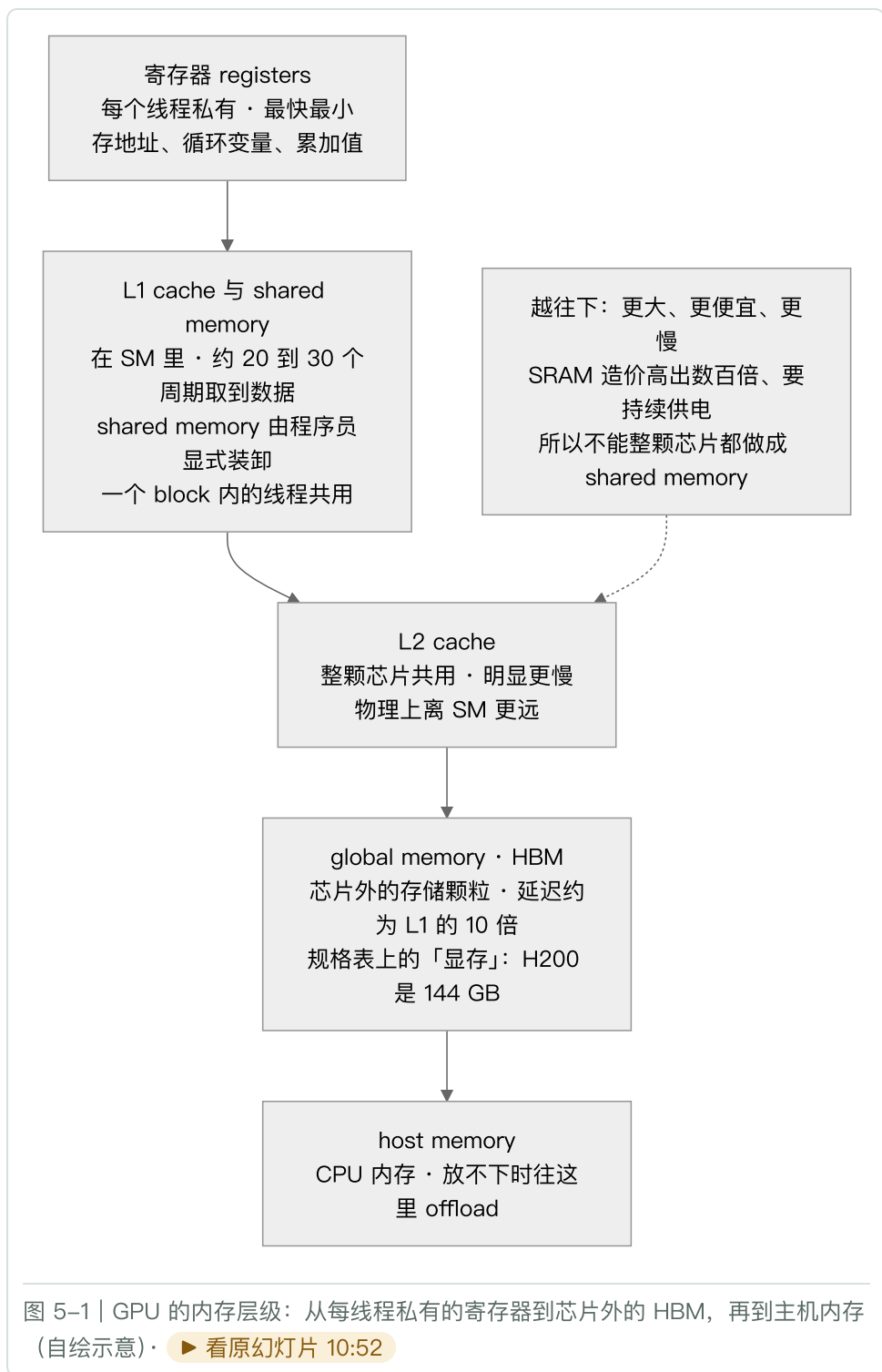
1. 这一讲的位置：算力是硬通货，串行提速早已到头

- **三段结构**：先讲 GPU 的硬件与编程模型（讲者说系统这块反而最"讲道理"——每一步都能推出来）；然后是六个让 GPU 跑快的技巧；最后把它们拼起来，当场"重新发明" FlashAttention。开头挂出一张谜题图：两个 $n \times n$ 方阵相乘的吞吐随 n 变化，本该单调上升，却布满锯齿和断崖——承诺讲完就能解释（第 11 节）。
- **为什么要懂硬件**：第 1 讲 Percy 说过，scaling 的一大半是把资源用满；不懂硬件模型就谈不上高效。哪怕只做架构设计，也得知道模型会怎样在硬件上执行。
- **算力从哪来**：更快的芯片、更高的利用率、更多的芯片、更好的并行——这几年训练比的是谁能调动更多算力，最近一年推理也是。
- **串行提速的终结**：90 年代的答案是提高时钟频率——晶体管变小、频率变高，即 Dennard scaling；这条路在 2000 年代走到头，晶体管

数量还在涨，但变小不再等于变快。剩下的路是横向扩展：让更多单元同时执行指令。GPU 与后面两讲的并行是同一范式的两层。

- **GPU 算力的超指数增长**：一张按代际画的 FLOP/s 图——K20、M40 那会儿还很温和，到 P100、V100 陡然起飞。讲者点出两个推手：2017 年 V100 引入 Tensor Core；此后是结构化稀疏和 FP8 等更短的数字格式（第 7 节会看到低精度为什么等于算力）。
- **推荐资源**见文首：讲者自称不是系统的人，讲义大量参考 Horace He、GPU MODE 和 Google 那本"TPU 书"——后者的练习和作业题很像。

2. GPU 长什么样：SM，和一座内存金字塔



- **CPU 与 GPU 的哲学：**CPU 为串行执行设计——复杂的分支和控制流，所以控制单元大、ALU 少，追求低延迟。GPU 追求吞吐：单个任务可能很久才完成、中途被换下换上，但几百个轻量核心同时干活，总量大。
- **SM (streaming multiprocessor)：**GPU 的基本单元，相当于一个“核”——独立的计算单元，内含若干流处理器（可并行跑不同线

程)、加速部件和自己能直接访问的内存。GPU 靠堆 SM 扩展：讲者说 A100 有 128 个 SM，各自独立编程、独立执行。

小注：GA100 完整芯片是 128 个 SM，量产的 A100 只启用 108 个——讲者后面解释 wave quantization 时用的就是 108。H100 是 132 个。

- **内存才是主角**：讲者反复强调，现代硬件和 LLM 优化是被内存定义的。层级从上到下（图 5-1）：寄存器最快最小；L1 cache 与 shared memory 在 SM 内部，A100 的微基准约 20—30 个周期取到数据；L2 明显慢；global memory 比 L1 慢约 10 倍。规格表上的“显存”就是 global memory——H200 说自己有 144 GB，说的是它。

小注：H200 规格表写的是 141 GB HBM3e (4.8 TB/s)，"144" 是讲者口头的约数。H100 SXM 的数字供对照：HBM3 80 GB、3.35 TB/s；L2 50 MB；每个 SM 的 L1 与 shared memory 合计 256 KB (shared 最多可配 228 KB)，寄存器堆 256 KB；BF16 Tensor Core 稠密算力约 989 TFLOP/s，FP8 约 1,979 TFLOP/s。

- **物理位置决定延迟**：global memory 是芯片外的存储颗粒；L2 和 L1 在芯片上，L1 更是在每个 SM 里面——离计算单元近，所以快。
- **那为什么不整颗芯片都做成 shared memory？** 贵几百倍，也更耗电（第 11 节的问答里讲者展开了三个原因）。所以实用的加速器都是层级结构，得学会尊重它。反例是 Groq：巨量 SRAM 的设计，对推理这类负载很适合——讲者说它最近被 NVIDIA 收了。

小注：据 2025 年 12 月的报道，NVIDIA 以约 200 亿美元取得 Groq 的技术授权并接收其核心团队，严格说是“授权加雇佣”而非整体收购。

- **问答：shared memory 和 L1 cache 有什么区别？** 二者物理上都是 SRAM、都在 SM 里，区别在用法：cache 是自动的，存最近访问过的数据，程序员控制不了；shared memory 是可编程的，你显式把东西放进去、拿出来——tiling 全靠它。L2 比 L1 慢主要是物理距离和互联，并非天生：TPU 上对应 L2 的那层就快得多，硅片上的取舍不同。

3. 编程模型：thread、warp、block，以及 SIMT

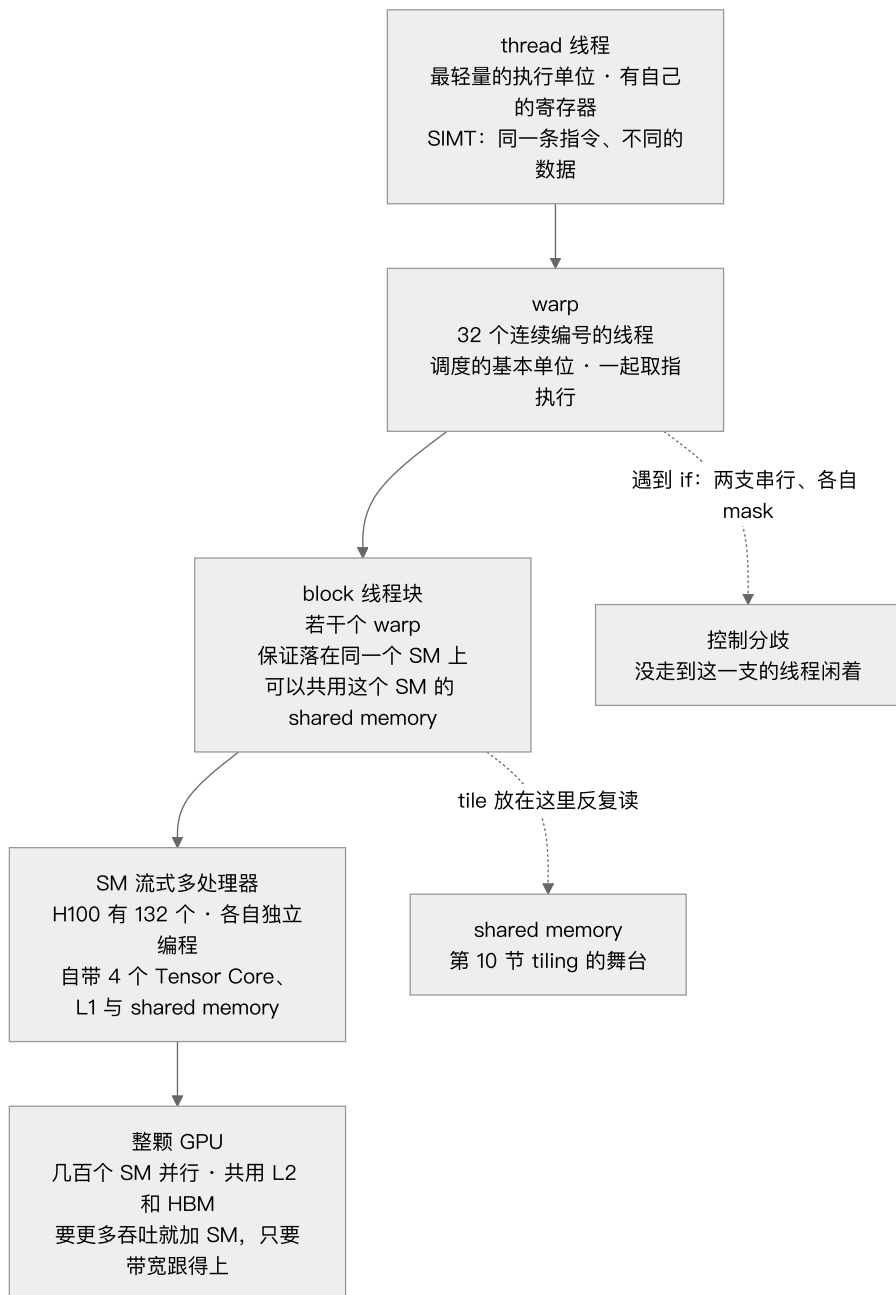


图 5-2 | GPU 执行模型的四层：线程组成 warp，warp 组成 block，block 落在一个 SM 上（自绘示意）· [▶ 看原幻灯片 13:54](#)

• 三个角色

- *thread*: 轻量的并行执行单位。GPU 的线程遵循 SIMT (single instruction, multiple threads): 所有线程执行相同的指令，只是输入不同——能各干各的活，但指令必须一样。这是"可编程性"和"效率"之间的取舍。
- *block*: 一组线程，保证被调度到同一个 SM 上，因此可以共用该 SM 的 shared memory——这是 tiling 的前提：同一个 block 的线程反复读同一块片上数据。

- *warp*: 32 个连续编号的线程组成的调度单位，一起取指、一起执行，调度器不必逐线程决策。

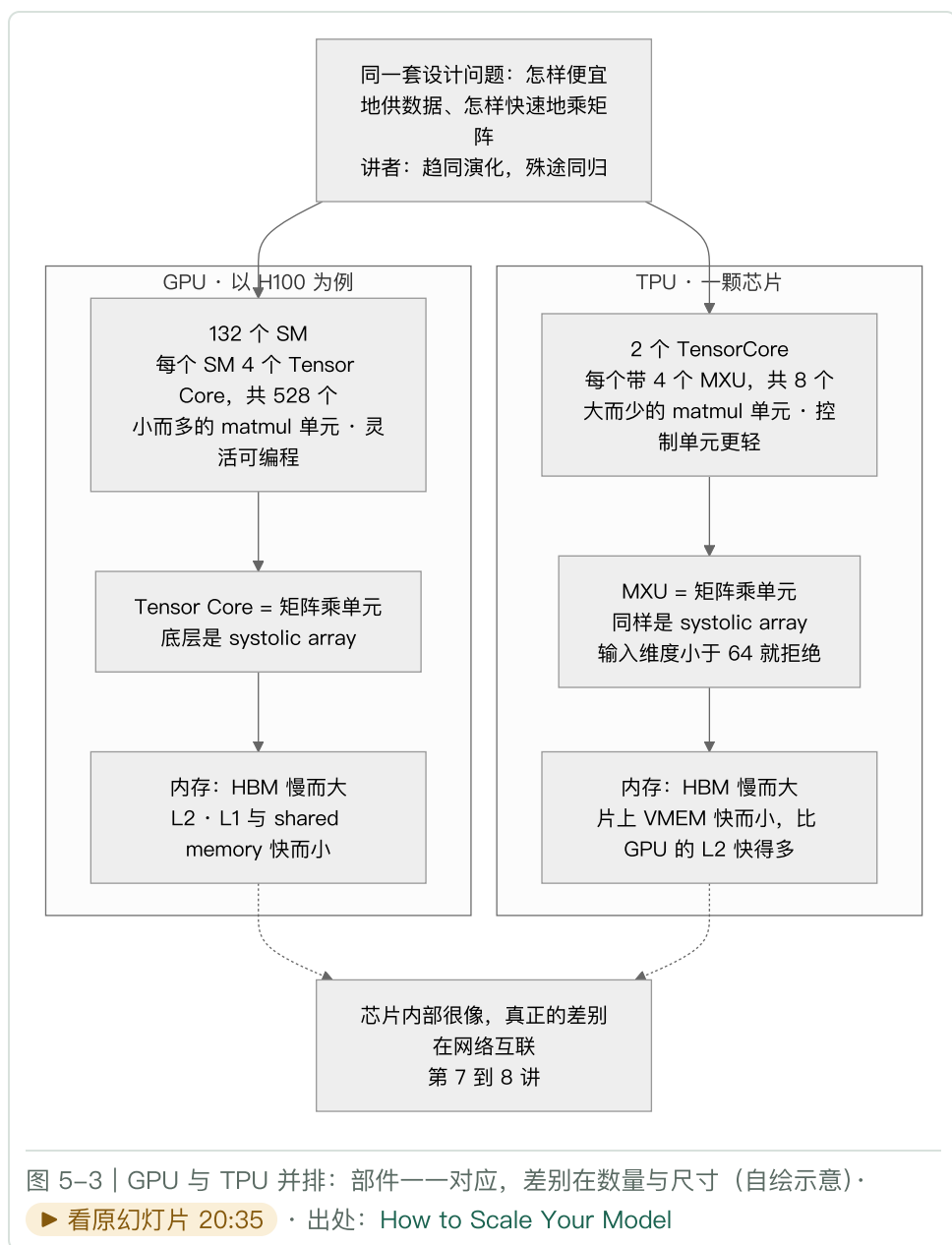
- **问答**: 所有线程执行同一条指令，是 block 内还是 warp 内？——warp 内；调度器决定下一个执行哪个 warp。一个 block 有多少个 warp？——和硬件有关，讲者没记住。

小注：CUDA 里一个 block 最多 1,024 个线程，即 32 个 warp；实际用多少由 kernel 自己指定。

- **设备端代码能碰到的内存**（由近到远）：寄存器（存数组起止地址之类）→ local memory（SM 内）→ shared memory（block 内共享，线程间传数据、复用数据放这里）→ global memory（远处的 DRAM，要付延迟）→ constant memory（很少见人用）→ host memory（CPU 内存，放不下时往这儿 offload）。一句话：一出 shared memory 就慢，整节课的主题就是“把访问归拢，少读 global memory”。
- **GPU 为什么成功**: 要吞吐就加 SM，只要内存带宽跟得上；编程“看似简单”——SIMD 风格下连 PTX 这种底层指令都写得下去，你不是在编排每个线程，而是“这是指令、这是输入，全部跑一遍”，像 map；线程极轻，随时可以停下换另一个 warp 上，某些任务卡住时吞吐也不掉。
- **matmul 成为特权算子**: Tensor Core 之前，人们就用图形硬件的大规模并行做科学计算——讲者放了一篇早期论文，用着色器硬凑出快速矩阵乘（应为 Larsen & McAllister, 2001）。V100 起 NVIDIA 直接给了做 matmul 的专用电路，从此 matmul 与其他可并行运算之间出现吞吐鸿沟：比 GPU 上任何别的浮点运算快 10 倍以上。推论：近未来任何随算力扩展的架构都会以 matmul 为核心。

小注：H100 上 FP32 CUDA core 约 67 TFLOP/s，BF16 Tensor Core 稠密约 989 TFLOP/s，相差约 15 倍。

4. TPU: 趋同演化，芯片里差在“少而大”，芯片外差在网络



- **为什么只讲两页：**多数人写代码还是在 GPU 上；但 TPU 越来越流行，而且它是 GPU 的"另一条演化路线"，对照着看能分清什么是本质、什么只是设计选择。结论先行：高层非常像——要造省电的 ML 加速器，最后都走到同一个地方。
- **核心区别：**TPU 更简单、更专门为 ML 负载优化——控制单元更轻，矩阵乘单元大得多。组成部件一样：一块专门乘矩阵的电路、做并向量运算的部件、某种控制系统；内存也一样是两层：慢的 HBM，加更快的片上本地内存（TPU 里叫 VMEM，字幕识别不清，推断）。差别在各部件的尺寸和灵活性。
- **术语陷阱：**TPU 把它的"SM"叫 TensorCore；GPU 把它的矩阵乘单元叫 Tensor Core。同名异物，靠上下文分辨。

- **数量对比**（讲者的数）：H100 有 132 个 SM，一颗 TPU 只有 2 个 TensorCore；GPU 的矩阵乘单元有 528 个（每个 SM 4 个），TPU 只有 8 个 MXU。GPU 靠"小而多"，灵活、可编程；TPU 靠"大而少"，锁定在大 matmul 上——讲者最近一篇论文做 batch size 扫描，只能扫到 64 为止，因为 TensorCore 拒绝接受小于 64 维的输入。

小注："2 个 TensorCore、8 个 MXU"对应 TPU v4 / v5p 一颗芯片的配置（每个 TensorCore 4 个 MXU）。学生问 systolic array 多大，讲者没答上：TPU 的 MXU 在 v4 / v5 是 128×128，v6e Trillium 起放大到 256×256；GPU 的 Tensor Core 每个只处理很小的矩阵块，所以靠数量取胜。

- **底层同源**：JAX 书里有两张表把 GPU 概念逐一映射到 TPU 概念，映射相当精确——Tensor Core 对 MXU，二者用的还是同一种电路：systolic array（脉动阵列），数据像脉搏一样流过阵列完成乘加。本讲的每个概念都能平移到 TPU。
- **真正的大差别在网络**：讲者今天不展开——芯片本身"活着就是为了乘矩阵"，GPU 与 TPU 最大的差别在怎样把芯片连起来（第 7–8 讲并行）。

小注：课外补一句：TPU 用 ICI 把芯片直连成 2D / 3D torus（一个 v4 pod 4,096 颗）；GPU 是 NVLink / NVSwitch 连成 8 卡节点，再用 InfiniBand 或以太网跨节点。

5. 三条曲线增速不同：为什么今天的优化几乎全是内存优化

- **硬件的三个部件各涨各的**：一张按年份画的图——灰线是算力（FLOP/s），涨得很快；绿线是内存带宽（搬数据的速度），涨得慢得多；蓝线是"并行"（设备之间的互联速度），也很慢。
- **含义**：GPU 编程的早年，算力并不比搬运快多少，不用太操心内存；越往右，算力与带宽的剪刀差越大——所以今天的优化几乎都是内存优化，往后把硬件用满会越来越费劲。

小注：这张图应出自 Gholami et al., 2024 《AI and Memory Wall》：过去二十年硬件峰值算力每两年约 3.0 倍，DRAM 带宽每两年约 1.6 倍，互联带宽每两年约 1.4 倍。

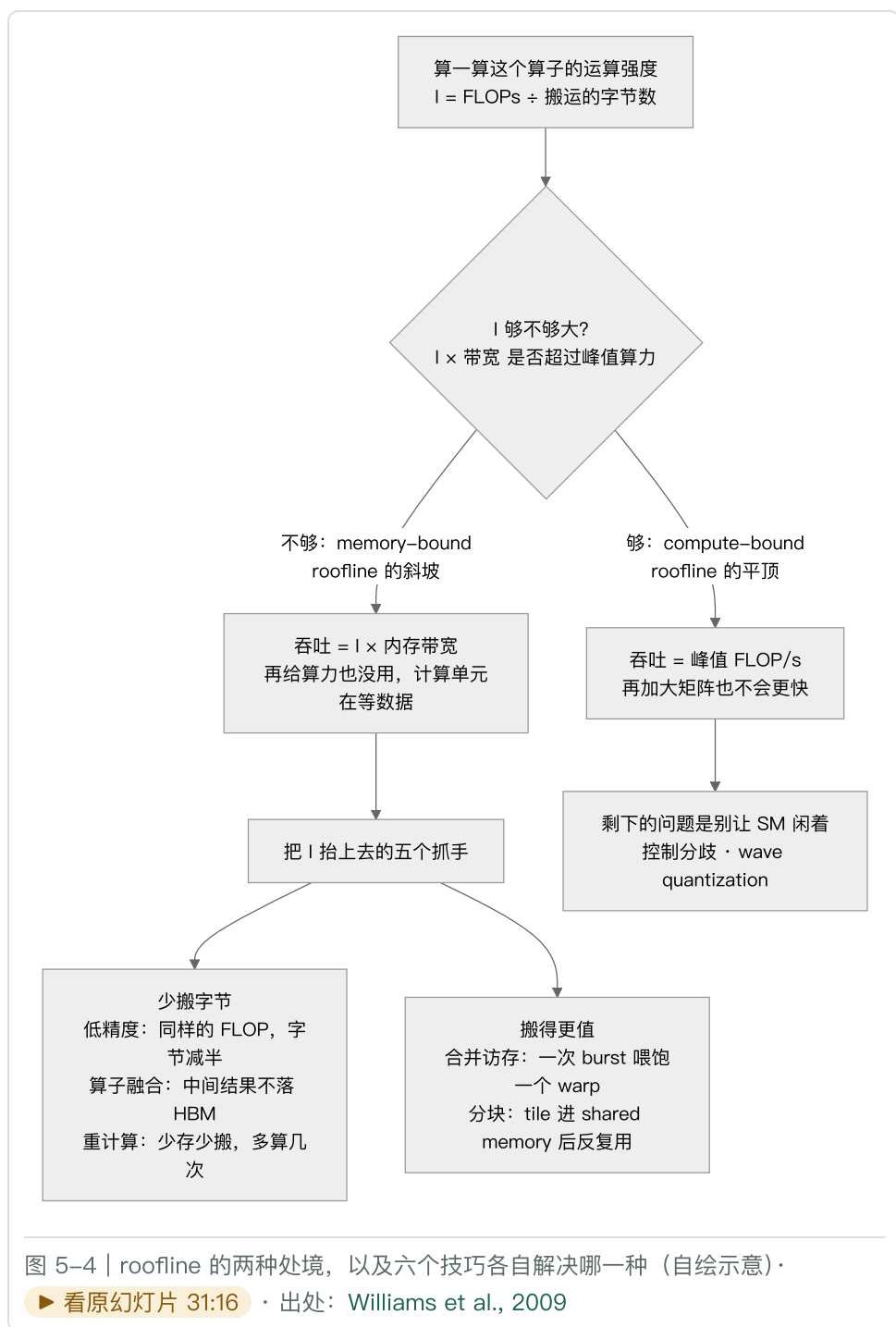
- **问答：这个剪刀差会改变芯片的设计吗？** 讲者说推理那边已经"更疯狂"：去年开始流行 prefill / decode 分离——prefill 是重 matmul，放一种芯片；decode 受内存带宽限制，放另一种芯片。Step-3（中

国的开源模型) 更进一步按层拆: attention 走一种加速器, MLP 走另一种。剪刀差越大, 任何能省着用"珍贵的快内存"的招都值钱, 推理比训练更受内存约束 (第 10 讲)。

小注: Step-3 论文里这叫 AFD (Attention-FFN Disaggregation)。

- **第一部分的心智模型:** GPU 是大规模并行的机器; 算力涨得比内存快得多, 所以一切都要尊重内存层级——能放 shared memory 的别碰 global memory。

6. Roofline 与运算强度: 六个技巧的共同目标



- **谜题图的骨架**：横轴是方阵边长，纵轴是每秒处理的元素数。总体向右上——矩阵越大、活越多，硬件越容易被喂饱；但为什么是锯齿状的、有的点在顶上有的在底下，是第二部分要回答的。
- **Roofline**（第 2 讲 Percy 提过）：吞吐先随“每搬一个字节做多少运算”线性上升（斜坡，memory-bound：给再多算力也没用，计算单元在等数据），到某一点后计算单元饱和，曲线变平（compute-bound：再加大工作量也不会更快）。内存更快，斜坡就整体上抬。写高效 GPU 代码的目标只有一个：**站到平顶上**。

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}, \quad \text{attainable FLOP/s} = \min(\text{peak FLOP/s}, I \times \text{memory bandwidth})$$

I 是运算强度 (arithmetic / operational intensity): 每从内存搬一个字节能做多少次浮点运算; 右式说吞吐被两条"屋顶"里较低的那条卡住—— I 小时受带宽限制 (斜坡), I 大到超过"峰值算力 $\div$ 带宽"这个拐点后受算力限制 (平顶)。

小注: 拿 H100 的规格代进去, 拐点约 $989 \div 3.35 \approx 295$ FLOP/byte。

BF16 的 $n \times n$ 方阵乘做 $2n^3$ FLOPs、至少搬 3 个 n^2 的矩阵共 $6n^2$ 字节, $I \approx n/3$, 所以理想情况下 n 要到 900 左右才可能 compute-bound——这就是谜题图整体向右上爬的原因; 而逐元素运算的 I 只有 $1/4$ 到 $1/8$, 永远在斜坡底部。

- **六个技巧:** 第一个 (控制分歧) 性质不同, 是 GPU 独有的执行模型问题; 其余五个都是同一件事——尽量减少内存搬运, 把 I 抬上去。

7. 技巧 1 与 2: 别写 if; 把数字变短

- **控制分歧 (control divergence):** 每个线程执行同一条指令, 遇到 if 会怎样? CPU 选一支执行完事; GPU 是两支都走一遍——先执行走一支的线程, 另一支的线程被 mask 掉干等, 再反过来。一个条件语句等于两支的时间之和, 而且总有一部分线程闲着。编程简单了, 代价是效率。所以 GPU 代码里 ReLU 之类几乎都写成"乘一个 0/1 mask"而不是 if。

GPU 上写 ReLU: 不用 if, 让整个 warp 走同一条乘法指令

$y = x * (x > 0)$ # mask 与 x 同形, 一条指令全部线程同时完成

若写成 if $x > 0$ 则 $y = x$ 否则 $y = 0$: 两支各占一轮, 不走那支的线程闲着

这三行说明的是"用数据代替控制流": 把分支变成算术, SIMT 才不会被串行化。

- **低精度: 为什么它等于算力。** 讲者把 Bill Dally 的超指数增长图搬出来: 曲线里相当一部分来自数字表示——FP32 $\rightarrow$ BF16 $\rightarrow$ INT8, 位数减半, 要搬的字节就减半。

小注: Dally 本人的讲座里把单芯片推理性能十年约 1,000 倍拆成: 数字表示约 16 倍、复杂指令约 12.5 倍、制程约 2.5 倍、稀疏约 2 倍 (应为此出处; 课上只引了"数字表示是大头"这一点)。

$$\text{FP32 elementwise: } \frac{4 \text{ B read} + 4 \text{ B write}}{1 \text{ FLOP}} = 8 \text{ B/FLOP } (I = \frac{1}{8}), \quad \text{BF16: } 4 \text{ B/FLOP } (I = \frac{1}{4})$$

讲者的"傻例子": 对一个向量逐元素做一次运算, FP32 下每个元素读 4 字节、写 4 字节, 只换来 1 个 FLOP, 即 8 字节/FLOP; 精度减半, 每 FLOP 的字节数也减半——他随即更正说这个量是运算强度的倒数。

- **Tensor Core 里的低精度并不"纯"**: 输入先降精度 (downcast) 再相乘, 部分和的累加用全精度, 输出可能再以 FP32 给出。低精度是"黑魔法", 不在于降位数 (谁都会), 而在于弄清哪个运算能用哪种格式: matmul 的权重和激活可以低精度, softmax 可能得 FP32, 指数运算也许 BF16 能扛住。走到今天花了好几年经验性的试错, 一边降精度一边保住训练稳定。
- **FP8 之后没有标准格式**: 16 位大家基本用 BF16 (CME295 第 4 讲的混合精度到此为止); 8 位就分化了——E4M3 (4 位指数、3 位尾数) 和 E5M2 (指数多、尾数少), 各有用处, 位数太少就没有一刀切的格式。
- **MXFP8** (讲者说"又酷又疯"): 传统 FP8 训练是元素 8 位, 外加一个 FP32 的缩放因子——只有 4 位指数, 太容易上溢或下溢, 得靠缩放把数值拉进可表示的范围。可一个矩阵里各处的量级差很多 (序列的一段激活可能比另一段大得多), 一个缩放因子照顾不过来。于是改成每个子块一个: MXFP8 的元素用尾数更多的 FP8, 每 32 个元素配一个缩放因子, 缩放因子本身也是 8 位——E8M0, 只有指数位, 全是 2 的幂。

$$x_i \approx s_b \cdot q_i, \quad i \in \text{block } b \text{ (32 elements)}, \quad s_b = 2^{e_b} \text{ (E8M0)}, \quad q_i \in \text{FP8 (E4M3)}$$

x_i 是原始值, q_i 是它的 8 位量化值, s_b 是第 b 个块共用的缩放因子——只能是 2 的幂, 所以缩放本身不引入舍入, 只是移动指数。

- **这个设计的坑**: 转置。以前转置不必重新量化; 现在转置后的矩阵和原矩阵"每 32 个一组"的分块模式对不上, 得整个重新量化。实际做法: 训练时每个量化矩阵存两份, 一份原样、一份转置好的。再加上只量化那些经验上"安全"的层——又是一轮试错。
- **收益**: matmul 大约省 20—30%, 矩阵越大省得越多; 到不了 2 倍, 因为量化、反量化的开销不小。问答: 最后一层为什么难量化? 它直接决定 loss, 量化它容易既不稳又掉点; 第一层讲者没有直觉。
- **MXFP4**: 整个格式一页就能画完——可表示的值只在 -6 到 6 之间。结构同上: 按块共用缩放因子, 讲者说每 16 个元素一个、缩放因子是 FP8 的 E4M3。有论文用 FP4 训练过, 但还没听说谁在真实的大模型上成功; 讲者判断下一代模型大概率会上。

小注：按 OCP 的 MX 规范，MXFP4 也是每 32 个元素一个 E8M0 缩放因子；"每 16 个元素、E4M3 缩放因子"是 NVIDIA Blackwell 上 NVFP4 格式的定义——讲者描述的应为 NVFP4（或者幻灯片本身就是 NVFP4）。FP4 训练的论文，应为 NVIDIA 的 NVFP4 预训练报告或微软的 FP4 训练论文之一（推断，见文末表）。

• 问答

- 只量化矩阵吗？什么都能量化（比如 ReLU 之后的激活），但只有 matmul 值得——吞吐提升大，别处的开销通常不划算；算力上乘量化数基本是线性加速，只是被量化、反量化摊薄了。
- 缩放因子要训练吗？不训练，没有梯度——由实现决定：扫一遍取最大最小值，或用运行统计；推理时倒可以拟合。
- 结构化稀疏？MoE 就是成功的结构化稀疏；Chris Ré 做了各种结构化矩阵，但计算收益对表达损失的账经验上没算过来。
- 部署到笔记本的 SOTA？先训一个更大的模型再量化下来；有量化 scaling law 刻画取舍；实践是量化感知训练加事后量化，业界仍在做"量化的科学"。

8. 技巧 3 与 4：算子融合与重计算——拿计算换搬运

- **算子融合 (operator fusion)**：把 GPU 想成一座工厂：仓库是内存，车间是计算单元，中间一条传送带。车间再大，传送带就那么宽；运算越多，原料和半成品在两头来回跑得越多。更好的做法是一座大车间，原料进一次、成品出一次，带宽只付两回。就这么简单，但你会惊讶它有多常没有发生。
- **例子**： $\sin^2 x + \cos^2 x$ 。PyTorch 的计算图是 $x \rightarrow \sin, \cos \rightarrow$ 各自平方 $\rightarrow$ 相加；不做处理时每个算子各自从 global memory 读、写回，中间结果全部落地。融合后是一个 kernel（在 GPU 上一次启动、由大量线程执行的一个函数）：读一次 x ，在 SM 里把整条链算完，写回一次。
- **谁来做**：这种"把一段图压成一个单元"的简单融合，编译器会自动做——torch.compile、JAX 的编译（XLA，和 JAX 结合得更深）都会把这类基本运算融合成一次 CUDA 调用；复杂的融合要手工干预，那是第 6 讲（Triton、kernel）的内容。
- **重计算 (recomputation)**：反向传播要用前向存下的激活（Percy 的 CS221 讲过的计算树）。换成系统视角看同一件事：怎样最少地访存？例子： $x \rightarrow \text{sigmoid} \rightarrow \text{sigmoid} \rightarrow \text{sigmoid} \rightarrow \text{out}$ 。正常做法：前向

读 x 一次，写出 s_2 、 s_1 、 out 三次；反向镜像一遍——一共 8 次访存，而算的只是几个 sigmoid，运算强度极低。

- **反过来：**假设算力便宜到用不完、访存却很贵，那就把中间激活全扔掉。前向读 x 、写 out （2 次）；反向读入 d_{out} 和 x ，现场重跑前向拿到激活，写出 dx （3 次）——5 次访存，是原来的 $5/8$ 。多算了一点，少搬了一截。CME295 第 4 讲里 FlashAttention “重算却更快”的现象，根源就在这里；第 12 节回到它。

9. 技巧 5：合并访存——DRAM 是按 burst 出货的

- **DRAM 的脾气：**global memory 是 DRAM，读它不是一个字节一个字节地给。存储单元排成阵列，选中一段、激活放大器很慢，但选中之后同一段里相邻的数据几乎白送——这一段叫 burst section，比如 128 字节：读第一个位置，同一 burst 里其余位置一并送到。
- **合并 (coalesced) 访存：**一个 warp 里线程的读请求若落在同一个 burst 里，就叫合并访存，一次 burst 喂饱整个 warp；多个 warp 对应多个 burst。反之每个线程各撞一个 burst，就是把整段都读上来却只用一个数。
- **矩阵的例子：**一个 4×4 的行主序矩阵，内存里的顺序是第一行、第二行……让 4 个线程每个周期各读一列中的一个元素——第一轮就撞了 4 个不同的 burst，等于把整个矩阵读了一遍，只为拿一列；反过来让 4 个线程沿一行读，一次 burst 就全拿到了（讲者的记法：行主序矩阵里，线程沿主轴移动的读法不合并）。所以数据怎样排布直接影响大块读取的速度——matmul 里某些访问模式是“特权”的。
- **问答：为什么整段免费？** 讲者的理解：放大器按列（或按行，看图怎么画）选中单元，加电压选中这一步耗时，选中后同一列内多个元素的读出就便宜了；顺带得到整段，但不会白送别的段。

10. 技巧 6：分块——把 tile 搬进 shared memory 后榨干它

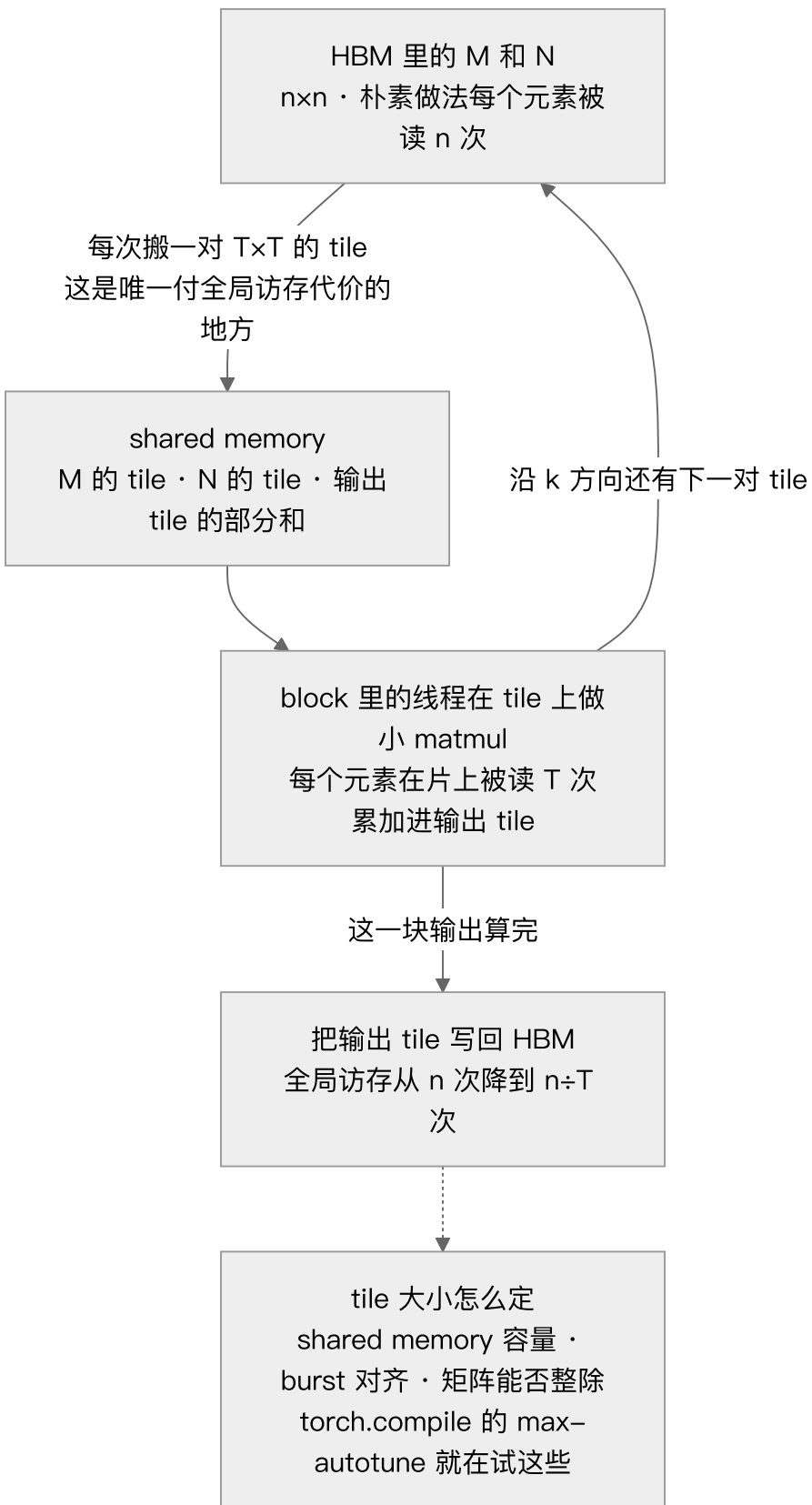


图 5-5 | 分块 matmul: tile 进 shared memory 后反复读, 只在搬 tile 时付 HBM 的代价 (自绘示意) · [▶ 看原幻灯片 58:51](#)

- **思想**：讲者留到最后的、影响最大的一招。把要处理的空间切成 tile，每个 tile 整块装进 shared memory，在片上反复读、把能算的全算完，再换下一块——把对 global memory 的零散访问归拢成整块搬运。
- **matmul 上怎么做**： $n \times n$ 乘 $n \times n$ ，朴素实现里每个输入元素会被读 n 次。矩阵天然可以切成子矩阵：把 M 和 N 各切成 2×2 的 tile，先把两个左上块搬进 shared memory——这是唯一付全局访存代价的地方——在片上做子矩阵乘，部分和累加到同样放在片上的输出 tile； k 方向的几对 tile 依次处理完，再把输出块写回。

$$\text{global reads per input element: } n \xrightarrow{\text{tile size } T} \frac{n}{T}, \quad \text{on-chip reads per element: } T$$

n 是矩阵边长， T 是 tile 边长：分块后每个输入元素从 global memory 读 n/T 次、在片上读 T 次； $T = n$ 的极端就是"全局读一次、片上读 n 次"。也就是全局访存少了 T 倍——tile 越大省得越多。

```
# 一个 block 负责输出 C 的一个 T×T tile；沿 k 方向逐对搬 tile
acc = zeros(T, T) # 部分和，先留在片上
for k0 in range(0, n, T):
    a_tile = load_to_shared(A[i0:i0+T, k0:k0+T]) # 全局访存：每个元素只
    在这里读一次
    b_tile = load_to_shared(B[k0:k0+T, j0:j0+T])
    sync() # 等 block 里所有线程装完
    acc += a_tile @ b_tile # 片上反复读，T 次复用
    sync()
    C[i0:i0+T, j0:j0+T] = acc # 算完才写回 HBM
```

这段伪代码是"一个 block 算一个输出 tile"的骨架：HBM 只在 load 和最后的写回处出现，中间全在 shared memory 里。

- **问答**：这么做是因为整个矩阵放不进片上吗？——对，Groq 那种全 SRAM 的加速器可以整个塞进去，很快也很贵。数据并行、张量并行也像分块？——是，各种并行都带着 tiling 的影子（第 7 讲）。PyTorch 会自动做吗？——任何 matmul kernel 底下都在 tiling，非常规运算才要自己想"局部复用"；但 PyTorch 也不完美，下面就是它的不完美。
- **tile 的微妙之处**：tile 128×128 ，矩阵 256×256 正好切成 4 块；边长加 1 变成 257，就多出两条几乎是空的细长 tile。tile 大小于是成了要

优化的对象：burst 宽度、shared memory 容量、矩阵尺寸一起决定最优 tile。torch.compile 的 max-autotune 就是干这个的——打开后会跑上十几分钟的基准测试，逐个尝试 tile 尺寸，收益不小。

- **对齐与 padding**：如果 burst 段恰好和 tile 一样宽，读一个 tile 只需几次整段读取；矩阵边长偏移 1，每行的起点都错开，tile 与 burst 段不再对齐，读一个 tile 至少要两个 tile 的量。解法是 padding，把边界补到与 burst 段对齐。Karpathy 在 nanoGPT 上把词表从 50257 补到 50304，快了 25%——补零反而加速，看懂了对齐就不奇怪。

小注：出自 Karpathy 2023 年 2 月的推文；50304 是 50257 往上最近的 64 的倍数（也是 128 的倍数）。字幕把两个数识别成了 5257 和 5304。

- **问答：那我该操心 tile 大小吗？** 不必——除非你自己写 kernel，tile 是底层系统的事；但要操心矩阵尺寸：一般取 2 的幂，最好还能被 32 整除。

11. 解谜：那张 matmul 吞吐图的三层结构

- **第一层：斜坡**——运算强度。方阵越大，每搬一个字节做的运算越多，直到把计算单元喂饱（第 6 节）。
- **第二层：颜色**——可整除性。按矩阵边长的可整除性着色：蓝色只能被 1 整除（奇数），橙色被 2 整除，绿色被 8，红色被 16，紫色被 32。奇数和只被 2 整除的吞吐很差；到 16 和 32 就没有区别了。这不是“2 的幂有魔法”，而是 16、32 已经大到能填满一个 burst 窗口，整个 tile 都能合并读取。
- **第三层：周期性断崖**（Percy 管这叫 GPU trivia，讲者说自己很爱）——橙线上每隔一段就掉一截，比如从 1792 到 1793 只加了一维，吞吐骤降。tile 是 256×128：

$$\# \text{tiles} = \left\lceil \frac{n}{256} \right\rceil \cdot \left\lceil \frac{n}{128} \right\rceil : \quad 1792 \rightarrow 7 \times 14 = 98 \leq 108, \quad 1793 \rightarrow 8 \times 15 = 120 > 108 \Rightarrow 2 \text{ waves}$$

n 是边长，108 是 A100 的 SM 数：1792 时 98 个 tile 一波就能全部铺到 108 个 SM 上；1793 时两个方向各多出一个 tile，变成 120 个，第一波 108 个做完还剩 12 个，第二波只有 12 个 SM 干活、其余闲着——tile 数只多了两成，时间却接近翻倍。这叫 **wave quantization**（波次量化）。

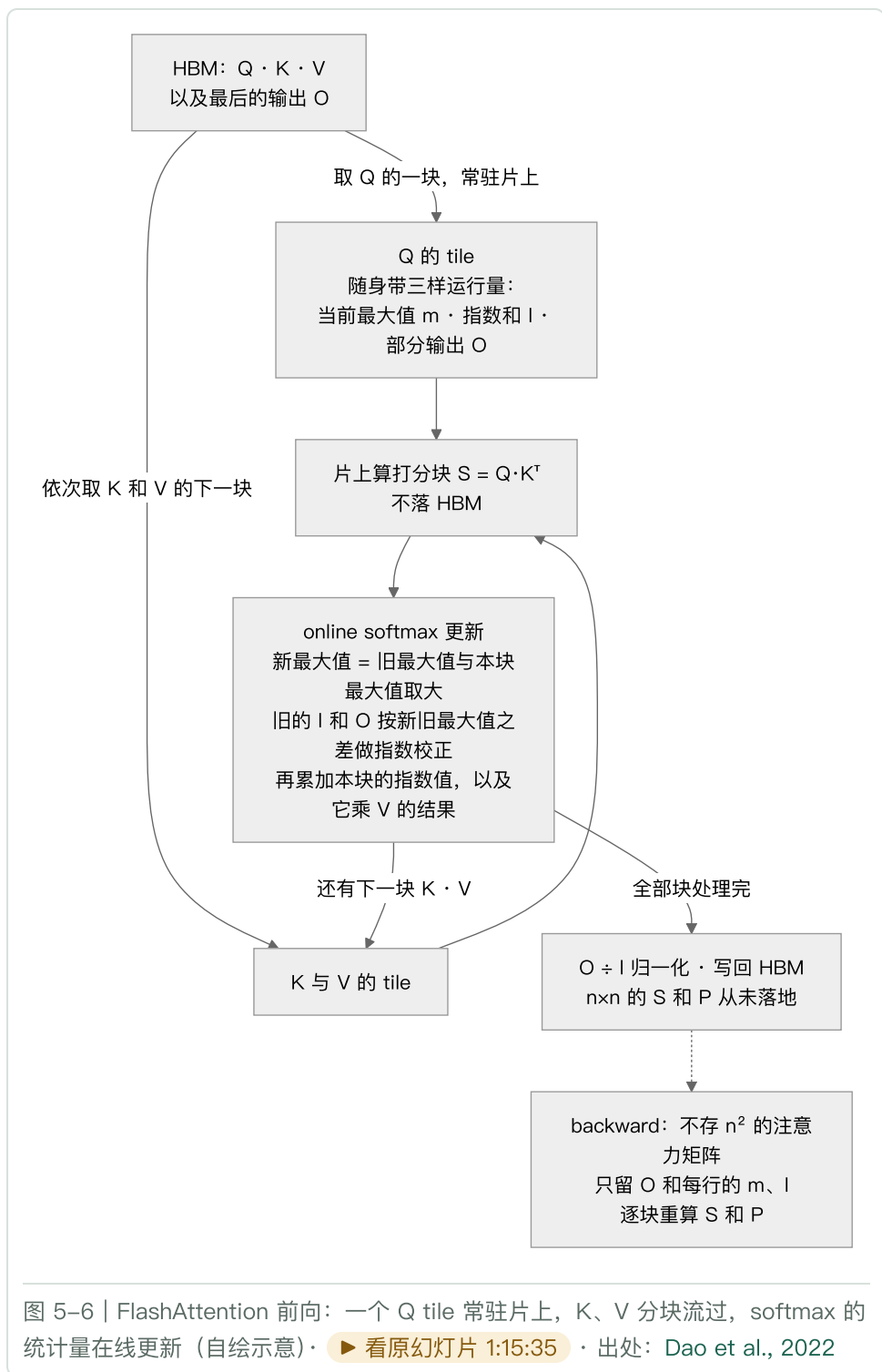
- **第二部分的小结**：memory、memory、memory。合并读（coalescing）；把运算合起来少读几次（fusion）；非读不可的搬进

shared memory 反复用 (tiling)；再拿别的资源换内存 (quantization、recomputation)。

- 问答

- 多堆 SRAM 不行吗？三个原因：造价高；必须物理上靠近计算单元，信号传播有极限；SRAM 要持续供电才能保持数据，很耗能——总拥有成本不划算，快内存要用在刀刃上。
- 被 64 整除会比 32 更好吗？不会，这是"整除 burst 大小"的性质，够了就够了；别的现象可能偏爱更大的因子，但这里不会。
- 为什么是 108？——那是 SM 的数量，"去问 Jensen"。
- wafer-scale（整晶圆芯片，如 Cerebras）呢？讲者不熟，只知道编译极难，会有波的干涉之类的物理效应。

12. 终点：FlashAttention = 分块 matmul + online softmax + backward 重算



- 它是纯系统的胜利：从 PyTorch 的朴素 attention 到一个精心融合的 kernel，延迟大幅下降，显存也省——能做更长的 attention。收益几乎全部来自减少 HBM 的搬运。论文自己的概括只有两句：tiling 加 recomputation，使访存次数不再随序列长度平方增长。CME295 第 4 讲看过它的概貌（三趟 HBM 往返、40.3 GB 变 4.4 GB），这里把每一步对应到本讲的技巧上。

- **要处理的运算**： QK^T 的 matmul、一个 softmax、再乘 V 的 matmul（讲者口头说“三个 matmul”，应为两个）。matmul 已经会了：论文的图 1 就是分块 matmul，没有任何新东西。
- **拦路虎是 softmax**：它是全局运算——按行归一化，把所有 tile 绑在一起，看上去没法分块。标准做法是整行取指数、减最大值（数值稳定）、除以总和。
- **核心观察：online softmax**。一边扫一边维护两个运行量——目前的最大值和“以它为基准的指数和”；每遇到更大的数就换掉最大值，并把已累计的量按新旧最大值之差做指数校正；扫完统一除以分母。因为是在线的，就能一个 tile 一个 tile 地算，不必看到其余 tile。

$$m' = \max(m, \max_j s_j), \quad l' = l e^{m-m'} + \sum_j e^{s_j-m'}, \quad O' = O e^{m-m'} + \sum_j e^{s_j-m'} v_j$$

对某一行： m 是到目前为止的最大打分， l 是以 m 为基准的指数和， O 是尚未归一化的输出累加；来了新一块打分 s_j （配对的值向量 v_j ），先更新最大值 m' ，再把旧的 l 、 O 乘上 $e^{(m-m')}$ 校正到新基准，加上本块的贡献；最后一块处理完后 O/l 就是精确的 attention 输出，与整行一起算完全一致。

```
# 一个 Q tile 的 FlashAttention 前向: m、l、O 三个运行量随块更新
m, l, O = -inf, 0, zeros(T, d)
for K_blk, V_blk in tiles(K, V):          # 每次只搬一对 tile 进 SRAM
    S = Q_blk @ K_blk.T                  # T×T 的打分块，不落 HBM
    m_new = max(m, S.max(axis=1))
    alpha = exp(m - m_new)                # 旧统计量的校正因子
    P = exp(S - m_new)
    l = alpha * l + P.sum(axis=1)
    O = alpha * O + P @ V_blk
    m = m_new
write(O / l); write(m, l)                # 只有 O 和每行的 (m, l) 回到 HBM
```

这段伪代码把上面的更新式放进循环： $n \times n$ 的 S 、 P 从未整体存在过，HBM 上只多了每行两个标量。

- **拼起来**（讲者引的可视化应出自 FlashAttention-2）：虚线框是 SRAM 里分块处理的部分，蓝框留在 HBM。 QK^T 分块相乘，结果留在 SRAM 里直接取指数、更新 softmax 的运行统计量，再到下一块；运行量很少（每行一个最大值、一个分母），放寄存器或 shared

memory 都行；指数值算好就顺手乘 V，最后除以分母。两个 matmul 和 softmax 在片上一口气做完——这就是 fusion 加 tiling。

- **backward 的 recomputation**：如果按常规保存激活，attention 要存 n^2 大小的注意力矩阵。FlashAttention 不存，反向时逐块重算——正是第 8 节的以算换搬。讲者没展开 backward。
- **总结**：要理解到硬件底层才知道为什么做某些决定——别做"矩阵维度要 32 的倍数"的 cargo cult，要知道 burst 与 tile 为什么导致它；算力增长的方式意味着要围着 matmul 设计；算力与内存的剪刀差意味着数据搬运要精打细算；FlashAttention 这类懂硬件的工作，包括懂硬件的架构设计，是未来性能的关键。

13. 六个技巧放在一起看

技巧	针对什么	代价 / 前提	谁替你做
避免控制分歧	SIMT 下 if 让一部分线程空转	用 mask 乘法代替分支	自己写 kernel 时注意；框架算子多已如此
低精度	每 FLOP 搬的字节数	哪些层、哪些运算能降是试出来的；量化开销让 FP8 只省 20—30%	Tensor Core 的硬件支持；库负责缩放因子
算子融合	中间结果在 HBM 的往返	复杂融合要手写 kernel（第 6 讲）	torch.compile / XLA 做简单的
重计算	存激活带来的读写	多算几次前向	activation checkpointing；FlashAttention 的 backward
合并访存	被浪费的 DRAM burst	数据布局、对齐、padding	kernel 与矩阵尺寸的选择
分块	同一元素被反复从 HBM 读	tile 尺寸要调；对齐与 wave quantization	所有 matmul kernel；max-autotune 调 tile

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
谜题图：方阵 matmul 吞吐对边长	总体向右上爬，但布满锯齿与断崖；讲到解谜时再回来看看颜色和 1792 → 1793	1:06	—
GPU 的 FLOP/s 逐代增长	P100 / V100 之后起飞；V100 处标 Tensor Core，之后是结构化稀疏与 FP8	6:14	—

图	看什么	跳转	出处
A100 内存延迟表	L1 / shared 约 20—30 周期；L2 更慢；global 约为 L1 的 10 倍	10:52	—
GPU → TPU 概念映射表	每个 GPU 概念——对应一个 TPU 概念；Tensor Core 对 MXU	19:02	How to Scale Your Model
三条增速曲线	灰线算力最陡，绿线内存带宽、蓝线互联都平缓；剪刀差随时间张开	25:13	Gholami et al., 2024（应为）
Roofline	斜坡 memory-bound，平顶 compute-bound；目标是平顶	31:16	Williams et al., 2009
MXFP8 分块缩放示意	每种颜色的子块配一个 E8M0 缩放因子；想一下转置后为什么对不上	38:52	Rouhani et al., 2023
MXFP8 训练数据流	每个矩阵量化两份（原矩阵与转置）；只量化选定的层	41:57	Mishra et al., 2025（应为）
重计算的两张计算图	上：存 s1、s2，8 次访存；下：只存 out，反向重跑前向，5 次	50:40	—
行主序矩阵的合并访存	4 个线程沿一列读要撞 4 个 burst；沿一行读只要 1 个	56:19	—
分块 matmul 示意	M00、N00 两个 tile 进 shared memory，部分和累加到输出 tile	58:51	—
解谜：可整除性着色	蓝奇数、橙 2、绿 8、红 16、紫 32；16 与 32 重合	1:06:28	—
1792 → 1793 的断崖	98 个 tile 对 120 个 tile，A100 有 108 个 SM；第二波只有 12 个在干活	1:08:00	—
FlashAttention 的分块可视化	虚线框在 SRAM，蓝框在 HBM；运行中的 softmax 统计量随块传递	1:15:35	FlashAttention-2（讲者推测出自这篇）

提到的工作

名称	在本讲里的作用
How to Scale Your Model（Google / JAX 团队）	讲者最推荐的读物："TPU 书"，现在也有 GPU 章节；GPU → TPU 概念映射表出自它；作业练习与它相似
Making Deep Learning Go Brrrr（Horace He）	讲者参考的 GPU 性能博客
GPU MODE（原 CUDA MODE）	讨论 GPU 与 CUDA kernel 的爱好者社区
Dennard scaling（Dennard et al., 1974）	串行提速的旧规律，2000 年代失效，引出并行扩展
NVIDIA K20 / M40 / P100 / V100 / A100 / H100 / H200	代际算力图上的坐标；V100 引入 Tensor Core；A100 108 个 SM；H100 132 个 SM、528 个 Tensor Core；H200 的显存容量

名称	在本讲里的作用
Groq	全 SRAM 的加速器设计，推理友好；讲者说被 NVIDIA 收了（见小注）
Google TPU（v4 / v5p 一类）	2 个 TensorCore、8 个 MXU；MXU 与 Tensor Core 同为 systolic array；差别在网络
Fast Matrix Multiplies using Graphics Hardware（应为 Larsen & McAllister, 2001）	Tensor Core 之前用着色器硬凑矩阵乘的早期黑客工作
AI and Memory Wall（Gholami et al., 2024）	算力、内存带宽、互联三条曲线的图应出自这里
Step-3（StepFun, 2025）	问答里的例子：把 attention 和 MLP 拆到不同加速器（AFD）
prefill / decode disaggregation	推理硬件"更疯狂"的例子：两阶段放不同芯片（第 10 讲）
Roofline model（Williams et al., 2009）	斜坡与平顶：判断 memory-bound 还是 compute-bound
Bill Dally 的算力增长图	超指数增长中数字表示（FP32 → BF16 → INT8）贡献很大
FP8 Formats for Deep Learning（Micikevicius et al., 2022）	E4M3 / E5M2 两种 FP8 格式的来源（讲者未点名）
Microscaling Data Formats（Rouhani et al., 2023, OCP MX 规范）	MXFP8 / MXFP4：按块共用 E8M0 缩放因子
Recipes for Pre-training LLMs with MXFP8（Mishra et al., 2025）	MXFP8 训练要存两份量化矩阵、只量化部分层——课上的数据流图应出自此类 NVIDIA 资料（推断）
Pretraining LLMs with NVFP4（NVIDIA, 2025）· FP4 训练（Wang et al., 2025）	"有论文用 FP4 训练过"的候选（推断）；前者也是"每 16 个元素一个 E4M3 缩放因子"的出处
MoE · Chris Ré 的结构化矩阵（如 Monarch）	问答：结构化稀疏的成功例子与"没算过账"的例子（推断为 Monarch 一类）
torch.compile · JAX / XLA	自动做简单的算子融合；max-autotune 自动搜 tile 尺寸
Stanford CS221（Percy）	反向传播计算树的出处
nanoGPT（Karpathy）	词表 50257 → 50304 提速 25% 的 padding 例子
Cerebras 一类 wafer-scale engine	问答里提到，编译极难
FlashAttention（Dao et al., 2022）	本讲终点：tiling + recomputation 的精确 attention
FlashAttention-2（Dao, 2023）	讲者引用的 SRAM / HBM 分块可视化应出自这篇

名称	在本讲里的作用
Online normalizer calculation for softmax (Milakov & Gimelshein, 2018)	online softmax 的原始出处（讲者未点名）

术语对照

English	中文
SM (streaming multiprocessor)	流式多处理器：GPU 的基本计算单元，相当于一个核
Tensor Core / MXU	矩阵乘单元（GPU 叫 Tensor Core，TPU 叫 MXU）；TPU 的 TensorCore 则相当于一个处理器
systolic array	脉动阵列：数据流过阵列完成乘加的矩阵乘电路
thread / warp / block	线程 / 32 个线程的调度单位 / 保证落在同一 SM 上的线程组
SIMT / SIMD	单指令多线程 / 单指令多数据：所有线程执行同一条指令
control divergence	控制分歧：warp 内线程走不同分支时两支串行执行
registers / shared memory / L1 / L2	寄存器 / 可编程的片上共享内存 / 一级缓存 / 二级缓存
global memory / HBM	全局内存 / 高带宽显存：芯片外、大而慢
SRAM / DRAM	静态 / 动态随机存取存储：片上快存与片外主存的两种工艺
host memory	主机内存：CPU 侧的内存
VMEM	TPU 的片上向量内存（推断）
latency / throughput	延迟 / 吞吐
memory bandwidth	内存带宽：每秒能搬多少字节
arithmetic (operational) intensity	运算强度：每访存一个字节对应的 FLOP 数
roofline model	屋顶线模型：吞吐 = 峰值算力与"强度 × 带宽"中较小者
memory-bound / compute-bound	受内存限制 / 受算力限制
FLOP / FLOP/s	浮点运算次数 / 每秒浮点运算次数
Dennard scaling	登纳德缩放：晶体管变小则频率变高、功耗密度不变的旧规律
kernel	内核：在 GPU 上一次启动、由大量线程执行的一个函数
PTX	NVIDIA GPU 的中间层汇编

English	中文
operator fusion	算子融合：多个运算合成一个 kernel，中间结果不落 global memory
recomputation / activation checkpointing	重计算 / 激活检查点：不存激活，反向时重算
burst section	DRAM 一次读出的连续段（如 128 字节）
coalesced memory access	合并访存：warp 内线程的访问落在同一 burst
row-major	行主序：一行的元素在内存中连续
tiling / tile	分块 / 块：装进 shared memory 反复使用的子矩阵
padding / alignment	补齐 / 对齐
wave quantization	波次量化：tile 数超过 SM 数导致最后一波利用率低
max-autotune	torch.compile 的自动调参模式：穷举 tile 配置做基准
E4M3 / E5M2 / E8M0 / E2M1	浮点格式记法：E 后是指数位数、M 后是尾数位数
scaling factor	缩放因子：把数值拉进低精度可表示的范围
MXFP8 / MXFP4 / NVFP4	按块共用缩放因子的 8 位 / 4 位微缩放格式
quantization-aware training / post-training quantization	量化感知训练 / 训练后量化
structured sparsity	结构化稀疏
prefill / decode disaggregation	预填充与解码分离部署
online softmax	在线 softmax：一遍扫描、维护运行最大值与指数和
wafer-scale engine	整晶圆芯片

字幕勘误

"map moles / map mole" → matmuls；"Grok / Rock" → Groq；"S-man" → 应为 VMEM（TPU 片上快存）；"FPA / MXFPA" → FP8 / MXFP8；"step one three" → Step-3；"Chris Ray" → Chris Ré；"5257 to 5304" → 50257 → 50304；"birth size / birth section" → burst size / burst section；"Jax Compile" → JAX 的编译（XLA）；"Torch Compile" → torch.compile；"max auto tune" → max-autotune；"soft max / sub-quadratically" → softmax / sub-quadratic；"partial summons" → partial sums；"tensor core in the tensor core" → TensorCore 里的 MXU；"CUDA mode" → 现名 GPU MODE；"flops" → FLOPs 或 FLOP/s（看上下文）；"Dennard scaling"

无误；"LM optimization" → LLM optimization；"Nvidia as am I" → 语句破碎，意为"别把 GPU 当成 NVIDIA 网页上的一段文字"。

带走的问题

1. 用第 6 节的 roofline 给你手头的一个算子记账：估它的 FLOPs 和搬运字节数，算出运算强度，和 H100 约 295 FLOP/byte 的拐点比一比——它在斜坡上还是平顶上？六个技巧里哪一个对它有效、哪些无效？
2. 第 10 节说分块让每个元素的全局访存从 n 次降到 n/T 次。shared memory 只有约 200 KB，BF16 下 T 最大能取多少？如果把 tile 换成非方形（256×128），又是在权衡什么？
3. 1792 → 1793 的断崖来自 108 个 SM 与 120 个 tile。如果 tile 改成 128×128，同样的矩阵会不会出现断崖？断崖会挪到哪些边长？这和 Karpathy 的词表 padding 是同一个现象吗？
4. FlashAttention 的 backward 重算 n^2 的注意力矩阵：多做了多少 FLOPs、少搬了多少字节？在什么序列长度以下重算反而不划算？把它和第 8 节"8 次访存变 5 次"的例子放在同一个记账框架里。
5. MXFP8 为了转置存两份矩阵、只量化"安全"的层，最终只省 20—30%。如果下一代硬件把 FP4 的 matmul 再翻一倍，量化、反量化和缩放因子的开销会不会反过来成为瓶颈？这时"低精度 = 算力"的等式还成立吗？

CS336 第 6 讲 | Kernel、Triton 与 XLA

(Kernels, Triton, XLA)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 6 讲；
接着第 5 讲 (Tatsu 讲的 GPU / TPU 概览) 往代码里走 视频：
<https://www.youtube.com/watch?v=xnDHaNUvHBg> (1:26:41；英文字幕为
自动生成，专名和缩写错得不少，见文末勘误) 讲者：Percy Liang (推断：
开场说"Tatsu 周一讲了 GPU 的高层概览，这一讲是它的续篇"，中途又说"去
年我做这个演示时") 课程主页：<https://stanford-cs336.github.io/>；本讲围
绕的东西：Triton (OpenAI 的 kernel 语言)、NVIDIA 的 PTX 与 CUTLASS、
PyTorch 的 profiler 与 `torch.compile`；作业要写的是 FlashAttention
(Dao et al., 2022)

小注：标题里的 XLA (Google 的编译器，JAX / TPU 那条路线) 在这段视频
里完全没有讲到——全程是 NVIDIA GPU、PyTorch 与 Triton；"eager 执行
对编译执行"的对比在本讲只以 `torch.compile` 的形式露面。想看 XLA 只能
等课程网页的讲义。

一句话：第 5 讲说清了 GPU 长什么样，这一讲讲怎么在它上面写代
码、并把性能榨出来：编程模型很干净——thread 组成 thread
block, block 组成 grid；HBM 全局共享、shared memory 一个
block 内共享、寄存器每个线程私有——而硬件很脏，warp 锁步、寄
存器占用率、bank 冲突、访存合并、尾波效应全在决定快慢；所以动
手前先 benchmark 和 profile：profiler 会告诉你一行 `a + b` 背后调
的是哪个 CUDA kernel，也会告诉你朴素写的 GELU 慢在每个算子
各起一个 kernel、在 HBM 之间来回搬 (`torch.compile` 会把整条
链融合成一个 Triton kernel)；最后用四个难度递增的 Triton 例子
——逐元素的 GELU、一行装进一个 block 的 softmax、一行装不下
时分 tile 归约的 row sum、分块 matmul 顺手融合 ReLU——把"一
个 block 从 HBM 读一块进 shared memory、算完写回"的套路打
通，直通作业里的 FlashAttention。

时间轴

时间	内容
0:05	开场：接着 Tatsu 周一的 GPU 概览，这一讲讲代码、Triton kernel 与 benchmark / profile
0:35	GPU 复习：SM、寄存器、L1 / shared memory、L2、HBM；越大越慢，B200 的 HBM 8 TB/s

时间	内容
3:06	编程模型：thread → thread block (CTA) → grid；为什么要有 thread block 这一层
8:45	硬件例子一：warp——32 个线程锁步、分支被串行化、靠切换 warp 隐藏访存延迟
11:20	例子二：warp 占用率与 thread coarsening；算一次 128 线程 × 160 寄存器 → 18%
14:25	例子三：shared memory 的 32 个 bank 与 bank conflict；swizzling
16:58	例子四：memory coalescing——一个 warp 的访存合并成一条 cache line
18:28	例子五：block 占用率与尾波（148 个 SM 对 160 个 block）；小结；问答
21:36	Benchmarking：先量再改；warm-up、多次取平均、CUDA event 与 synchronize；matmul 的耗时曲线
26:15	Profiling：a + b 背后的 add kernel、a @ b 背后的 CUTLASS kernel，名字里能读出架构和 tile 形状
30:23	GELU 三匹马：naive、内置、torch.compile；profiler 里看到多 kernel 对一个融合 kernel
36:34	CUDA 与 Triton 的心智模型：想每个 thread 做什么，还是想每个 block 做什么
39:39	第一个 Triton kernel：分配输出、切 block、launch 语法；pid、offsets、mask、load、store；问答
50:27	PTX：编译器生成的线程级汇编；ld.global / st.global、寄存器、thread coarsening 变成 8 个元素；问答
57:05	路线图；softmax：naive 版 5·MN 次读、3·MN 次写；Triton 版一行一个 block
1:05:22	一行装不进一个 block：切 tile、每个线程一个累加器、最后归约；row sum kernel
1:11:34	终曲 matmul：naive 读 O(MKN)、理想化、tiling；stride；kernel 走读；融合 ReLU
1:21:48	总结；问答：Triton 之外的选择（PTX、ThunderKittens、CuTe 等 DSL）；下一讲多 GPU

核心内容

1. 复习：存储层级与编程模型是怎么对上的

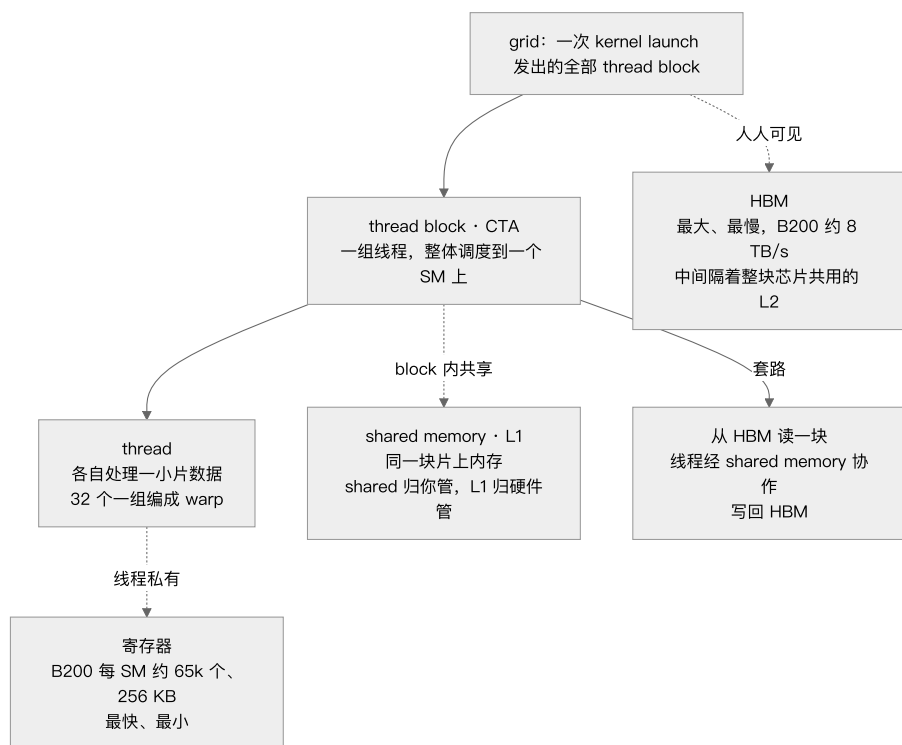


图 6-1 | 编程模型的三层 (grid / block / thread) 各对应哪一级存储 (自绘示意)。

► 看原幻灯片 3:06

- 硬件复习** (第 5 讲讲过，这里只把要用的数字带上；CME295 第 4 讲的 HBM / SRAM 二分是它的粗略版)：一张 NVIDIA GPU 由一堆 **SM** (streaming multiprocessor，流式多处理器，真正干活的单元) 加一块 **HBM** (high bandwidth memory，规格表上的"显存") 组成。从 A100 到 H100 到 B200，每张卡的 SM 数一直在 100 到 200 之间，没怎么变；每个 SM 上的寄存器数也没怎么变——B200 约 65k 个、合 256 KB；SM 上还有 L1 cache 和 shared memory，两者是同一块物理内存，区别是 shared memory 由你显式控制、L1 由硬件自动管，容量同一量级；L2 cache 不按 SM 分，整块芯片共用，比 L1 大；HBM 最大，也是一代代真正在涨的那个数。带宽和容量反着来：寄存器最快，L1 次之，L2 再次，HBM 最慢——虽然 B200 的 8 TB/s 放在别处也不算慢。心智模型一句话：**大的远且慢，快的近且小**。
- 编程模型** 只有三层：**thread** (线程) 各自对一小片数据执行同一段代码；线程编成 **thread block** (线程块，NVIDIA 叫 CTA)；block 排成 **grid** (网格)。launch 一个 kernel，就是把整个 grid 撒到 GPU 上同时跑。H100 / B200 还有 thread block cluster (几个 block 之间可以共享 distributed shared memory)，B200 又加了介于寄存器和

shared memory 之间的 tensor memory 给 tensor core 用——这些对程序员多半不可见，本讲不管。

- **为什么要有 block 这一层**：如果只有一群线程各拿一个元素，逐元素的操作（比如 GELU）完全够用，一个线程一个元素。可 softmax、matmul 这类要在线程之间**通信**的操作不行——严格说也行，让每个线程算输出的一个元素、中间结果全走 HBM，只是 HBM 太慢，这是最坏的策略。正解是用 SM 本地的 shared memory，而 block 就是"共用同一块 shared memory 的那群线程"：一个 block 被整体调度到某个 SM 上，从 HBM 读进一块数据，线程们经 shared memory 协作着算，再写回 HBM。后面的 tiling 玩的全是这一套；Triton 干脆让你原生按 block 来想。
- **存储的归属**：HBM 对所有线程全局可见；shared memory 归一个 block；寄存器归一个线程。

小注：CTA 在 NVIDIA 文档里的全称是 cooperative thread array，字幕听成了 concurrent；"约 65k 个寄存器"应为 65,536 个 32 位寄存器， $65,536 \times 4 \text{ B}$ 正好是讲者说的 256 KB。

2. 编程模型很干净，硬件很脏：五个例子

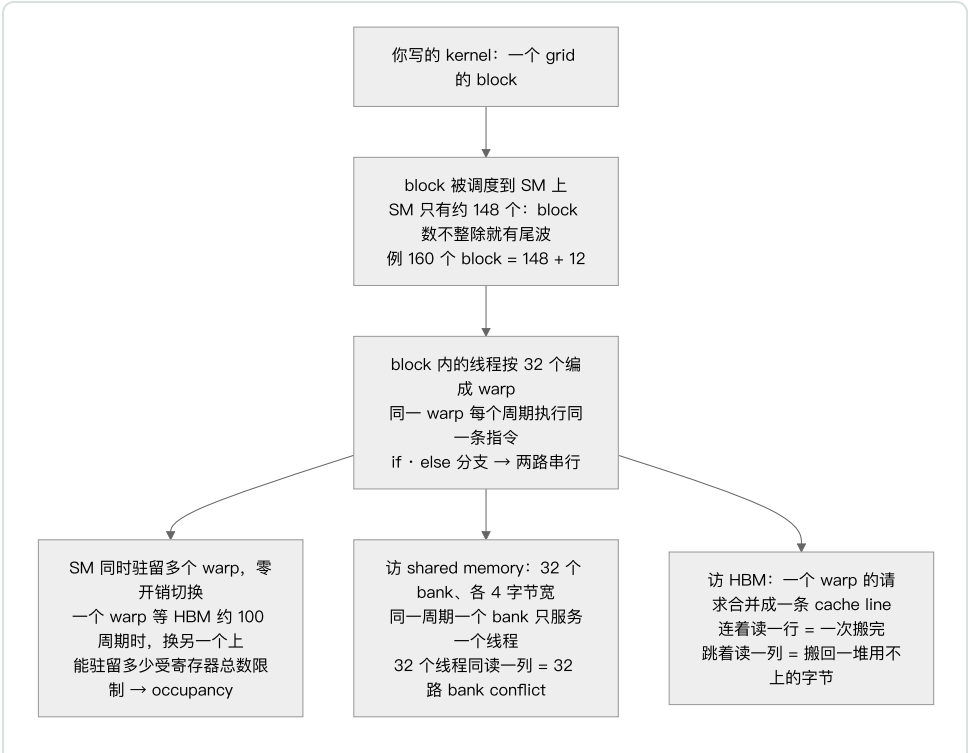


图 6-2 | 从 launch 到执行，硬件在哪几处"不听编程模型的话"（自绘示意）·

► 看原幻灯片 8:45

- **两层之间的落差**：编程模型是硬件的抽象，写 kernel 只需要定义有哪些 block、每个 block 里的线程做什么，和写 Python 差不多；只求正确的话到此为止。可我们谈 kernel 就是为了性能，而性能对硬件极其敏感。Percy 给了五个例子，一部分是第 5 讲的复习。
- **例一：warp 与控制分歧**。block 里的线程实际上按 32 个一组编成 **warp**（64 个线程的 block 就是两个 warp），同一个 warp 里的线程在 SM 上**锁步**执行——每个时钟周期必须跑同一条指令。于是 if A else B 这种分支叫 **control divergence**（控制分歧）：warp 只能先让走 A 的线程跑、其余等着，再让走 B 的跑，分支被串行化了，所以 kernel 里要尽量避免分支。
- **例二：延迟隐藏与 occupancy**。一个 SM 上同时驻留着好几个 warp，warp scheduler 在它们之间**零开销**切换（CPU 上做不到），目的是**隐藏延迟**：某个 warp 读 HBM 要等上百个周期，SM 不会干等，立刻换另一个 warp 去做 tensor core 运算。但能驻留多少 warp 受硬件限制：每个线程最多用 255 个寄存器，SM 的寄存器总数固定，每个线程用得越多、能同时跑的线程就越少——这就是 **occupancy**（占用率）下降。occupancy 低不一定坏：线程少但每个线程干的活多，也可能更快；**thread coarsening**（线程加粗）就是有意为之——逐元素操作让一个线程处理 8 个元素而不是 1 个，线程少了，调度更省事。课上算了一笔账：

$$\text{regs/block} = 128 \times 160 = 20,480, \quad \left\lfloor \frac{65,536}{20,480} \right\rfloor = 3 \text{ blocks}, \quad \frac{3 \times 128 / 32}{64} = \frac{12}{64} \approx 18\%$$

一个 block 有 128 个线程、每个线程用 160 个寄存器，一个 block 就要 20,480 个寄存器（讲者念的是“两万”）；SM 的寄存器只够同时放 3 个这样的 block，即 384 个线程、12 个 warp；而 SM 最多可驻留 64 个 warp，所以 occupancy 只有 12/64，讲者说 18%（精确是 18.75%）——寄存器用得多了，把并发压没了。

- **例三：bank conflict**。shared memory 分成 32 个 **bank**，每个 4 字节宽，同一周期每个 bank 只能被一个线程访问（同一地址除外）；多个线程撞上同一个 bank 就得排队串行。最坏情形：矩阵按行存放，32 个线程同时去读同一列——各撞同一个 bank 的不同位置，32 路冲突，全部排队。你会说那就按行读，可 matmul 得一个矩阵按行、另一个按列，还会有转置，读的顺序不总由你定；解法叫 **swizzling**（把数据在 shared memory 里错位摆放），本讲不展开。profiler 能看到 bank conflict 和 occupancy。

- **例四：memory coalescing**（访存合并）。一个 warp 的 32 个线程访问 HBM 时，请求会合并成一次 32 到 128 字节的事务（一条 **cache line**）一次搬回。最好的情况叫 full coalescing：线程 0 读 M[0,0]、线程 1 读 M[0,1].....整条 cache line 正好全用上；反过来沿着列读，每个线程各搬回一整条线却只用其中一个数。它和 bank conflict 感觉相像，但约束完全不同：那个是 shared memory 的，这个是 HBM 的。
- **例五：block 占用率与尾波**。逻辑上 block 想定义多少都行，物理上 SM 只有那么多个（讲者用的数是 148）。launch 160 个 block，先跑 148 个，等它们跑完再跑剩下 12 个——最后这一“波”里绝大多数 SM 闲着。所以 block 数最好能被 SM 数整除，或者说别让最后一波太瘦。
 - 问答：一个 SM 能不能同时容纳两个 block？如果一个 block 已经把 SM 的 tensor core 用满，再塞一个也快不了；根本问题是 block 不能拆开摊到别的 SM，所以办法是**改 block 大小、从而改 block 数**，把尾巴消掉。
- **小结**：grid、block、thread 的模型很优雅，HBM 全局、shared memory 归 block、寄存器归线程也很清楚；但 warp、bank conflict、coalescing、occupancy 这些硬件细节才决定性能，而它们很难凭空知道——profiler 给一部分信息，SM 数、各级容量得自己查，调度器有时还会做你控制不了的事。比编程模型脏得多。

小注：字幕里的“20 到 128 字节”应为 32 到 128 字节——CUDA 的全局访存事务是 32、64 或 128 字节。问答里学生问“为什么一个 SM 有 4 个 warp scheduler”，讲者说不清楚：自 Volta 起每个 SM 分成 4 个 processing block (sub-partition)，各带一个 warp scheduler 和一条 dispatch 通道，4 就是这么来的（课外事实）。

3. Benchmarking：先量、再改、再量

```
def bench(fn, warmup=3, iters=10):
    for _ in range(warmup):          # 触发懒编译等一次性开销，不计时
        fn()
    times = []
    for _ in range(iters):
        start, end = torch.cuda.Event(enable_timing=True),
    torch.cuda.Event(enable_timing=True)
        start.record(); fn(); end.record()
    torch.cuda.synchronize()        # GPU 是异步的，等它真正跑完
```

```
times.append(start.elapsed_time(end))    # 毫秒
return sum(times) / len(times)          # 讲究的话看整个分布或 P95
```

这就是课上"从零开始"写的计时器的形状：warm-up、CUDA event、synchronize、多次取平均，四样缺一不可。

- **哲学**：这一节内容不多，但要立一条纪律：**benchmark、profile，改代码，再 benchmark、profile**。Percy 故意把它放在 Triton 之前讲：先量出瓶颈在哪，再决定要不要写 kernel。
- **benchmarking 量的是端到端时间**：一个数，不告诉你时间花在哪，但它就是你最终关心的量，而且能看趋势——比如随维度怎么涨。现成的工具有，但本课"从零开始"，自己写一遍是为了讲几个坑：
 1. **warm-up**（第 2 讲提过）：有些东西是懒编译的，第一次跑的开销不该算进去——你在乎的是反复运行时的速度，初始条件无所谓；
 2. **多跑几次取平均**，因为有方差；讲究的话看整个分布或 P95；
 3. 用 **CUDA event**：start 事件 record、跑计算、end 事件 record，然后 **synchronize**——GPU 上一切都是异步的，不设这个同步屏障，你量到的是"发出命令"的时间而不是"跑完"的时间。
- **看曲线**：把两个随机方阵相乘的维度一路放大，时间理论上按立方涨，但直到维度接近 2000 之前曲线几乎是平的——GPU 是为大矩阵乘造的，2×2 的矩阵跑起来纯属浪费，那段平台就是固定开销在主导。

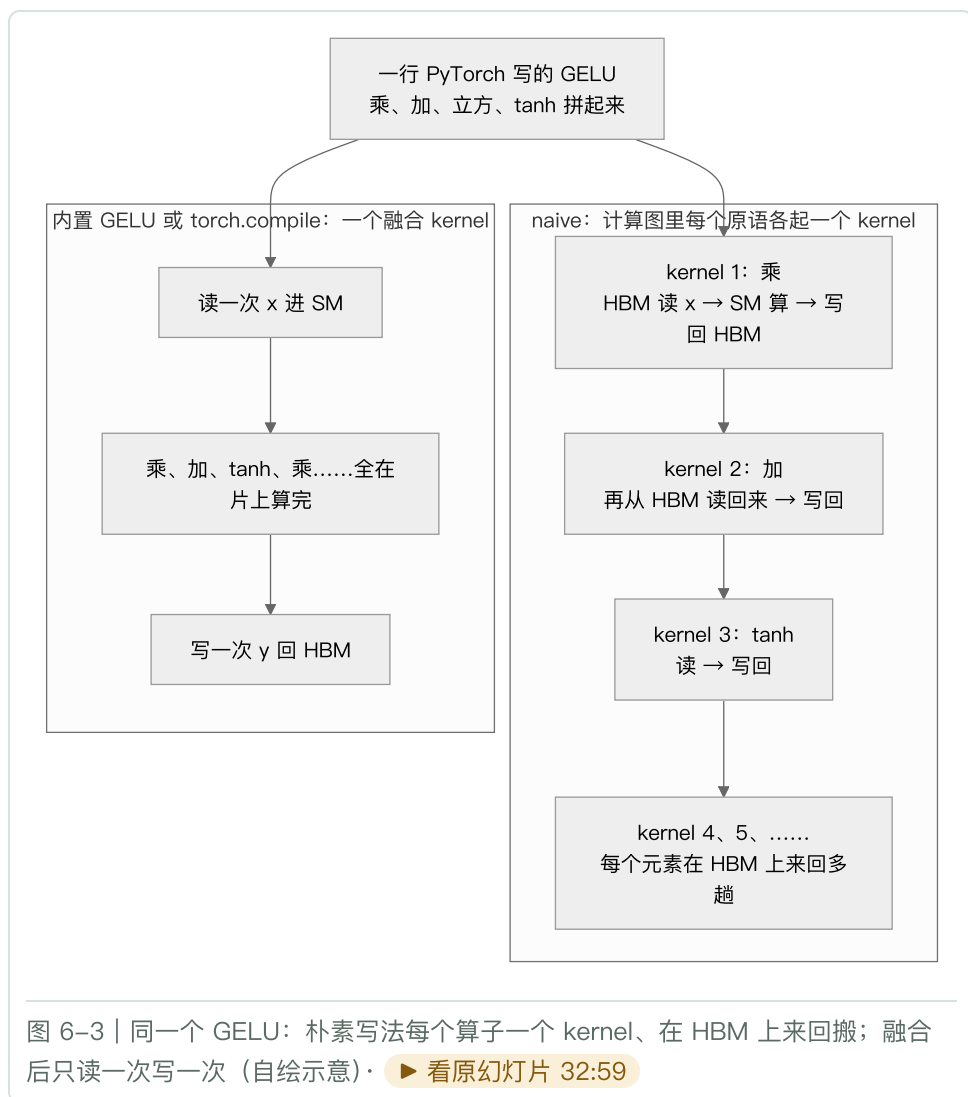
小注：讲者没说"现成工具"指哪个；PyTorch 自带 torch.utils.benchmark，Triton 自带 triton.testing.do_bench，做的正是 warm-up、同步与重复计时这几件事（推断）。

4. Profiling：一行 PyTorch 背后是哪个 kernel

- **profiling 告诉你时间花在哪**。就算不在乎时间，它也是理解"底下到底发生了什么"的最好办法——高层语言写一行、跑一下、出结果，中间全被藏起来了。本讲用 PyTorch 自带的 profiler（先 warm-up，再在 profiler 的 context 里跑一次）；作业里用的 Nsight 信息更细，课上没时间讲。
- **a + b**：两个张量相加，profiler 里出现一个名字很长的 kernel（... CUDA functor add...）——原来加法底下是一个专门的 kernel，在这个例子里当然占 100% 的时间。

- **a @ b**：换成一个更长的名字，里面能读出信息：**CUTLASS** 是 NVIDIA 的 CUDA 线性代数库；**sm100** 指 Blackwell 架构，说明这个 kernel 是专为它写的；**f32** 是精度；后面的 **64·64·16** 是 tile 的形状（第 9 节会讲 tile）。把矩阵改成 128×128，调用的 kernel 换了一个，tile 形状变成 32·32·16——**输入的维度不同，PyTorch 会挑不同的 kernel。**
- **三条观察**：看得到到底调了哪些 CUDA kernel（一般就是名字长的那些）；同一个算子随张量形状换不同 kernel；kernel 的名字本身就交代了实现（库、架构、精度、tile）。作业里会逼你做这两件事，没得选。

5. GELU 的三匹马：naive、内置、torch.compile



$$\text{GELU}(x) = x \Phi(x) \approx \frac{1}{2} x \left[1 + \tanh\left(\sqrt{2/\pi} (x + 0.044715 x^3)\right) \right]$$

Φ 是标准正态分布的累积分布函数；右边是常用的 $\tanh$ 近似，算起来更省——课上实现的就是这个近似式，它由乘、加、立方、 $\tanh$ 几个逐元素原语拼成。

小注：近似式里的常数 0.044715 和 $\sqrt{2/\pi}$ 出自 GELU 原论文 (Hendrycks & Gimpel, 2016)，幻灯片上有，讲者没有念。

- **三个实现**：(1) naive——把近似公式原样敲进 PyTorch；(2) 内置——`torch.nn.functional` 里的 `gelu`；(3) 编译——对 naive 函数调一下 `torch.compile`，得到一个算同样东西的新函数（每个 PyTorch 用户都该知道的一招）。三者随机输入上答案一致。
- **benchmark**：naive 约 3.75（讲者念的数，单位应为 ms），内置快得多，编译版也快得多、但没有内置快。答案相同，性能天差地别——为什么？
- **上 profiler**（图 6-3）
 - naive 版：一堆 kernel——binary functor、unary、add、 $\tanh$ ……PyTorch 表达式的计算图里每个原语各自变成一个 **kernel**。慢的原因：每 launch 一个 kernel，都要把数据从 HBM 拉到 SM、算、写回 HBM，下一个 kernel 再从 HBM 拉回来……算子之间的中间结果必须经过 HBM。
 - 内置版：只有一个 GELU CUDA kernel——没有魔法，就是有人用 CUDA 手写了一个放进标准库，因为大家都用 GELU。
 - 编译版：也只有一个 kernel，而且是 **Triton kernel**——编译器看了计算图，把整条链融合（kernel fusion）成一个 kernel，用 Triton 写出来。
- **结论**：naive = 多个 kernel、多趟 HBM 读写、无融合，慢；内置和编译 = 一个 kernel，每个元素读一次、写一次。
- **问答**：为什么 Triton kernel 比 CUDA 的快？——这个例子里它并不快：编译出的 Triton kernel 是一个 kernel，但比内置的 CUDA kernel 慢。去年测的时候两者更接近；这种数字很依赖硬件，也没人认真调过，看个大意就好。

6. Triton：以 thread block 为单位思考

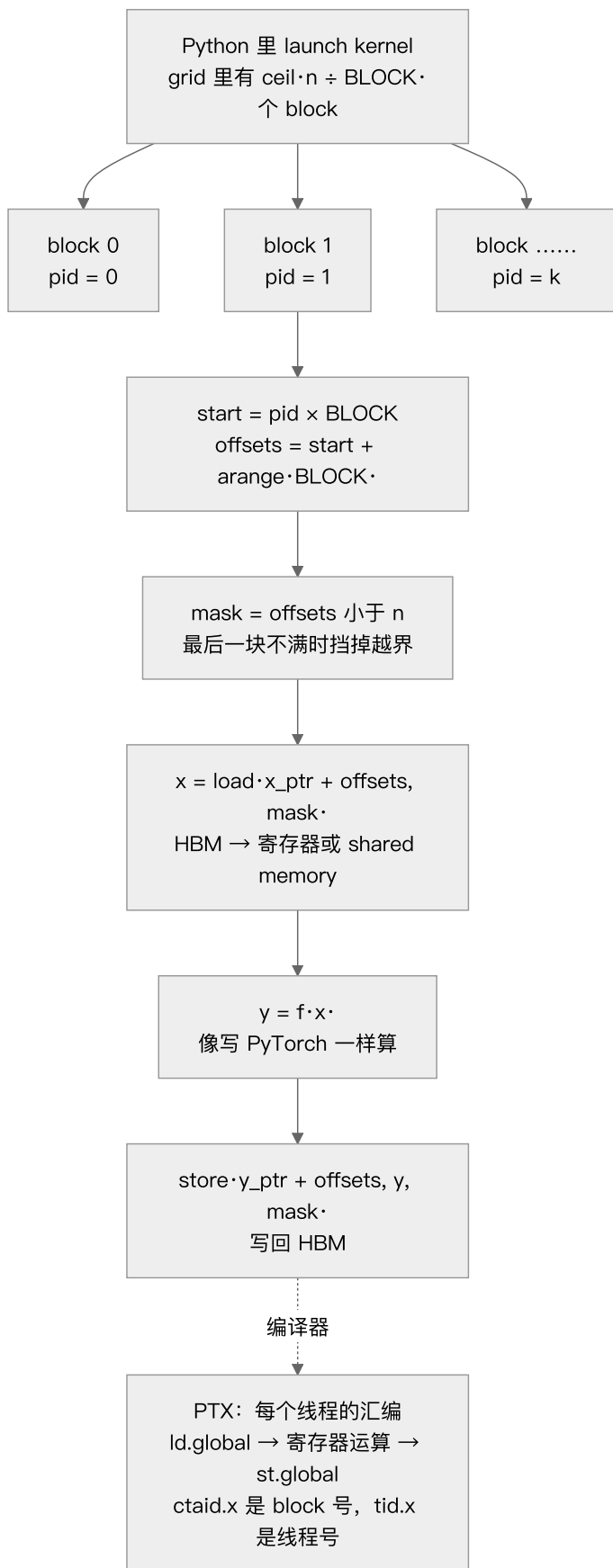


图 6-4 | 一个 Triton kernel 的骨架：grid 里每个 block 醒来、认领自己那段 offsets、读、算、写，再由编译器降到 PTX（自绘示意）· [▶ 看原幻灯片 42:42](#) · 出处：Triton (OpenAI)

- **CUDA 的心智模型是"每个线程做什么"**：NVIDIA 的 CUDA 是多年来写 kernel 的正统，代码里拿到一个线程 ID，然后针对这个线程执行。好处是贴近硬件上真正发生的事，控制粒度最细；坏处是线程之间要通信时（block 内的线程一起读 HBM、同步、再算），同步和 shared memory 的记账全得你自己做。纯逐元素的操作，CUDA 完全够用，甚至更简单；操作一复杂，Triton 的抽象才显出价值。
- **Triton 的心智模型是"每个 block 做什么"**：OpenAI 开发，如今已相当标准。一个 block 把数据装进 shared memory、操作、写回全局内存——block 是介于"单个元素"和"整个矩阵"之间的中间层次：PyTorch 里你定义几个巨大的矩阵、一声 matmul，思考的是怎么把事情凑成大矩阵乘；Triton 是它和逐元素之间的混合体。对本课和入门足够强，要榨干最新硬件的每个新特性时未必给你全部自由度。
- **第一个 kernel 的准备**（普通 PyTorch 部分）：Triton 里不再是函数式的"返回一个值"，而是显式地读、写，所以要先分配输出张量；输入可以任意大、塞不进一个 SM，所以要切 block——约 8K 个元素、block 大小取 1024，就是 8 个 block；launch 用方括号语法（kernel 名后面跟着 grid 的形状，再跟普通的参数），对 grid 里每个 block 调用一次 kernel 函数。
- **kernel 内部**（图 6-4）：参数里的 x、y 现在是指针——就是整数地址，得习惯；每个 block 醒来第一件事是问"我是谁"：program_id 给出 block 号 pid；start = pid × BLOCK 是它在 x 里的起点；offsets = start + arange(0, BLOCK) 是它负责的那一段下标；元素数不整除 block 大小时，用 mask 挡住最后一块的越界部分（非最后一块的 mask 全真）；load 是指针算术——x 的地址加上 offsets，从 HBM 把这一段读进来，之后就当它是个普通向量做计算；store 按同样的 offsets 和 mask 写回。**所有 kernel 都长这样**：输入、输出、醒来、认领下标、读、算、写。

```
@triton.jit
def gelu_kernel(x_ptr, y_ptr, n, BLOCK: tl.constexpr):
    pid = tl.program_id(0)                                # 我是第几个 block
    offsets = pid * BLOCK + tl.arange(0, BLOCK)           # 我负责的那一段下标
    mask = offsets < n                                     # 最后一块可能不满足
    x = tl.load(x_ptr + offsets, mask=mask)               # HBM → 片上
```

```

    y = gelu_formula(x)                                # 和 naive 版一模一样的
逐元素公式
    tl.store(y_ptr + offsets, y, mask=mask)            # 片上 → HBM

y = torch.empty_like(x)                                # 输出要自己分配
grid = (triton.cdiv(x.numel(), 1024),)                # 8K 个元素 → 8 个
block
gelu_kernel[grid](x, y, x.numel(), BLOCK=1024)

```

逐元素 kernel 的全部骨架：醒来、认领、读、算、写；换一个逐元素函数，只改中间那一行。

• 问答

- 和 CUDA 有什么区别？逐元素的例子里几乎一样，CUDA 甚至更简单（醒来、认领线程、算一个元素）；这里是"向量化"的版本，一个 block 操作一段。等到操作不止逐元素，CUDA 就烦人得多。
- 怎么用上 tensor core？你控制不了，硬件（和编译器）决定往哪放。
- 从 HBM 到 shared memory 到寄存器，一步步到底发生了什么？先说破：这段代码某种意义上是"谎言"——GPU 并不真的在跑 Triton，它是写给我们看的计算规格；编译器把它变成 PTX 再去执行（第 7 节）。概念上，x 的指针是 HBM 里的地址，load 取回那一段数据，局部变量 x 实际住在寄存器或 shared memory 里，住哪由 Triton 决定，你不用管。
- load 会不会卡住？会——一条 load 要阻塞若干周期；但它跑在某个 SM 的某个 warp 上，SM 同时驻留多个 warp，等数据时 warp scheduler 换别的 warp 上，数据到了再切回来（第 2 节的延迟隐藏）。

7. PTX：编译器替每个线程写的汇编

- PTX 是 NVIDIA GPU 的中间汇编语言，Triton 编译器的输出。看它不是为了写它，是为了对"底下发生了什么"有个体感：
 - 这时看到的是一个线程在做什么——block 这一层已经被编译掉了；
 - ld.global 从 HBM 装进寄存器（r 开头是整数寄存器，f 开头是浮点寄存器），接着一串 mov、mul（寄存器乘常数放进另一个寄存器），最后 st.global 写回 HBM——和 Triton 里 load、算、store 的三段一一对应；

- 代码里同一段反复出现了 8 遍：这就是 **thread coarsening**——编译器发现这个线程太轻，自作主张让它一次处理 8 个元素；
- 这段代码只编译一次，每个线程跑的是同一份，靠传入的编号区分自己：ctaid.x 是 block 号，tid.x 是 block 内的线程号；
- PTX 里仍有很多没说的：跑在哪个 SM、warp 怎么排——那些由硬件控制，你看不到。

• **问答：**PTX 是不是只由编译器生成？多数情况是。确实有人手写 PTX——如果你认为自己比编译器强；NVIDIA 的编译器已经相当成熟，一般不需要；换到不那么成熟的加速器上，有时就得伸手进去多扶一把。为什么一个 SM 有四个 warp scheduler？讲者说不清楚原因（见第 2 节小注）。

小注：PTX 之下还有一层真正的机器码 SASS，由 ptxas 生成、随架构而变，讲者没提；平时说“看汇编”多半看的是 PTX，因为它跨代稳定、可读。

8. 归约：一行装得进一个 block，和装不进的时候

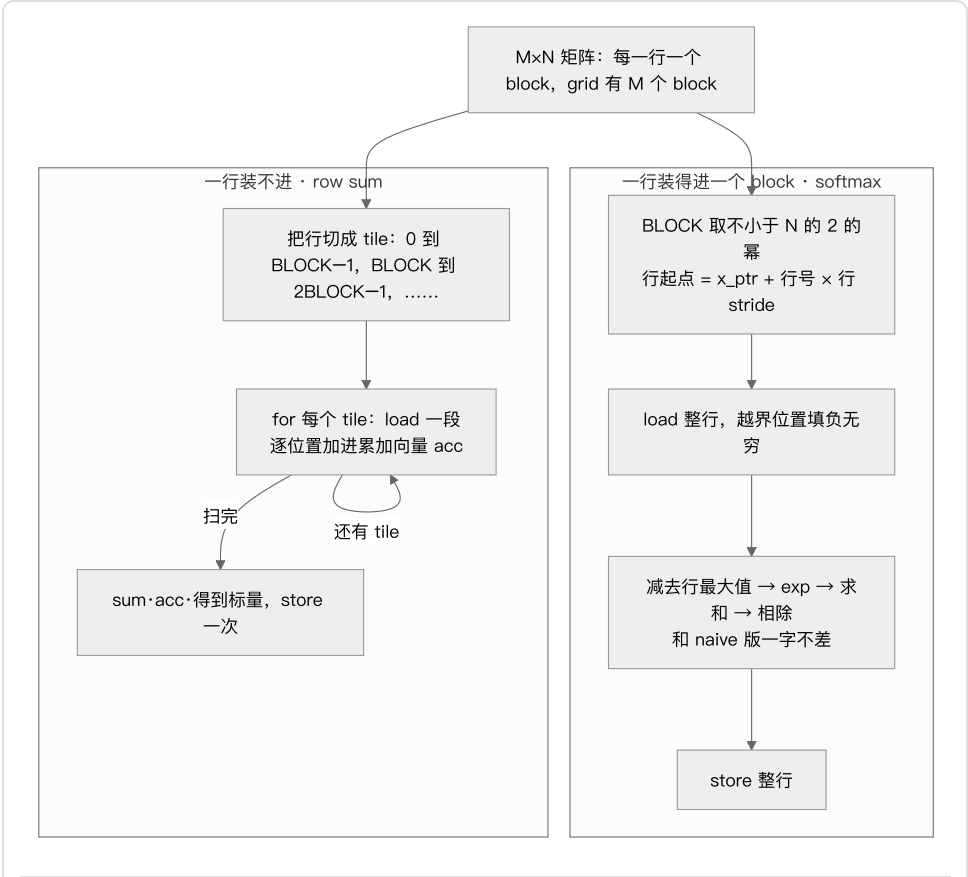


图 6-5 | 两种行归约：一行装进一个 block 时像写 PyTorch；装不进时 block 内要循环扫 tile 再归约（自绘示意）· [▶ 看原幻灯片 1:06:54](#)

$$\text{softmax}(x)_j = \frac{e^{x_j - m}}{\sum_k e^{x_k - m}}, \quad m = \max_k x_k$$

x 是矩阵的一行， m 是这一行的最大值；减掉 m 是为了数值稳定，结果与不减完全相同；分母是整行共用的归一化常数，所以 softmax 不是逐元素操作，而是"逐行"操作。

- **softmax**：对矩阵每一行做指数并归一化，attention 和输出概率都靠它。naive PyTorch 写法（作业一里有）：按行求 max、减掉、exp、按行求和、相除。数一数读写：**这是五个不同的 kernel**（除非 torch.compile），每个都要从 HBM 读、往 HBM 写——约 $5 \cdot MN$ 次读、 $3 \cdot MN$ 次写，而原则上远用不了这么多。
- **Triton 版**：脚手架和 GELU 一样，核心计算和 naive 版几乎一样。关键决定是一行一个 **block**：softmax 不是逐元素，但是逐行——行与行之间不交互，而 block 之间本来就没有 shared memory，正好。准备：分配输出；BLOCK 取"不小于列数的 2 的幂"（图个吉利）；block 数 = 行数 M ；除了输入输出指针，还要传 **stride**（告诉 kernel 往下跳一行要走多远）。kernel 里：醒来知道自己是哪一行；列下标 0 到 BLOCK-1；行起点 = 数据起点 + 行号 $\times$ 行 stride；load 整行，**被 mask 挡掉的位置填 $-\infty$** （它在 softmax 里相当于 0）；然后 max、减、exp、sum、除，写回。**只要一行装得进一个 block，写起来就像普通 PyTorch。**
 - 问答：按列做 softmax？指针是任意的，把列偏移乘上行 stride 就行。
- **一行装不进一个 block 时**（例：约 4K 列、BLOCK 只有 1024）：把行切成 4 个 **tile**，block 内循环扫过各个 tile，每个线程维护一个累加器，最后再归约。讲者换成 row sum 举例（比 softmax 好想）：block 仍然一行一个；tile 0 是第 0—3 列、tile 1 是第 4—7 列、tile 2 是第 8—11 列……第一趟每个线程拿到自己那个位置的数，第二趟把下一 tile 对应位置的数加进来，扫完所有 tile 后手里是一个"每线程一个数"的累加向量，sum 一下得到标量，写出去。累加器住在寄存器还是 shared memory 由 Triton 定，block 大到一定程度就只能进 shared memory。
- **block 与 tile 的区别**（讲者特意强调）：GELU 里也把数据切成了段，但那些段是 **block**，彼此独立、各自被调度；这里的 **tile** 属于同一个 block，由它顺序处理——从这里开始，代码不再像 PyTorch，因为数据装不进 shared memory 了。

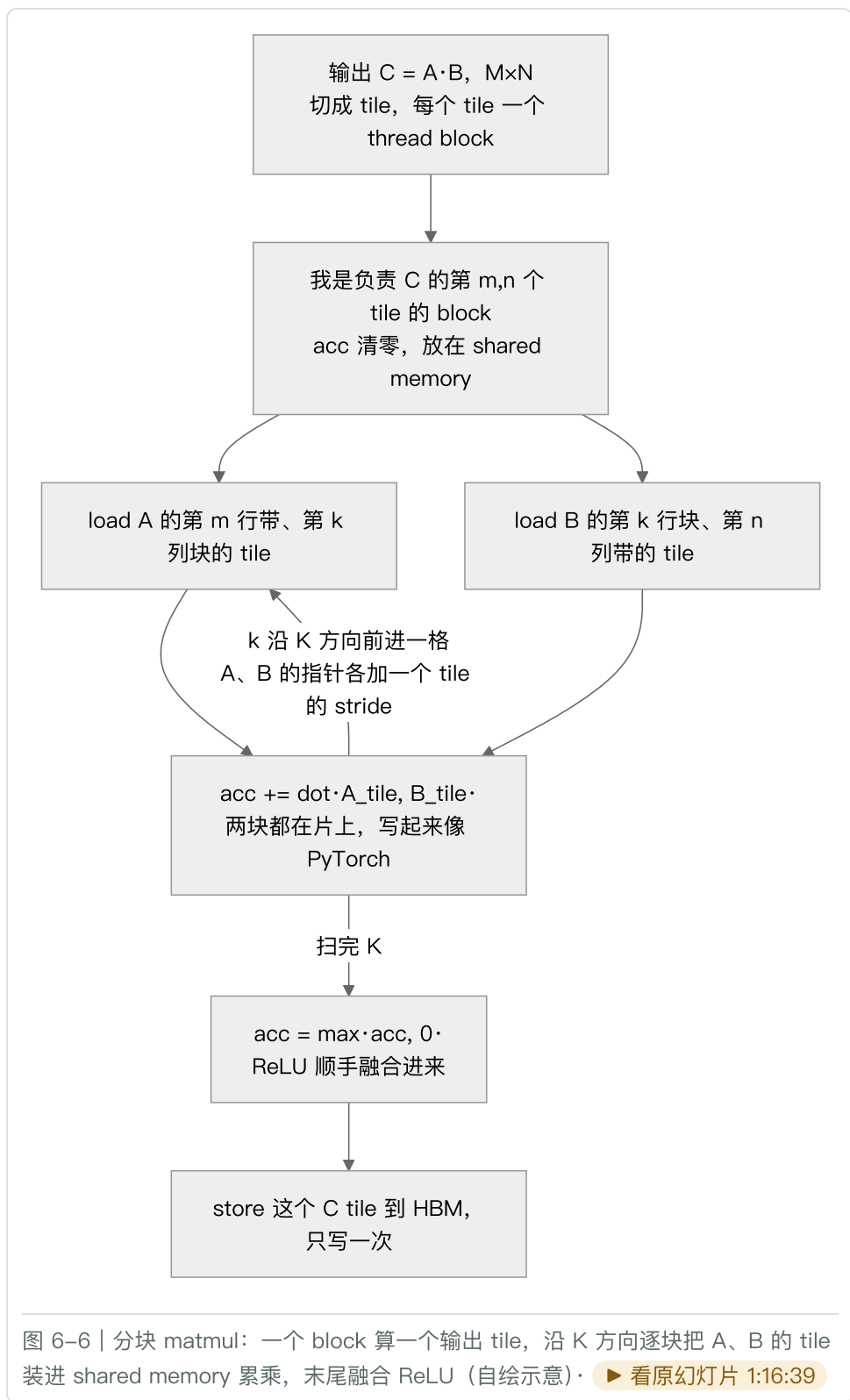
```

@triton.jit
def rowsum_kernel(x_ptr, out_ptr, N, stride_row, BLOCK:
tl.constexpr):
    row = tl.program_id(0)                                # 一行一个 block
    acc = tl.zeros((BLOCK,), dtype=tl.float32)           # 每个位置一个累加器
    for start in range(0, N, BLOCK):                       # 逐 tile 扫过这一
行
        cols = start + tl.arange(0, BLOCK)
        x = tl.load(x_ptr + row * stride_row + cols, mask=cols < N,
other=0.0)
        acc += x
    tl.store(out_ptr + row, tl.sum(acc))                   # 最后归约一次，写一
个数

```

"一行装不进"的通用形状：block 内一个 for 循环沿 tile 推进，片上累加，最后写一次。

9. 终曲：分块 matmul，顺手融合 ReLU



- **naive kernel**: $C = A \cdot B$, A 是 $M \times K$ 、 B 是 $K \times N$ 、 C 是 $M \times N$ 。最直接的写法是一个线程负责 C 的一个元素: 沿 K 循环, 从 HBM 读 A 的一行、 B 的一列的对应元素, 乘、累加, 最后写出那一个数。正确, 但读写糟糕: 对每个 (m, n, k) 都要读一次 HBM, 读的次数是 $O(MKN)$ 量级; 按第 2 讲的 **arithmetic intensity** (算术强度: 做的运

算数除以搬的字节数，越高越好）算，运算也是 $O(MKN)$ ，比值是常数——这就是 memory-bound 的写法。

- **冗余在哪**：算 C 同一行的相邻两个元素，A 的那一行被反复读。要是能只读一次就好了。
- **理想化方案**：把整个 A 和 B 装进 shared memory，然后算 C——读的次数从三次方降到平方，算术强度变成 $O(N)$ 量级，正是第 2 讲说的理想上限。问题：A、B 通常太大，装不进 shared memory。
- **tiling**：经典的折中——“全局看像 naive，局部看像理想化”。把 C 切成 tile，**每个 tile 一个 thread block**，不同 tile 由不同 block 完全独立地算。我这个 block 醒来，沿 K 方向逐块前进：把 A 的对应 tile 和 B 的对应 tile 装进 shared memory，两个小块相乘（这时像理想化方案），累加进 shared memory 里的部分和；扫完整个 K 之后，把这个输出 tile 写回 HBM——只写一次。算术强度升到 $O(\text{tile 边长})$ ：到不了 $O(N)$ （那要求全装下），但 tile 够大就不坏。第 5 讲 Tatsu 画过同一张图。

$$\text{naive: } \frac{O(MNK) \text{ FLOPs}}{O(MNK) \text{ reads}} = O(1), \quad \text{ideal: } \frac{O(MNK)}{O(MK + KN + MN)} = O(N), \quad \text{tiled: } O(T)$$

三种写法的算术强度（运算数除以 HBM 读取数）：naive 每次乘加都读一次，是常数；全装进 shared memory 时每个元素只读一次，强度随矩阵边长 N 线性增长；tiling 介于两者之间，T 是 tile 的边长。

- **kernel fusion 的顺手活**：既然已经在写 kernel，想在 matmul 后面接一个逐元素激活（这里是 ReLU，一层 linear 后接 ReLU 是常事），在写回之前顺手做掉就行——否则多一个 kernel 就多一趟 HBM。
- **stride**：张量是 multidimensional 数组，在内存里却是一维的；stride 告诉你从（行，列）这样的多维下标怎么算出线性地址。一个 4 列的矩阵，往下一行走 4 个位置、往右一列走 1 个；转置了就反过来。kernel 里的下标运算全靠它。

$$\text{addr}(i, j) = \text{base} + i \cdot s_{\text{row}} + j \cdot s_{\text{col}}$$

i、j 是行号和列号，s_row、s_col 是两维的 stride；行优先存放的 4 列矩阵 s_row = 4、s_col = 1，转置后两者互换。

- **kernel 走读**：launch 部分不新鲜；醒来知道自己负责 C 的第 (m, n) 个 tile；一串下标运算（该看 A 的哪些行、B 的哪些列、0 到 K 的循环），得到 A、B 在本 tile 处的指针；分配累加矩阵；然后是和 row sum 同样形状的循环，只不过同时沿 A 的行带和 B 的列带前进：load 小 A、load 小 B、dot 一下（片上的东西写起来就像

PyTorch)，指针各前进一个 tile；末尾套上 ReLU，写出。下标要盯紧，算法的形状则很清楚。

```
@triton.jit
def matmul_relu_kernel(a_ptr, b_ptr, c_ptr, K, sa_m, sa_k, sb_k,
sb_n, sc_m, sc_n, BM: tl.constexpr, BN: tl.constexpr, BK:
tl.constexpr):
    pm, pn = tl.program_id(0), tl.program_id(1)          # 我负责 C 的第 pm, pn 个 tile
    rm, rn, rk = pm * BM + tl.arange(0, BM), pn * BN + tl.arange(0,
BN), tl.arange(0, BK)
    a_ptrs = a_ptr + rm[:, None] * sa_m + rk[None, :] * sa_k  # A 行带里的第一个 tile
    b_ptrs = b_ptr + rk[:, None] * sb_k + rn[None, :] * sb_n  # B 列带里的第一个 tile
    acc = tl.zeros((BM, BN), dtype=tl.float32)            # 部分和，留在片上
    for _ in range(0, K, BK):                                # 沿 K 逐 tile 推进
        acc += tl.dot(tl.load(a_ptrs), tl.load(b_ptrs))      # 两个小块在片上相乘
        a_ptrs += BK * sa_k; b_ptrs += BK * sb_k
    acc = tl.maximum(acc, 0.0)                                # 融合 ReLU
    tl.store(c_ptr + rm[:, None] * sc_m + rn[None, :] * sc_n, acc)
```

分块 matmul 的形状（为了短，省掉了边界 mask）：二维 grid、每个 block 一个输出 tile、沿 K 的循环里 load 两块、dot、累加，写回前融合激活。

10. 收尾与问答：Triton 之外还有什么

- **总结：**程序员能控制的是编程模型这一层——PyTorch、Triton，乃至 PTX（想怎么特化都行）；但代码最终跑在硬件上，SM、bank、内存、寄存器都是有限的，你带着大矩阵和 Transformer 进来，得塞进这些约束里——所以 benchmark 和 profile 是理解“硬件的脏如何变成性能”的唯一办法。Triton 是按 block 思考的好语言：不必显式同步线程、不必手管 shared memory；把计算拆成 block，每个 block 读、算、写。四个例子难度递增：逐元素 → 一行归约 → 一行装不下的“婴儿版 tiling” → matmul 的正经 tiling。单 GPU 到此为止，下一讲多 GPU（第 7 讲）。

- **问答：Triton 之外的选择、能离最优多近？** 每种语言都有自己的归纳偏置，某些事容易、某些事难；Triton 是训 Transformer 的人做的，Transformer 相关的事在它上面都相对容易。极端情况可以直接写 PTX，但别拿它当第一步。还有一批 DSL——ThunderKittens、CuTe 等——它们不是简单地在栈上更高或更低，只是各有各的取舍。
- **问答：高维张量是整块装进去算，还是逐元素处理再写回？** 抽象地答不了，取决于计算的性质，课后再聊。

小注：ThunderKittens 是 Stanford Hazy Research 的 kernel DSL (Spector et al., 2024)；字幕里的"cute"应指 NVIDIA CUTLASS 3 里的 CuTe 布局库（推断）。至于标题里的 XLA：它是 Google 的线性代数编译器，JAX 与 TPU 走的都是"先把整段程序 trace 成图、再整体编译"的路线，本讲没有展开；相近的思想在本讲只以 torch.compile 露了面。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
GPU 结构简图	SM 数 100–200、每 SM 寄存器 256 KB 几代没变，涨的是 HBM；带宽与容量反着排	0:35	NVIDIA 规格
occupancy 算例	128 线程 × 160 寄存器 → 3 个 block → 12 个 warp → 12/64 ≈ 18%	12:53	—
bank conflict 图	32 个 bank 各 4 字节；32 个线程同读一列 = 32 路冲突	14:57	CUDA C Programming Guide
coalescing 图	沿行读，整条 cache line 全用上；沿列读，搬回一堆没用的	17:28	同上
CUDA event 计时代码	record、算、record、synchronize 的顺序；多次取平均	24:39	—
matmul 耗时曲线	维度到约 2000 之前几乎水平，之后才按立方涨	25:43	—
matmul 的 profiler 输出	kernel 名里读出 cutlass、sm100、f32、tile 64·64·16；128×128 换成 32·32·16	28:17	CUTLASS
GELU 三版 benchmark	naive 3.75 对内置与编译版；答案一样、时间不一样	31:56	—
naive GELU 的 profiler	一串 kernel：binary functor、unary、add、tanh；每个原语一个	32:59	—
compiled GELU 的 profiler	只剩一个 kernel，名字表明是 Triton 生成的	34:30	torch.compile

图	看什么	跳转	出处
第一个 Triton kernel	pid → start → offsets → mask → load → 算 → store	42:42	Triton
PTX 片段	ld.global 在头、st.global 在尾；同一段重复 8 遍 = thread coarsening	50:58	PTX ISA
tile 累加动画	每个线程一个累加器，逐 tile 加，最后 sum	1:06:54	—
分块 matmul 图	一个 C tile 一个 block；A 的行带与 B 的列带逐块装进 shared memory	1:16:39	—

提到的工作

名称	在本讲里的作用
NVIDIA A100 / H100 / B200	三代 GPU：SM 数与寄存器几乎不变、HBM 在涨；B200 有 thread block cluster 与 tensor memory；148 个 SM、8 TB/s 的数字说的是 B200
AMD GPU · Google TPU	一句带过：本讲只谈 NVIDIA
CUDA	NVIDIA 的 kernel 语言，"每个线程做什么"的心智模型；内置 GELU kernel 用它写
Triton (OpenAI)	本讲主角："每个 block 做什么"；torch.compile 生成的也是它
PTX	NVIDIA GPU 的中间汇编，Triton 编译器的输出；有人手写
CUTLASS	NVIDIA 的 CUDA 线性代数库；PyTorch matmul 底下调的 kernel 名里就有它
PyTorch profiler · Nsight	看每行代码调了哪个 kernel；作业里用 Nsight 看更细的
torch.compile	对任意 PyTorch 函数做图编译，把逐元素链融合成一个 Triton kernel
GELU (Hendrycks & Gimpel, 2016)	三匹马的赛道：naive、内置、编译
softmax · row sum · matmul + ReLU	四个 Triton 例子的后三个：行归约、分 tile 归约、分块矩阵乘并融合激活
FlashAttention (Dao et al., 2022)	作业目标；本讲的 tiling 与融合是它的全部原料（CME295 第 4 讲讲过大意）
swizzling	避免 bank conflict 的 shared memory 布局技巧，只点名
ThunderKittens · CuTe 等 DSL	问答里 Triton 之外的选择（CuTe 为推断）

术语对照

English	中文
kernel	核函数：在 GPU 上跑的一段程序，一次 launch 一个 grid
SM (streaming multiprocessor)	流式多处理器：GPU 里真正执行的单元，每张卡 100—200 个
HBM (high bandwidth memory)	高带宽显存：最大、最慢、全局可见
shared memory / L1 cache	片上共享内存 / 一级缓存：同一块物理内存，前者你管、后者硬件管
L2 cache	二级缓存：整块芯片共用
register	寄存器：线程私有，最快
thread / thread block (CTA) / grid	线程 / 线程块 / 网格：编程模型的三层
thread block cluster · tensor memory	线程块簇（H100 起）· 张量内存（B200）：新硬件多出来的两层，本讲不用
warp	线程束：32 个线程一组，锁步执行
lockstep	锁步：同一周期执行同一条指令
control divergence	控制分歧：warp 内线程要走不同分支，被串行化
warp scheduler	warp 调度器：在驻留的 warp 间零开销切换
latency hiding	延迟隐藏：一个 warp 等内存时换别的 warp 算
occupancy	占用率：实际驻留的 warp（或 block）占上限的比例
thread coarsening	线程加粗：让一个线程处理多个元素
bank / bank conflict	存储体 / 存储体冲突：shared memory 的 32 个 4 字节宽分区，同周期撞上就排队
swizzling	错位布局：重排 shared memory 里的数据以避免 bank conflict
memory coalescing	访存合并：一个 warp 的 HBM 访问合并成一条 cache line
cache line	缓存行：一次搬运的单位，32 到 128 字节
tail effect / wave	尾波：最后一波 block 填不满所有 SM
benchmarking / profiling	基准计时（端到端多久） / 性能剖析（时间花在哪、调了什么）
warm-up	预热：先跑几次，排除懒编译等一次性开销
CUDA event · synchronize	CUDA 事件计时 · 同步：GPU 异步，等它跑完再读表
CUTLASS	NVIDIA 的 CUDA 线性代数模板库

English	中文
kernel fusion	核融合：把多个算子合成一个 kernel，中间结果不经 HBM
torch.compile	PyTorch 的图编译入口：生成融合的 Triton kernel
pointer / pointer arithmetic	指针 / 指针算术：Triton 里张量参数是地址，加偏移得到要读的位置
program_id (pid)	程序号：Triton 里当前 block 的编号
offsets · mask	偏移 · 掩码：本 block 负责的下标；挡掉最后一块的越界部分
load / store	读 / 写：HBM 与片上之间的显式搬运
stride	步长：多维下标换成线性地址时每一维的乘数
PTX	NVIDIA GPU 的中间汇编语言
ld.global / st.global	PTX 里从全局内存读 / 写
ctaid.x / tid.x	PTX 里的 block 号 / 线程号
reduction	归约：把一行（或一块）合成一个数，如求和、求最大
tile / tiling	块 / 分块：一次装进 shared memory 处理的那一小块；分块计算
accumulator	累加器：逐 tile 累加的部分和
arithmetic intensity	算术强度：运算次数除以搬运字节数（第 2 讲）
memory-bound / compute-bound	受访存限制 / 受计算限制
DSL (domain-specific language)	领域专用语言：ThunderKittens、CuTe 这类写 kernel 的小语言
XLA	Google 的线性代数编译器（JAX / TPU 路线）：标题有、本讲未讲

字幕勘误

"M100s" → A100s；"Blackwall" → Blackwell；"Tatsuo" → Tatsu；"concurrent thread arrays" → cooperative thread arrays (CTA)；"20 to 128 bytes" → 32 to 128 bytes (cache line)；"site" → Nsight（作业用的 profiler）；"teaching you try and" → teaching you Triton；"value example / Triton value kernel" → GELU example / Triton GELU kernel；"A range" → arange；"TF.load" → tl.load；"color F32" → cutlass ... f32（kernel 名）；"mammal" → matmul；"cute" → CuTe（推断）。

带走的问题

1. 用第 2 节的算法给你自己的 kernel 记一次账：block 256 线程、每线程 96 个寄存器，在 65,536 个寄存器、64 个 warp 上限的 SM 上 occupancy 是多少？把每线程寄存器压到 64 会怎样？occupancy 翻倍一定更快吗——什么情况下 thread coarsening 反而赢？
2. naive softmax 的 $5 \cdot MN$ 次读、 $3 \cdot MN$ 次写是怎么数出来的（哪个算子读什么、写什么）？torch.compile 之后理论上最少要几次？如果一行装不进一个 block，Triton 版 softmax 要扫几遍 tile——求 max、求 sum、写出各一趟——能不能像 FlashAttention 那样合成一趟（online softmax，CME295 第 4 讲）？
3. 分块 matmul 的算术强度是 $O(\text{tile 边长})$ ：tile 取多大受什么限制（shared memory 容量、寄存器、bank conflict、SM 数与尾波）？为什么 PyTorch 给 128×128 的矩阵换了一个 $32 \cdot 32 \cdot 16$ 的 kernel，而不是沿用 $64 \cdot 64 \cdot 16$ ？
4. "编译版 GELU 是一个 Triton kernel，却比手写 CUDA 的内置版慢"——可能的原因有哪些（thread coarsening 程度、向量化宽度、tanh 的实现、launch 配置）？你会用 profiler 里的哪些指标去证实或排除？
5. Triton 让你按 block 思考、不碰 warp 和 shared memory；FlashAttention 的作业正好卡在"一行装不进一个 block"的情形。把第 8 节的 row sum 循环推广到 attention：沿 K、V 的 tile 前进时，除了累加向量还要维护什么（running max、归一化常数），写回前要做什么修正？

CS336 第 7 讲 | 并行（上）(Parallelism I)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 7 讲, 2026 年 4 月 20 日; 系统部分的第三讲 (第 5 讲 GPU / TPU → 第 6 讲 kernel → 第 7–8 讲多卡并行) 视频: <https://www.youtube.com/watch?v=SzpOcwdlL0Y> (1:21:02; 英文字幕是人工校对的 CC 轨, 比自动字幕干净得多, 只剩少数缩写和听不清的地方, 见文末勘误) 讲者: Percy Liang (推断: 讲义是可执行的 Python、逐行"走"代码的风格, 两次预告"Tatsu 周三讲 FSDP / ZeRO 和流水线的细节", 与课程表一致) 课程主页: <https://stanford-cs336.github.io/>; 本讲不围绕某一篇文章, 围绕的是 PyTorch 的 `torch.distributed` 和 NVIDIA 的 NCCL; 三种并行方法的原始出处讲者没有点名, 按推断标在"提到的工作"表里

一句话: 多卡训练和单卡优化是同一道题——计算单元离数据远, 游戏是编排计算、少搬数据——只是"远"从 HBM 变成了别的 GPU (节点内 NVLink 5 的 1.8 TB/s 约是 B200 HBM 8 TB/s 的四分之一, 跨节点的 InfiniBand 和以太网再低几级); 所有搬运都用 1980 年代就有的集合通信原语表达, 训练真正用到的是 all-gather、reduce-scatter 和二者拼成的 all-reduce (课上测得有效带宽约 400 GB/s, 且与卡数、拓扑无关), 在 `torch.distributed` 里各是一行调用; 拿一个多层 MLP 做例子, 数据并行只是在 backward 之后加一行梯度 all-reduce、代价是每张卡都得放下整套参数, 张量并行切矩阵的列、每层都要 all-gather 激活所以只能在 NVLink 域内做, 流水线并行切层、靠微批填气泡但能忍慢网络——放不下整套参数怎么办 (FSDP / ZeRO) 和气泡怎么调度, 留给第 8 讲。

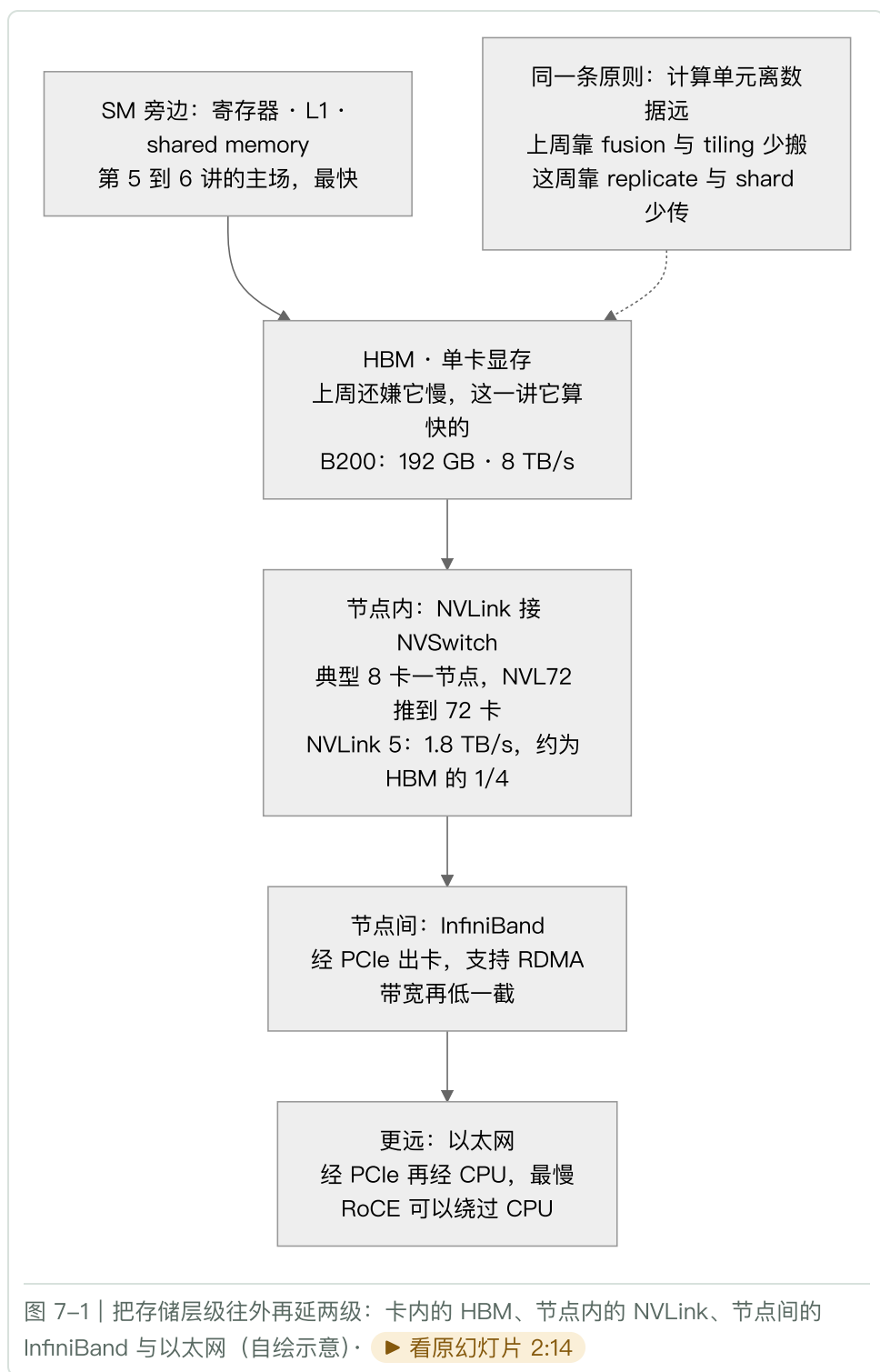
时间轴

时间	内容
0:05	开场: 从单卡 kernel 到多卡; 计算离数据"更远了一层"; 广义的存储层级
3:17	为什么要多卡: 放不下 (B200 192 GB 对 1T 参数) 与想更快; 讲义是可执行 Python, 课堂走单进程模式
4:57	本讲两部分: 集合通信、硬件、 <code>torch.distributed</code> ; 三种并行在 MLP 上实现
5:28	集合通信: 1980 年代就有的原语; rank 与 world size; 八个操作点名
8:04	热身四件: broadcast (初始化用一次)、scatter、gather、reduce
12:12	主角三件: all-gather、reduce-scatter、all-reduce = reduce-scatter + all-gather
16:31	all-to-all: MoE 的动态路由; 均衡时就是转置; 记忆口诀; 问答

时间	内容
21:53	硬件：PCIe 加以太网的经典拓扑 vs NVLink / NVSwitch / InfiniBand；NVLink 5 1.8 TB/s 对 HBM 8 TB/s
26:03	绕过 CPU：RDMA；NVL72 把 72 卡放进一个 NVLink 域；RoCE 与 Meta
29:43	NCCL 干什么；问答：rack 与 tray、RDMA 与 InfiniBand 的关系、第 9 张卡、TPU
35:57	torch.distributed：nccl / gloo 后端；spawn、rank、init_process_group、barrier
40:05	演示：all_reduce 就地求和；reduce_scatter_tensor；async 与通信计算重叠；all_gather_into_tensor"举例证明"
46:30	基准：1 亿元素的 all-reduce 1.6 ms；有效带宽的算法与约 400 GB/s；reduce-scatter 没有那个 2；问答 synchronize 与 barrier
55:09	第二部分：多层 MLP 上的三种切法；数据并行 DDP：batch 128 切 4 份、梯度 all-reduce 取平均
1:02:35	张量并行：切每层的列，每层 all-gather 激活；backward 对偶地 reduce-scatter；autograd 不会替你做
1:09:25	流水线并行：切层、微批、send / recv；气泡与通信计算重叠
1:14:42	没讲的：DP 里也能重叠；序列并行、专家并行、组合；选法看硬件：TP 不出 NVLink 域，PP 能忍慢网
1:17:52	收尾：原语 vs 编译器（TPU / JAX）；总结；重算、存本地、存别的卡；下周三 Tatsu

核心内容

1. 从一张卡到一千张卡：问题没变，只是数据"更远了一层"



- **上周和这周:** 第 5—6 讲盯着一张 GPU 的内部——HBM、L2、L1、寄存器和一堆 SM, 讲怎么写 kernel 让单卡跑快。这一讲把图往外扩: 不是一张卡, 而是 4 张、1,000 张连在一起的卡, 要把这些算力全用来训练模型。讲者的判断: 用一大堆 GPU 很容易, 用好它们很难。
- **同一条原则:** 放大了看, 两种情形是一回事——计算单元 (ALU、Tensor Core) 离数据远。单卡时“远”是指 HBM; 多卡时你要的那块

数据可能在另一张 GPU 上，得想办法搬过来。所以做法也同源：上周靠 fusion 和 tiling 减少对 HBM 的访问（读进 shared memory，能算的都算完再写回）；这周靠恰当地 replicate（复制）和 shard（分片）减少卡与卡之间的通信。

- **广义的存储层级**（图 7-1）：单卡内 L1 / shared memory 最快；HBM 上周还被嫌慢，这一讲反而算快的；再往外是节点内多卡，靠 NVLink 和 NVSwitch 连；再往外是多节点，只能靠 InfiniBand 或以太网，看你有什么网络。越往外越慢，和内存层级完全类似。
- **为什么要多卡**：表面答案是"要 scale"，讲者拆成两条：
 1. 放不下：参数、激活、梯度、优化器状态加起来超过单卡 HBM。B200 是 192 GB，训一个 1 万亿参数的模型，单卡无论如何放不下。
 2. 想更快：就算放得下，也可以把东西切开分给更多卡，训练更快。这里有取舍——用少几张卡放得下但算力少，摊开就要付通信带宽的代价，怎么并行是要算一笔账的。

小注：1 万亿参数按 bf16 存权重就是 2 TB，是一张 B200 的十倍以上；按第 2 讲的记账再加上梯度和 Adam 状态，还要翻几倍。

- **讲义的形式**：和前几讲一样是可执行的 Python，但这一讲真跑起来是多进程的；课堂上逐行走的是一个"单进程模式"，所以看到的 rank 永远是 0；多进程的真实输出讲者另附了链接。作业 2 要在真机上用 torch.distributed 把这些写一遍。
- **本讲两部分**：第一部分是分布式计算的积木——集合通信的编程模型、硬件怎么连、在 PyTorch 里怎么调；第二部分在一个多层 MLP 上实现三种并行：数据并行、张量并行、流水线并行。用 MLP 而不用整个 Transformer，是因为 MLP 本来就是 Transformer 里算力的大头，核心计算已经都在里面了。

2. 集合通信原语：八个操作，三个主角

- **什么叫 collective**：分布式编程里的一组原语，1980 年代就有，不是为 LLM 发明的，今天用的还是这一套。"集合"的意思是你指定一种跨所有设备的通信模板（"把这些加起来发给每个人"），而不是逐对管理"这张卡发给那张卡"——写起来容易得多，系统也能替你做更多事。

小注：这套原语后来在 MPI（Message Passing Interface，1994 年定稿）里标准化，名字沿用至今；NCCL 的接口就是照着 MPI 的集合操作起的名。

- **术语：**rank 指一个设备（这门课里就是一张 GPU，也可以是 TPU），编号 0 到 $W-1$ ；world size W 是设备总数。课上的例子 $W = 4$ 。
- 八个操作按用途分三档：

操作	之前	之后	训练里出现在哪
broadcast	rank 0 持有 0 1 2 3	每个 rank 都持有 0 1 2 3	初始化：加载 checkpoint 后广播给所有 rank，只做一次
scatter	rank 0 持有 0 1 2 3	rank k 拿到第 k 个分量	不直接用，是理解 reduce-scatter 的台阶
gather	rank k 持有第 k 段	rank 0 把各段拼接起来	理解 all-gather 的台阶
reduce	rank k 持有一个数	rank 0 得到 $0+1+2+3 = 6$	理解 all-reduce 的台阶；gather 可以看成"归约运算是拼接"的 reduce
all-gather	rank k 持有第 k 段	每个 rank 都拼出完整向量	分片的参数在 forward 前收齐（第 8 讲 FSDP）
reduce-scatter	rank k 持有向量 k、k+1、k+2、k+3	第 j 个分量跨 rank 求和后放到 rank j：6、10、14、18 各在一处	backward 之后把各 rank 的梯度加起来再分回去
all-reduce	同上	每个 rank 都得到 6 10 14 18	数据并行：梯度求和后复制到所有 rank
all-to-all	rank i 的第 j 段	送到 rank j；均衡时相当于把矩阵转置	MoE 的动态路由：token 送到持有对应 expert 的 rank

- **三档的意思：**前四个（broadcast、scatter、gather、reduce）是热身，用来建立感觉，训练的主路径上几乎不出现；all-gather、reduce-scatter、all-reduce 会在分布式训练里反复出现；all-to-all 是 MoE 专用（第 3 节末）。
- **reduce 的定义：**函数式编程里的那个 reduce——对各 rank 上的数据施加一个可结合、可交换的运算（sum、max、min 都行），结果落在指定的 rank 上。
- **记忆口诀**（讲者的）：reduce 就是归约；scatter 分发、gather 集中，互为逆运算；前面加 all 表示目的地是所有设备——all-reduce、all-gather 由此得名。
- **问答**
 - 和 NumPy 的 broadcasting 有关系吗？概念上一样（一个东西复制给多个），但实例完全不同，这里说的是设备间通信。
 - gather / reduce 的目标非得是 rank 0 吗（20:23）？不是，调用时指定哪个 rank 就落到哪；不必提前定死，但执行那一刻必须确

定。

- 这些只是概念还是真有代码？马上就看代码（第 5 节）。

3. all-reduce = reduce-scatter + all-gather：为什么要拆开看

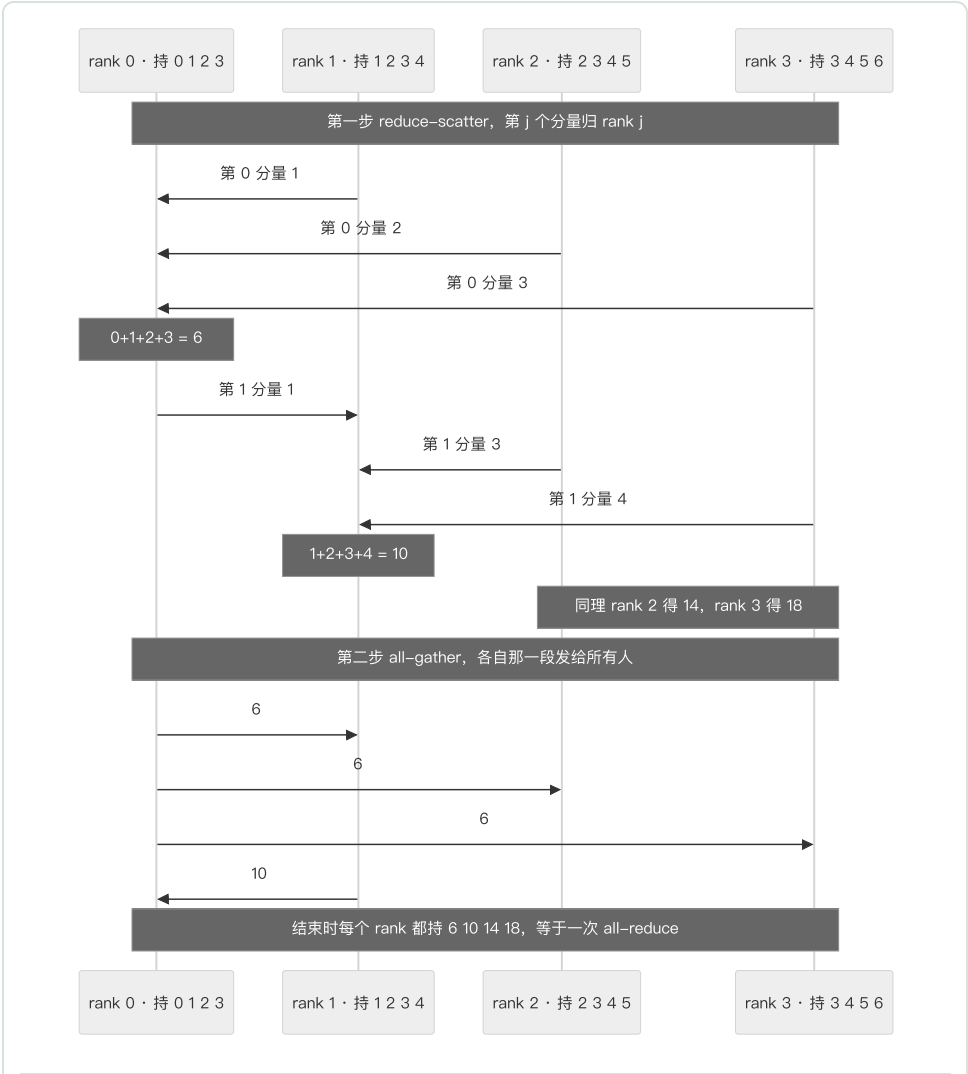


图 7-2 | 课上的四 rank 例子：先 reduce-scatter 把第 j 个分量的和放到 rank j，再 all-gather 让人人都有整份；两步连做就是 all-reduce（自绘示意）·

▶ 看原幻灯片 15:25

- **reduce-scatter 做什么**：每个 rank 手里都是一个完整长度的向量；对第 j 个分量跨 rank 归约，结果只放到 rank j。课上的例子：四个 rank 分别持有 0 1 2 3、1 2 3 4、2 3 4 5、3 4 5 6，第 0 个分量加起来是 6 放到 rank 0，第 1 个分量 10 放到 rank 1，依此 14、18。训练里的对应：backward 之后各 rank 算出的是不同数据上的梯度，要加起来，再把结果分回去各管一段。
- **all-gather 做什么**：gather 到所有 rank——每个 rank 把自己那段发给所有人，每个人都拼出完整向量。训练里的对应：参数分片在各

rank 上，forward 前要收齐整份参数。

- **all-reduce 就是两步连做**：先 reduce-scatter 得到 6、10、14、18 各在一处，再 all-gather 把它们复制到每个 rank。讲者说 all-reduce 反而最好懂——一堆张量，求和，复制到所有节点；数据并行第一个用的就是它。
- **为什么还要拆**：all-reduce 是一个整块的操作，简单，但它要求每个 rank 都放得下整份结果。往后要做 ZeRO / FSDP 这类"参数放不下"的方案，就得把它拆成两半，在两半之间插手——只 all-gather 当前层需要的参数、只 reduce-scatter 自己负责那一片的梯度。这是第 8 讲的内容，本讲先把两半各自练熟。CME295 第 4 讲把 ZeRO 的三级分片一页带过，这里给出了它赖以成立的通信原语。
- **all-to-all**：最一般的一种——每个 rank 给每个 rank 发一条消息。课上的例子：rank i 的第 j 段要送到 rank j，rank 0 把 0、1、2、3 分别送到 rank 0、1、2、3，rank 1 把 4、5、6、7 分别送到 rank 0、1、2、3.....结果是 rank j 收到所有人发来的"第 j 列"。若把各 rank 的数据排成矩阵，均衡情形下 all-to-all 就是转置；它也支持不均衡的划分，可以配置给任何 rank 发任意字节，但通常希望越均衡越好——第 4 讲 MoE 的负载均衡就是为此。MoE 里每个 rank 既持有一部分数据也持有一部分 expert，路由是动态的，得看了数据才知道哪些激活该送到哪个 expert，所以天然是 all-to-all。本讲不再展开。

4. 硬件：节点内 NVLink，节点间 InfiniBand，再远就是以太网

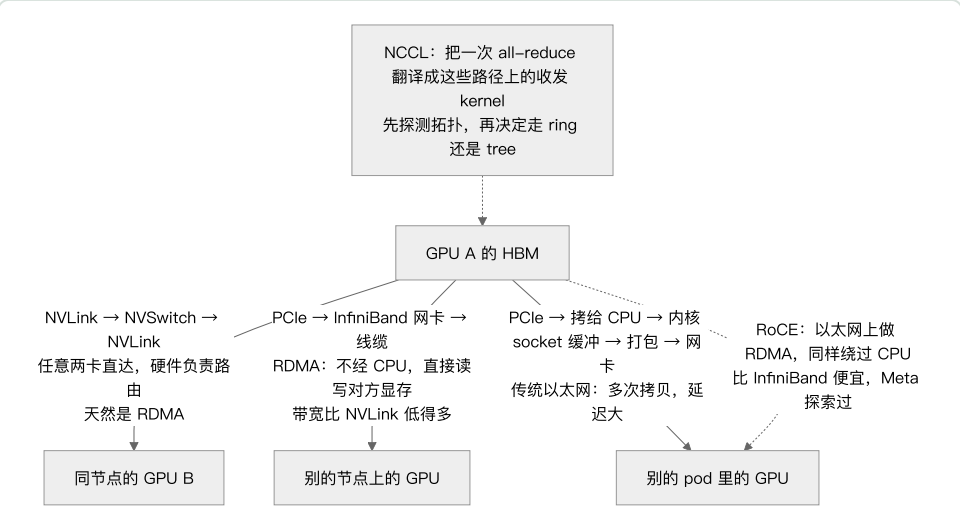


图 7-3 | 同一份数据要去三种远近的 GPU，各走什么路；RDMA 的意义是不让 CPU 插手（自绘示意）。 ▶ 看原幻灯片 23:25

- **经典的图：**一台服务器里有几颗 CPU，一条 PCIe 总线（过去接鼠标键盘的那种），GPU 挂上面，还有内存；服务器之间用以太网相连。同节点的 GPU 通过 PCIe 通信，跨节点得走以太网。讲者的比喻：买了张游戏卡，和朋友的卡连起来"训个大模型"，就是这个拓扑。
- **认真训练的样子：**每节点 8 张 GPU（8 是典型值；幻灯片上的 256 个节点是编的），通过 NVIDIA 的 NVLink 接到 NVSwitch 上。校准一下数量级：NVLink 5 的总带宽是 1.8 TB/s；B200 的 HBM 是 8 TB/s，所以卡间比卡内慢约 4 倍——对设备之间的通信来说已经很快了，但仍比 HBM 慢，而 HBM 又比 shared memory / L1 慢得多。从编程的角度，接在同一个 NVSwitch 上的 GPU 可以看成任意两两直连，硬件负责把数据送到交换机、交换机负责路由。
- **再往外：**GPU 数量一多，NVSwitch 就罩不住了（不可能有一个 NVSwitch 管 10 万张卡），节点被组成 pod，用 InfiniBand 相连——此时 GPU 不再直连 GPU，要经 PCIe 出卡、走专门的 InfiniBand 线缆，速度低得多；InfiniBand 也到头时，pod 之间只剩以太网，要经 PCIe 再经 CPU，更慢。节点越多越慢，和内存层级一个道理。
- **RDMA：别让 CPU 插手。**传统以太网上，GPU 要发数据得先拷给 CPU，进操作系统内核的 socket 缓冲区（这里的 kernel 是 OS 内核，不是 GPU kernel），打成网络包，再拷到网卡发出去，延迟很大。RDMA（remote direct memory access）让一张 GPU 直接读写另一张 GPU 的显存，CPU 完全不参与。NVLink / NVSwitch 天然如此，InfiniBand 也支持 RDMA，标准以太网不支持。
- **两个新进展**
 - NVL72：NVIDIA 把 B200 / B300 做成机架级产品，72 张 GPU 全部接进同一个 NVLink 域。凡人的世界是"8 张卡之间飞快，出了这 8 张就慢一大截"；肯花钱，这个边界可以推到 72 张。
 - RoCE（RDMA over Converged Ethernet）：以太网上也做 RDMA、绕过 CPU，算是对 InfiniBand（和 NVIDIA 的多数产品一样很贵）的回应。Meta 有论文探索过，讲者说他们的模型"可能就是在 RoCE 上训的"。

小注：NVIDIA 的资料里 GB200 NVL72 是 18 个 compute tray（每个 2 颗 Grace CPU 加 4 颗 Blackwell GPU，每颗 Grace 带 2 颗 GPU）加 9 个 NVLink switch tray，合计 36 颗 CPU、72 张 GPU；讲者问答里说的"每 tray 8 卡、9 个 tray"应是把 switch tray 的个数和 GPU 总数对上。Meta 那篇应为 SIGCOMM 2024 的《RDMA over Ethernet for Distributed AI Training at Meta Scale》；Llama 3 论文自己写明 405B

是在 RoCE 集群上训的，小模型用的是 InfiniBand，两种网络都是 400 Gbps 的链路。小注：对照上一代：H100 的 NVLink 4 每卡 900 GB/s、HBM3 3.35 TB/s（同样约 4 倍）；InfiniBand NDR 每端口 400 Gb/s $\approx$ 50 GB/s，比 NVLink 低一个数量级以上——这就是“张量并行不出 NVLink 域”（第 11 节）的数字依据。

- **NCCL**（念作 nickel）：NVIDIA Collective Communications Library，最底层。你说“我要 all-reduce”，它去探测硬件拓扑、找出 GPU 之间的路径，然后启动 GPU kernel 去收发数据——别忘了 GPU 上跑的一切都是 kernel，通信也有通信 kernel。课上不再深入，知道它在就行。
- **问答**
 - rack 和 tray 是什么？rack 就是数据中心里的机架；tray 是一层托盘，G 指 Grace CPU，两颗 CPU 各接若干 GPU，整层再接进 NVSwitch（讲者自称不是硬件专家，准确数字见上面的小注）。
 - RDMA 和 InfiniBand 什么关系？RDMA 是“想要的性质”——一张卡能直接读写另一张卡的显存；NVLink / NVSwitch、InfiniBand、RoCE 是实现它的三种硬件路径。
 - NCCL 对多节点优化过吗？细节讲者不清楚，但 NVIDIA 的大客户就是训大模型的公司，整个软件栈都在为这类负载优化，没优化才奇怪。
 - 9 张卡怎么办？看第 9 张落在哪：如果 8 卡一节点，第 9 张在另一个节点又没有 NVLink，那它算力不多、通信还贵，很糟；都在同一个 NVSwitch 域里就没什么问题。
 - 和 TPU 有什么不同？TPU 简单得多，细节课后聊（第 5 讲说过：芯片里像，差别在网络）。

5. 用 torch.distributed 把它写出来

- **接口**：PyTorch 的 torch.distributed 给集合通信提供了干净的接口，不必直接碰 NCCL；它支持多种后端——GPU 上用 nccl，CPU 上用 gloo（并行编程早于 GPU，这些操作在 CPU 上一样能做；讲者在笔记本上走代码用的就是 gloo）。库里也有 FSDP 这类高层封装，这门课不用——要从零造。

```
import torch, torch.distributed as dist
def worker(rank, world_size):
    backend = "nccl" if torch.cuda.is_available() else "gloo"
    dist.init_process_group(backend, rank=rank,
```

```

world_size=world_size) # master 地址只用来对表
    x = torch.arange(4, dtype=torch.float32) + rank # rank k 持
有 k, k+1, k+2, k+3
    piece = torch.empty(1)
    dist.reduce_scatter_tensor(piece, x, op=dist.ReduceOp.SUM) # 输
入不动, 输出是我负责的那一段
    full = torch.empty(4)
    dist.all_gather_into_tensor(full, piece) # 各段拼回整
份: 6 10 14 18
    dist.all_reduce(x, op=dist.ReduceOp.SUM) # 就地一步到
位, 结果与 full 相同
    dist.barrier(); dist.destroy_process_group()
torch.multiprocessing.spawn(worker, args=(4,), nprocs=4) # 同一个函数
复制 4 份, 各占一个进程

```

一次集合操作在代码里就是一行：先建进程组，再把张量和归约运算交给库，后端负责启动通信 kernel。

- **执行模型**：spawn 把同一个函数复制 W 份，各在一个进程里异步运行；函数拿到自己的 rank（0 到 W-1）和 W。讲者的 spawn 是个包装，课堂的单进程模式走的是“禁用分布式”的分支，正常情况调的是 torch.multiprocessing.spawn。
- **setup 里配的 master 地址和端口**不是数据通道，只用于元数据和协调；真正的数据走 NCCL，否则会慢得离谱。
- **barrier**：同步栅栏，所有进程都到这一行才继续。进程之间完全异步——一个可能先跑完，也可能任意交错——想保证某段代码先于另一段执行就放 barrier；代价是可能白等。讲者为了让打印顺序好看，放了比平时多得多的 barrier。
- **all_reduce**：传入张量和归约运算（sum），async_op = False；后端（gloo 或 nccl）启动 kernel、完成通信，结果就地写回——之后每个 rank 上都是各列之和。打印出来的顺序由硬件心情决定，因为各进程异步。
- **reduce_scatter_tensor**：不是就地，要分别给输入和输出张量；输入不动，输出得到“自己负责的那个分量”的归约结果。
- **async_op = True 是什么意思**（44:25 的提问）：这是一个整块的操作，启动 CUDA kernel 去通信；CUDA 本来就相对进程异步，现在进程之间又异步。设成异步后这行立刻返回，你可以先干别的——典型用法是通信和计算重叠：发起 all-reduce 后去加载下一步的数据，需

要确认完成时再 wait 或 barrier。本讲不展开，但它是第 8 讲和作业里的关键。

- **all_gather_into_tensor**: 把 reduce-scatter 的输出当输入，再分配一个输出，之后每个 rank 都拿到所有分片——讲者管这叫"举例证明"all-reduce = reduce-scatter + all-gather。最后 destroy_process_group 清理，是好习惯。
- **问答**: rank 就是 GPU 吗？这门课里是。

6. 通信有多快：有效带宽怎么算

- **基准怎么做**: all-reduce 一个 1 亿元素的张量。和第 2 讲的基准一样先热身；计时前后各做一次 cuda.synchronize 加 barrier，因为这里有两种异步——CUDA kernel 和多个进程——要确保一切都停下、都做完了才开始计时。每个 rank 各报一个时间，想要一个数就取平均。
- **1.6 ms 是快是慢**: 光看时间没意义，要像第 2 讲算 MFU 那样算有效带宽：该发多少字节 ÷ 总时间。

$$\text{bytes}_{\text{sent}} = 2(W-1)S, \quad \text{BW}_{\text{eff}} = \frac{2(W-1)S}{Wt} \xrightarrow{W \rightarrow \infty} \frac{2S}{t}$$

S 是张量的字节数（元素大小 × 元素个数）， W 是 world size， t 是一次 all-reduce 的墙钟时间。 $W-1$ 来自"把 W 份加起来要做 $W-1$ 次加法"， 2 来自既要发出去归约又要结果送回来；分母乘 W 是因为所有 rank 都在等这段时间。代进课上的数：1 亿个元素（讲者没说 dtype，按默认的 float32 推断是 400 MB）， $W = 4$ ， $t = 1.6 \text{ ms}$ ，得到约 375 GB/s，即讲者说的"约 400 GB/s"。

- **两条性质**: W 一大， $(W-1)/W$ 趋近 1，有效带宽就是 $2S/t$ ——与卡数无关，卡越多带宽也不变，这是好事；它也与拓扑无关，NCCL 是走 ring 还是 tree 由它自己决定，公式不用管。
- **reduce-scatter 的账**: 同样的输入输出、热身、计时，公式里没有那个 2:

$$\text{BW}_{\text{eff}}^{\text{reduce-scatter}} = \frac{(W-1)S}{W t_{\text{rs}}}$$

t_{rs} 是一次 reduce-scatter 的时间。测出来带宽同样在 400 GB/s 上下（有随机波动）：all-reduce = reduce-scatter + all-gather，搬的数据是两倍，时间也是两倍，两者相消，带宽相同。

小注：这正是 NCCL 官方基准 nccl-tests 的"bus bandwidth"口径——算法带宽 S/t 乘以 $2(W-1)/W$ ——用来在不同卡数之间公平比较；ring all-reduce

里每张卡恰好收发 $2(W-1)/W \times S$ 字节，所以它也是每条链路的真实负载。讲者没说这台机器是什么卡；若是 H100，NVLink 单向 450 GB/s，测到 400 GB/s 上下是合理的。

- **问答：为什么要 `cuda.synchronize`？** (53:30) 说到底还是在做 CUDA 操作，只是多个进程各带一张卡；CUDA 默认异步，跑到下一行 Python 时上一个操作可能没做完，所以要同步。先 barrier 再 synchronize 行不行？讲者想了想：不行——先过 barrier 时各自的 kernel 可能还没跑完，之后各自同步各自的，等于没同步。

7. 第二部分：在一个多层 MLP 上看三种切法

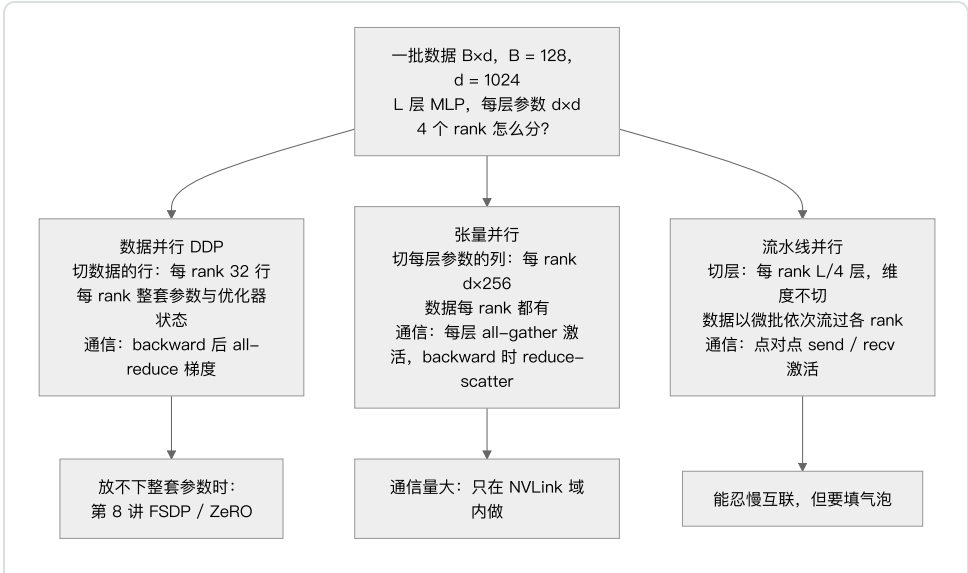


图 7-4 | 同一个 MLP、同一批数据，三种切法各自切什么、传什么（自绘示意）。
▶ 看原幻灯片 55:40

- **为什么用 MLP：**多层 MLP 是 Transformer 里真正的算力瓶颈，三种并行的核心计算在它身上都能看到；大模型只是多了很多簿记，反而看不清算法。
- **设定：**batch size 128、维度 1,024 的数据矩阵；L 层，每层一个 $1,024 \times 1,024$ 的参数矩阵，随机初始化，交给优化器； $W = 4$ 。
- **三种切法** (图 7-4)：数据并行切数据的行，每张卡整套参数、只看一部分数据，然后同步；张量并行不切数据、切每一层，每张卡拿到每层的一部分，通信量大得多；流水线并行按层切，每张卡拿到几层的全部维度，数据依次流过。讲者提醒这是概念图，别抠细节。CME295 第 4 讲的 DP / tensor / pipeline 三分法就是这张图，那边只说了“切什么”，这里把“传什么、传多少”补上。

8. 数据并行 DDP：标准训练加一行

```

local = data[rank * B // W : (rank + 1) * B // W]      # 每个 rank 只
拿自己那 B/W 行
for step in range(num_steps):
    x = local
    for w in params:                                     # 每个 rank 都
持有整套参数
        x = torch.relu(x @ w)
        loss = x.square().mean()
        loss.backward()                                 # 到这里各 rank
的梯度还不一样
        for w in params:                                 # 唯一多出来的一
步
            dist.all_reduce(w.grad, op=dist.ReduceOp.AVG) # 梯度取平均, 各
rank 从此一致
    optimizer.step(); optimizer.zero_grad()

```

数据并行对训练循环的全部改动就是 backward 之后那个 all-reduce 循环，forward 长什么样它不关心。

- **切数据**：把数据矩阵的行分成 W 份，rank k 拿第 k 份——local batch size = $128 / 4 = 32$ 行，就是按下标切片再放到对应的卡上。实践中每个 rank 应该自己加载自己的数据，别经过一个瓶颈，这里只是演示。
- **训练循环**：forward 只在本地那 32 行上逐层算，backward 照常。到这里本来就该结束了——但每个 rank 数据不同，梯度也不同。让数据并行成立的关键一步：对每个参数的 .grad 做一次 all-reduce、取平均。之后每个 rank 的梯度完全一致，再各自更新参数。这是标准训练和 DDP 之间唯一的差别，一行代码，讲者说它"相当优雅"。

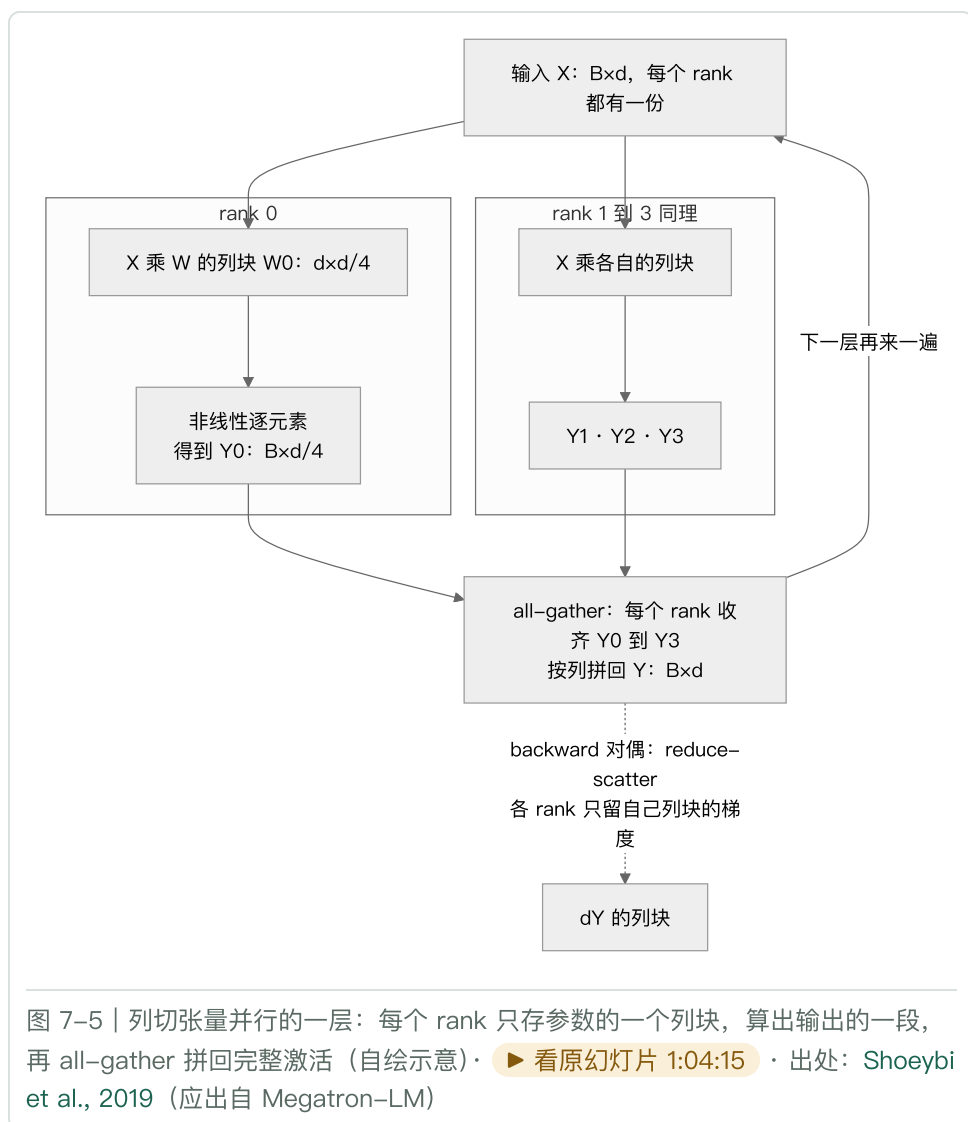
$$\bar{g} = \frac{1}{W} \sum_{k=1}^W g_k = \nabla_{\theta} \left(\frac{1}{W} \sum_{k=1}^W \mathcal{L}_k(\theta) \right)$$

g_k 是 rank k 在自己那 $1/W$ 的数据上算出的梯度， $\mathcal{L}_k$ 是它的局部 loss， θ 是所有 rank 都持有的同一份参数；平均梯度等于整个 batch 上 loss 的梯度，所以每个 rank 更新完仍然一致，效果如同它看过全部数据。

- **不变量**（讲者在 1:02:04 的总结）：各 rank 的 loss 不同，梯度起初不同，all-reduce 之后相同，因此参数始终相同。每个 rank 都像拿着全部数据在更新参数，实际只处理了 $1/W$ 。
- **问答**

- batch size 得大于 1 吧？至少要等于 W 才说得通，通常要大得多。
- 要是 W 的整数倍吗？最好是；不是也能补零之类，但整除对谁都省事。
- 换成 Transformer 呢？完全一样——DDP 很模块化，只在 backward 之后平均梯度，不关心 forward 长什么样。
- **代价与去向：**all-reduce 简单，但要求每张卡放得下全部参数（以及梯度和优化器状态——DDP 里每个 rank 都在做重复的更新，换来的是不必把优化器状态搬来搬去）。参数放不下时就得更聪明：这是第 8 讲 Tatsu 讲的 FSDP 和 ZeRO——用第 3 节拆开的两半，参数、梯度、优化器状态逐级分片。CME295 第 4 讲的 ZeRO-1 / 2 / 3 那一页，到那里会算清每级省多少字节。
- **能做但没做的优化：**这里是 backward 全部结束后一次性做所有 all-reduce；聪明的做法是某个参数的梯度一算出来就开始发送，让通信和后续的 backward 重叠——作业 2 会探索。

9. 张量并行：切矩阵的列，每层 all-gather 一次激活



- **切法:** 不切数据 (简化起见每个 rank 都有全部数据 $B \times d$), 切每一层的参数。 $\text{local_num_dim} = d / W = 256$; 每个 rank 上每层的参数是 $d \times 256$ ——沿列切开, 所以叫 column tensor parallel (按行切也行, 课上不讲)。

$$\sigma(XW) = \sigma(X [W_0 \ W_1 \ W_2 \ W_3]) = [\sigma(XW_0) \ \sigma(XW_1) \ \sigma(XW_2) \ \sigma(XW_3)]$$

X 是 $B \times d$ 的输入, W_k 是参数矩阵的第 k 个列块 ($d \times d/4$), σ 是逐元素的非线性。矩阵乘法可以按列块拆成几个小乘法各自做, 逐元素的 σ 与拼接可交换, 所以每个 rank 只用自己的列块就能算出输出的一段, 最后拼起来即可。

```
d_local = d // W # 每个 rank 只
存 W 的 d_local 列
x = data # 简化: 每个
rank 都有全部数据 B x d
for w_local in params: # w_local 形状
```

```

d × d_local
    y_local = torch.relu(x @ w_local)                # B ×
d_local, 非线性逐元素, 可先算
    pieces = [torch.empty_like(y_local) for _ in range(W)]
    dist.all_gather(pieces, y_local)                  # 收齐其余 rank
的列块
    x = torch.cat(pieces, dim=1)                      # 拼回 B × d,
交给下一层
# backward 与之对偶: 对拼接后的梯度做 reduce-scatter, 各 rank 只留自己列块的
梯度

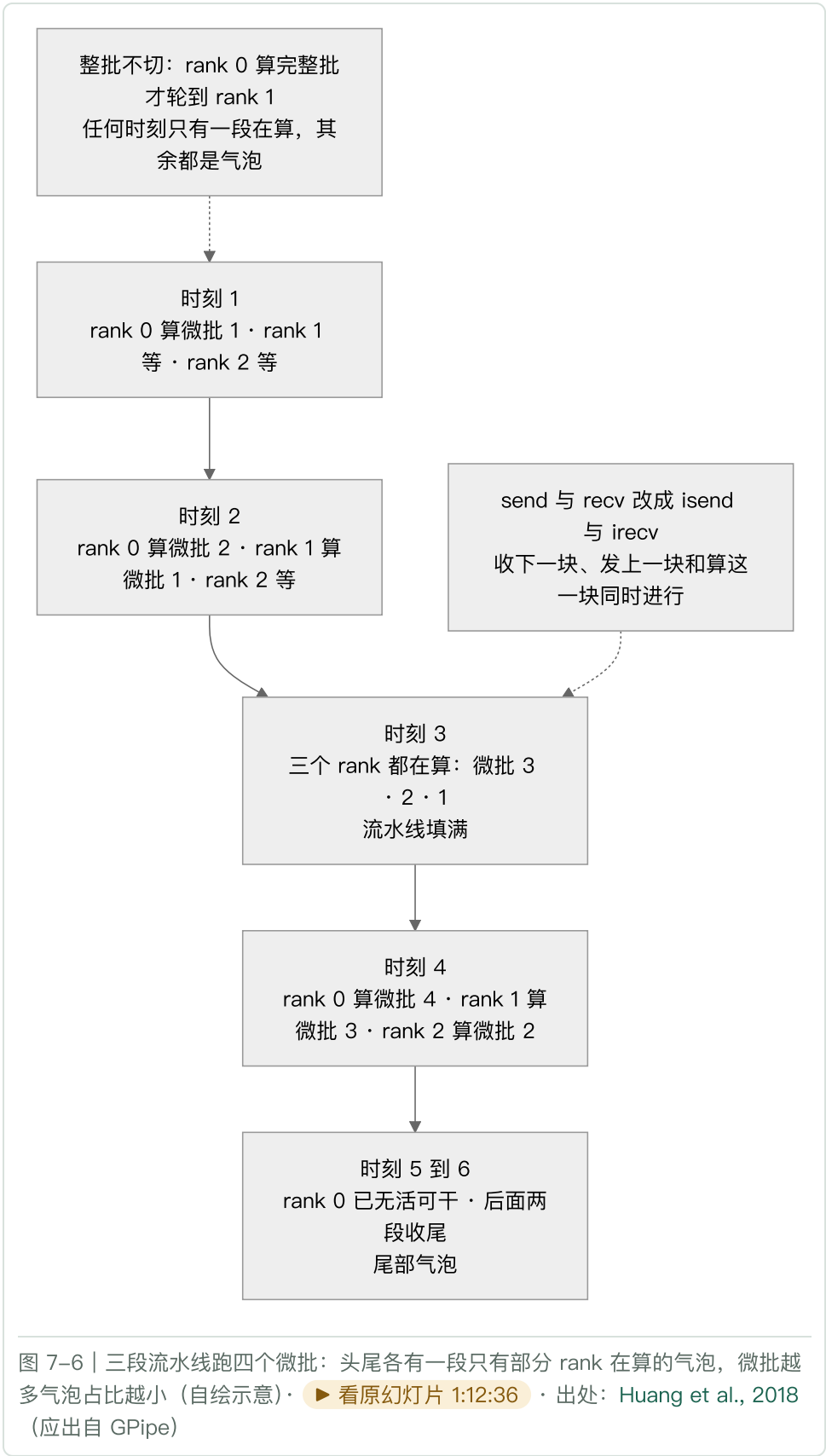
```

每一层都要经过"局部 matmul → 局部非线性 → all-gather → 拼接"四步, 通信嵌在模型内部。

- **forward**: 逐层: X 乘自己那一片参数得到 $B \times 256$ 的激活片, 非线性可以直接做 (逐元素); 但下一层需要完整的 $B \times d$ 输入, 各 rank 的激活片都得凑齐——这正是 all-gather: 每个 rank 分配 W 个 $B \times 256$ 的槽, all-gather 后每个人的对应槽里都有了各 rank 的片, 按列拼接得到 $B \times d$ 。每一层都要来一遍。
- **和数据并行的区别**: DDP 优雅在把模型当黑盒; 张量并行必须动模型内部, 它强烈依赖"一个大矩阵乘可以拆成几个小矩阵乘再合并"这一事实。
- **问答**
 - backward 怎么办? 对偶: forward 里 all-gather 激活, backward 里对梯度做 reduce-scatter, 各 rank 只留自己列块对应的那份。all-gather 与 reduce-scatter 的这种对偶会反复出现。
 - autograd 会自动做吗? 只调 .backward 不会, 它不知道有并行; PyTorch 有现成的封装替你做, 但这里是 CS336, 通信要自己管——设计如此。
- **通信量**: 每一层都要传一整份 $B \times d$ 的激活, 相当大, 所以张量并行只在高带宽的地方做——节点内、NVLink 域内; 出了 NVLink 域就别做 (第 11 节)。

小注: 课上的版本每层 all-gather 一次, 是为了教学; Megatron-LM (Shoeybi et al., 2019) 的做法是把 MLP 的两层配成"第一层按列切、第二层按行切", 中间的激活不必拼回, 整个 MLP 块 forward 只需一次 all-reduce (backward 再一次), attention 按 head 切也是同一思路。CME295 第 4 讲说"tensor parallelism 把一个大矩阵切两半", 切法就是这里的列切。

10. 流水线并行：切层，用微批填气泡



- **切法：**每个 rank 拿到一段连续的层 ($\text{local_num_layers} = L / W$)，每层完整的 $d \times d$ 、完整的维度；数据从 rank 0 进，算完自己的层送

给 rank 1，依次往后——一切深网络最自然的方式。

- **微批**：除了切层，还把 batch 切成若干 micro-batch。rank 0 拿到数据后分块；对每个微批：从前一个 rank 接收，在自己的层上做 forward，发给下一个 rank。这里用的是点对点的 send / recv（前面没讲，但一看就懂：recv 从 rank-1 收一个张量，send 把 x 发到 rank+1）。

```
my_layers = params[rank * L // W : (rank + 1) * L // W] # 这个 rank
只存 L/W 层
for mb in data.chunk(num_microbatches): # 微批：小块流水，少等一点
    if rank > 0:
        dist.recv(mb, src=rank - 1) # 点对点：从上一段接激活
    for w in my_layers:
        mb = torch.relu(mb @ w)
    if rank < W - 1:
        dist.send(mb, dst=rank + 1) # 送给下一段
# 换成 dist.isend / dist.irecv 并在需要时 .wait()，通信才能和计算重叠
```

流水线的一段就是“收、算自己的几层、发”三步，参数只有自己那几层的。

- **气泡**：rank 0 在算时后面的卡都在等，rank 1 开始算了 rank 0 又闲着——流水线里总有人在等别人的张量，这就是 pipeline bubble，相当浪费。微批的用处是把批切小，算完一小块立刻送走，让后面的卡早点开工、前面的卡少闲一会儿，气泡就少了。

$$\text{bubble fraction} \approx \frac{W - 1}{M + W - 1}$$

W 是流水线段数（rank 数），M 是微批数；M = 1（整批不切）时气泡占 (W-1)/W，M 越大越接近 0。

小注：这个比例出自 GPipe (Huang et al., 2018)，讲者没有写公式，只说微批能减少气泡；第 8 讲会讲 1F1B 这类调度怎么在不加大 M 的前提下压气泡和显存。

- **通信计算重叠**：这个朴素版本没做，但对流水线并行尤其重要——算这一块的同时就该在收下一块、发上一块。做法是把 send / recv 换成 isend / irecv（前面加个 i），变成异步，代码上要多管一些事。
- Tatsu 周三会细讲微批与调度。

11. 选哪种并行：由带宽决定，不由喜好决定

- **张量并行**：通信最多，每层都要发一整份激活，所以只在节点内、NVLink 上做，不会跨出 NVLink 域。
- **流水线并行**：能忍受慢得多的互联——一些去中心化训练的工作就用它，因为 GPU 分布在地球两端；那种场景绝不会用张量并行。
- **常见的组合**：节点内张量并行，然后数据并行或 FSDP，再需要时套一层流水线并行。作业里会让你在给定拓扑下做这种组合。
- **critical batch size**：数据并行看起来能一直加卡，但 batch 加到某个临界值之后再大就不再帮忙，算力就浪费了——这时候不如转去做张量并行。这些考量课程后面会陆续展开。
- **没讲的并行 (1:15:14)**：序列并行把整条序列切段，用来并行化 attention 的计算；专家并行把 MoE 的 expert 分到不同卡上，第 3 节的 all-to-all 就用在这里；以及各种组合。讲者认为 MLP 已经给出理解基础所需的大部分，大模型只是簿记更多。
- **原语 vs 编译器**：这门课故意用 PyTorch、而且是最原始的集合通信调用，为的是看清机制上发生了什么。另一条路（尤其在 TPU / JAX 的世界）是只定义模型和分片策略——"这块数据要在这里、这里和这里"——由编译器决定需要哪些通信操作。很诱人，但会把从零造东西的乐趣抽走一大半。
- **收尾的模式 (1:19:56)**：并行的切法有按数据、按张量或 expert、按流水线或序列；本讲只做了 DDP，下一讲 FSDP / ZeRO；张量并行要快互联，流水线并行不那么要，但要下功夫消气泡。更高一层看，同一个模式反复出现：一样东西可以重算，可以存在本地内存（第 5 讲重计算的取舍），现在多了第三个选项——存在别的 GPU 上。数据并行每个 rank 都在做重复的工作、都保存整套参数，理由就是不用把优化器状态搬来搬去。硬件会越来越快，但我们永远想要更大的模型，这套层级结构永远都在。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
单卡到多卡的总图	上周盯的那一个方框（HBM、L2、L1、SM）现在变成 4 个、1,000 个，之间画着连线	0:38	—
广义存储层级	三级：卡内 L1 / shared memory 与 HBM → 节点内 NVLink / NVSwitch → 跨节点 InfiniBand / 以太网	2:14	—

图	看什么	跳转	出处
四个热身操作	每张左边是"之前"各 rank 持有什么、右边是"之后"; reduce 那张把 0 1 2 3 加成 6	8:36	—
reduce-scatter 示意	第 j 个分量跨 rank 相加后落到 rank j: 6、10、14、18 各在一处	14:21	—
all-reduce 示意	同一个输入, 先 reduce-scatter 再 all-gather, 四个 rank 都得到 6 10 14 18	15:25	—
all-to-all 示意	位置决定目的地: rank j 收到所有人的"第 j 列", 看出来就是转置	17:36	—
NVLink / NVSwitch / InfiniBand 拓扑	8 卡一节点接 NVSwitch; 读 NVLink 5 的 1.8 TB/s, 对照 B200 HBM 的 8 TB/s	23:57	—
传统以太网 vs RDMA	左边数据经 CPU、内核 socket 缓冲、网卡多次拷贝; 右边 GPU 直写对方显存	26:35	—
all_reduce 演示的输出	before 每个 rank 一行不同的向量、打印顺序乱; after 全部相同	40:36	—
有效带宽的公式	分子 $2 \times (W-1) \times S$, 分母 $W \times \text{时间}$; 1 亿元素 1.6 ms 得约 400 GB/s	49:15	—
三种切法的示意图	同一块"数据 $\times$ 参数"的矩形: DP 横切数据、TP 竖切每层、PP 按层切	55:40	—
DDP 的关键一行	for 循环里对每个 param.grad 做 all_reduce 取平均, 其余和单卡训练一模一样	59:21	—
列切张量并行	参数矩阵沿列切成 W 块每 rank 一块; forward 后 all_gather 进 W 个槽再拼接	1:04:15	Megatron-LM (应出自)
流水线的 recv / forward / send	微批循环: 从 rank-1 收、算自己的层、发给 rank+1	1:11:32	GPipe (应出自)

提到的工作

名称	在本讲里的作用
集合通信原语 (1980 年代的并行编程)	全讲的语言: broadcast 到 all-to-all 八个操作; 小注: MPI 1994 年把它们标准化
NVLink / NVSwitch	节点内互联: NVLink 5 总带宽 1.8 TB/s; 接在同一交换机上的卡两两直达
GB200 NVL72	72 张 GPU 一个 NVLink 域的机架级产品
InfiniBand	节点间互联, 支持 RDMA, 贵
RDMA	GPU 直接读写对方显存、绕过 CPU 的性质; NVLink、InfiniBand、RoCE 三种实现
RoCE (RDMA over Converged Ethernet)	以太网上的 RDMA; Meta 探索过

名称	在本讲里的作用
Meta 的 RoCE 论文 · Llama 3	"Meta 有论文"应为 SIGCOMM 2024 那篇；讲者说"可能就是在 RoCE 上训的"模型应为 Llama 3（推断）
NCCL	把集合操作翻译成拓扑感知的收发 kernel
gloo	torch.distributed 的 CPU 后端；讲者笔记本上走代码用它
torch.distributed · torch.multiprocessing.spawn	本讲所有代码的接口：init_process_group、all_reduce、reduce_scatter_tensor、all_gather_into_tensor、send / recv、barrier
DDP (DistributedDataParallel)	数据并行的名字；本讲手写了它的核心
FSDP · ZeRO (Rajbhandari et al., 2019)	参数放不下时的数据并行，第 8 讲；靠拆开的 reduce-scatter 与 all-gather
Megatron-LM (Shoeybi et al., 2019)	列切 / 行切张量并行的出处（讲者未点名，应出自这里）
GPipe (Huang et al., 2018)	微批填气泡的流水线并行的出处（推断）
序列并行 · 专家并行	只点了名：切序列以并行 attention；分 expert，用 all-to-all
critical batch size（应为 McCandlish et al., 2018）	数据并行加卡的上限，超过就浪费算力
JAX / XLA 的编译器分片	"定义模型和分片策略，编译器决定通信"的另一条路
去中心化训练	流水线并行能跨慢网的例子，讲者没点具体工作
作业 2	在真机上写 torch.distributed；通信计算重叠；组合并行

术语对照

English	中文
rank / world size	设备编号 / 设备总数
collective operation	集合通信操作：指定跨所有设备的通信模板，而非逐对收发
broadcast / scatter / gather / reduce	广播 / 分发 / 收集 / 归约
all-gather	全收集：每个设备都拼出完整数据
reduce-scatter	归约分发：每个分量跨设备归约后各落一处
all-reduce	全归约：归约后复制到所有设备 = reduce-scatter + all-gather

English	中文
all-to-all	全交换：每个设备给每个设备发一段；均衡时是转置
point-to-point (send / recv)	点对点通信：一个设备发给另一个
isend / irecv / async_op	异步版本：立刻返回，之后 wait
overlap of communication and computation	通信与计算重叠
barrier	同步栅栏：所有进程到齐才继续
cuda.synchronize	等 GPU 上排队的 kernel 全部做完
process group / backend (nccl, gloo)	进程组 / 通信后端
NCCL	NVIDIA 的集合通信库，把集合操作变成通信 kernel
PCIe	主板上的通用总线，GPU 出卡的必经之路
NVLink / NVSwitch / NVLink domain	卡间直连链路 / 交换机 / 能两两直达的那组卡
InfiniBand	节点间高速网络，支持 RDMA
Ethernet	以太网
RDMA	远程直接内存访问：不经 CPU 读写对方显存
RoCE	以太网上的 RDMA
NIC	网卡
kernel socket buffer	操作系统内核里的网络缓冲区（此处 kernel 不是 GPU kernel）
rack / tray / pod	机架 / 托盘（一层） / 一组节点
effective bandwidth / bus bandwidth	有效带宽：该发的字节 ÷ 总时间；nccl-tests 叫 bus bandwidth
ring / tree (topology)	环形 / 树形通信路线，由 NCCL 选
replicate / shard	复制 / 分片
data parallelism (DDP)	数据并行：切 batch，每卡整套模型，梯度 all-reduce
tensor parallelism (column / row)	张量并行：切参数矩阵的列 / 行
pipeline parallelism	流水线并行：切层
micro-batch	微批：流水线里切小的 batch 块
pipeline bubble	流水线气泡：有卡闲着等别人的时段
sequence parallelism	序列并行：切序列，用来并行 attention
expert parallelism	专家并行：MoE 的 expert 分到不同卡，走 all-to-all

English	中文
FSDP / ZeRO	分片式数据并行：参数、梯度、优化器状态分片（第 8 讲）
critical batch size	临界 batch size：再大就不再省时间
compiler-driven sharding	编译器决定通信的分片方式（JAX / XLA 路线）

字幕勘误

"B21"（192 GB 的那张卡）→ B200；"SFTP" → FSDP；"0"、"zero"（与 FSDP 并列时）→ ZeRO；"NV72" → NVL72；"[? great ?]"（GB200 的 G）→ Grace；"nickel" 是 NCCL 的读法；"parem.grad" → param.grad；"weight"（等异步操作完成）→ wait；"put an I before these" → isend / irecv；"pointwise operations"（send / recv）→ point-to-point；"MOE" → MoE；"Cuda" → CUDA；"collect operations" → collective operations；"the inverse of scatter is scatter" → gather（口误或字幕）；"[INAUDIBLE] may or may not have been trained" 的主语应为 Llama 3。

带走的问题

1. 用第 6 节的公式反推：8 张卡的 NVLink 域内 all-reduce 一份 bf16 的 70B 梯度（140 GB）要多久？换成跨节点、每端口约 50 GB/s 的 InfiniBand 呢？拿它和一步训练的计算时间比一比，就明白为什么 DDP 也要做通信计算重叠。
2. all-reduce 的有效带宽与 W 无关，张量并行的每层 all-gather 却让人不敢跨节点——把两者"每张卡每步收发的字节数"和"每步做几次"各写一遍，差别在字节数还是在次数？
3. DDP 让每个 rank 重复保存参数、梯度和优化器状态，换来不用搬优化器状态；FSDP 反过来。对 1T 参数的模型，各自在 192 GB 的卡上算得过来吗？分片之后多出来的通信是第 3 节的哪两半？
4. 流水线并行用 M 个微批把气泡压到约 $(W-1)/(M+W-1)$ ，但每个微批的激活都得留到 backward——显存怎么随 M 变？这和第 5 讲的重计算能怎么组合？
5. 讲者说"哪种并行取决于硬件"：给你 4 个节点、每节点 8 张卡、跨节点 InfiniBand，模型放得下一个节点却放不下一张卡，TP / DP / PP

怎么排？把 NVLink 域当边界画一画，再想想 critical batch size 会不会先卡住 DP。

CS336 第 8 讲 | 并行（下）(Parallelism II)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 8 讲，系统部分的收尾（第 5—8 讲：GPU → kernel → 并行）；课程表标注 2026 年 4 月 22 日 视频：<https://www.youtube.com/watch?v=6-cXp-aOmdg> (1:20:10；英文字幕为自动生成，库名、芯片名和几处术语错得不少，见文末勘误) 讲者：Tatsunori Hashimoto (全程；第 7 讲 Percy 用代码把机制跑了一遍，这一讲自称讲"知识、细节和八卦"——每种并行的算力、显存、通信账，以及真实训练怎么配) 课程主页：<https://stanford-cs336.github.io/> · 讲义反复引用的三篇：ZeRO (Rajbhandari et al., 2019)、Megatron 的大规模训练实验 (Narayanan et al., 2021)、Reducing Activation Recomputation (Korthikanti et al., 2022)；实战部分以 DeepSeek-V3 和 Llama 3 的技术报告为主

一句话：单卡的算力和显存都不够，所以"计算单元"变成了整个数据中心，而数据中心里的链路分快慢（节点内 NVLink 快、节点间慢）——这一讲把每种并行的账算清：数据并行每步通信 2 倍参数量但一点显存不省；ZeRO 把优化器状态、梯度、参数逐级分片，前两级靠 all-reduce = reduce-scatter + all-gather 的恒等式白得，第三级（FSDP）多一次 all-gather 但能藏在计算底下；再往上要靠模型并行传激活：pipeline 通信最省（B·S·H、点对点）但有 bubble、要大 batch 填，tensor parallel 通信最饥渴（约 8 倍 B·S·H 的 all-reduce）只能在 8 卡节点内用，expert parallel 是 MoE 时代对 TP 的替代，sequence parallel 把 TP 除不掉的 10·SBH 激活也切掉——最后的配方出奇简单：先在最快链路上用 TP / EP 切到 8，再用 pipeline 或 ZeRO-3 切到装得下，剩下的卡全给数据并行，batch 不够就梯度累积；Llama 3 405B 的 TP 8 / PP 16 / DP 128、DeepSeek-V3 的 64 路 EP 都是这条规则的实例。

时间轴

时间	内容
0:05	开场：Percy 讲了机制，这讲讲细节与实战；4D 并行；作业要给定拓扑和模型选最优并行
1:05	为什么并行：算力与显存两个瓶颈；节点内快、节点间慢；只在集合通信层面记账；all-reduce ≡ reduce-scatter + all-gather
4:39	硬件底座：TPU 环面网格 vs GPU 胖树；Dally 与 Dean 的对谈；当天早上 Google 新 TPU 转向全互联

时间	内容
8:44	华为 Ascend 910：单芯片弱、384 芯片一柜、功耗 4 倍；"新的计算单元是整个数据中心"
11:16	算法总纲：数据并行与模型并行两大类；朴素数据并行的三项指标
13:17	显存账：每参数约 16 字节、约 5 份权重；优化器状态是大头；ZeRO 图 120 → 1.9
16:50	ZeRO Stage 1: reduce-scatter 梯度、各更新一片、all-gather 参数，通信量与 DDP 相同
19:24	Stage 2 边反传边发梯度；Stage 3 / FSDP 按需取参数，两次 all-gather 一次 reduce-scatter，靠重叠掩盖；问答
28:34	为什么不能止步于 FSDP：batch size 是资源（critical batch size）；激活显存一点没省
30:39	模型并行总纲：传的是激活；pipeline parallel 的 bubble、micro-batch；放在最慢的链路上
36:43	更聪明的调度：DeepSeek 的前后向交错；zero-bubble 把反传拆成 B 与 W
39:14	tensor parallel：切宽度；f / g 的前后向对偶；列切与行切；只在节点内 8 卡；TPU 网格能切更多
44:48	激活显存精算：34·SBH 加注意力项；TP 除不掉的 10·SBH；sequence parallel；配合重计算到 34·SBH/T
53:02	expert parallel：MoE 优先 EP；DeepEP 与 Hybrid EP；all-to-all 与延迟；与 DP 的耦合；attention 与 MoE 的 TP 解耦；context parallel
1:01:39	总表：没有支配策略；算账图：每卡 batch 2,000 时 FSDP 够用，batch 变小要加 TP；3D / 4D 并行
1:06:14	决策流程与 Megatron 五条准则；问答：sequence parallel 只是附件；looped transformer
1:09:28	Megatron 大规模实验：TP 到 8 停、PP 递增、DP 最后降到 6，利用率平坦；重计算换 batch
1:12:30	实战配方：OLMo、DeepSeek V1 / V3、Yi、Llama 3 405B、Gemma 2、Mixtral、Nemotron 3 Super、Qwen3；收尾

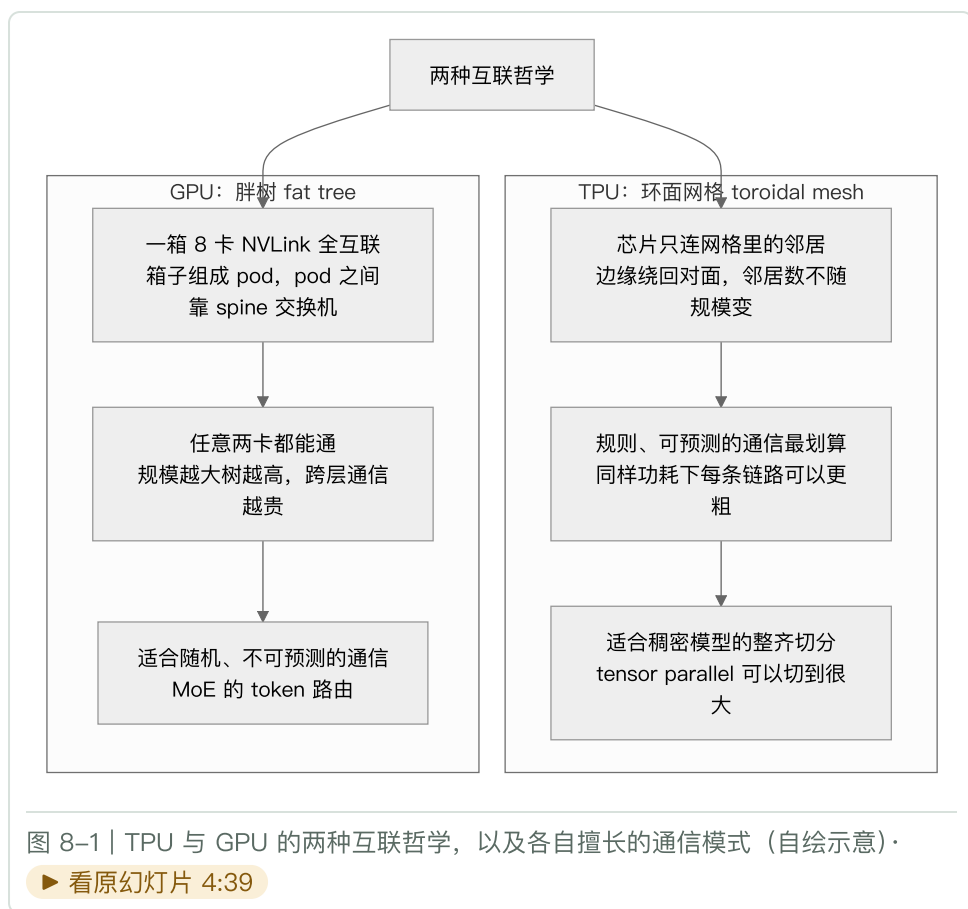
核心内容

1. 为什么要并行：两个瓶颈、两种链路、一个恒等式

- **这一讲的位置**：第 7 讲 Percy 用代码演示了集合通信原语和数据 / 张量 / 流水线并行的机制；这一讲不再重推这些原语，而是回答三个问题：每种并行每一步花多少算力、显存、通信；什么时候该用哪种；真正的大规模训练是怎么把它们叠在一起的。作业里要做的就是这件事：给定网络拓扑和模型，找出最优并行配置。CME295 第 4 讲把 DP、ZeRO 和三种模型并行各用一页带过，这里逐项算账。

- **两个瓶颈。**算力：单张芯片远达不到需要的速度（讲者的对照是世界上最快的超算有 exaFLOP/s 量级，单 GPU 差好几个数量级），解决办法是把很多机器连起来。显存：模型装不进一张卡，只能切成小块分到多卡上。
- **两种链路**是贯穿整讲的概念对象：节点内（intra-node）互联很快，可以做通信很重的事；节点间（inter-node）慢，只能用“尊重带宽”的算法。后面每种并行“该放在哪一层链路上”都由此决定。
- **讨论的层次**：不谈发包，只在集合通信（collective）层面记账。四个原语（第 7 讲的定义）：all-reduce——每卡各有一份同形张量，结束时每卡都拿到逐元素求和的结果；reduce-scatter——同样求和，但每卡只拿到结果的一个分片；all-gather——每卡各有一个分片，结束时每卡都拿到拼齐的整体；all-to-all——每卡把自己的数据按目的地切开，分别发给对应的卡。
- **一个恒等式**：all-reduce 的代价等于 reduce-scatter 加 all-gather（第 7 讲 Percy 讲过：环形 all-reduce 本来就是这两步实现的）。它是第 4 节 ZeRO “白得显存”的全部依据。有学生问为什么偏偏强调这种分解——因为接下来的算法用的正是它；别的分解当然也存在。
- **换一种世界观**：计算单元不再是 GPU，而是整个数据中心。要的是三件事——显存可控、算力可控、资源不闲置。

2. 硬件底座：TPU 的环面网格与 GPU 的胖树



- 算法本身与硬件无关, 但网络拓扑决定了各家的并行配方, 所以先看两种互联哲学。第 5 讲说过 TPU 像一个"轻量 GPU", 真正的差别在网络。
- **TPU: toroidal mesh (环面网格)**。每块芯片只和网格里的邻居直连, 边缘绕回到对面 (真实结构是三维的, 讲者说记住二维图就够)。好处是拓扑极简单、可以无限扩展——不管集群多大, 每块芯片的邻居数不变。只跟邻居说话的通信最划算: 同样功耗下每条链路可以做得更粗。
- **GPU: fat tree (胖树)**。哲学是"任意两卡都能通": 最底层一箱 GPU 之间高速直连, 箱子组成 pod, pod 之间靠 spine 交换机。节点数越多树越高, 跨层通信越贵、拓扑越复杂。换来的是灵活: 通信模式随机、不可预测时也能应付。
- **哪种 workload 适合谁** (讲者转述 Bill Dally 与 Jeff Dean 的对谈): MoE 的 token 要路由到不同 expert, 通信模式不可预测, GPU 合适; 稠密模型做整齐、可预测的切分 (第 7 节的 tensor parallel), TPU 合适。
- **正在趋同**: 讲课当天早上 Google 发布了新一代 TPU, 讲者备课时发现推理芯片改成了树状、更接近全互联的拓扑; 训练芯片的跨机柜互

联（他提到叫 Virgo 的网络层）也长得更像 GPU 那一套。原因不难猜：今天的主流模型都是 MoE，服务 MoE 时 token 在 expert 之间飞来飞去，推理时这些通信就是瓶颈。讲者称之为趋同进化：是 workload 在定义网络。

小注：字幕里的 "TPU AI / 8T" 应为 TPU 8i（推理）与 TPU 8t（训练），2026 年 4 月 22 日 Google Cloud Next 上发布（推断，讲者只说"今天早上"）。

- **另一条路：用功耗换互联。**第 5 讲有人问"SRAM 那么好为什么不全用 SRAM"，答案是 Groq。同理，"全互联那么好为什么不用光纤交换机把所有芯片连起来"，答案是华为 Ascend 910：单芯片比 H200 弱不少（矩阵乘慢），但一个机柜用光交换把 384 块芯片连成一体，靠数量补单芯片的差距；代价是功耗约为同等 NVIDIA 系统的 4 倍。讲者的结论：想在功耗和制造上都高效，设计会落到一处；愿意暴力堆资源，会落到另一处。

小注：应指 CloudMatrix 384（384 块 Ascend 910C）。据 SemiAnalysis 2025 年 4 月的分析，它的总 BF16 算力约为 GB200 NVL72 的 1.7 倍，功耗约 3.9 倍，单芯片算力约为 GB200 的三分之一——和课上"单芯片弱、靠数量、功耗 4 倍"的说法一致。

3. 数据并行与显存账：为什么是每参数 16 字节

- **数据并行 (DP)：**先忘掉 Adam，只看 SGD。每步取 B 个样本，把 B 切给 M 台机器，每台算自己 B/M 个样本的梯度，再把梯度同步求和后更新。三项指标：算力扩展完美——只要每卡样本够多；通信每步约 2 倍参数量——一次 all-reduce；显存零节省——每卡一份完整模型、完整激活。
- **显存有多糟：**直觉上"显存 = 参数"，实际远不止。经验法则是要存约 5 份权重大小的东西、每参数约 16 字节：参数本身；梯度（更新前的累加器）；可能还有一份高精度的累加副本；Adam 的一阶矩和二阶矩，各一份且往往要高精度。后几样统称优化器状态（optimizer state），是显存的大头。

$$\text{bytes per parameter} = \underbrace{2}_{\text{bf16 weight}} + \underbrace{2}_{\text{bf16 grad}} + \underbrace{4 + 4 + 4}_{\text{fp32 master, } m, v} = 16$$

权重和梯度各 2 字节（bf16），fp32 主权重、Adam 的 m、v 各 4 字节；这就是"5 份、16 字节"的来历，其中 12 字节是优化器状态，占四分之三。

小注：字节的拆分按 ZeRO 论文的混合精度记账（优化器状态 $K = 12$ 字节 / 参数）；课上只说了“约 5 份、16 字节”。

- **ZeRO 的那张图 (15:19)**：绿色的优化器状态最大，橙色的梯度和蓝色的参数一样大。朴素 DP 把这三样在每张卡上原样复制，总显存随卡数线性增长。把优化器状态分片，是一大笔节省；再把梯度分片、把参数也分片，图最右边一列从 120 降到 1.9。问题是代价——直觉上没有免费午餐，越往下通信越贵；讲者说值得惊讶的是，其中大半是免费的。

小注：120 → 1.9 是 ZeRO 论文 Figure 1 的例子：7.5B 参数、64 卡，每卡 120 GB → Stage 1 31.4 GB → Stage 2 16.6 GB → Stage 3 1.9 GB。

4. ZeRO 三级：前两级白送，第三级靠重叠

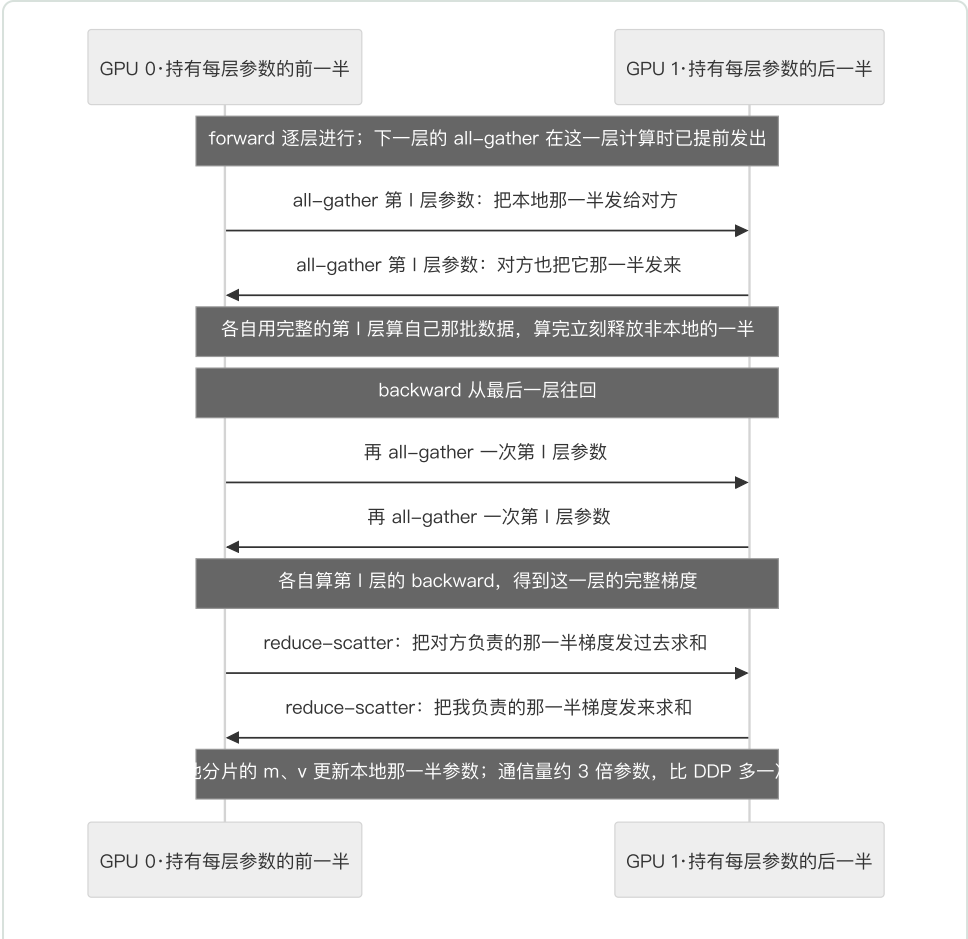


图 8-2 | ZeRO-3 / FSDP 在两张卡上的一步：每层两次 all-gather、一次 reduce-scatter，用完即放（自绘示意）。▶ 看原幻灯片 21:25 · 出处：Rajbhandari et al., 2019

- **Stage 1**：只分优化器状态。每张卡仍有完整参数和梯度，但只负责更新参数的一个切片（GPU 0 负责第一片）。一步四个动作：① 各卡在

自己的数据上算出完整梯度；② reduce-scatter——我把梯度切片发给各片的负责人并求和，通信量约 1 倍参数；③ 各卡用自己那片的 m、v 更新自己那片参数；④ all-gather 把更新后的切片拼回完整参数，再 1 倍参数。合计仍是 2 倍参数——和 DDP 的一次 all-reduce 一模一样，显存却把 12 字节那块除以了卡数 N。这就是恒等式的用处：显存白得。

- **Stage 2：再分梯度。**麻烦在于现在连完整梯度都不能物化。解法只是一个系统技巧：反向传播是逐层往回扫的，算完一层的梯度就立刻 reduce-scatter 给负责它的卡，然后释放；分批发和一次发总量相同，所以还是不多花通信。
- **Stage 3 / FSDP：连参数也分。**同一招——按需取。正向：走到第 l 层时 all-gather 这一层的参数，算完立刻释放；反向：再 all-gather 一次算 backward，得到的梯度立刻 reduce-scatter 出去，再释放。任何时刻每卡手里只有一层的完整参数、一片梯度和一片优化器状态。通信是两次 all-gather 加一次 reduce-scatter，约 3 倍参数——比 DDP 多一次 all-gather，而且每层都在通信，看着很吓人。
- **为什么几乎免费。**两个想法：其一就是"扫到哪、通信到哪、用完即放"；其二是通信与计算重叠——讲者用 PyTorch FSDP 论文里的时间线图解释：CPU、GPU 计算流、GPU 通信流三条并行；all-gather 完第 0 层就开始算第 0 层的 forward，同时通信流已经在 all-gather 第 1 层。只要计算比通信长、网络够快，通信就全藏在计算底下，只剩几个小 bubble。实践里 FSDP 的 GPU 利用率非常接近单卡——作业要自己写一个 FSDP wrapper：包住任意 module，正向"gather → 算 → 放"，反向再来一遍。

$$C_{\text{DDP}} = 2\Psi = \underbrace{\Psi}_{\text{reduce-scatter}} + \underbrace{\Psi}_{\text{all-gather}} = C_{\text{ZeRO-1}} = C_{\text{ZeRO-2}}, \quad C_{\text{ZeRO-3}} = 3\Psi$$

Ψ 是参数个数，C 是每步的通信量（按讲者的单位"多少倍参数"计）。ZeRO-1、-2 与 DDP 完全相同；ZeRO-3 多一次 all-gather。

$$M_{\text{DDP}} = 16\Psi, \quad M_{\text{ZeRO-1}} = 4\Psi + \frac{12\Psi}{N}, \quad M_{\text{ZeRO-2}} = 2\Psi + \frac{14\Psi}{N}, \quad M_{\text{ZeRO-3}} = \frac{16\Psi}{N}$$

N 是数据并行的卡数；每卡显存里，哪一项被分片，哪一项就除以 N。代入 $\Psi = 7.5\text{B}$ 、 $N = 64$ 正好得到上面那张图的 120、31.4、16.6、1.9 GB。

小注：这组式子是 ZeRO 论文的记账，课上只展示了结果数字。

```
# ZeRO Stage 1: 每卡持有完整参数和梯度，只持有 1/N 的优化器状态
def zero1_step(model, batch, rank, N):
    grads = backward(model, batch)          # 完整梯度，只来自本卡的数据
    my_grad = reduce_scatter(grads, rank)    # 收到"我负责的那 1/N"的求和梯度
    my_params = adam_update(params[rank], my_grad, m[rank], v[rank])
    params = all_gather(my_params)          # 拼回完整参数，供下一步 forward
```

四行对应四个动作；reduce-scatter 加 all-gather 的通信量恰好等于 DDP 的一次 all-reduce。

```
# ZeRO Stage 3 / FSDP: 参数也分片，逐层"取来-用完-释放"
def fsdp_step(layers, x):
    acts = []
    for l in layers:                          # forward
        w = all_gather(l.shard); acts.append(x); x = l.forward(w, x);
    free(w)
    g = loss_grad(x)
    for l in reversed(layers):                # backward
        w = all_gather(l.shard)               # 第二次 all-gather
        g, grad_w = l.backward(w, acts.pop(), g)
        l.grad_shard = reduce_scatter(grad_w); free(w, grad_w)
    for l in layers:
        l.shard = adam_update(l.shard, l.grad_shard) # 只更新本地分片
```

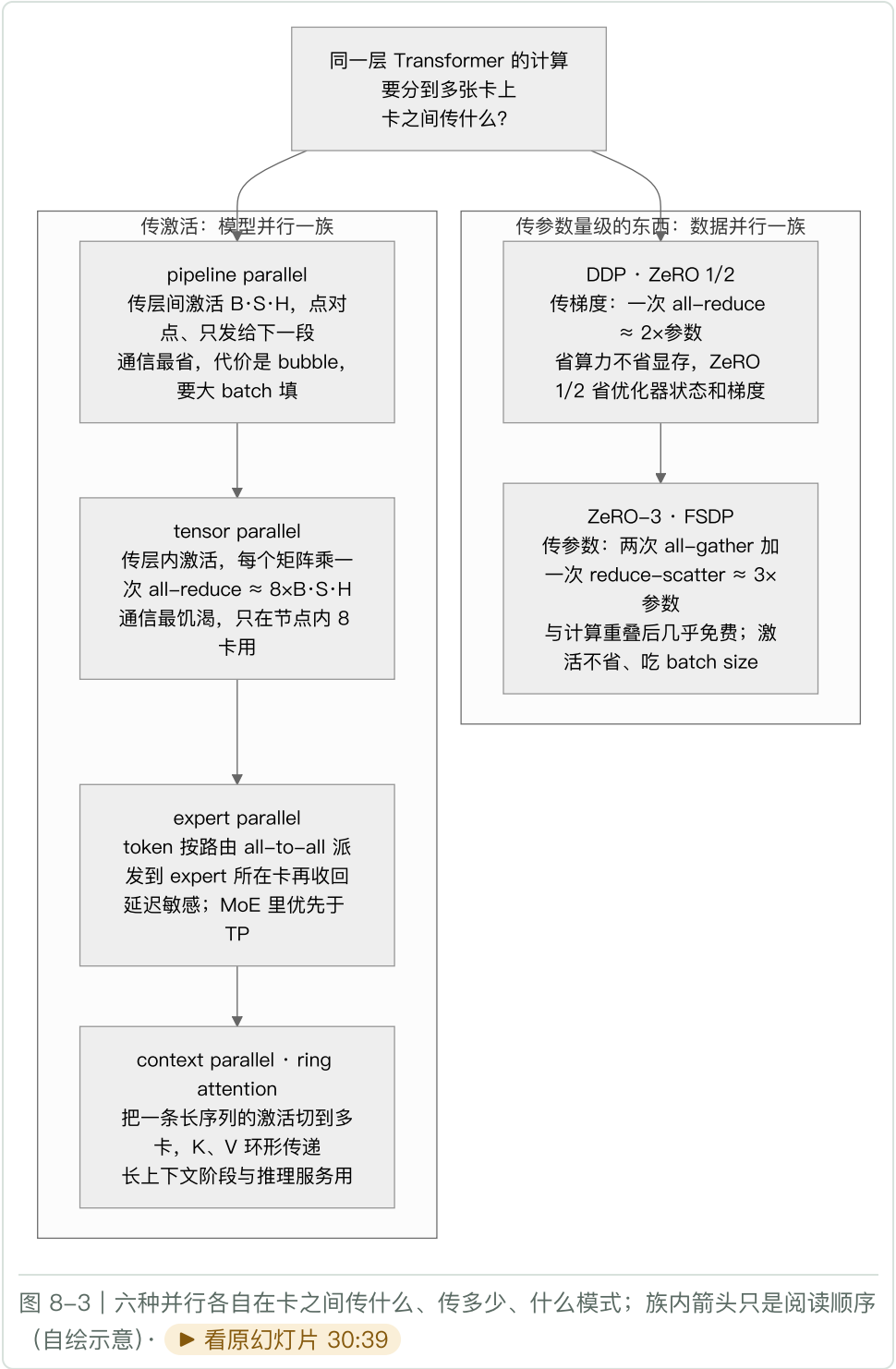
每层两次 all-gather、一次 reduce-scatter；实际实现里下一层的 all-gather 会在这一层计算时提前发出，这就是重叠。

• 问答

- 这不是流水线：流水线是不同层放在不同卡上；FSDP 里每张卡都跑完整个模型，只是没有哪张卡同时持有全部参数——每步计算前"要参数"、算完"放参数"。
- 每张卡都持有每一层的一部分？对，参数、梯度、优化器状态三样都是这样分的。
- 通信次数乘了层数，为什么总量不乘？次数确实乘了，但每次只传一层（一个小 MLP），加起来正好是整个网络一遍。
- 装得下多大：讲者给的表（应为 8 张 A100 的机器）——基线连 7B 都装不下，Stage 3 能装到约 50B。换 H100 / H200 怎么算？

线性换算即可，比如乘 141 除以 80（H200 的 141 GB 对 80 GB）。

5. 为什么不能止步于 FSDP：batch size 是资源，激活没省



- 讲者说 FSDP 干净优雅，真想就此下课，但有两个理由必须继续往"更丑更毛"的方法走。

- **数据并行消耗的资源是 batch size**：batch 是 8，就最多只能用 8 张卡。能不能卡越多 batch 越大？不能无限大：存在一个临界 batch size (critical batch size)，超过它之后，多加一个样本带来的收益不如拿这个样本多走一步 SGD——小 batch 时加样本等价于多走步，之后收益递减，无限大的 batch 并不等于无限多步。于是陷入两难：batch 小让 GPU 闲着，batch 大让优化吃亏。

小注：critical batch size 这个概念应出自 McCandlish et al., 2018 (用梯度噪声尺度预测临界 batch size)。第 9、11 讲的 scaling laws 会再碰到它。

- **激活显存一点没省**：ZeRO 切的是参数、梯度、优化器状态；激活 (activations，每层为反传留下的中间结果) 以及其他动态显存分毫未动。要继续压显存，得更细粒度地切模型本身。
- **模型并行的概念分界**：FSDP 也切参数，但它只是一层包装，计算仍是单卡的原样计算，飞来飞去的是参数。模型并行则让卡之间传激活：第 0 层在 GPU 0、第 1 层在 GPU 1，要传的就是两层之间的激活。三种：pipeline parallel 切层，tensor parallel 切矩阵，expert parallel 切 expert。图 8-3 把每种方案“传什么、多大、什么模式”放在一起。

6. Pipeline parallel: bubble、micro-batch，以及放在最慢的链路上

- **做法**：按层切，正向传激活、反向传偏导。朴素地做，4 张卡各管四分之一的层，任何时刻只有一张卡在干活：卡 0 算完交给卡 1 就闲到最后，反向同样一次只有一张卡活跃——利用率惨不忍睹。空转的部分叫 bubble。
- **填 bubble 靠流水**：把 batch 切成若干 micro-batch，卡 0 算完第一个立刻算第二个，同时第一个已交给卡 1。bubble 占比大约是流水段数除以 micro-batch 数，要让它趋于零，需要很大的 batch——这就是为什么说 batch size 是一种资源：数据并行花它，流水线也花它，花在不同的地方。

$$\frac{t_{\text{bubble}}}{t_{\text{ideal}}} \approx \frac{p-1}{m}$$

p 是流水段数 (stage 数)， m 是每步的 micro-batch 数；讲者口头说的是“段数除以 micro-batch 数”， $(p-1)/m$ 是 GPipe 和 Megatron 论文里的精确形式，含义相同：bubble 随 m 按 $1/m$ 消失。

- **为什么还要用它：**流水线臭名昭著（folklore：并行代码在实现 pipeline parallel 之前都还能看懂），但它有两个不可替代的优点。一是省显存（层切开了），且可与数据并行叠加；二是通信性质最好——只传激活，大小是 $B \cdot S \cdot H$ （batch $\times$ 序列长 $\times$ 隐藏维），而且是点对点，只发给下一段，不是 all-to-all。所以实践里流水线放在最慢的链路上：跨 pod、跨数据中心的并行就用它。有学生问“比 FSDP 好在哪”：传的东西小—— $B \cdot S \cdot H$ 几乎总小于一整个参数矩阵。
- **代价看 Megatron 的扫描：**batch 大、流水段多时，利用率能接近不做流水线；batch 一小，利用率迅速垮掉。
- **更聪明的调度。**一类是把不同 micro-batch 的前向和后向交错排布（讲者用的是 DeepSeek 报告里的图，应为 V3 的 DualPipe）。更巧的是 zero-bubble pipelining，它不靠排班，而是看反传的结构：反传在每个节点做两件事——把偏导继续往前一段传（B），和算本层权重的梯度（W）。前者在关键路径上，前一段拿不到它就没法开工；后者是计算图上的叶子，什么时候算都行。于是把两者拆开：B 尽早算，W 推迟到流水线有空隙时补——几乎能把 bubble 填满，代价是实现复杂得多。

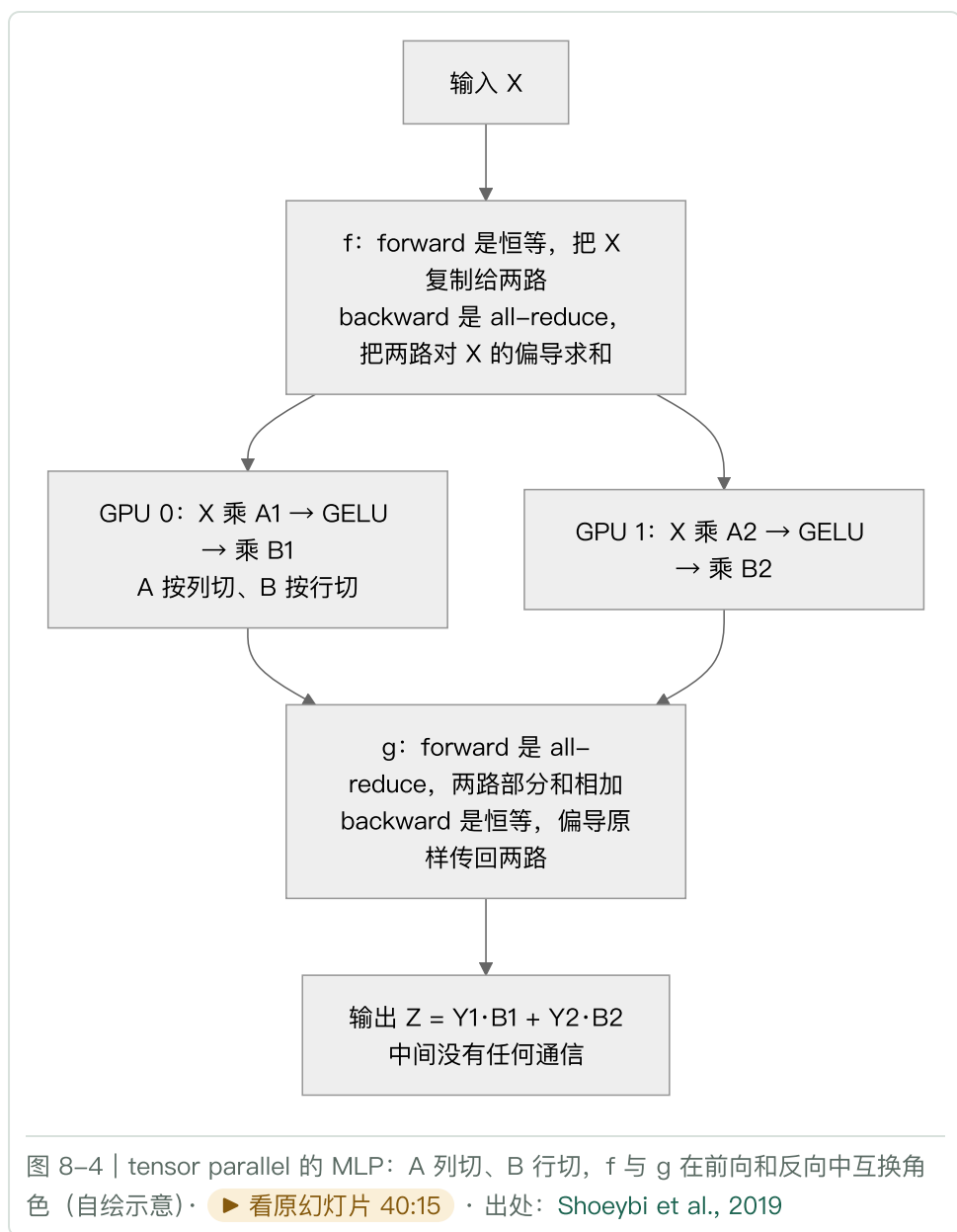
小注：Zero Bubble (Qi et al., 2024) 论文摘要：相同显存上限下吞吐比 1F1B 高至多 23%，放宽显存限制时高 31%。DeepSeek-V3 报告的 DualPipe 也做了同样的 B / W 拆分。

```
# zero-bubble 的核心：把一层的 backward 拆成两半，B 走关键路径，W 见缝插针
def backward_split(layer, grad_out):
    grad_in = layer.backward_input(grad_out)    # B：偏导继续往前一段传，
    立刻发出
    send_to_prev_stage(grad_in)
    pending_W.append((layer, grad_out))         # W：权重梯度先记账，不算

def fill_bubble():
    while pipeline_idle() and pending_W:
        layer, g = pending_W.pop(0)
        layer.grad_w += layer.backward_weight(g)  # 有空隙时再算
```

B 和 W 在标准实现里是同一个 backward 调用；拆开之后 W 成了可以自由挪动的填充物。

7. Tensor parallel：切宽度、前后向对偶、只在节点内



- **切宽度**: 流水线沿深度切, tensor parallel (TP) 沿宽度切。原理和 tiling 一样 (第 5、6 讲): 矩阵乘可以拆成小矩阵乘再把部分和加起来——同一个原语反复出现。
- **以 MLP 为例**: $X \rightarrow \text{乘 } A \rightarrow \text{GELU} \rightarrow \text{乘 } B \rightarrow Z$ 。TP 把 A 按列切成 A1、A2, B 按行切成 B1、B2, 两张卡各算一路, 最后把部分和加起来。
- **f / g 的对偶**是写 TP 的关键: 前向时 f 是恒等 (把 X 复制给两路)、g 是 all-reduce (两路相加); 反向时正好翻转——g 变成恒等 (偏导原样传回两路), f 变成 all-reduce (把两路对 X 的偏导求和)。
- **哪里列切、哪里行切**: 每个 Transformer block 里, 输入侧的矩阵按列切 (MLP 的上投影、attention 的 Q / K / V 投影), 第二阶段按行

切（MLP 的下投影、attention 的输出投影）。LayerNorm、非线性、MoE 的 router 这类小算子全卡复制，不值得切。

$$Z = [\text{GELU}(XA_1) \text{ GELU}(XA_2)] \begin{bmatrix} B_1 \\ B_2 \end{bmatrix} = \text{GELU}(XA_1) B_1 + \text{GELU}(XA_2) B_2$$

A 按列切成 A1、A2 后，GELU 可以逐块独立算（逐元素的非线性不跨列）；B 按行切后两路各得一个部分和，一次 all-reduce 相加即为 Z——中间不需要任何通信，这是把切法选成"先列后行"的原因。

```
# tensor parallel 的 MLP: 本卡只持有 A 的一列块和 B 的一行块
def tp_mlp_forward(x, A_col, B_row):
    x = f(x)                                # 前向恒等；反向对 x 的偏导做 all-reduce
    y = gelu(x @ A_col)                      # 本卡这一路，不通信
    z_partial = y @ B_row
    return g(z_partial)                     # 前向 all-reduce 求和；反向恒等
```

f 和 g 是两个自定义的 autograd 函数，前向和反向的角色互换；整个 block 里只有它们两处通信。

- **代价与用武之地：**每个矩阵乘都要通信一次激活大小的 all-reduce，前向一次、反向一次，频繁而饥渴，所以只在节点内做：一箱 8 张 GPU 紧耦合，TP 最多开到 8；一旦跨节点，互联慢一个量级，性能立刻掉一截。TPU 没有"8 卡一箱"和"箱外"的分界，整个 pod 是一张网格，能在规则通信模式上维持高带宽，所以 TPU 上的 TP 可以开得多——这是 TPU 与 GPU 并行配方的主要差别。
- **TP 与 PP 对比：**都切模型、都省参数和（部分）激活。TP 的优点：没有 bubble、网络够快时能跑满、实现简单；缺点：通信量大得多——PP 是 B·S·H 的点对点，TP 大约是 8 倍 B·S·H 的 all-reduce。结论：有高速互联用 TP，其他场合用 PP。

小注："8 倍"可以这样凑出来（推断）：一层里 attention 和 MLP 各一次 all-reduce，前向、反向各来一遍共 4 次，而环形 all-reduce 每卡收发数据约为张量大小的 2 倍。

8. 激活显存精算：TP 除不掉的 10·SBH，与 sequence parallel

- **显存不只是参数：**PyTorch profiler 的曲线上，参数和优化器状态是平的底，红色的大驼峰是激活——正向逐层堆高，反向开始后梯度也要占位，所以峰值出现在反传刚开始、激活还没释放多少的时候。模型变大、序列适中时，激活远超参数显存（讲者用的是 Korthikanti 等人论文里的表），任何省显存的方案都绕不开它。

- **全存时每层要多少**： $34 \cdot S \cdot B \cdot H$ 加 $5 \cdot A \cdot S^2 \cdot B/H$ (S 序列长、 B batch、 H 隐藏维、 A 注意力头数；单位字节)。 $S \cdot B \cdot H$ 这个依赖是根本性的——序列里每个位置、batch 里每个样本、隐藏维每一维都得存点东西。第二项是注意力打分矩阵那一块，含 dropout 掩码；用 FlashAttention 式的重计算（第 5 讲）可以整个去掉。
- **只做 TP 省不干净**：TP 切的是 attention 和 MLP 里的矩阵乘，34 份里的 24 份和第二项都能除以 T (TP 的路数)。但 LayerNorm、dropout，以及 attention 和 MLP 的输入（要作为残差留给反传）这 10 份 $S \cdot B \cdot H$ 没被切——就算像 Google 那样开 1,000 路 TP，仍然要付 $10 \cdot SBH$ 。
- **Sequence parallel (SP)**：名字容易误导（“按序列切”的思想更自然地属于 context parallel）。做法：剩下这 10 份都是 LayerNorm、dropout 这类没什么计算量的轻算子，就把它们沿序列轴切到各卡上，需要时再 all-gather 回来，用完 reduce-scatter 回去——和 FSDP 的“分片存放、按需物化”一个思路。同样有对偶：前向 g 是 all-gather、 g^{-1} 是 reduce-scatter，反向互换。

$$M_{\text{act}} = sbh \left(34 + \frac{5as}{h} \right) \xrightarrow{\text{TP}} sbh \left(10 + \frac{24}{t} + \frac{5as}{ht} \right) \xrightarrow{+\text{SP}} sbh \left(\frac{34}{t} + \frac{5as}{ht} \right) \xrightarrow{+\text{selective recompute}} \frac{34 sbh}{t}$$

每层激活的字节数： s 序列长、 b batch、 h 隐藏维、 a 头数、 t 是 TP 路数。TP 把矩阵乘部分除以 t ，SP 把剩下的 10 份也除以 t ，选择性重计算去掉注意力打分那一项——最后的 $34 \cdot sbh/t$ 是讲者说的“正常训练里激活显存合理的下界”，手算“模型装不装得下”时用它，再加上参数、梯度和优化器状态。

小注：34 的拆分 (Korthikanti et al., 2022)：attention 块 11 份加 $5as^2b$ ，MLP 19 份，两个 LayerNorm 4 份；TP 能除的 24 份是 attention 里的 8 份和 MLP 里的 16 份，课上说成“MLP 是 24 份”是口头简化。论文摘要：激活显存降 5 倍、重计算开销减 90% 以上；530B 模型在 2,240 张 A100 上 MFU 54.2%，对比全量重计算的 42.1%，快 29%。

- **问答：MLP 那 $24/T$ 能不能也重计算掉？** 能，重计算可以做得比这更多，但 MLP 的重计算等于反传时再跑一遍 MLP，太贵；attention 的重计算便宜——分块做，还省掉了 S^2 的存储，划算。

9. Expert parallel 与 context parallel

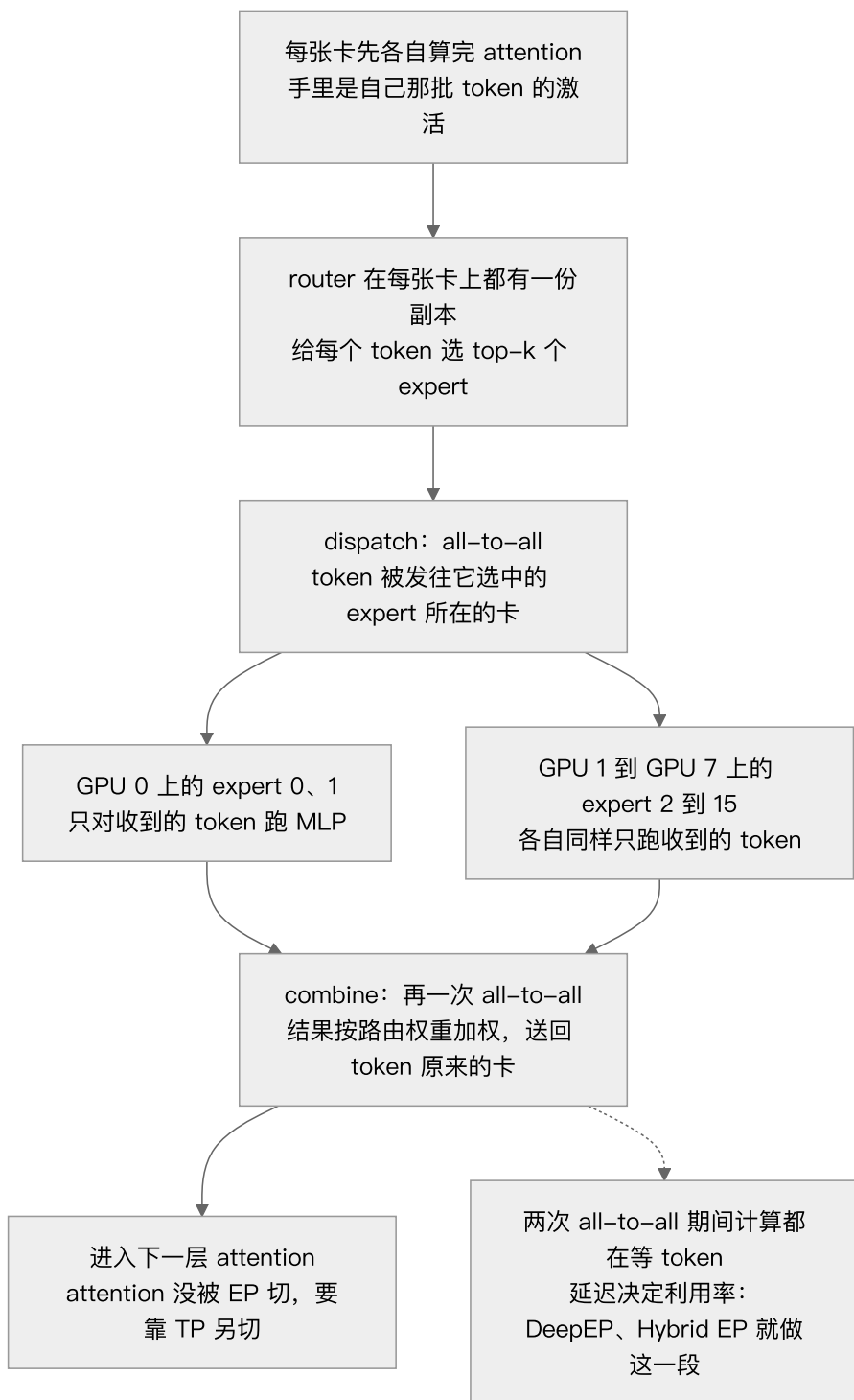


图 8-5 | expert parallel 的一层：两次 all-to-all 把 token 送到 expert 所在的卡再收回，attention 不在 EP 的管辖内（自绘示意）· [▶ 看原幻灯片 57:05](#)

- **EP 是 MoE 时代对 TP 的替代：**MoE（第 4 讲）现在是标配，它带来的系统红利就是 expert parallel——把整个 MLP（一个或几个 expert）放到不同的卡上，token 路由到哪个 expert 就发到哪张卡。系统性质和 TP 相似：高带宽原语、也能减少激活。

- **MoE 里为什么优先 EP 而不是 TP**（Megatron 的并行指南）：TP 把矩阵越切越碎，矩阵乘变小，GPU 利用率下降；MoE 层路由的是稀疏的 token 激活，比 TP 里搬运稠密的大矩阵激活轻；token 只送到需要它的地方，省掉不必要的开销——既然要做 MoE，就顺势把 expert 铺到各卡上。
- **但 EP 一点也不简单**：讲者说 FSDP 之后的所有东西（TP、PP、EP）系统上都不平凡。两个例子：DeepSeek 的 DeepEP（V3 时代，讲者最喜欢的东西之一）在 GPU 网络原语层面做路由与派发，把操作合并、让硬件接管；NVIDIA 的 Hybrid EP 是同类工作。难在哪：每个 MLP 都要做一轮 all-to-all 派发，而且计算在等 token 到达——延迟极敏感。轶事：DeepSeek 为了榨干性能，找到了未公开的 PTX（GPU 机器码级）指令来加速通信——前沿就是这个级别的功夫。
 小注：DeepSeek-V3 报告里写明用了定制的 PTX 指令做跨节点通信；DeepEP 于 2025 年 2 月开源。
- **和别的并行拼在一起的限制**：其他并行大多像乐高一样随便搭（DP 加 TP 很容易），EP 有额外约束。老库的做法是 EP 的副本集等于 DP 的副本集：DP = 8 就把 expert 铺在这 8 个副本上，EP 是 DP 划分的子集——这就给 EP 的规模封了顶，也牵制着 DP 和 TP 的搭配。
- **attention 与 MoE 的 TP 解耦**：MoE 只改 MLP、不改 attention，所以 EP 只并行了 MLP。attention 想切就得靠 TP，希望 TP 大；可 TP 大加 EP 大会把 MoE 的矩阵切成碎片、利用率崩掉——attention 想要高 TP，MLP 想要低 TP。近几年的解法是让两者各用各的 TP 度：attention 一套，MoE 层另一套，代价是系统更复杂。
- **Context parallel / ring attention**：把一条长序列的激活切到多卡，K、V 沿环传递（原始论文在 TPU 的网格拓扑上做得很好）。用在长上下文扩展阶段和推理服务里。讲者没展开，因为思想与前面重叠。
- **问答**：pipeline 和 FSDP 对 MoE 还适用吗？完全适用；标准建议是 EP 像 TP 一样开到 8 条快链路（现在大家已经不守这条了），DP / FSDP、PP、TP 在所有前沿模型里都在大量使用。

10. 拼起来：没有支配策略、算账、决策流程

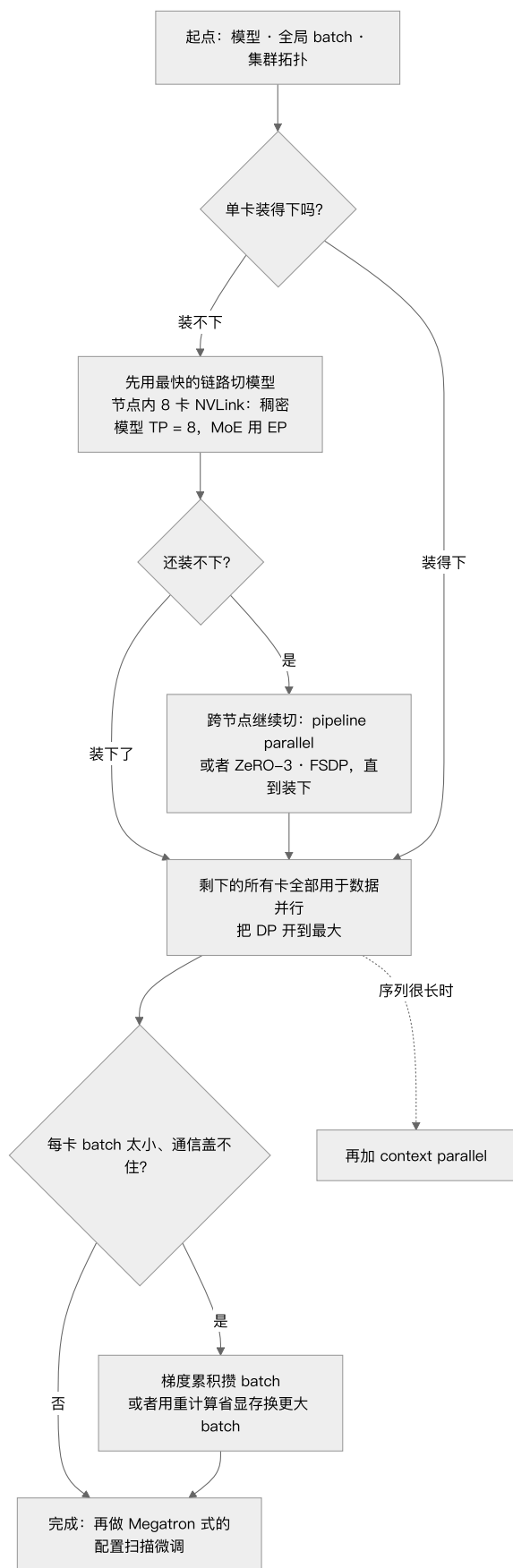


图 8-6 | 选并行配置的决策流程：先在快链路上切模型，再跨节点切，剩下全给数据并行（自绘示意）· [▶ 看原幻灯片 1:06:14](#)

- 总表里红字的意思：**讲者把 DDP、FSDP 和各种模型并行列成一张表，主观标红每种的缺点——想说的只有一句：没有一种策略严格占优，全是折衷。FSDP 很好，但不省激活、消耗全局 batch；TP 省激活、不吃 batch，但要快网络；慢链路又得靠别的方案去用满。所以大规模训练几乎都是多种并行的组合。

策略	卡之间传什么	每步通信量（讲者口径）	通信模式	省哪块显存	主要限制
DDP	梯度	2 倍参数	all-reduce	不省	吃 batch size
ZeRO-1 / 2	梯度分片、参数分片	2 倍参数	reduce-scatter + all-gather	优化器状态、梯度	吃 batch size；激活不省
ZeRO-3 / FSDP	参数分片、梯度分片	3 倍参数	2 次 all-gather + 1 次 reduce-scatter	参数、梯度、优化器状态	激活不省；吃 batch size；靠重叠
pipeline	层间激活	B·S·H	点对点	参数、激活	bubble，要大 batch
tensor	层内激活	约 8 倍 B·S·H	all-reduce	参数、激活	只能在节点内的 24 / 34 8 卡
sequence（附于 TP）	轻算子的激活	若干 all-gather / reduce-scatter	集合	剩下的 10·SBH	不单独使用
expert	token	每层两次 all-to-all	all-to-all	MLP 参数、激活	延迟敏感；与 DP 耦合
context / ring	长序列的 K、V	环形传递	点对点	长序列激活	长上下文阶段用

- 算账（作业要做）：**对每种切法算每层的计算量和通信量，在给定卡数下看两者随规模怎么变；只要计算时间比通信时间长，通信就能藏起来。画成图是"每卡 batch size 对利用率"：顶上的虚线是 roofline 式的效率边界（第 5 讲），线上是算力吃满，线下是在等通信。讲者的读法：每卡 batch 2,000 时只用 FSDP 就完全 compute bound；batch 变小，FSDP 会掉到通信瓶颈，这时加进 TP 能把曲线往小 batch 推一段，再加别的策略继续推——这就是 3D、4D 并行（"我不觉得有 5D"）。

$$T_{\text{compute}} \geq T_{\text{comm}} \implies \frac{6 B_{\text{tok}} \Psi_{\ell}}{F} \geq \frac{6 \Psi_{\ell}}{W} \iff B_{\text{tok}} \geq \frac{F}{W}$$

以 FSDP 的一层为例（推断，课上只给了"计算时间大于通信时间就能藏住"这条准则）： Ψ_{ℓ} 是这一层的参数数， B_{tok} 是每卡每步的 token 数， F 是每卡算力（FLOP/s）， W 是每卡可用带宽（字节/s）；正反向计

算约 $6 \cdot B_{\text{tok}} \cdot \Psi_{\ell}$ FLOPs，三次集合通信按 bf16 约搬 $6 \cdot \Psi_{\ell}$ 字节，两边的 Ψ_{ℓ} 消掉——每卡每步的 token 数要不小于“算力除以带宽”。这就是为什么 batch 一小 FSDP 先垮，也是为什么算力增速快于带宽的年代（第 5 讲）这个阈值越来越高。

- **决策流程其实很简单**（图 8-6）：模型装不下，就先在最快的链路上切模型——一台机器 8 张卡，就开 8 路 TP 或 EP；还装不下，用 pipeline parallel 或 ZeRO-3 一路切到装下为止；装下了，剩下所有卡全部拿去做数据并行；每卡 batch 太小，就用梯度累积换利用率。Megatron 的 MoE 并行指南（对稠密模型同样适用）把这条规则倒过来写成五条：① 模型并行最小化、数据并行最大化；② EP 和 TP 留在 NVLink 内，也就是一箱之内；③ 跨节点用 pipeline；④ MoE 优先 EP；⑤ 长序列用 context parallel。
- **问答**：sequence parallel 单独算一维吗？不算，它是 TP 的附件，专为减激活。如果传闻中的 looped transformer（层循环复用）成真，系统会怎样？对 FSDP 不友好——它建立在“取权重、丢权重”的循环上，循环模型没法丢；但参数效率高，模型并行可能就没那么重要了。

11. 大规模实验与真实配方：TP 到 8 停，DP 开到最大，EP 开始变大

- **Megatron 的老论文仍是最好的教材**（NVIDIA 与当时在 Stanford 的 Matei Zaharia、Deepak Narayanan；讲者说网络的基本面这几年没变，结论仍然适用）：在很多种配置上跑大规模训练，看并行策略该随规模怎么变。结果正是上面的规则：DP 先开满；TP 随规模增加，到 8 就停；之后靠 PP 一路加；规模最大时 DP 反而降到 6——因为光是装下模型就需要那么多 TP 和 PP，剩下的预算才给 DP。把这些叠起来，卡数大到离谱时利用率仍然平、仍然高——这就是巨型数据中心、乃至跨数据中心训练能成立的原因。

小注：论文摘要（Narayanan et al., 2021）：1T 参数模型在 3,072 张 A100 上达到 502 PFLOP/s，为理论峰值的 52%；提出的交错式流水线调度再提升 10% 以上吞吐。“DP = 6”来自该论文最大的配置：TP 8 × PP 64 × DP 6 = 3,072。

- **三条量化证据**：TP 该在 8 停下，超过就出问题；PP 需要大 batch——图上蓝线与橙线的差距随流水段数增大；重计算是划算的——多算一些换来更少显存，显存换成更大 batch，batch 换来更高利用率。这条“多算反而更快”的逻辑和 FlashAttention 一样（第 5 讲）。

• 真实训练怎么配（讲者挑的样本）：

模型	并行配置（课上所述）	讲者的点评
OLMo 7B（AI2, Dolma 数据集）	纯 FSDP	小模型（7B 上下）纯 FSDP 就能扩到很多卡，很多 7B 级模型都这么训
DeepSeek V1	ZeRO-1 数据并行 + tensor + sequence + pipeline	经典组合
DeepSeek-V3（MoE）	pipeline + 64 路 expert parallel, 8 台机器一组作为 EP 域；不用 TP	用流水线上的同类技巧避免 EP 的低利用率段，相当 exotic
Yi	ZeRO-1 + TP + PP	经典 DP / TP / PP 三件套；换成 MoE 就把 TP 换成 EP，目标相同、EP 更高效
Llama 3 405B（稠密）	主预训练 TP 8 / CP 1 / PP 16 / DP 128；长上下文阶段调高 CP、调低 DP	少见的分阶段完整披露；训练期间 GPU 故障 148 次，容错是另一门分布式系统课
Gemma 2（Google）	FSDP + TP + SP，仅此两类	TPU 阵营"不用流水线，在大网格上做 TP"的实证；能否无限扩展存疑
Mixtral 8x22B（NVIDIA Megatron Bridge 配方）	EP 8 / PP 4 / TP 4（TP 给 attention 层）	符合"EP 在 8 左右"
Nemotron 3 Super	大量 EP 加 context parallel	走 DeepSeek-V3 路线，做长上下文扩展
Qwen3	EP 32 / PP 8 / TP 2（TP 切 attention 矩阵）	DeepSeek 配方：大 EP、小 TP

小注：DeepSeek-V3 报告：16 路 PP、跨 8 个节点的 64 路 EP、ZeRO-1 DP，流水线用 DualPipe。Llama 3 报告：主预训练用 16,384 张 H100（8 × 1 × 16 × 128），长上下文阶段 TP 8 / CP 16 / PP 16 / DP 8；"148"是 54 天快照期内归因于 GPU 故障的中断次数（占 419 次意外中断的 30.1%）。OLMo 论文记载 7B 分别在 LUMI（AMD MI250X）和 MosaicML 的 A100 集群上各训了一遍。

- **三条共性：**所有模型都把数据并行开到最大；TP 几乎总在 8 以下；EP 现在可以很大——很大程度上归功于 DeepSeek-V3 为大规模 EP 建的基础设施。Megatron Bridge 里还有 NVIDIA 对各模型的并行基准：即便同是 TP，子配置的差别也会显著影响性能，系统集成的功夫不小。
- **收尾：**要在多 GPU、多节点、甚至多数据中心的尺度上谈效率，快链路、慢链路、batch size 都是要用满的资源，而组合它们的规则出奇

地简单。下一讲：scaling laws。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
TPU 环面网格 vs GPU 胖树	左：芯片只连邻居、边缘绕回；右：箱 8 卡、pod、spine 三层树	4:39	—
ZeRO 三级显存图	绿色优化器状态最大；最右列 120 → 31.4 → 16.6 → 1.9	15:19	ZeRO
Stage 1 的四步	reduce-scatter 梯度 → 各更新一片 → all-gather 参数；数一数通信量	16:50	同上
FSDP 重叠时间线	三条流：CPU、GPU 计算、GPU 通信；all-gather 第 1 层与 forward 第 0 层重叠；空档是 bubble	23:27	PyTorch FSDP（应为）
A100 可装模型表	基线装不下 7B，Stage 3 约 50B；换 H200 乘 141 除 80	27:31	—
流水线 bubble 图	4 张卡、时间从左到右，任一时刻只有一格在算；下图 micro-batch 填空	32:09	应出自 GPipe
Megatron batch × pipeline 扫描	batch 大、段数多时利用率接近不做流水线；batch 小时迅速下滑	35:12	Narayanan et al., 2021
zero-bubble 调度图	B（往前传偏导）先排，W（权重梯度）填空隙	38:13	Qi et al., 2024
TP 的 MLP 切分与 f / g	A 列切、B 行切；f 前向恒等反向 all-reduce, g 相反	40:15	Megatron-LM
profiler 显存曲线	底部平的是参数与优化器状态，红色驼峰是激活，峰值在反传刚开始	45:19	—
激活显存四行表	34 加注意力项 → TP 只除 24 与第二项 → 加 SP 全除 T → 加重计算剩 34/T	51:00	Korthikanti et al., 2022
并行策略总表	红字是每种的缺点；找不到一行没有红字	1:01:39	—
算账图	横轴每卡 batch，纵轴利用率，虚线是效率边界；FSDP 曲线在小 batch 处掉下去，加 TP 往左推	1:04:43	—
Megatron 规模扫描 · Llama 3 配置表	TP 到 8 停、PP 递增、DP 最后 6；Llama 3 三阶段各自的 TP / CP / PP / DP	1:09:58 · 1:15:03	Narayanan et al., 2021 · Llama 3

提到的工作

名称	在本讲里的作用
ZeRO (Rajbhandari et al., 2019)	三级分片的出处；120 → 1.9 的显存图
PyTorch FSDP (Zhao et al., 2023)	ZeRO-3 的 PyTorch 实现；通信与计算重叠的时间线图（应为这篇）
Megatron-LM (Shoeybi et al., 2019)	tensor parallel 的列切 / 行切与 f / g 对偶
Efficient Large-Scale Training with Megatron-LM (Narayanan et al., 2021)	讲者反复引用的"老论文": batch × pipeline 扫描、TP 到 8 停、DP 降到 6、利用率平坦
Reducing Activation Recomputation (Korthikanti et al., 2022)	34·sbh 的激活公式、sequence parallel、选择性重计算
GPipe (Huang et al., 2019)	流水线 bubble 与 micro-batch 的经典图（应出自此）
Zero Bubble Pipeline Parallelism (Qi et al., 2024)	把反传拆成 B 与 W，填满流水线
DeepSeek-V3 (2024)	前后向交错的流水线调度 (DualPipe)；64 路 EP；PTX 轶事
DeepEP	DeepSeek 的 EP 派发库，字幕作 "DPP"
Hybrid EP (NVIDIA)	同类的低层 EP 派发实现，只点了名
Megatron-Core 的 MoE 并行指南	"EP 优于 TP"的理由与五条并行准则
Megatron Bridge	NVIDIA 发布的各模型推荐训练配置与基准 (Mixtral、Qwen3、Nemotron 等)
Ring Attention (Liu et al., 2023)	context parallel 的原始论文，在 TPU 网络上验证
OLMo · Dolma (AI2, 2024)	7B 纯 FSDP 训练的例子
DeepSeek LLM (V1)	ZeRO-1 + TP + SP + PP
Yi (01.AI, 2024)	ZeRO-1 + TP + PP
Llama 3 (2024)	分阶段并行配置全披露；148 次 GPU 故障
Gemma 2 (2024)	TPU 上 FSDP + TP + SP、不用流水线
Mixtral 8x22B · Nemotron 3 Super · Qwen3	Megatron Bridge 配方里的 EP / PP / TP 数字
An Empirical Model of Large-Batch Training (McCandlish et al., 2018)	critical batch size 的出处（应为这篇）
Groq · Huawei Ascend 910 (CloudMatrix 384)	全 SRAM 与全光互联：两种"暴力换效率"的硬件路线
Google 新一代 TPU（应为 8i / 8t）	当天发布，拓扑转向全互联；趋同进化的例子
Bill Dally 与 Jeff Dean 的对谈	GPU 适合 MoE、TPU 适合稠密切分

术语对照

English	中文
intra-node / inter-node	节点内 / 节点间：一台机器内的卡之间 vs 机器之间，前者快一个量级
collective (communication)	集合通信：多卡一起参与的一次通信原语，记账的基本单位
all-reduce	全归约：每卡都拿到所有卡张量的逐元素和
reduce-scatter	归约分散：求和后每卡只拿结果的一个分片
all-gather	全收集：把各卡的分片拼成整体，每卡一份
all-to-all	全交换：每卡把数据按目的地切开分别发送；MoE 派发用
point-to-point	点对点：只在两张卡之间传
toroidal mesh (torus)	环面网格：TPU 的互联，只连邻居、边缘绕回
fat tree / spine switch	胖树 / 脊交换机：GPU 集群的分层全互联
NVLink	NVIDIA 节点内 GPU 直连的高速链路；"一箱 8 卡"的边界
PTX	NVIDIA GPU 的中间汇编，机器码级指令
data parallelism (DP) / DDP	数据并行 / 分布式数据并行：切 batch，每卡一份模型
sharding	分片：把一个张量切成若干份分到各卡
optimizer state	优化器状态：Adam 的一阶矩 m、二阶矩 v，加 fp32 主权重
ZeRO stage 1 / 2 / 3	逐级分片优化器状态 / 梯度 / 参数
FSDP (fully sharded data parallel)	全分片数据并行：PyTorch 对 ZeRO-3 的实现
communication-computation overlap	通信与计算重叠：预取下一层参数，把通信藏在计算底下
bubble	空泡：流水线或通信等待中设备空转的时间
critical batch size	临界 batch size：超过它再加样本不如多走一步
model parallelism	模型并行：切模型本身，卡间传激活
pipeline parallelism (PP) / stage	流水线并行 / 段：按层切，每段若干层
micro-batch	微批：把一步的 batch 再切小，让流水线各段同时有活干
1F1B	一前一后交错的流水线调度，zero-bubble 论文的基线
zero-bubble pipelining	零空泡流水线：把反传拆成 B（传偏导）和 W（算权重梯度）
tensor parallelism (TP)	张量并行：切矩阵

English	中文
column-wise / row-wise split	按列切 / 按行切
sequence parallelism (SP)	序列并行：把 TP 切不到的轻算子激活沿序列轴分片
context parallelism (CP) / ring attention	上下文并行 / 环形注意力：长序列切到多卡，K、V 环形传递
expert parallelism (EP)	专家并行：MoE 的 expert 分到不同卡
dispatch / combine	派发 / 合并：EP 里 token 发往 expert、结果送回 的两次 all-to-all
router	路由器：MoE 里给 token 选 expert 的小网络，TP 下全卡复制
activation memory	激活显存：为反传保留的每层中间结果
(selective) activation recomputation	(选择性) 激活重计算：不存、反传时重算，只对便 宜的部分做
residual	残差输入：每个子层的输入要留给反传
gradient accumulation	梯度累积：多个小 batch 的梯度累加后再更新，攒 出大 batch
3D / 4D parallelism	同时使用 DP、TP、PP，再加 EP 或 CP
utilization / MFU	利用率 / 模型算力利用率：实际 FLOP/s 占峰值的 比例
roofline	屋顶线：第 5 讲的模型，这里横轴换成了 batch size
long-context extension	长上下文扩展：预训练末尾把序列拉长的阶段
fault tolerance / redundancy	容错 / 冗余：应对训练中的 GPU 故障

字幕勘误

"DPP" → DeepEP; "deep TPU AI and T / PPU AI / TPU 8T" → 应为 TPU 8i / 8t; "zero stage" → ZeRO stage; "reduced scatter" → reduce-scatter; "paralyzed / paralyzing" → parallelized / parallelizing; "MOE" → MoE; "8X E E100" → 8× A100 (提问); "P200" → H200; "Dolma ... was a 7B model" → OLMo 7B (讲者随即自纠); "NeMo Tron 3 Super" → Nemotron 3 Super; "8x24 22B" → 8x22B; "Qwen 3" → Qwen3; "scaling loss" → scaling laws; "roof line" → roofline; "Matei and Deepak" → Matei Zaharia 与 Deepak Narayanan; "5 AS / H" → $5 \cdot a \cdot s/h$ (乘在 sbh 上); "conversion evolution" → convergent evolution; "Virgo network" 未能核实。

带走的问题

1. 用第 3—4 节的账给一个 70B 模型在 512 张 80 GB 卡上记账：
DDP、ZeRO-1 / 2 / 3 各需每卡多少显存？再按 $34 \cdot sbh/t$ 估每层激活（比如 $s = 8k$ 、 $b = 1$ 、 $h = 8192$ 、80 层、 $t = 8$ ）——哪一项先把卡撑爆？
2. ZeRO-3 比 DDP 多一次 all-gather 却"几乎免费"，条件是什么（第 10 节的 $B_{tok} \geq F/W$ ）？带宽不变、下一代 GPU 算力翻倍，这个阈值怎么变？它和"batch size 是资源"是同一件事吗？
3. Pipeline 通信最省却有 bubble，TP 无 bubble 却通信最饥渴——为什么 TP 停在 8、pipeline 跨节点？如果 NVLink 域扩大到 72 卡，或者像 TPU 那样整个 pod 是一张网格，配方会怎么变？Gemma 2 的"不用流水线"能推到 Llama 3 的规模吗？
4. EP 与 TP 在 MoE 里的冲突：attention 想要高 TP，MLP 想要低 TP。DeepSeek-V3 选 EP 64、不用 TP，Qwen3 选 EP 32 / TP 2，Mixtral 选 EP 8 / TP 4——各自的 expert 数、expert 大小与 attention 形状怎么解释这些选择？
5. "重计算换显存、显存换 batch、batch 换利用率"：把它和 FlashAttention（第 5 讲）以及 zero-bubble 的 W 延后放在一起，能否给出统一的表述——什么时候多做 FLOPs 反而更快？critical batch size 给这条路线设了什么上限？

CS336 第 9 讲 | Scaling laws (上) (Scaling Laws I)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 9 讲，课程日程表标注 2026 年 4 月 27 日；scaling laws 的第一讲（第 11 讲是进阶篇），中间隔着 Percy 的第 10 讲推理 视频：
<https://www.youtube.com/watch?v=Q15rhEWZPQ4> (1:17:57；英文字幕为自动生成，人名和论文名错得多，见文末勘误) 讲者：Tatsunori Hashimoto（全程；多处以第三人称提到 Percy——"Percy 第一讲说过 FLOPs 约等于数据乘参数""Percy 爱说 scaling law 是设计出来的""下一讲 Percy 讲推理"）课程主页：<https://stanford-cs336.github.io/> · 本讲围绕的论文：Kaplan et al., 2020 (OpenAI 的 scaling laws) · Hoffmann et al., 2022 (Chinchilla) · Hestness et al., 2017 (神经网络 scaling 的源头) · Porian et al., 2024 (Kaplan 与 Chinchilla 为何不一致) · Besiroglu et al., 2024 (Epoch AI 重拟合 Chinchilla)

一句话：一次大 run 动辄数百万美元，不可能在上面调参，所以要把所有优化搬到小尺度，再靠一条"简单、稳健的规律"外推上去——这条规律就是 scaling law：把 loss 和数据量、参数量、算力画在 log-log 坐标上，几乎总是一条直线（幂律），只是斜率约 -0.1 ，比统计课里均值估计的 -1 慢得多，像是在十维空间里做非参数回归；架构、优化器、深宽比、MoE 稀疏度、batch size、学习率都能这样研究，而且一个反复出现的现象是"干预只改截距、不改斜率"，所以小尺度上选出的最优往往就是大尺度上的最优；Chinchilla 用三种方法（lower envelope、IsoFLOP、参数拟合）得到"算力翻倍，参数和 token 各翻 $\sqrt{2}$ 倍"、每参数约 20 个 token，而 Kaplan 之所以得出"该训巨型模型" ($N \propto C^{0.73}$)，只是因为不数 embedding 参数、warm-up 太长、batch 固定太大这三个细节——scaling law 是设计出来的，不是转个曲柄就出来的；今天做产品的人则故意"过训练"，因为算力大头花在服务而不是训练上。

时间轴

时间	内容
0:05	开场：暂别系统部分；两讲 scaling laws，本讲讲基础，第 11 讲讲开源模型技术报告、 μ P、优化器
1:05	场景：一万张 B200 用一个月，怎么保证唯一的一次大 run 成功；scaling law 作为"信仰"与工程工具

时间	内容
4:07	历史：generalization bound 就是理论版的数据 scaling law；1993 年 Cortes 与 Vapnik 在贝尔实验室拟合学习曲线外推
6:39	Banko 与 Brill、2012 年机器翻译的 Pow3 / Pow4 函数形式、Hestness 2017——2017 年就能预见今天
11:19	现代 LM 的各种 scaling law：算力 / 数据 / 参数对 loss，下游指标是 sigmoid；data scaling law 的定义：log-log 上一条直线
15:54	三页统计：均值估计 σ^2/n 斜率 $-1 \rightarrow$ 神经网络约 $-0.1 \rightarrow$ 非参数回归 n 的 $-1/D$ 次方；问答："模型比数据大"指幂律区
21:29	用数据 scaling law 做决策：data mixing laws、DataDecide、重复数据最多 4 个 epoch、无限算力下的集成
27:05	干预只改截距不改斜率；数据过滤随规模变松；问答：坐标轴与"小范围看不出幂律还是指数"
30:10	模型工程：LSTM 对 Transformer、Tay 等的架构 scaling 研究（Performer 差、GLU 好）、SGD 对 Adam
35:43	层数与 aspect ratio 这类 scale-invariant 量；参数怎么数；MoE 稀疏度的 scaling
40:47	batch size：critical batch size 的两个区间、估计步骤、随 loss 的幂律；
48:26	learning rate：1/width 与 μP 两种哲学
51:00	上游困惑度对下游：困惑度最好的不是最好的模型；标准流程；问答：单次 run、直接拟下游、train 与 test loss
56:38	Chinchilla 问题：更多数据还是更大模型；Rosenfeld 与 Kaplan 的联合函数形式；Kaplan 的 0.73 / 0.27
1:02:13	Chinchilla 的三种方法：lower envelope 得 67B、IsoFLOP 得 63B、参数拟合
1:06:48	Kaplan 与 Chinchilla 为什么不一致：参数计数、warm-up、batch size 三步；Pearce 与 Song 的模拟
1:12:22	方法三的悬案：Epoch AI 从图里抠数据重拟合，原文欠拟合，修正后回到 20 倍
1:14:27	你大概不想要 20 倍：算力大头在服务，产品模型要"过训练"；IsoFLOP 是长青工具；总结

核心内容

1. 为什么要 scaling laws：一次大 run 不能拿来调参

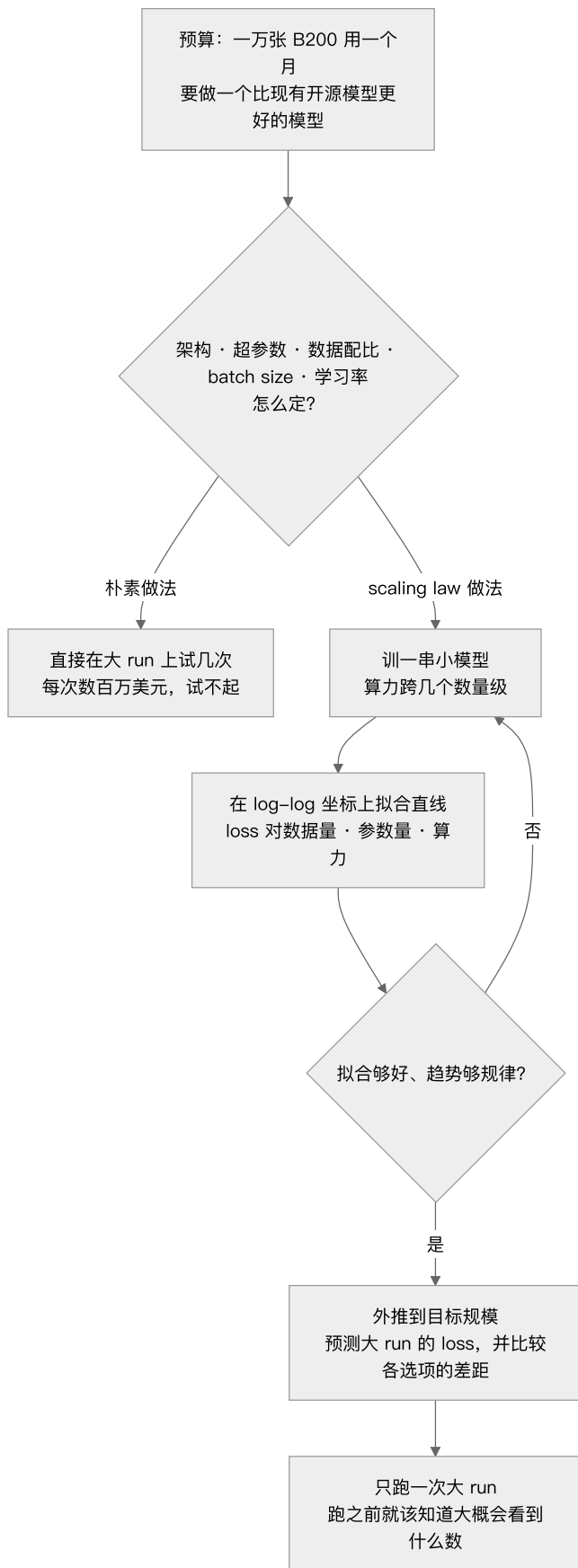


图 9-1 | scaling law 的工程闭环：所有优化在小尺度做，靠一条稳健的规律外推，大 run 只跑一次（自绘示意）· [► 看原幻灯片 2:36](#)

- **这一讲的位置：**系统部分（第 5—8 讲）暂告一段落，回到深度学习本身。本讲是基础篇：经典工作、基本思想、和机器学习 101 的联系；第 11 讲是进阶篇——读开源模型的技术报告，讲 μP 一类参数化和初始化，今年还加优化器。因为排课，中间隔着 Percy 的第 10 讲推理。
- **场景：**一位富有的朋友给你 10,000 张 B200 用一个月（第 1 讲算过它的 2.25 PFLOP/s），要你做一个很好的开源模型。基础设施有了（作业 2），预训练数据假设有（作业 4），剩下全是选择题：架构、超参数、数据配比……而这次 run 可能值数百万美元。照抄文献的最佳实践（第 3 讲）能得到不错的模型，但要超过最前沿，就得自己优化这些选择——抄别人的选择不可能超过别人。
- **scaling law 是什么：**把小尺度模型的表现外推到大尺度的简单预测规则。大实验室做 scaling 的人把它当"生活方式"甚至"信仰"，它有时也相当刁钻。朴素做法是在大 run 上调超参数，太浪费；正确做法是所有优化都在小尺度做，只要小尺度和大尺度之间有一条简单、稳健的规律连着，外推就有信心（图 9-1）。
- **和 CME295 的关系：**CME295 第 4 讲把 Chinchilla 当结论用（每参数约 20 个 token）；这一讲讲它是怎么拟合出来的、为什么 Kaplan 得出了相反的结论、以及为什么你可能并不想要这个 20。

2. 历史：scaling law 不是新东西，1993 年就有人在做

- **理论那一头：generalization bound（泛化界）。**机器学习理论一直在问"我的模型会有多好"，答案是泛化界：在一个有限假设类上，测试误差最多比训练误差差某个量，而这个量依赖样本数——它本身就是一条"loss 上界随训练集增长怎么变"的理论曲线。讲者说理论家看 scaling law 会有回家的感觉：它是经验版的 sample complexity（样本复杂度：达到某个误差要多少样本）。
- **最早的 scaling law 论文：**贝尔实验室的 Corinna Cortes、Vladimir Vapnik 等做严肃理论的人，1993 年就问过——在大数据集上训练分类器太贵，能不能在小样本上拟合分类器、对误差率的衰减拟一条曲线、用它估计大样本的表现？讲者说这几乎就是字面意义上的数据 scaling law。

小注：应为 Cortes, Jackel, Solla, Vapnik, Denker 的 Learning Curves: Asymptotic Values and Rate of Convergence (NeurIPS 1993)；讲者只报了人名和年份。

- **NLP 那一头：**Banko 和 Brill 是"与其研究算法不如去收数据"的经典引用——各种方法的表现都随数据量以可预测的方式变好。2012 年一篇机器翻译学习曲线的论文（字幕作 "Collobert et al."，应为 Kolachina et al., 2012）专门研究"BLEU 随数据量该用什么函数形式"，结论是三参数和四参数的幂律（ $Pow3 / Pow4$ ）——和今天用的一模一样。
- **神经网络 scaling 的源头：**Hestness et al., 2017（字幕作 "Hassabis"）。讲者每次讲 scaling law 都提它，因为它的引用量配不上它的前瞻性：2017 年就对语音识别、机器翻译、语言模型等多种系统拟合了数据量的幂律，远早于 OpenAI；还讨论了 emergence（涌现：accuracy 比 loss 不连续得多，能力像是突然出现）、按算力 scaling、以及"既然靠算力 scaling，系统优化的速度就会变成准确率"。讲者的感慨：2017 年认真读了它，今天的很多现象当时就能预见。
- **问答：这些幂律是纯经验的还是有依据？** 本质上是曲线拟合，没有"只能是幂律"的金科玉律；但理论整天在算误差率如何随各种量衰减，物理学家又爱想极限行为，两者都是候选函数形式的来源。第 3 节会给一个幂律为什么自然的推导。

3. 数据 scaling law: log-log 上的直线，以及幂律从哪里来

log-log 图上一条直线
loss 正比于 n 的 $-\alpha$ 次方
斜率 $-\alpha$ 就是学习速度

参数模型：估均值 · 线性回归
误差 σ^2/n 或 d/n
斜率 -1

模型类更灵活，速率就更慢

非参数模型：估任意光滑函数
把单位立方体切成小格，每格取平均
 D 维时误差约 n 的 $-1/D$ 次方，斜率 $-1/D$

神经语言模型：Hestness · Kaplan 拟出的斜率
约 $-0.1 \cdot -0.3 \cdot -0.1$

读法：像在约 10 维空间里
做非参数回归
Bahri 等据此论证网络是非参数平滑器，讲者半信

以上都以还在幂律区为前提
接近不可约误差时曲线会拐平
要么模型远大于数据，要么把渐近线一起拟合再扣掉

- **现代 LM 的 scaling law 长什么样 (11:49)**：x 轴是 log 算力、log 数据量或 log 参数量，y 轴是 log 测试 loss，各种“资源”在对数轴上都把 loss 拉成直线。更奇特的也有：下游 benchmark 的准确率对算力呈 sigmoid；预测类工作把日期放在 x 轴，能力的上包络居然是线性的。LM 的表现随规模和资源的变化，比直觉里规则得多。
- **最简单的一种：数据 scaling law**：固定训练流程，模型（默认）远大于数据集，按规律增加数据量，看误差怎么降。超参数调好时它应当单调，从随机猜测一路降到任务的**不可约误差**（irreducible error，或 noise floor：任务本身的熵，再多数据也降不下去），整体是 sigmoid 形。
- **经验观察**（Kaplan et al., 2020 的图；作业 3 里你会自己画一张）：模型远大于数据、增加数据量，log-log 坐标上的点排成一条很清楚的直线。log-log 上的直线叫 scale-free 或幂律（power law）关系，含义有两条：误差按多项式速度衰减；而且离渐近线还很远——接近渐近线时曲线会拐平。

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \mathbb{E}[(\hat{\mu} - \mu)^2] = \frac{\sigma^2}{n} \Rightarrow \log \text{err} = -\log n + \log \sigma^2$$

x_i 是从方差为 σ^2 的高斯分布抽的 n 个样本， $\hat{\mu}$ 是样本均值；均方误差是 σ^2/n ，取对数后是 $\log n$ 的线性函数，斜率 -1 ——这就是一条 scaling law，“统计课三页幻灯片”的第一页。

- **幂律为什么自然**：任何形如“常数加 n 的 $-\alpha$ 次方”的误差，扣掉常数项画在 log-log 上都是直线。参数估计（估均值、线性回归）的经典速率是 $1/n$ （回归是 d/n ），所以斜率应当是 -1 。
- **但神经网络的斜率是 -0.1 左右**：讲者给的三个拟合值——Hestness 的两个约 -0.1 和 -0.3 ，Kaplan 的约 -0.1 。仍是多项式衰减，但比估均值慢得多。哪种估计问题会这么慢？非参数问题。

$$\text{err}(n) \approx a n^{-\alpha} + L_{\infty}, \quad \alpha_{\text{parametric}} = 1, \quad \alpha_{\text{nonparametric}, D} \approx \frac{1}{D}, \quad \alpha_{\text{neural LM}} \approx 0.1$$

n 是样本数， L_{∞} 是不可约误差， α 是 log-log 图上直线斜率的绝对值。第二页幻灯片的推导：在单位立方体里估一个任意光滑函数 f （观测带高斯噪声），最笨的估计是把空间切成边长为 n 的 $-1/4$ 次方的小格、

每格取平均——二维时有 $\sqrt{n}$ 个格子、每格 $\sqrt{n}$ 个样本，误差约 $1/\sqrt{n}$ ；推广到 D 维，误差约 n 的 $-1/D$ 次方，斜率 $-1/D$ 。

- **一种读法：**斜率 -0.1 意味着这些网络像是在约 10 维空间里做非参数回归（最近邻那一类估计器），这就是它们从数据里学习的速度。

Bahri 等人（字幕作 "Barri"）走得更远，主张 scaling 指数字面上就说明网络是非参数平滑器，还用内在维度的估计佐证；讲者觉得有意思，但证据有点单薄，不完全买账。

小注：Hestness 报告的指数按任务不同：机器翻译约 -0.13 、图像分类约 -0.31 、词级语言模型约 -0.07 ；Kaplan 的 $\alpha_N \approx 0.076$ 、 $\alpha_D \approx 0.095$ ——都对得上讲者"约 -0.1 到 -0.3 "的说法。Bahri et al., 2021 的论文题为 Explaining Neural Scaling Laws（应为）。

- **问答："模型比数据大"是什么意思？** 不是参数数和 token 数一比一。模型相对数据太小时会进入不可约误差区——这个模型类已经把数据拟合到头，再加数据也没用。研究 scaling law 通常只看幂律区，要么就把渐近线明确拟合出来再修正。大多少都行，经验上比数据大 10 倍左右就够。

```
# 拟合一条数据 scaling law, 再外推 (自己写的示意)
import numpy as np
n = np.array([1e7, 3e7, 1e8, 3e8, 1e9])          # 训练 token 数
L = np.array([4.10, 3.72, 3.40, 3.12, 2.88])      # 各自的验证 loss (示意数字)
slope, intercept = np.polyfit(np.log(n), np.log(L), 1) # log L = slope * log n + b
alpha = -slope                                     # 幂律指数, LM 上通常 0.1 左右
L_at = lambda n_new: np.exp(intercept) * n_new ** slope
print(alpha, L_at(1e11))                          # 外推两个数量级
# 若曲线已开始拐平, 应改拟合 L = a * n^(-alpha) + L_inf (非线性最小二乘),
# 否则直线会把不可约误差错当成斜率变小
```

这就是"经验版泛化界"的全部机械动作：取对数、拟直线、外推；唯一的判断是你是否还在幂律区。

4. 拿数据 scaling law 做工程决定：配比、重复、过滤，以及"只改截距"

- **单独一条数据 scaling law 用处有限：**它只说模型学得多快，能做预报，做不了别的。工程上想问的是：最优的数据配比（data mixture，

各来源按什么比例混) 是什么? 数据该不该**重复** (多个 epoch)? 只重复高质量的部分?

- **经典模型给的直觉**: 把数据 scaling law 看成经验版泛化界, 那么数据集的组成影响的是曲线的**截距** (offset), 而**斜率**由模型类决定, 与分布无关。玩具例子: 线性回归、两种数据源, 只用一种误差怎么都高, 混起来才好——把截距写出来, 就能看到数据多样性为什么有帮助。
 - **Data mixing laws (23:32)**: 新闻和 Wikipedia 各用多少? 用很小的模型、很少的数据, 把"配比 → 表现"拟合成函数, 逐步放大模型看趋势怎么变, 像外推数据 scaling law 一样外推到全量训练, 再取最优——"小算力上拟合、找最小值、放大"的最简单版本。
 - 现实更吵: 真做过数据配比的人会告诉你, 实际上是训一批小模型、直接挑最好的配比、放大, 根本不用 scaling law。
DataDecide 是这类做法的大规模实证, 结论正是小尺度直接选就行——和"截距变、斜率不变"自治: 斜率不变, 小尺度的最优配比就是大尺度的最优配比。
 - **重复数据 (25:33)**: 算力在涨, 数据没涨, 所以很多人关心数据重复多次会怎样。Scaling Data-Constrained Language Models 的结果: 标准训练配方下最多约 4 个 epoch 完全不吃亏; 再往上, 实际得到的 scaling 曲线 (深色) 明显差于"有新鲜数据时"的投影 (虚线); 也能写出一个修正过的函数形式来预测重复下的行为。
 - **推到极端: 无限算力 (26:03)**: 讲者和 Percy 合带的一位学生最近的工作——允许无限次 epoch、无限算力, 固定数据里最多能榨出什么? 不能一直重复 pass, 也不能一直把模型做大, 两者都收益递减, 只好转向模型集成 (ensembling) 之类的手段。图上红线是标准数据 scaling law, 正则化、集成等干预都让线整体下移, 斜率却惊人地相似。
- 小注: 应为 Kim, Kotha, Liang, Hashimoto 的 Pre-training under infinite compute (2025); 学生名字字幕不清 ("Perseus"), 推断为 Konwoo Kim。
- **本讲反复出现的一课**: 自己拟合过 scaling law 就会发现, 斜率极少改变, 干预通常只改截距。第 5 节的 SGD 对 Adam、LSTM 对 Transformer 还会再看到。
 - **数据过滤随规模而变 (27:36)**: 你我明天过滤数据一定过滤得很狠, 只留最高质量的——反正算力少, 训不完整个互联网; 算力多的人不

想在高质量数据上反复重复，过滤器就越来越松，训到质量越来越低的数据上。"数据质量"这种常被当成静态的东西其实是动态的，最优过滤器不随规模固定（第 13—14 讲再回来）。

- **问答：无限算力那张图为什么两个轴都像线性？** x 轴其实按倍增取点，是对数轴，画得不好；y 轴范围太小，线性和对数看不出差别，拟的仍是幂律。顺带的警告：只有一小段算力范围时根本分不清幂律还是指数——放大到足够近什么都像直线（Taylor 展开）。看到任何函数形式的声明都要保留一点怀疑。

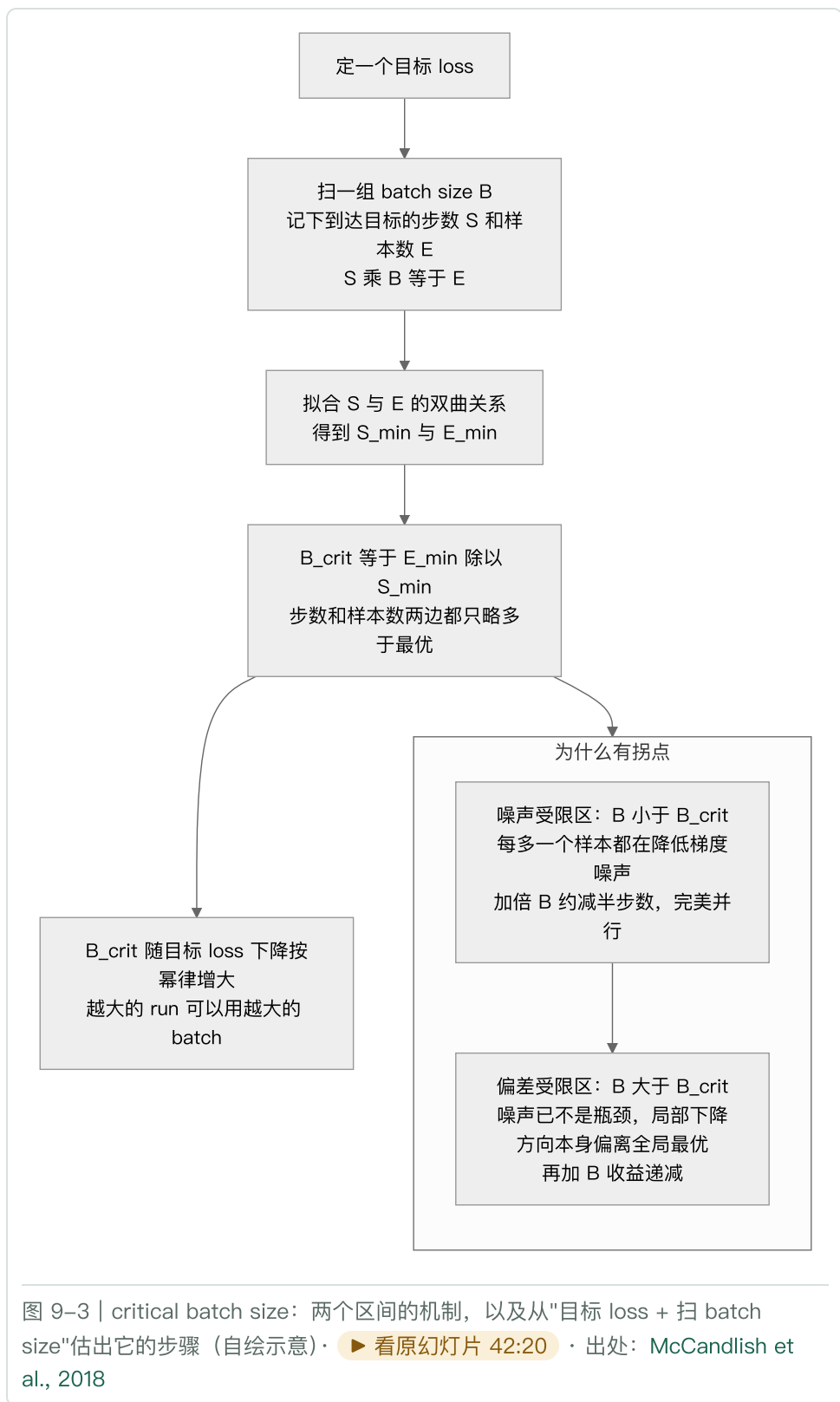
5. 模型工程的 scaling law：架构、优化器、深宽比、参数怎么数、MoE 稀疏度

- **换个问法：**现在要设计一个大模型。也许你是激进派，觉得 LSTM 才是未来；或者想用 SGD 而不是 Adam；资源有限时还要在"训久、训大、多收数据"之间取舍。scaling law 给这些取舍一个量化的做法。第 3 讲讲超参数时说的是"看别人怎么做"，这里试着给第一性原理的答案——前提是你信 scaling law。
- **Transformer 真的比 LSTM 好吗 (31:40)：**暴力做法是训一个 GPT-3 大小的 LSTM；scaling law 做法是在一系列算力上训 Transformer 和不同层数的 LSTM。Kaplan 的图上 LSTM 截距明显更高，斜率可能也更差——这就足以决定放大 Transformer。今天每篇架构论文（Mamba、Gated DeltaNet，第 4 讲）都有这么一张"vanilla Transformer 对我们的模型"的图：你的曲线要么整体在下面，至少斜率不能更差——斜率更差意味着放大到后来一定输。
- **Tay 等在 T5 系列上的架构 scaling 研究 (33:10)：**同一套架构越训越大，比较各种改动。讲者喜欢它，因为它在远小于今天的算力上就抓到了前沿模型真正采用的改动：Performer（一种高效注意力）scaling 很差，今天没人用；gated linear unit 全程红线压绿线，今天人人都在用（第 3 讲的 SwiGLU）；Switch Transformer 趋势不错（最大的 run 不知怎么了）；mixture of softmaxes 按趋势看也有效，只是今天不用了。所以很多人把 scaling law 当范式："干预在 scaling law 上看不出来，就不是好干预"。
- **SGD 对 Adam**（Hestness 的图，递归 highway 网络做语言模型）：又是那个神秘现象——截距不同、斜率几乎一样。固定数据和模型时，连"换优化器"这么大的干预都很少改变斜率；讲者每次看到都意外，但确实如此。

- **层数与 aspect ratio (35:43)**: 第 3 讲说深宽比随便取个合理值就行; scaling 分析可以看得更准。按层数画曲线: 1 层极差, 多于 1 层就意外地有竞争力, 但每个算力档位上更多层都更好。层数不是 **scale-invariant** (尺度不变) 的量——模型变大就该更深; aspect ratio (d_{model} 除以层数) 才可能是: Kaplan 的扫描里各种模型大小的最优值都在每层约 100 的 d_{model} 附近或略小, 深模型略偏向更小的比值, 但谷底基本不动。这就是"固定 aspect ratio 整体放大"这条策略的信心来源。
- **参数不是生而平等的 (37:14, 本讲要回来两次的大坑)**: Kaplan 发现把 embedding 参数算进去, 按深度画的曲线很怪, 于是只数 **非 embedding 参数**——理由是"这些才是做计算的参数"。第 9 节会看到它对结论的影响。要点是 scaling law 不是魔法。Percy 爱说: 跨尺度的可预测性是**设计出来的** (engineered) ——选对 x 轴、把超参数设对, 才可能跨好几个数量级都可预测; scaling law 研究者的日常就是在找那个"正确的方式"。
- **MoE 的 scaling (38:46)**: MoE 已是主流 (第 4 讲), 总参数和激活参数解耦后, "一个参数值多少"要重新想。Apple 和 MIT 的分析把 loss 画成总参数和激活参数的函数: 总参数越多, 最优稀疏度越高 (颜色越来越深); 固定算力可以只变稀疏度, 问这份算力该花在激活参数上还是更高的稀疏度上; 固定激活参数、只加不激活的总参数, loss 也在改善——没被激活的参数同样在帮你降 loss。这些量都有可写出的函数形式和可预测的规律。

小注: 应为 Abnar et al., 2025 的 Parameters vs FLOPs: Scaling Laws for Optimal Sparsity for Mixture-of-Experts Language Models; 讲者只说"Apple 和 MIT 的人"。

6. 两个非得重调的量: batch size 与 learning rate



- **为什么单挑这两个：**真训新的大模型，上面的大多数东西你都不会动——没人激进到换 LSTM，多半选个 DeepSeek 风格的 MoE。但 batch size 和 learning rate 每个 run 都得重新定，而且耦合：改一个就得改另一个。batch size 还有系统那一面（第 7—8 讲）：数据并行

靠大 batch 才并行得开，所以要问"batch 能开到多大才开始吃亏"，以及这个上限怎么随模型大小变。

- **critical batch size** (临界批大小, 42:20): batch 从小往大加, 起初是完美回报——每多一个样本都在降低 SGD 一步的梯度噪声, 此时你是**方差受限** (variance-limited, 或 noise-limited) 的; 加到梯度的噪声尺度附近, 瓶颈变成**偏差** (bias): 梯度下降只看目标函数的局部结构, 局部下降方向和全局最优本来就有分歧, 噪声再低也只能朝那个方向走, 收益递减。交界处就是 critical batch size——batch 尽可能大、又不付出巨大效率损失的折中点。
- **怎么估** (44:23, 理论来自目标函数局部二次近似的一堆计算, 这里只记机械步骤, 图 9-3): 定一个目标 loss; 扫若干 batch size; 记下各自到达目标所需的步数 S 和样本数 E (S 乘 B 就是 E)。OpenAI 的人论证 S 和 E 满足下面的双曲关系: 想少用样本就得多走步, 反之亦然。让两边平衡, 解出 $B_{\text{crit}} = E_{\text{min}} / S_{\text{min}}$, 其中 S_{min} 、 E_{min} 是对观测数据拟合出的"最少步数"和"最少样本数"; 这个 batch 下步数和样本数都比各自的最优多一点, 但两边平衡。论文还给了另一种估法: 梯度协方差的迹除以梯度范数的平方 (噪声尺度), 感兴趣去读原文。

$$\left(\frac{S}{S_{\text{min}}} - 1\right)\left(\frac{E}{E_{\text{min}}} - 1\right) = 1, \quad B_{\text{crit}} = \frac{E_{\text{min}}}{S_{\text{min}}}, \quad B_{\text{noise}} = \frac{\text{tr}(\Sigma)}{\|G\|^2}$$

S 是到达目标 loss 的优化步数, E 是为此处处理的样本数, S_{min} 、 E_{min} 是两者各自的下限 (拟合参数); B_{crit} 是两边平衡时的 batch size; B_{noise} 是用梯度的协方差矩阵 Σ 和真实梯度 G 算出的噪声尺度, 是同一个量的另一种估法。

```
# 估 critical batch size (McCandlish 等的机械步骤, 自己写的示意)
from scipy.optimize import curve_fit

target = 3.0                                     # 想达到的 loss
S, E = [], []                                    # 到达目标所需的步数、样本数

for B in [64, 256, 1024, 4096, 16384]:
    steps = train_until(loss=target, batch_size=B)
    S.append(steps); E.append(steps * B)         # S·B = E
# 拟合 (S/S_min - 1)(E/E_min - 1) = 1, 等价于 S = S_min · E / (E - E_min)
f = lambda E, S_min, E_min: S_min * E / (E - E_min)
(S_min, E_min), _ = curve_fit(f, E, S)
```

$B_{\text{crit}} = E_{\text{min}} / S_{\text{min}}$

两边都只略多于最优的折中

点

同一个目标 loss 换几个 batch size 各跑一遍、拟合两个常数、相除——它贵在“每个目标 loss 都要扫一遍”，所以下面要问它怎么随目标 loss 变。

- **为什么它出现在 scaling 这一讲 (46:56)**：critical batch size 是个合理的选择，那它怎么随目标 loss 变？目标 loss 可以理解成算力——想要更好的模型，该用多大的 batch？答案：loss 越低，critical batch size 越大，而且又是一条幂律。直觉上说得通：越接近最小值，优化的是越细微的差别，噪声影响越大，方差降低就越值钱。所以大规模 run 可以、也应该用很大的 batch。

$$B_{\text{crit}}(L) \approx \frac{B_*}{L^{1/\alpha_B}}$$

L 是目标 loss， B_* 、 α_B 是拟合常数；loss 每降一点，可用的 batch 按幂律变大。

小注：Kaplan et al., 2020 对 WebText2 拟出的常数是 $B_* \approx 2 \times 10^8$ 个 token、 $\alpha_B \approx 0.21$ ，讲者没有念数字。

- **learning rate (48:26, 第 11 讲细讲)**：最优学习率随规模漂移。标准图景：只做宽度 scaling 的 MLP，模型越宽，最优学习率越小——参数更多、一次改的东西更多，每步就该走得小一点；常见经验法则是按 $1/\text{width}$ 缩小。另一派给网络重新参数化：改初始化尺度、改各部分的优化器步长，强行让学习率最优点在各尺度上不动，这就是 μP (maximal update parametrization) 一类方法——有人大获成功，有人效果一般。
- **两种哲学**：一种是估计最优点怎么随规模移动，然后预测（最优点的移动相当规律）；另一种是重参数化让最优点不动，小尺度选好学习率直接上。两种都有大规模 run 成功用过；讲者的观感是 scaling law 那一派略多。

7. 上游困惑度不等于下游，以及 scaling law 的标准流程

- **一张让人清醒的图 (51:00, Tay 等的架构论文)**：负困惑度对参数量是一条漂亮的直线。你于是决定发布困惑度最好的那个模型，结果它不是最好的：下游最好的是一个 32 层的变体，困惑度差得多。讲者说这是他见过上游到下游相关性最差的例子之一，但教训是普遍的：

scaling law 是困惑度那一侧的对象——干净、规律、可预测；困惑度到下游的迁移远没有看上去那么确定。

- 讲者的轶事：去各处做后训练的往届学生总抱怨，预训练的人把模型一交："困惑度很好，剩下是你的问题了"——而问题常常从预训练就开始了。别做那种人，要连迁移一起想。

小注：模型名的 "NL-32" 这种命名方式出自 Tay 等 2021 年的 Scale Efficiently (应为，讲者说取自"架构论文")；那篇的主张之一就是下游表现要看形状而不只看参数量。

- **标准流程 (52:31)**：大 run 跑之前，你应当大致知道会看到什么——不只是"能不能训起来"，而是几乎能预测最终 loss 的数值、换一个优化器能赚多少。做法就是图 9-1：训一批小模型，在 log-log 上拟直线，拟合够好就有信心差距会持续，然后部署到大 run。讲者称之为"朴素版"，但和实践相当一致——不然呢，凭空开一个大 run？
- **问答三则 (53:31)**
 - 每个点跑几次、取平均还是取最小？几乎所有点都是单次 run。困惑度非常干净：训练数据同质、量大，评测集也大，重跑一次差别在小数点后第二位。学习率和 critical batch size 的 scaling law 则噪声"相当可怕"，做方差缩减的人却不多。
 - 为什么不直接对下游指标拟合？可以，也有人做，但下游的问题就是上面那张图：一条规律的直线你有理由相信它延续；一条锯齿线你说不清是直线还是曲线，更谈不上外推。哲学是：先用低方差的量建立 scaling 规律，再另行建立或相信"它会迁移到下游"。
 - 拟 test loss 还是 train loss？讲科学规范时是 test loss (Kaplan 那样)；但除了无限算力、重复数据这类研究，预训练都是单 pass 的 SGD，泛化差距极小，train 和 val 几乎一样——讲者见过连 validation loss 都不算的预训练代码库。

8. Chinchilla：算力固定时，钱花在数据上还是模型上

同一批训练 run: 多种模型
大小, 每个都记录整条训练
曲线
x 轴 FLOPs, y 轴 loss

方法一 · lower envelope

取所有训练曲线的下包络
每个 FLOPs 值上最低的
loss 来自哪个 run

这些点的模型大小对
FLOPs 作图
log-log 直线 → 该预算下
67B

换一种独立的估法核对

方法二 · IsoFLOP

固定几档 FLOPs 预算, 按
3 倍递增
每档扫 N 与 D 的取舍: N
减半、D 加倍

每档的终点 loss 对 N 是一
条 U 形曲线
取最低点或拟二次曲线取最
低点

各档最低点连成直线 →
63B

再换一种

方法三 · 参数拟合

假设联合函数形式: L 是 N
与 D 的两条幂律之和
 $E + A/N^\alpha + B/D^\beta$

对全部 run 做曲线拟合估出
常数
再在 6ND 等于 C 的约束下
求最小值

图 9-4 | Chinchilla 用同一批训练 run 回答“该训多大”的三种办法，前两种对他们的预算给出 67B 和 63B（自绘示意）· [▶ 看原幻灯片 1:03:13](#) · 出处：Hoffmann et al., 2022

- **问题 (56:38)**：scaling law 最出名的用途，你未必亲手做过，但一定用过它的经验法则。有人递给你一笔算力，要花在训练上；Percy 第 1 讲说过 FLOPs 大致是数据量乘参数量（第 2 讲推出了系数 6）。那么要更多数据还是更大模型？一个极端很清楚：很小的模型灌海量数据，曲线早早变平（图上顶部那条深紫色线），纯属浪费——同样的 token 数不如去训那个大得多的黄色模型。要精确地优化，就得理解数据量和模型大小是**怎么一起**决定表现的。

$$C \approx 6ND$$

C 是训练总算力（FLOPs），N 是参数量，D 是训练 token 数；系数 6 来自第 2 讲的记账（forward 每参数每 token 2 次运算，backward 4 次）。本讲把它当约束条件：预算 C 给定，N 和 D 只能一个换一个。

- **联合函数形式 (58:10)**：Kaplan 和 Rosenfeld 几乎同时提出了大致等价的形式。Rosenfeld 的极简单：两条 scaling law 相加；Kaplan 的复杂一点，思想一样。检验它的办法是取极限：数据无限，数据项归零，剩下纯模型大小的 scaling law；模型无限，就剩纯数据的——别人递给你一条 scaling law，先把每个变量的极限都取一遍。Rosenfeld 等还证明了它**外推得住**：只在低算力、低数据的角落（图上的绿点）拟合，却能相当准确地预测高算力、大模型区域的误差。

$$\begin{aligned} L(N, D) &= E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} && \text{Rosenfeld \cdotp Chinchilla 方法三} \\ L(N, D) &= \left[\left(\frac{N_c}{N} \right)^{\alpha_N / \alpha_D} + \frac{D_c}{D} \right]^{\alpha_D} && \text{Kaplan} \end{aligned}$$

N 是参数量，D 是 token 数；第一行里 E 是不可约误差，A、B、 α 、 β 是拟合常数——N 或 D 趋于无穷时对应的项消失，退化成单变量的幂律；第二行是 Kaplan 的写法， N_c 、 D_c 、 α_N 、 α_D 也是拟合常数，两种形式在极限行为上一致。

小注：Chinchilla 论文对第一行拟出的常数是 $E = 1.69$ 、 $A = 406.4$ 、 $B = 410.7$ 、 $\alpha = 0.34$ 、 $\beta = 0.28$ ；Kaplan 的 $\alpha_N \approx 0.076$ 、 $\alpha_D \approx 0.095$ 、 $N_c \approx 8.8 \times 10^{13}$ （非 embedding 参数）、 $D_c \approx 5.4 \times 10^{13}$ token。课上没有念这些数字。

- **有了联合形式就能优化**：在 FLOPs 固定的约束下最小化 $L(N, D)$ ，一个简单的非线性优化。Kaplan 的解：N 正比于 C 的 0.73 次方，D 正

比于 C 的 0.27 次方（讲者念的时候翻了一下，随即更正）。含义是每参数的 token 数随算力下降——算力越多，越该训大模型。GPT-3 时代大家训数千亿、甚至万亿参数的稠密模型，部分就是被这条处方推着走的。

- **2022 年 DeepMind 的反转 (1:01:13)**：Hoffmann 等说这些预测错得离谱——图上那三颗星（当时的巨型模型）都太大了，正确的趋势是另一条蓝线，该训的是那颗青色的星（Chinchilla），小得多却好得多；结论你们都知道：每参数 20 个 token。讲者要细讲，是因为这场分歧说明 scaling law 不是转曲柄出答案的机器，它敏感、需要小心、要尊重过程。

小注：Chinchilla 论文图 1 里的三颗星是 GPT-3 (175B)、Gopher (280B)、Megatron-Turing NLG (530B)，青色的星是 70B 参数、1.4T token 的 Chinchilla；课上没有逐个念名字。

- **三种拟合方法 (1:02:13, 图 9-4)**：讲者欣赏 Chinchilla 用三种独立的办法估同一个量，相当于对建模假设做稳健性检查。写成 N 正比于 C 的 a 次方、 D 正比于 C 的 b 次方：方法一 $a = 0.50$ 、 $b = 0.50$ ；方法二 0.49、0.51；方法三 0.46、0.54，略有出入（第 9 节）；Kaplan 是 0.73、0.27。
 - **方法一 · lower envelope**（下包络）：每条彩色线是一个模型的训练曲线（loss 对已用 FLOPs）。取所有曲线的下包络——每个 FLOPs 值上任何 run 达到过的最低 loss——每个包络点对应一个模型大小，“模型大小对 FLOPs”散点是一条直线。Kaplan 也部分用了这办法。对他们的预算，答案是 67B。简单，但“下包络到底在哪”有不少棘手之处。
 - **方法二 · IsoFLOP**：现在最流行，也是讲者的最爱——容易、稳健。选几档 FLOPs 预算（按 3 倍递增，或 1、3、6），每档内只有一个自由度： N 和 D 的取舍——模型减半、数据加倍。一档里各 run 的终点 loss 对 N 是一条 U 形曲线；取最低点，或拟二次曲线取最低点；各档最低点对 FLOPs 连线，就是另一条预测：63B，和方法一非常接近。
 - **方法三 · 参数拟合**：既然有联合函数形式，最自然的做法就是拿一大堆 run 直接对这个 loss 曲面做曲线拟合，估出常数。最暴力，也最讲究——多变量的曲面怎么拟很关键，和方法一一样有自己的坑，不像 IsoFLOP 那么直截了当。


```

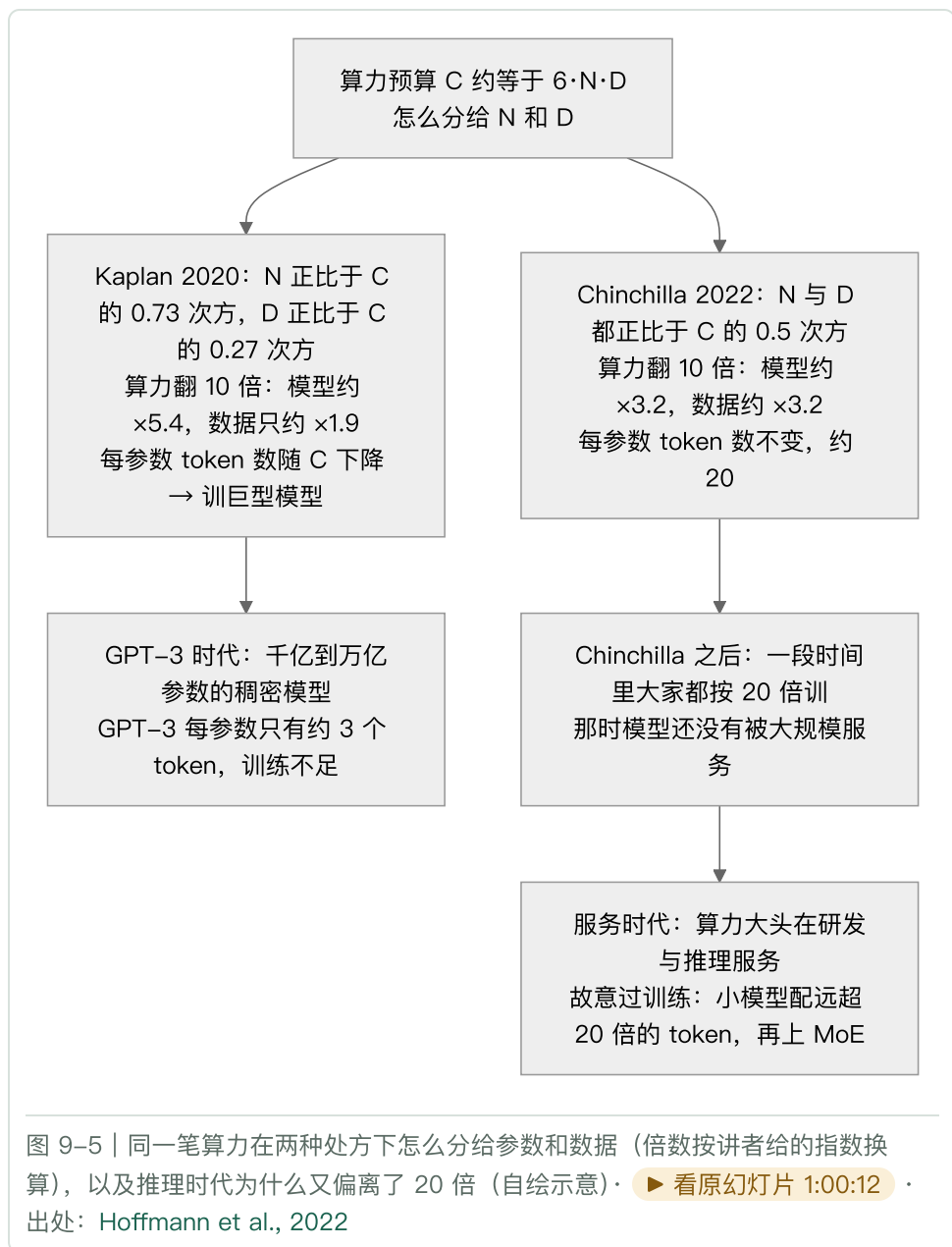
# IsoFLOP 扫描（方法二），自己写的示意
import numpy as np
n_opt = {}
for C in [1e18, 3e18, 1e19, 3e19, 1e20]:          # 预算按 3 倍
    递增
        Ns = np.logspace(7, 10, 12)              # 每档扫一串模
        型大小
        losses = [train(N=N, D=C / (6 * N)) for N in Ns]  # 固定 C, N 减
        半则 D 加倍
        a, b, c = np.polyfit(np.log(Ns), losses, 2)      # 对 log N 拟
        二次曲线
        n_opt[C] = np.exp(-b / (2 * a))                # 抛物线的最低
        点
        slope, _ = np.polyfit(np.log(list(n_opt)),
        np.log(list(n_opt.values()))), 1)
        print(slope)                                  # Chinchilla
        约 0.5, Kaplan 约 0.73
        D_over_N = lambda C: C / (6 * n_opt_at(C) ** 2)  # a≈0.5 时这
        个比值不随 C 变

```

每档预算只需要一个一维扫描，读出 U 形曲线的底，再看底部怎么随预算移动——斜率 0.5 意味着 D 与 N 同比例增长，每参数的 token 数是常数。

$$N_{\text{opt}} \propto C^a, \quad D_{\text{opt}} \propto C^b, \quad a + b = 1; \quad \text{Chinchilla: } a \approx b \approx 0.5 \Rightarrow \frac{D}{N} \approx 20, \quad \text{Kaplan: } a \approx 0.73, b \approx 0.27$$

a、b 是最优参数量和最优 token 数随算力增长的指数，由 $C \approx 6ND$ 知两者之和为 1；a = b = 0.5 时 D/N 是常数（Chinchilla 读出来约 20），a 大于 b 时 D/N 随算力下降（Kaplan：越有钱越该训大模型）。



9. Kaplan 和 Chinchilla 为什么不一致: 三个"小细节", 和方法三的悬案

Kaplan 2020 的复现
 N 正比于 C 的 0.73 次方

① 参数怎么数：把
embedding 和输出层参数算
回来

曲线移动一截

② 学习率 warm-up：小模
型在 warm-up 结束前根本
没收敛

再移一截

③ batch size：不再固定一
个大 batch，按模型大小调

与 Chinchilla 完全一致
 N 正比于 C 的 0.5 次方

另一个角度的解释

Pearce 与 Song：不训模型
用 Chinchilla 隐含的训练曲
线模拟 Kaplan 的设置
两个效应：算力尺度太低 +
不数 embedding 带来的非
线性

以及方法三的悬案

Epoch AI：Chinchilla 方法
三为何与一、二不合
从论文图里抠出数据重拟合
→ 原文欠拟合
修正后也回到每参数约 20
个 token

图 9-6 | 从 Kaplan 走到 Chinchilla 只需要改三个细节；两项后续工作从不同角度解释了同一个分歧（自绘示意） · [▶ 看原幻灯片 1:07:50](#) · 出处：Porian et al., 2024

- **谜题 (1:06:48)**: Chinchilla 的做法和 Kaplan 没有本质不同——不是 Kaplan 做了什么疯狂的事、Chinchilla 想出了什么高明得多的办法。两边都合理，预测却差这么远；事后看大家倾向于 Chinchilla 更对，你自己做一遍多半也得到接近 Chinchilla 的结果。为什么？这就是 scaling law 的"混乱现实"：它随实现细节、超参数、甚至 x 轴的选择而变。
- **Porian 等的三步复现 (1:07:50, 论文 Resolving Discrepancies in Compute-Optimal Scaling of Language Models, 末位作者是本校毕业的 Yair Carmon——字幕作 "Y Yi", 应为)**: 先按 Kaplan 的设置复现出 Kaplan 的结果，然后一次改一件事 (图 9-6):
 1. **参数怎么数**。Kaplan 排除了 embedding 参数 (第 5 节)，这还说得过去，但他们把最后一层 (softmax 前的输出矩阵) 也一起排除了——理由是它和 embedding 互为对偶、形状相同 (词表大小乘隐藏维度，对隐藏维度乘词表大小)。结果这一算一不算，对 scaling law 的形状有实质影响。
 2. **学习率 warm-up**。更像疏忽而非观点分歧：Kaplan 的很多模型太小，小到 warm-up 结束时都还没收敛，学习率设得很不理想。
 3. **batch size**。Kaplan 给所有模型固定了一个大 batch，对小模型是次优的；按模型大小调 batch 之后，结果和 Chinchilla 完全一致。
 - 每一步看着都是小差别，合起来就是 0.73 对 0.5 的鸿沟。
- **scaling law 某种意义上是下界 (1:10:21)**: 它说的是"沿用这套配方放大会得到这个"。配方本身有毛病——warm-up 离谱、batch 离谱——放大出来的就是一条坏的 scaling law。所以拟合用的小 run 要尽量贴近真正的大 run 该有的样子。
- **Pearce 与 Song 的模拟 (1:10:51, 字幕作 "Pearson and Song")**: 一个模型都不训。用 Chinchilla 的函数形式生成隐含的训练曲线，模拟"像 Kaplan 那样不数 embedding 参数、算力又小得多"会看到什么，再对比用总参数量的情形。两个效应：Kaplan 处在低得多的算力尺度，对小改动敏感；不数 embedding 参数带来的非线性，在那个尺度上足以搅乱结果。角度和 Porian 等略不同，但都指向"细节决定 scaling law"。
- **方法三的悬案 (1:12:22, 讲者觉得好玩)**: Chinchilla 推翻了 Kaplan，自己却留了个尾巴——方法三从没和方法一、二对上。0.46 / 0.54 看着和 0.5 / 0.5 差不多，作者也不以为意，但含义不小：方法一、二说参数和 token 同比例放大 (固定比值，20 倍由此而来)；

方法三的指数不同，意味着算力足够大时 token 会远远多于参数，渐近结论完全是另一回事。Epoch AI 的人拿不到原始数据和代码，就从论文的图里把数据点抠出来，重新拟合方法三的曲面。结果是原文欠拟合了自己的数据，重拟合的误差低得多；按重拟合的模型算，几乎正好回到每参数 20 个 token。讲者的评语：作者比他们自己知道的更正确——方法三的不合只是一次拟合失误，三种方法终究是稳健的。

小注：Besiroglu et al., 2024 的 Chinchilla Scaling: A Replication Attempt；摘要还指出原文报告的置信区间窄得不合理——那需要 60 万次以上实验，而他们可能只跑了不到 500 次。重拟合后的 α 与 β 大致相等（约 0.35 与 0.37），与方法一、二的“同比例放大”一致；原文的 $\beta = 0.28$ 明显偏小，正是 a、b 不对称的来源。

10. 你大概不想要 20 倍；IsoFLOP 是长青的工具

- **产品模型不在乎训练算力 (1:14:27)**：已是常识，但值得说清。对前沿实验室算力去向的外部调查显示，大头不在训练 run，而在研发和推理服务。既然是服务，你要的是小而能干的模型，不是训练最省却又大又臃肿的模型——所以要“过训练”（overtrained，讲者加引号：对产品来说这才是恰当的训练量）。
- **token 倍数的历史**：GPT-3 肯定训练不足，讲者说约每参数 3 个 token；Chinchilla 把它推到现代的 20；此后一段时间大家都按 20 训——那是模型很酷但还没被大规模服务的年代；服务一旦成真，就进入过训练时代，接着上 MoE，一切取舍都朝着推理成本优化（第 10 讲）。Chinchilla 重要，不是因为 20 比 1 是黄金比例（做研究也许是），而是它教会了我们怎么拟合、怎么看 scaling law。

小注：GPT-3 是 175B 参数、300B token，每参数约 1.7 个 token，讲者的“3”应是口头约数。Llama 3 8B 用 15T token，每参数近 2,000 个，是“过训练”的典型；CME295 第 4 讲的带走问题问过这一点，本讲给了答案：训练算力最优不等于总成本最优。

- **IsoFLOP 长青 (1:15:59)**：固定一档 FLOPs，扫其他所有自由度，读数一般都可靠。讲者的学生拿它做过扩散模型，第 5 节的 MoE 稀疏度研究也是 IsoFLOP 式的设计。拿不准“这些取舍怎么定”时，IsoFLOP 永远是好的默认。
- **总结**：投入的资源和得到的表现之间有一条 log-线性的规律，延伸到参数量、算力、MoE 的稀疏度……于是第 3 讲里那些“任意的选择”可以用证据来定，不必真去跑大模型。数据 scaling 最自然，只是指数有点怪；模型 scaling 让工程决策成为可能；scaling 整体上是一个稳

健的预测器。下一讲 Percy 讲推理，第 11 讲回来讲更进阶的 scaling 话题。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
三种 x 轴的 scaling law 三联图	算力、数据、参数各自在对数轴上都把 log loss 拉成直线；下方更奇特的 sigmoid 与"日期做 x 轴"	11:49	Kaplan et al., 2020
数据 scaling law 的直线与三个指数	模型远大于数据时点排成直线；17:24 读三个拟合斜率（Hestness 约 -0.1、-0.3，Kaplan 约 -0.1），和统计课的 -1 对照	14:20	Kaplan et al., 2020 · Hestness et al., 2017
重复数据的 scaling 曲线	深色实际曲线在 4 个 epoch 以内贴着虚线（新鲜数据的投影），之后分开	25:33	Muennighoff et al., 2023
无限算力：基线 与 干预	红线是标准数据 scaling law，正则化、集成把线整体下移，斜率几乎不变；x 轴其实是倍增的对数轴	26:03	Kim et al., 2025（应为）
LSTM 对 Transformer	不同层数的 LSTM 截距更高、斜率可能更差；架构论文标配的一张图	31:40	Kaplan et al., 2020
T5 系列的架构 scaling	Performer 曲线掉队；GLU 的红线全程压住绿线；Switch 趋势好但最大的点异常	33:10	Tay et al., 2022
层数与 aspect ratio 扫描	1 层的曲线远离其他；aspect ratio 的 U 形谷底在各模型大小下都约 100，深模型略左移	35:43	Kaplan et al., 2020
MoE 稀疏度曲面	总参数越多颜色越深（最优稀疏度越高）；固定激活参数、加不激活的参数，loss 仍下降	38:46	Abnar et al., 2025（应为）
critical batch size 两区间 · B_crit 对 loss	左：完美 scaling 区到收益递减区的拐点；右（46:56）：loss 越低 B_crit 越大，又是幂律	42:20	McCandlish et al., 2018
上游对下游	负困惑度对参数量是直线；下游最好的模型在困惑度上并不最好	51:00	Tay et al., 2021（应为）
Rosenfeld 的外推	只用左下角的绿点拟合，预测右上高算力、大模型区域的误差，几乎全中	59:10	Rosenfeld et al., 2019
Chinchilla 图 1 与三种方法的指数表	三颗星是当时的巨型模型，青色星是 Chinchilla；表里方法一、二各 0.5，方法三 0.46 / 0.54，Kaplan 0.73 / 0.27	1:01:13 · 1:02:13	Hoffmann et al., 2022
lower envelope 与 IsoFLOP 曲线	前者取所有训练曲线的下包络；后者每档预算一条 U 形曲线，谷底连线	1:03:13 · 1:04:14	同上
Kaplan → Chinchilla 三	参数计数、warm-up、batch size 各移一截，最后与 Chinchilla 重合；1:13:24 是	1:07:50	Porian et al., 2024 ·

图	看什么	跳转	出处
步图 · Epoch AI 的重拟合	从图里抠出的数据点，原文拟合残差大，重拟合后回到 20 倍		Besiroglu et al., 2024

提到的工作

名称	在本讲里的作用
Cortes, Vapnik 等（贝尔实验室，1993）	最早的"数据 scaling law": 小样本拟合学习曲线，外推大样本的误差（应为 Learning Curves: Asymptotic Values and Rate of Convergence）
Banko & Brill, 2001	NLP 里"多收数据胜过改算法"的经典引用
Kolachina et al., 2012（应为，字幕作 "Collobert"）	机器翻译学习曲线的函数形式：三参数、四参数幂律
Hestness et al., 2017	神经网络 scaling 的源头：多领域数据幂律、涌现、算力 scaling；SGD 对 Adam 的对比图也出自这里
Kaplan et al., 2020	本讲反复引用：三联图、数据 scaling、LSTM 对比、aspect ratio、非 embedding 参数、 $N \propto C^{0.73}$ 、 B_{crit} 对 loss
Bahri et al., 2021	主张 scaling 指数说明网络在做非参数平滑（讲者半信）
Data Mixing Laws（Ye et al., 2024）	用小模型拟合"配比 → 表现"再外推
DataDecide（Magnusson et al., 2025）	大规模实证：小尺度直接选最优配比就够
Scaling Data-Constrained Language Models（Muennighoff et al., 2023）	重复数据最多约 4 个 epoch 不吃亏；重复下的修正函数形式
Pre-training under infinite compute（Kim et al., 2025，应为）	无限算力下重复与加大模型都收益递减，转向集成；干预不改斜率
Mamba · Gated DeltaNet	架构论文里"我们的模型对 vanilla Transformer"标配图的例子（第 4 讲）
Scaling Laws vs Model Architectures（Tay et al., 2022）	T5 系列上的架构 scaling: Performer 差、GLU 好、Switch 好、mixture of softmaxes 有效
Scale Efficiently（Tay et al., 2021，应为）	上游困惑度与下游表现脱节的那张图
Performer · Switch Transformer · mixture of softmaxes	Tay 等比较过的架构改动
Abnar et al., 2025（应为，"Apple 和 MIT"）	MoE 的总参数、激活参数、稀疏度的 scaling law

名称	在本讲里的作用
An Empirical Model of Large-Batch Training (McCandlish et al., 2018)	critical batch size 的定义、估计与噪声尺度
μP (Yang et al., 2022)	重参数化让最优学习率不随宽度漂移；第 11 讲细讲
Rosenfeld et al., 2019	两条幂律相加的联合函数形式；小角落拟合、大区域外推
GPT-3 (Brown et al., 2020)	按 Kaplan 处方训出的巨型模型；训练不足的例子
Chinchilla (Hoffmann et al., 2022)	三种方法、 $a = b \approx 0.5$ 、每参数约 20 个 token
Resolving Discrepancies in Compute-Optimal Scaling (Porian et al., 2024)	参数计数、warm-up、batch size 三步从 Kaplan 走到 Chinchilla
Reconciling Kaplan and Chinchilla Scaling Laws (Pearce & Song, 2024)	不训模型，用模拟的训练曲线解释分歧：低算力尺度加非 embedding 参数的非线性
Chinchilla Scaling: A Replication Attempt (Besiroglu et al., 2024, Epoch AI)	从图里抠数据重拟合方法三，发现原文欠拟合
DeepSeek-V3	"今天你多半会选一个 DeepSeek 风格的 MoE"的例子
NVIDIA B200	场景里的算力：一万张用一个月
Epoch AI 等对前沿实验室算力去向的调查	算力大头在研发和服务，不在训练 run（讲者没报具体来源）

术语对照

English	中文
scaling law	规模定律：小尺度表现外推到大尺度的简单预测规则
power law / scale-free	幂律 / 无尺度：log-log 图上的直线
log-log plot	双对数坐标图
slope / intercept (offset)	斜率 / 截距：斜率是学习速度，截距是整体水平
exponent	指数：斜率的绝对值
irreducible error / noise floor	不可约误差 / 噪声底：任务本身的熵，数据再多也降不下去
asymptote	渐近线
generalization bound	泛化界：测试误差相对训练误差的理论上界
sample complexity	样本复杂度：达到某个误差需要多少样本

English	中文
hypothesis class	假设类：模型能表示的函数集合
parametric / non-parametric	参数 / 非参数模型：后者的收敛速率随维度变慢
intrinsic dimension	内在维度：数据实际占据的维度
extrapolate	外推
emergence	涌现：能力看似突然出现，因为准确率比 loss 不连续
data mixture	数据配比
epoch / repetition	一遍数据 / 重复数据
ensembling	模型集成
data filtering	数据过滤：随规模变松
aspect ratio	深宽比：d_model 除以层数
scale-invariant	尺度不变：最优值不随模型大小漂移的量
embedding / output (unembedding) layer parameters	词嵌入 / 输出层参数：形状互为转置，Kaplan 两者都不数
non-embedding parameters	非 embedding 参数
active parameters / sparsity (MoE)	激活参数 / 稀疏度：每个 token 真正用到的参数比例
critical batch size	临界批大小：完美 scaling 区到收益递减区的拐点
noise-limited / variance-limited	噪声受限 / 方差受限：更大 batch 直接换进展
bias-limited	偏差受限：局部下降方向本身偏离全局最优
gradient noise scale	梯度噪声尺度：梯度协方差的迹除以梯度范数平方
learning rate warm-up	学习率预热
width scaling	宽度 scaling：只加宽不加深
μ P (maximal update parametrization)	让学习率最优点不随宽度变的参数化
perplexity	困惑度：loss 的指数
upstream / downstream	上游（预训练 loss） / 下游（任务表现）
transfer	迁移：上游改善能否变成下游改善
singleton run	单次 run：每个数据点只跑一遍
one-pass SGD / generalization gap	单遍训练 / 泛化差距：单遍时 train 与 val 几乎一样
compute-optimal	算力最优：预算固定时最划算的 N 与 D
joint scaling law / functional form	联合 scaling law / 函数形式

English	中文
lower envelope	下包络：每个 FLOPs 值上所有 run 的最低 loss
IsoFLOP	等算力曲线：固定预算扫 N 与 D 的取舍
curve fitting / underfit	曲线拟合 / 欠拟合
tokens per parameter	每参数的 token 数：Chinchilla 约 20
undertrained / overtrained	训练不足 / 过训练：后者是产品模型的常态
B200	NVIDIA Blackwell 一代的 GPU

字幕勘误

"Hassabis" → Hestness (et al., 2017); "Collobert et al 2012" → 应为 Kolachina et al., 2012; "Barri et al." → Bahri et al.; "Perseus" → 学生姓名（推断为 Konwoo Kim）; "E T / ETE" → Yi Tay (Tay et al.); "new P" → μP ; "Y Yi" → Yair (Carmon); "Pearson and Song" → Pearce and Song; "Hoffman and all" → Hoffmann et al.; "data decide" → DataDecide; "isoflops" → IsoFLOPs; "flops" → FLOPs; "MOE" → MoE; "deep seek for inspired" → DeepSeek 风格（原话不清）; "the trip optimal trade-off" → the optimal trade-off; "scaling loss" → scaling laws; "Kaplan 2021" → Kaplan 2020（讲者口误）; "D over 20 is 20 rule" → 每参数 20 个 token 的经验法则; "Chinchilla-Kaplan" 有时被切成 "chinchilla / cap lin"。

带走的问题

- 用 $C \approx 6ND$ 和 Chinchilla 的 $a = b = 0.5$ ：算力从 10^{22} 涨到 10^{24} FLOPs, N 和 D 各涨多少倍？换成 Kaplan 的 0.73 / 0.27 呢？两种处方下 10^{24} FLOPs 的模型分别多大、每参数多少 token？哪一种更接近今天的开源模型？
- "干预只改截距、不改斜率"如果是普遍规律，意味着什么样的工程流程是合理的（先小后大、直接挑最优）？它在哪些情况下失效——重复数据超过 4 个 epoch、LSTM 对 Transformer？在只有两个数量级算力的小实验里，怎样区分"斜率真的变了"和"Taylor 展开让什么都像直线"？
- critical batch size 随目标 loss 下降按幂律增大。把它和第 7—8 讲的数据并行连起来：给定目标 loss 和 B_{crit} ，纯数据并行最多能开多少

张卡而不浪费？反过来，Kaplan 给所有模型固定一个大 batch，为什么会毁掉小模型的 scaling law、进而把 a 推向 0.73？

4. 方法三的欠拟合让 β 偏小、结论偏向"token 应远多于参数"。如果你只能做 IsoFLOP，需要多少档预算、每档几个点，才有把握区分 $a = 0.50$ 和 $a = 0.46$ ？这两个指数在 10^{25} FLOPs 上给出的最优 N 差多少倍？
5. 把推理成本算进来之后，"最优"该怎么重新定义？写一个包含训练 FLOPs (6ND) 和预期服务 token 数（每个 token 约 $2N$ FLOPs）的总成本函数，看最优 N 往哪边移、移多少；这和第 10 讲的推理账、第 4 讲 MoE 的取舍怎么衔接？

CS336 第 10 讲 | 推理 (Inference)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 10 讲, 2026 年 4 月 29 日 视频: <https://www.youtube.com/watch?v=EfM546A79aM> (1:25:30; 英文字幕为自动生成, 专名与缩写错得不少, 见文末勘误) 讲者: Percy Liang (全程; 开场说"上一讲 Tatsu 讲了 scaling laws", 收尾说"下一讲 Tatsu 回来讲 scaling laws 下") 课程主页: <https://stanford-cs336.github.io/> · 本讲的骨架和不少图取自 Google 的 How to Scale Your Model (讲者称 scaling book, 推理一章) · 反复引用的论文: GQA · DeepSeek-V2 的 MLA · Speculative Decoding · vLLM / PagedAttention

一句话: 推理和训练用的是同一个模型, 却是完全不同的工作负载——训练把整段序列当成张量的一个维度并行地喂给矩阵乘, 推理只能一个 token 一个 token 地往外生成, 于是解码阶段每从 HBM 搬一个字节只做约 1 次浮点运算 (H100 要 295 才吃满算力), 瓶颈是搬参数和 KV cache, 不是算力。整讲的账都从这里出发: 延迟 = 要搬的字节 ÷ 带宽, 吞吐 = batch ÷ 延迟; batch 放大, 吞吐升、延迟升、显存爆。所以所有手段——GQA / MLA / 跨层共享 / 滑窗 / 量化 / 剪枝——归根到底是同一句话"把 KV cache 和参数的字节缩小, 但别掉精度", 再加上 speculative decoding (无损的换法) 和 continuous batching、paged attention (服务器层面的系统手段)。

时间轴

时间	内容
0:05	推理为什么值得一讲: 聊天、补码、agent、批处理、评测、RL 的 rollout 都是推理
1:40	训练是一次性成本, 推理天天付: OpenAI 每天约 8.6T token, 不到 4 天就超过一个模型的 32T 训练 token
2:43	agent 时代: token 大多不是给人读的, token 数就是花掉的算力, 没有天花板
4:13	谁在做推理: 闭源 API、开源权重服务商; vLLM、SGLang、TensorRT-LLM、llama.cpp
5:14	三个"快": TTFT、latency、throughput, 各自适用的场景
7:17	本讲的一个 bit: 训练沿序列并行, 推理逐 token 串行, 算术强度上不去
8:50	记号 (借自 scaling book): B、T、D、H、N、K、F、S; Transformer block 的"电路图"

时间	内容
14:29	复习算术强度：瘦矩阵乘的强度约等于 B；H100 要 B 大于 295；B = 1 时强度 1
18:37	问答：幻灯片上 K 和 G 写反了，K 应是组数、G 是每组 query 头数
20:41	朴素解码是 $O(T^3)$ ；因果性让前缀可复用 → KV cache；prefill 与 generation 两阶段
25:22	逐层记账：MLP 的强度 → $B \cdot T$ ，attention 的强度 → $ST/(S+T)$
33:37	结论：prefill 算力受限，generation 内存受限；attention 为什么加大 B 也救不了
35:41	Llama 2 13B 上 H100：参数 26 GB、KV cache 每序列 838 MB → 延迟、吞吐的公式
41:22	batch size 的两难：B = 1 时 0.008 s/token、124 token/s；加大 B 吞吐升延迟升，256 就爆显存；公交车比喻；TTFT = prefill
45:59	缩 KV cache 之一：GQA 缩 N/K 倍；Llama 例子改 1:5 后延迟吞吐双改善，B = 256 装得下
51:35	缩 KV cache 之二：MLA（16k 维压到 512，RoPE 要补丁）；CLA 跨层共享；问答：为何不直接缩模型维
56:41	缩 KV cache 之三：滑动窗口与混合层；问答：线性注意力、Mamba、DeltaNet 与滑窗怎么比
1:02:20	DeepSeek 的压缩 + 稀疏注意力；小结：跨层、跨头、降维、局部、线性、扩散模型
1:04:23	量化：QAT 与 PTQ；GPTQ 用 Hessian 逐层修正；AWQ 给大激活通道保留高精度
1:07:25	剪枝加蒸馏：NVIDIA 把 15B 剪成 8B；问答：怎么判断哪一部分重要
1:11:29	speculative decoding：草稿模型猜几个、目标模型并行校验，接受概率 $\min(1, q/p)$ ；甜点 3–4 个
1:16:37	动态负载：Orca 的 continuous batching 与 selective batching
1:19:39	paged attention：两种碎片、按块存 KV、前缀共享、copy-on-write
1:23:47	总结：一切归于“缩 KV cache 但别掉精度”；新架构是最大的未开发空间

核心内容

1. 推理为什么值得一整讲：成本、agent 与 RL

- **问题本身很简单**：模型训好了，给一个 prompt，尽量准、尽量快地产出回复。但它只占一讲，重要性却在涨。
- **用途清单**：和助手聊天、代码补全、agent、批量处理数据、评测（需要生成的那一类）、以及训练内部——做 RL 时要用当前模型生成 rollout，打分，再更新权重。第 16 讲 RLVR 的每一步都要先做推理。

- **成本结构**：训练是一次性成本，贵，但付完就完了；推理是反复成本，每天都在付。讲者给的量级：OpenAI 估计每天产出约 **8.6T token**；一个今年发布的模型训练用了 **32T token**——不到 4 天，推理生成的 token 就超过了它的整个训练集 ($32 \div 8.6 \approx 3.7$ 天)，前沿模型训练用的 token 更多，但推理这边只会更大。

小注：字幕里这个"训练用了 32T token"的模型写作 GPT-4，但 GPT-4 并非 2026 年发布，具体是哪个模型无法从字幕确认，只保留数字。

- **agent 改变了推理的意义**：chatbot 时代，prompt 进、回复出、人来读——快到人读不过来就够了。agent 时代，一个查询进去，模型要思考、推理、调工具、自省，最后才给人一段输出；生成的 token 大多不是给人读的。所以应当把"生成的 token 数"直接理解为"花掉的算力"，而且没有上限——问题够难，就要更多算力、更多 token。
- **谁在做**：闭源 API 厂商必须自己服务模型；一批服务商在跑开源权重；开源社区的包——**vLLM**（最常见的默认选择）、**SGLang**（对 agent 负载特别合适，还没那么流行）、**TensorRT-LLM**（NVIDIA 出品，很快但更窄）、**llama.cpp**（在 CPU 上跑）。
- 讲者的结论：推理快 2 倍、哪怕只快 10%，都是大事。

2. 三个"快"的指标，和本讲唯一要记住的一个 bit

- **TTFT** (time to first token)：用户敲下问题到第一个 token 出现的时间。交互式应用最在意——干等的时间越长体验越差；token 一旦开始出现，快慢反而没那么要紧，人读不了那么快。
- **latency** (延迟)：站在单个用户的角度，一个查询的 token 以多快的速度流出来 (秒/token 或它的倒数)。同样是交互式应用关心的。
- **throughput** (吞吐)：站在系统的角度，所有查询合计每秒产出多少 token。批处理关心它——"有 1 PB 数据要过一遍模型"，只要总任务早点做完，单个查询先出后出无所谓。
- 大多数优化会同时改善延迟和吞吐，但有一条根本的取舍 (batch size, 第 7 节) 让两者反向。
- **本讲的一个 bit** (讲者说如果只记一件事就记这个)：训练时 (包括 SFT) 所有 token 一次全看见，序列只是张量的一个维度，attention 和 MLP 都是一个大张量乘过去；推理因为自回归，token 只能一个一个串行生成，**没法沿序列维并行**。这就是推理和训练是完全不同工作负载的根源——后面会看到，它让算术强度很难拉高、算力用不满。

- 讲者的路线：先做数学（算术强度、吞吐、延迟怎么想），再讲降成本的手段（缩 KV cache、量化、剪枝），最后 speculative decoding 和线上服务的实际问题。

3. 记号，以及"瘦矩阵乘"的算术强度

- 记号**（沿用 scaling book；同一个字母既表示维度也表示它的长度）：
B 批维 = 序列数；T 序列维 = token 数；D 模型维；H 头维；N 头数， $D = N \cdot H$ ；K 是 KV 头数（问答之后修正：K 应是组数，G 是每组的 query 头数，幻灯片写反了）；F 是 MLP 的中间维，按惯例记 $F = 4D$ 。另有 S 与 T 两个序列维：S 是输入（被条件化的）token 数，T 是要产出 logits 的 token 数——训练时 $S = T$ ，推理时 $T = 1$ 、S 是前文长度。
- 张量乘法的三种维度**：收缩维（两个操作数都有，结果里消失，图上红色）；普通维（只在一边，留在结果里，黑色）；批维（两边都有，结果里也留着，不收缩，蓝色）。attention 里的 B 和头数正是蓝色批维——"记住这个蓝色 B"，第 5 节它就是推理难的根源。
- Transformer block 的"电路图"**： $X \rightarrow \text{attention} \rightarrow \text{MLP}$ 。Q 是 $B \times T \times N \times H$ ，K、V 的头数是 K 而不是 N；attention 那一步在 B 和头上批、在 H 上收缩；MLP 就是 gate、up、down 三个矩阵乘。讲者认为这是他见过最清晰的 Transformer 定义——每个张量的形状都写明，依赖关系一目了然。
- 算术强度复习**（第 2 讲讲过，这里换成非方阵）：X 是 $B \times D$ ，W 是 $D \times F$ ，推理一律 BF16（每个数 2 字节）。要做的事：从 HBM 读 X（ $2BD$ 字节），读 W（ $2DF$ ），做矩阵乘（ $2BDF$ 次 FLOPs），把结果写回（ $2BF$ ）。读写是"元素个数"量级的平方项，矩阵乘是立方项——这就是矩阵乘能有高强度的原因。

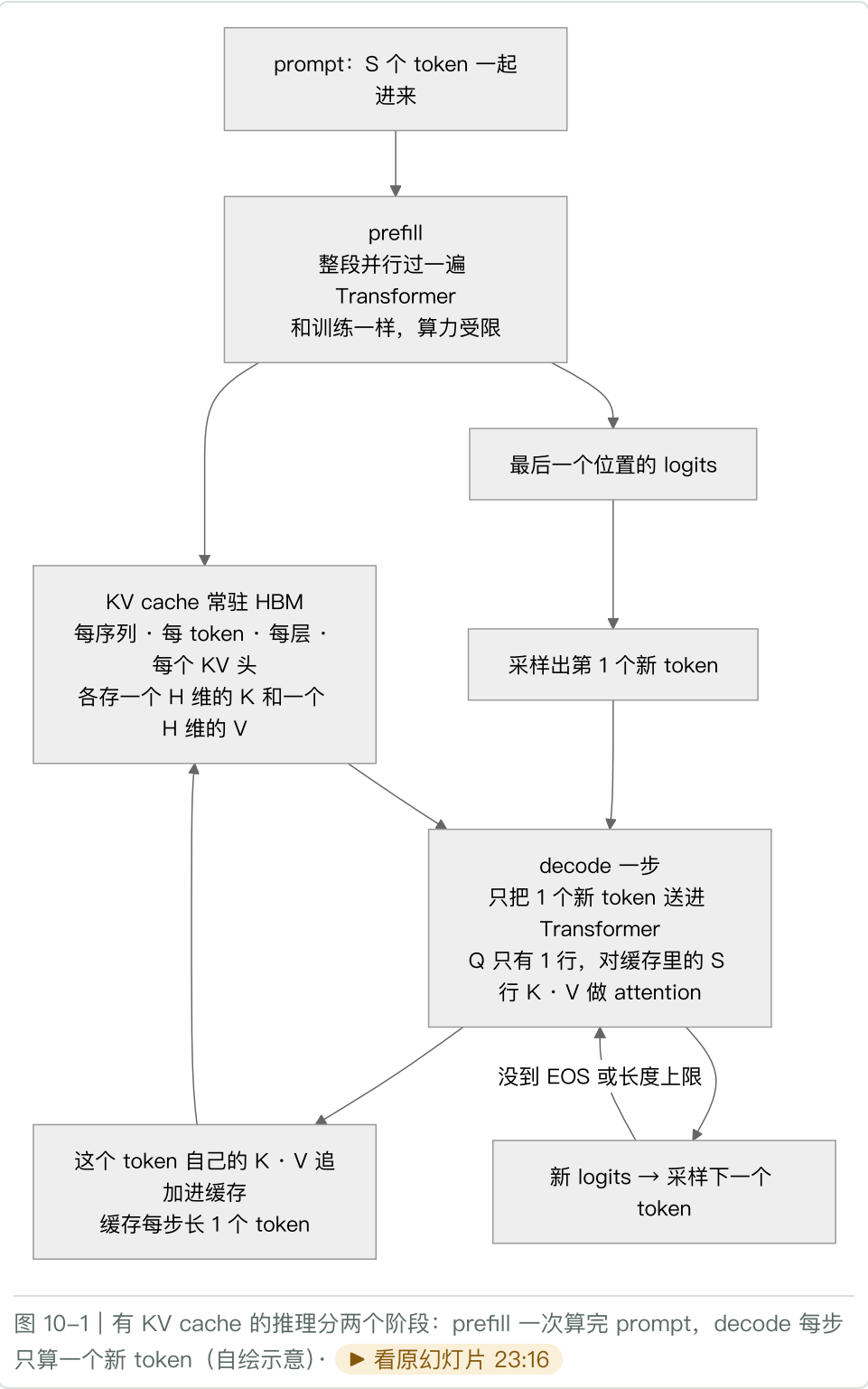
$$I_{\text{matmul}} = \frac{2BDF}{2BD + 2DF + 2BF} \xrightarrow{B \ll D, F} B$$

I 是算术强度（每搬一个字节做多少 FLOPs），分子是矩阵乘的 FLOPs，分母是读两个输入、写一个输出的字节数；令 $D = cB$ 、 $F = cB$ 并让 c 趋于无穷（D、F 远大于 B），分母被 $2DF$ 主导，比值就是 B。第 2 讲方阵的 $n/3$ 是它的特例。

- 和硬件比**：加速器强度 = 规格表上的 FLOP/s $\div$ HBM 带宽（第 2 讲：H100 是 $989\text{e}12 \div 3.35\text{e}12 \approx 295$ ）。算法强度高于它 $\rightarrow$ 算力受限（好）；低于它 $\rightarrow$ 内存受限（坏）。对这个矩阵乘，H100 上 B 大于 295 才算力受限。

- 极端情形 $B = 1$: 强度是 1。读进一整块 $D \times F$ 的权重，却只做 $2DF$ 次运算。推理看到的正是这种"瘦"张量，不是训练里那种满的矩阵。

4. KV cache: 从 $O(T^3)$ 到 prefill + generation



- 朴素做法：prompt 是 "Never going to give you", 整段过一遍 Transformer 得到输出分布，采样一个 token，接到 prompt 后面，

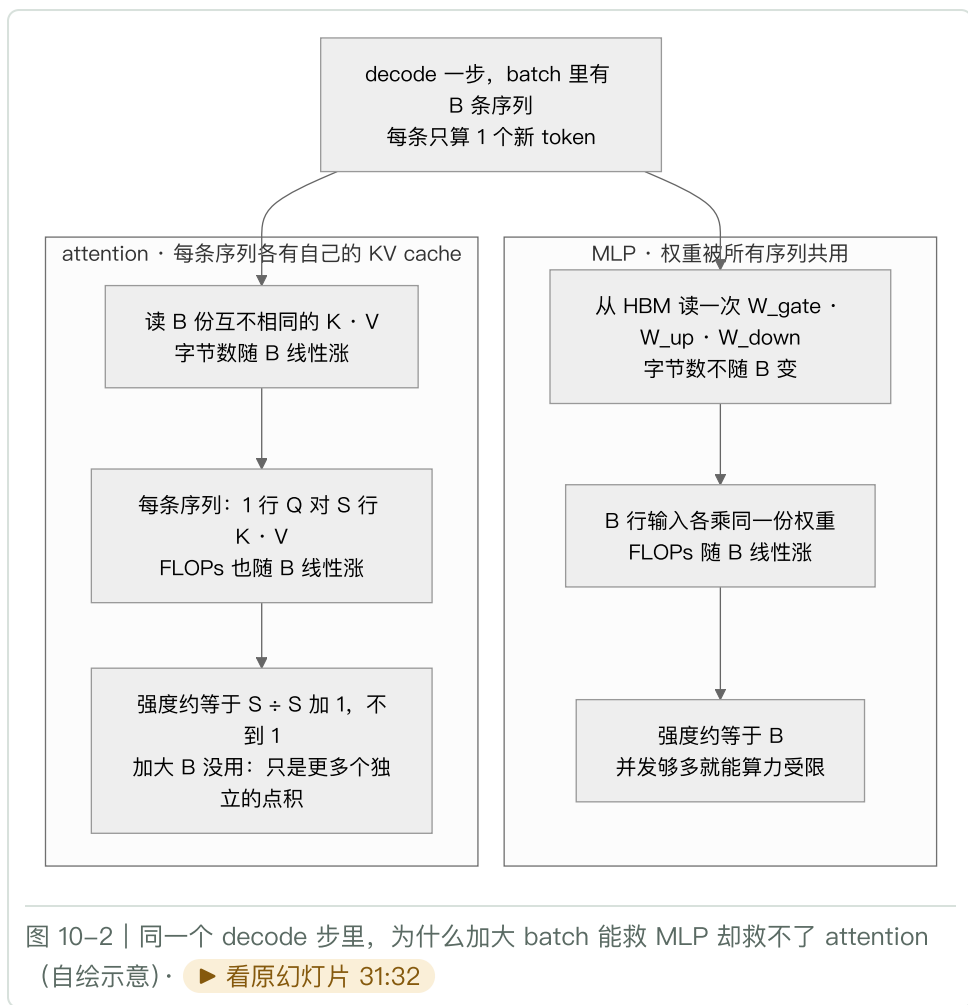
再把整段过一遍……任何“输入序列、输出分布”的黑盒都能这么用，能用，但很糟：attention 本身是 $O(T^2)$ ，每生成一个 token 都重做一遍，生成 T 个 token 就是 $O(T^3)$ 。

- **观察**：这是因果（causal）Transformer——前面 token 的激活、K、V 不会因为后面追加了什么而改变（双向模型就不行，加一个 token 全变）。所以不同前缀之间大量计算是共享的。
- **KV cache**：把已算出的 K、V 存在 HBM 里，相邻两步之间直接复用。形式化地说，每个序列（共 B 个）、每个 token（共 S 个）、每一层、每一个 KV 头，各存一个 H 维的 K 向量和一个 H 维的 V 向量。读者在 CME295 第 3 讲见过这个概念；本讲往下算它的字节数，以及它对算术强度的决定性影响。
- **两个阶段**（图 10-1）：**prefill**——拿到 prompt，一次性把它编码成 KV cache，整段都看得见，所以能像训练一样并行；**generation**（decode）——逐 token 串行地生成回复，但至少不必再为看过的 token 重算 K、V，每步只算新 token 那一份并追加进缓存。

```
def generate(model, prompt_ids, max_new):
    kv, logits = model.prefill(prompt_ids)    # 一次并行：填满 S 个位置
    # 的 K、V，返回末尾位置的 logits
    out = [sample(logits)]
    for _ in range(max_new - 1):              # decode: 每步只送 1 个
    # token 进模型
        logits, kv = model.step(out[-1], kv) # Q 只有 1 行；这一步的
    # K、V 追加进 kv
        out.append(sample(logits))
        if out[-1] == EOS:
            break
    return out
```

prefill 是一次批量调用，decode 是一个每步只处理一个 token、不断往缓存里追加的循环——两段代码的算术强度天差地别，下一节算给你看。

5. 逐层记账：MLP 靠加大 batch 能救，attention 谁也救不了



- **只数矩阵乘**: 其他算子的 FLOPs 少得多, 而且可以融合进矩阵乘的 kernel (第 6 讲)。用 S、T 两个序列维抽象地算, 最后再代入 prefill ($T = S$) 和 generation ($T = 1$)。
- **MLP**: 读 X, 读全部权重, 算 up 投影写回 HBM, 算 gate 写回, 算 down 投影写回。FLOPs 正比于 $B \cdot T \cdot D \cdot F$; 字节数里权重那一项正比于 $D \cdot F$ 、不含 B 和 T。和第 3 节一样假设 $B \cdot T$ 远小于 D、F, 强度就是 $B \cdot T$ ——MLP 本质上就是一个大矩阵乘, 只是矩阵多几个、维度多几个; batch 维和序列维在 MLP 里完全独立、地位相同。
 - prefill: $B \cdot T$ 够大 (大 batch、长序列) 就没问题。
 - generation: $T = 1$, 强度退化成 B, 而 B 在这里是**并发请求数**。批处理场景你能控制它; 线上聊天它等于此刻有多少用户在同时用, 可高可低、随时间变——第 13 节的 continuous batching 就是为这个准备的。总体不算坏: 只要并发够多, 序列长度帮不上忙, batch 能。
- **attention**: 读 Q、K、V, 算打分, softmax (不计), 乘 V, 写回。FLOPs 正比于 $B \cdot S \cdot T \cdot D$, 字节数正比于 $B \cdot D \cdot (S + T)$ 。矩阵乘所以

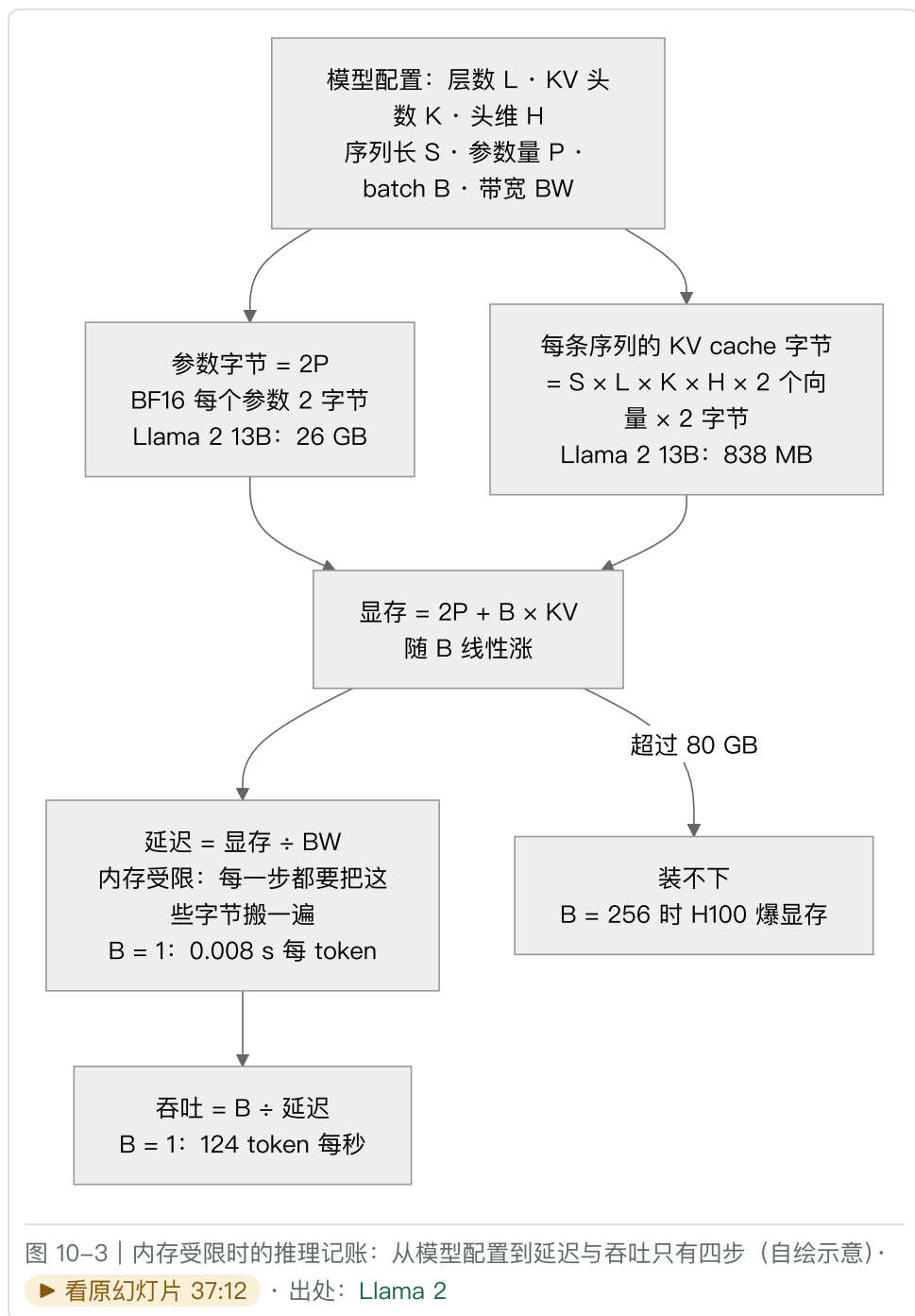
FLOPs 总比字节高一阶，关键只在那个因子长什么样：

$$I_{\text{attn}} \propto \frac{ST}{S+T}, \quad \text{prefill } (T=S) : \frac{S}{2}, \quad \text{generation } (T=1) : \frac{S}{S+1} < 1$$

S 是已有的前文 token 数，T 是这一步要产出 logits 的 token 数；prefill 时强度随序列长度线性上升（注意 B 根本不出现），generation 时不到 1——而 H100 要 295 才能吃满算力。

- **为什么 B 帮不上 attention**（图 10-2）：MLP 的权重不依赖 B，加载一次给所有序列用，B 越大摊得越薄；attention 里每条序列有自己的一份 KV cache，读的字节和做的 FLOPs 都随 B 一起涨，相当于 B 个互不相关的小矩阵乘——多做几个并不提高强度。这正是第 3 节那个“蓝色 B”：按一个坐标批起来做，本质上和点积一样，强度糟糕。
- **四格总结**：prefill 的 MLP 强度 $B \cdot S$ ，很好；prefill 的 attention $S/2$ ，不如前者但可用；generation 的 MLP 强度 B，可用，前提是并发够多；generation 的 attention 约等于 1，是**根本瓶颈**——只要还用 Transformer，这一格就改不了。所以 prefill 算力受限，generation 内存受限。讲者：以后再听人说 “inference is memory bound”，你知道为什么了。

6. 把账算到具体的模型上：Llama 2 13B 跑在 H100



- **内存受限带来的简化：**要估时间，只看要搬多少字节（假设通信与计算重叠，瓶颈就是字节数）。好处是账好算，坏处是加速器大部分时间在空转。
- **例子的配置：**Llama 2 13B, H100。要用的量：序列长度、D、F、query 头数、KV 头数（原版没有 GQA, $K = N$ ）、头维、层数、词表大小，以及 H100 的 HBM 带宽。
- **参数：**embedding、MLP、Q/K/V 投影加起来约 13B，和广告数一致；BF16 每参数 2 字节 → 26 GB。

- **KV cache**: token 数 $\times$ KV 头数 $\times$ 头维 $\times$ 层数 $\times 2$ (K 和 V 各一份) $\times 2$ 字节 (BF16):

$$C_{kv} = S \cdot L \cdot K \cdot H \cdot 2 \cdot 2 \text{ bytes}, \quad M = 2P + B C_{kv}$$

C_{kv} 是一条序列的 KV cache 字节数 (S 个 token、L 层、K 个 KV 头、每头 H 维, K 与 V 各一份, 每个数 2 字节), M 是总显存: 参数 2P 字节只有一份, KV cache 每条序列一份。讲者算出来的形式是 $M \approx 838 \text{ MB} \times B + 26 \text{ GB}$ 。

$$\text{latency} = \frac{2P + B C_{kv}}{BW} \text{ s/token}, \quad \text{throughput} = \frac{B}{\text{latency}} = \frac{B \cdot BW}{2P + B C_{kv}} \text{ token/s}$$

BW 是 HBM 带宽; 内存受限时每一步 decode 都得把参数和全部 KV cache 搬一遍, 所以延迟是 B 的线性函数 (斜率是 KV cache、截距是参数), 吞吐是 B 除以它——B 变大时参数那一项被摊薄, KV 那一项摊不掉, 所以吞吐上升但有渐近线, 不可能到无穷。

- **数字**: $B = 1 \rightarrow$ 延迟 **0.008 s/token**, 吞吐 **124 token/s**。

小注: 反推讲者的配置——Llama 2 13B 是 40 层、40 个头、头维 128, 每个 token 的 KV cache = $2 \times 40 \times 40 \times 128 \times 2$ 字节 = 819,200 字节, 乘 1024 个 token 正好 838 MB, 所以配置里的序列长度应为 1024; 带宽是 H100 SXM 的 3.35 TB/s, $(26 \text{ GB} + 0.838 \text{ GB}) \div 3.35 \text{ TB/s} = 8.0 \text{ ms}$, 倒数 125 token/s, 与课上数字一致。

```
def decode_stats(P, L, K, H, S, B, bw=3.35e12, nbytes=2):
    param_bytes = nbytes * P                                # BF16: 每参数 2 字节,
    只有一份
    kv_per_seq = S * L * K * H * 2 * nbytes                # K 和 V 各一份, 每条序
    列一份
    mem = param_bytes + B * kv_per_seq                      # decode 每一步都要搬完
    这些
    latency = mem / bw                                       # 秒 / token (内存受限的
    假设)
    throughput = B / latency                                 # token / 秒
    return mem, latency, throughput
```

代入 Llama 2 13B ($P = 13e9$, $L = 40$, $K = 40$, $H = 128$, $S = 1024$) 和 $B = 1$, 得到 26.8 GB、8 ms、125 token/s; 改 B、改 K, 就是接下来两节的全部实验。

- **多卡**: 本讲不展开 (分片方案见 scaling book 的推理章)。最平凡的一种: 起 M 份模型副本, 延迟不变, 吞吐乘 M。

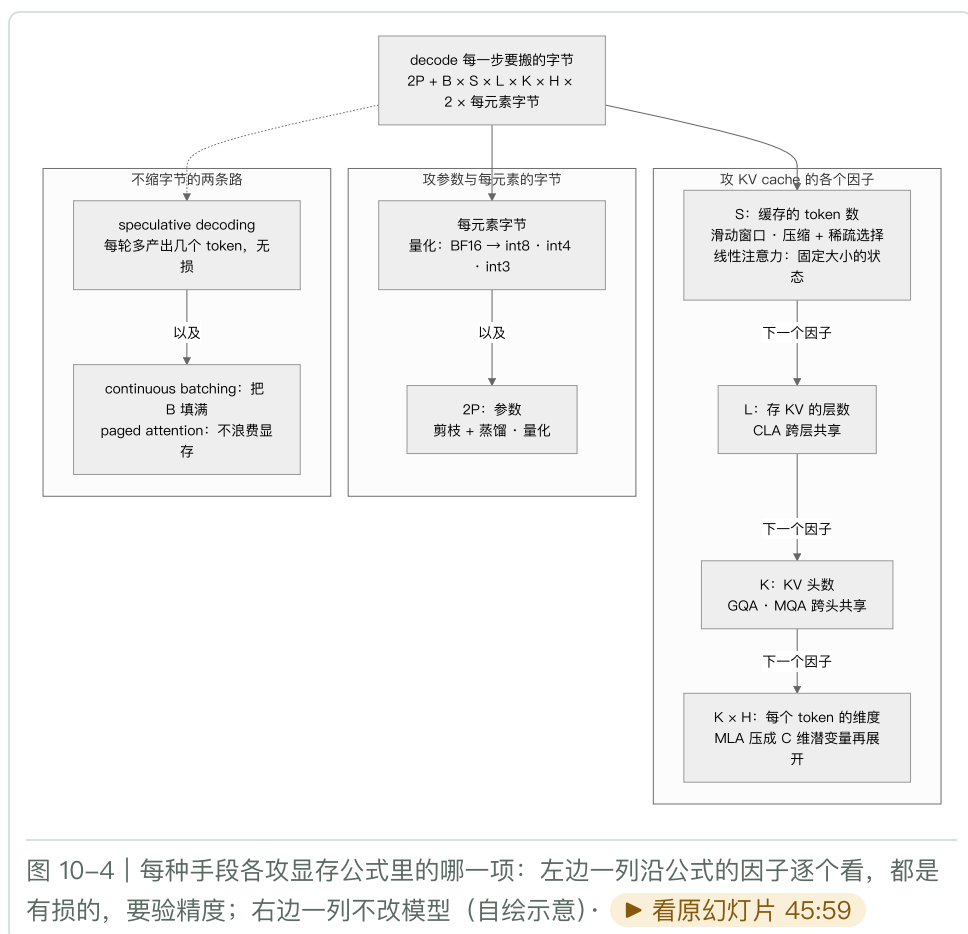
7. batch size 的两难：延迟、吞吐、显存与 TTFT

- **加大 B**：延迟变差——KV cache 更大，每步搬得更多；而且一个 batch 里的请求要等所有人一起走。吞吐变好——参数只加载一次，摊给更多序列。讲者反复说“快”有两个意思，看你要哪一个，调 batch size 的方向完全相反。
- **再加大到 256**（想要高吞吐处理海量文档）：吞吐更好、延迟更差，但 KV cache 的显存超过 H100 的 80 GB，**OOM**。换 B200 能再大一点，但总有上限：永远到不了吞吐的渐近线，而且越往后收益越小。

小注：按第 6 节的公式自己算， $B = 64$ 时显存 $26 + 0.838 \times 64 \approx 79.6$ GB，刚好塞进 80 GB（讲者后面用的正是 64）； $B = 256$ 要约 240 GB；吞吐的渐近线是 $BW \div C_{kv} = 3.35 \text{ TB/s} \div 838 \text{ MB} \approx 4,000 \text{ token/s}$ 。

- **公交车比喻**：小 batch 像打车，随到随走但一次运不了几个；大 batch 像公交，得等车坐满、大家一起走，单人等得久，但整体运力高。
- **结论**：小 batch $\rightarrow$ 延迟好、吞吐差；大 batch $\rightarrow$ 吞吐好、延迟差。但延迟和吞吐并不总是对立的：**减少要搬的字节，两者一起改善**（下一节 GQA 的例子）；真正对立的只有 batch 这一个维度。
- **TTFT 就是 prefill 的时间**——prefill 做完才能开始生成。要 TTFT 快就用小 batch；要吞吐就用大 batch。

8. 缩 KV cache（一）：GQA、MLA、CLA——跨头共享、降维、跨层共享



- **动机**：内存是推理的瓶颈，而 KV cache 占显存的大头——batch 够大时甚至超过参数本身。所以第一件事就是缩 KV cache，但要小心别把精度丢太多。手段从改架构到做系统都有，推理是个横切的话题。

小注：用第 6 节的数自己算，Llama 2 13B 在 1024 上下文下 B 超过 31 时 KV cache ($0.838 \text{ GB} \times B$) 就超过参数的 26 GB。

- **GQA**（定义见 CME295 第 2 讲，T5 时代的证据见本课第 3 讲第 11 节）：MHA 是 $K = N$ ，每个头各有一份 K、V；MQA 是 $K = 1$ ，"没人用，太差"；GQA 在两者之间找平衡。GQA 论文（2023）的图：横轴每样本耗时（和延迟、吞吐都相关），MHA 最慢； $K = 1$ 快得多；把 K 一路加到 8 仍然很快；再往上耗时明显涨。它缩 KV cache N/K 倍——内存受限，所以直接变快。
- **回到 Llama 例子**：B = 64、原版 $K = N$ 时有一组吞吐和延迟；改成 1:5 的 GQA（40 个头共用 8 份 K、V），显存大减，**延迟和吞吐一起改善**——两者并非总是对立，减少字节两头都赚，对立的只是 batch 维。既然显存省下来了，把 B 加到 256：之前 OOM，现在装得下；延迟略差，吞吐按比例上升。参数是联动着调的：缩 KV cache 换来更大的 batch，再换别的取舍。

- **精度**：GQA 论文在一堆评测上显示不掉点；但后来 DeepSeek 的论文表明它其实掉。讲者：凡不是纯数学的结论都要打折扣。
- **MLA** (multi-head latent attention, DeepSeek-V2；第 4 讲第 9 节写过公式)：不减 K、V 的个数——每个 token 还是一份——而是压缩它们。正常做法是激活乘一个矩阵得 K、再乘一个得 V，各是 $N \cdot H$ 维、和模型维一样大；MLA 先把激活投影到 C 维的潜变量 (DeepSeek-V2 从约 16,000 维压到 512，很激进)，缓存只存这 C 维，用的时候再从潜变量把 K、V 展开出来。

$$c_t = x_t W_c \in \mathbb{R}^C, \quad K_t = c_t W_{uk}, \quad V_t = c_t W_{uv}, \quad C \ll N \cdot H$$

x_t 是 token 的激活， c_t 是存进缓存的 C 维潜变量， W_{uk} 、 W_{uv} 是展开矩阵；每个 token 的缓存从 $2 \cdot N \cdot H$ 个数变成 C 个数。

- **麻烦**：MLA 与 RoPE 不兼容——RoPE 直接作用在 K 上，压缩后展开就套不进去了；DeepSeek 的做法是另加少量维度专门放 RoPE。即便如此缩得仍然很多。延迟和吞吐的改善"就是简单算术"：缓存越小越快，接近线性直到某个点。精度表上，GQA 的数比 MHA 小，而 MLA 比 MHA 还好一点点——当作差不多即可。

小注：DeepSeek-V2 论文里 128 个头 $\times$ 头维 128 = 16,384，即课上的"16,000"；潜变量 $d_c = 512$ ；解耦出来放 RoPE 的维度是每头 64。摘要给的账：KV cache 缩 93.3%，最大生成吞吐是 DeepSeek 67B 的 5.76 倍（第 4 讲也记过）。

- **问答**：和直接缩模型维比呢？消融没做这一项；讲者猜不分青红皂白地缩 D 只会更差——诀窍是找到模型里"能挤"的地方，而哪里能挤事先不知道，只能实验。
- **CLA** (cross-layer attention)：只在一部分层算 K、V，其他层直接用上一层的缓存。GQA 跨头共享，CLA 跨层共享。论文的图：给定一种方法，可以扫 K 和头维得到一条"精度对 KV cache 大小"的曲线，CLA 把这条 Pareto 前沿往外推。

小注：应为 Brandon et al., 2024 (MIT)；摘要的账：叠在 MQA 之上再缩 2 倍 KV cache，精度几乎不变。

9. 缩 KV cache (二)：滑动窗口、混合层、线性注意力、DeepSeek 的稀疏注意力

- **滑动窗口 / 局部注意力**：老想法，也很自然。生成一个 token 时只看最近的 k 个，而不是整个 n^2 的注意力矩阵。于是 KV cache 与序列

长度无关（只剩 B 乘其他因子），长上下文尤其划算。有效上下文比窗口大：层数叠起来，信息一层一层往后传得更远。变体：不必每层都做密集的局部窗，可以隔开；"global + sliding window"——固定网格上的一些全局位置加一个局部窗。

- **代价**：表达力受损，精度掉。"没有免费午餐——至少这顿很贵。"解法是**混合**：一部分层全注意力，一部分层局部（第 3 讲记过 Llama 4、Gemma 4、Command A 的 3:1）。永远在精度和速度之间平衡。
- **问答：线性注意力对滑窗**（第 4 讲整讲讲这个，这里只记 Percy 的回答）：线性注意力不存 KV cache，而是把全部历史压成一个固定大小的状态——最朴素的做法是把 K、V 累加成一个向量，肯定与序列长度无关；Gated DeltaNet、Mamba 是"压缩但少遗忘"的版本。它们也被拿来替代滑窗，效果不错；还可以三者并用——全注意力、滑窗、线性各管一摊：滑窗保局部的高分辨率，线性给过去的粗略摘要。长上下文用线性更好吗？没有免费午餐——做 needle in a haystack 时把整段历史压进一个小状态，信息就是会丢，可能就是找不回来。Mamba / DeltaNet 比滑窗**更强**：递推时可以只看最近的状态，所以能表达滑窗的部分行为，"有更多余地"；滑窗一旦选定就到头了。
- **DeepSeek 还在折腾 attention**：MLA 之后又有压缩 + 稀疏的组合（字幕里的名字：compressed sparse attention、DeepSeek sparse attention、heavily compressed attention；讲者自己也说记不清这些缩写）。机制：先把每 M 个 KV token 压成 1 个；再用一套轻量的 query 和 key 做一次很小的 attention 算出 index score，据此选出要保留的 token 子集——"闪电般地决定留谁"；然后再进一步压缩。

小注：带轻量索引器的 DeepSeek Sparse Attention (DSA) 应出自 DeepSeek-V3.2 (2025 年底)；课上那张图把压缩、稀疏选择、重度压缩三段合在一起，应是其后续版本，未能确认出处，不加链接。

- **这一段的小结**：KV cache 就是显存，显存决定延迟和吞吐，所以缩它直接换来速度，关键是别掉精度。手段：降维（跨层、跨头、头维）、局部注意力、线性注意力，还有很多；另一条路是**扩散模型**——非自回归的生成，可以快得多。

10. 量化：从系统而不是架构的角度缩字节

- **思路**：把数的精度降下来。字节少了，延迟和吞吐都好，照例要盯精度。选项从 BF16 一路到 int4（第 5 讲讲过训练侧的 FP8 /

MXFP8，这里是推理侧；第 2 讲问答里"先用 bf16 训好再量化到低比特做推理"指的就是这一节）。

- **量化感知训练 (QAT)**：怕量化把模型搞坏，就在训练时的 forward 里做 quantize → dequantize，把量化误差模拟进去，让权重适应它。
缺点：要做大规模训练，贵。
- **训练后量化 (PTQ)**：训完再量化，便宜得多，是通常的做法。
 - 朴素版：每个 tensor（或每层）定一个 scale 和 zero point，各自量化。一般效果不好。
 - **GPTQ**：用 Hessian 信息逐层量化，把每一步的量化误差记下来，传给尚未量化的权重去补偿。
 - **AWQ** (activation-aware quantization)：更讲究的一种。观察：有些激活通道的值特别大，和它们相乘的那些权重更要紧——就给这些权重多留精度。图上：FP16 的权重矩阵整体量化到 int3，但挑出对应大激活通道的少数几列保留 FP16。

小注：GPTQ 出自 [Frantar et al., 2022](#)，摘要：175B 参数的模型约 4 个 GPU 小时就能量化到 3–4 bit；AWQ 出自 [Lin et al., 2023](#)，摘要：只保护约 1% 的显著权重就能大幅降低量化误差。

11. 剪枝加蒸馏：把大模型"拆了再修"

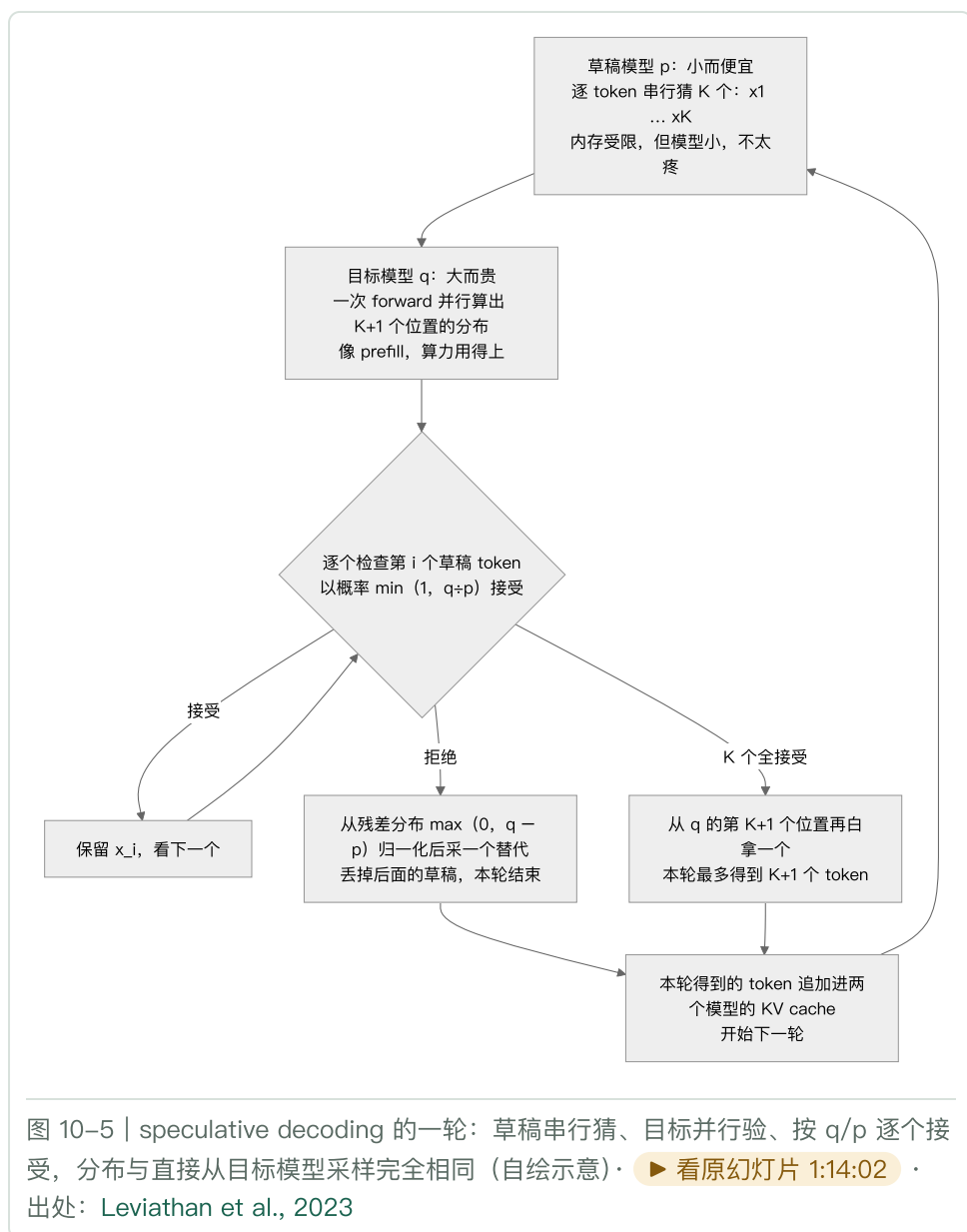
- **做法**（粗暴但有效）：拿一个大模型，撕掉一部分，再修好。NVIDIA 的论文：先估计模型各部分的重要性，删掉不重要的隐藏单元、头、甚至整层；得到的模型此时不好用，再 **post-train**——在你关心的数据或任务上继续训（用蒸馏）把它"治好"。结果：**15B 剪成 8B**，精度基本不掉，训练量比从头训小得多。

小注：应为 NVIDIA 的 Minitron，[Muralidharan et al., 2024](#)，摘要：Nemotron-4 15B 剪成 8B 和 4B，每个模型比从头训少用最多 40 倍的训练 token，全家族训练算力省 1.8 倍，重训只用了不到 3% 的原始数据，MMLU 比从头训高最多 16%。

- **讲者的总结框架**：目标是减参数或减 KV cache，两条路——定义一个更快的架构从头训；或者定义更快的架构、从原模型初始化（架构可以不同，拼成一个 Frankenstein），再用蒸馏修复。
- **问答**
 - 怎么区分重要的层和不重要的层？拿一个校准集过一遍模型，看激活的幅度：接近零的多半是死单元，大的要留。

- 激活为什么大就重要？万一都很大呢？这是经验事实——某些通道就是比别的大得多；若不成立，这一类方法也就不成立了。模型训出来就是这样，于是可以利用。
- 一个神经元对所有样本恒为 100 呢？不能直接删，删了全坏；但高均值、低方差的单元可以折成一个 bias。

12. Speculative decoding：唯一无损的加速——让小模型猜、大模型判



- 前面的手段都是有损的——把 KV cache 狠狠压下去，精度可能受伤。有一种很优雅的办法是无损的：speculative sampling / speculative decoding。

- **利用的不对称**：prefill 能把一整段 token 并行编码，而且顺手给出每个位置的概率——快、算力受限、一切都好；generation 却只能一个一个来。所以**检查比生成快**：给你一段序列，判断它好不好，比一个 token 一个 token 地把它生成出来快得多。
- **想法**：用一个便宜的草稿模型 p 先猜几个 token（比如 4 个），再用真正关心的目标模型 q 一次并行地审这几个 token，决定接受哪些。两边都不太疼：草稿模型小，即使内存受限、串行生成，也不算贵；目标模型大而贵，但让它一次处理一批 token，算力就用上了。课上放了一段动画：大模型逐 token 生成很慢；开了 speculative decoding 后小模型先吐一串，大模型批一下，于是 token 一阵一阵地往外冒。
- **算法**（几篇论文差不多同时提出，课上讲的是其中一篇）：从草稿模型 p 采样 K 个 token；用目标模型 q 并行算出这些位置的 logits；然后逐个决定：

$$\Pr[\text{accept } x] = \min\left(1, \frac{q(x)}{p(x)}\right), \quad \text{reject} \Rightarrow x' \sim \frac{\max(0, q(\cdot) - p(\cdot))}{\sum_y \max(0, q(y) - p(y))}$$

p 是草稿模型对这个 token 的概率，q 是目标模型的；q 相对 p 越大越倾向接受；一旦拒绝，就从“q 比 p 多出来的那部分”归一化成的残差分布里采一个替代 token 并结束本轮。这就是 rejection sampling 的思路，区别在于 rejection sampling 拒绝时两手空空，这里每一轮都保证拿到一个严格服从目标模型分布的样本。证明课上跳过（和 rejection sampling 同一套论证）。

```
def spec_step(draft, target, ctx, K):
    xs, P, out = [], [], []
    for _ in range(K):
        # 小模型串行猜 K 个
        p = draft.dist(ctx + xs); x = sample(p); xs.append(x);
        P.append(p)
        Q = target.dists(ctx + xs)
        # 大模型一次并行：K+1 个位置的分布
        for i, x in enumerate(xs):
            if random() < min(1, Q[i][x] / P[i][x]):
                # 按 q/p 接受
                out.append(x); continue
            out.append(sample(normalize(clip0(Q[i] - P[i]))))
            # 拒绝：从残差分布换一个，退出
        return out
    out.append(sample(Q[K]))
    # 全部接受：末位
```

再白拿一个

return out

一轮里目标模型只做一次 forward，却可能产出 1 到 $K + 1$ 个 token；接受与残差采样合起来保证输出分布就是目标模型的。

- **草稿长度**：论文的图显示，草稿 token 太少，没充分利用目标模型的并行；太多，后面的越来越容易被拒。甜点在 **3—4 个** 左右。
- **草稿模型从哪来**：比目标模型小得多，且越像目标越好——所以要蒸馏。于是前面所有缩 KV cache 的"花招"在这里全用得上：先想办法把模型缩小，缩完满意就直接上线；不满意就把它当草稿，让主模型来把关。之后有一整支文献在原版上改进，课上不展开。

小注：Leviathan et al., 2023 (Google) 摘要：T5-XXL 上相对标准 T5X 实现加速 2—3 倍，输出完全相同；同期的另一篇是 DeepMind 的 Chen et al., 2023。

13. 线上服务的动态负载：continuous batching 与 paged attention

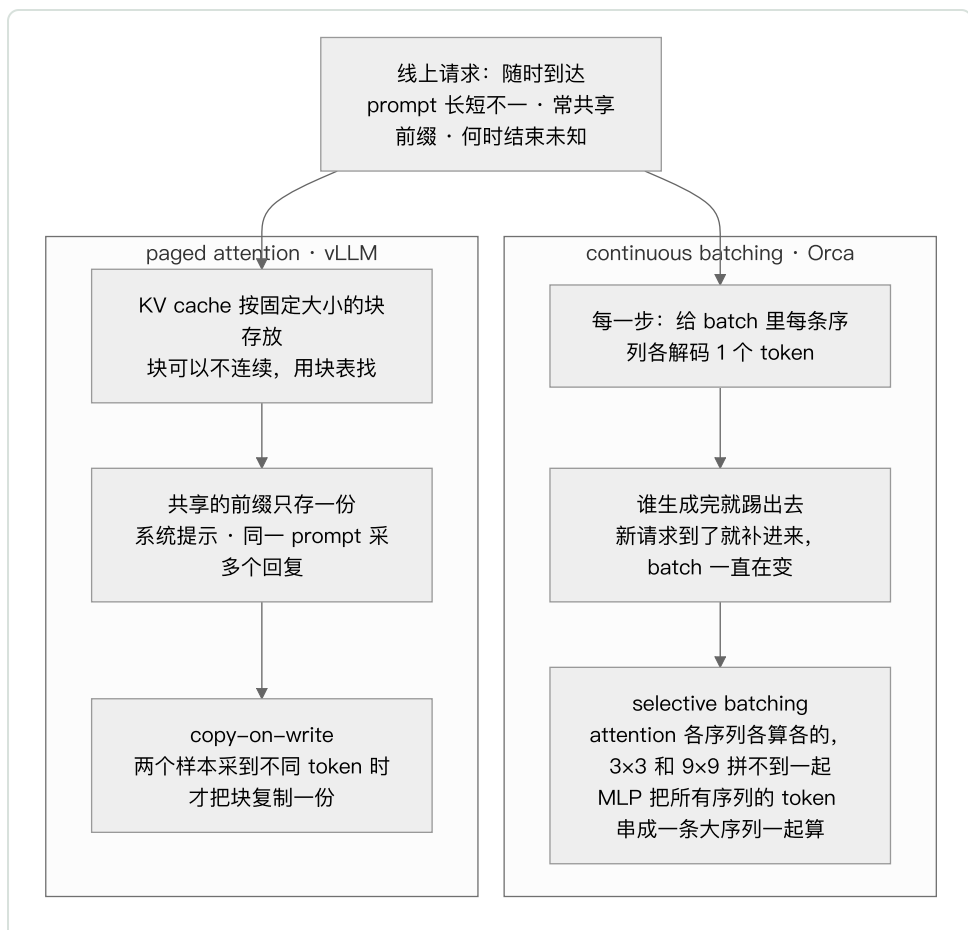


图 10-6 | 服务器层面的两件事：batch 里的成员随时进出；KV cache 像操作系统的内存页那样分块、共享、写时复制（自绘示意）· [▶ 看原幻灯片 1:17:37](#) · 出处：Kwon et al., 2023

- **场景**：在线网站，用户随时来聊。请求到达时间不同、共享的前缀不同、长度不同——一团乱，完全不像训练里整整齐齐、同样长度的块。
- **Orca 与 continuous batching**（很早的系统）：几条请求的前缀长短参差；逐步解码，每一步给所有序列各出一个 token；哪条序列结束了就把它踢出 batch，新到的请求随时补进来——batch 是持续更新的，所以叫 continuous。第 5 节说 generation 的 MLP 强度等于并发数，这就是把并发数一直填满的办法。
- **selective batching**：批处理要求所有切片形状一致，可每条请求长度不同怎么办？attention 部分没办法——它依赖各自的长度， 3×3 和 9×9 的计算凑不成一个张量，只能各算各的；但 MLP 这些非 attention 的部分（FLOPs 的大头）与位置无关，可以把所有序列的 token 首尾相接成一条“大序列”一起算。

```
def serve_step(batch, queue, model, max_batch):
    while queue and len(batch) < max_batch:                # 有空位就补新请
```

求：先 prefill 再入队

```
req = queue.pop(0); req.kv = model.prefill(req.prompt);
batch.append(req)
toks = model.step_all(batch) # 一步：每条序列
各出 1 个 token
for req, tok in zip(batch, toks): # MLP 合并算，
attention 各算各的
    req.out.append(tok); req.kv.append(tok)
batch[:] = [r for r in batch if r.out[-1] != EOS and len(r.out) <
r.max_new] # 完成的踢出
```

每调用一次就是一个解码步：补位、齐步走一格、淘汰——batch 的成员和长度每一步都在变。

- **paged attention** (vLLM 论文的核心想法；vLLM 现在功能多得多)：KV cache 到底放在显存哪里？请求进来总得给它找地方，于是出现碎片化——就像当年的硬盘要 defrag。两种碎片：*内部碎片*——不知道什么时候会停，只好按最大长度（比如 1024 个 token）预留一整块缓冲区，大部分用不上也不能给别人；*外部碎片*——请求之间留下的空隙太小，谁也用不了。
- **解法**（做系统的人懂操作系统：“这个问题我们解决过一次了”）：把每条序列的 KV cache 切成固定大小、可以不连续的块——课上的例子把 "Four score and seven years ago our fathers brought forth" 按每块 4 个 token 切开，块放在哪里无所谓，只要按块对齐、记住索引就行。
- **顺手得到共享**：两条请求的相同前缀可以指向同一批块。系统提示 (system prompt) 的 KV cache 只算一次，所有请求复用；同一个 prompt 要采多个回复时，prompt 的块共享，只有各自的回复不同。**copy-on-write**：两个样本先共享同一块，采到同样的 token 就继续共用，采到不同 token 时才把这个块复制一份、各走各的——尽可能多地共享前缀缓存。还有一堆 kernel 上的优化没时间讲；总的思路是用操作系统的那套比喻来管理推理。

小注：vLLM 论文 (Kwon et al., 2023) 摘要：灵感来自操作系统的虚拟内存与分页；KV cache 显存浪费接近零；同等延迟下吞吐是当时主流系统的 2—4 倍；正文估计此前的系统因碎片和过量预留浪费 60—80% 的 KV 显存。Orca 应为 Yu et al., OSDI 2022, 论文报告同等延迟下吞吐比 FasterTransformer 高 36.9 倍。

14. 收尾：每个手段各攻哪一项

手段	缩的是显存公式里的哪一项	有损？	课上的数 / 代价
GQA / MQA	KV 头数 K，缓存缩 N/K 倍	是（GQA 论文说不掉，DeepSeek 说掉）	K = 8 仍快；MQA 没人用；Llama 例子 1:5 后延迟吞吐双改善
MLA	每个 token 的 KV 维度压成 C 维潜在量	是（表上和 MHA 持平）	16k → 512；RoPE 要另加维度
CLA	存 KV 的层数	是	把精度对缓存大小的 Pareto 前沿往外推
滑动窗口 / 混合层	缓存的 token 数 S → 窗口大小	是，损表达力	要和全注意力层交替
线性注意力 / SSM	S → 固定大小的状态	是，needle in a haystack 会丢	比滑窗"有更多余地"
压缩 + 稀疏（DeepSeek）	S：每 M 个压 1 个，再轻量打分选子集	是	课上只过了一张图
量化	每元素字节	是	BF16 → int4 / int3；QAT 贵，PTQ 便宜
剪枝 + 蒸馏	参数 P	是，要 post-train 修复	15B → 8B，精度基本不掉
speculative decoding	不缩字节，每轮多出几个 token	否，精确	草稿 3—4 个是甜点
continuous batching	把 B 一直填满	否	attention 各算各的，MLP 合并
paged attention	显存浪费（两种碎片）；前缀共享	否	块大小 4 的例子；copy-on-write
多副本	吞吐 × M，延迟不变	否	分片方案见 scaling book

- **讲者的总结：**推理极其重要；同一个模型，却是和训练完全不同的任务——内存受限，线上还是动态的。量化、新架构、剪枝蒸馏、speculative sampling，全都由一条原则驱动：**缩 KV cache，别太伤精度**；再加上分页、投机执行这些从系统领域借来的思想，服务器才跑得起来。
- **没来得及展开、但讲者认为潜力最大的：**新架构——状态空间模型、线性注意力、扩散模型。KV cache 和 attention 的构造方式让 Transformer 从根子上就是一个"对推理不友好"的架构；如果能设计一个像 Transformer 当年那样成功、但一开始就为推理而设计的架构，可能打开很大的空间（第 4 讲是这条线的架构侧）。下一讲 Tatsu 回来讲 scaling laws 下。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
推理的用途与成本	每天 8.6T token 对一个模型 32T 的训练 token：不到 4 天	1:40	—
三个指标	TTFT、latency、throughput 各对应交互式还是批处理	5:14	scaling book
张量记法与 Transformer block 电路图	红色收缩维、黑色普通维、蓝色批维；attention 里那个蓝色 B (10:53)	9:50	同上
瘦矩阵乘的强度推导	字节 2BD + 2DF + 2BF 对 FLOPs 2BDF；B 远小于 D、F 时约等于 B；H100 要 295	14:59	同上
朴素解码与 KV cache 动画	先看整段重算的版本，再看 prefill 填缓存、每一步只送一个 token (23:16)	20:41	—
attention 的强度与四格总结	因子 ST/(S+T)：prefill S/2、generation 不到 1；总结表 (33:37)	29:59	—
Llama 2 13B 的计算器	显存 838 MB × B + 26 GB；B = 1 → 0.008 s、124 token/s；B = 256 爆显存 (41:22)	39:16	Llama 2
GQA 的每样本耗时图	MHA 最慢；K = 1 最快；K 到 8 仍好，再大就涨	47:31	GQA
MLA 示意与精度表	潜变量 512 维、RoPE 另加维度；表里 MLA 略高于 MHA、GQA 偏低 (53:39)	52:06	DeepSeek-V2
CLA 的 Pareto 图	同一方法扫 K 与头维得一条曲线；CLA 的曲线在外侧	56:10	Brandon et al., 2024
滑窗、隔层与混合层	局部窗、隔开的层、global + local；全注意力层与局部层交替	57:12	—
AWQ 示意	整块 FP16 → int3，大激活通道对应的那几列留 FP16	1:06:54	AWQ
speculative decoding 动画与算法	小模型一串、大模型批一下；接受概率 min(1, q/p) (1:14:02)；草稿数 3—4 是甜点 (1:15:36)	1:13:01	Leviathan et al., 2023
Orca 与 paged attention 示意	参差的 batch 逐步解码、完成即出；块大小 4、块表、copy-on-write 分裂 (1:21:12)	1:17:37	Orca · vLLM

提到的工作

名称	在本讲里的作用
How to Scale Your Model (Google, 讲者称 scaling book)	本讲的骨架：记号、Transformer 电路图、推理章的图；多卡推理请读它

名称	在本讲里的作用
vLLM (Kwon et al., 2023)	开源推理引擎的默认选择；paged attention 的出处
SGLang · TensorRT-LLM · llama.cpp	分别：适合 agent 负载、NVIDIA 的快而窄、在 CPU 上跑
Llama 2 (Touvron et al., 2023)	13B 版本做记账例子：26 GB 参数、每序列 838 MB KV cache
NVIDIA H100 · B200	80 GB 显存、295 FLOPs/byte 的分界线；B200 能开更大的 batch
GQA (Ainslie et al., 2023)	跨头共享 K、V；每样本耗时图；缩 N/K 倍
MQA (Shazeer, 2019)	K = 1 的极端，"没人用，太差"
DeepSeek-V2 (2024)	MLA：16k 维压到 512；顺带指出 GQA 其实掉点
RoPE (Su et al., 2021)	与 MLA 不兼容，要另加维度
Cross-Layer Attention (Brandon et al., 2024)	跨层共享 KV；Pareto 前沿外推（应为这篇）
滑动窗口 / 局部注意力、global + local、混合层	缩 S 的老办法；有损，要与全注意力交替（第 3 讲有各家的配比）
线性注意力 · Mamba · Gated DeltaNet	问答里与滑窗的对比：固定状态、更有余地、但会丢信息（第 4 讲主角）
DeepSeek 的压缩 + 稀疏注意力	每 M 个压 1 个、轻量打分选子集；名字与出处见第 9 节小注（推断）
扩散语言模型	非自回归，另一条快路，只点了名
GPTQ (Frantar et al., 2022)	训练后量化：Hessian 逐层、误差传给未量化的权重
AWQ (Lin et al., 2023)	激活感知：大激活通道的权重留高精度
Minitron (Muralidharan et al., 2024, NVIDIA)	剪枝 + 蒸馏：15B → 8B（应为这篇）
Speculative Decoding (Leviathan et al., 2023) · Speculative Sampling (Chen et al., 2023)	无损加速；接受概率 $\min(1, q/p)$ ；同期两篇
Orca (Yu et al., OSDI 2022)	continuous batching 与 selective batching

术语对照

English	中文
inference	推理：模型训好后拿 prompt 产出回复

English	中文
prefill / decode (generation)	预填充（并行编码 prompt、填 KV cache） / 逐 token 解码
autoregressive	自回归：下一个 token 依赖已生成的全部 token
TTFT (time to first token)	首 token 时间：prefill 的耗时
latency / throughput	延迟（单个查询每 token 多久） / 吞吐（系统合计每秒多少 token）
arithmetic intensity	算术强度：每搬一个字节做多少 FLOPs
accelerator intensity	加速器强度：FLOP/s ÷ 内存带宽，H100 约 295
compute-bound / memory-bound	算力受限 / 内存受限
HBM (high bandwidth memory)	高带宽显存：参数和 KV cache 住在这里，每步都要搬
contracting / batching dimension	收缩维（两边都有、结果里消失） / 批维（两边都有、结果里保留）
KV cache	缓存的历史 key / value：每序列、每 token、每层、每 KV 头各一对 H 维向量
MHA / MQA / GQA	多头 / 多查询（K = 1） / 分组查询注意力（K 组共享 K、V）
MLA (multi-head latent attention)	多头潜变量注意力：KV 压成低维潜变量再展开
CLA (cross-layer attention)	跨层注意力：若干层共用同一份 KV
sliding window / local attention	滑动窗口 / 局部注意力：只看最近 k 个 token
hybrid / interleaved layers	混合层：全注意力层与局部或线性层交替
linear attention / SSM	线性注意力 / 状态空间模型：历史压成固定大小的状态
needle in a haystack	大海捞针：长上下文检索测试
Pareto frontier	帕累托前沿：精度与缓存大小的最优取舍曲线
quantization	量化：降低数的精度
QAT / PTQ	量化感知训练 / 训练后量化
scale / zero point	量化的缩放系数 / 零点
Hessian	海森矩阵：GPTQ 用它衡量量化误差的影响
activation channel	激活通道：AWQ 按它挑要保护的权重
pruning / distillation	剪枝 / 蒸馏
calibration set	校准集：估计重要性、定量化参数用的小数据集
speculative decoding / sampling	投机解码 / 投机采样

English	中文
draft model / target model	草稿模型 / 目标模型
rejection sampling	拒绝采样
residual distribution	残差分布：q 比 p 多出来的部分归一化
continuous batching	连续批处理：batch 成员随时进出
selective batching	选择性批处理：attention 各算各的，MLP 合并算
paged attention	分页注意力：KV cache 按块存放，块表寻址
internal / external fragmentation	内部碎片（预留没用上） / 外部碎片（空隙太小）
block table	块表：序列的逻辑块到物理块的映射
copy-on-write	写时复制：共享的块在分叉时才复制
prefix caching	前缀缓存：系统提示等公共前缀的 KV 只算一次
OOM (out of memory)	显存溢出
diffusion language model	扩散语言模型：非自回归生成

字幕勘误

"v l m / VLM paper" → vLLM；"s g lang" → SGLang；"tensor r t" → TensorRT–LLM；"llama c p p" → llama.cpp；"t t f t" → TTFT；"chat g p t" → ChatGPT；"HPM / HPN" → HBM；"mammal / mat mals / matmal" → matmul；"up projection dimension in NLP" → MLP；"bottom neck" → bottleneck；"each 100" → H100；"generated nation intensity" → generation intensity；"JQ / JQA" → GQA；"multi-layered attention / multi-latent attention" → multi-head latent attention (MLA)；"rope" → RoPE；"GateNet" → Gated DeltaNet；"N3" → int3；"FE16" → FP16；"logic / logics" → logits；"KV fit cache" → KV cache；"page attention" → paged attention；"student naively" → do it naively；"Tats" → Tatsu；"GPT–4, which came out earlier this year" → 模型名存疑（见第 1 节小注）；"transpose slide"（18:37 的提问）→ 应是 K 与 G 的定义那一页。

带走的问题

- 用第 6 节的公式给 Llama 2 13B 换个配置记账：上下文 4096、B = 16 时 KV cache 多大、延迟多少、还装得进 80 GB 吗？把 K 从 40

改成 8 (GQA) 再算一遍——哪一项变了、哪一项纹丝不动？吞吐的渐近线各是多少？

2. 第 5 节说 generation 时 attention 的强度约为 1 且不随 B 涨。MLA 把每个 token 的 KV 从 $2 \cdot N \cdot H$ 压到 C，改变的是字节还是 FLOPs？强度变了吗？既然强度没怎么变，延迟为什么仍然改善——这和“内存受限时只看字节”是一回事吗？
3. speculative decoding 是无损的，那它的加速从哪来？若草稿模型与目标模型几乎一样（接受率接近 1），每轮目标模型一次 forward 最多换来几个 token？若草稿就是 MQA 版的目标模型，第 8 节的 GQA 掉点还重要吗？
4. continuous batching 把 B 填满，paged attention 让 B 能更大，可第 7 节说 B 大延迟就差。一个线上聊天产品应当怎么在 TTFT、每 token 延迟和吞吐之间定 B？讲者没讲的 prefill / decode 分离到不同卡上跑（disaggregated serving），按本讲的四格表看，解决的是哪一格的问题？
5. 讲者说 Transformer “从根子上对推理不友好”，第 4 讲的 Gated DeltaNet 状态固定。用本讲的账说明一个线性注意力层在 decode 时读的字节随 B 怎么变，它更像 MLP 还是更像 attention？既然如此，混合模型为什么还要保留少量全注意力层（第 9 节问答的 needle in a haystack）？

CS336 第 11 讲 | Scaling laws（下）（Scaling Laws II）

Stanford CS336: Language Modeling from Scratch（2026 春）· 第 11 讲（课程表：2026 年 5 月 4 日），Tatsunori Hashimoto 的第二堂 scaling laws 课，上接第 9 讲，中间隔着 Percy Liang 的第 10 讲推理 视频：<https://www.youtube.com/watch?v=vTfEyOyzV9E>（1:17:03；英文字幕为自动生成，模型名、人名和 μP / Muon 这类词错得多，见文末勘误） 讲者：Tatsunori Hashimoto（全程；讲 μP 推导时自称 Tatsu，多次说“两讲之前我讲过 batch size”） 课程主页：<https://stanford-cs336.github.io/> · 本讲围绕的论文：MiniCPM、DeepSeek LLM、Step Law、Wen et al. 的优化器大对比、A Spectral Condition for Feature Learning、Lingle 的 μP 压力测试

一句话：第 9 讲的 Kaplan / Chinchilla 是 2022 年以前的“正典”，这一讲看真正训大模型的人（MiniCPM、DeepSeek、Qwen、Kimi K2、Hunyuan、Llama 3、MiniMax-01、StepFun）怎么把 scaling law 用到实处——核心矛盾是学习率、batch size、初始化这些敏感超参会随规模漂移，而对付它只有两条路：要么像 DeepSeek / StepFun 那样在小模型上网格搜索、把最优超参本身拟成算力或数据量的幂律再外推；要么像 MiniCPM / Cerebras 那样用 μP 改逐层初始化和逐参数学习率，让最优学习率干脆不随宽度变。配套的 WSD 学习率让 Chinchilla 式实验从“每个数据量从零重跑”变成“回滚再衰减一次”；优化器（Muon）的故事说明小规模胜利到大规模会缩水甚至崩掉，而 Chinchilla 比是最常被忽略的混杂变量。讲者的结论：scaling 至今是一门手艺，没有银弹。

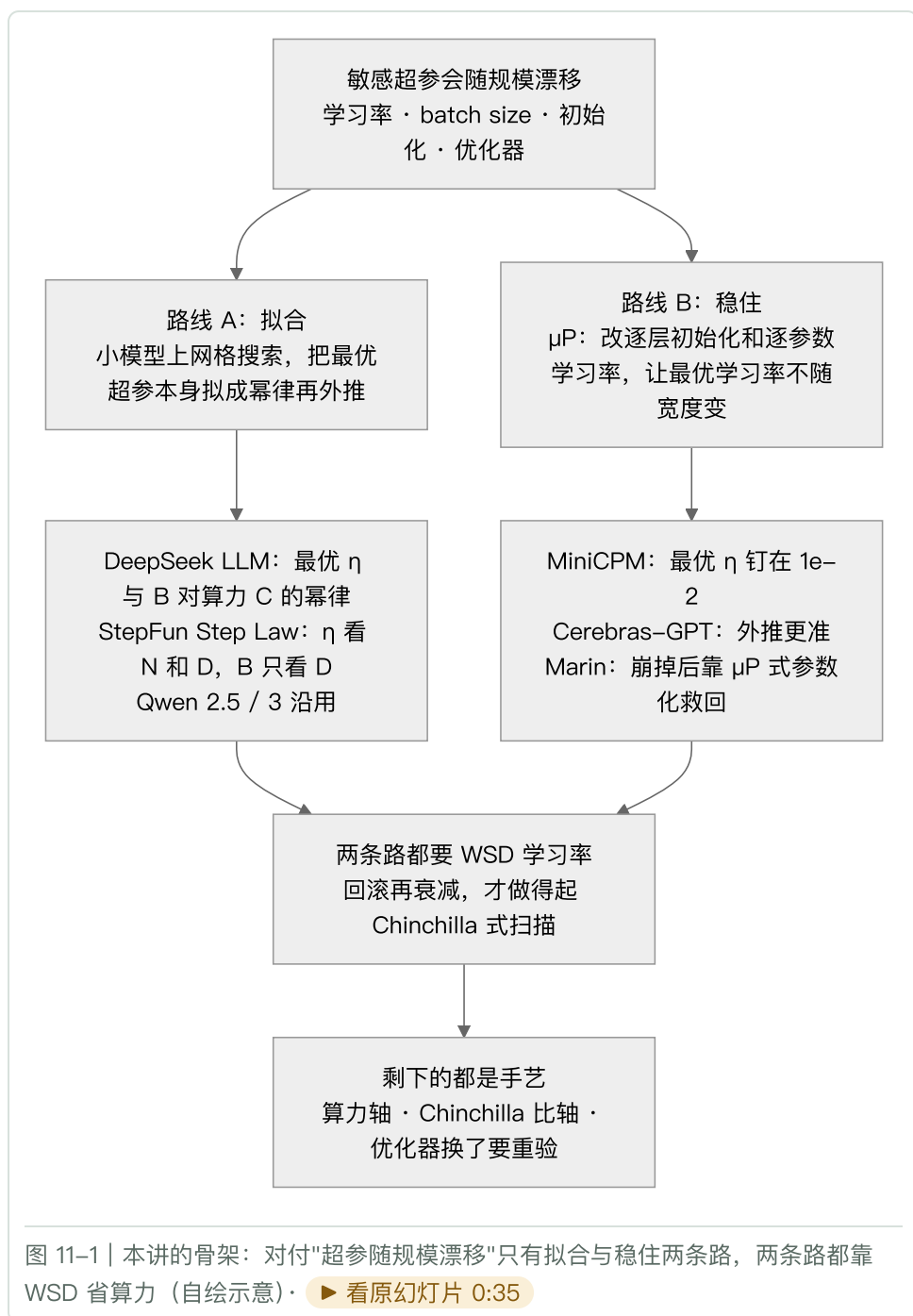
时间轴

时间	内容
0:05	开场：三个问题——经典 scaling law 在真实开源规模上灵不灵、学习率等优化细节怎么定、哪些超参随规模变敏感
1:36	2022 之后的 scaling 论文多出自中国开源社区；精讲两篇路线相反的：MiniCPM 与 DeepSeek LLM
3:39	MiniCPM： μP 的做法清单；小模型阶梯往上外推约 5 倍
7:13	μP 之下最优学习率钉在 $1e-2$ ；最优 batch size 仍随数据量按幂律变
9:45	Chinchilla 复现的痛点：cosine 要先知道终点；WSD 学习率与“回滚再衰减”
12:48	WSD 对 cosine 的曲线；用 WSD 做 Chinchilla 方法 1、3，拟出的指数与原文不同

时间	内容
15:53	DeepSeek LLM: 网格搜索 → 学习率和 batch size 对算力的 scaling law
20:01	DeepSeek 的 Chinchilla 复现 (isoFLOP、两段 decay); 小模型外推到两个正式模型
22:33	Qwen 2.5 / 3、Kimi K2 (稀疏度 48)、Hunyuan (96 token 每激活参数)、Llama 3 (sigmoid)、MiniMax-01 (架构对比); 问答: post-training 怎么进 scaling
30:42	StepFun 的超参 scaling law: 各家公式连输入变量都不同; 1B / 100B 切片的等高线
35:19	关键发现: batch size 只看数据量; 学习率随模型减、随数据增; 迁到 MoE 还行, 换数据会漂
39:54	问答: 直接用别人的 law 还是自己 grid; "scaling law 有一大半是 vibes"
41:26	优化器: nanoGPT speedrun 上 Muon 的领先到大规模缩水; Wen et al.: 调好 Adam 与 weight decay, 盯住算力和 Chinchilla 比两个轴
47:33	Marin 的失败曲线: 漂亮的直线在 1e20 量级之后崩掉
49:36	Muon: 矩阵参数、Newton-Schulz 正交化、Kimi K2 全程使用; 问答: SVD、超参、交互
57:14	μP : 目标、三个旋钮、Cerebras-GPT 的验证; 两个不变量 (激活 $O(1)$ 、一步之后的变化 $O(1)$)
1:02:54	A1、A2 的推导: σ 与 η 的缩放规则 (SGD 用 $\text{fan_out} / \text{fan_in}$, Adam 用 $1 / \text{fan_in}$)
1:12:41	μP 压力测试: SwiGLU、RMSNorm 大多没事; 学 gain、Lion、大 weight decay 会破; 收尾: scaling 是手艺, 没有银弹

核心内容

1. 这一讲的位置：正典之后，前沿怎么做



- **上接第 9 讲：**Kaplan、Hestness、Chinchilla 三篇是 2022 年以前的"经典正典"。今天问三个问题：(1) 这些东西在真实的开源模型规模上灵不灵；(2) 学习率之类的优化细节怎么定——这是"造模型的人该懂"的常识；(3) 优化器的行为随规模变，所以初始化、学习率、batch size 都得作为规模的函数来想。
- **讲者的观察：**2022 年之后，由真正训大模型的团队写的 scaling 论文数量在减少，而且多数来自中国的开源社区。今天挑出一串，目标是"从 Chinchilla 速通到 Kimi K2"——后者是最近还给出 scaling 细节的发布。

- **重点两篇**：MiniCPM（2024）与 DeepSeek LLM。都是认真做过 scaling 研究、模型也够强的团队，而且对同一个问题走了相反的路：一个想把超参稳住，一个想把超参拟出来。图 11-1 是全讲的骨架。
- **一个趋势**：像"怎么做 Chinchilla、怎么定学习率"这些已被当成常识，新报告（Qwen 3 之类）不再展开，所以反而是这几篇老一点的论文最有教学价值。 μP 第 9 讲只提了一嘴，这里是第一次真正展开。

2. MiniCPM：用 μP 稳住学习率、拟合 batch size、用 WSD 省算力

- **背景**：中国开源社区产学合作的小模型，2024 年时是 1-2B 档的最强（现在 Gemma 的 2B / 3B 更强）。
- **研究设计**：目标不是暴力训几百个 1.5B 模型，而是训一串小得多的模型（scaling ladder），一次把超参定对；阶梯上最大的模型和最终发布的模型差约 5 倍。要钉住的是公认最敏感的几个量：最优 batch size、最优学习率，以及 token 与参数量的配比（顺带复现 Chinchilla）。
- **μP 的做法清单**（MiniCPM 论文列的五条）：缩放 embedding 的输出；残差连接按层数的平方根缩放；矩阵形状的张量按 fan-in / fan-out 比调初始化；逐张量设学习率（讲者提醒：没见过 per-parameter 学习率的人会觉得这条很"异类"）；缩放 LM head。原理留到第 8 节推导，这里只需知道：它改的是初始化和学习率，图的是最优学习率不随规模变。
- **结果 1：最优学习率不动了**。按模型大小逐条扫学习率，每条曲线的最低点几乎都在 10^{-2} （最小的模型略偏，也很接近）。这是 μP 的成功案例——做到这一步，学习率就不用再调了。
- **结果 2：batch size 稳不住，但能拟**。它随数据量和模型大小变。做法：每个固定的 batch size 跑一条训练曲线（图上一列点），沿等 loss 的等高线拟二次曲线、取最低点，得到"达到这个 loss 最省的 batch size"；结论是最优 batch size 对目标 loss 呈幂律——正是第 9 讲 Kaplan 的 critical batch size：loss 越低，batch 该越大。用法：先由 scaling law 反推目标 loss，再由这条幂律定 batch size。

$$B_{\text{crit}}(L) \approx \frac{B^*}{L^{1/\alpha_B}}$$

B_{crit} 是"再增大 batch 就开始浪费样本"的临界 batch size（以 token 计）， L 是训练要达到的 loss， B^* 和 α_B 是拟合常数；loss 越低，允许

的 batch 越大。MiniCPM 这条曲线和第 5 节 StepFun 汇总表第一行的 OpenAI 公式都是这个形状。

小注：Kaplan et al., 2020 的拟合值是 $B^* \approx 2 \times 10^8$ token、 $\alpha_B \approx 0.21$ ；字幕把幻灯片上的常数听成了"2e18"。

- **结果 3：WSD 学习率。**痛点在 Chinchilla 式实验：固定 FLOPs 下要换 token 数和模型大小，而 cosine 调度必须在开训前知道总步数，所以 8M 序列的跑法不能从 4M 序列那一跑的末尾续上，只能从头再来——代价近似是平方级的。MiniCPM 的解法是现在人人叫的 warmup—stable—decay，一个梯形：
 - warmup：固定步数（不是总步数的比例），与训练总长无关；
 - stable：学习率恒定，占绝大部分；
 - decay：末尾快速衰减，通常占总步数的 10—20%，讲者说一般衰到最大学习率的约 10%。
- **为什么好：**stable 阶段的任何一个 checkpoint 都能当起点——回滚到它，继续 stable 往前跑，再 decay 一次；每次多花的只是 decay 那约 10%。于是"数据轴上的扫描"不必反复重跑预训练（图 11-2）。
- **曲线长什么样：**和调好的 cosine（橙线）比，WSD 在 stable 阶段看着一直落后，进入 decay 后突然把差距追回，持平甚至反超。讲者的圈内印象：很多场合 cosine 略好，WSD 基本一样好，是一个很通用的默认；另一个看点是"衰减学习率"这件事本身对最后的 annealing 有多大作用——没玩过的人会吃一惊。

小注：讲者说的"Martin Jaggi 那边做过大型的 WSD 研究"应为 Hägele et al., 2024——出自 EPFL，不是 ETH。

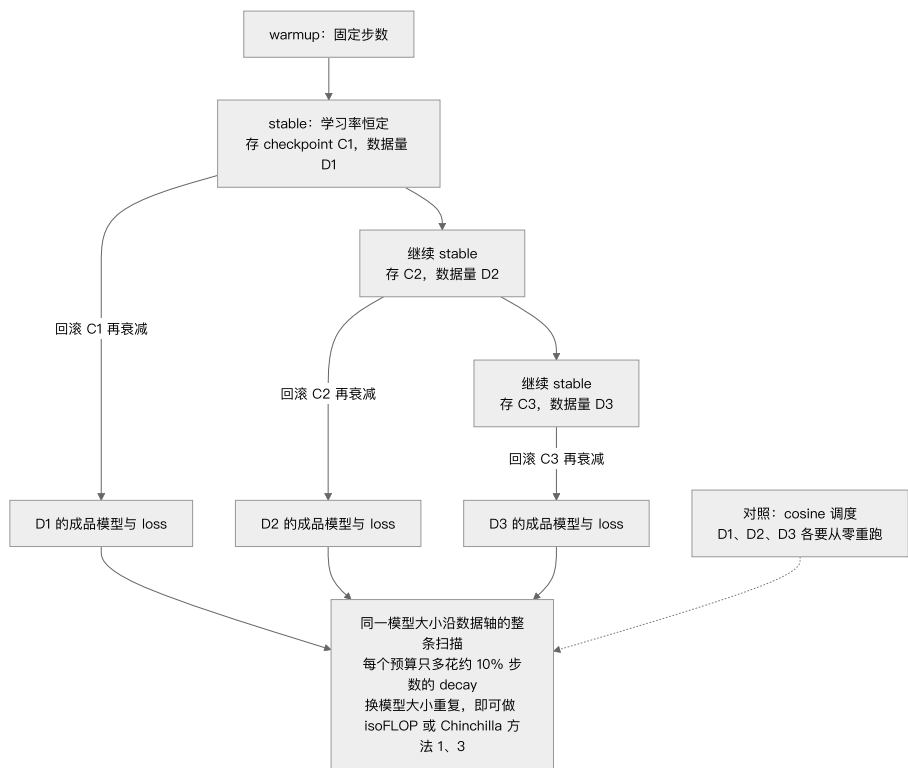


图 11-2 | WSD 的"回滚再衰减": 一条 stable 主线加若干次短衰减, 换来数据轴上的整条扫描 (自绘示意) · [▶ 看原幻灯片 11:16](#) · 出处: Hu et al., 2024

```
# WSD 之下做"数据轴扫描": 一条 stable 主线, 多次短衰减
snaps = []
for step in range(total_steps):
    lr = warmup(step) if step < warm else lr_max # stable 段恒定
    train_step(model, lr)
    if step in budgets: # 4M、8M、16M 序列.....
        snaps.append((step, model.snapshot()))
for step, snap in snaps: # 每个预算只多花 decay 那 10%
    m, n_decay = snap.copy(), int(0.1 * step)
    for k in range(n_decay):
        train_step(m, lr_max * (1 - k / n_decay)) # 线性衰到 0 附近
    record(step, evaluate(m))
```

这段说明 WSD 为什么把 Chinchilla 式实验从"每个预算重跑"变成"每个预算多衰减一次": stable 主线只跑一遍。

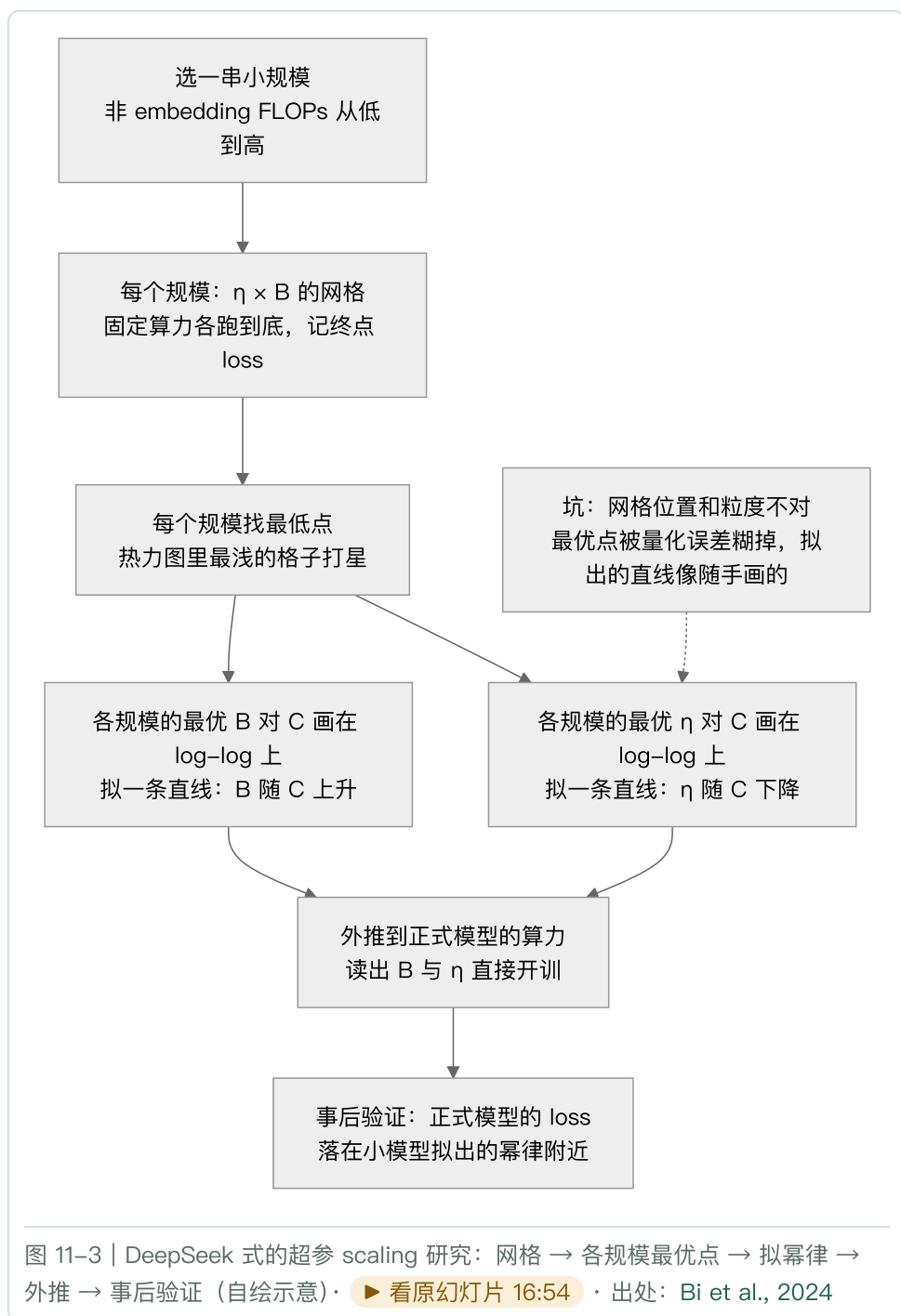
- **Chinchilla 复现**: 有了 WSD, 一条长跑加多次回滚衰减就得到数据维度的扫描, 再换模型大小就能做 isoFLOP 或别的分析。MiniCPM 选

了复现 Chinchilla 的方法 1 (lower envelope) 和方法 3 (参数化拟合) ——讲者认为是第 9 讲讲过的三种方法里最不可靠的两种——曲线倒还好看：方法 1 的下包络基本是直线，方法 3 在算力对非 embedding 参数的关系上也平滑。但联合拟合的指数与 Chinchilla 差别很大，论文据此声称每个参数应配远多于 Chinchilla 的 token；讲者说不清是真的还是拟合本身有问题。

小注：MiniCPM 论文给的算力最优配比约 192 token / 参数，是 Chinchilla 的 20 的近 10 倍。

- **带走两点：** μP 是稳住学习率的有用技巧；WSD 是人人该知道的基本功。

3. DeepSeek LLM：不稳住，直接把最优超参拟成 scaling law



- 讲者对这篇（MoE 之前的第一篇 DeepSeek 论文）评价很高：开源世界里执行得最漂亮的 scaling 分析之一，实验品味好，"R1 之前就看得出这群人会走得远"。
- **策略**：不做 μP ，承认学习率和 batch size 随规模变，但只要变化可预测就够——直接给它们拟 scaling law。做法见图 11-3：各规模上扫学习率与 batch size 的网格，固定算力看终点 loss，取最低点；把各规模的最优值对非 embedding 训练 FLOPs 画出来拟直线。正式模型（图上的星）在更高算力处取了大得多的 batch 和小得多的学习率。

- **讲者的挑剔：**batch size 那条确实像直线；学习率那条"不是我这辈子见过最好的线性拟合"，但模型训得出来、结果合理，能用。这条路的弱点：网格不在对的位置就有量化误差，拟出来像一条随手画的线。学生问同一算力下为什么最优学习率散得那么开——讲者：因为同一算力下模型大小等还在变。

$$\eta_{\text{opt}} = a C^{-\alpha}, \quad B_{\text{opt}} = b C^{\beta}$$

C 是非 embedding 的训练 FLOPs, η_{opt} 与 B_{opt} 是网格里找到的最优学习率与最优 batch size, a、b 是 log-log 直线的截距、 α 、 β 是斜率；算力越大，学习率越小、batch 越大。

小注：DeepSeek LLM 论文的拟合值是 $\eta_{\text{opt}} = 0.3118 \cdot C^{(-0.1250)}$ 、 $B_{\text{opt}} = 0.2920 \cdot C^{(0.3271)}$ ；图上两颗星是 7B 和 67B，各训 2T token。

- **Chinchilla 复现：**2024 年是"每个开源团队都把整套 scaling law 重做一遍"的年代。DeepSeek 也用 WSD 式调度，只是有个怪变体——两段 decay 而不是一段（讲者不知道为什么，之后也没流行）。因为选了 isoFLOP，曲线比 MiniCPM 干净得多：固定算力下的扫描和模型—数据的权衡都很整齐，得到与 Chinchilla 相似的配比。要点：Chinchilla 在别人的、够大的算力范围上被复现了，那套分析是站得住的。

小注：DeepSeek LLM 的"多步"调度：warmup 2000 步到峰值，跑完 80% token 时降到峰值的 31.6%，90% 时降到 10%。

- **收官图：**小模型拟出的幂律（灰点）外推到两个正式模型（星），预测相当接近——"可以更好，但一点也不差"。讲者遗憾现在的发布报告里这种图越来越少。
- **一句话总结：**对付敏感超参有两条路，稳住（ μP ）或拟合（scaling law），DeepSeek 代表后者。

4. 之后的开源报告各用 scaling law 定什么

报告	用 scaling law 回答什么	课上给的数字或结论
Qwen 2.5 → Qwen 3	最优 batch size 与学习率，含 MoE	与 DeepSeek 分析同一套；Qwen 3 直接说"沿用 2.5 的做法"，已是标准配方
Kimi K2	MoE 稀疏度对 loss 的关系	同一算力下越稀疏 loss 越低；取稀疏度 48，因为再往上收益递减
Hunyuan（腾讯的 MoE）	每个激活参数配多少数据	固定稀疏度，得到 96 token / 激活参数

报告	用 scaling law 回答什么	课上给的数字或结论
Llama 3	isoFLOP 配比；loss 到下游准确率的映射	配比与 Chinchilla 略有不同；准确率对 loss 呈 sigmoid
MiniMax-01	架构选择：lightning attention / softmax / 混合	三者 scaling 基本相当、所需参数量相近，据此选混合架构

- **MoE 的必修课**：换成 MoE 的公司都得重做一遍"激活参数多少、稀疏多少"的 scaling law——固定算力、换稀疏度看 loss，量化了稀疏度与 loss 的关系才能理性地决定放多少稀疏。这是 scaling law 在架构决策上的用法。
- **MiniMax-01** 是第 9 讲说的"scaling law 的用处之一：比较架构变体"的现成例子——复现 Kaplan 那种"算力增大时学习曲线怎么变、各架构需要多少参数"的图，结论是混合、线性（lightning attention）、softmax 三者相当，于是放心用混合架构上线。
- **Llama 3 那张图**：左边是算力增大时 benchmark 答案上的 log prob（负对数似然）稳定下降；右边是 log loss 越好、下游准确率越高，作者在紫点上拟了一条 sigmoid。讲者不信它是"唯一真理"（有系统性偏离），但承认 loss 与下游准确率之间常有紧耦合——我们一直只谈 log loss，这一步把它接到了准确率上（第 12 讲评测再展开）。

$$\text{Acc(NLL)} \approx \frac{1}{1 + \exp(-k(\text{NLL}_0 - \text{NLL}))}$$

NLL 是模型在 benchmark 正确答案上的负对数似然，Acc 是下游准确率，k 和 NLL_0 是拟合参数；两步走：先由算力预测 NLL，再由 NLL 预测准确率。

小注：Llama 3 报告用 ARC Challenge 演示这两步，其 isoFLOP 拟合给出 3.8×10^{25} FLOPs 下最优 402B 参数、16.55T token。Kimi K2 的"稀疏度 48"是 384 个 expert 里每个 token 选 8 个（ $384 / 8$ ）。

- **为什么新报告越来越简略**：核心机器（Chinchilla、学习率 scaling）人人会了，再放进论文没有增量价值——这是讲者的猜测。
- **问答：post-training 标准化之后，scaling 里什么变了？** 讲者：仍是大开放问题。没有把 post-training 一起算进去的"整体 scaling"，因为 post-training 有时会改变"该做什么样的 pre-training"；最接近的是研究预训练的覆盖度或多样性能否预测 post-training 的表现，还很初步（第 15–16 讲）。

5. StepFun 的超参 scaling law: batch 只看数据量, 学习率随模型减、随数据增

- **为什么选它**: 超参、尤其学习率至今没有真正可靠的研究, StepFun 的这篇预印本最接近——他们训的是可信的大模型, 又烧了大量算力做网格。

小注: 论文摘要: 从零训了 3,700 多个模型、合计 100T token、近 100 万 H800 GPU 小时; 拟出的最优点与穷举最优在测试集上的 loss 只差 0.094%。

- **各家公式连输入都不一样**: 论文有张汇总表。第一行 OpenAI (Kaplan): batch size 是终点 loss 的函数 (critical batch size); DeepSeek: 学习率和 batch 都是算力的函数; MiniCPM 也有一套; StepFun 自己: batch 是数据量的函数。大家对"哪些变量该进公式"都没共识。讲者的提醒: 最后一行也别当圣经, 这些公式脆, 但实验现象值得学。
- **实验设计**: 和 DeepSeek 一样, 训一大堆模型把学习率 $\times$ batch size 的空间铺满, 跨模型大小和数据量; 只是粒度高得多, 能画出等高线。
- **超参地形是凸的、几乎光滑的**: 1B 参数、100B token 的切片上, 固定 batch 看学习率、或固定学习率看 batch, 每一条都像个碗。意义: 网格搜索这条路可行, 最低点不会离得太远; 要是锯齿状, 整个方案都要打问号。顺带给了"超参空间大体光滑"的直觉。
- **最稳的规律: 最优 batch size 只依赖数据量 D**。右图 x 轴是 D, 不同颜色是不同模型大小, 全落在同一条趋势线上; log-log 下是 D 的幂律。讲者说在相关论文里没见过反例。
- **学习率更微妙**: 模型越大最优学习率越小; 数据越多最优学习率越大——后一条反直觉, 可能脆弱, 也有论文认为 D 的方向相反; 但在这组实验里趋势清楚。
- **学习率其实很皮实**: 明天让你训一个语言模型, $1e-3$ 还是 $1e-4$? 大家心里都有这个范围, 它通常就是好起点。
- **迁移**: 这套 law 一定程度能迁到 MoE (黄星是预测, 红叉是直接 MoE 上搜到的最优), 只要按激活参数控制; 但换训练数据时最优学习率和 batch 会漂——说明这些常数依赖数据, 比别的都更"看情况"。

$$\eta_{\text{opt}}(N, D) = c_1 N^{-\alpha} D^{\beta}, \quad B_{\text{opt}}(D) = c_2 D^{\gamma}, \quad \gamma \approx \frac{1}{2}$$

N 是参数量, D 是训练 token 数, c_1 、 c_2 、 α 、 β 、 γ 是拟合常数; 学习率随 N 减、随 D 增, batch size 与 N 无关、大约随 D 的平方根增长 (课上的说法是"根号 D 乘常数")。

小注: 论文的拟合值是 $\eta = 1.79 \cdot N^{(-0.713)} \cdot D^{(0.307)}$ 、 $B = 0.58 \cdot D^{(0.571)}$ 。

- **和 DeepSeek 对上号:** 按 Chinchilla 配比时 N 和 D 都由算力驱动, N 压学习率、 D 抬学习率, 两下部分抵消后总体上算力越大学习率越小、batch 越大——就是 DeepSeek 那种形状, 只是指数不同。
- **问答: 拿别人的 law 用, 还是自己 grid?** 看算力预算和"必须多准"。若你的算力范围接近人家网格覆盖的范围, Step Law 是很好的默认值, 讲者宁愿信它也不信拍脑袋; 但真做自己的大跑, 总有小差异 (比如你坚持用大 weight decay), 不知道会不会一样地 scale, 所以各家报告都在重做 Chinchilla——为的是确认"一阶正确"。讲者的金句: scaling law 看着像科学, 其实一大半是 vibes——你永远无法确定人家的实验设置和你的足够像。

6. 优化器: 小规模胜利, 到大规模还剩多少

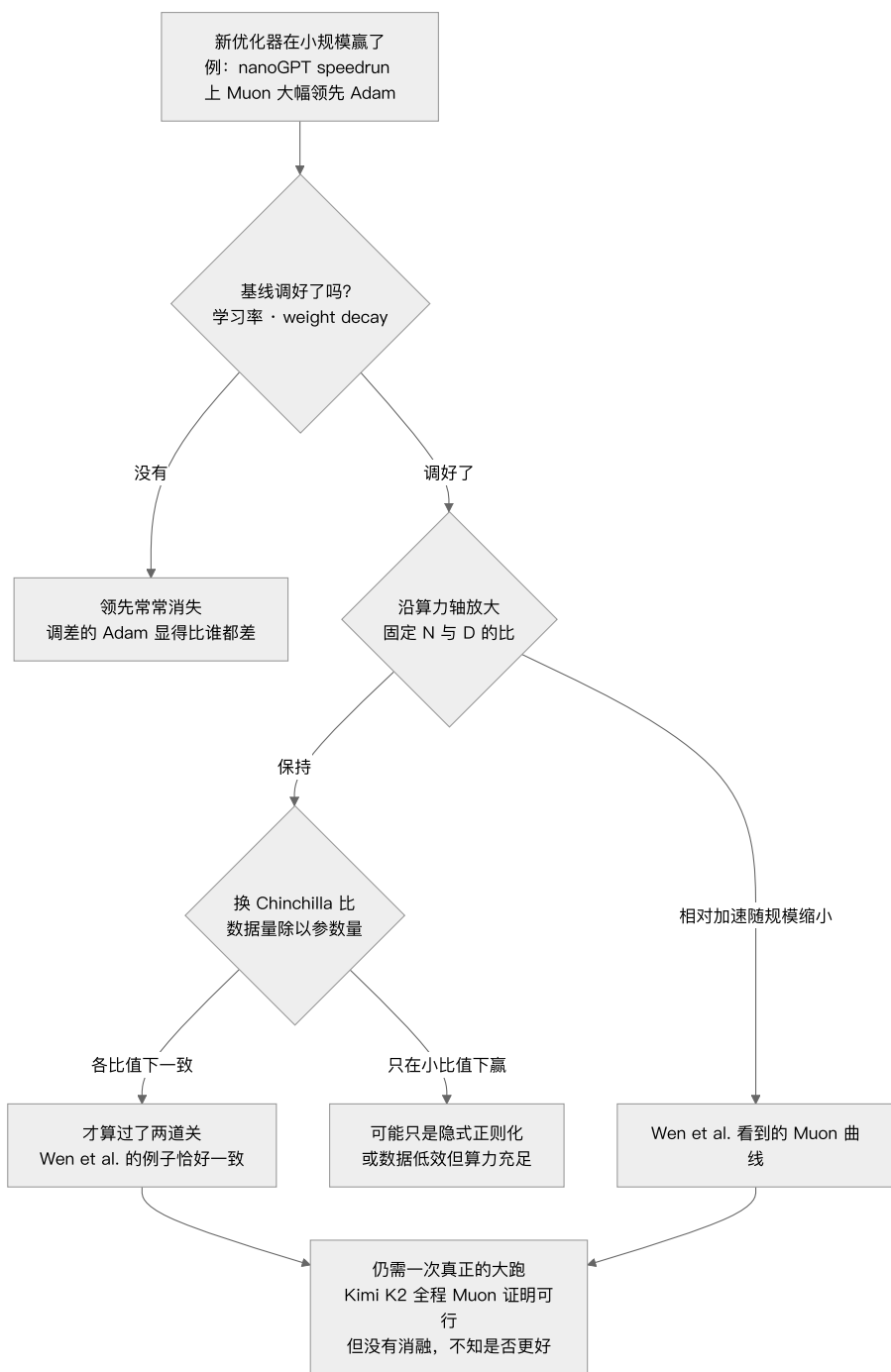


图 11-4 | 评价一个随规模而变的算法：先调基线，再沿算力轴、再沿 Chinchilla 比轴，最后仍要一次大跑（自绘示意）· [► 看原幻灯片 45:02](#)

- **为什么放在这讲：**第 3 讲架构课太满没讲优化器，而优化器是随规模变的東西，正好放在 scaling 里讲。
- **动机图：**nanoGPT speedrun（作业 1 的灵感来源）——很小的模型、很短的时间，比谁先到 3.x 的 loss，很多人在上面爬山。榜上最显眼的事件是 Muon（紫线）相对 Adam（蓝线）的大幅领先，而且并不比 Adam 慢多少。问题：放大之后还好吗？有 scaling 研究说没

那么好。这是研究里最棘手的一环：优化器肯定随规模变，小规模赢了，大规模怎么办。

小注：speedrun 的目标是 FineWeb 上验证 loss 3.28，模型是 124M 的 GPT-2 small 量级。

- **Wen et al. 的大规模优化器对比**（Kaiyue Wen、David Hall、Tengyu Ma、Percy Liang）给了几条基本功和两条 scaling 教训：
 - 每个模型的最优超参不同、甚至 scaling 指数不同，所以 Adam 必须调好：调差的 Adam（深棕线）看着比一众新算法都差，学习率稍一调，“提升”就没了；
 - 学习率调好了还有 weight decay：沿用同一个小 weight decay，绿线明显更差——严格说不是 scaling 问题，是一般的实验卫生；
 - **算力轴**：固定模型大小与数据量的比，x 轴就是算力；以 Adam 为 1.0 看相对加速，Muon 在小规模很好，随规模增大提升递减——不是你想看到的图；
 - **Chinchilla 比轴**（数据量 / 参数量）：有的算法在过参数化（比值小）时好——可能是隐式正则化，或数据低效但算力多；有的在比值大时好——知识往参数里装得更高效。这是大混杂变量，连做 scaling 很讲究的论文也常忽略或没算力做；这篇里各比值下优化器之间的比例一致，但不总是这样。

小注：应为 Wen et al., 2025 《Fantastic Pretraining Optimizers and Where to Find Them》；摘要的数字约为：模型 0.1B—1.2B、Chinchilla 比 1—8 倍，最快的优化器相对调好的 AdamW 的加速从 0.1B 的约 1.4 倍缩到 1.2B 的约 1.1 倍（凭记忆，未核对原文）。

- **建立 scaling 本身就不容易——Marin 的失败曲线**：Will Held 在 Marin（Percy 的开源训练项目）里验证一套超参：Cautious AdamC（一个 Adam 变体）、batch size 按平方根缩放等标准配置，做的就是别人那种 Chinchilla 分析。因为不是论文，失败的跑也公开：到 10^{20} 量级（图上的竖虚线）之前是教科书般漂亮的直线，再往上先是差一点、差很多、然后整条跑崩掉。修法是更小心的 μP 式参数化和换一个优化器，之后跨更多数量级都顺了。教训：跨多个数量级都漂亮的趋势也可能突然咬你；相当比例的时间你会得到这种图、还不知道错在哪。

7. Muon：把矩阵参数的更新正交化

- **讲它的门槛：**课程只讲进过大规模训练的东西，Muon 现在够格 (Kimi K2)。
- **出发点：**带动量的梯度下降对所有参数一视同仁，但语言模型的参数有两类——向量参数 (RMSNorm 的 gain) 和矩阵参数 (attention、MLP 的权重)。矩阵有奇异值，可以对奇异值做文章——讲者觉得这是个有点"跳出框框"的想法。
- **算法：**算梯度 $\rightarrow$ 动量步 $B_t = \mu B_{t-1} + G_t \rightarrow$ 不直接用 B_t 更新，而是先做 Newton-Schulz (具体是 5 步迭代) 把 B_t 正交化：写成 $U S V^T$ ，把所有奇异值都变成 1，用 $U V^T$ 当更新——大的奇异值压回单位，小的抬到单位。
- **直觉：**Adagrad / Adam 逐坐标除以梯度的大小，让每个坐标的步长差不多；Muon 是在谱范数意义下让每个"方向"的步长都是单位——这只对矩阵有定义，所以 Muon 是矩阵优化器，向量参数照旧用 AdamW。
- **系统面：**Newton-Schulz 只用矩阵乘法，是正交化的有限步近似而非精确值，GPU 上高效。学生问 SVD 在 GPU 上快不快——不快，但他们不做 SVD。讲者拆成三个巧思：把矩阵当矩阵看、正交化的想法、用 Newton-Schulz 落地。

$$B_t = \mu B_{t-1} + G_t, \quad B_t = U S V^T \Rightarrow W_{t+1} = W_t - \eta U V^T$$

G_t 是第 t 步的梯度矩阵， B_t 是动量缓存， μ 是动量系数， U 、 S 、 V 是 B_t 的奇异值分解， $U V^T$ 是把所有奇异值置 1 后的正交化更新；实现里 $U V^T$ 由 5 步 Newton-Schulz 迭代近似。

```
def muon_step(params, grads, state, lr, mu=0.95):
    for p, g in zip(params, grads):
        b = state[p] = mu * state[p] + g                # 动量
        if p.ndim == 2:                                # 矩阵参数：正交化
            o = newton_schulz5(b)                       # 只用 matmul, 近似 U V^T
            p -= lr * o
        else:                                            # 向量参数：交给
            adamw_update(p, g, state, lr)
```

这段只想说明 Muon 的"分流"：矩阵参数走正交化更新，向量参数照旧。

- **故事的收尾**：先是 speedrun 上惊艳；然后 Wen et al. 等人放大后说提升缩水，讲者以为故事到此为止、没人会花大算力把它训到完整预训练规模；结果 Kimi K2 出来，整个模型全程用 Muon（加了一堆防爆的补丁，他们遇到不少不稳定）。K2 很强、训练曲线正常，所以 Muon 在大规模"能用"；比 Adam 好不好？K2 没做消融，那个规模上无人知道。

小注：Kimi K2 报告：1T 总参数、32B 激活；优化器叫 MuonClip——Muon 加 QK-Clip，用来压住 attention logit 爆炸。

- **讲者的态度**：知道一个东西在大规模上灵不灵极难；但"小规模实验再往上迁"依然是我们做科学的方式，speedrun 这类榜值得尊重，好点子最终会进大模型。会不会有人在大规模做消融、回答"Muon 是否优于 AdamW"，还是开放问题。
- **问答**
 - Muon 和 AdamW 的超参共用吗？不，各是各的。讲者顺带提 Jeremy Bernstein 的主张：每一层该有自己的学习率、甚至自己的优化器——推到极限就是 Transformer 里每个参数都不同、各配一个优化器，超参也得各调各的（讲者说自己不想干，但也许是优化的未来）。
 - 只调学习率和 batch，它们和别的超参交互怎么办？确实担心，但组合是指数级的，铺不满；实践是先狠狠地网格最敏感的（学习率永远最重要），再对 weight decay 之类做单变量扫描，求局部最优。

8. μP 的推导：两个不变量，逼出初始化和学习率的缩放

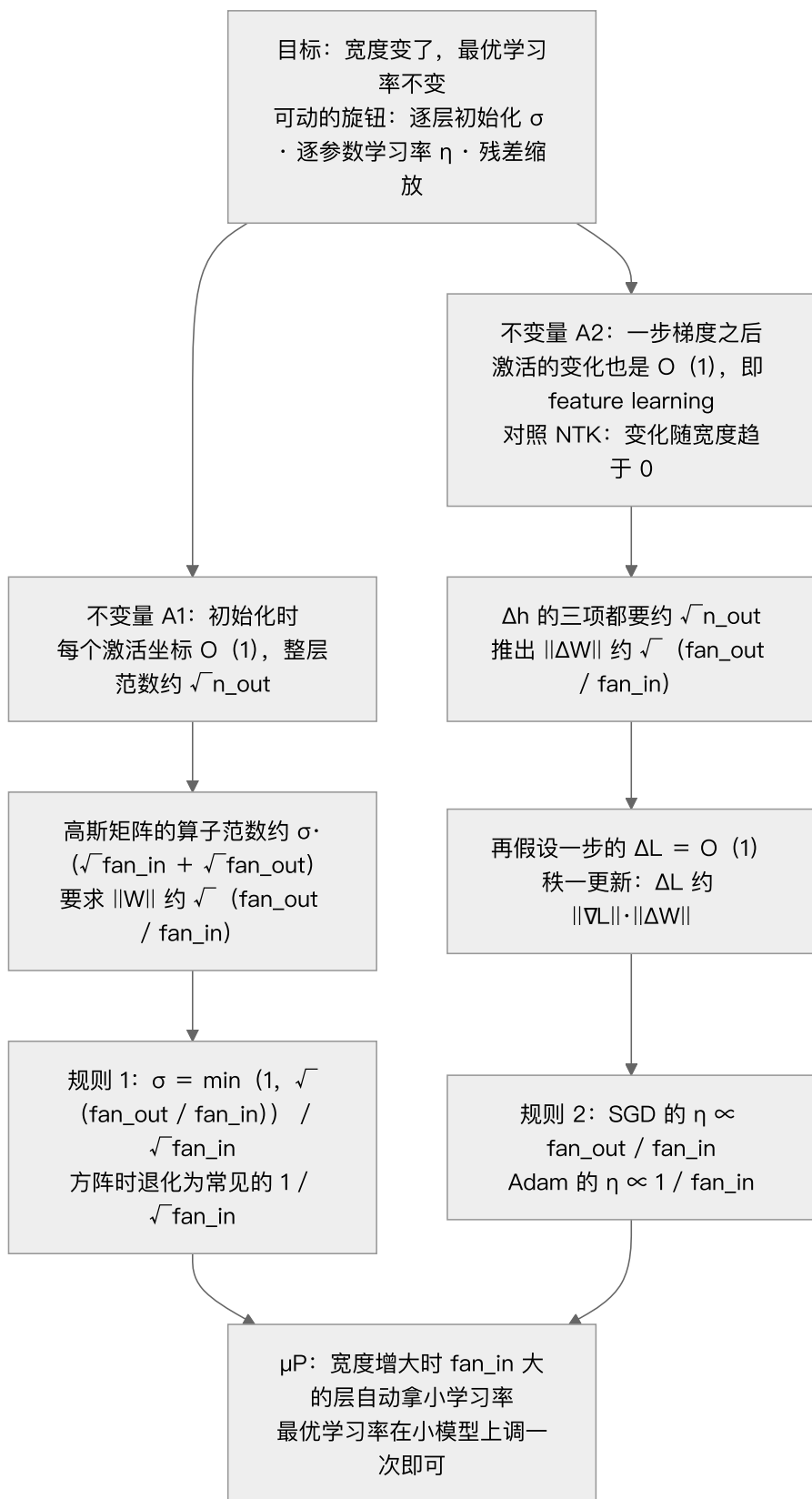


图 11-5 | μP 的推导链：两个不变量加一个“一步 loss 变化 $O(1)$ ”的假设，逼出逐层的 σ 和 η （自绘示意）· [▶ 看原幻灯片 1:00:51](#) · 出处：Yang, Simon & Bernstein, 2023

- **μP 是什么：**一个有点神秘的对象——论文和实现都很多，彼此对底层数学和实现细节并不一致，但共享一组核心想法，而且大体有效。讲者讲的是他自己对这套“纲领”的理解。
- **目标图：**模型变宽，最优学习率通常会移动；我们想让它不动。允许动的旋钮有三个：逐层初始化、逐参数学习率、有时还按模型大小缩放残差连接。
- **验证：**Cerebras（本是芯片公司，但有训练团队，负责人是 Hestness——第 9 讲 scaling law 那篇的作者）的 Cerebras-GPT 用 Chinchilla 配方训了 0.1B 到 13B，另训一组 μP 变体来调超参；μP 那组的 scaling law 拟合更稳，外推预测几乎正中，非 μP 那组的 loss 预测波动大得多。加上 MiniCPM 等，μP 在规模上得到过验证。

小注：Cerebras-GPT 一共 111M 到 13B 七档，μP 变体做到 2.7B。

- **读什么：**Greg Yang 开创了这条线（Tensor Programs 系列，讲者觉得难读）；Jeremy Bernstein 参与的一篇换了个框架的综述最易读（应为图 11-5 出处那篇“谱条件”）；还有物理学家和 Cerebras 的几篇。
- **两个不变量（只改宽度，取极限）：**
 - A1：初始化时喂随机数据，激活的大小不随宽度变——每个坐标 $O(1)$ ，整层范数约 $\sqrt{n_l}$ （ n_l 是该层的隐藏单元数）；随宽度爆炸或归零都说明参数化选错了。
 - A2：一步梯度之后，激活的变化也是 $O(1)$ ，这就是 feature learning——网络确实在学特征。对照 NTK（neural tangent kernel）那种极限，激活的变化随宽度消失，大网络不再学特征，那不是我们要的。
- **A1 的推导（深层线性网络，权重是方差 σ^2 的高斯）：**随机高斯矩阵的算子范数有标准的集中不等式；当 fan-out 不比 fan-in 大很多时，输出的范数近似等于输入范数乘算子范数（严格说是上界，高维下近似取等）。归纳：假设第 $l-1$ 层范数是 $\sqrt{n_{l-1}}$ ，选一个 σ_l 让 $\|W_l\|_{\text{op}} \approx \sqrt{(n_l / n_{l-1})}$ ，则第 l 层范数 $\approx \sqrt{n_l}$ ，归纳成立。 σ 从哪来？讲者承认是“从帽子里掏出来的 ansatz”，代回去能验证。

$$\|W_l\|_{\text{op}} \approx \sigma_l (\sqrt{n_{l-1}} + \sqrt{n_l}) \Rightarrow \sigma_l = \frac{\min(1, \sqrt{n_l/n_{l-1}})}{\sqrt{n_{l-1}}}, \quad \|h_l\| \approx \|W_l\|_{\text{op}} \|h_{l-1}\| \approx \sqrt{n_l}$$

W_l 是第 l 层的权重（ n_l 行、 n_{l-1} 列，即 fan-out $\times$ fan-in）， σ_l 是它每个元素的初始化标准差， h_l 是第 l 层的激活；这样选 σ_l ，算子范

数约为 $\sqrt{(n_l / n_{l-1})}$ ，激活范数逐层保持 $\sqrt{n_l}$ 。方阵时 $\sigma_l = 1 / \sqrt{n_{l-1}}$ ，就是标准参数化。

- **A2 的推导**（单样本、SGD、线性网络）：
 - 一步之后权重的变化 ΔW_l 是秩一的：loss 对该层输出的梯度与输入激活 h_{l-1} 的外积（前向是秩一乘法，反向就是秩一外积）。
 - 激活的变化：把 $h_l = W_l h_{l-1}$ 展开，得到三项——上一层激活变化传过来的直接项、权重变化引起的交叉项、两者同时变的项。要求三项都是 $\sqrt{n_l}$ 的量级（假设不相消）；第一项由归纳假设免费得到；另两项是 $\|\Delta W_l\|_{op} \cdot \sqrt{n_{l-1}}$ ，于是解出 $\|\Delta W_l\|_{op} \approx \sqrt{(n_l / n_{l-1})}$ 。
 - 最后一个、也是讲者最不放心的假设：一步之后 loss 的变化是 $O(1)$ ——模型在任何规模下都取得"差不多的可观进展"。Taylor 展开 $\Delta L \approx \langle \nabla L, \Delta W \rangle$ ，秩一时 Frobenius 内积化成两个算子范数之积，解出梯度的算子范数 $\approx \sqrt{(n_{l-1} / n_l)}$ ，再由 $\Delta W = -\eta \nabla L$ 得 $\eta_l = n_l / n_{l-1}$ 。Adam 的推导不同，得到 $1 / n_{l-1}$ 。

$$\Delta h_l = W_l \Delta h_{l-1} + \Delta W_l h_{l-1} + \Delta W_l \Delta h_{l-1}, \quad \text{每项} \sim \sqrt{n_l} \Rightarrow \|\Delta W_l\|_{op} \approx \sqrt{\frac{n_l}{n_{l-1}}}$$

Δh_l 是一步之后第 l 层激活的变化，三项分别来自上一层的变化、本层权重的变化、以及两者的交叉；让每一项都保持 $\sqrt{n_l}$ 的量级，就把本层允许的更新大小定了下来。

$$\Delta L \approx \langle \nabla_{W_l} L, \Delta W_l \rangle \approx \|\nabla_{W_l} L\|_{op} \|\Delta W_l\|_{op} = O(1) \Rightarrow \eta_l^{SGD} \propto \frac{n_l}{n_{l-1}}, \quad \eta_l^{Adam} \propto \frac{1}{n_{l-1}}$$

ΔL 是一步之后 loss 的变化， ∇L 是 loss 对本层权重的梯度；由 $\Delta L = O(1)$ 和上一式定下的 $\|\Delta W_l\|$ ，SGD 的学习率要按 fan-out / fan-in 缩放，Adam 则按 $1 / \text{fan-in}$ 缩放——fan-in 大的层学习率小。

- **结果一览**：初始化 $\sigma_l = \min(1, \sqrt{(n_l / n_{l-1})}) / \sqrt{n_{l-1}}$ ——比值为 1 时等于标准参数化；学习率：SGD 从"常数"改成 n_l / n_{l-1} ，Adam 改成 $1 / n_{l-1}$ ，于是 fan-in 大的层自动拿小学习率，得到逐层自适应的学习率。

```
def mup_layer(fan_in, fan_out, base_lr, optimizer="adam"):
    ratio = (fan_out / fan_in) ** 0.5
    sigma = min(1.0, ratio) / fan_in ** 0.5 # 规则 1: 方阵时就是 1/sqrt(fan_in)
    if optimizer == "sgd":
        lr = base_lr * fan_out / fan_in # 规则 2: SGD 按 fan_out/fan_in
```



```

else:
    lr = base_lr / fan_in                                # Adam 按
1/fan_in
    return torch.randn(fan_out, fan_in) * sigma, lr      # 每层各自的  $\sigma$ 
与学习率

```

这段说明 μP 落到代码上只是"每个线性层按自己的 $\text{fan-in} / \text{fan-out}$ 算一个 σ 和一个学习率", base_lr 在小模型上调一次即可。

- **高一层的意思**： μP 有趣不只在结果，还在推导方式——取网络的 scaling 极限，断言几个不变量，再加假设，反推超参必须满足的约束，约束本身就是超参的缩放规律。这是"物理学家的数学"（讲者说无意冒犯，这套东西本来就是物理学家发展的）：只追踪各项的数量级、假设不相消，而不是严格证明；它是一种和 CS / ML 里常见套路很不一样的算法设计原则。

9. μP 的压力测试与"到底有没有用"

- 一位独立研究者的论文（应为 Lingle, 2024）把 μP 当成一套调超参的程序做大规模压力测试：embedding、attention、MLP 输入输出、softmax 线性层，各自在 μP 和非 μP 下有不同的缩放规则。

小注：论文摘要：十几组消融最大到 1.2B 参数 / 33B token，一组大实验到 10B 参数 / 190B token；结论是多数设置下 μ -Transfer 给出近似最优的学习率。

- **复现了 MiniCPM 的头条结果**： μP 做对、只在控制下改宽度，最优学习率的不变性就出现；基础 μP 和一个给 attention 投影加 bias 的变体都迁移得很好。
- **理论不覆盖但实际没事的**：SwiGLU、各种初始化变体、RMSNorm——严格说都不在 μP 的理论里，但大多能一起用。
- **会破的三样**：
 - RMSNorm 里学习 gain 参数——破，不过很多情况下去掉 gain 也不掉点，所以不算大问题；
 - Lion 这类靠梯度符号的优化器——破（sign-gradient 在精神上和 Muon 同类，暗示还会有别的东西破）；
 - 大的 decoupled weight decay——最严重的失败，是唯一没过的压力测试。
- **结论**：想稳住学习率， μP 确实有用——不用 μP 时宽度增大最优学习率"可预测但幅度很大"地移动，用了就不动。它是控制超参漂移的工具

之一，不是定论； μP 式的初始化程序和"直接拟合 scaling law"都还有很大空间。

10. 收尾：scaling 是科学还是手艺

- 初见 scaling law 像科学：画直线、走流程，就能知道大规模会怎样。现实要脏得多、未知得多。
- 人们确实拿它选架构、选优化器、选超参——但这是门手艺：你不知道它会不会一直外推下去，只能做合理的事把成功概率拉高； μP 、搜索最优学习率，都是控制漂移的手段，没有银弹（"也许明年就有个模块说我们解决了，现在还没有"）。
- 把本讲的工具按"它管哪一种漂移"放在一起：

工具	管什么	代价或前提
WSD 学习率	让数据轴扫描不必重跑	每个预算多一次 decay；stable 段的 loss 不能直接比
最优超参的 scaling law（DeepSeek、StepFun）	学习率、batch 随算力或数据的漂移	网格要铺对位置；换数据常数会漂
μP	学习率随宽度的漂移	要改初始化和逐层学习率；大 weight decay、Lion、学 gain 会破
算力轴 + Chinchilla 比轴	判断优化器 / 架构的小规模优势是否保得住	两个轴都要扫，算力开销大
一次真正的大跑	最终验证	没有消融就只知道"能用"，不知道"更好"

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
μP 下的学习率扫描	每条线一个模型大小，最低点都在 $1e-2$ 附近；最小模型略偏	7:43	Hu et al., 2024
最优 batch size 对 loss	每列点是一个固定 batch 的跑法；等 loss 线上拟二次曲线取最低点；log-log 下呈幂律	8:43	同上
WSD 梯形与对比曲线	橙线是调好的 cosine；WSD 在 stable 段落后、decay 时追回	12:48	同上
DeepSeek 的 $\eta \times B$ 热力图与拟合线	固定算力，最浅的格子打星；灰点是各规模最优，星是正式模型；batch 那条像直线，学习率那条散	16:54	Bi et al., 2024
DeepSeek 的外推验证	灰点是小模型拟合，两颗星是正式模型，预测接近	21:32	同上

图	看什么	跳转	出处
Kimi K2 稀疏度扫描	同一 FLOPs 下越稀疏 loss 越低；48 之后收益递减	24:04	Kimi K2
Llama 3 两步图	左：算力 → 答案的 NLL；右：NLL → 准确率，紫点上的 sigmoid 有系统偏离	25:35	Llama 3
各家超参公式汇总表	输入变量各不相同：终点 loss、算力、数据量	31:42	Li et al., 2025
StepFun 的等高线与趋势图	1B / 100B 切片是碗形；右图各颜色的模型落在同一条 batch 对 D 的线上	34:16	同上
优化器相对加速对算力	Adam 为 1.0；Muon 在小规模领先、随规模递减；另一组按 Chinchilla 比分面	45:02	Wen et al., 2025
Marin 的失败 scaling 图	竖虚线（1e20 量级）之前直线，之后逐步偏离直到崩掉	48:04	Marin
Muon 伪代码	第 5 行的 Newton—Schulz 把动量矩阵正交化，第 6 行才更新	50:06	Muon
μP 规则汇总表	蓝框： σ 多了 $\min(1, \sqrt{(\text{fan_out} / \text{fan_in})})$ ；SGD 学习率 $\text{fan_out} / \text{fan_in}$ ，Adam $1 / \text{fan_in}$	1:10:39	Yang, Simon & Bernstein, 2023
μP 压力测试表	哪些变体仍迁移（SwiGLU、RMSNorm）、哪些破（学 gain、Lion、大 weight decay）	1:13:41	Lingle, 2024

提到的工作

名称	在本讲里的作用
Kaplan et al., 2020 · Hestness et al., 2017 · Chinchilla (Hoffmann et al., 2022)	第 9 讲的"正典"；Kaplan 的 critical batch size 是 StepFun 汇总表的第一行
MiniCPM (Hu et al., 2024)	μP 稳住学习率、batch size 幂律、WSD、Chinchilla 方法 1 / 3
DeepSeek LLM (Bi et al., 2024)	最优学习率 / batch 对算力的 scaling law；isoFLOP 复现 Chinchilla；两段 decay；外推验证
Hägele et al., 2024 (EPFL)	WSD 与 cosine 的系统比较（讲者记成了 ETH）
Qwen 2.5 · Qwen 3	超参 scaling law 成为标准配方，含 MoE
Kimi K2 (Moonshot, 2025)	稀疏度 scaling law（取 48）；全程用 Muon 的大模型
Hunyuan (腾讯 MoE, 2024)	MoE scaling law: 96 token / 激活参数
Llama 3 (2024)	isoFLOP 配比；loss → 准确率的 sigmoid 两步预测

名称	在本讲里的作用
MiniMax-01 (2025)	lightning / softmax / hybrid 架构的 scaling 对比
Step Law (Li et al., 2025, StepFun)	超参 scaling law 的最大规模研究；batch 只看 D
Gemma	如今 2-3B 档更强的对照
nanoGPT speedrun (Keller Jordan)	作业 1 的灵感；Muon 的出生地
Muon	矩阵参数的正交化更新；Newton-Schulz 5 步
Fantastic Pretraining Optimizers (Wen et al., 2025)	调好基线；算力轴与 Chinchilla 比轴
Marin (Will Held 的实验)	失败的 scaling 曲线，以及 μP 式参数化的救场
Cautious AdamC	Marin 那组实验试的 Adam 变体
Cerebras-GPT (Dey et al., 2023)	μP 让 scaling law 外推更稳
Tensor Programs (Greg Yang) · $\mu Transfer$	μP 的源头；讲者觉得难读
A Spectral Condition for Feature Learning (Yang, Simon & Bernstein, 2023)	推荐读物；本讲推导的框架（应为这篇）
NTK (Jacot et al., 2018)	A2 的反面：变化随宽度消失、不学特征
Lingle, 2024	μP 的大规模压力测试
Lion (Chen et al., 2023)	符号梯度优化器；会破 μP
Adam / AdamW / Adagrad	对照与向量参数的默认优化器

术语对照

English	中文
scaling ladder	规模阶梯：一串由小到大的模型，用来外推到目标规模
hyperparameter drift	超参漂移：最优学习率、batch size 随规模移动
μP (maximal update parametrization)	最大更新参数化：改初始化和逐层学习率，让最优学习率不随宽度变
$\mu Transfer$	用 μP 在小模型上调好超参再迁到大模型
standard parametrization (SP)	标准参数化：初始化方差 $1 / fan_in$ ，学习率各层相同
$fan-in$ / $fan-out$	一层权重的输入维 / 输出维
per-parameter (per-tensor) learning rate	逐参数（逐张量）学习率

English	中文
critical batch size	临界 batch size：再增大就开始浪费样本
optimal batch size / learning rate	最优 batch size / 学习率：网格里 loss 最低的那个
isoFLOP	等算力曲线：固定 FLOPs 换模型大小与 token 数
Chinchilla ratio	Chinchilla 比：训练 token 数除以参数量
non-embedding FLOPs / parameters	不含 embedding 的算力 / 参数量
cosine schedule	余弦学习率调度：要事先知道总步数
WSD (warmup—stable—decay)	预热—恒定—衰减：梯形学习率
rewind and re-decay	回滚再衰减：从 stable 段的 checkpoint 重新衰减
annealing	退火：末尾降学习率把模型收敛到位
terminal loss	终点 loss：训练结束时的 loss
contour plot	等高线图
quantization error (网格)	网格太粗造成的最优点偏差
grid search / univariate sweep	网格搜索 / 单变量扫描
decoupled weight decay	解耦的权重衰减 (AdamW 那种)
sparsity (MoE)	稀疏度：总 expert 数除以每 token 激活数
active parameters	激活参数：每个 token 实际用到的参数
lightning attention	MiniMax 的线性注意力变体
sigmoid fit	用 S 形曲线拟 loss 到准确率的映射
speedrun	速通榜：固定目标 loss 比谁最快
Muon	矩阵参数的正交化动量优化器
Newton—Schulz iteration	只用矩阵乘法的正交化近似迭代
orthogonalize	正交化：把奇异值全变成 1
spectral norm / operator norm	谱范数 / 算子范数：矩阵最大的奇异值
Frobenius norm	矩阵各元素平方和开根
rank-one update	秩一更新：两个向量的外积
feature learning	特征学习：一步之后激活变化 $O(1)$
NTK (neural tangent kernel)	神经正切核：无限宽极限下不学特征的那种行为
concentration (matrix)	随机矩阵的集中不等式
ansatz	先猜一个形式再验证的假设
implicit regularization	隐式正则化：优化器本身带来的偏好
ablation	消融实验
vibes	讲者的说法：凭感觉的判断

字幕勘误

"Hessness" → Hestness; "Chinchilla 2" (23:33 处, 会误导) → Kimi K2; "Kimmy K2 / Kimmy" → Kimi K2; "Gwen 2.5 / Gwen 3 / Quen" → Qwen; "MOE / MOEs" → MoE; "deep sea recipe / deep seek" → DeepSeek; "many CPM / mini CPM" → MiniCPM; "Moraine" → Marin; "Kaya / Kai Yu" → Kaiyue (Wen); "μon / MuON / mu on" → Muon; "new P / MUP / mu P" → μP ; "Step One / step on" → StepFun; "Newton-Schultz" → Newton-Schulz; "scaling loss" → scaling laws; "WS learning rate" → WSD; "tech pieces of text" → tokens of text; "Swiglu" → SwiGLU; "nano GPT" → nanoGPT; "2 e 18" (Kaplan 的 batch size 常数) → $2e8$; "soft max" → softmax; "iso-flop / isoflops" → isoFLOP; "equal loss contour" 正确; "lightning attention" 正确。

带走的问题

1. MiniCPM 和 DeepSeek 对"最优学习率随规模漂移"给出的两种解法, 各自的前提是什么? 如果你的架构里有 SwiGLU、RMSNorm 学 gain 和大 weight decay (第 9 节), 哪条路更稳? 把两条路合起来——先 μP 、再对剩余的漂移拟 scaling law——有什么好处和风险?
2. 用 WSD 做一次 isoFLOP: stable 主线跑 D 个 token、要 5 个数据预算点、decay 占 10%, 总算力是纯 cosine 方案 (5 次从零重跑) 的几分之几? WSD 在 stable 阶段的 loss 比 cosine 差, 那 stable 段中途的 checkpoint 能不能直接拿来比较模型?
3. StepFun 说最优 batch size 只随 D 变、与 N 无关, Kaplan 说 critical batch size 随目标 loss 变——在 Chinchilla 配比下这两种说法能否互相换算? 换了训练数据之后哪一个更可能失效?
4. 图 11-4 的两个轴里, 为什么 Chinchilla 比是"最常被忽略的混杂变量"? 对一个据说自带隐式正则化的优化器, 你会怎么设计实验让它在 $1\times$ 、 $4\times$ 、 $8\times$ 的比值上各自暴露? Kimi K2 没有消融, "Muon 在 1T 规模能训"和"Muon 比 AdamW 好"之间还差什么证据?
5. μP 推导里最不可靠的假设是"一步的 $\Delta L = O(1)$ "; 如果它不成立 (比如大模型每步进展更小), 推出的 $\eta \propto n_{\lfloor l \rfloor} / n_{\lfloor l-1 \rfloor}$ 会往哪个方向错? Adam 为什么变成 $1 / n_{\lfloor l-1 \rfloor}$ 而不是 $n_{\lfloor l \rfloor} / n_{\lfloor l-1 \rfloor}$ ——从"Adam 的更新每个坐标都是 $O(1)$ "出发自己推一遍。

CS336 第 12 讲 | 评测 (Evaluation)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 12 讲
(课程日程表标注 2026 年 5 月 6 日; 视频里没有出现日期) 视频:
<https://www.youtube.com/watch?v=JpAxdTWQJxM> (1:18:34; 英文字幕为自动生成, benchmark 名、模型名错得不少, 见文末勘误) 讲者: Percy Liang (全程; 讲污染检测时以第三人称提到"Tatsu 组的一个想法", 讲私有评测时说自己读博时有一堆没上网的被拒论文可以拿来评困惑度) 课程主页:
<https://stanford-cs336.github.io/> · 本讲不围绕某一篇文章, 而是串起一批 benchmark: MMLU · GPQA · Humanity's Last Exam · Chatbot Arena · AlpacaEval · SWE-bench · ARC-AGI; 讲者自己组的工作有 Cybench、AIR-Bench 和报告 train-test overlap 的立场文

一句话: 评测回答的是"训出来的模型好不好", 可"好"没有唯一定义——榜单分数、分数对成本、人的偏好、真金白银的用量各是一种答案——所以这一讲给的是一张地图: 困惑度是造模型的人自己的仪表盘 (随规模平滑变化、能拟 scaling law、有私有语料就能评, 但要看到分布, 榜单上验不了真); 考试类靠越出越难续命 (MMLU 57 科已到 90 多分, MMLU-Pro 改 10 选项后从 33 回到近 90, GPQA 从 GPT-4 的 39% 到 94, HLE 还只有 64.7); 对话类没有标准答案, 只能靠成对比较加 Elo 或 LLM 裁判 (AlpacaEval 与 Arena 相关 0.98, 但裁判偏爱长回答); 智能体类评的是模型加 scaffold (SWE-bench Verified 从 16% 到 93); ARC-AGI 想把推理从知识里剥出来, 推理模型一出就从 0 冲到解决; 安全类连"安全"的定义都还在争——而让这些数字可信的, 是四件功课: 想清楚评测的目的、防训练—测试重叠 (顺序检验、报告重叠、新鲜题、私有题)、审计数据集 (空回复也能拿 38%)、亲自看输出。

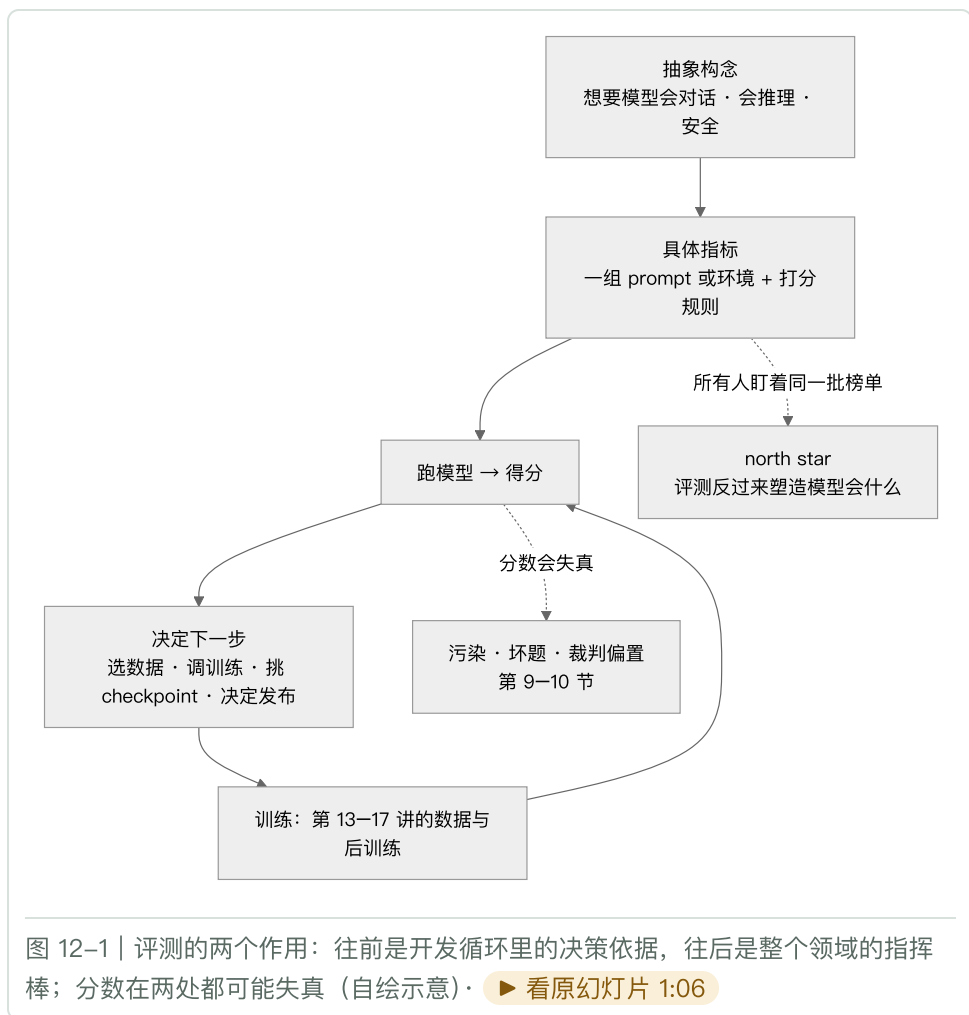
时间轴

时间	内容
0:06	开场: 训练一个 LM 的拼图只差数据; 但先得定义"要什么行为"
1:06	评测是什么: 抽象构念 → 具体指标; 评测设定 AI 发展的 north star
2:40	"好模型"的四种看法: Artificial Analysis 智能榜、智能对推理成本、Arena 偏好、OpenRouter 用量
5:16	困惑度: 定义; 2010 年代的 in-distribution 范式 (PTB、WikiText-103、1B Word); 2016 年纯神经的决定性结果

时间	内容
7:50	GPT-2 的 zero-shot 评测：WebText 40 GB、1.5B 参数、PTB 35 对 SOTA 46
9:51	"perplexity is all you need"的论证；反例"Stanford 1885"；条件困惑度
12:53	伪装的困惑度：LAMBADA、HellaSwag；困惑度榜单的信任问题；困惑度小结
18:11	考试类：MMLU 57 科与 few-shot；MMLU-Pro 4 → 10 选项；GPQA 的 61 位 PhD 与 diamond 子集
25:53	问答：题在不在训练集里？污染是间接的；Humanity's Last Exam 与私有保留集；多选题的价值与局限；答案怎么比对
31:30	对话类：Chatbot Arena 的成对比较与 Elo；谁在投票、风格与正确性混淆、谄媚
38:09	AlpacaEval 的 LLM 裁判与长度偏置；怎么评一个指标（相关 0.98）；WildBench 的 checklist；开放式评测小结
44:52	智能体类：SWE-bench 16 → 93、Terminal-Bench 89 题、Cybench 40 题、MLE-bench；scaffold 的四件事
53:37	纯推理：ARC-AGI 1 / 2 / 3 的轨迹；o1 / o3 的拐点；问答：64x64 网格怎么输入
1:00:18	安全类：HarmBench、AIR-Bench、GCG 越狱；安全的语境性、各类风险、双重用途
1:05:24	生态效度：GDPval、临床任务集、用 LLM 分析真实用量
1:08:56	训练—测试重叠：四条路——顺序检验、报告重叠、新鲜评测、私有评测
1:13:04	数据集质量：SWE-bench Verified 的由来、MMLU 坏题、空回复拿 38%、Docent
1:15:37	评测为了什么：四种目的；评方法还是评模型（nanoGPT speedrun）；总结与取舍

核心内容

1. 评测在造模型的循环里是什么位置

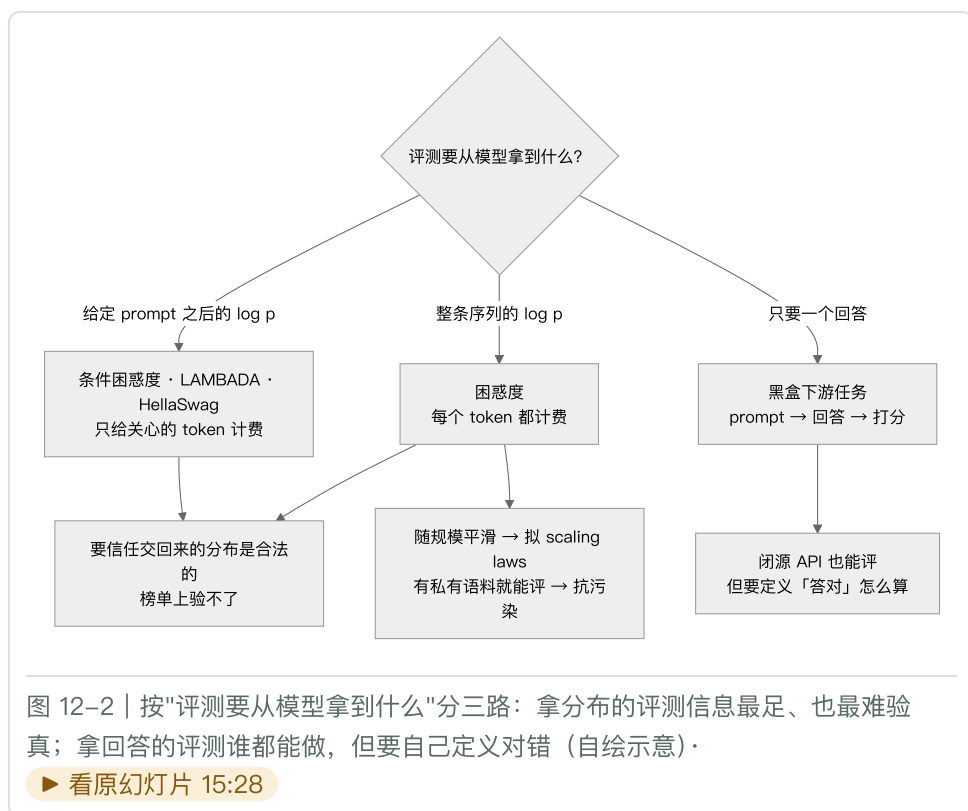


- **这一讲的位置**：到上一讲为止，训练一个语言模型需要的零件都讲过了——架构（第 3-4 讲）、优化器与训练循环（第 2 讲）、kernel 与并行（第 6-8 讲）、scaling laws（第 9、11 讲）、推理（第 10 讲）。还缺的一块是数据（第 13-14 讲）：数据决定模型的行为，喂代码就会写代码，只喂 DNA 序列就不会说英语。但谈数据之前，得先说清“想要模型有什么行为”——这就是评测。
- **评测看着机械，其实很深**：定义一组 prompt，发给模型，收回答，算准确率，似乎没什么好讲。但评测塑造了 AI 的发展方向：它设定 north star，开源闭源的每家开发者都把它当进度表。所以认真设计评测，就等于在暗中决定模型将来会什么、不会什么。
- **核心难题**：起点永远是一个抽象构念（construct）——“会对话”“会推理”；评测是把它落成一个具体指标，背后是一组具体的 prompt 或环境。整讲反复出现的都是这一步转换及其失真。
- **"好"的四种看法（2:40）**
 - 榜单分数：artificialanalysis.ai 的 intelligence index 几乎成了“智能”的默认口径，榜从 GPT-5.5 往下排，分数是若干数据集的合

成；

- 分数对成本：同一个网站把 intelligence index 对推理花费画成散点——两者相关，但并不严格对齐，花钱多的一般更聪明，却不是一条直线；
 - 人的偏好：Arena AI（原 Chatbot Arena）的做法，用户成对投票排出名次（第 4 节）；
 - 真金白银的用量：OpenRouter 是多模型的统一入口，公布各模型的调用量——经济学视角，"有人付费就是好"，但只代表 OpenRouter 上的那群用户。
 - 讲者的态度：没有哪一个是正确答案，也不确定存在正确答案；列出来是为了让人意识到"好"可以有多种定义。
- **和前面课程的关系**：CME295 第 8 讲和 CS329A 第 8 讲从用模型的人的角度讲过 benchmark 分类、LLM-as-judge 和 Arena 的毛病；这一讲是造模型的人的视角——开发时用什么信号（第 2 节）、比较时信什么（第 3—8 节）、数字怎么被污染、怎么查（第 9—10 节）。

2. 困惑度：造模型的人自己的仪表盘



$$\text{PPL}(D) = \exp \left(- \frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i | x_{<i}) \right)$$

D 是测试语料，共 N 个 token， p_θ 是模型；指数里是每个 token 的平均负对数似然（也就是训练用的 loss），取指数后得到困惑度。一句话：模型给这份语料分了多少概率质量，按 token 归一化让数字可比。

- **为什么从困惑度讲起**：语言模型的本身就是 token 序列上的分布 $p(x)$ ，评一个分布最自然的量是它给测试集分了多少概率——困惑度、似然、log loss 说的是一回事。训练时最小化的是训练集困惑度，评测时自然就量测试集困惑度。
- **2010 年代的范式**：当年任何一篇语言模型论文都是这一套。标准数据集是 Penn Treebank、WikiText-103、1 Billion Word Benchmark（取自机器翻译语料）；训练用 train split，评测用 test split，这是 **in-distribution** 评测，进展用困惑度下降来衡量。讲者点了一篇 2016 年的论文（应为 Jozefowicz et al. 的 *Exploring the Limits of Language Modeling*）：CNN 加 LSTM 在 1B Word 上把困惑度压下一大截——那时大家还在讨论 n-gram 和混合模型，这是第一个证明神经网络是正道的决定性结果。
- **GPT-2 换了玩法**（2019）：在 WebText（Reddit 外链的网页，40 GB 文本）上训练，然后**零样本**去评别人的标准数据集——这是 **out-of-distribution** 评测。最大的模型 1.5B 参数；在小数据集上提升尤其大，PTB 上做到 35，而当时的 SOTA 是 46；在 1B Word 这种自带大量同分布训练数据的集上没能超过 SOTA，但考虑到模型根本没见过那份训练集，已经很惊人。讲者顺带提醒：当年不一定做了仔细的去污染，可能有重叠。这开启了“在大语料上训、在标准 benchmark 上评”的范式，今天习以为常，当时是新鲜事。
- **"perplexity is all you need" (9:51)**：讲者说别太当真，但它是驱动很多人做 scaling 的心法。论证是：真实分布记作 T，模型记作 P；能达到的最好困惑度是 T 的熵，只在 P 等于 T 时取到；所以一路压困惑度，唯一的极小点就是真实分布——有了它，给问题就能生成解，给提问就能生成回答，往下压就通向 AGI。GPT-3 之前收益还不明显的年代，正是这个信念撑着大家继续放大模型。

$$\mathbb{E}_{x \sim T}[-\log P(x)] = H(T) + \text{KL}(T \parallel P) \geq H(T)$$

模型 P 在真实分布 T 下的交叉熵等于 T 自身的熵加上 T 到 P 的 KL 散度；KL 项非负，只在 P 等于 T 时为零——这就是讲者口头说的“最好的困惑度是熵、唯一极小点是真实分布”，写成式子的样子（讲者没有写这个式子，只说了结论）。

- **困惑度也许"多于所需"**：一句"Stanford was founded in 1885"，困惑度对每个 token 都收费。预测 1885 那个 token 很有价值——它等于一道问答题，测的是世界知识；可句首的词、founded 这样的词并不有趣，困惑度照样按每一分偏差收费。补救是**条件困惑度**：在 prompt 上条件化，只量后面回答部分的困惑度，把注意力收到你在意的 token 上。

$$\text{PPL}(y \mid x) = \exp \left(-\frac{1}{|y|} \sum_{t=1}^{|y|} \log p_{\theta}(y_t \mid x, y_{<t}) \right)$$

x 是 prompt（不计费），y 是回答，|y| 是回答的 token 数；只对回答部分的 token 求平均负对数似然。

- **伪装成准确率的困惑度**：有些 benchmark 名义上算准确率，骨子里是下一 token 预测。
 - LAMBADA (2016)：给一段上下文，填最后一个词。选词经过精心挑选——必须读完长上下文才能填对。早期 GPT 系列论文很看重它，因为当时的信念是长上下文建模是通向推理的钥匙；把困惑度"磨尖"到某类现象上，就是这么做的。
 - HellaSwag：多选题形式的句子续写（一位女士带着狗在外面，接下来她……），四个候选选最合理的一个。本质上还是在比哪个续写的概率高。
- **想办困惑度榜单的人请注意 (15:28)**：参赛者交回来的是 log prob，你得信它是合法分布、加起来等于 1。否则我写一个"永远返回 log prob 为 0"的模型，困惑度完美，但那根本不是分布。合同层面验不了，只能看代码。下游任务就简单得多：黑盒模型，给 prompt 收回答，算准确率。困惑度处理的是分布，天然要多一份小心；对 VAE 这类只能算一个界的模型更麻烦——还得信那个界的数学是对的。
- **小结，以及造模型的人怎么用它**：困惑度在模型开发中仍然用得很重——它随规模平滑变化，第 9、11 讲的 scaling law 拟的就是它；训练时盯的 loss 曲线是留出集困惑度；防污染时用私有语料评它（第 9 节）。这三件事用户看不到、开发者天天在做，第 1 讲叫它内部评测。但对不信"困惑度通向一切"的人，你得拿出反映真实场景的 benchmark；对信的人，一条漂亮的困惑度曲线就够了。

```
def perplexity(model, docs, ctx_len):  
    nll, n = 0.0, 0  
    for doc in docs:  
        # 私有语料也行：只要是文本就能  
        评
```

```

ids = tokenize(doc)
for s in range(0, len(ids) - 1, ctx_len):
    x = ids[s : s + ctx_len + 1]
    logp = model.log_probs(x[:-1]) # 每个位置对下一个 token 的
log p
    nll -= logp.gather(x[1:]).sum()
    n += len(x) - 1
return math.exp(nll / n) # 按 token 归一化再取指数

```

这段伪代码是困惑度评测的全部：切窗、取 log prob、累加、归一化——它不需要标注、不需要标准答案，这正是私有困惑度评测好做的原因。

3. 六个家族一览，从考试类开始

困惑度 · 评分布
打分: log prob 按 token 归一化

只给关心的 token 计费

考试 · 评知识与推理
打分: 字母比对 · 答案抽取

没有标准答案了

对话 · 评模型说了什么
打分: 成对偏好 · Elo · LLM 裁判

从说到做

智能体 · 评模型做了什么
打分: 单元测试 · flag · 竞赛成绩

把知识剥掉

纯推理 · ARC-AGI
打分: 网格题 · 交互游戏

能力之外还要无害

安全 · 拒答 · 越狱 · 风险分类
打分: 拒答率 · 攻击成功率

图 12-3 | 本讲的六个评测家族按讲授顺序排成一条线，每个框里写评什么、怎么打分，边上写这一步换掉了什么；讲者的总结是它们在难度、真实性、效度上各有取舍（自绘示意）· [▶ 看原幻灯片 1:17:40](#)

- **为什么是考试：**人类就是靠考试互相测的。考试的好处是对科目、难度有精细控制，可以设计成答案唯一，于是好判分。LM 的 benchmark 文化很大一部分就建在这个想法上。

Benchmark	设计	课上给的数字
MMLU (Hendrycks et al., 2020)	57 个学科，一队学生从网上搜罗题目；名字里有"语言理解"，实际考知识和推理；用 few-shot prompt 评 GPT-3	小模型勉强高于随机，GPT-3 明显高于随机；从 GPT-3.5 turbo（2023 年初）到 GPT-4，如今 90 多分
MMLU-Pro (2024)	去掉噪声题和琐碎题；4 选项扩到 10 选项；默认配 chain of thought	刚出时准确率回落到 33，现在约 88，快 90
GPQA (Google-proof QA)	61 位 Upwork 上的 PhD 出题；写题 → 专家验证 → 改题 → 第二位专家 → 非专家带着 Google 做；diamond 子集 = 两位专家都答对且至多一位非专家答对	PhD 专家 65%（基本是 4 选 1）；非专家 30 分钟加 Google 只略高于随机；GPT-4 39% → 现在 94
Humanity's Last Exam (2025)	多模态、多学科、多选加简答；众包出题，用钱和署名激励；多轮审核，先用前沿模型过滤掉太容易的；留一份不公 开的私有集	刚出时各模型个位数；2026 年 Mythos 也只有 64.7

小注：GPQA 论文摘要的原始数字——专家 65%（剔除明显失误后 74%），带 Google 的非专家 34%，GPT-4 39%。"Mythos"是讲者念的模型名，应为 Anthropic 2026 年的 Claude Mythos（推断）。

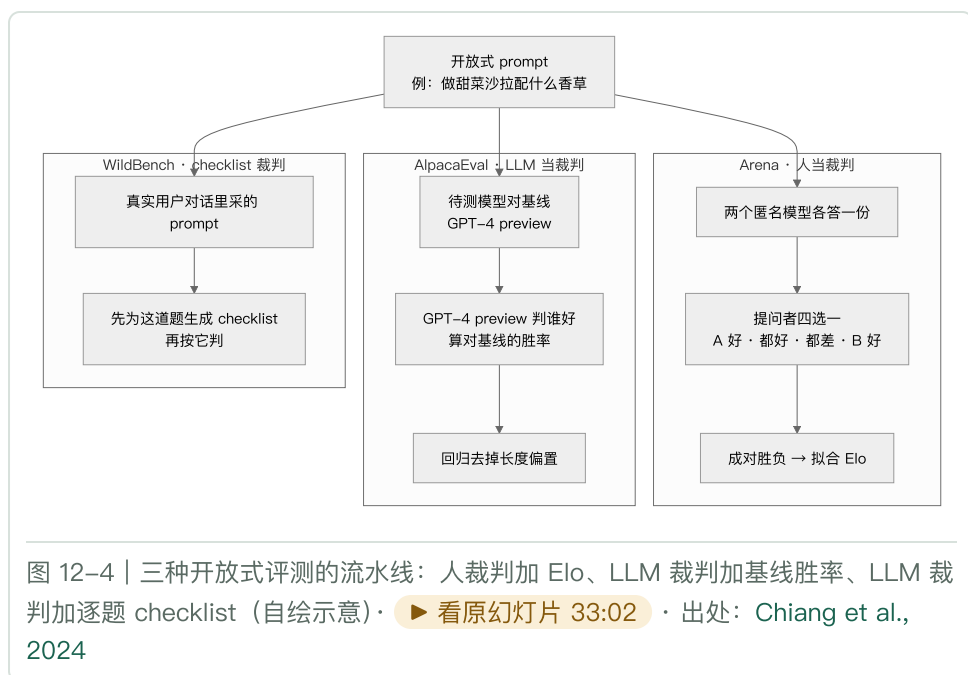
- **MMLU 当年有多激进：**GPT-3 刚出来的时候，没人确定语言模型是通用任务求解器，多数人还把它当"能生成流畅英文的东西"。MMLU 的团队赌的是"走在曲线前面"，并且用 few-shot prompt 来评——先写一句"下面是高中数学的题目"，再放几组题和答案，最后放待答的题，让模型接着预测。今天看平淡无奇，当时让模型处理这么复杂的 prompt 是相当大胆的。
- **续命的套路：**benchmark 饱和了就出更难的——MMLU 太容易就有 MMLU-Pro；能靠 Google 搜到答案就太容易，于是有 GPQA；模型什么都会了就众包一份"人类最后的考试"。每篇数据集论文的图都长一个样：旧 benchmark 上各模型都很高，我这个上各模型都很低。讲者的评语是保质期不长——但饱和的 benchmark 对造模型的人仍有用：给小模型选型、拟 scaling law 时，它们还是好信号。

- **问答：题目在不在训练集里？**（25:53）讲者的诚实回答是不知道，因为不知道训练集里有什么，榜单数字都得打折扣，全凭信任。而且污染很微妙：多半不是直接拿测试集训练，而是题目改编自的来源被训练进去了。HLE 的私有保留集是一种应对，但评测时仍要把题发给 API，只能希望那些 prompt 不被留下来训练。
- **多选题的价值与局限**（29:29）：多选题活到现在，是因为难度可以随便加——说多选题太简单是误解，它限制的是能问的问题类型，不是难度。真正的问题是考试和真实使用相距太远：除了评 HLE，没人会拿 HLE 的题去问模型；真实的问题是开放式的，可能没有正确答案，甚至不成句。
- **问答：答案怎么比对？**（30:30）多选题就是模型生成一个字母，和标准答案比是否相等。至于怎么生成：可以直接采样一个字母，现在更常见的是先生成一段 chain of thought 再给答案，这就需要从文本里**抽取答案**——抽取规则会影响分数，LM 评测对这一步可能非常敏感，讲者说这一讲不展开。

```
def score_mc(model, q, choices, gold):  
    # 方式 A: 黑盒——让模型生成，再从文本里抽出字母  
    text = model.generate(fewshot_prefix + q + "\nAnswer:")  
    pred_a = extract_letter(text)          # 抽取规则本身会影响分数  
    # 方式 B: 白盒——比较每个选项的 log p, 这就是「伪装的困惑度」  
    scores = [model.log_prob(q + c) for c in choices]  
    pred_b = "ABCD"[argmax(scores)]  
    return pred_a == gold, pred_b == gold    # 两种口径未必一致
```

同一道多选题有两种打分口径：黑盒生成加抽取，或者白盒比 log prob；讲者提醒抽取那一步会左右分数，而白盒口径又回到了第 2 节的困惑度。

4. 对话类：没有标准答案时，靠成对比较和裁判



- **问题长什么样**：没人拿多选题问自己的 AI 助手（除非想让它替自己考试）。真实的问题像“做甜菜沙拉，哪些香草搭、哪些不搭”——问题开放，回答也开放，没法用相等来判，甚至没有 ground truth。
- **Chatbot Arena 的巧思 (33:02)**：让人来判，但判法设计得聪明。网站对所有人开放，随便谁都能来聊；不同于普通助手，一条 prompt 会拿到两个匿名模型的回答，用户四选一：A 好、都好、都差、B 好。攒下大量成对比较，再拟合 Elo 评分。榜上讲者点了一句 Claude Opus 排得不错。

$$P(A \succ B) = \sigma(r_A - r_B) = \frac{1}{1 + e^{-(r_A - r_B)}}, \quad \hat{r} = \arg \max_r \sum_{(i \succ j) \in \mathcal{D}} \log \sigma(r_i - r_j)$$

r_A 、 r_B 是两个模型的评分，A 赢 B 的概率是评分之差的一个平滑递增函数；参数就是各模型的评分，拟合目标是让观测到的所有成对胜负的概率最大。讲者只说“某个函数”，这里写成 logistic 形式。

小注：logistic 形式就是 Bradley–Terry 模型，Chatbot Arena 论文实际用的是它；国际象棋的 Elo 用的是同一族函数，只是底数取 10、尺度取 400。

- **好在哪**：prompt 是真的。人来这个网站的动机是免费用模型，所以默认他们真想办点事，得到的就是真实世界的 prompt。Elo 的美妙之处在于不要求所有模型答同一批 prompt——和象棋一样，不必人人对战，比较图连通就能排名；这一点很重要，因为人不可能给所有模型都打分。而且新 prompt、新模型随时进来，天然可以持续更新。
- **担心在哪 (35:35)**

- 投票的是谁？"互联网上随机来的人"是什么分布，谁也不知道；论文里有人口统计，但那说明不了全部。可能有垃圾投票，可能有人为自己提交的模型刷票，有点像蛮荒西部。
- 二元偏好把**风格**和**正确性**揉在一起。象棋的 Elo 干净，因为只有输赢；"哪个回答更好"远没那么清楚。
- 提问者当裁判有好处——他有意图，能判断意图有没有被满足；但他提问往往正因为不知道答案，那两个回答哪个对，他怎么判？
- **谄媚** (sycophancy)：讨喜的回答会被抬高，正确但不中听的会吃亏。
- **AlpacaEval** (2023, 38:09)：指标是对基线模型的**胜率**——基线是 GPT-4 preview，裁判也是 GPT-4 preview，偏置显而易见，可以靠多个裁判集成缓解。这是 LLM-as-a-judge 的典型实例。它的第一个大问题是裁判偏爱长回答：一批微调模型只因回答更长就冲上榜首，属于刷榜；后续论文用一个简单的回归把长度偏置扣掉。
- **怎么评一个指标**：有了指标能评模型，可新指标本身好不好，谁来评？没有答案。能做的是理智检查：看它和别的指标的相关性，前提是你确实想模仿那个指标。AlpacaEval 和 Chatbot Arena 的相关系数是 0.98，不想上 Arena 或不想等人投票的，可以拿它当替身；但相关性是对某一批模型算的，比 GPT-4 preview 更强的模型上未必成立。这个榜已一年多没维护。
- **WildBench**：prompt 来自真实的人机对话（它也开了免费服务收集提问），裁判是 LLM；创新点是为每道题**生成一份专属 checklist**——直接问 LLM "好不好"是个定义不清的任务，checklist 把任务圈定下来。它也报告了和 Arena 的相关性，讲者承认这有点循环：Arena 就是 ground truth 吗？至少大家都在同一条船上。
- **开放式评测小结** (43:21)
 - 成对比较优于绝对打分：两个回答很接近时，"这个略好"仍是有效的方向信号；"7 分还是 8 分"信号弱得多。
 - 人和 LLM 裁判各有各的偏置，都得留意；也许只能多找几个裁判，人和模型都说你更好，那大概是真的更好。
 - 开放式评测本质上定义不清，要靠 rubric 或 checklist 提高可靠性——对人类裁判同样适用，不给评分标准就让众包工人打分，结果多半是胡来。
- **和前面课程的关系**：CME295 第 4 讲列过 Arena 的五类问题，CS329A 第 8 讲讲过 LLM-as-judge；这一讲新增的是榜单背后的拟

合模型、长度偏置的修法，以及用相关性验证一个指标。

```
def fit_ratings(pairs, models, steps=1000, lr=0.01):  
    r = {m: 0.0 for m in models}          # 每个模型一个评分  
    for _ in range(steps):  
        for winner, loser in pairs:      # 一场匿名对战 = 一条成对比较  
            p = sigmoid(r[winner] - r[loser])  
            r[winner] += lr * (1 - p)    # 冷门取胜，评分动得多  
            r[loser] -= lr * (1 - p)  
    return r                             # 比较图连通即可，不必人人对战
```

按成对胜负做梯度上升拟合评分：每条比较只涉及两个模型，所以不要求所有模型答同一批题，新模型打几场就能进榜。

5. 智能体类：评的是模型加 scaffold

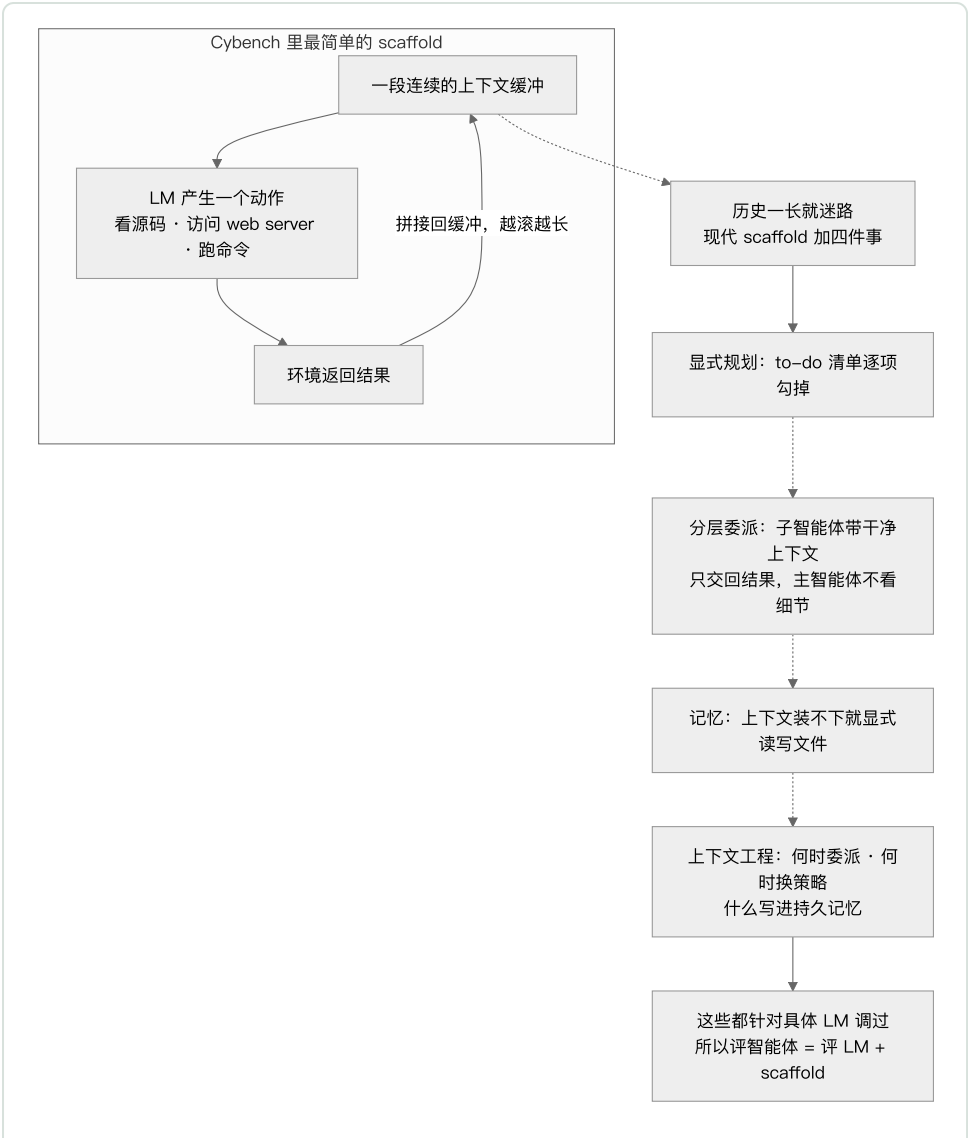


图 12-5 | 最简单的智能体循环，以及讲者列的现代 scaffold 四件套；结论是智能体分数里混着 scaffold 的功劳（自绘示意）· [▶ 看原幻灯片 49:58](#) · 出处: Zhang et al., 2024

- 从"说什么"到"做什么": 对话类评的是模型说了什么，智能体类评的是模型做了什么。智能体 = 语言模型 + scaffold，后者是决定何时调用模型、给它哪些工具的那套逻辑。讲者的判断是智能体会改变我们看待语言模型的方式，它扩展了模型的能力面。

Benchmark	任务与环境	怎么打分	课上给的数字
SWE-bench	给一个代码库和一条 GitHub issue，提交 PR	单元测试：原本失败的要通过，其余不能弄坏	SWE-bench Verified：2024 年约 16%，现在约 93 (Mythos)
Terminal-Bench 2.0	环境就是终端；任务靠敲命令完成，简单而通用	各任务自带检验	93 人众包，89 道任务；人做需 1 小时到一周以上，看专家还是新手

Benchmark	任务与环境	怎么打分	课上给的数字
Cybench	40 道 capture-the-flag: 看源码、访问服务器、拿到 flag	拿到 flag 这个唯一字符串即成功	刚出时最好的模型约 10%，现在完全解决
MLE-bench	Kaggle 竞赛：看数据、读说明、写代码、训模型、提交	按 Kaggle 成绩评级	榜首仍是那几个前沿模型，但 scaffold 之间差异很大

- **四个榜的看点：**SWE-bench 开了在真实代码库里评智能体写代码的先河，打分极其直接——PR 能不能让测试通过（榜上看的是 Verified 版，缘由见第 10 节）；Terminal-Bench 的环境最简单、最普适，榜上能看到同一个模型配不同 agent 准确率不同；Cybench（讲者组的工作）把人类的 CTF 竞赛搬给智能体，课上展示的 scaffold 极简（图 12-5 上半），历史越滚越长，所以后面必须有更好的上下文管理；MLE-bench 榜上 LM 一侧还是那几位常客，差异主要来自 scaffold。
- **scaffold 很重要，所以这已经不只是 LM 评测 (52:02)。**要解决真正复杂的任务要靠四件事：显式规划（一路 chain of thought 意识流下去，上下文一堆积就迷失，要维护 to-do 清单逐项勾掉）；分层委派（子智能体带干净的上下文去干活，只交回结果，主智能体不看细节——一种封装）；记忆（上下文装不下就显式读写文件）；上下文工程（何时委派、何时换策略、什么写进持久记忆）。这些都针对具体模型调过，所以评智能体等于同时评模型和 scaffold。
- **给做智能体 benchmark 的人：**这一节和第 10 节合起来是三条提醒——分数里 scaffold 和模型不可分，报告时两者都得写清；环境是质量问题的藏身处，测试用例可能不完整，通过了测试不等于方案能用；一定要看轨迹 (trace)，不能只看通过率。CS329A 第 8 讲讲过智能体评测的框架，这里补的是各榜单的数字和"scaffold 这一列"。

6. 纯推理：ARC-AGI 想把推理从知识里剥出来

- **动机：**前面所有任务都要语言和世界知识。能不能把推理、或者说流体智力 (fluid intelligence)，从"知道多少事实"里剥离出来单独测？有人认为这才是更纯的智能。
- **ARC-AGI (2019 年起)：**目标就是这个——每道题都要 100% 可被人解出、但对 AI 很难；每道题都是独一份，背下的事实和做过的旧题都帮不上忙。2019 年还是 GPT-2 时代，可以说相当有预见性。第一代的题是几张网格图，猜下一张——比如把黄色补齐成一个矩形，人看十秒就会。

- **轨迹 (55:39)**

- ARC-AGI-1 刚出时，GPT-3 一代的预训练模型得分是 0——正如设计者所愿：预训练学的是互联网上的事实和语言模式，对这些题没有直接帮助。
- 2024 年 OpenAI 的 o1、o3 出现，分数陡然起飞，ARC-AGI-1 基本被解决。
- 2025 年的 ARC-AGI-2 还没完全解决，但看起来在路上。
- 上个月刚出的 ARC-AGI-3 改成了交互式环境——线上玩的一种游戏，不用语言，纯靠找规律——目前分数极低。讲者说明年再讲这门课，这页幻灯片肯定要更新。

- **讲者的评价：**是推理能力解锁了这条曲线。预训练的知识有没有用？没有预训练就没有推理模型的爆发，所以它重要，只是在 ARC 的历史上长期不可见。把推理从知识里剥离非常难，ARC 大概是最好的尝试，但未必真能解耦——只要有人在乎，什么 benchmark 都能被冲上去，这些题终究来自某种先验。另一个限制：它要求人类可解，所以覆盖不到超人类的推理（IMO 金牌、开放数学问题）。但它确实暴露了当前模型的缺口。

- **问答：这么图形化的题怎么喂给语言模型？ (59:15)** 讲者记得是 64×64 的网格，可以给图像，也可以转成 ASCII 或其他文本表示；无论哪种，都有一个空间推理的成分，和自然语言无关。

小注：ARC-AGI-1 和 -2 的网格最大 30×30；64×64 是 ARC-AGI-3 交互式游戏的网格尺寸（推断，讲者自己也说不确定）。

7. 安全类：连"安全"是什么都还在定义

- **对照汽车：**汽车的安全有明确定义——撞墙测试、安全气囊、假人，背后是几十年的游说和标准化。AI 的安全没有这么好的答案，讲者只给了几种人们在想的东西。
- **HarmBench：**用有害的 prompt 去问，期望模型拒绝。这是最主流的安全观——防止坏人拿模型做坏事。
- **AIR-Bench (讲者组参与)：**从整体看安全——把欧盟、中国、美国的监管框架和各公司的政策收集起来，建一套"可能出什么问题"的分类法，再据此构造 prompt 来评。
- **越狱 (jailbreaking)** 是安全里自成一体的子问题：模型被训练得会拒绝有害指令，但聪明人能绕过去。早期工作 GCG 用逐坐标的优化算法自动搜索绕过安全训练的 prompt；惊人的是在开源模型上优化出的

攻击能**迁移**到闭源模型——一串乱码后缀加一句"一步步毁灭人类的计划"，当时 OpenAI 的模型照答不误。讲者希望这些攻击现在已失效；那份"计划"是否真有害另说，期望行为总归是拒绝。

- **安全到底是什么 (1:03:22)**

- 它高度依赖语境：政治、法律、社会规范因国家而异。
 - 风险种类多，需要的关注也不同：幻觉（医疗、法律、金融场景尤其要紧）——它和能力正相关，模型越准就越少胡说；谄媚；协助犯罪——这一项和能力是相反的；不平等；人失去批判性思维——又是和能力正相关的风险。把"确保 AI 对人是好事"当作安全的定义，题目就大得多了。
 - 双重用途：网络安全智能体既能攻破系统，也能做渗透测试让系统更安全——同一种能力是风险还是收益，是把双刃剑。
- 对齐相关的训练方法在第 17 讲。

8. 生态效度：评测和真实使用隔着多远

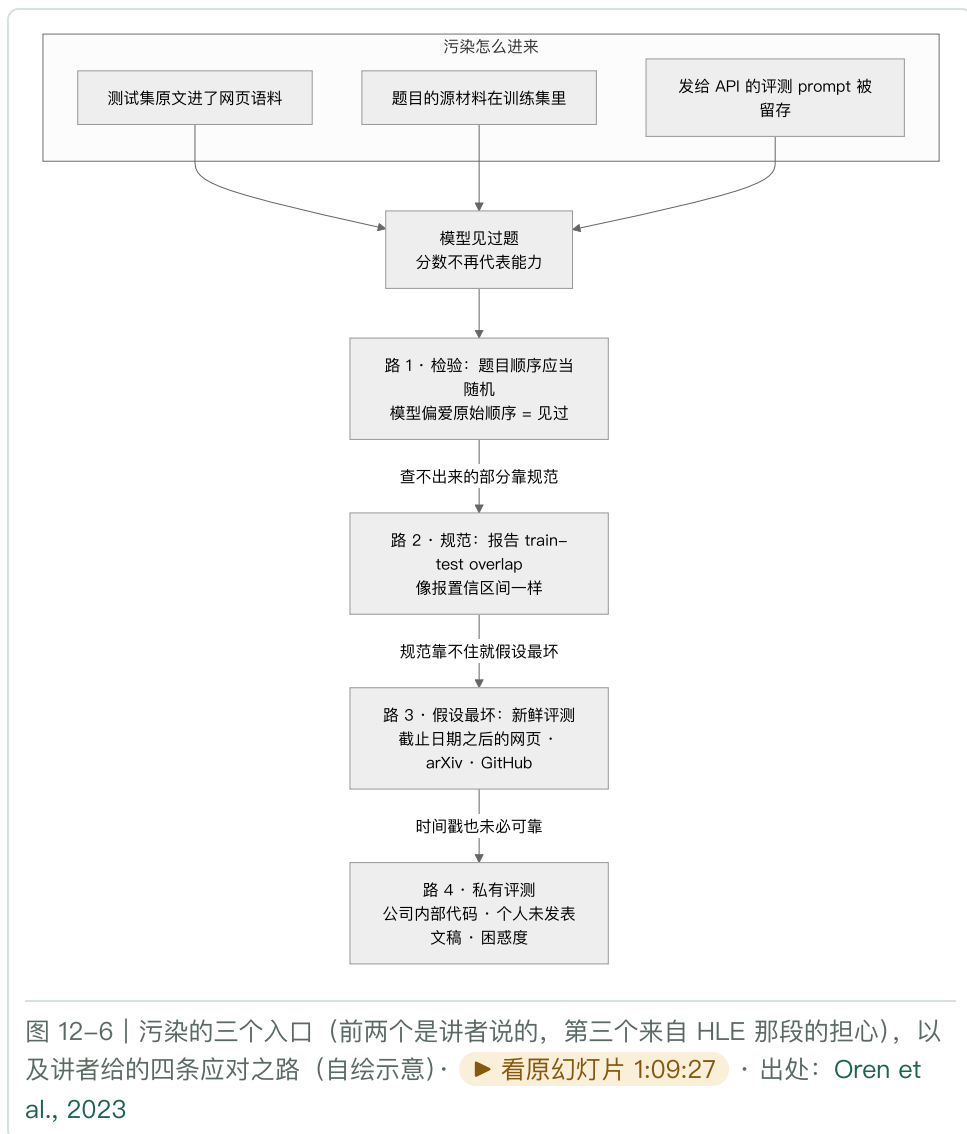
- **生态效度** (ecological validity)：评测多大程度上抓住了真实使用。考试离真实使用最远；Arena 的 prompt 来自真人，但那群人、那些用例是不是对的分布，也说不准。下面几个 benchmark 至少在**用例层面**追求真实（不是逐条 query 层面）：
 - **GDPval** (OpenAI)：按美国 GDP 取前 9 个行业，请平均约 14 年经验的从业者出任务——护士、礼宾、房产经纪、影视剪辑等等。
 - **临床任务集**（应为讲者组的 MedHELM）：此前医学 benchmark 多半是执业考试题，但人也不是考过试就直接上手术台（中间还有漫长的过程），模型更不该考过试就直接部署。这个项目从 29 位临床医生那里收集了 121 项任务，代表临床医生真正会拿去问模型的事，和多选题考试完全不同。
 - **用 LLM 分析真实数据**（应为 Anthropic 的 Clio）：真正知道用户在干什么的是模型开发者。出于隐私不能直接看用户数据，但可以让语言模型去看、去归纳一般模式——于是能知道人们拿 Claude 做哪些类型的事。真实性和隐私是拉扯的：最理想的是直接从真实 query 流里采样来评，可那正是隐私不允许的。

小注：GDPval 的公开数字——9 个行业、44 种职业、1,320 项任务，出题者平均 14 年经验（应出自 OpenAI 的 GDPval 报告）。MedHELM 论文摘要：5 大类、22 子类、121 项任务，分类法由 29 位临床医生验证。讲

者全程没有提 HELM 这个名字，但 MedHELM 和 AIR-Bench 都出自那条线。

- **给造模型的人：**这一节就是第 1 讲里"外部评测"的那一头——生态效率高的评测贵、慢、一次性；内部评测（困惑度、已饱和的 benchmark）便宜、平滑、可反复用。开发循环里两头都要，但用途不同。

9. 训练—测试重叠：四条路



- **为什么现在成了问题：**机器学习第一课就是别在测试集上训练。基础模型之前这很清楚——ImageNet、SQuAD 都有 train / test split，大家玩同一个游戏。现在的模型在整个互联网乃至更多的数据上训练，而且你不知道数据里有什么。
- **路 1：推断模型有没有见过测试集。**Tatsu 组的一个巧妙想法：benchmark 里题目的顺序本应是随机的；如果模型偏爱某个特定顺

序、而那正是 benchmark 发布时的顺序，多半是把它训进去了。

```
def order_test(model, benchmark, n_shuffles=100):
    canon = model.log_prob(concat(benchmark))          # 发布时的题目
顺序
    shuffled = [model.log_prob(concat(shuffle(benchmark)))
                for _ in range(n_shuffles)]           # 打乱顺序的对
照
    # 没见过题的模型对任何顺序一视同仁: canon 应落在 shuffled 的分布里
    p_value = mean(s >= canon for s in shuffled)
    return p_value < 0.05                             # 显著偏爱原顺
序 = 见过
```

一个可交换性检验：把原始顺序的对数似然和打乱顺序的对比，只有“见过这份文件”的模型才会对原始顺序另眼相看；它只需要 log prob，所以对开放权重模型和暴露 log prob 的 API 都能做（细节按论文，这里是示意）。

- **路 2：规范——报告 train-test overlap。**这是社区规范的问题。统计学里报估计值必带置信区间，是标准做法；讲者组的立场文主张：你要是声称某个 GPQA 分数，就该同时给出“没在测试集上训练”的证据。

小注：那篇立场文调查了 30 家模型开发者，只有 9 家报告 train-test overlap（4 家公开训练数据、5 家公开方法）。

- **路 3：干脆假设最坏——所有公开评测都被训过了，那就用新鲜题。**LiveCodeBench、Uncheatable Eval 一类做法：持续抓取模型截止日期之后的新网页、新 arXiv 论文、新 GitHub 仓库来出题。看着很稳，但时间戳也不绝对安全：今天发到 GitHub 的代码可能是从别的旧仓库改来的，“新”未必真新。
- **路 4：私有评测。**公司都这么做：Google、OpenAI 有内部代码库，默认不在互联网上，自己也不会拿来训练，就能拿来评。个人也行——讲者读博时那些从没上网的被拒论文就是他的私有测试集。这类评测**特别适合困惑度**：只要有一份好语料就能算 log prob，不需要标注。
- **给造模型的人的污染卫生：**四条路不互斥——开发时用路 4 的私有困惑度盯训练，对外报告走路 2，比较他人模型时用路 1 和路 3 交叉验证。第 13—14 讲的数据流水线会回到去污染（decontamination）：那是训练侧的功课，这一讲是评测侧的。

10. 数据集质量、评测的目的与总结

- **SWE-bench 为什么要有 Verified**：原版 SWE-bench 有一批任务的单元测试不够严格，还有别的毛病，后来修出了 Verified 版。很多 benchmark 走过同样的路——GSM8K、MMLU 都被仔细审计过，发现有的题坏了：题干说“根据下面的曲线”，可根本没给曲线；问“婴儿有没有穿袜子”，无从判断。
- **智能体 benchmark 更难审**：有一篇论文（应为 Zhu et al., 2025 的 *Establishing Best Practices for Building Rigorous Agentic Benchmarks*）指出智能体 benchmark 问题很多，而且比看题、看选项难查得多——它是一整个环境。代码题有通过 / 不通过的测试当然好，但测试用例可能不完整，全部通过也不等于解法能用。还有的情形，智能体交一个空回复就能拿到约 38%——这类平凡解必须小心避开。

小注：那篇论文的摘要点名 SWE-bench Verified 的测试用例不足、 τ -bench 把空回复计为成功；38% 应是 τ -bench 的某个子集上“什么都不做”的通过率（推断）。论文给出的 ABC 清单用在 CVE-Bench 上，把高估的分数压低了 33%。

- **看输出**：Docent 这个工具用语言模型检查智能体的轨迹、找出问题——是对重度量化的 benchmark 文化的一种定性补充。讲者的建议是无论造 benchmark 还是跑 benchmark，一定亲自看输出、做审计，确认量到的是你以为在量的东西。

小注：Docent 出自 Transluce (<https://docent.transluce.org/>)。

- **评测为了什么 (1:15:37)**：没有一个评测能统治一切，所以必须先说清目的，而这常常没人说。四种典型目的，会导向不同的 benchmark 组合：
 1. 用户或公司做采购决策——某个用例上选 A 还是 B；
 2. 研究者想量化心里那个“智能”的直觉；
 3. 为商业或政策理解收益与危害；
 4. 模型开发者要反馈来改进模型。
- **评的到底是什么**：基础模型之前，研究者评的是方法——train / test split 固定，变的只有算法。今天评的多半是模型和系统，什么都可以变；例外是 nanoGPT speedrun，它刻意评算法：多快能把一个模型训到某个水平。评最终模型有意义，因为那才是要上线的东西，但要明确说出你在评什么。

- **总结 (1:17:40)**：没有唯一正确的评测，先选定要量什么。六个家族——困惑度、考试、对话、智能体、推理、安全——在难度、真实性、效度上各有取舍；很难同时做到真实、困难、生态效度高、又没有训练一测试污染，通常得牺牲其中一项，牺牲哪一项由目的决定。下周开始讲训练数据。

家族	评什么	打分方式	最大的软肋（课上的话）
困惑度	分布	log prob 归一化	榜单验不了真；每个 token 都计费
考试	知识与推理	字母比对、答案抽取	离真实使用远；保质期短；污染
对话	说了什么	成对偏好、Elo、LLM 裁判	风格与正确性混淆；裁判偏置；定义不清
智能体	做了什么	测试、flag、竞赛成绩	分数里混着 scaffold；环境和测试可能有洞
纯推理	剥离知识的推理	网格题、交互游戏	未必真能解耦；只到人类水平
安全	拒答、越狱、风险	拒答率、攻击成功率	"安全"本身依赖语境、风险多元、双重用途

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Artificial Analysis 智能榜与"智能对成本"散点	榜从 GPT-5.5 往下排；散点里贵的一般更聪明，但偏离对角线的不少	3:10	artificialanalysis.ai
GPT-2 零样本困惑度表	各列是不同数据集，各行是模型大小；PTB 一行 35 对 SOTA 46，1B Word 一行没赢	8:21	Radford et al., 2019
"Stanford was founded in 1885"	逐 token 看困惑度在哪里计费：1885 值钱，句首和 founded 不值钱	11:51	—
MMLU 的 few-shot prompt 与分数曲线	prompt 的结构：学科说明、几组问答、待答题；曲线从 GPT-3.5 turbo 到 90 多分	20:12	Hendrycks et al., 2020
GPQA 出题流水线	写题 → 专家验证 → 改题 → 第二位专家 → 非专家带 Google；diamond 的两个条件	24:20	Rein et al., 2023
HLE 各模型分数	刚出时全是个位数；2026 年的图上最高 64.7	28:28	Phan et al., 2025
Chatbot Arena 界面与 Elo 公式	两份匿名回答、四个按钮；胜率是评分差的函数，拟合评分使成对胜负最可能	33:32	Chiang et al., 2024

图	看什么	跳转	出处
AlpacaEval 与 Arena 的相关图	相关系数 0.98；注意点都是 GPT-4 preview 一代及以下的模型	40:46	Dubois et al., 2024
SWE-bench Verified 曲线	2024 年约 16% 起步，如今约 93	46:55	Jimenez et al., 2023
Terminal-Bench 榜	同一个模型配不同 agent，分数不同——scaffold 那一列	48:58	tbench.ai
Cybench 的 scaffold 图	一段连续记忆：动作 → 环境反馈 → 拼回缓冲	49:58	Zhang et al., 2024
ARC-AGI 轨迹图	长期贴着 0，2024 年 o1 / o3 处陡升；ARC-AGI-2 在追，ARC-AGI-3 又归零	55:39	arcprize.org
GCG 越狱示例	乱码后缀加有害请求，闭源模型照答；在开源模型上优化、迁移到闭源	1:02:52	Zou et al., 2023
MMLU 坏题示例	题干引用不存在的曲线："婴儿有没有穿袜子"无从判断	1:14:05	—

提到的工作

名称	在本讲里的作用
Artificial Analysis	"好 = 榜单分数"和"好 = 分数对成本"的例子；intelligence index
OpenRouter 用量榜	"好 = 有人付费使用"的经济学视角
Penn Treebank · WikiText-103 · 1B Word Benchmark	2010 年代 in-distribution 困惑度评测的标准数据集
Jozefowicz et al., 2016	"2016 年那篇论文"（应为）：CNN 加 LSTM 在 1B Word 上大幅降低困惑度，纯神经胜出
GPT-2（Radford et al., 2019）	WebText 40 GB、1.5B 参数、零样本评标准数据集；PTB 35 对 46
LAMBADA（Paperno et al., 2016）	需要长上下文才能填的最后一个词；伪装成准确率的困惑度
HellaSwag（Zellers et al., 2019）	多选题形式的句子续写；本质上也是困惑度
MMLU（Hendrycks et al., 2020）	57 科、few-shot；从勉强高于随机到 90 多分
MMLU-Pro（Wang et al., 2024）	去噪、10 选项、配 chain of thought；33 → 近 90
GPQA（Rein et al., 2023）	Google-proof；61 位 PhD、diamond 子集；专家 65%、GPT-4 39%、现在 94
Humanity's Last Exam（Phan et al., 2025）	众包、多轮审核、私有保留集；Mythos 64.7
Chatbot Arena（Chiang et al., 2024；现称 Arena AI）	匿名成对投票加 Elo；真实 prompt；投票人群、风格与正确性、谄媚

名称	在本讲里的作用
AlpacaEval · Length–Controlled AlpacaEval (Dubois et al., 2024)	LLM 裁判对基线的胜率；长度偏置的回归修正；与 Arena 相关 0.98
WildBench (Lin et al., 2024)	真实对话里的 prompt；逐题生成 checklist 的 LLM 裁判
SWE–bench (Jimenez et al., 2023) · SWE–bench Verified	GitHub issue 到 PR，单元测试判分；Verified 修掉了测试不严的任务；16% → 93
Terminal–Bench	终端环境；93 人众包、2.0 版 89 题；同模型不同 agent 分数不同
Cybench (Zhang et al., 2024)	40 道 CTF；最简 scaffold 的示例；10% → 完全解决
MLE–bench (Chan et al., 2024)	Kaggle 竞赛全流程；scaffold 差异大
ARC–AGI (Chollet, 2019) · ARC Prize	剥离知识的推理测试；1 / 2 / 3 三代的轨迹
OpenAI o1 · o3	2024 年让 ARC–AGI–1 起飞的推理模型
HarmBench (Mazeika et al., 2024)	有害 prompt 期望拒答的安全评测
AIR–Bench 2024 (Zeng et al., 2024)	从欧盟、中国、美国监管框架和公司政策建风险分类法
GCG (Zou et al., 2023)	逐坐标优化的自动越狱；开源模型上优化、迁移到闭源
GDPval (OpenAI, 2025)	按 GDP 取 9 个行业、约 14 年经验的从业者出任务
MedHELM (Bedi et al., 2025)	"121 项任务、29 位临床医生"的临床任务集（应为）
Clio (Tamkin et al., 2024)	用 LLM 在保护隐私的前提下分析真实用量（应为）
ImageNet · SQuAD	基础模型之前 train / test split 清晰的例子
Oren et al., 2023	路 1：靠题目顺序检验测试集污染（"Tatsu 组的想法"）
Zhang et al., 2024 (立场文)	路 2：模型开发者应报告 train–test overlap
LiveCodeBench (Jain et al., 2024) · Uncheatable Eval	路 3：截止日期之后的新鲜题（两个名字都按字幕推断）
GSM8K · MMLU 的审计	坏题的例子：缺失的曲线、婴儿的袜子
Zhu et al., 2025	智能体 benchmark 的质量问题：测试不完整、空回复得分（应为）
Docent (Transluce)	用 LLM 审计智能体轨迹的工具
nanoGPT speedrun	今天少见的"评方法不评模型"的例子

术语对照

English	中文
construct	构念：想量的抽象概念，如"会推理"
north star	指路星：所有人共同追逐的指标
perplexity / log loss / likelihood	困惑度 / 对数损失 / 似然：同一件事的三种说法
in-distribution / out-of-distribution evaluation	同分布 / 分布外评测：测试集与训练集是否同源
zero-shot / few-shot	零样本 / 少样本：prompt 里给不给示例
entropy	熵：真实分布下能达到的最好困惑度的对数
conditional perplexity	条件困惑度：只对 prompt 之后的回答计费
cloze task	完形填空任务（LAMBADA）
leaderboard	排行榜
exam benchmark	考试类基准
multiple choice	多选题
saturation	饱和：模型分数逼近上限，benchmark 失去区分度
shelf life	保质期：benchmark 从发布到饱和的时间
Google-proof	搜不到答案的：靠搜索引擎解不了
diamond subset	钻石子集：GPQA 里两位专家都对、至多一位非专家对的题
held-out / private set	保留集 / 私有集：不公开的测试题
contamination / decontamination	污染 / 去污染：测试数据混进训练数据，及其清除
answer extraction	答案抽取：从生成文本里取出最终答案
chain of thought	思维链
pairwise comparison	成对比较
Elo rating / Bradley-Terry model	Elo 评分 / Bradley-Terry 模型：由成对胜负拟合的评分
LLM-as-a-judge	用 LLM 当裁判
win rate	胜率：对某个基线模型的获胜比例
length bias / length-controlled	长度偏置 / 长度校正
checklist / rubric	检查清单 / 评分细则
sycophancy	谄媚：讨好用户而非说实话
ecological validity	生态效度：评测与真实使用的接近程度
agent / scaffold	智能体 / 脚手架：决定何时调模型、给什么工具的外围逻辑
sub-agent / hierarchical delegation	子智能体 / 分层委派

English	中文
context engineering	上下文工程
trace	轨迹：智能体一步步的动作与观察记录
capture-the-flag (CTF)	夺旗赛：攻入系统取得一个唯一字符串
fluid intelligence	流体智力：不依赖已有知识的推理能力
jailbreak	越狱：绕过安全训练让模型执行有害指令
refusal	拒答
dual use	双重用途：同一能力既可为善也可为恶
train-test overlap	训练—测试重叠
fresh eval	新鲜评测：用模型截止日期之后的数据出题
audit	审计：逐条检查题目或输出
speedrun	速通：固定目标比谁训得快

字幕勘误

"Hendricks" → Hendrycks; "GBQA" → GPQA; "Humanities Last Exam" → Humanity's Last Exam; "Methos" → Mythos; "01 and 03" → o1 and o3; "SweetBench / side bench" → SWE-bench; "I bench" → Cybench; "MLE engineering" → MLE-bench; "harm bench" → HarmBench; "Airbench" → AIR-Bench; "Sickle fancy" → sycophancy; "betting crimes" → abetting crimes; "GDP Val" → GDPval; "Tatsuo's group" → Tatsu's group (Tatsunori Hashimoto); "Code Bench" → LiveCodeBench (推断); "Untreated Eval" → Uncheatable Eval (推断); "Gia GSM8K" → GSM8K; "genetic benchmarks" → agentic benchmarks; "Lambada" → LAMBADA; "soda" → SOTA; "decontam" → decontamination; "closed tasks" → cloze tasks; "log prob" → log prob; "Squad" → SQuAD; "chat about arena / JetBot" → Chatbot Arena; "ELO" → Elo; "Wiki Text 103" → WikiText-103; "Docent" 无
误但字幕未大写。

带走的问题

- 困惑度榜单验不了参赛者交回的 log prob 是不是合法分布。如果你必须办这样一个榜，能设计出什么合同来逼近可验证——比如要求暴露完整的下一 token 分布并抽查归一化，或者同时要求生成并对照？为

什么讲者说私有评测"特别适合困惑度", 这和第 2 节的"要看到分布"是否矛盾?

2. 顺序检验只能抓到"整份文件按原顺序被训进去"这一种污染。题目被改写、题目的源材料被训进去、评测 prompt 经 API 回流——这三种它各漏掉多少? 把路 1 到路 4 组合成一套你自己会执行的污染卫生流程, 并说明每一步的成本。
3. Elo 只要求比较图连通。一个新模型只打了几十场, 它的评分方差有多大? 投票里的长度偏好和风格偏好会怎样进入评分? 如果要给 Arena 也加一个"长度校正", 回归里该放哪些变量?
4. 讲者说空回复在某个智能体 benchmark 上能拿约 38%。你在做的智能体 benchmark 里, 哪些平凡策略(空回复、只跑测试、只回显题目)必须作为基线报告? 榜单上怎么把 scaffold 的贡献和模型的贡献分开——固定 scaffold 换模型、固定模型换 scaffold, 各能回答什么?
5. 真实、困难、生态效度高、无污染, 讲者说通常只能三选二或四选三。对你自己的产品评测, 你会牺牲哪一项? 把这个选择和第 10 节的四种目的对上号, 再想想 CME295 第 8 讲和 CS329A 第 8 讲里的评测框架各自默认牺牲的是哪一项。

CS336 第 13 讲 | 数据（上）：来源与数据集 (Data I: Sources, Datasets)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 13 讲 (课程表上是 2026 年 5 月 11 日)；数据单元的第一讲：第 12 讲讲是怎么评，这两讲回答"该在什么数据上训练"——本讲讲数据从哪来、各代数据集怎么造；第 14 讲讲过滤、去重、混合与合成数据 视频：
<https://www.youtube.com/watch?v=-qm0ln33G24> (1:22:01；英文字幕为自动生成，数据集名、人名和缩写错得不少，见文末勘误) 讲者：Percy Liang (全程；开场说"按这门课的精神，从零讲起"，结尾预告"下次继续讲数据"并点到 assignment 4，与课程表一致) 课程主页：<https://stanford-cs336.github.io/>；本讲不围绕某一篇文章，而是串起一条开源数据集谱系：C4 · The Pile · RefinedWeb · Dolma · DCLM · Nemotron-CC · Common Pile；法律部分的主线是 Consent in Crisis (Longpre et al., 2024)

一句话：数据是语言模型里最重要、也最少被公开的环节——Llama 3 论文把架构和训练流程写得一清二楚，数据只有一句"多种来源"，原因是竞争秘方和版权风险；所谓"在整个互联网上训练"并不成立，网络是一堆活服务器，能拿到的只是爬虫够得着、登录墙和 robots.txt 没挡住、terms of service 和许可证没禁止的那一部分，而且网上几乎一切内容都自动受版权保护，训练只能靠许可证或 fair use 的四要素，2025 年的 Anthropic 案给出的答案是"这次训练是 fair use，盗版书不是" (15 亿美元和解，约每本 3,000 美元)；实际的原料来自 Common Crawl (每月 30—50 亿页，一次 dump 约 372 TB) 加上 Wikipedia、GitHub、arXiv 这些自带 dump 的高质量口袋，而从 2019 年的 WebText (40 GB) 到 C4 (156B token)、The Pile、Llama 1 (1.2T)、RefinedWeb (5T)、FineWeb (15T)、DCLM (240T 的池子筛到 3.8T，约 1.4%)、Nemotron-CC (合成改写 6T)，每一代数据集的差别都在"怎么定义高质量"——规则、像不像 Wikipedia、Reddit 外链、还是训一个 fastText 分类器——这一步至今靠经验和 vibes，也正是第 14 讲和 assignment 4 的题目。

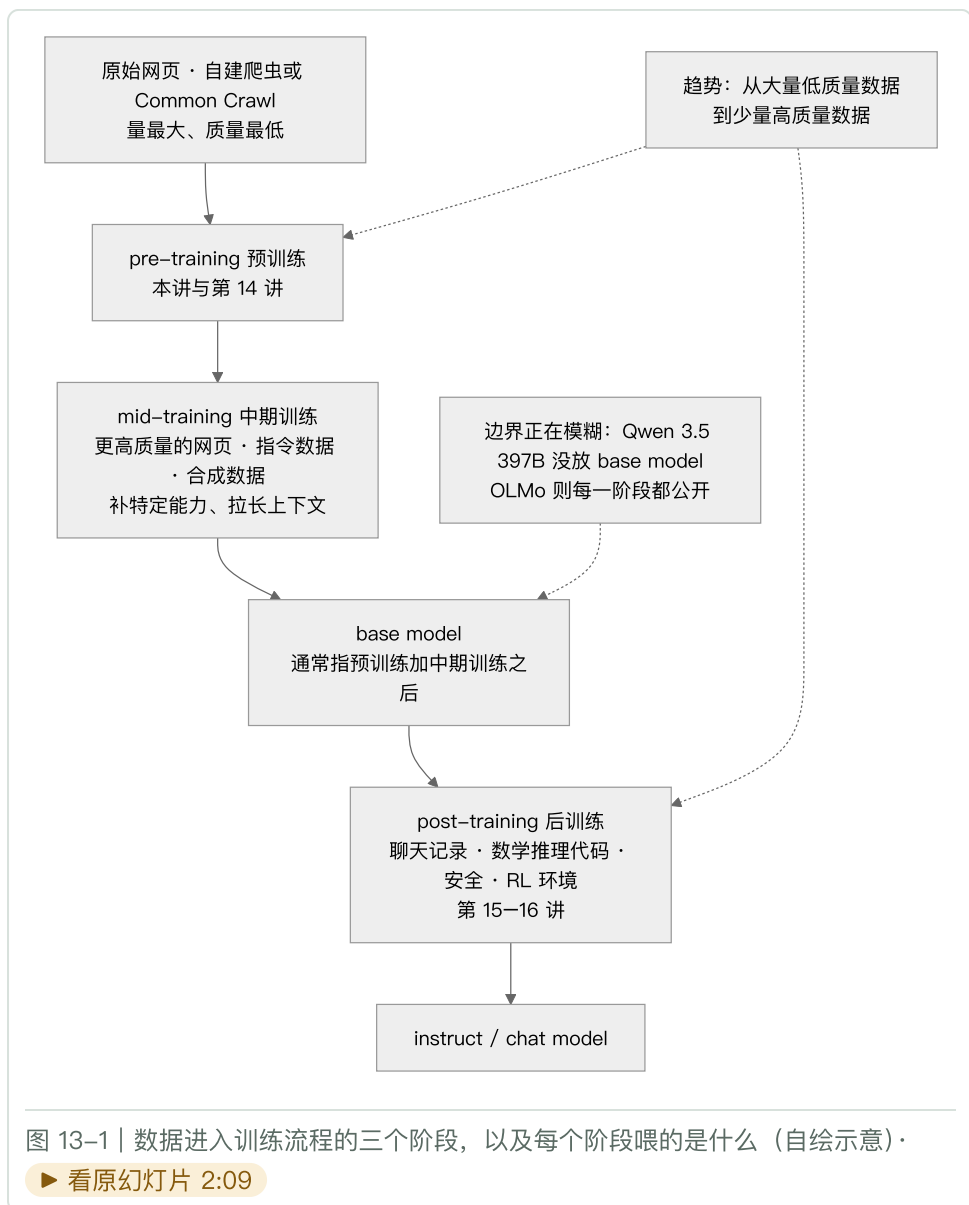
时间轴

时间	内容
0:05	开场：数据是最重要、也最少被公开的环节；Llama 3 论文对数据只字不提的两个原因；数据是长尾问题

时间	内容
2:09	数据出现在三个阶段：pre-training / mid-training / post-training；base model 与 instruct model；OLMo 全公开
4:41	"在整个互联网上训练"不成立：网络是活服务器，得靠爬虫；动态内容、登录墙、robots.txt、Cloudflare、限速
9:19	法律层：terms of service 与许可证；Consent in Crisis：限制在 2023 年后陡增
11:21	爬虫一天打一百万次把网站压垮；影子图书馆 LibGen、Anna's Archive
14:25	版权法：1709 / 1976；什么能受保护、门槛多低、保护多久；网上一切几乎都有版权
18:30	合法使用的两条路：许可证（public domain、Creative Commons、付费）与 fair use 四要素
23:04	fair use 的例子：Google Books 案；版权不等于逐字记忆；对语言模型意味着什么
27:15	判例现状：NYT 诉 OpenAI、Anthropic 案 15 亿美元和解、Meta 案；问答：许可证事后变更
32:04	Common Crawl：2007 年起每月 30–50 亿页；爬虫就是图遍历；WARC 与 WET；HTML 转文本的工具很要紧
36:09	三个高质量口袋：Wikipedia（dump 投毒）、GitHub（GH Archive、Software Heritage）、arXiv
43:30	问答：模型生成的数据能不能用；爬取里混进盗版书怎么办
45:03	数据集谱系 I：BERT 的 BooksCorpus、GPT-2 的 WebText、CCNet、C4、GPT-3
53:47	数据集谱系 II：The Pile（Books3、Gutenberg、Stack Exchange、Enron）、Gopher 的 MassiveWeb
59:21	Llama 1 的数据表与 RedPajama；"网络就够了"的 RefinedWeb → FineWeb；Dolma
1:04:29	DCLM：240T 的池子筛到 1.4%，fastText 分类器；Nemotron-CC：教育价值打分与合成改写，6T
1:10:45	代码：The Stack v1 → v2，PR 怎么线性化；Common Pile：只用许可数据能走多远
1:19:30	小结：数据不会从天上掉下来；过滤是 200T 到 3T 的大缩减；下一讲

核心内容

1. 为什么先讲数据：最重要，却最少被谈起



- **本讲的位置：**前面十几讲解决的是“给定数据怎么训”，第 12 讲解决“怎么知道模型好不好”；这两讲回答“该在什么数据上训”。讲者的立场很直接：数据是语言模型里最该做对的一件事。
- **证据是各家的沉默：**Llama 3 是 open-weight 模型，论文对架构完全透明，连训练流程都写了，唯独数据只有一句“来自多种来源”（0:36）。保密有两个理由：数据是竞争的秘方，不想让对手知道自己正在做什么；以及版权责任——说了训练了什么，就可能被告。
- **数据工作从来没消失：**foundation model 之前，“做数据”等于为监督学习标注；现在预训练几乎不标注了，但清洗与整理的活一点没少。数据永远是瓶颈，因为它本质上是长尾问题，而且随人力线性扩展——架构和系统只容得下少数人，数据的活可以无限并行，所以各家的数据团队都很大。

- **三个阶段**（图 13-1）：预训练吃原始网页文档；mid-training 用更高质量的数据补特定能力、给长上下文；post-training 用聊天记录，做 RL 就用环境，越来越任务化。实际的界线很模糊，也可能不止三段，但这是基本模板；趋势是从"大量低质量"走向"少量高质量"。文献里的 base model 通常指预训练加中期训练之后，instruct / chat model 指后训练之后——可最新的大模型干脆不放 base model（讲者举 Qwen 3.5 397B：只有最终版，没有中间 checkpoint）。反过来 AI2 的 OLMo 每一阶段都公开，所以课上拿它当样本（3:41）：预训练是网页、学术论文、数学页面和证明；中期是更高质量的网页、指令数据和大量合成数据；后训练是聊天日志、更多的数学、推理和代码，安全也在这一步做。
- CME295 第 4 讲把这一段压成了一页（Common Crawl、GPT-3 的 300B token、Llama 3 的 15T）；这里把每个名字拆开看它是怎么来的。

2. "在整个互联网上训练"? ——网络到底是什么，爬虫能碰到哪些

- **先过 type check**：互联网是一堆活着的服务器，你发一个请求、它回一个响应；模型没法"在服务器上训练"，除非它是个上网的 RL agent。稍准确的说法是"在公开的万维网上训练"——也不对，因为你拿到的只是某个爬虫的一份快照。
- **爬虫**：有人（不一定是你）写一个 crawler，从一个种子 URL 集合出发，边爬边发现新页面、下载。但爬不全，障碍分几层：

障碍	例子	性质
动态内容 / deep web	现在很多网站是 app，URL 不再是内容的完整描述，得点按钮、提交表单；Discord 这类根本爬不到	技术
登录与付费墙	Facebook、X、LinkedIn、New York Times：海量内容锁在 walled garden 里；只有平台自己能——Meta 用 Facebook 的数据、xAI 用 X 的，别人不行	技术 + 合同
robots.txt	放在站点根目录，声明谁可以爬什么；nytimes.com 禁了一长串 AI 爬虫（OAI-SearchBot、PerplexityBot、ChatGPT-User、ClaudeBot.....）；不是法律约束，是"好公民"约定：名字在名单上就别爬	约定
反爬	Cloudflare 之类的机器人检测——你看到的验证码就是它判断你可能是机器人；封 IP 或封国家；限速	技术
terms of service	用网站等于接受一份合同，常写着"机器人走开""不得用于 AI 训练"	法律

障碍	例子	性质
内容许可证	就算 ToS 没说，内容本身也可能没给你训练的许可（第 3 节）	法律

- **限制在收紧：**Consent in Crisis (Longpre et al.) 对常见数据集里的 URL 做纵向审计，同时看技术限制 robots.txt 和法律限制 ToS。图上 (10:19) robots.txt 的"完全限制"到 2023 年之前一直平稳，2023 年中之后陡升到接近 50%；ToS 那张图，2016 年几乎没人给页面写条款，现在多数页面有，而且多数写着不能用于 AI。所以 2020 年还能爬到的互联网，今天能合法爬的部分小得多。

小注：论文摘要的口径——2023 到 2024 一年之间，C4 里约 5% 以上的 token、或最活跃维护的关键来源的 28% 以上，被 robots.txt 完全限制；按 ToS 算，C4 的 45% 已受限。课上的"接近 50%"应是读的 ToS 那条线或头部站点那条线。

- **还没谈版权，爬虫已经会惹麻烦：**有站长抱怨 Anthropic 的爬虫 24 小时内打了它一百万次，Read the Docs 也被打爆 (11:52)。违反 ToS 或 robots.txt 是一回事，纯粹把人家服务器压垮是另一回事——花的是别人的托管费，坏的是别人用户的体验。

小注：应为 2024 年 7 月 iFixit 的公开抱怨 (ClaudeBot 一天近百万次请求) 和 Read the Docs 同期的博客 (推断，讲者说忘了站名)。

- **影子图书馆：**LibGen、Anna's Archive 之类，完全无视版权、绕过付费墙，把本该付费的书和论文免费放出来；被下架、被诉，就把服务器搬到别的国家；主张"该自由的东西被解放了"，但法律上就是盗版和侵权。里面书和论文很多，第 3 节和第 7 节会再回到它。

3. 版权：什么受保护，怎样合法地用

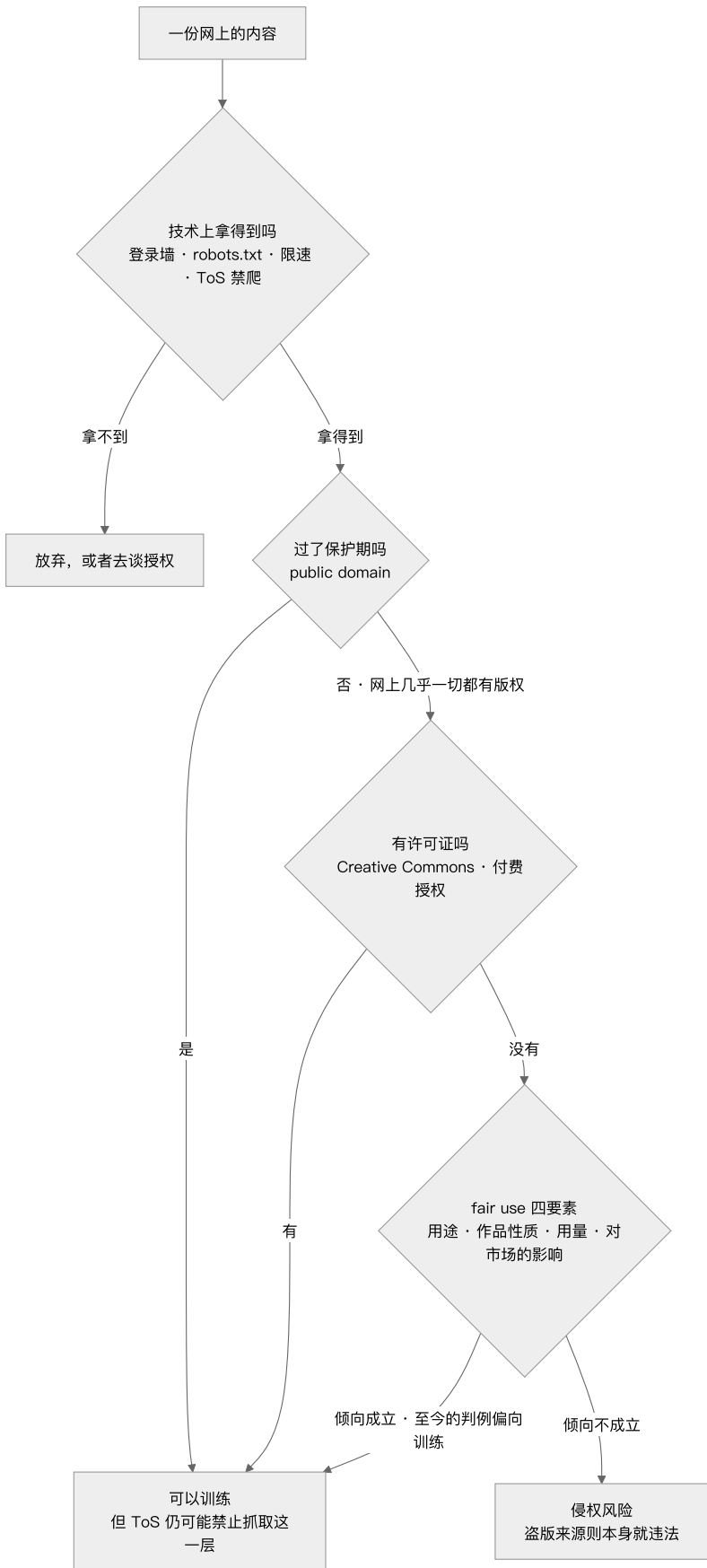


图 13-2 | "这份内容能不能拿来训练"的判断顺序：先技术、再保护期、再许可证、最后才是 fair use（自绘示意）· [▶ 看原幻灯片 18:30](#)

- **问题：**假设你守规矩——尊重 ToS、遵守限速——爬到了数据，能不能训练？这个问题没有完全解决，但过去一年有不少进展。它属于知识产权法，而这套法律的目的是激励智力创作，不是"技术上什么都不许"；四种形式里（copyright、patent、trademark、trade secret），和语言模型数据最相关的是版权。
- **版权是什么：**1709 年英国开始用版权来激励创作；美国的现代版权由 1976 年的 Copyright Act 定义，保护的是固定在任何有形媒介上的原创作品。几个要点：
 - 集合不受保护（电话簿，除非编排本身有创意）；保护的是表达不是想法——quicksort 算法没有版权，某个具体实现有。
 - 1976 年之后门槛大降：不需要发表、不需要登记，"固定下来"就行——写在网页上就自动有版权。要起诉才需要登记，65 美元，比律师费便宜得多。
 - 保护期讲者说是 75 年，到期进入 public domain，人人可用；理由是先保护创作者一阵子，过后再保护已无意义。

小注：美国现行的保护期是作者终身加 70 年，雇佣作品是发表后 95 年或创作后 120 年取短者；"75 年"是课堂上的粗略说法。

- **结论：**互联网上的一切几乎都有版权。但"有版权"不等于"不能用"，两条路（图 13-2）：
 - **许可证：**合同法里 licensor 对 licensee 说"我不告你，你可以按许可的方式用"。Creative Commons（2001 年创立）让作品像 public domain 一样自由流通、不必等 75 年——Wikipedia、OpenCourseWare、Khan Academy 都是。没有 CC 的，有钱可以买：模型开发商和内容平台之间的授权交易已经不少。
 - **fair use：**Copyright Act 第 107 条——没有许可也可以用。四个要素，都是倾向不是硬规则，真上法庭要综合权衡：用途与性质（教育强于商业，转化性使用强于原样托管）；作品性质（事实性强于虚构性——一页二战史实比一首诗的保护弱）；用量（片段强于整本）；对原作市场的影响（替代了原作者的变现就不利，开辟新市场则有利）。最后这条回到版权的初衷：对齐经济激励。
- **例子：**看完电影写摘要；照着想法重新实现算法而不抄代码；Authors Guild 诉 Google——Google Books 展示受版权书籍的片段，打了 11 年，最终判 Google 胜（所以今天还能用它），这成了讨论"训练是不是 fair use"的先例。

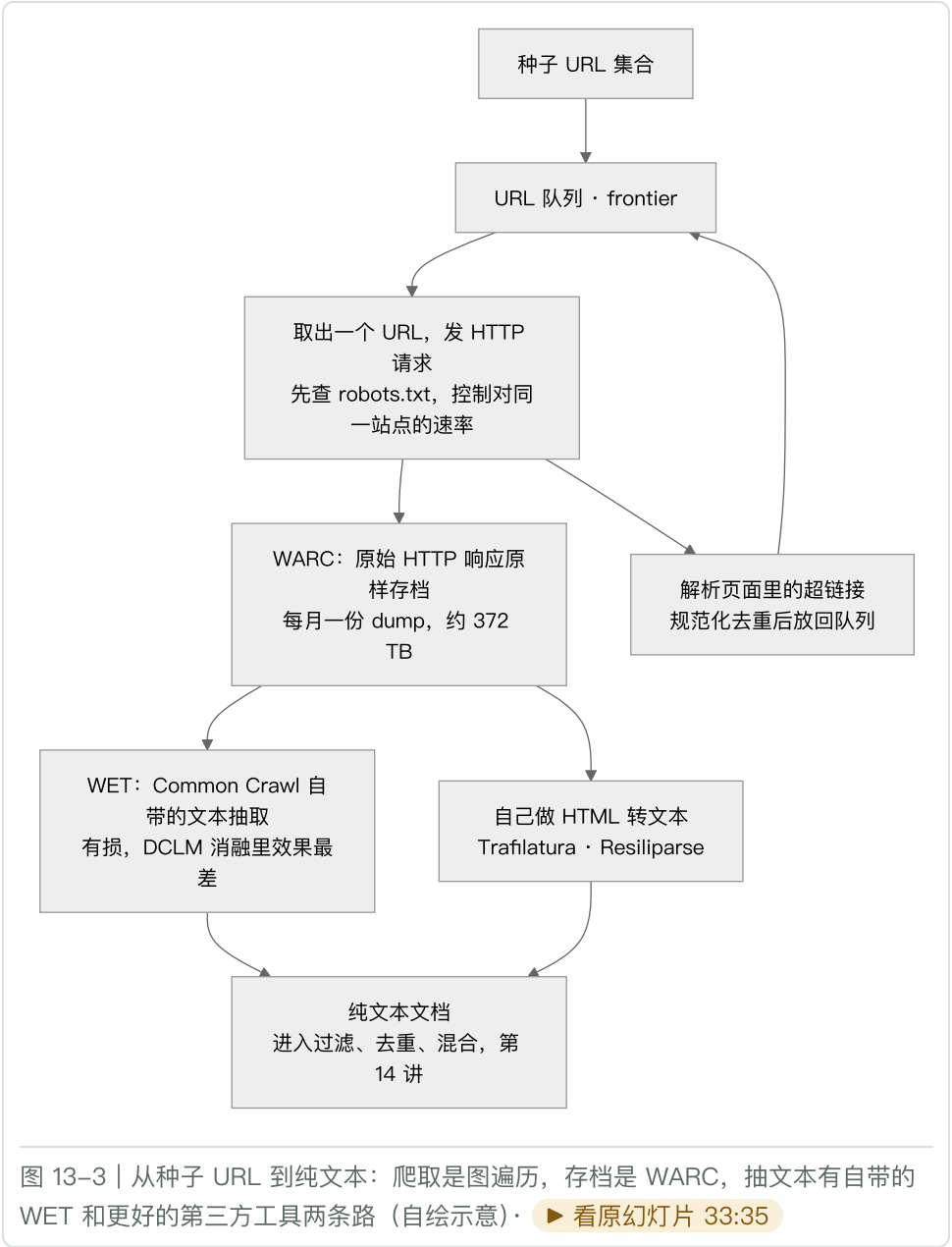
- **对做 ML 的人可能陌生：**版权不是逐字记忆的问题。论文都盯着 verbatim memorization，可那只是侵权的一种——情节和角色也能受保护（Harry Potter 这个角色本身可以主张版权，不限于某一本书），而戏仿反而更可能是 fair use。版权讲的是语义和经济，不是 n-gram 重叠。
- **对语言模型的含义 (25:10)：**
 - 复制本身就可能侵权——copyright 这个词里就有 copy——哪怕复制完什么都不做。
 - 训练直觉上带有转化性：和原样托管明显不同，模型拿数据是手段不是目的，学的是世界怎么运作，不是那份具体表达。讲者强调这是直觉，不是既定事实。
 - 但语言模型确实会影响市场（第四要素），这一点对 fair use 不利。
 - 就算 fair use 或许可证成立，ToS 还是另一层：YouTube 的视频有许可，但 ToS 禁止用机器人下载或抓取。
- **判例现状 (27:45)：**
 - 2023 年 New York Times 诉 OpenAI：你用我们的新闻训练，证据是能让 ChatGPT 几乎逐字吐出文章。仍在审。
 - 诉 Anthropic：指控盗版了数百万本书来训 Claude。去年的标志性裁决：这一次的训练是 fair use；但盗版不行——这和训练无关，盗版本身违法。有意思的是 Anthropic 还买了书、拆掉书脊扫描数字化，法院说这是 fair use；但买了不能抵消之前的盗版。结果是 15 亿美元和解，约每本 3,000 美元。
 - 诉 Meta：你用我们的书训练——而且如第 7 节会看到，Llama 论文自己写了出来。紧随 Anthropic 案的判决：训练是 fair use；用种子下载书的部分仍待决，按先例看对 Meta 不利。
 - 小结：训练至今没有被判“不是 fair use”，但裁决都很窄，不能推广成“任何版权内容都能训”；盗版明确违法（本来就知道）；领域仍在快速演变。

小注：三案应为 NYT v. Microsoft & OpenAI（2023 年 12 月起诉）、Bartz v. Anthropic（2025 年 6 月裁决、9 月和解）、Kadrey v. Meta（2025 年 6 月裁决）——推断，讲者没有报案名。

- **问答：**一份数据先有许可、后来许可变了怎么算？讲者先声明自己不是律师：先前许可覆盖的那批文档应该仍然可以训；但许可通常不只

针对一批固定的文档——Reddit 这样的站点不断有新内容，许可变更意味着之后的内容不能再用。

4. 来源一：Common Crawl，以及一次爬取是怎么工作的



- **谁来爬：**多数模型开发商有自己的爬虫，为的是完全掌控数据。没有的可以用 Common Crawl：2007 年起每月一次爬取，每次 30—50 亿页，和上次有重叠但会刻意找新页面；官网说累计 3,000 亿页，讲者觉得偏大，按每月的量乘下来对不上。总共有多少 URL 没人说得清，Google 的索引按其自述至少 100 PB。一次 dump 讲者说约 20 亿页、372 TB（和前面的 30—50 亿略有出入，两处都按原话记），主要是文本，不含图片。

- **爬取概念上简单，细节全在实现**（图 13-3）：本质是图遍历——种子 URL 进队列，弹出、下载、抽出页面里的超链接、入队，多机并行。要做的决策：下载哪些页；尊重 robots.txt；别压垮服务器；网页会变，需要一个回访策略——常变的页面多回访，不变的别浪费；URL 是动态的——同一 URL 因浏览器状态不同给出不同内容，不同 URL 又常指向同一内容，不小心就大量重复，mirror 站点更是明摆着的重复。

```
frontier = deque(seed_urls); seen = set(seed_urls)
while frontier:
    url = frontier.popleft()
    if not robots_allows(url, agent="my-bot") or
rate_limited(host(url)):
        continue # 好公民：看
robots.txt, 别压垮同一台服务器
    resp = http_get(url)
    archive.write_warc(url, resp) # 原始响应原样存档，晚
点再抽文本
    for link in extract_links(resp.html):
        if canonical(link) not in seen: # 很多 URL 指向同一份
内容
            seen.add(canonical(link)); frontier.append(link)
    schedule_revisit(url, change_rate(url)) # 常变的页面多回访
```

这就是讲者口中的“图遍历加一堆策略”：真正的难点不在循环本身，而在 robots.txt、限速、回访和去重这几行里。

- **两种格式**：WARC 是原始 HTTP 响应的存档——HTTP 就是“发一个带 URL 的 GET，回一个响应”，WARC 存的就是那个响应。Common Crawl 还多做一步，给出抽好文本的 WET 文件——这必然有损，而且不一定是用网络数据的最好方式。

小注：Common Crawl 每次 dump 发三种文件：WARC（原始响应）、WAT（元数据）、WET（抽出的纯文本）；字幕里的“WAT”按上下文应为 WET。

- **HTML 转文本的工具本身有影响**：Trafilatura、Resiliparse 之类很多，而且怎么转真的会改变下游效果。DCLM 论文的消融（35:39）拿 Common Crawl 自带的 WET 对比这两个工具，后者更好。

```
from warcio.archiveiterator import ArchiveIterator
import trafilatura
```

```

for record in ArchiveIterator(open("CC-MAIN-xxxx-00000.warc.gz",
"rb")):
    if record.rec_type != "response":                # 只要
HTTP 响应, 跳过请求与元数据记录
        continue
    if "text/html" not in record.http_headers.get_header("Content-
Type", ""):
        continue
    html = record.content_stream().read()
    text = trafilatura.extract(html)                # 去导航、
广告、样板, 只留正文
    if text:
        yield {"url": record.rec_headers.get_header("WARC-Target-
URI"), "text": text}

```

读 WARC 的骨架：一条记录一个响应，正文抽取交给专门的工具——换成 WET 就等于把这一步交给 Common Crawl 替你做，而消融说明它做得不如你自己做。

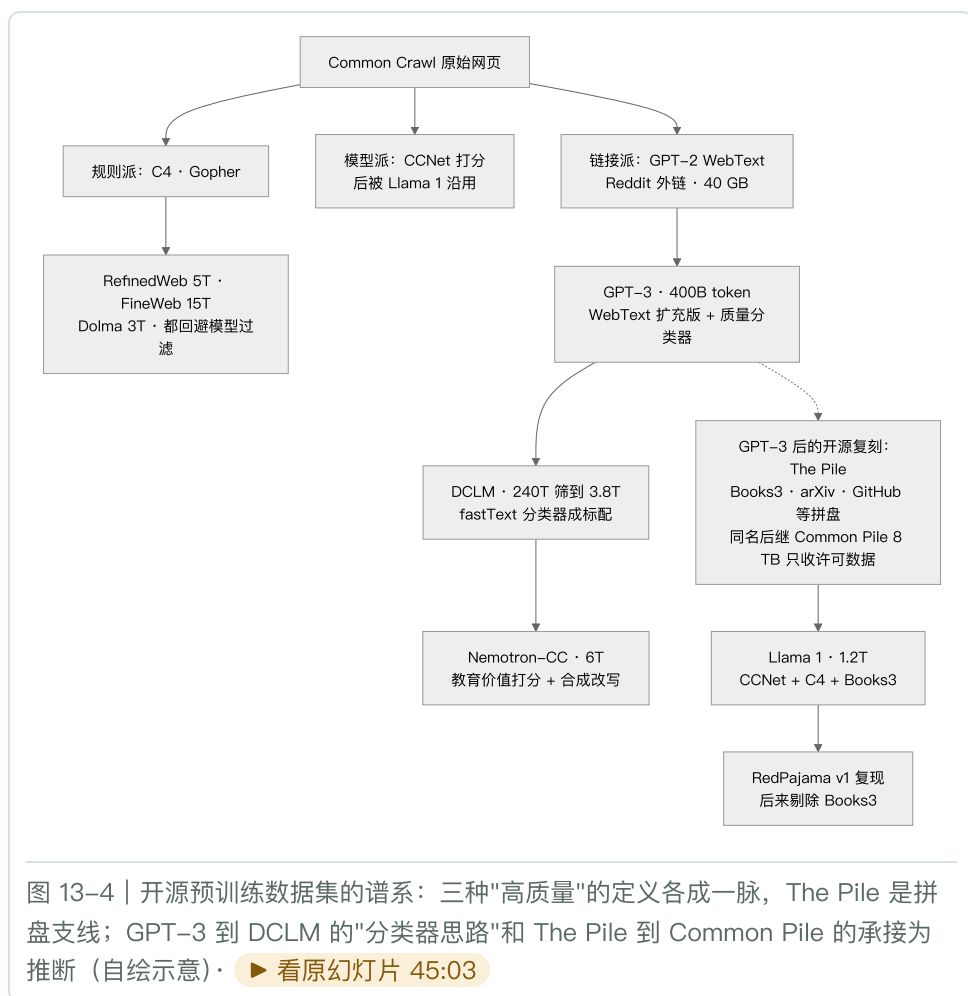
5. 来源二：三个高质量的“口袋”——Wikipedia、GitHub、arXiv

- **网络不是均匀的**：不是随机抽一部分网站就成了数据集，有些角落质量特别高，讲者挑了三个。
- **Wikipedia**（2001 年起，各语言共 6,700 万篇）：不允许原创研究，一切都要有引用，所以理论上“不含网上没有的东西”——但它引用书，书拿不到，所以 Wikipedia 依然极好；条目有 notability 门槛。谁都能编辑，破坏会被管理员或机器人回滚；和任何 peer production 系统一样，少数人做了大部分工作（某位编辑有 500 万次编辑）。每隔几周做一次 dump 供整包下载——别去爬 Wikipedia，他们也不希望你爬。
 - **dump 投毒**（Carlini 等，38:46）：以为破坏总会被回滚就安全？dump 有固定节奏，在 dump 前一刻编辑，编辑随后被回滚，但 dump 里已经带上了恶意内容。能往训练页面注入内容，就能让模型对某个触发词（比如 iPhone）产生负面情感。教训：一旦考虑对手，所谓高质量来源也可能有毒。讲者说此事后来已修复。
- **GitHub**（2008 年起，和 Common Crawl 同龄；4.2 亿个仓库，2,800 万公开）：代码不只是为了写代码的能力，也为了通用推理。一个仓库不是一个文件，是带提交历史、issue、PR、评论的目录；代码重复极多，因为复制和 fork。GitHub 认可在许可宽松（MIT、Apache）的公开仓库上训练。两类数据：仓库本身走 git 协议下载，

别去抓网页；元数据（issue、PR 等）来自 GH Archive——逐小时快照的事件流，每一条评论、star、动作都有记录。Software Heritage 专注仓库而非元数据，把 GitHub、GitLab、Bitbucket 等汇聚在一起。

- **arXiv** (1991 年起，从物理开始；300 万篇投稿，每篇有元数据、PDF、可选的 LaTeX 源)："在 arXiv 上训练"是什么意思并不显然——PDF 要转文本，或者用一堆 LaTeX 文件。不是同行评审但有审核；作者可以保留权利或选 CC，所以许可非常清楚：元数据一律宽松许可，论文挑 CC 的下载（讲者说多数是 CC）；同样是批量下载，不爬。
- **问答 (43:30)**：模型生成的数据有没有限制？短答案：大概可以，第 11 节和下一讲再谈。爬取里混进盗版书站怎么办？看不完所有网站，保证不了——Common Crawl 里多半有不该训的受版权的书和内容，只能诉诸 fair use；书和网页在版权上没有本质区别，都受保护，只是出版了的书作者上法庭更容易维权。要真谨慎，有一条路，第 11 节讲 (Common Pile)。

6. 数据集谱系 I (2018–2020)：BERT、WebText、CCNet、C4、GPT-3



- **BERT (2018)**：Wikipedia 加 books。books 来自 Smashwords——谁都能上传出版电子书的网站，2024 年约 50 万本；2015 年一篇论文把免费的那些抓下来做成语料，那是“没人在意”的年代。这份 BooksCorpus 在学界流传多年，后因违反 ToS 下架——免费能拿到，不等于合法能用。BERT 的另一点：序列是文档而不是句子，此前的语言建模研究都以句子为单位。
- **GPT-2 的 WebText (2019, 47:06)**：想要高质量网页，Common Crawl 太脏。点子很巧：取 Reddit 上 karma 大于 3 的帖子的外链——好帖子必然链向好网站。得到 40 GB 文本。从未发布，但有开源复现 OpenWebText，用得很多。

小注：GPT-2 报告的数字是 4,500 万条链接、约 800 万篇文档、40 GB；字幕里的“一百万页”应是听错。
- **CCNet (当时的 Facebook)**：目标是为预训练造大而高质量的数据集，尤其关心低资源语言，所以不要只对英语有效的手工流程。做了去重、语言识别（训一个分类器只留目标语言）；质量过滤的想法是留

下"像 Wikipedia"的文档——在 Wikipedia 上训一个语言模型，给新文档打分：

$$\text{PPL}_{\text{wiki}}(d) = \exp \left(- \frac{1}{|d|} \sum_{t=1}^{|d|} \log p_{\text{wiki}}(x_t \mid x_{<t}) \right)$$

d 是一篇文档， x_t 是它的第 t 个 token， p_{wiki} 是只在 Wikipedia 上训的语言模型（应为 n -gram 模型，推断）；困惑度越低越像 Wikipedia，按它排序留头部。这样筛出的数据量远大于 Wikipedia 本身，效果也超过只在 Wikipedia 上训练；这个工具后来在别的论文里反复出现。

- **C4** (Google, 2019, 49:40)：出自 T5 论文——那篇更出名的是"把所有 NLP 任务都当 text-to-text"，但这份很长的报告里很大一块贡献是 C4。当时的共识：Common Crawl 对自然语言基本没用，直接训就是垃圾；问题是怎么从这团糟里挖一个又大又干净的子集。C4 的路线是定一堆规则，结果相当有效——留以标点结尾、超过五个词的行；删少于三句的页面；删含脏词的页面；删 terms of use 之类的样板；删含花括号的页面（顺手滤掉了代码，可见当时没想过代码模型）；只留英文。

```
def c4_keep(page):
    lines = [l for l in page.text.split("\n")
              if l.rstrip().endswith((".", "!", "?", "'")) and
              len(l.split()) >= 5]
    if len(lines) < 3:                                # 页面太短
        return None
    doc = "\n".join(lines)
    if "{" in doc or has_bad_words(doc) or "terms of use" in
doc.lower():
        return None                                # 花括号顺手滤
掉了代码
    if lang_id(doc) != "en":
        return None
    return doc
```

规则派的全部家当就这几行 if；它便宜、可解释、可控，代价是"好文本"被定义成了"长得像英文散文的文本"。

- C4 的结果是 156B token、800 GB，远大于 GPT-2 的 40 GB。后来的分析 (51:11) 看了 C4 里有哪些网站：Wikipedia 不少，专利站点占比很高。同一篇论文还做了一个 WebText 式的数据集（同样用 Reddit 上 karma 大于 3 的外链）：用 12 个 Common Crawl dump

只得到 17 GB——WebText 是 40 GB，说明 Common Crawl 并不完整；用这些数据在了一批 NLP 基准上有提升。

- **GPT-3 (52:43)**：Common Crawl 自己处理，加上扩充版的 WebText、Books1 和 Books2（论文只说"基于互联网的书籍语料"，至今是谜）、Wikipedia；约 500 GB、400B token。处理 Common Crawl 时训了一个质量分类器，区分"他们认为的高质量"和其余；WebText 和 Common Crawl 有重叠，做了模糊去重。

小注：GPT-3 论文的表里各来源合计约 500B token（过滤后的 Common Crawl 约 410B），实际训练用了 300B token——CME295 第 4 讲说的就是这个 300B；课上的"400B"应是指 Common Crawl 那一项。

- **到这里已有四种"高质量"的定义**：规则（C4）、像 Wikipedia（CCNet）、Reddit 外链（WebText）、自训分类器（GPT-3）。后面的每个数据集都是在这四种里选一种或混着用。

7. 数据集谱系 II（2020—2023）：The Pile、Gopher、Llama 1、RedPajama

- **The Pile**（EleutherAI, 54:17）：GPT-3 之后的开源运动之一，草根项目——一群人在 Discord 上凑高质量来源，凑出一份至今仍有意思、很多样的清单：Pile-CC、PubMed、Books3、arXiv、GitHub、Wikipedia、IRC 聊天记录、另一份书籍语料、哲学论文、Enron 邮件……Enron 2002 年破产，邮件被公开，成了少有的邮件数据集——对"邮件"来说分布很怪，但只有这个。
 - **书**：Project Gutenberg 1971 年起，只收有版权许可的书，基本是 public domain；打包成 PG-19（2019 年前的 Gutenberg），既然要等 75 年才进 public domain，2019 年前的差不多就是全部。**Books3 (55:47)**：来自影子图书馆 Bibliotik 的 20 万本书，"你喜欢的作者都在"；2020 年没人在意，大家拿来训模型；现在已下架，不能再用、也不该再用。
 - **Stack Exchange**：2008 年起的用户问答。格式本身就接近真实应用——预训练数据不都是 Wikipedia 那种"原始文本"，网上本来就存在长得像监督数据的东西，也许正是这类数据教会了模型某些问答行为，"不是什么都神奇地涌现"。元数据（投票数）可用于过滤；同样是 dump 下载，不用爬。
- **Gopher**（DeepMind, 2021, 57:49）：模型没发布，后来被 Chinchilla 取代（第 9 讲），但数据描述写得很好，值得读——除了不

告诉你数据里有什么的那部分。自建 MassiveWeb，加 C4、书、新闻、GitHub、Wikipedia（后几样怎么来的没说）。MassiveWeb 只留英文、去重，质量过滤用手工规则，理由是可控。到这里"规则派"和"分类器派"的分野已经很清楚。文本量巨大，模型只训了其中很小一部分。

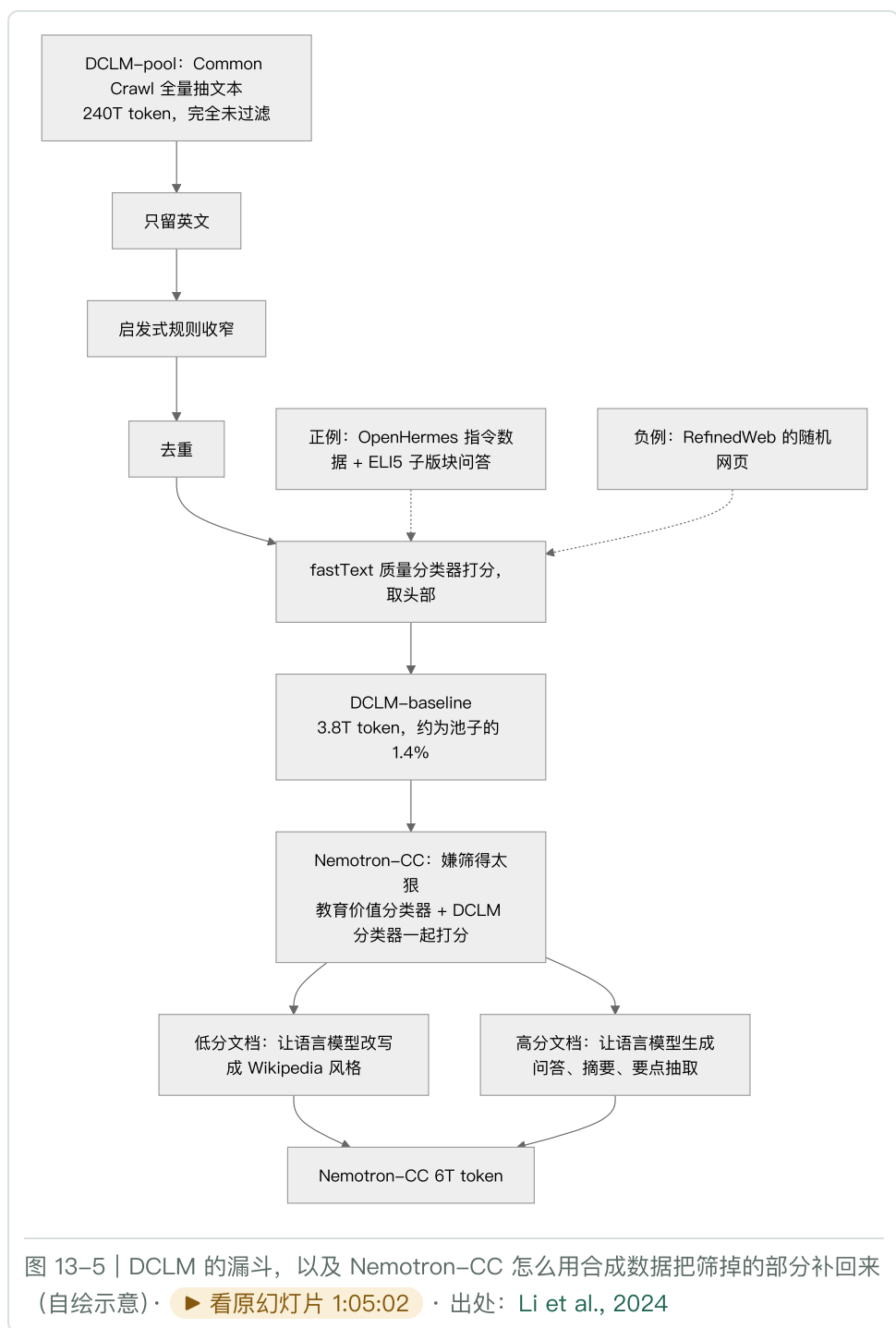
- **Llama 1 (59:51)**：论文好在把数据处理写清楚了——大概是最后一个非开源却谈数据的模型。Common Crawl 用 CCNet 处理，但分类目标换成"页面是否被 Wikipedia 引用"，而不是"像不像 Wikipedia"——Wikipedia 文章可能太程式化，它引用的那些页面想必是好的；加 C4；GitHub 只取宽松许可；Wikipedia；Books3 和 Project Gutenberg；arXiv 用 LaTeX 源处理；Stack Exchange。合计 1.2T token。Books3 让他们惹了大麻烦——向全世界宣布"我用了这个"，一查来自 The Pile，再查来自影子图书馆，这就是后来没人再谈数据的原因。数据集没发布，但描述足够详细，Together 的 RedPajama v1 复现了它；RedPajama 起初也含 Books3，后来剔除——早年一个版权决定，影响是分水岭式的。

小注：Llama 1 论文发表于 2023 年 2 月；讲者口误说 2022。

8. "网络就够了"：RefinedWeb → FineWeb → Dolma

- **RefinedWeb (1:01:53)**：立论是"web data is all you need"——网络本身就是一切，何必再要 GitHub、arXiv、Stack Exchange 那些专项来源？做法：把 HTML 转文本做好；用 Gopher 规则过滤（基本是"留下像英文的"）；明确说避免 ML 过滤，怕引入偏差、把网络筛成过窄的子集；去重。5T token，发布了约 600B。
- **FineWeb (Hugging Face)**：复现 RefinedWeb 并改进——当时所有 Common Crawl dump 全用；仍用手工规则（同样为了不注入偏差）；去重；去 PII；15T token。数据集尺寸在快速增长。
- **Dolma (AI2, 1:03:26)**：自己处理的 Common Crawl，加 The Stack（第 10 节）、C4、Reddit（来自 Pushshift 项目——那时还拿得到，后来锁上了），以及 AI2 自家爬的学术论文库 Semantic Scholar 派生的数据集。Common Crawl 的处理：语言识别用模型，质量过滤仍回避模型，毒性用规则加分类器。3T token。
- 三者的共同点：规则派的顶点——数据越做越大（5T → 15T），但"什么是好文本"仍由人手写的规则定义，理由都是怕分类器把网络筛歪。

9. 模型打分成为常态：DCLM 与 Nemotron-CC



- **DCLM (DataComp-LM) 是转折点：**模型打分的质量过滤从此成了主流。初衷是定义一条标准流水线，让人用统一的方式比较各种数据方法；实际上大家主要是拿它发布的数据集训模型。
- **漏斗（图 13-5）：**把 Common Crawl 全量抽成 DCLM-pool，240T token，完全未过滤——比任何人实际训练的 token 都多，但大多低质。然后只留英文、规则收窄、去重、模型过滤，最后剩 1.4%，也就是 3.8T token，效果相当好。

- 分类器的训练"有点怪但有效" (1:06:05)：正例是 OpenHermes（GPT-4 生成的指令数据）和 ELI5（Reddit 上"像我五岁一样解释"子板块的问答）；负例随机取自 RefinedWeb——"基本就是网络"；训一个 fastText，也就是线性分类器：

$$s(d) = \sigma(w^\top \phi(d) + b), \quad \text{keep } d \iff s(d) \geq \tau$$

$\phi(d)$ 是文档 d 的词与 n -gram 词袋向量， w 、 b 是学出来的权重和偏置， σ 是 sigmoid， $s(d)$ 就是"像正例"的概率， τ 是保留阈值；模型简单到极点，关键全在正例的选择——它在替你定义什么叫高质量。

```
pos = load("OpenHermes-2.5") + load("r/explainlikeimfive") # 想要的
样子：指令回答 + 通俗问答
neg = sample(refinedweb, n=len(pos)) # 不想要的
样子：普通网页
clf = fasttext.train_supervised(labeled(pos, "good") + labeled(neg,
"bad"))
scores = [clf.predict(doc)["good"] for doc in
dedup(rules_filter(english(pool)))]
keep = top_fraction(scores, tau) # 按分数取
头部，阈值是超参数
```

这个"神奇的"分类器胜过他们试的其他方法，一度成为开源社区质量过滤的金标准。

- 算一下：

$$\frac{3.8 \text{ T}}{240 \text{ T}} \approx 1.6\%$$

3.8T 是 DCLM-baseline 的 token 数，240T 是池子；和课上说的 1.4% 同一量级——每六七十个 token 才留一个，过滤才是数据流水线里最大的一刀。

- Nemotron-CC (NVIDIA, 1:07:39)：DCLM 筛得太狠，扔掉了绝大多数，只剩 3.8T，而我们要更多 token。办法更"讲究"：让现有的语言模型按"教育价值"给网页打分，造出标签，训一个 fastText 分类器；再加上 DCLM 的分类器；然后大举合成——讲者说这大概是最早真正把合成数据用于预训练的主要数据集之一：分类器判为低质的，让模型改写成更像 Wikipedia 的样子；高质的，让模型据此生成各种任务（给一篇 Wikipedia 文章生成问答、"请总结这份文档"、抽取关键信息）。6T token，比 DCLM 大不少；高质量子集打败此前的数据集。

小注：论文摘要写的是 6.3T token，应为约 4.4T 全局去重后的原文加约 1.9T 合成文本；讲者的"6T"是取整。

- **参照系**：Llama 3 训了 15T，Qwen 3 训了 36T。但"token 数"不一定是"独特 token"：论文里的数常包含重复——跑两个 epoch 就是两倍，看这些数要小心（第 9、11 讲的 scaling law 里 D 指的是训练 token，不是独特 token）。

$$D_{\text{reported}} = \sum_i e_i D_i^{\text{unique}}$$

D_i^{unique} 是第 i 个来源的独特 token 数， e_i 是它被重复的 epoch 数；论文报告的训练 token 是二者乘积之和，所以 36T 未必意味着见过 36T 不同的文本。

- **过滤方法一览**：都长得差不多——拿一份网络爬取，要么用规则，要么用模型；用模型就得先定义"什么是好数据"，训分类器，给所有文档打分再选。权衡：240T 的低质，还是 1T 的高质，甜点在中间的某处。

10. 代码：The Stack v1 → v2，以及怎么把 PR 变成序列

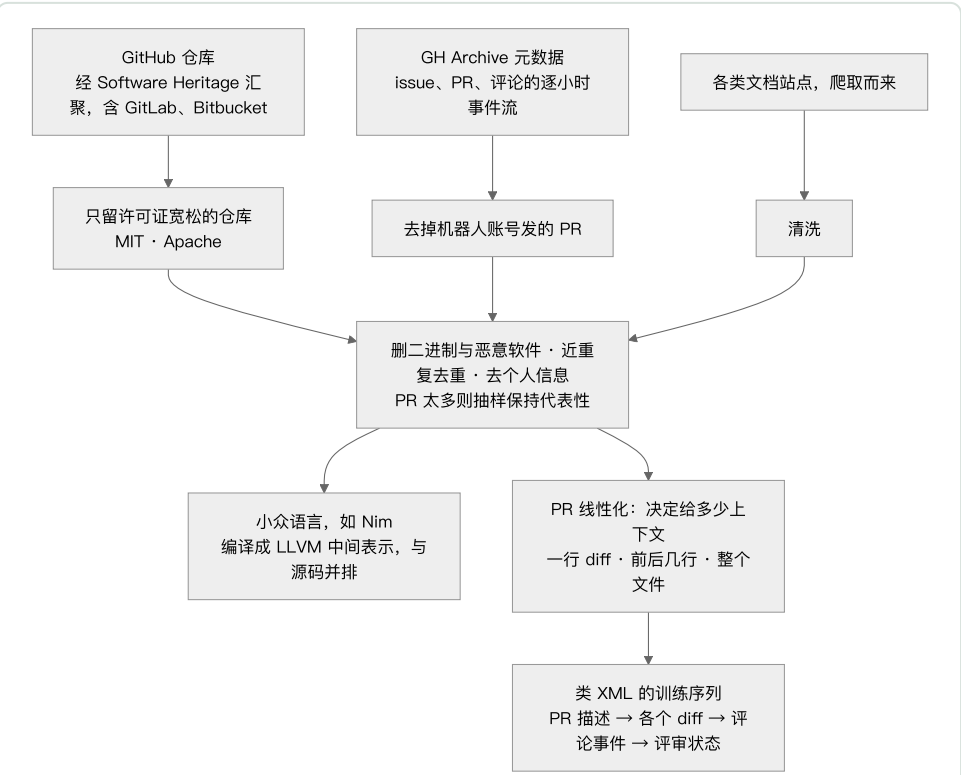


图 13-6 | The Stack v2 的三路来源、清洗步骤，以及把非线性的 PR 元数据压成训练序列的两个设计点（自绘示意） · [▶ 看原幻灯片 1:11:47](#) · 出处：Lozhkov et al., 2024

- **The Stack v1 (1:11:17)**: 到 2022 年已经清楚代码很重要，这个项目想做一份真正好的代码数据集。做法：克隆 1.37 亿个仓库，只留宽松许可的，去近重复，得到 3 TB 代码。

小注：The Stack 论文摘要写的是 3.1 TB、30 种编程语言。

- **v2 (2024, 图 13-6)**: 加了元数据 (issue、评论、PR, 来自 GH Archive)、Software Heritage 的仓库、爬取的各类文档。处理：删二进制文件、删恶意软件、过滤机器人 (尤其 PR 里大量是机器人发的)、去重、去 PII; PR 太多就抽样, 保持代表性和可管理的规模。一个巧点子: 编程语言也有长尾——Python、C 很多, Nim 这种小众语言很少; 把代码编译成 LLVM 中间表示 (C 编译器等都能出这个), 把小众语言的源码和它的 IR 并排放, 让模型学到"共享的低层表示 (数据多)"和"小众语言 (数据少)"之间的映射。v2 主要是代码, 再加了些别的, 用来训他们的代码模型。
- **PR 怎么变成序列 (1:14:19)**: PR 和它的元数据天然不是线性序列, 要设计线性化。第一个决定是给多少上下文——一个事件可能只改了一行, 但要学会它, 可能需要前后几行、甚至整个文件。最终的训练 token 长成类 XML 的结构: 先是 PR, 然后一串 diff, 评论则以事件的形式出现 (评论发布、评审状态变化)。模型学的不只是写代码, 还有代码周围的软件开发过程。

11. 只用许可数据能走多远: Common Pile

- **态度**: 几乎一切都有版权, 只有一部分是宽松许可或 public domain, fair use 又没定论; 极度厌恶风险的人会说"不确定就是不行"。Common Pile 就是这个态度的产物: 只用许可数据能走多远?
- **翻遍互联网找许可数据 (1:15:53)**: The Stack v2 的代码; 政府会议记录 (很多是宽松许可); wiki; 一些网页; 一些新闻站; 学术论文; 在线论坛; public domain 的作品; 教育资源.....共 8 TB, 对许可数据来说相当可观。
- **比看上去难得多**: 不是看一眼许可证写着 Apache 或 CC 就行。
 - **license laundering**: 人们对许可证很随意, 把受版权的东西贴个 CC BY 就放上网——谁都能这么写, 很难辨真假。
 - **集合的许可不覆盖单件**: Dolma 是宽松许可的集合, 但集合的许可不延伸到每一篇文档 (集合本身能否有版权、能否贴许可是另一回事)。Hugging Face 上很多数据集标着宽松许可, 往下挖到单件就不是了。

◦ **放弃合成数据**：用未许可数据训出的模型（哪怕权重是 MIT 开源）生成的数据，法律上不清楚——诚实地说有点 data laundering。这也回答了第 5 节的问答。

- **结果 (1:18:27)**：与 Llama 1、MPT、Qwen 等比较，一批基准上"不差"：肯定不如 Qwen，但胜过 2023 年那批老模型。讲者的结论：能做得不错，但不上更多 token 很难竞争；这不是终局，许可数据里应该还能榨出更多。

小注：论文模型叫 Comma v0.1，7B 参数，分别在 1T 和 2T token 上训练；和它比的 Qwen 是用几十 T token 训的，差距里有相当一部分是 token 数量的差距。

12. 小结与下一讲

- **数据不会从天上掉下来**，也不是上 Hugging Face 点下载就完事：网站给 dump 或有人建爬虫，才有原料；过滤、转换、去重，每一步都影响最终模型，其中过滤是最大的一刀——从 200T 到不到 3T (1:20:31)。
- **数据是区分语言模型的关键**：架构大同小异（第 3 讲），数据处理能拉开差距；法律与伦理问题比一讲能讲的多。
- **过程很乱，靠 vibes**：不像课程其他部分基于第一性原理，数据处理现在是定个分类器、定条规则、设个阈值，改进空间很大——做 assignment 4 时想想有没有更好的办法，也许就是研究方向。下一讲继续讲数据：后训练数据，以及更多的过滤。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Llama 3 论文的"数据"段	架构、训练流程写得很细，数据只有一句"多种来源"	0:36	Llama 3
OLMo 的三阶段数据表	预训练：网页、论文、数学；中期：高质量网页、指令、合成；后训练：聊天、数学推理代码、安全	3:41	OLMo
nytimes.com 的 robots.txt	一长串被禁的 AI 爬虫名；它不是法律，是约定	7:49	—
Consent in Crisis 两张图	上图 robots.txt 的红色"完全限制"在 2023 年中陡升到近 50%；下图 terms 从 2016 年几乎没有到现在多数都有	10:19	Longpre et al., 2024

图	看什么	跳转	出处
判例表	NYT 诉 OpenAI 待决；Anthropic 案训练 fair use 但盗版不行，15 亿美元和解；Meta 案训练 fair use，种子下载待决	27:15	—
Common Crawl 的数字	每月 30–50 亿页、累计 3,000 亿页（讲者存疑）、一次 dump 约 372 TB	32:34	Common Crawl
WARC 与 WET	WARC 是原始 HTTP 响应，WET 是有损的文本抽取	35:08	—
DCLM 的 HTML 转文本消融	WET 对比 Trafilatura、Resiliparse：抽取工具本身改变下游效果	35:39	Li et al., 2024
Wikipedia 投毒	在定期 dump 之前的窗口编辑，回滚来不及，dump 里已经带毒	38:46	Carlini et al., 2023
C4 的规则清单与网站分析	规则逐条：标点、词数、句数、脏词、样板、花括号、英文；后来的分析里专利站点占比很高	50:40	Raffel et al., 2019 · Dodge et al., 2021
The Pile 的来源清单	Pile–CC、PubMed、Books3、arXiv、GitHub、Wikipedia、IRC、哲学论文、Enron 邮件……	54:17	Gao et al., 2020
Llama 1 的数据表	各来源一览，Books3 赫然在列；合计 1.2T token	59:51	Touvron et al., 2023
DCLM 漏斗	240T → 英文 → 规则 → 去重 → 分类器 → 1.4%；分类器正例是 OpenHermes 加 ELI5	1:05:02	Li et al., 2024
Stack v2 的 PR 线性化样例	类 XML：PR、一串 diff、评论事件、评审状态	1:14:19	Lozhkov et al., 2024
Common Pile 的对比表	比 2023 年的 Llama 1、MPT 强，比 Qwen 弱	1:18:27	Kandpal et al., 2025

提到的工作

名称	在本讲里的作用
Llama 3 (2024)	"只谈架构不谈数据"的样本；训练 15T token 的参照系
OLMo (AI2)	每一阶段的数据都公开的模型，课上用它展示三阶段各喂什么
Qwen 3.5 397B · Qwen 3	前者是"不放 base model"的例子；后者训练 36T token 的参照系
Consent in Crisis (Longpre et al., 2024)	robots.txt 与 ToS 的纵向审计：限制在 2023 年后陡增

名称	在本讲里的作用
LibGen · Anna's Archive · Bibliotik	影子图书馆；Books3 的来源
Authors Guild 诉 Google	Google Books 片段展示，11 年后判为 fair use；训练是否 fair use 的先例
NYT 诉 OpenAI · Anthropic 案 · Meta 案	判例现状：训练被判 fair use，盗版不是；15 亿美元和解
Common Crawl	2007 年起的公共爬取，每月 30—50 亿页；WARC / WET 两种格式
Trafilatura · Resiliparse	HTML 转文本的工具；DCLM 消融里都优于自带的 WET
Poisoning Web-Scale Training Datasets (Carlini et al., 2023)	卡 Wikipedia dump 的时间窗口投毒
GH Archive · Software Heritage	GitHub 元数据的逐小时事件流；跨平台的代码仓库汇聚
BERT (2018) · BooksCorpus (Zhu et al., 2015)	Wikipedia 加 Smashwords 免费书；后者因 ToS 下架
GPT-2 (2019) · OpenWebText	WebText: Reddit 上 karma 大于 3 的外链，40 GB；开源复现
CCNet (Wenzek et al., 2019)	用 Wikipedia 语言模型给文档打分；去重与语言识别；后被 Llama 1 沿用
C4 / T5 (Raffel et al., 2019) · Documenting C4 (Dodge et al., 2021)	规则过滤的代表，156B token；后者分析了 C4 里的网站构成
GPT-3 (Brown et al., 2020)	自训质量分类器加模糊去重；Books1 / Books2 至今是谜
The Pile (Gao et al., 2020)	EleutherAI 的多来源拼盘；Books3 从这里流向 Llama 1
PG-19 (Rae et al., 2019)	Project Gutenberg 2019 年前书籍的打包
Gopher (Rae et al., 2021)	MassiveWeb 与手工规则过滤；数据描述写得很好
Llama 1 (Touvron et al., 2023)	最后一个详述数据的非开源模型，1.2T token；Books3 惹祸
RedPajama (Together, 2023)	Llama 1 数据的开源复现，后来剔除 Books3
RefinedWeb (Penedo et al., 2023)	"网络就够了": Gopher 规则、回避 ML 过滤，5T token
FineWeb (Penedo et al., 2024)	RefinedWeb 的复现与改进，15T token
Dolma (Soldaini et al., 2024)	AI2 的 3T token 数据集；Reddit 来自 Pushshift
DCLM (Li et al., 2024)	240T 池子筛到 1.4%；fastText 分类器成为质量过滤的金标准

名称	在本讲里的作用
OpenHermes · ELI5	DCLM 分类器的正例：GPT-4 生成的指令数据加通俗问答
fastText (Joulin et al., 2016)	DCLM 与 Nemotron-CC 用的线性文本分类器
Nemotron-CC (Su et al., 2024)	教育价值打分加合成改写，6T token
The Stack (Kocetkov et al., 2022) · The Stack v2 (Lozhkov et al., 2024)	许可代码 3 TB；v2 加元数据、LLVM IR 并排、PR 线性化
Common Pile (Kandpal et al., 2025)	只用许可数据的 8 TB；license laundering 与集合许可的坑
MPT (MosaicML)	Common Pile 对比表里 2023 年的老模型之一

术语对照

English	中文
pre-training / mid-training / post-training	预训练 / 中期训练 / 后训练：数据从大量低质到少量高质的三段
base model / instruct model	基座模型（预训练加中期训练之后） / 指令模型（后训练之后）
crawler / crawl	爬虫 / 一次爬取：从种子 URL 出发做图遍历、下载页面
seed set / frontier	种子 URL 集合 / 待爬队列
deep web	深网：点按钮、提交表单才能到的内容，超链接够不着
walled garden	围墙花园：登录或付费才能看的内容平台
robots.txt	站点根目录下声明"谁可以爬什么"的文件；约定而非法律
rate limit	限速：同一来源单位时间内的请求上限
terms of service (ToS)	服务条款：使用网站即接受的合同
intellectual property	知识产权：版权、专利、商标、商业秘密
copyright	版权：保护固定下来的原创表达，不保护想法
public domain	公有领域：保护期已过，人人可用
license / licensor / licensee	许可证 / 授权方 / 被授权方
Creative Commons (CC)	让作品像 public domain 一样流通的许可证
fair use	合理使用：无许可也可使用，按四要素权衡
transformative use	转化性使用：改变了原作的用途或性质
verbatim memorization	逐字记忆：侵权的一种，不是全部

English	中文
shadow library	影子图书馆：无视版权的盗版书库
piracy	盗版
data dump	数据快照：站点定期打包的完整下载，替代爬取
WARC / WAT / WET	原始 HTTP 响应存档 / 元数据 / 抽出的纯文本
HTML-to-text extraction	HTML 转文本：去导航、广告、样板，留正文
boilerplate	样板文本：条款、导航等重复出现的非正文
mirror site	镜像站：内容完全重复的站点
deduplication (exact / fuzzy / near-duplicate)	去重（精确 / 模糊 / 近重复）
language identification	语言识别
quality filter / quality classifier	质量过滤 / 质量分类器
heuristic rules	启发式规则：人手写的过滤条件
perplexity	困惑度：语言模型对一段文本的"意外程度"，越低越像训练分布
fastText	基于词与 n-gram 词袋的线性文本分类器
PII removal	去个人信息
toxicity filter	毒性过滤
synthetic data / rephrasing	合成数据 / 改写：让模型重写或生成训练文本
epoch	一遍：数据被重复训练的次数；报告的 token 数常含重复
karma	Reddit 帖子的净赞数
notability	显著性：Wikipedia 收录条目的门槛
peer production	同侪生产：Wikipedia 式的众包协作
data poisoning	数据投毒：往训练数据里注入恶意内容
permissive license	宽松许可证：MIT、Apache 等允许自由使用的许可
license laundering / data laundering	许可洗白 / 数据洗白：给受版权内容贴假许可；用未许可数据训的模型再生成数据
collection license	集合许可：整个数据集的许可，不延伸到单篇文档
LLVM IR	LLVM 中间表示：多种语言共享的低层代码表示
linearization	线性化：把 PR 这类非线性结构压成 token 序列
pull request (PR)	合并请求：代码改动加评审过程
risk-averse	厌恶风险：不确定就不用

字幕勘误

"Shane Lampray" → Shayne Longpre; "WAT file" (两处) → WET file (Common Crawl 的文本抽取格式, WAT 是元数据); "Brazilia Parser" → Resiliparse; "Data Comp at WLM / DCL M / DCLAM" → DataComp-LM (DCLM); "archive" → arXiv; "web text" → WebText; "Open or Hermes" → OpenHermes; "Eli5" → ELI5; "Nematron" → Nemotron-CC; "Gwen 3" → Qwen 3; "Doma" → Dolma; "MBT" → MPT; "Bibliotek" → Bibliotik; "137 new repos" → 137M repos (1.37 亿个仓库); "fear it" → fair use; "PI removal / PI reduction" → PII removal; "Red Pajama" → RedPajama; "Refined Web" → RefinedWeb; "fine-web" → FineWeb; "Olmo" → OLMo; "Libgen" → LibGen; "Books 3" → Books3; "Stack V2" → The Stack v2; "PushShift" → Pushshift; "11 labs" → ElevenLabs; "a million pages" (WebText) → 应为 8 million; "Software Heritage Foundation" → Software Heritage。

带走的问题

1. DCLM 的漏斗从 240T 到 3.8T, 每一级各扔掉多少? 分类器的正例 (OpenHermes 加 ELI5) 实际上在定义"高质量"——如果你的目标是代码或多语言, 正例该换成什么? 这和第 12 讲里"benchmark 的选择决定了你在优化什么"是不是同一种风险?
2. fair use 四要素里"对市场的影响"对语言模型最不利: 一个训练在 NYT 文章上的模型替代了读 NYT, 这条怎么权衡? Anthropic 案说"训练是 fair use、盗版不是", 那 Common Crawl 里混进来的盗版书, 按讲者的说法只能诉诸 fair use——它算哪一种?
3. Wikipedia 的 dump 投毒说明"高质量来源"也不安全。如果你在给 agent 建知识库或 RAG 语料, 哪些来源有类似的"定期快照"漏洞? 除了回滚, 还有什么机制能挡住卡时间窗口的攻击?
4. Nemotron-CC 用改写和生成把 3.8T 变成 6T, 其中"独特 token"有多少? 第 9、11 讲的 scaling law 用 D 计训练 token, 合成 token 和重复 epoch 该不该按同样的权重算进 D?
5. Common Pile 8 TB 只用许可数据, 效果落后于 Qwen——差距来自 token 数量还是数据分布? 如果许可数据是硬约束, 第 14 讲要讲的过滤、去重、混合和合成数据里, 哪一步最能把差距补回来, 又会不会踩到"data laundering"的线?

CS336 第 14 讲 | 数据（下）：过滤、去重、配比与合成数据（Data II）

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 14 讲, 数据部分的下半场 (第 13 讲讲数据从哪来、有哪些数据集; 这一讲讲拿到原始数据之后的流水线); 课程表标注 2026 年 5 月 13 日 视频:

<https://www.youtube.com/watch?v=5sxHosTLPF8> (1:24:46; 英文字幕为自动生成, 工具名、数据集名和算法名错得不少, 见文末勘误) 讲者: Percy Liang (全程; 讲到 μP 时说"Tatsu 讲过", epoch 实验出自他的学生 Michael Ryan, 数据源清单来自他主持的 Marin 项目) 课程主页: <https://stanford-cs336.github.io/> · 去重的算法主线出自 *Deduplicating Training Data Makes Language Models Better* (Lee et al., 2021); 配比部分以 *RegMix* (Liu et al., 2024) 和 *Olmix* (Chen et al., 2026) 为主; 合成数据以 *OpenThoughts* (Guha et al., 2025) 和 *SWE-smith* (Yang et al., 2025) 为例

一句话: 原始网页变成能训模型的 token 要过四道工序, 每道工序都由一个"想要什么"的定义和一个"怎么在 100T token 上跑得动"的算法组成——过滤是"给一小份目标数据 T 、一大池原料 R , 找出 R 里像 T 的子集", 做法就是拿 T 当正例、 R 的随机样本当负例训一个 fastText 线性分类器再逐篇打分 (GPT-3、LLaMA、OpenWebMath、phi-1 全是这个骨架的实例), 最后只留个位数百分比, 而且阈值没有最优值, 取决于你打算训多少 token; 去重要在线性时间里找到"几乎一样"的文档, 靠的是 MinHash 让哈希碰撞的概率恰好等于 Jaccard 相似度, 再用 LSH 的 b 个 band $\times$ r 个哈希把这个概率削成一个陡峭的 S 形开关 (阈值约为 $(1/b)^{(1/r)}$, 真实系统用 $b = 20$ 、 $r = 450$); 配比要给几十个数据源定权重, 最大的坑是 epoch——10T 低质 + 10B 高质各占一半、训 1T token, 高质源就被重复了 50 遍, 所以要么像 UniMax 那样给 epoch 数设上限, 要么用一群小模型拟合"权重 $\rightarrow$ loss"的回归再优化 (RegMix / Olmix), 并用 simulated epoching 让小规模也体会到大规模的数据稀缺; 后训练数据则几乎全是"环境 $\times$ 任务 $\times$ 教师模型"的合成流水线, OpenThoughts 的 120 万条、SWE-smith 的 5 万个任务、SWE-zero 的 30 万条无执行轨迹、当天刚发布的 1200 万条, 都是它的实例。

时间轴

时间	内容
0:05	开场：回顾第 13 讲（数据要 dump 或爬、要处理、有 ToS 和版权问题）；今天：转换 → 过滤 → 去重 → 配比，末尾讲后训练数据
1:09	转换回顾：HTML 线性化是有损的、规则式的；PDF 与 FinePDFs (4:14)：截断、重爬、OCR 贵，但平均质量高
6:51	过滤的抽象骨架：目标集 T、原料池 R，找 R 里像 T 的子集；要能泛化、要快到扫完 100T token；两类打分器（KenLM 生成式、fastText 判别式）
11:34	语言识别：fastText 的 176 种语言；OpenWebMath 的三步定向过滤：15B token 打赢 20 倍的数据
14:09	质量过滤实例：GPT-3、LLaMA、phi-1（GPT-4 标 100k 篇 → 训便宜的分类器）；毒性过滤：Jigsaw
17:12	没有万能阈值：训练 token 越多越能容忍低质数据；157M 模型在 100 个 WARC 上的 epoch 实验；问答：置信区间、更高质量数据训更久
22:47	去重：精确重复（镜像站、fork）与近似重复（许可证、页眉页脚、模板、标点差异）；C4 里出现 61,000 次的商品描述；为什么去重
27:21	去重的设计空间（粒度、匹配方式、留几份）； n^2 比较不可行 → 哈希；精确去重与 C4 的三句跨度
31:30	Jaccard 相似度；MinHash：碰撞概率恰好等于 Jaccard，特征矩阵 + 随机置换的证明；100 个哈希函数估出 0.6
37:41	LSH：b 个 band × r 个哈希的"或一与"结构；S 形曲线；加 r 右移变陡、加 b 左移；阈值 $(1/b)^{(1/r)}$ ；真实系统 b = 20、r = 450
49:06	数据配比：Marin 的数据源清单、The Pile 的权重；vibes / 均匀 / 按 token 数比例；两条约束：多样性、每个源是有限的
53:42	epoch 陷阱：10T + 10B、各一半、训 1T → 高质源 50 个 epoch；问答：配比在 batch 里怎么实现；UniMax 的 epoch 上限
59:51	回归式配比：小模型群 → 拟合"权重 → loss" → 优化 → 放大；RegMix；设计选择（Dirichlet、回归模型、目标指标）；Olmix 的方法总表
1:04:30	两次信仰之跃；规模依赖：小规模不 epoch 会高估高质源；两种修法：cap epoch、simulated epoching（和 μP 同一思想）
1:10:44	问答：下采样太小怎么办；一个数据集内部按 domain × quality 再切格子配比（WebOrganizer）
1:12:46	后训练数据的通用配方：环境 · 任务 · 教师；OpenThoughts 的 120 万条与四条发现
1:17:29	agentic coding 数据：SWE-smith 5 万个合成任务、SWE-zero 30 万条无执行轨迹、SWE-rebench、当天新论文的 1200 万条
1:23:46	总结：定义好数据 → 训轻量分类器；去重省 FLOPs 防过拟合；配比小规模试大规模用；数据工作又脏又具体

核心内容

1. 这一讲在流水线里的位置，以及"转换"为什么是有损的

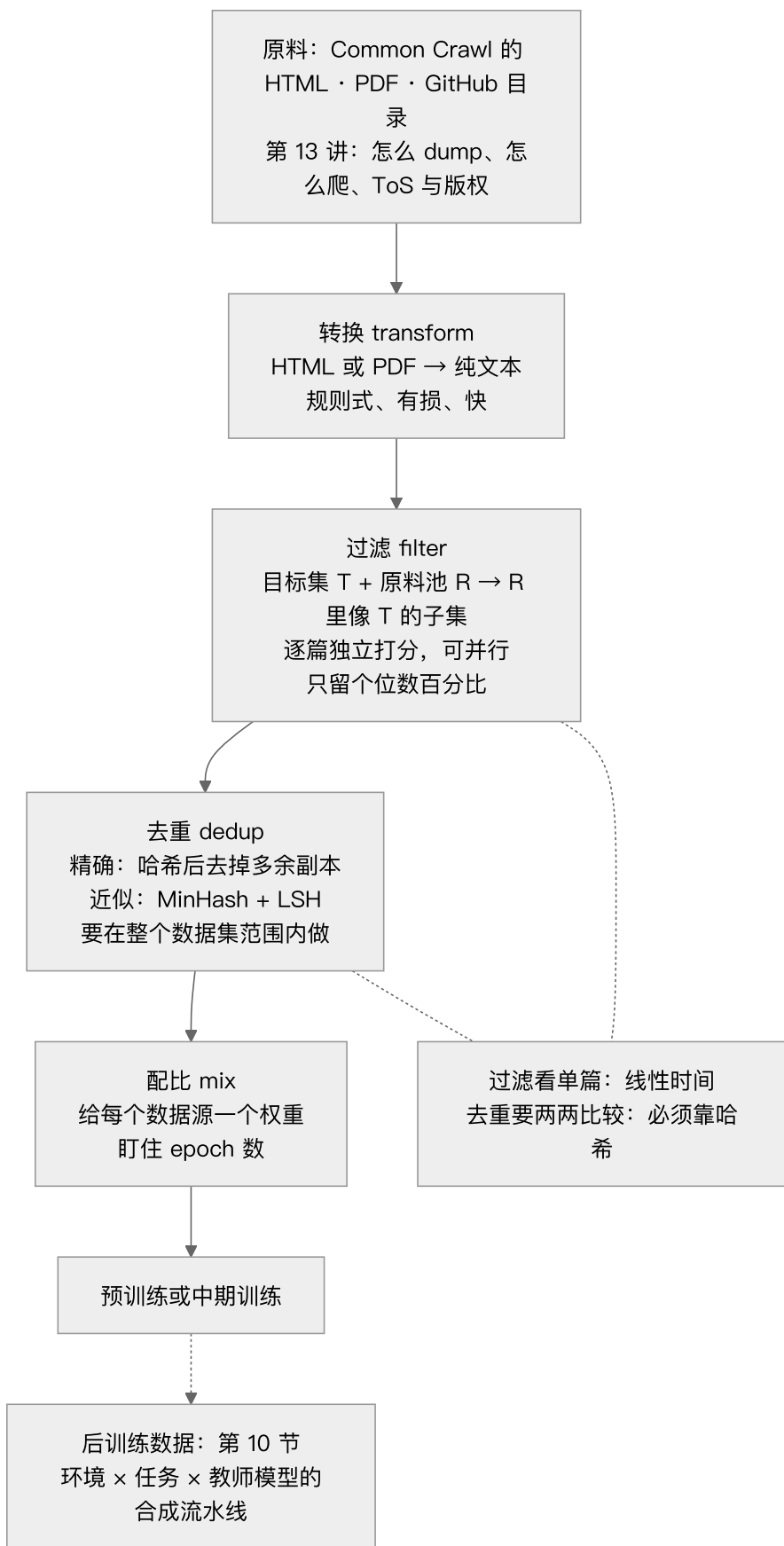
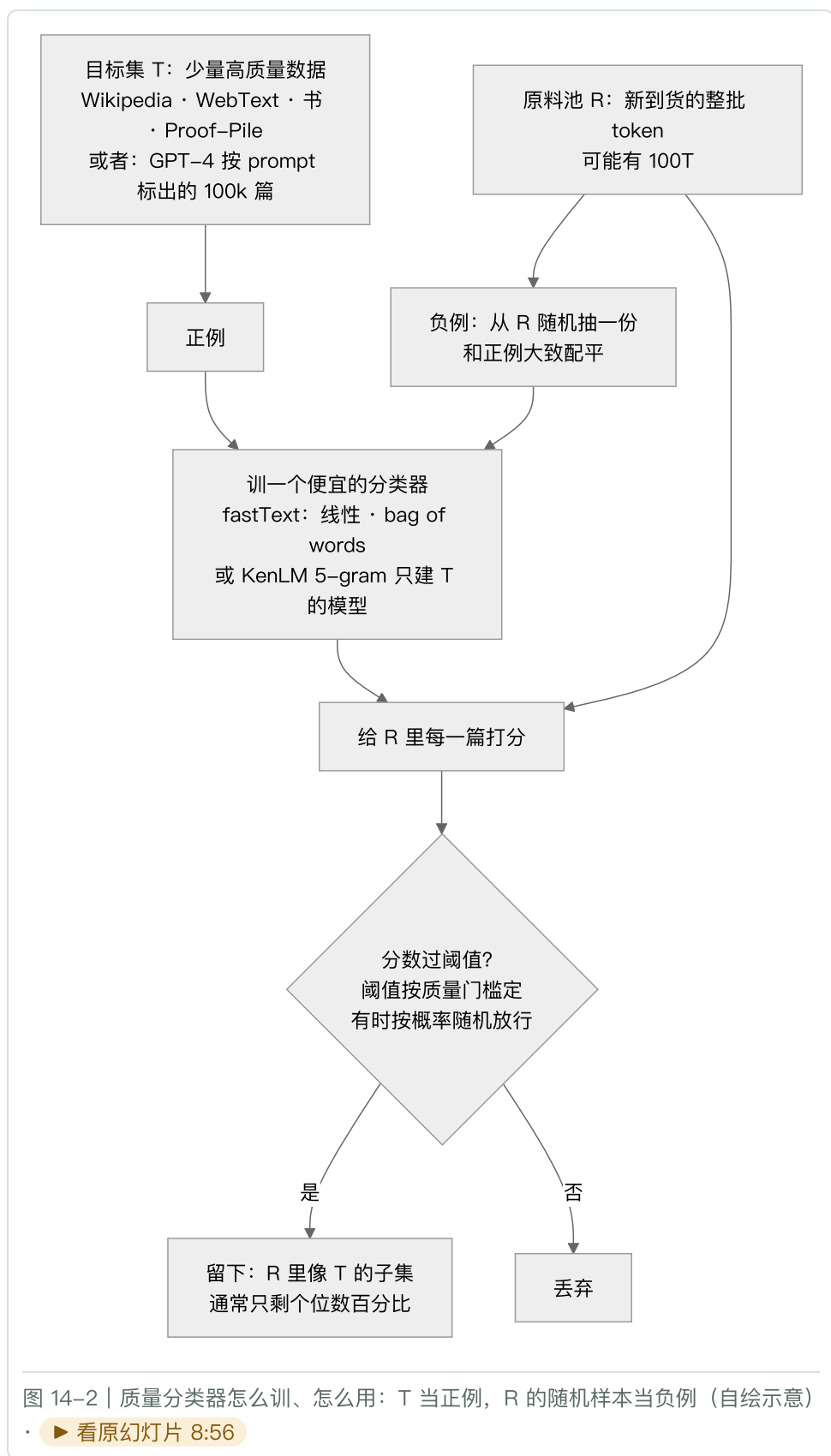


图 14-1 | 从原始抓取到可训练数据的四道工序，以及本讲各节对应的位置（自绘示意）· [▶ 看原幻灯片 0:37](#)

- **接着第 13 讲讲：**上一讲的结论是数据不会从天上掉下来——互联网是一堆在线服务，数据要么被 dump 出来要么被爬下来，之后还要处理，而且绕不开 terms of service 和版权（要么拿许可，要么诉诸 fair use）。这一讲从"手里已经有一批原始抓取"开始，走完转换、过滤、去重、配比四步，前四分之三讲预训练数据，最后一段看后训练数据现在怎么合成。
- **转换 (transformation)：**原始数据不是文本——Common Crawl 里是 HTML，有时是 PDF，GitHub 给的是目录。精力主要花在 HTML 上：去 boilerplate（导航、广告、页眉页脚、菜单）抽"正文"，可什么算正文并不总清楚（导航元素有时反而能教模型"网页长什么样"），图片和表格也没有定论。这一步本质上有损：层级的 HTML 要压成一维 token 序列，简单表格能转 markdown，嵌套表格只能放弃或近似。用规则而不用模型，是因为规则式处理器极快、这一步也不需要多少"智能"（Percy 认为模型式干预有空间，但必须同样快）；代价是任何规则都有失误率，工具选择也影响结果——第 13 讲比过 Resiliparse 和 Trafilatura，扩展 DCLM 评测上 Resiliparse 更好。
- **PDF**（Hugging Face 的 FinePDFs 工作）：Common Crawl 以文本为主，URL 没带扩展名时抓之前未必知道是 PDF，而且很多 PDF 因为太大被截断了，要重新爬；扫描件本质是图片，得用 VLM 做 OCR，比处理文本贵得多。值得做，因为 PDF 虽只占互联网很小一部分，**平均质量比 HTML 高**——肯费事做成 PDF，多半有东西要说；代价是清洗更重：HTML 有 H1、P 这类语义标签，PDF 天生只管版式，语义结构基本丢了。

小注：FinePDFs 是 Hugging Face FineWeb 团队 2025 年发布的 PDF 语料，托管在 [Hugging Face](#)；讲者说的"这篇博客里有一大堆细节"应指它的发布文章。

2. 过滤的通用骨架：目标集 T、原料池 R，和一个必须很快的打分器



- 一个骨架套所有过滤: 手里有一小份想要的目标数据 T (通常量小、质量高) 和一大池原料 R (上一步转换出来的新鲜 token), 目标是找 R 里和 T 相似的子集。语言识别 (只要英文或只要德文)、质量过滤

(要百科不要垃圾邮件，这是最主要的用途)、毒性过滤，都是这个形状。

- **两个硬要求**：一是要能泛化——T 你已经有了，过滤的意义是找到 T 之外的同类；二是要**极快**——它要跑遍整个互联网，量级可能是 100T token。过滤的典型结果是只留下个**位数百分比**。
- **怎么打分**：用 T 和 R 估一个模型，导出打分函数，按分数留人。两类做法：
 - **生成式**：只对 T 建模。必须便宜，所以不是训大语言模型，而是 KenLM 这种 5-gram 模型，用困惑度当分数。
 - **判别式**（现在的主流）：T 是正例，R 的随机子集是负例，配平后训分类器；工具几乎都是 fastText——一个 bag-of-words 的线性分类器，胜在快。然后给每篇新文档打分，按你的质量门槛定阈值，留下高分的，有时按分数随机放行而不是一刀切。

$$\hat{S} = \{x \in R : s_{T,R}(x) \geq \tau\}, \quad s_{T,R} = \text{fastText}(T \text{ 为正, sample}(R) \text{ 为负}) \text{ 或 } -\log \text{ppl}_{\text{KenLM}(T)}$$

R 是原料池，T 是目标集，s 是从 T 和 R 估出来的打分函数（判别式分类器的正类概率，或生成式模型下的困惑度取负）， τ 是阈值；过滤的输出 $\hat{S}$ 就是 R 里分数过线的那部分。

```
# 判别式质量过滤的骨架（示意）
pos = sample(target_T)                # 想要的数据：Wikipedia、
书、GPT-4 标过的 100k 篇.....
neg = sample(raw_R, n=len(pos))       # 从原料池随机抽同样多当负例
clf = fasttext.train_supervised(pos + neg) # 线性 bag-of-words，快到
能扫 100T token
kept = []
for doc in stream(raw_R):             # 逐篇独立打分：线性时间，可
并行
    p = clf.predict_proba(doc)         # 像 T 的概率
    if p > tau or random() < keep_prob(p): # 硬阈值，或按分数随机放行
        kept.append(doc)
```

这段展示的是骨架本身：正例来自 T、负例来自 R、分类器要便宜、打分逐篇独立；GPT-3、LLaMA、OpenWebMath、phi-1 只是换了 T 和分类器。

- **要不要用模型来过滤**：几年前的一些数据集（第 13 讲提过）刻意不用模型式过滤，怕引入偏差；现在基本人人都用——除非你算力充裕到可以什么都训，否则不聪明地过滤就是把 FLOPs 浪费在低质内容上。“算力穷人”是常态。

3. 骨架的实例：语言识别、数学、GPT-3 / LLaMA / phi-1、毒性

实例	目标集 T	原料池 R	打分器	结果
语言识别	多语言站点 (Wikipedia 各语言版、翻译站)	任何文本	Meta 训好的 fastText, 直接用, 支持 176 种语言	阈值按经验定
OpenWebMath (2023)	Proof-Pile (数学语料)	Common Crawl	规则 (有没有 LaTeX 命令) → KenLM 困惑度 → fastText"是不是数学写作"	15B token; 数学能力胜过用 20 倍未过滤数据训的模型
GPT-3	Wikipedia、WebText (高分 Reddit 帖子链出的页面)、书	网页	线性分类器, 分数够高才留	第 13 讲讲过, 这里放进骨架
LLaMA 1	被 Wikipedia 引用的页面 (不是 Wikipedia 文章本身)	网页	同上	同一思路
phi-1 (Microsoft)	GPT-4 用 prompt"判断教育价值"在 100k 篇上标出的正例	The Stack of Python 子集	随机森林 (用 fastText 也行)	比直接用原料训得更快、上限更高
毒性	Jigsaw toxic comments: Wikipedia 讨论页的人工标注	网页	同样的正负例分类器	过滤掉不想学的内容

- **语言识别**：Meta 的 fastText 语言识别模型现成可用，176 种语言，在 Wikipedia 这类多语言站点上训的。这是相对容易的问题——看几个词就知道是西班牙语还是日语；code switching 和方言有麻烦，但不是训好模型的瓶颈。
- **OpenWebMath**：想让模型擅长数学，就去找数学文本。三步：规则看有没有 LaTeX 命令；在 Proof-Pile 上训的 KenLM 算困惑度，低于阈值才留（生成式那一路）；再训 fastText 判断是不是数学写作。两套阈值：**有 LaTeX 的门槛低，没有的门槛高**。得到 15B token，用它训的模型比用 20 倍普通数据训的更会数学。质量没有统一定义——定义成数学，就得到数学。

小注：论文摘要的数字是 14.7B token，"超过用 20 倍以上通用语言数据训的模型" (Paster et al., 2023)。

- **phi-1 的两级结构**：T 不是现成数据集，而是一个贵的分类器的输出——写一条 prompt（判断教育价值），让 GPT-4 在 R 的 100k 篇子

集上打标，标为正的就是 T；再用 T 训便宜的分类器（他们用随机森林）扫全量。结果曲线里，这份数据"更少步数就到更高的值"。GPT-3 和 LLaMA 的做法第 13 讲讲过，重点是它们落在同一个骨架里。

- **毒性过滤**：Jigsaw 的数据来自"让网上讨论更好"的项目，标注的是 Wikipedia 讨论页（争议话题下吵得很凶），同样定义正负例、训分类器。
- **到此为止的配方**：认准你想要的数据——要么"有个现成数据集我喜欢，想要更多"，要么写条 prompt 让语言模型先粗筛一批——训一个小分类器，去 Common Crawl 里捞。

4. 阈值没有最优值：质量门槛取决于你打算训多少 token

- **一个容易忽略的事实**：分类器给的是连续分数，没法说"0.9 最好"——想要什么数据，取决于要训什么模型，尤其是训练 token 数。直觉是：训得越久，越能容忍低质量数据；训得短，就要高质量。当然最好是"训得久 + 全是高质量"，但数据池就那么大，这不是你能选的。
- **Michael Ryan 的初步实验 (18:12)**：157M 参数的模型，数据池故意取得很小——100 个 WARC，只是 Common Crawl 的零头——然后越训越久，看 loss。
 - 蓝线是 DCLM 那套过滤后的数据：loss 从高处一路下来；图上每条竖线是一个 epoch，数据不多，只能反复过。第二遍还在学（loss 还在降），再往后开始过拟合。
 - 另一条是 Resiliparse 直出、基本没过滤的数据：开头差很多，但训得久、也 epoch 之后，下降只是变慢。
 - **趋势**：不 epoch 的区间高质量数据更好；token 很多之后，高质量数据就不再那么占优。而且高质量那条你根本不想训到那个区间——已经过拟合了，会在拐点早停；可早停的那个点，比"拿低质数据训更久"还差。

小注：WARC (Web ARChive) 是 Common Crawl 发布抓取结果的归档格式，一个月的抓取就有几万个这样的文件，100 个只是零头；字幕把它识别成了"works"。

- **问答**
 - 图上每个点是一次训练 run，要不要重复跑、报置信区间？理想上应该；论文里少见，因为每个 run 都贵。实际经验：预训练这类实验结果相当稳定。

- 如果能拿到更多高质量数据并训得更久，会不会同样边际递减？会——任何数据集都是有限的，终究递减；但整条曲线会在更低的位置一直往下。
- **小结：**对算力穷人（也就是绝大多数人）来说过滤至关重要；算力无限才可以什么都训。配方就是：搞清好数据长什么样 → 训分类器 → 外推到全部原料。

5. 去重：重复从哪来、为什么去、设计空间在哪

- **两类重复**（过滤完仍然大量存在）
 - **精确重复：**镜像站——镜像的意义就是复制，而爬虫未必知道这是镜像，全都爬一遍；fork 的代码仓库——就算改了几个文件，99% 还是一样的。
 - **近似重复：**不是镜像也不是衍生，只是差几个 token 的相同文本，多半来自复制或共同来源。例子：terms of service 和许可证（MIT license 到处都有，除非抄的人打错字，那一段是精确复制，但它所在页面的其余部分各不相同）；网站共用的页眉页脚；LM1B 里同一篇文章的两个版本只差一个逗号；模板化内容——像一条广告，把 Canada 换成 USA 又发一遍。拿这些变体训练是在浪费 GPU。
 - **极端案例：**对 C4 的一次审计发现，同一段商品描述（一个防毒面具）在数据集里出现了 61,000 次，追溯不出为什么。网络很怪——所以要看数据。

小注：这个例子和第 7 节的 LSH 参数都出自 [Lee et al., 2021](#)。摘要的说法是 C4 中有一个 61 个词的英文句子重复了 6 万多次；去重之后，模型吐出记忆文本的频率降到十分之一，用更少的训练步数达到同样或更好的准确率，而且标准数据集验证集里 4% 以上的内容与训练集重叠。

- **为什么去重：**一是训练更高效——去重缩小数据集，却几乎不丢信息；二是避免记忆——版权内容重复太多就会被背下来，还有隐私问题；主要还是别把 FLOPs 浪费在重复上。同一套工具还用于去污染（decontamination）：确保测试集不在训练集里，这一点可以说更重要。
- **设计空间**
 - 单位：句子、段落，还是文档？
 - 匹配：精确匹配？存在共同的子项？共同子项的比例（这就是近似去重）？

- 找到之后：全删，还是只留一份？
- **算法上的难点**：过滤是逐篇判断“这篇好不好”，线性时间、可并行，即便如此还要靠规则和小模型提速；去重却是**项目和项目之间的比较**，不能做 n^2 的全比较，网络规模下必须是线性时间算法——出路是哈希。
- **哈希函数**：把一个值（比如字符串）映成小得多的值；两个不同的项映到同一个值叫碰撞。密码学哈希抗碰撞（比特币在用），哈希表用的那类更快、碰撞不是世界末日——这里用后者。
- **精确去重**：把字符串哈希，找完全匹配，删到只剩一份；写成 MapReduce 风格就好并行。C4（T5 论文）的单位是**三句话的跨度**：找到两篇文档共享三句话，就从其中一篇里挖掉这三句——连贯性被破坏了，但他们就是这么做的。精确去重不够用，因为网络数据的重复大多是近似的。

6. 近似去重的度量与哈希：Jaccard 和 MinHash

- **先定义“近似匹配”**：Jaccard 相似度 = 交集大小 / 并集大小。例：{1, 2, 3, 4} 和 {1, 2, 3, 5}，交集 3 个、并集 5 个，Jaccard = 0.6。取值在 0（不相交）到 1（相同）之间。于是可以定义：Jaccard 超过某个阈值（比如 0.99）就算近似重复。

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad h_{\min}(A) = \min_{a \in A} h(a), \quad \Pr_h[h_{\min}(A) = h_{\min}(B)] = J(A, B)$$

A、B 是两篇文档各自的元素集合（比如 n-gram 集合），J 是它们的 Jaccard 相似度；h 是一个随机哈希函数， $h_{\min}(A)$ 是对 A 里每个元素哈希后取最小值，即 MinHash；第三个式子是 MinHash 的核心性质——在随机选 h 的意义下，两个集合 MinHash 相等的概率恰好等于它们的 Jaccard。

- **怎么在线性时间里找**：这是算法界已经解决的问题，第一步是 **MinHash**（不是最终答案）。它是一个随机哈希函数，使得碰撞概率恰好等于 Jaccard(A, B)。妙处在于：哈希让事情线性，Jaccard 是我们需要的度量，MinHash 把两者接上了（期望意义下）。反常之处也在这里——通常设计哈希函数是要避免碰撞，这里要的却是**受控的碰撞**：相似的东西比不相似的更容易撞在一起。
- **做法**：对集合里每个元素做哈希，取最小值。取最大值也行，只是一种打破平局的约定。

- **为什么成立**（讲者的图景）：把两个集合画成特征矩阵——行是所有元素 1 到 5，列是 A 和 B。一个随机哈希函数等价于对行做一次随机置换（比如 4, 3, 1, 5, 2）；MinHash 相等，当且仅当按这个顺序排最前的那个元素同时属于 A 和 B。每个元素被排到最前的概率相同：排最前的是 1、2、3 之一，两边相同；是 4 或 5，两边不同。所以概率就是交集除以并集。
- **代码验证**：生成 100 个不同的哈希函数（每个由一个 seed 决定），数 MinHash 相等的比例，得到约 0.6。
- **关键收益**：不用 n^2 。算出每个集合自己的 MinHash，然后只看谁和谁碰撞。

7. LSH：把"碰撞概率等于相似度"削成一个开关

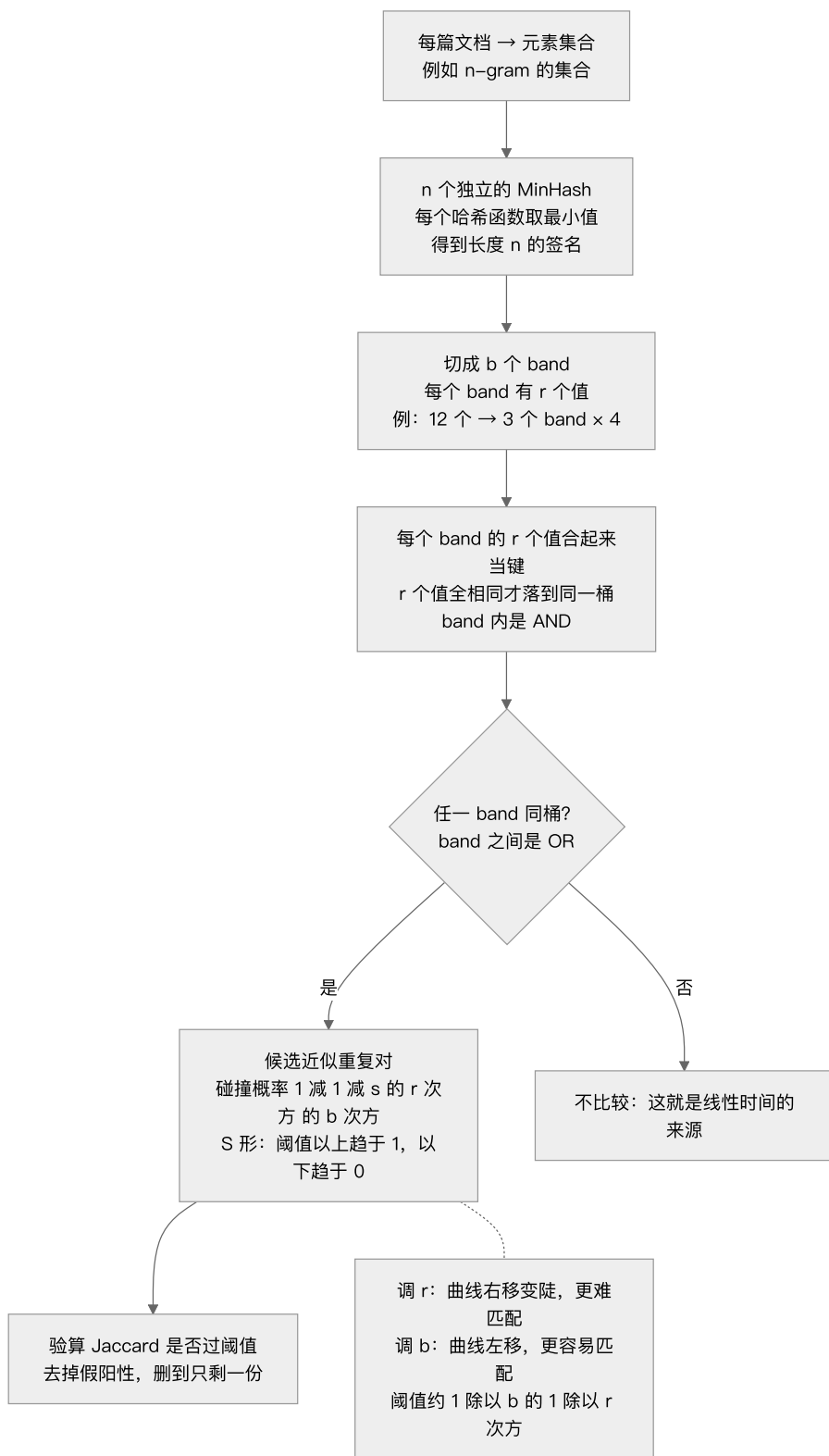


图 14-3 | MinHash 签名加 LSH 分 band 怎样在线性时间里找到近似重复 (自绘示意) · [▶ 看原幻灯片 39:12](#) · 出处: Lee et al., 2021

- **MinHash 还不够:** 我们要的是"找出 Jaccard 大于 0.99 的 A、B", 可单次碰撞只是一个概率等于 Jaccard 的随机事件, 方差太大。得把概率削尖: 高于阈值几乎必碰, 低于几乎不碰。

- **LSH** (locality sensitive hashing, 局部敏感哈希) 是理论计算机科学的经典想法：用更多**独立**的哈希函数，按"或—与"结构组合—— n 个哈希函数分成 b 个 band、每个 r 个 (12 个 $\rightarrow 3$ 个 band $\times 4$)；一个 band 里 r 个哈希**全部**相同才算触发 (AND)，**任何一个** band 触发就判定碰撞 (OR)。

$$\Pr[\text{某个固定 band 全部匹配}] = s^r, \quad \Pr[A \text{ 与 } B \text{ 碰撞}] = 1 - (1 - s^r)^b$$

s 是 A、B 的 Jaccard 相似度， r 是每个 band 里的哈希函数数， b 是 band 数；一个 band 要 r 个哈希全撞上，概率是 s 的 r 次方，指数级地小；"没有任何 band 撞上"的概率是 $(1 - s^r)^b$ ，取补就是至少一个 band 撞上的概率。

- **算一个**： $s = 0.8$ 、 $b = 5$ 、 $r = 10$ ——固定 band 匹配的概率是 0.8 的 10 次方，很低；碰撞概率约 0.4 ，比单个 band 高，因为有 b 次机会。

小注：代进公式： $0.8^{10} \approx 0.107$ ， $1 - (1 - 0.107)^5 \approx 0.43$ ，与讲者说的"零点四几"一致。

- **画出来是 S 形**：横轴相似度，纵轴碰撞概率， 0 对 0 、 1 对 1 ，中间一个相变——阈值以下尽量低，以上尽量高，正是要的。
- **读表** (相似度 0.7 到 0.98)：
 - $b = 10$ 、 $r = 10$ ：碰撞概率从 0.25 到 1 。阈值设在 0.9 的话，低于的仍有假阳性，可以事后验算 Jaccard 过滤掉；高于的基本都留下。
 - 加大 r (每个 band 里的哈希数)：曲线变陡并**右移**，什么都更难匹配—— 0.7 处从 0.25 掉到 0.008 ， 0.9 处只剩 0.72 。
 - 加大 b (band 数)：曲线**左移**，更容易匹配—— 0.9 处从 0.72 升到 0.92 ，低处也涨一点，但不多。

小注：讲者只说了"加大 r " "加大 b "，没念具体值；按公式反推， 0.008 和 0.72 对应 $b = 10$ 、 $r = 20$ ， 0.92 对应 $b = 20$ 、 $r = 20$ (推断)。

- **想多陡都行**，代价是更多哈希函数。真实系统的量级：去重论文里 $b = 20$ 、 $r = 450$ ——每篇文档算 $9,000$ 个 MinHash。

$$s^* \approx \left(\frac{1}{b}\right)^{1/r}, \quad \Pr[\text{碰撞} \mid s = s^*] = 1 - \left(1 - \frac{1}{b}\right)^b \approx 1 - \frac{1}{e} \approx 0.63$$

s^* 是相变发生的相似度阈值：在这一点固定 band 匹配的概率恰好是 $1/b$ ，于是碰撞概率是一个与 b 、 r 几乎无关的常数 (讲者念作 0.64)；想

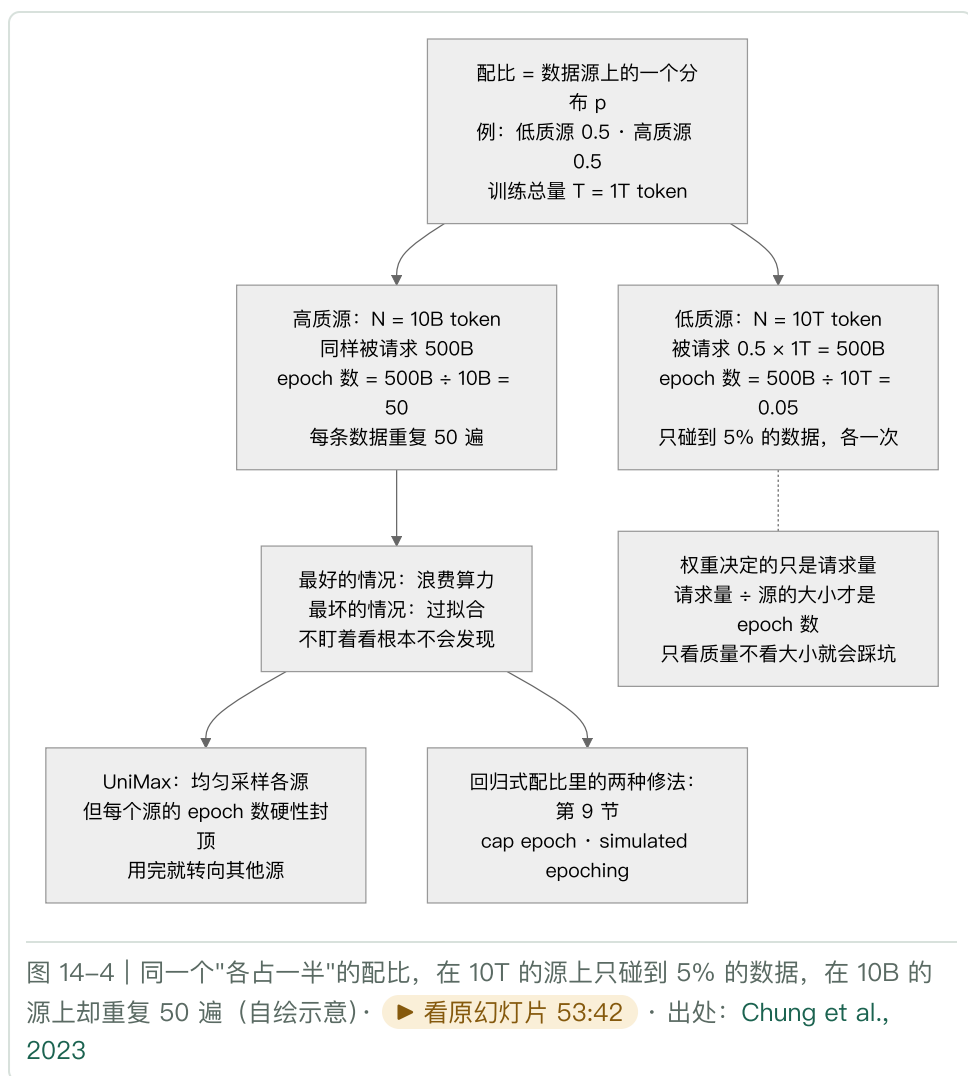
按 Jaccard 大于 0.9 去重，就把 b、r 调到让 s^* 等于 0.9，再同时调大两者让相变更陡——阈值以下趋于 0，以上趋于 1。

```
# MinHash 签名 + LSH 分 band (示意)
def minhash(doc, seeds):                                     # n 个独立哈
    # 希函数，各由一个 seed 决定
    grams = set(ngrams(tokens(doc), 5))                     # 文档 → 集
    # 合，例如 5-gram
    return [min(h(g, s) for g in grams) for s in seeds]     # 每个哈希函
    # 数取最小值
buckets = defaultdict(list)
for doc in corpus:                                          # 线性时间：
    # 每篇只算自己的签名
    sig = minhash(doc, seeds)                               # 长度 n = b
    # * r
    for band in range(b):                                   # b 个
        # band, 各 r 个值
        key = (band, tuple(sig[band * r:(band + 1) * r])) # r 个值全相
        # 同才落到同一桶
        buckets[key].append(doc.id)                         # 同桶 = 候选
    # 近似重复对
# 之后只对候选对验算 Jaccard，再删到只剩一份
```

这段展示的是线性时间从哪来：每篇文档只和“同桶”的文档比，桶的键由一个 band 的 r 个 MinHash 值决定，而不是两两比较。

- **名字：**这套方法叫 MinHash LSH——LSH 对任何哈希函数族都成立，语言模型的数据处理里用的是近似 Jaccard 的 MinHash 这一种。
- **一个实践备注：**数据集常常一份一份进来，各自去过重；但应当跨整个数据集去重，因为数据集之间也经常冗余。这一步常被省掉，不该省。

8. 数据配比：权重从哪来，以及 epoch 这个最大的坑



- **到这一步：**转换 → 过滤 → 去重之后，手里是一小批高质量文档——但这只是一个数据源的事，语言模型是在很多个源上训的。Marin 的网站上跟踪着下一个模型会用的源：Nemotron 那套网页数据、刚才说的 FinePDFs、Institutional Books、代码……老一点的 The Pile 也是一堆源，每个配一个权重。**配比（data mixture）就是数据源上的一个分布。**
- **权重从哪来**（讲者说这远谈不上第一性原理，甚至是它的反面）
 - *vibes*：凭直觉手动设——比你想的常见得多；就算新论文用了某种方法，最后也往往再手调。
 - *均匀*：每个源等概率，每采一个 chunk 先采一个源。
 - *按比例*：按各源的 token 数采样，也算理性——但一个巨大的低质源会吃掉大部分 token，不太对；直觉上应该上调高质量的源。
- **两约束**
 - *多样性*：源之间往往不可比——文学、代码、论文，说“这篇论文比这段代码质量高”没有意义；想让模型什么都行，就不能把质量全押

在论文上。

- **每个源是有限的**：小源权重太大就会用完，只能 epoch——反复训练字面上相同的 token——后果很坏。这一点要展开算。
- **算一遍**（图 14-4）：低质源 10T token，高质源 10B token（高质源一般都小）。天真地均匀配比各 0.5，训 1T token——这里的“1T”是步数乘以 batch size，不是 unique token。低质源被请求 $0.5 \times 1T = 500B$ ，除以它的 10T 是 0.05：只碰到 5% 的数据，各一次。高质源同样被请求 500B，可它只有 10B，每条数据要重复 **50 次**。一些大模型的训练就在这里翻过车：只看数据的质量定分布，忘了看每个源里到底有多少数据。

$$e_i = \frac{p_i T}{N_i}, \quad \text{UniMax 约束: } p_i T \leq c N_i \quad \text{对每个源 } i$$

e_i 是第 i 个源被训练的 epoch 数， p_i 是它在配比里的权重， T 是训练总 token 数（步数乘 batch size）， N_i 是这个源的大小；UniMax 要求每个源被请求的 token 数不超过它自身大小的 c 倍， c 就是 epoch 上限。

• 问答

- 为什么需要 50 个 epoch？——不需要。最好的情况是浪费算力，最坏是过拟合；50 只是“不注意就会发生”的数。教训：看清自己实际在每个源上做了多少 epoch。
- 配比在训练时怎么落地——每步换一个源，还是 10 步代码、10 步别的？——填 batch 时对**每条序列**采样它来自哪个源，一条序列只来自一个源，不按 token 采；一个 batch 应当是混合的，这样方差小。
- **UniMax**：这个问题很早就被注意到——训多语言模型时低资源语言的问题非常明显。此前的做法是把按比例分布取一个幂来压平；UniMax 更直接：**均匀采样各源，但对每个源的 epoch 数硬性封顶**（比如 20 个 epoch），用完就不再取，转向其他源——像一张安全网。有一个简单的过程能在这个约束下解出配比。

9. 回归式配比：小模型群、两次信仰之跃，和 simulated epoching

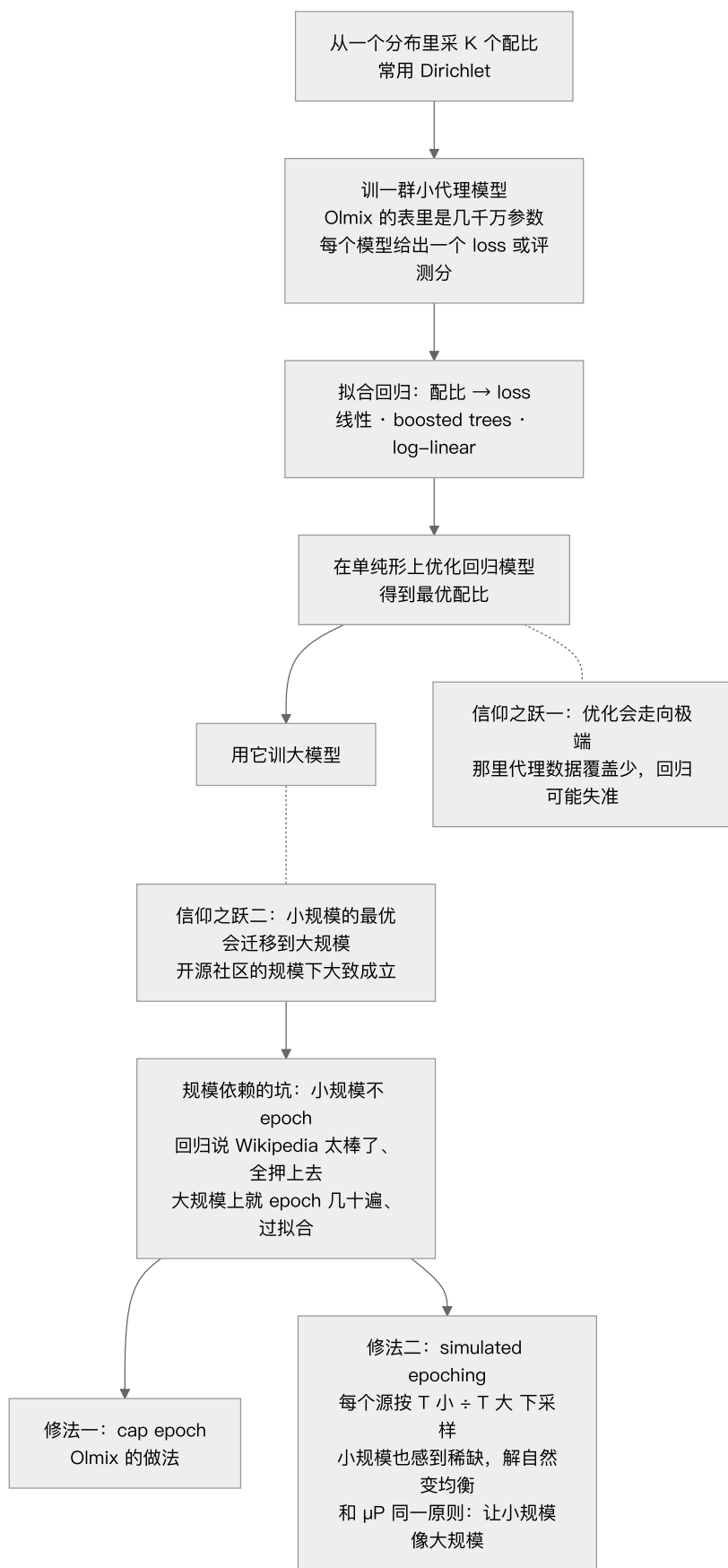


图 14-5 | 回归式配比的闭环，以及它的两个信仰之跃和两种修法（自绘示意）·

- **动机**：50 个源就要填 50 个数。按比例不够；按估出来的某个质量指标加权，也还是启发式。最有原则、也最容易看懂发生了什么的，是回归式方法（RegMix 等）。
- **步骤**：要训一个大模型，先到小规模（比如 300M 参数）试不同的配比——训一群（swarm）小模型，每个给出一个目标指标（下游评测或困惑度都行）；用这些点拟合一个回归：配比权重 $\rightarrow$ loss。于是你有了一个极便宜的模型，能回答“如果用这个配比训，loss 会是多少”；对它优化得到最优配比，拿去训大模型。和 scaling laws（第 9、11 讲）一个套路：便宜的计算找答案，再放大。

$$\hat{L}(p) \leftarrow \text{fit on } \{(p_k, L_k)\}_{k=1}^K, \quad p^* = \arg \min_{p \in \Delta^m} \hat{L}(p)$$

p_k 是第 k 个采出来的配比（ m 个源上的分布，常从 Dirichlet 采）， L_k 是用它训的小模型的 loss， $\hat{L}$ 是拟合出的“配比到 loss”的回归函数， Δ^m 是 m 维单纯形； p^* 是在回归函数上解出的最优配比，用于大规模训练。

• 设计决策

- **试哪些配比**：需要一个“分布上的分布”，常用 Dirichlet。
- **回归方法**：线性模型、boosted decision trees；Olmix 的表里 log-linear 效果不错。
- **目标指标**：常基于下游评测，要非常小心过拟合——预训练是在训通用模型，不是拟合几个评测：评测全是代码，回归就会上调所有代码数据，之后让它写诗就露馅。按比例和均匀配比没这个问题，因为它们不看评测。
- **小规模和大规模差多少**：代价与准确度的权衡——太小没有代表性，太大就等于在最大规模上调超参，没意义。
- Olmix 的总表把一批方法放进同一框架比：代理模型大小（几千万参数）、采多少个配比（随域数 m 变）、Dirichlet 还是指数分布、log-linear 回归、优化解法。

小注：Olmix (Chen et al., 2026, AI2, 为 Olmo 3 做的配比工作) 的摘要：代理模型 3000 万参数 (1500 万以上即可)，代理 run 的数量随域数线性增长；它得到的最好配比比自然分布好 12.2%，数据效率 3.05 倍。这些数课上没念。

• 两次信仰之跃（都要小心）

1. 回归模型是在随机采的配比上训的。随机采一个配比来预测没问题——经典泛化，分布内；但**优化**会把你推向极端，那里的覆盖少，回归可能失准。

2. 小规模的最优配比要能迁移到大规模。在开源社区的规模上大致成立，至少没有明显错；但显然有规模依赖效应——回想第 4 节：训更多 token 时低质数据也变得可以接受，所以最优点并不相同，只能“往好处想”。

- 一个必须处理的规模依赖效应：还是 10T 低质 + 10B 高质。小模型、少 token 时根本不 epoch，回归就会说“Wikipedia 太棒了，全押上去”；到大模型上用这个配比，高质数据被 epoch 几十遍，过拟合。两种修法：
 - *cap epoch* (Olmix 的做法)：直接给 epoch 数封顶。
 - *simulated epoching* (名字不止一个)：原则是让小规模看起来像大规模——这门课的通用主题，Tatsu 讲 μP (第 11 讲) 也是它：参数化得让超参能迁移。天真缩放的问题是小规模不 epoch、大规模 epoch，是两种性质不同的操作。做法：把每个源按比例下采样——小 run 10B token、大 run 1T token，就是 1/100——再做回归。这样“全押 Wikipedia”在小规模就会露馅：只有一点点 Wikipedia，马上 epoch 很多次、loss 很差，优化自然转向更均衡的配比。你在低规模上模拟了高规模的数据稀缺。

```
# 回归式配比 + simulated epoching (示意)
scale = T_small / T_big                                # 例: 10B ÷
1T = 1/100
sources = {k: subsample(v, scale) for k, v in sources.items()} # 让
小模型也感到稀缺
runs = []
for _ in range(K):                                     # 一群小代理
模型
    p = dirichlet(alpha, m)                             # 采一个配比
    loss = train_small(mix(sources, p), tokens=T_small) # 权重太偏就
会 epoch, loss 会差
    runs.append((p, loss))
L_hat = fit_log_linear(runs)                            # 配比 →
loss
p_star = minimize(L_hat, simplex=True, epoch_cap=20)    # 也可以再加
epoch 上限
train_big(mix(full_sources, p_star), tokens=T_big)
```

这段展示的是闭环的形状，以及 simulated epoching 插在哪里：下采样发生在采配比之前，让“过度偏向小源”在小规模就得到惩罚。

- **小结：**Wikipedia、Common Crawl、代码、数学各配多少——回归式配比是个好框架：小规模拟合"权重 $\rightarrow$ loss"，优化，推广到大规模；但 epoch 和过拟合要用 capped epoching 或 simulated epoching 管住。更大的教训：**你在优化任何东西时，都有优化错目标的危险。**
- **问答**
 - 下采样之后一个源太小怎么办？——确实可能只剩极少 token；那样最优解会给它很小的权重，放大之后也许恰好是对的。原则上行得通，但可能有"舍入误差"：本想训一遍，结果一遍都没训到；可以规定每个源至少训一遍来兜底。
 - 配比只在源之间做，还是也在一个数据集内部做？——讲者说忘了讲，好问题。Nemotron 的论文和一些 OLMo 的工作里：把 Common Crawl 按域分组（AI2 的 WebOrganizer 能按主题分），再按质量分档，得到一张"域 $\times$ 质量"的二维格子，每个格子当一个源来配比；这是自定义域的办法，再加上别人交给你的额外数据源。

10. 后训练数据：环境 $\times$ 任务 $\times$ 教师模型的合成流水线



- **和前面的分界：**到此为止讲的是预训练或中期训练的数据，基本与任务无关——回归式配比虽然在最小化某个 loss，数据本身仍是通用的，目的是打基本功。后训练的数据则**高度依赖任务**。这一段不做综述，只挑几个最近发布、尤其是编程方向的数据集。
- **通用配方：**定义一组环境（代码的话就是 GitHub 仓库），定义一组任务或 prompt，然后从一个强模型（教师）那里收集回答。言外之意：

至少在开源社区，后训练数据几乎全是合成的。教师也可以换成人——慢、贵；几年前在前沿就得花大钱请很多人写回答，现在即便前沿也是人机混合。不变的是：总有一个教师在给回答。

- **OpenThoughts**：起因是 o1 发布后大家都盯着数学和科学的推理，问题是怎么做出好的后训练数据。最终版本是 **120 万条**样本，由一个教师模型生成。
 - **任务从哪来**：很多源，人写的（Stack Exchange、NuminaMath）和合成的都有；编程部分有练习题、code golf（不算真实）、code review（更真实一些），此外还有数学、化学。这是很多人合作的大项目。
 - **系统分析的发现**：源不必全上，选几个好的就行；每题采多份回答（比如 16 份）有帮助；**更强的模型不一定是更好的教师**——QwQ-32B（现在看又老又小）当教师比当时最强的开源模型之一 DeepSeek-R1 更好；简单的答案过滤没有帮助。
 - **流水线**：各源 → 去重 → 随机下采样题目 → 每题生成多份回答 → 最终数据集。120 万是样本数，除以 16 才是题目数。

小注：论文摘要（Guha et al., 2025）：用 QwQ-32B 当教师、规模到 120 万条的 OpenThoughts3 训出的 7B 模型在 AIME 2025 得 53%、LiveCodeBench 51%、GPQA Diamond 54%，比 DeepSeek-R1-Distill-Qwen-7B 分别高 15.3、17.2、20.5 个百分点。120 万 ÷ 16 ≈ 7.5 万道题是按讲者的说法算出来的。

- **agentic coding**：最近的兴趣从“会写代码的模型”转向“能做软件开发的模型”，数据集随之复杂起来。
 - **SWE-smith**：给一个仓库，让语言模型的 agent 先把它弄到能用（装依赖等），再自动生成任务——改代码、埋 bug——经验证得到任务实例。任务是合成的，但有 **5 万个**，在当时（去年）算大。
 - **SWE-zero**（NVIDIA；名称按字幕发音复原，推断）：软件工程任务和数学不同，依赖极重——大多数 GitHub 仓库根本跑不起来，回滚到某个 PR 当时的状态更是一团糟。与其给每个仓库做 docker 镜像，不如利用一个事实：模型已经强到**不需要执行反馈**也能解掉很多任务——允许执行大约 80，不允许也有将近 70——说明模型对代码有某种内部语义。于是生成 **30 万条** agent 轨迹，全部来自真实 GitHub PR（不像 SWE-smith 是合成任务），用 OpenHands 脚手架；为防 agent 作弊，“zero”版本不许运行 Python，只能用 sed、grep 这类基本操作；从一个大编程模型蒸馏再过滤——编程

模型有时会无视指令偷偷执行。另有 1.3 万条需要执行反馈的轨迹（字幕作"sweet hero"，应为论文里带执行反馈的子集名，未能核实）；先在 30 万条上微调，再在这 1.3 万条上微调，有进步，离前沿还远。

- *SWE-rebench*: 另一次"抓尽可能多的 PR"的尝试：收集仓库、尝试安装、大多失败、再努力，然后让语言模型给回答。
- *当天刚发布的一篇*：把 SWE-zero 的思路放大到 1200 万条轨迹，用的是 SWE-rebench 的任务——那边只让 3.2 万个任务跑起来了，12 万个没跑起来；zero 式的做法不在乎能不能执行，全都能用。因为数据大了，用的是一个很小的模型。
- **趋势与取舍**：数据集从不需要环境的数学，到编程，再到规模越来越大的编程数据。任务有三档真实度：全合成、半合成（真环境 + 合成任务）、全真实；回答来自有能力的模型，但它们还得是好教师；代码环境很折磨人，过滤和细节一大堆，课上没时间展开。

11. 收尾：四道工序的一句话总结，和"数据工作不长这样"

- **过滤**：定义什么是好数据，训一个轻量分类器，扫一遍网页抓取，得到像它的一小份。
- **去重**：为了不过拟合、不浪费 FLOPs；近似去重靠 MinHash + LSH。
- **配比**：小规模试，大规模用；盯住 epoch。
- **后训练数据**：环境 × 任务 × 教师，几乎全合成。
- **讲者的提醒**：真正的数据工作又脏又具体，高度依赖领域，要靠盯着一个个具体样本看才能做出高质量数据集——这一讲展示的是数据的版图，不是数据工作本身的样子。
- **这一讲没讲到的**：DoReMi 一类在训练中在线调权重的方法、退火 / cooldown 阶段专门的数据、PII 处理，课上都没出现；退火阶段的数据应在第 15 讲（推断）。

小注：课程主页上 Assignment 4 (Data) 就是把这一讲自己做一遍：从 Common Crawl 的 WARC 文件抽文本，做语言识别、PII 遮蔽、有害内容和质量分类器，做精确与 MinHash 去重，再训小模型比 loss——课上没有提作业。

图	看什么	跳转	出处
HTML 抽取工具对比	第 13 讲的回放：Resiliparse 与 Trafilatura 在扩展 DCLM 评测上的差距，说明规则式工具的选择也影响模型	3:44	DCLM
过滤的骨架图	一小份目标 T、一大池原料 R，要找的是 R 里像 T 的子集；后面所有实例都套这张图	6:51	—
OpenWebMath 流水线	三步：LaTeX 规则 → KenLM 困惑度 → fastText；两套阈值，有 LaTeX 门槛低	12:37	Paster et al., 2023
phi-1 的训练曲线	用 GPT-4 标注再训分类器筛出的数据：更少的步数到更高的值	16:12	Gunasekar et al., 2023
epoch 实验 (157M 模型、100 个 WARC)	蓝线 DCLM 先低后过拟合；未过滤的 Resiliparse 先高、后来反超；竖线是 epoch 边界	18:12	—
C4 里的防毒面具描述	同一段商品描述出现 61,000 次；"看数据"的理由	25:50	Lee et al., 2021
特征矩阵与随机置换	行是元素、列是 A 和 B；哪一行被置换到最前决定 MinHash 是否相等，读出"概率 = 交集 / 并集"	34:04	—
LSH 的 S 形曲线	横轴相似度、纵轴碰撞概率；看相变发生的位置和陡度	42:47	—
b、r 的数字表	b = 10、r = 10 时 0.7 到 0.98 对应 0.25 到 1；加 r 后 0.7 处掉到 0.008、0.9 处 0.72；加 b 后 0.9 处升到 0.92	43:50	—
Marin 的数据源清单	下一个模型要用的源：Nemotron 网页数据、FinePDFs、Institutional Books、代码	49:37	Marin
epoch 的算术	p 乘 T 除以 N：低质源 0.05、高质源 50；这页是整节配比的核心	53:42	—
Olmix 的方法总表	各回归式方法的代理模型大小、采样的配比数、分布、回归模型、优化方式并排比	1:03:59	Chen et al., 2026
OpenThoughts 流水线图	各源 → 去重 → 下采样题目 → 每题 16 份回答 → 120 万条	1:16:58	Guha et al., 2025
SWE-zero 的执行与不执行	允许执行约 80、不允许约 70：模型不跑代码也能解题的证据	1:19:32	未能核实

提到的工作

名称	在本讲里的作用
Common Crawl	原料池；HTML 为主，夹杂 PDF；WARC 是它的归档格式
Resiliparse · Trafilatura	HTML 抽正文的规则式工具；第 13 讲比过，Resiliparse 在扩展 DCLM 评测上更好；本讲

名称	在本讲里的作用
	epoch 实验里 Resiliparse 直出代表"不过滤"
DCLM (Li et al., 2024)	过滤后数据的代表, epoch 实验里的蓝线; 扩展评测集
FinePDFs (Hugging Face, 2025)	PDF 语料: 截断要重爬、扫描件要 VLM 做 OCR、平均质量高
KenLM	生成式过滤器: 5-gram 模型, 用困惑度打分
fastText (Joulin et al., 2016) · 语言识别模型	判别式过滤的标准工具, 线性 bag-of-words; Meta 的现成语言识别模型支持 176 种语言
OpenWebMath (Paster et al., 2023)	定向过滤的范例: 规则 + KenLM + fastText, 15B token 胜过 20 倍数据
Proof-Pile	数学语料, OpenWebMath 拿它训 KenLM
GPT-3 (Brown et al., 2020)	质量分类器: Wikipedia、WebText、书为正例, 网页为负例
LLaMA (Touvron et al., 2023)	正例是被 Wikipedia 引用的页面
phi-1 / Textbooks Are All You Need (Gunasekar et al., 2023)	两级过滤: GPT-4 按 prompt 标 100k 篇, 再训随机森林扫 The Stack 的 Python 子集
The Stack (Kocetkov et al., 2022)	phi-1 的原料池
Jigsaw Toxic Comment Classification	毒性过滤的训练数据: Wikipedia 讨论页的人工标注
C4 / T5 (Raffel et al., 2020)	精确去重的例子: 三句话跨度, 删到只剩一份; 61,000 次的商品描述也出自对它的审计
LM1B (Chelba et al., 2013)	近似重复的例子: 同一篇文章只差一个逗号
Deduplicating Training Data Makes Language Models Better (Lee et al., 2021)	61,000 次的审计; MinHash LSH 的真实参数 $b = 20$ 、 $r = 450$; 去重减少记忆
MinHash · LSH	来自算法界的经典工具, 讲者没报出处; 应出自 Broder (1997) 与 Indyk-Motwani (1998) (推断)
Marin	Percy 主持的开放训练项目; 网站上跟踪下一个模型的数据源
The Pile (Gao et al., 2020)	多源加权配比的早期例子
Nemotron · Institutional Books	Marin 数据源清单里的两项; Nemotron 的论文还被引为"域 × 质量"格子配比的例子 (应为 Nemotron-CC, 推断)
UniMax (Chung et al., 2023)	均匀采样 + 每个源 epoch 数硬上限; 多语言场景提出
RegMix (Liu et al., 2024)	回归式配比: 小模型群 → 拟合 → 优化 → 放大
Olmix (Chen et al., 2026)	回归式配比的统一框架和方法总表; 用 cap epoch 处理规模依赖; 字幕作"Omix"

名称	在本讲里的作用
μP (Yang et al., 2022)	"让小规模像大规模"的同一原则；第 11 讲 Tatsu 讲过
WebOrganizer (Wettig et al., 2025)	AI2 的网页按主题分域工具，配合质量分档做格子配比（应为这篇）
OLMo	AI2 的开源模型系列；"一些 OLMo 的工作"里也做了数据集内部的配比
OpenThoughts (Guha et al., 2025)	推理后训练数据：120 万条，每题 16 份回答；QwQ-32B 比 DeepSeek-R1 更好的教师
Stack Exchange · NuminaMath	OpenThoughts 的人写题目来源
QwQ-32B · DeepSeek-R1 · o1	教师模型的对比；o1 是 OpenThoughts 的起因
SWE-smith (Yang et al., 2025)	合成软件工程任务：agent 把仓库弄到能用、改代码埋 bug，5 万个任务
SWE-zero (NVIDIA)	30 万条来自真实 PR 的无执行轨迹，OpenHands 脚手架，只许 sed / grep；名称按字幕复原，未能核实
OpenHands (Wang et al., 2024)	SWE-zero 用的 agent 脚手架
SWE-rebench (Nebius, 2025)	大规模收集真实 PR 的另一次尝试；当天新论文用它的任务
当天发布的 1200 万条轨迹论文	SWE-zero 思路放大：3.2 万个能执行 + 12 万个不能执行的任务全用；讲者没报题目，未能核实

术语对照

English	中文
data pipeline	数据流水线：转换 → 过滤 → 去重 → 配比
boilerplate	样板内容：导航、广告、页眉页脚等非正文
content extraction	正文抽取
linearize	线性化：把层级或二维的 HTML 压成一维 token 序列
lossy	有损的
OCR / VLM	光学字符识别 / 视觉语言模型（用来读扫描版 PDF）
target data (T) / raw data (R)	目标数据 / 原料数据：过滤骨架里的两个输入
quality filtering	质量过滤
toxicity filtering	毒性过滤
language identification (LID)	语言识别
code switching	语码混用：一段话里混用多种语言

English	中文
generative / discriminative classifier	生成式 / 判别式分类器：只对 T 建模 vs 正负例训分类器
n-gram model / perplexity	n-gram 语言模型 / 困惑度：KenLM 的打分方式
bag of words	词袋：不看顺序只看词频的表示
linear classifier	线性分类器
threshold / stochastic keep	阈值 / 按概率随机放行
positive / negative examples	正例 / 负例
educational value	教育价值：phi-1 让 GPT-4 判断的标准
epoch	轮：把一个数据源完整过一遍；这一讲的核心变量
overfitting	过拟合
confidence interval	置信区间
exact / near duplicate	精确重复 / 近似重复
mirror site / fork	镜像站 / 分叉的仓库
decontamination	去污染：确保测试集不在训练集里
hash function / hash collision	哈希函数 / 哈希碰撞：两个不同的项映到同一个值
cryptographic hash	密码学哈希：抗碰撞，这里不需要
MapReduce	分而治之的并行计算范式，精确去重按它写便于扩展
Jaccard similarity	Jaccard 相似度：交集大小除以并集大小
characteristic matrix	特征矩阵：行是元素、列是集合，标明谁包含谁
permutation	置换：随机哈希函数等价于给元素随机排序
MinHash	最小哈希：集合里每个元素哈希后取最小值
locality sensitive hashing (LSH)	局部敏感哈希：相似的项更容易落进同一桶
band / signature / bucket	带 / 签名 / 桶：r 个哈希一组叫 band，n 个 MinHash 组成签名，band 的值决定桶
false positive	假阳性：阈值以下却碰撞了的对
phase transition	相变：S 形曲线里从"几乎不碰"到"几乎必碰"的陡坡
data mixture	数据配比：各源上的一个分布
uniform / proportional mixing	均匀配比 / 按 token 数比例配比
cap	上限：UniMax 给每个源的 epoch 数封顶
proxy model / swarm	代理模型 / 模型群：小规模上并行训的一批小模型
regression-based mixing	回归式配比：拟合"配比 → loss"再优化

English	中文
Dirichlet distribution	狄利克雷分布：单纯形上的分布，用来采配比
log-linear model	对数线性模型：Olmix 表里表现好的回归形式
simplex	单纯形：所有配比的取值空间
downstream eval	下游评测
in-distribution / extrapolate	分布内 / 外推
scale-dependent effect	规模依赖效应：小规模和大规模的最优不一样
simulated epoching	模拟 epoch：把源按比例下采样，让小规模也感到数据稀缺
downsampling	下采样
domain × quality grid	域 × 质量格子：一个数据集内部自动切出的"源"
post-training data	后训练数据
environment / task / teacher	环境 / 任务 / 教师：合成数据配方的三要素
distillation	蒸馏：用强模型的回答训小模型
agent trajectory	agent 轨迹：一次任务里的完整交互记录
scaffold	脚手架：跑 agent 的框架（如 OpenHands）
agent hacking	agent 作弊：绕过限制（如偷偷执行代码）
execution feedback	执行反馈：跑代码得到的结果
dependencies / docker image	依赖 / docker 镜像：让仓库能跑起来的环境
code golf / code review	代码高尔夫（最短代码解题） / 代码评审
semi-synthetic	半合成：真实环境 + 合成任务

字幕勘误

"Chromic Crawl / comic crawl / common call" → Common Crawl; "Brazil parse / Brazilia parse / resilient pars" → Resiliparse; "Traflera" → Trafilatura; "find PDFs" → FinePDFs; "heristic" → heuristic; "dduplication / due duplication / DDUP / dubbing" → deduplication / dedup; "klm" → KenLM; "fastax" → fastText; "latte" → LaTeX; "51 from Microsoft" → phi-1; "GPD4" → GPT-4; "100 works" → 100 WARC; "jakard / chakard / your card" → Jaccard; "minash" → MinHash; "sarcastic" → stochastic; "BNR" → b and r; "beach tries" → b tries; "paralyzed / paralyzable" → parallelized /

parallelizable; "epoing / epox" → epoching / epochs; "neatron / Neutron" → Nemotron; "durlay / dishlay" → Dirichlet; "ragmix" → RegMix; "Omix / OMIX" → Olmix; "unimax" → UniMax; "MUP" → μ P; "OMO" → OLMo; "sack exchange" → Stack Exchange; "num math" → NuminaMath; "01 came out" → o1; "QWQ 32B" → QwQ-32B; "deepseeek R1" → DeepSeek-R1; "Swissmith / Sweenith Smith" → SWE-smith; "sweet zero / three zero" → SWE-zero (推断); "sweet ben rebench" → SWE-rebench; "educa / education execution" → execution; "open hands" → OpenHands; "set and grap" → sed and grep; "one billion or benchmark" → One Billion Word Benchmark (LM1B); "R&T" → R and T; "Meta ... fast text" 无误; "security data collection" → targeted data collection (推断)。

带走的问题

1. 第 4 节说阈值取决于训练 token 数：数据池固定为 100 个 WARC，把模型从 157M 换成 1B（按 Chinchilla 配比 token 也要多 6 倍以上），质量阈值该往哪边挪？把这张 epoch 曲线和第 9 讲的数据 scaling law（重复数据收益递减）放在一起，能写出“最优阈值随 T 变化”的定性关系吗？
2. 按 $s^* \approx (1/b)^{(1/r)}$ 算，Lee et al. 的 $b = 20$ 、 $r = 450$ 对应的 Jaccard 阈值约是 0.99——那它抓的是什么级别的重复？课上 100 个哈希函数就把 Jaccard 估到 0.6，为什么判定“是否超过阈值”却要 9,000 个？“估计相似度”和“做一个陡峭的开关”对哈希数量的要求为什么差这么多？
3. 给 Marin 那样十来个源写一个 epoch 检查：UniMax 的硬上限和 simulated epoching 谁更保守？高质源被下采样到只剩几百万 token 时，回归的解会不会把它“舍入”成 0？“每个源至少训一遍”的兜底和 epoch 上限会不会冲突？
4. 回归式配比的目标指标若是下游评测，怎么组合评测集合才不会“全押代码”？换成各源 held-out 困惑度的加权和，会不会退化成按比例配比？两次信仰之跃里哪一次更容易用实验证伪？
5. QwQ-32B 比 DeepSeek-R1 更好的教师——“更强 ≠ 更好的教师”可能的机制是什么（回答长度、风格、对学生的可学性）？SWE-zero 的“不许执行”既防作弊又省环境，训出来的模型在真能执行时会不会反

而不会用执行反馈？1200 万条轨迹里那 12 万个跑不起来的任务，回答的正确性由谁保证？

CS336 第 15 讲 | 中期训练与后训练

(Mid/Post-Training: SFT, RLHF)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 15 讲
(课程表: 2026 年 5 月 18 日), post-training 的上半场; 下一讲 (第 16 讲) 接着讲 RLVR 与 thinking model 视频:

<https://www.youtube.com/watch?v=2oH6PWPrYFo> (1:19:54; 英文字幕为自动生成, 数据集名、人名和缩写错得不少, 见文末勘误) 讲者: Tatsunori Hashimoto (全程; 多处自称"我的学生做了 Alpaca""我和 Percy 合带的博后做了观点研究", 与课程表一致) 课程主页: <https://stanford-cs336.github.io/> · 本讲围绕的论文: InstructGPT、Learning to summarize from human feedback、Tulu 3、DPO、Llama 3、MiniCPM

一句话: 前 14 讲能把你送到一个"加强版 GPT-3", 可这种 base model 只能做文案之类不需要可靠性的小活; 从 GPT-3 到 ChatGPT 靠的是 post-training, 而它的算法部分薄得可怜——SFT 就是换了数据的预训练, RLHF 的目标在 InstructGPT 里一行写完, DPO 更是化成"抬高胜者、压低败者"两项——真正的杠杆全在数据: 谁写 prompt、谁写回答、写多长、带不带引用、几百条安全样本够不够、标注者是本科生还是时薪一百多美元的律师、要不要干脆让更强的模型来标。两条主线是"底座够强只需少量高质量示范"和"RLHF 是最大化奖励而非拟合分布, 所以会过度优化、坍缩、失去校准"; mid-training 则把指令数据直接塞进衰减期, 让"base model"名不副实。有没有一种奖励能放心一直优化下去——留给第 16 讲的 RLVR。

时间轴

时间	内容
0:05	开场: 从 GPT-3 到 ChatGPT 靠 post-training; 本讲 SFT + RLHF, 下一讲到 o1 类 thinking model
3:07	前沿 post-training 的信息极少: 能引的多是 2020-2022 年的论文; 2023 年 Scale AI 泄露文件; 秘诀在数据不在算法
5:39	两段式配方: 示范数据 SFT, 再用 RL 塑形; SFT 的方法等于预训练, 差别全在数据
7:40	SFT 数据谱系: FLAN → Self-Instruct → Alpaca / Vicuna → OpenAssistant → WizardLM / Tulu 3 → agentic SFT; 问答: 输入输出对的正确性有多重要
11:13	逐个看: FLAN 的不自然任务和幻觉摘要、"后训练也要规模"的错误假设; Alpaca 蒸馏 ChatGPT; OpenAssistant; Nemotron 的工具调用 SFT

时间	内容
17:50	三个大转向：更像人的对话、更专业的标注者、工具使用；长度与风格：风格偏好不等于能力提升
21:55	一条样本同时教知识和格式：参考文献、尾部知识与幻觉；Schulman 的"所以需要 RL"论证；问答
28:02	安全 SFT：Llama 2 的违规率对误拒率；Tulu 3 约 5 万条安全数据、从 WildChat 里挖；500 条就能大幅压低
33:42	SFT 数据小结；问答：怎么知道某种行为已在预训练里；SFT 抹特征、RL 推特征？
35:44	方法就是梯度下降；mid-training：指令数据混进衰减期；"base model 是个谎言"；MiniCPM 的两阶段配比
38:16	问答：衰减期的 prompt 要不要 mask、质量该更高还是更低；配比靠试错：衰减期跑约 10 次消融再反推预训练配比；Meta 版权官司里的消融文件
41:49	RLHF：从"拟合分布"到"最大化奖励"的概念转换；为什么要 RL：人评判比自己写强、验证比生成容易
45:54	RLHF 流程：T=1 采样、成对打分、奖励模型；InstructGPT 的 helpful / truthful / harmless 准则；Bard 泄露的 Likert 指南
48:58	标注者：学历与年龄、时薪 50 到 100 多美元、防 AI 代写、时间压力；标注人群决定模型观点；隐性传递；专家对众包
59:43	模型当标注者：GPT-4 评审对人（AlpacaFarm）；Zephyr 放弃人工数据；UltraChat / UltraFeedback / Tulu 3 全靠模型；Constitutional AI；长度偏置
1:05:23	算法：带 KL 的目标（InstructGPT 的式 2）；policy gradient → off-policy / TRPO → PPO；几种"去掉 PPO"的失败尝试
1:09:58	DPO 推导：非参数闭式解、隐含奖励、梯度的含义；DPO 对 PPO 不重要；Llama 3 的外循环；SimPO 等变体；AI2 前后矛盾的结果
1:16:34	三个坑：过度优化与 KL、模式坍缩、RLHF 后校准变差（GPT-4 的图）；过渡到 RLVR

核心内容

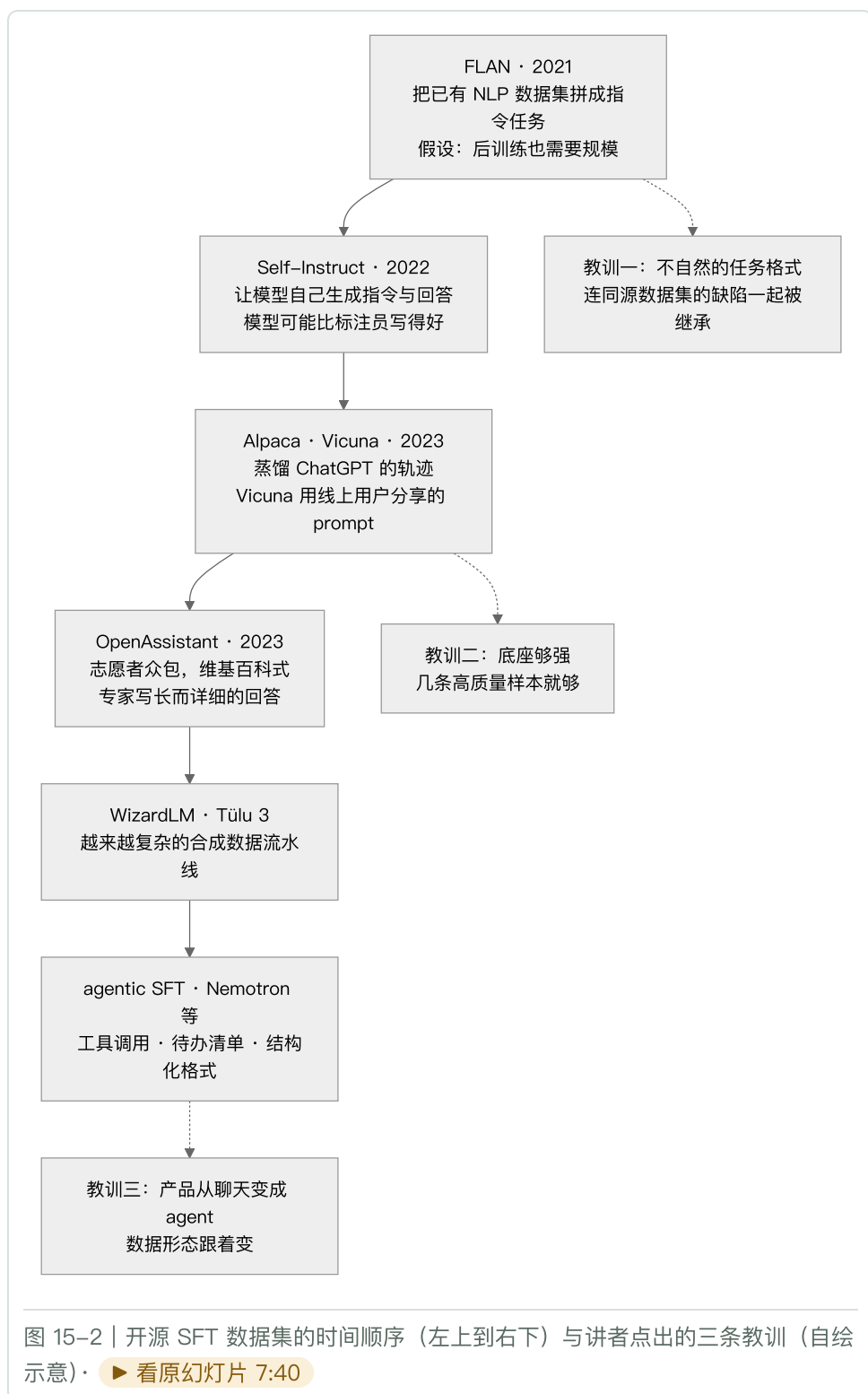
1. 大图：从 GPT-3 到 ChatGPT，算法薄、数据厚



图 15-1 | 从预训练到 RLVR 的整条时间线，以及每一段进的是什么数据（自绘示意）

- **这一讲的位置：**到第 14 讲为止，你能造出一个"加强版 GPT-3"——更多 FLOPs、更多数据。但 GPT-3 这种又大又强的 base model 能榨出的效用有限：只能做文案之类不需要可靠性、不需要指令遵循的小活。ChatGPT 之所以惊艳，差的就是 post-training。本讲走 GPT-3 → ChatGPT (SFT + RLHF)，第 16 讲走 ChatGPT → o1 类 thinking model (RLVR)。
- **预训练仍是根：**跳过预训练直接 post-train，什么都得不到。预训练是那锅广而杂的"原始汤"，post-training 的活是从里面把想要的行为提取出来——显式收数据、显式引导，外加大量脏活。
- **为什么材料偏旧：**能详细引用的几乎都是 ChatGPT 竞争白热化之前的论文——[Stiennon et al., 2020](#)、Anthropic 2022 年的 HH 论文 ([Bai et al., 2022](#))。此后各家把 post-training 数据当商业机密；开源配方又大多靠蒸馏，跟前沿实验室"自己收人工数据"这件又难又乱的事性质不同。
 - 例子：2023 年 Scale AI 内部文件泄露，为了让 Google Bard 追上 GPT-4，要求标注员写出比 GPT-4 更详细、更好的回答。
- **两段式配方：**先为每个 prompt 收标注者写的示范回答做 SFT；再做某种 RL，把行为往人认为好的方向塑。算法层面大家心里有数，但算法不是秘诀，杠杆在数据——"怎么 SFT"和预训练一模一样，只是换了数据，所以 SFT 这段几乎全讲数据。
- **与已学内容的关系：**CME295 第 4 讲讲过 SFT，第 5 讲和 CS224R 第 9 讲推过 RLHF / PPO / DPO；本讲不重复推导，字数花在"什么数据、多少、按什么顺序"上。

2. SFT 数据的谱系：六代数据集，三条教训



- **FLAN** (Google, 用于 T5 系列): 最老也最有远见——NLP 界早就攒了一堆"输入 → 输出"的监督数据集, 全拼起来训一遍。合理, 但事后看不对 (第 3 节)。

小注: 最早的 FLAN (Wei et al., 2021) 微调的是 137B 的 LaMDA-PT; 训 T5 的是后来的 Flan-T5 / Flan Collection (Chung et al., 2022; Longpre et al., 2023), 讲者说的应是后者。

- **Self-Instruct**: 模型越来越强、甚至可能强过一部分标注员，为什么不¹让模型自己写回答？
- **Alpaca / Vicuna**: 蒸馏一路。Alpaca（讲者学生的工作）蒸馏 ChatGPT；Berkeley 的 Vicuna 用线上用户分享的 prompt 做输入。
- **OpenAssistant**: 反过来完全靠人——志愿者像编维基百科一样写 prompt 和回答。
- **WizardLM、Tulu 3**: 既然模型是很好的合成数据生成器，就设计越来越复杂的流水线生成指令数据。
- **agentic SFT**: 最近一两年重心从聊天转向 agent 和工具使用，出现了专门生成工具调用与 agent 轨迹的流水线。
- **问答：输入输出对的正确性有多重要？** 顶层答案是尽量收最高质量的，坏样本教坏行为；但微妙之处在于拿各种奇怪的东西做 SFT，模型照样学会遵循指令——预训练的泛化让你能容忍差一些的数据。讲者提到 Percy 以前的学生有篇论文，训练时连回答都不给也能得到指令遵循。

小注：应为 Hewitt et al., 2024 《Instruction Following without Instruction Tuning》——该文去掉的其实是指令（只在回答上训练，称 response tuning），或只用单一任务的数据，模型仍表现出指令遵循；讲者顺口说成“连回答都不给”。

3. 逐个细看：FLAN 为什么不对，Alpaca 为什么对，数据形态怎么变

- **FLAN 的毛病 (11:44)**
 - **不自然**: 样例是一封 Enron 邮件后面接一句“给这封邮件写主题行”，答案在右边——没人会这样提问。它由现有数据集机械改造而来：Enron 邮件集有正文和主题，就造出“预测主题”任务；摘要任务取自 CNN/DailyMail 或 XSum。
 - **继承源数据集的缺陷**: 摘要比 ChatGPT 短得多，细看常有幻觉——细节在输入里根本没有。源数据集本身质量不高，缺陷和不自然的结构一并传下去。
 - **错误的先验**: 当时以为 post-training 也像预训练一样需要规模，所以拼了海量数据集；后来发现数据集一路缩小照样有效——底座够强时少量高质量样本就够，预训练的泛化走完剩下的路。FLAN 落在了质量—数量权衡的错误一端，但不把整个空间探索一遍谁也不知道。

- **Alpaca (14:16)**: ChatGPT 出来后蒸馏其轨迹，得到输入自然、输出更长更"聊天"的样本；训到原版 Llama 上可靠地诱导出 ChatGPT 式行为——预训练和 post-training 得同时到位。随之而来一股乐观：收到足够大、足够好的指令数据就能追上闭源实验室。
- **OpenAssistant**: 那股乐观下的大规模众包——志愿者出难题、写高质量回答，想复制维基百科。讲者记不清是"一万多条还是更多"，项目后来停滞。

小注：论文摘要的数字 (Köpf et al., 2023): 161,443 条消息、35 种语言、461,292 条质量评分、一万多棵完整标注的对话树、一万三千五百多名志愿者。

- **Nemotron 的 agentic SFT (17:19)**: NVIDIA 开源的 SFT 数据里很大一块是 agent 样本——assistant 字段旁并行出现工具调用，Claude Code 或 Codex 那种边做边勾的待办清单也在其中；结构化格式被显式监督进模型。
- **三个高层转向 (17:50)**:
 1. *chattiness*: 经典 NLP 数据集再好也像"输入 → 程序化输出"，人不想跟 benchmark 说话，于是转向更详细、更像人的回答。
 2. *更好的标注者、更多细节*: OpenAssistant 是代表——让专家写回答。
 3. *工具使用*: 为 agent 找对接口和 API，是最近的一次转向。

4. 收 SFT 数据的坑：风格不是能力，一条样本同时教两件事

- **假设你负责收人工 SFT 数据 (18:53)**: 要操心格式、回答里放多少知识、要多少条、收什么安全数据。讲者特别点出，**长度与风格差异是 post-training 里非常大的一块**——"Claude 和 ChatGPT 语气不同""ChatGPT 太啰嗦"，全是数据团队的有意决定。
- **风格偏好会骗人**: 做偏好评测时，人会显著偏向带项目符号、更长更详细的回答。这本身不算错，但会系统性地扭曲语气；更要紧的是看 engagement 信号很容易自欺：训不同的 post-training 数据集，AlpacaEval 那一系列的偏好分数大幅波动，标准能力 benchmark 却基本不动 (21:25)。模型没变聪明，只是更讨喜——**风格控制和能力控制要分开想**。
- **一条样本同时教知识和格式 (21:55)**: OpenAssistant 的一条高质量回答末尾附了文献引用。拿它做 SFT，下一 token 预测同时学到两件

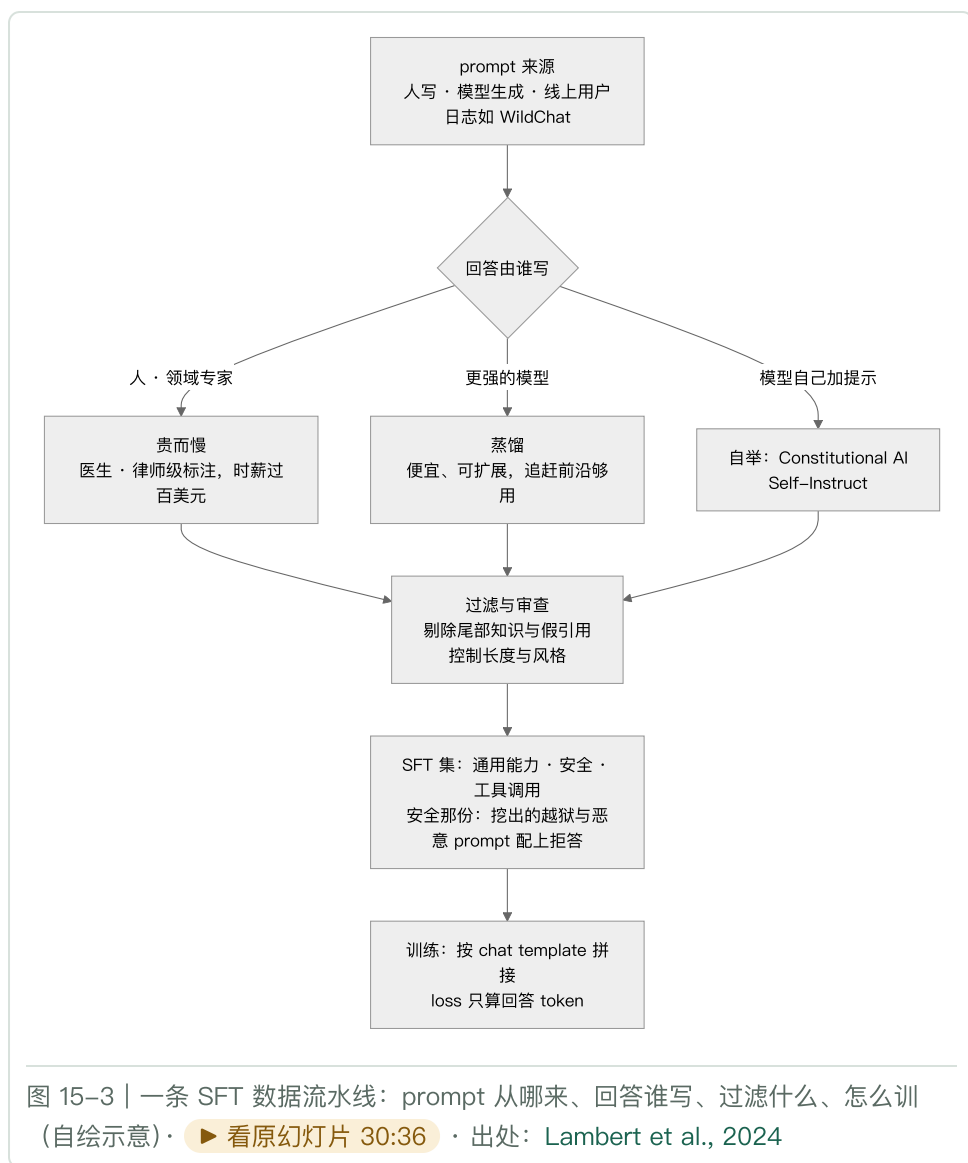
事——这条具体引用（知识）和"好回答该附引用"（格式）。后果是误泛化：模型不知道引用真假，却学会了"该有引用"，于是编一条。

- **尾部知识可能有害**：folklore（讲者认为有实证支持）——在 SFT 阶段逼模型说出它不知道的事实会诱发幻觉，只在已知事实上训练则不会。所以"最高质量的数据"不一定该训：模型不知道的尾部知识，尤其配上"Reference:"这类标记，是在教它强行输出未知内容。
 - 问答：什么叫尾部知识？没有正式定义；讲者用过的代理指标是 Wikipedia 条目长度——在冷门条目上训练，幻觉确实变多。

小注：应为 Gekhman et al., 2024 《Does Fine-Tuning LLMs on New Knowledge Encourage Hallucinations?》一类的结果；"policy-dependent"的论证出自 John Schulman 2023 年在 Berkeley 的演讲 RL and Truthfulness。

- **Schulman 的论证：这正是需要 RL 的原因**。要让模型知道"我知道什么、不知道什么"，监督必须依赖策略本身；外人往它嘴里塞知识没法让它校准。讲者的民间版解释 (26:30)：假设模型内部有一根"我知道"方向。SFT 逼它无论如何都给引用；RL 时它发现在"我知道"方向上给引用得高分、在"不知道"方向上得低分，于是把内部信号泛化到"要不要给引用"上。前提是内部确实有这个信号，否则 RL 也无能为力。
 - 问答：SFT 会惩罚这条引用吗？不会——SFT 本身没做错，错在从一条样本没法完美泛化：模板和内容被一起教了。

5. 安全 SFT：两条曲线、几千条数据、500 条就见效



- **安全落在 post-training 团队头上 (28:02):** post-training 的人得面对政治操纵、假信息、个性化的 spear phishing——你是滥用之前的最后一道防线，事实上的做法就是训模型拒答恶意输入。
- **公开信息更稀薄:** Llama 2 的安全 SFT 描述已经算详细的，却连用了多少条都没说。所有安全调优都在平衡两条曲线: *violation rate* (放过了多少坏请求) 和 *false refusal rate* (误拒了多少正常请求，比如"怎么 kill 一个 Python 进程"被拒)；做法是造定向数据在两者之间找 Pareto 前沿。规模通常几千到几万条，Llama 2 讲者估计是几千条。
- **唯一能看清全貌的公开配方: Tulu 3** (AI2 为 OLMo 做的 post-training 流水线)。安全部分约 5 万条，生成策略很朴素：此前的 WildChat 项目给大家免费聊天、换取聊天记录；从中大规模挖出不安全的尝试和越狱，再配上"抵抗越狱"或"拒绝"的标准回答。从模型卡

看，闭源公司做的也类似——看使用数据、找坏行为、让标注员打地鼠。

小注：Tulu 3 的安全 SFT 数据应出自同组的 WildJailbreak / WildGuardMix (Jiang et al., 2024) (推断)；WildChat 见 Zhao et al., 2024。

- **500 条就够转向 (32:09)**：底座够强时，几百条就能把系统掰过来——挖或合成一批不安全请求、写上拒答，只加 500 条，执行恶意指令和仇恨言论指令的比例就大幅下降。一个理解方式：预训练后的模型内部已经有“我是安全模型还是不安全模型”这根轴，不用很多样本就能拉出来。

小注：这张曲线应出自 Bianchi et al., 2023 《Safety-Tuned LLaMAs》(Hashimoto 是作者之一)：往指令数据里加约 3% 即几百条安全示范，安全性显著提高、能力基本不降；加太多则出现“过度安全”误拒。

- **但少不等于够**：OpenAI、Anthropic 要区分的是细粒度的安全边界，那仍需要大规模数据收集。
- **SFT 阶段小结 (33:42)**：SFT 最擅长“提取预训练里已有的行为”——东西已经在里面，少量高质量数据就能把对的模式拉出来；加数据、哪怕事实正确的数据有时反而伤（幻觉）；所以精力该放在质量而非数量上。
 - 问答：怎么知道某种行为已在预训练里？不知道；只能证明“不在”——极冷门的编程语言 SFT 很难泛化过去。
 - 问答：SFT 抹掉特征、RL 推高特征？讲者认为界线很模糊（expert iteration 就是带花活的 SFT）；真正的区别是反馈种类：SFT 是稠密的教师监督，RL 是自学的策略监督，强化的是自己的输出，偏离得不会那么远。

6. 方法只有一句话：loss.backward()，再加一个 mask 的选择

- 全课都在讲方法，这一节最无聊：就是梯度下降，幻灯片上放张 `loss.backward()` 的图当玩笑。CME295 第 4 讲写过 SFT 的 loss——形式上仍是下一 token 预测，只是通常只在回答 token 上算。
- 但有一个值得说的细节：**要不要 mask 掉 prompt**。有些 SFT 配方也预测 prompt；mid-training 里混进的指令数据则是纯预训练、不做任何 mask (38:16 的问答)。讲者说这差别不大。

```

def build_example(tokenizer, messages, mask_prompt=True):
    ids, labels = [], []
    for m in messages:
        # chat template: role 标记
        + 内容
        seg = tokenizer.encode(f"<|{m['role']}|>\n{m['content']}<|end|>\n")
        ids += seg
        train_on = (m["role"] == "assistant") or not mask_prompt
        labels += seg if train_on else [-100] * len(seg) # -100: 不算 loss
    return ids[:-1], labels[1:] # 右移一位, 交给普通的交叉熵
# loss = F.cross_entropy(logits.view(-1, V), labels.view(-1),
ignore_index=-100)

```

一条对话按 chat template 拼成 token 串，user 段的标签设成 -100 就被交叉熵忽略；把 `mask_prompt` 关掉，就是 mid-training 里"连 prompt 一起预测"的做法。

7. Mid-training: 把指令数据塞进衰减期, "base model"从此名不副实



- **趋势：**把指令微调变成预训练的一部分（36:14）。大量高质量数据、甚至指令数据被混进训练末尾——通常是学习率衰减那段。好处两个：指令微调的规模可以做大，高质量数据得到强调。效果显著，据

讲者所知所有人都在做，多数模型报告里都有"mid-training"或"第二阶段预训练"。

- **讲者的 pet peeve**：现在谁说"这是个 base model"都有点撒谎——传统上它指"在互联网数据上预测下一个词"，而今天的 base model 已经在 UltraChat 之类专为聊天合成的数据上训过。
- **MiniCPM 的两阶段图 (37:46)**：左边是标准预训练配比，右边切到更高质量也更"聊天"的一套——Stack Exchange 问答、UltraChat、各种 SFT 数据，通用预训练数据比例下调。预训练、指令微调、高质量数据三者的边界就这样被抹平。第 11 讲讲过 MiniCPM 的 WSD 学习率，这是同一张图的数据侧。
- **问答：衰减期的数据质量该更低吗？** 恰恰相反——衰减期是整个训练里最重要的一段：离部署最近，学习率也最低，所以最高质量的数据要放这里。书、中文语料、Wiki、Stack Exchange 通常都算高质量。
- **问答：配比怎么定？ (39:48)** 预训练和 post-training 的配比都是大量试错——有算法论文，但讲者认为不太可靠。两阶段训练的好处正在于此：mid-training 比完整预训练短得多，左边跑一次的算力右边能跑约 10 次；实际流程往往是在衰减期做一堆便宜的消融、得到各域的质量估计，再反推回预训练配比。从消融结果到配比可以拟模型但常常很脆，实际更像逐域消融、按影响排序、再决定放什么。
 - 为什么不把预训练整段换成高质量数据？token 不够——拿 Wikipedia 当整个预训练集，数据就用光了（第 13 / 14 讲讲过各来源的规模）。
 - 泄露出来的例子：Meta 因使用书籍数据被诉，法庭文件里就有研究员对各书籍子集做消融、估计各自用处的记录——配比就是这么定的。

8. RLHF：从"拟合分布"换到"最大化奖励"

- **一个概念转换 (42:20)**：预训练是生成式建模——拿一堆序列拟合分布；SFT 只是换了分布，仍在拟合。RLHF 不再是"拟合分布"而是"最大化奖励"：找一个策略让它在某个下游奖励上得分最高——用户停留时长也好、解出的数学题也好——有没有模仿某个分布不关心。后果很实在：策略可以把每个 prompt 的分布坍缩到一个点，只要奖励高就没问题。第 13 节的坍缩与校准问题都从这里来。
- **为什么不永远收 SFT 数据 (43:52)**
 - **人说的和人写的不一样**：讲者早年让自由撰稿人做新闻摘要，结果有几位更喜欢 ChatGPT 前身模型的摘要而不是自己写的。团队起

初怀疑他们没认真干活，访谈后发现他们看了模型输出说"这确实更好"——他们是专业写手、摘要质量也核过，只是人不是最优系统，评判和生成是两种能力。所以有时打分比写示范更值。

- **验证比生成容易**：数学是典型——验证一个证明比写出来容易得多，DeepSeek 等就沿着"模型自我验证"的路走。这一块留给第 16 讲的 RLVR。

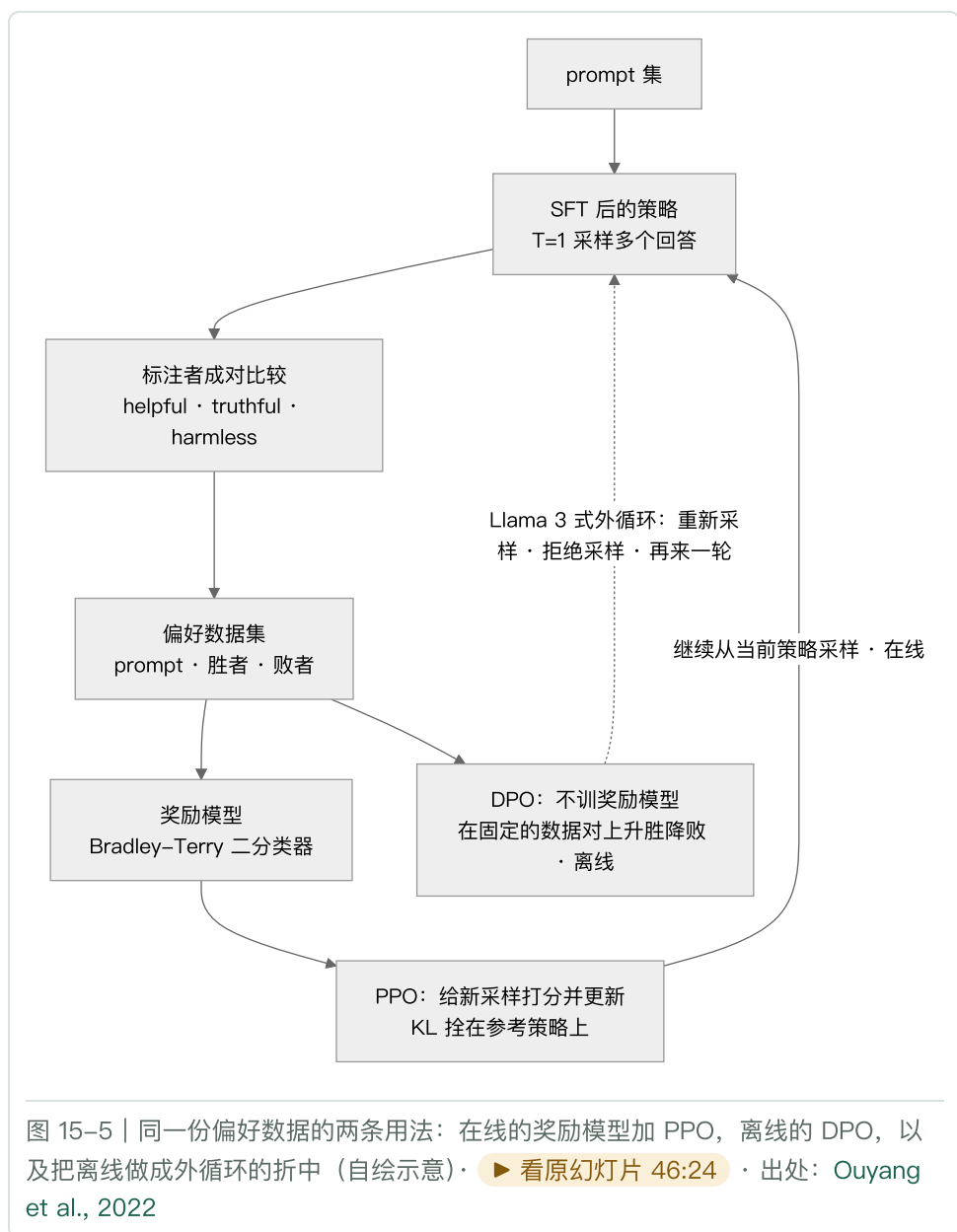
$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r(x, y)] - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

x 是 prompt, y 是策略 π_{θ} 采出的回答, r 是奖励 (人或奖励模型给的分), π_{ref} 是 SFT 后的参考策略, β 控制拴得多紧; 讲者说这就是 InstructGPT 的式 2——第二项的意思只有一句: 别离预训练出来的模型太远, 免得退化。

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l)) \right]$$

y_w 、 y_l 是同一 prompt 下标注者选出的胜者与败者, σ 是 sigmoid; Stiennon et al. 的奖励模型就是这样一个"哪个更好"的二分类器, 训完之后拿它当 r 去爬山。

9. 偏好数据：流程、指南，以及标注者到底是谁



- **流程 (46:24)**：SFT 后的模型已会听指令，给它 prompt，用 temperature 1 采几个回答（SFT 刚结束时模型仍相当多样）；标注者给出排序、有时只是二选一；在排序上训奖励模型；再用标准 RL 最大化它的分数。绕奖励模型一道，是因为训一个验证器可能比直接训一个“做得好”的模型容易。
- **标注指南**：最后一次能窥见工业界细节的是 InstructGPT 附录——标注者要在 helpful、truthful、harmless 之间权衡：有帮助指表达清楚、注意国际化、别太长；真实指别幻觉；无害指对可疑请求的拒答要加分。另一份是泄露的 Google Bard 指南：结构类似，但用 Likert 量表而非成对比较。
- **标注者在往上走 (48:58)**：整体向专家、向高成本移动。Scale AI 一个平台的调查里，本科或硕士学历占七成多，众数年龄约 35 岁，常做

创意写作和技术写作。更进一步是近一两年的定制标注：实验室要把系统推进白领工作，就需要医生、律师来写 SFT 数据和评回答——OpenAI 等公司的内部项目里，各主题的中位时薪超过 50 美元，专家超过 100 美元。但廉价标注也没消失——是一座金字塔，底层还曾让 Scale AI 的外包惹上麻烦。

- **难在哪**：最难的是“可验证”的标注者——确保没偷偷用 AI，几乎防不住；时间压力也让“真正正确”难以保证：Bard 那次泄露伴随劳资纠纷，标注员抱怨要在一分钟内核对一整段长回答。
- **标注者决定模型的观点 (53:03)**：post-training 是出厂前最后一道塑形，标注者的影响大得惊人。讲者与 Percy 合带的博后拿标准民意调查问题模型，看它最像哪个人群——各类 base model 接近新教徒和天主教徒、远离佛教徒和印度教徒；post-train 之后反过来，更像佛教徒、印度教徒和无神论者。为什么？翻 InstructGPT 附录的标注者构成：大量东南亚标注者加美国西海岸的人——正好对上。

小注：应为 Santurkar et al., 2023 《Whose Opinions Do Language Models Reflect?》(OpinionQA)。

- **更隐蔽的传递**：用一个被训成“喜欢猫头鹰”的模型生成一批看似无关的数据，在上面训练的模型也会继承这个偏好。这类“潜移默化”的转移很难查；讲者也承认“模型政治观点”这类研究本身有时相当脆弱。

小注：猫头鹰的例子出自 Subliminal Learning (Cloud et al., 2025)；讲者口中的 emergent misalignment 另有所指 (Betley et al., 2025：在不安全代码上微调后模型整体失准)。两者都是“数据里看不见的偏差被学走”，讲者把名字合到了一起。

- **专家与众包错的地方不同 (55:37)**：Hosking et al. 比较真正在乎做对的专家和平台上随机的众包者——非专家显著更看重格式；事实性、不一致这类实质错误由专家标出。事实性本来就难查，没有专家自然没人查。

小注：应为 Hosking et al., 2023 《Human Feedback is not Gold Standard》。

- **问答：怎么衡量标注者质量？** 没有金标准：要么靠足够详细、半客观的指南（定义事实、给出“Google 前三页搜不到反例”这类可操作标准）；要么看标注者间一致性——但它只说方差不说偏差，就算为零也不知道问题问对没有，或者大家都在用 ChatGPT。

- **问答：为什么招高学历标注者？** 主要因为有些数据只有这些人能标（核对法律引用得有律师）；平台整体也在往高质量走——LLM 变强后，不当面盯着，人就会用最便宜的 LLM 生成像样的答案，不少标注公司的卖点就是“我们有真人、替你验证”。

10. 让模型来标：从 AlpacaFarm 到 Zephyr 的教训

- **模型标注很好用 (59:43)：**不在前沿、目标是追上前沿时，模型标注非常好——现代模型多半比随便找的众包者强。领域专用模型没有独特优势，因为很难做得比强开源模型更好。
- **AlpacaFarm 的对照 (1:00:15)：**GPT-4 刚出时，讲者和学生把它的标注与精心整理的人工标注对比——系统排名相近，人与模型的一致度接近人与人的众数，成本低一个数量级。

小注：论文摘要 (Dubois et al., 2023)：模拟标注比众包便宜 50 倍，并收集了 10k 对真实人类偏好做校验。

- **Zephyr 的教训 (1:01:17)：**Hugging Face 造 Zephyr 时决心不做模型蒸馏，去找 OpenAI 用的那些供应商收人工数据，结果极其耗时、昂贵，效果还不比模型标注好，最终还是用了 AI 反馈——讲者认为很有说服力，他们认真试过了。问答里有人说 7B 现在算小；讲者答当时 7B 是体面的开源规模，换更大模型突然出现差异的可能性很低。
- **现状：**UltraChat 和 UltraFeedback 成了标配；Tulu 3 整条流水线全用模型标注。只要目标是追赶，人工数据基本没有位置；想推前沿，除有限场景外仍得靠人。
- **不只是蒸馏：**Anthropic 早期的 Constitutional AI 用提示让模型自己生成安全数据、再训自己，是最早的自举循环；Self-Instruct 是能力侧的同类想法。但律师、科学家那种世界知识模型自己生不出来，只能找人。
- **模型评审有和人一样的偏差，有时更糟 (1:04:22)：**把回答长度一路拉长，模型评判的胜率会一直涨——散点图里唯一的离群点是 ChatGPT 3.5，它不是靠长度而是真的更好。另一篇论文表明，只按长度做 RLHF 在很多 benchmark 上就能拿不错的分。

小注：后者应为 Singhal et al., 2023 《A Long Way to Go》；长度—胜率图应出自 AlpacaEval 系列 (Dubois et al., 2024 的 length-controlled 版本正是为此而设)。

11. 算法：PPO 的来路，和 DPO 的五步推导

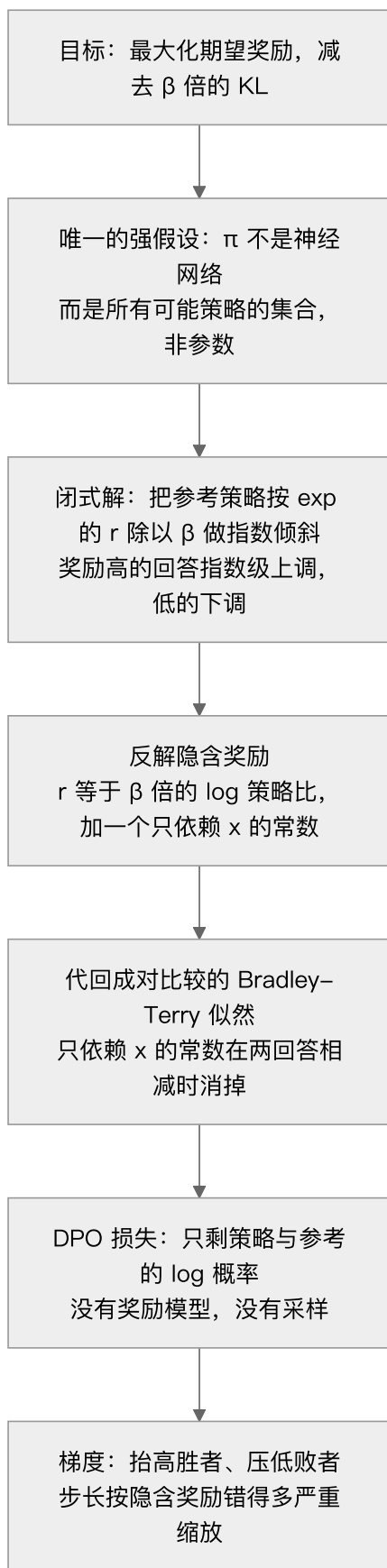


图 15-6 | DPO 的推导链：一个非参数假设换来闭式解，再把奖励用策略表示掉（自绘示意）· [▶ 看原幻灯片 1:11:31](#) · 出处：Rafailov et al., 2023

- 这是"婴儿级"RL (1:05:53)：目标是让从策略里采出的回答奖励最大；有人会说这是 bandit 不是多轮的真 RL——讲者说没错，所以算法也简单。CME295 第 5 讲和 CS224R 第 9 讲已推过 PPO 与 DPO，这里只记讲者的推进顺序和他强调的直觉。
- 从 policy gradient 到 PPO：

$$\nabla_{\theta} \mathbb{E}_{y \sim \pi_{\theta}} [r(x, y)] = \mathbb{E}_{y \sim \pi_{\theta}} [r(x, y) \nabla_{\theta} \log \pi_{\theta}(y | x)]$$

把梯度推进期望里得到右边：对采出的回答求 log 概率的梯度，再用奖励加权——讲者的说法是"看起来就是加了权的 SFT"。

- 问题是每走一步都得重新采样，而采样贵——推理复杂难做，训练反而算力密集、好办（第 10 讲）。于是想采一次、多用几步，这就是 off-policy；但走太远，局部的奖励估计会爆掉。TRPO：做 policy gradient 但别离当前策略太远，靠 importance weighting 修正；PPO：距离约束太难处理，换成启发式的 clipping，不鼓励跑到离原策略太远的地方。细节留到第 16 讲，作业里要用。
- 多年来大家想甩掉 PPO (1:08:56)，讲者列出几种失败的尝试，免得你重蹈覆辙：给好回答前置 good token、坏回答前置 bad token，生成时只用 good 前缀——把 RL 归约成 SFT，不行；只在好的那半上做 SFT，效果也不好；训个奖励模型、让 LM 采样、只留奖励模型挑中的再 SFT——不如 PPO，但多少有点用。
- DPO 的思路：去掉 PPO 里复杂的两样——奖励模型和一切 on-policy 的东西。直觉是深度学习里一切都是朝好的方向走梯度：对胜者做 SFT，对败者做"负 SFT"。最朴素的做法，但两项权重配对了就是个相当好的算法。推导只有五步（图 15-6）：

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

把 π 当成所有可能策略的集合（非参数，能逼近任何分布），带 KL 的目标就有闭式最优解：参考策略按奖励做指数倾斜——奖励好的回答指数级上调、差的指数级下调， $Z(x)$ 是让它归一化的配分函数。

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

把上式反过来解，得到"能诱导出这个最优策略的奖励"： β 倍的策略与参考之间的 log 比，外加一个只依赖 x 的常数——这个常数在成对比较里会被减掉。

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

把隐含奖励代回奖励模型的 Bradley–Terry 损失（第 8 节第二个式子），奖励模型消失了，只剩策略和参考模型在胜者、败者上的 log 概率——左边那项是"好的东西"，右边是"坏的东西"，两者互相制衡。

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E} \left[\underbrace{\sigma(\hat{r}_{\theta}(x, y_l) - \hat{r}_{\theta}(x, y_w))}_{\text{隐含奖励错得多、权重越大}} \left(\nabla_{\theta} \log \pi_{\theta}(y_w | x) - \nabla_{\theta} \log \pi_{\theta}(y_l | x) \right) \right]$$

r_{θ} 就是上面用策略表示的隐含奖励。讲者认为这是 DPO 最直观的形式：对每一对标注，抬高胜者的似然、压低败者的似然，步长由"隐含奖励模型错得多严重"决定——已经给胜者很高奖励就走小步；若两者概率几乎相同、模型完全没分出好坏，就走大步。

```
def dpo_loss(policy, ref, x, y_w, y_l, beta=0.1):
    # 每个 seq_logp 是回答 token 的 log 概率之和 (prompt 部分不算)
    pi_w, pi_l = policy.seq_logp(x, y_w), policy.seq_logp(x, y_l)
    with torch.no_grad():
        # 参考模型冻结
        ref_w, ref_l = ref.seq_logp(x, y_w), ref.seq_logp(x, y_l)
    margin = beta * ((pi_w - ref_w) - (pi_l - ref_l)) # 隐含奖励之差
    return -F.logsigmoid(margin).mean() # 再 loss.backward()
即可
```

一个 batch 的偏好对只需要四次前向（策略和参考各在胜者、败者上一次），没有采样、没有奖励模型——"看起来就像 SFT"，这正是讲者说它简单的地方。

12. DPO 还是 PPO? Llama 3 的外循环与一堆差别不大的变体

- **答案：多半不重要 (1:14:32)** ——除非你在前沿训最好的模型。
DPO 对 Llama 够用，对讲者也够用。翻 Llama 的技术报告：SFT → DPO → 用 DPO 后的模型生成候选、做拒绝采样 → 重复，外面套一层循环，但核心的 RLHF 原语就是 DPO。

小注：Llama 3 论文里这套 SFT + DPO 的循环做了六轮，每轮重新收偏好数据；这就是图 15–5 里"离线算法做成外循环"的折中——每轮的数据对当前策略而言是新采的。

```
policy = sft(base, demos) # 第一轮 SFT
for round in range(K):
    # Llama 3 式外循环
    cands = {x: sample(policy, x, n=8, T=1.0) for x in prompts}
    pairs = annotate(cands) # 人或奖励模型选出胜者、
    # 败者
    best = {x: top1(reward_model, ys) for x, ys in cands.items()}
    policy = sft(policy, demos + best) # 拒绝采样出的样本回灌
```

SFT

```
policy = dpo(policy, ref=policy, pairs=pairs) # 再在新数据对上做
```

DPO

每一轮的候选都从当前策略采出，所以离线的 DPO 也逐轮逼近在线；讲者的意思是"外面有循环，里面还是 DPO"。

- **变体**：SimPO 改了权重——不用 π_{ref} ，换成按回答长度归一化的项；length-normalized DPO 按长度归一化以避免长度作弊。讲者的判断：这些变体似乎都没有很大差别。

小注：SimPO 是 Meng et al., 2024；字幕里的"Y of L normalizer"应是幻灯片上的 $|y|$ 长度项。

- **结果极其依赖实验设置**：同在 AI2，一篇论文说从 DPO 换到 PPO 更好，Tulu 2 的论文又说 DPO 做对了能胜过 PPO——怎么执行决定谁赢，结果很脆。该记住的是：这些变体都足够接近"对的东西"，只要步长设对，"朝好的走、从坏的退"在很多场景下都够好。

小注：前者应为 Iverson et al., 2024 《Unpacking DPO and PPO》，后者为 Tulu 2 (Iverson et al., 2023)；Tulu 3 最终用的是 DPO 加上 RLVR。

13. RLHF 的三个坑：过度优化、模式坍缩、校准

- **过度优化 (1:16:34)**：InstructGPT 刚出时人们真问过"能不能靠 RLHF 一路到超级智能，多收点赞踩就行？"结果是很难——把 RLHF 往前猛推，就开始过拟合学到的奖励模型。所以 KL 正则在很多场合至关重要，尤其当你的优化器很强的时候。

小注：奖励模型过拟合随 KL 增长的定量研究见 Gao et al., 2022 《Scaling Laws for Reward Model Overoptimization》。

- **模式坍缩**：RL 后的模型多样性明显下降，集中在少数几种输出上——回到第 8 节的概念：RLHF 模型不再建模一个自带多样性的分布，它是一个只要奖励高就可以坍缩的策略。近年这是 RLHF 模型真正头疼的问题。
- **校准变差**：GPT-4 时代 OpenAI 少有地公开了一张"我们还有几个未解问题"的图，其中之一是 RLHF 之后模型不再校准。讲者认为至今没人真正解决；Anthropic 的论证是它天然会失去校准，有时能重新校准、有时不能。

小注：应为 GPT-4 技术报告里 MMLU 上 RLHF 前后的校准曲线，以及 Kadavath et al., 2022 《Language Models (Mostly) Know What They

- **为什么第 16 讲更要紧：**RLVR 靠熵与探索走遍可能解法，熵坍缩正是那里的核心矛盾（CS329A 讲过 GRPO / DAPO 对熵的处理）。
- **收尾：**RLHF 的数据收集同样很难——post-training 之乱，大半在于好数据难得；算法复杂，PPO 尤其难，好在有更简单的 GRPO（作业里用）；最大的问题是过度优化。下一讲的问题：有没有一种奖励怎么优化都不会过度优化——算力一直堆、性能单调涨？这正是 RLVR 影响如此之大的原因。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
SFT 数据谱系总览	左上到右下按时间：FLAN → Self-Instruct → Alpaca / Vicuna → OpenAssistant → WizardLM / Tulu 3 → agentic SFT	7:40	—
FLAN 样例	邮件正文后面挂着"写主题行"；摘要比 ChatGPT 短得多，细节在输入里找不到	11:44	Wei et al., 2021
Alpaca 与 OpenAssistant 样例	前者输入自然、输出长而"聊天"；后者专家长回答，末尾带一条文献引用（第 4 节的两种机制）	14:46	Alpaca · Köpf et al., 2023
风格对能力表	换 post-training 数据集，AlpacaEval 那列大幅波动，标准 benchmark 那几列几乎不动	21:25	—
Llama 2 的安全 SFT 与两条曲线	violation rate 对 false refusal rate 的权衡；注意描述里没有样本数	29:03	Touvron et al., 2023
500 条安全样本曲线	横轴是安全样本数，恶意指令的执行率在几百条处骤降	32:40	Bianchi et al., 2023 (应为)
MiniCPM 两阶段配比图	左边通用网页配比；右边衰减期切到 Stack Exchange QA、UltraChat 等，通用数据比例下调	37:46	Hu et al., 2024
InstructGPT 标注指南与 Bard 泄露指南	helpful / truthful / harmless 各自的措辞；Bard 用 Likert 量表而非成对比较	47:55	Ouyang et al., 2022
标注者画像与年薪	本科 / 硕士七成多、众数年龄约 35；中位年薪 50 美元以上、专家 100 美元以上	49:28	—
观点对齐表	base model 列接近新教 / 天主教；post-train 后右三列转向佛教 / 印度教 / 无神论	53:34	Santurkar et al., 2023
专家对众包的错误类型表	非专家那行格式类错误偏红（过多）；事实性、不一致集中在专家那行	55:37	Hosking et al., 2023

图	看什么	跳转	出处
长度对胜率散点	回答越长模型评判的胜率越高；唯一的离群点是 ChatGPT 3.5	1:04:52	—
DPO 梯度式	两个 log 概率梯度相减，前面的 sigmoid 权重就是"隐含奖励错了多少"	1:13:32	Rafailov et al., 2023
GPT-4 校准图	RLHF 前贴近对角线，RLHF 后明显偏离——讲者说至今没人解决	1:18:08	GPT-4 技术报告

提到的工作

名称	在本讲里的作用
Learning to summarize from human feedback (Stiennon et al., 2020)	讲者推荐读附录的 RLHF 论文：标注指南、成对奖励模型都在里面
Anthropic HH-RLHF (Bai et al., 2022)	另一份写得详细的早期 post-training 材料
InstructGPT (Ouyang et al., 2022)	两段式配方；式 2 的带 KL 目标；附录的标注指南与标注者构成
FLAN · Flan-T5	把 NLP 数据集拼成指令任务的第一代；不自然、继承缺陷、错估了规模需求
Self-Instruct (Wang et al., 2022)	让模型自己生成指令数据的先驱；能力侧的自举
Alpaca · Vicuna	蒸馏 ChatGPT；Vicuna 用线上用户分享的 prompt
OpenAssistant (Köpf et al., 2023)	维基百科式的志愿者众包；带引用的回答成为第 4 节的反面教材
WizardLM · Tulu 3	复杂的合成数据流水线；Tulu 3 是讲者眼中唯一完整公开的高性能 post-training 配方（含约 5 万条安全数据，全模型标注）
Nemotron (NVIDIA)	agentic SFT 数据的例子：并行工具调用与结构化格式
WildChat (Zhao et al., 2024)	免费聊天换日志；从中挖不安全请求与越狱，配上拒答
Llama 2 (Touvron et al., 2023)	安全 SFT 的公开描述；violation rate 对 false refusal rate
Safety-Tuned LLaMAs (Bianchi et al., 2023)	"500 条安全样本就见效"的曲线应出自这里
Instruction Following without Instruction Tuning (Hewitt et al., 2024)	"Percy 以前的学生"的论文：不给指令也能学会指令遵循（应为）
Gekhman et al., 2024	在模型不知道的新知识上微调会诱发幻觉（应为讲者所指的实证）

名称	在本讲里的作用
John Schulman 的 RL and Truthfulness 演讲 (2023)	"要校准就得依赖策略，所以需要 RL"的论证
MiniCPM (Hu et al., 2024)	两阶段训练、衰减期换配比的示例图 (第 11 讲讲过它的 WSD)
UltraChat · UltraFeedback	模型生成的 SFT 与偏好数据，如今的标配；也是"base model 名不副实"的原因
Zephyr (Tunstall et al., 2023)	Hugging Face 认真试过人工数据后仍回到 AI 反馈的 7B 模型
AlpacaFarm (Dubois et al., 2023)	GPT-4 标注对人工标注：排名相近、成本低一个数量级
Constitutional AI (Bai et al., 2022)	模型按提示自生安全数据再训自己：最早的自举循环
Whose Opinions Do Language Models Reflect? (Santurkar et al., 2023)	标注者人群决定模型观点 (讲者与 Percy 合带博后的研究，应为)
Subliminal Learning (Cloud et al., 2025) · Emergent Misalignment (Betley et al., 2025)	"猫头鹰偏好"通过无关数据传递；讲者把两个名字合在了一起
Human Feedback is not Gold Standard (Hosking et al., 2023)	专家看事实性，众包看格式
A Long Way to Go (Singhal et al., 2023) · Length-Controlled AlpacaEval	只按长度做 RLHF 也能刷分；长度对胜率的散点 (应为)
TRPO · PPO (Schulman et al.)	policy gradient → 信赖域 → clipping 的推进；PPO 细节留到第 16 讲
DPO (Rafailov et al., 2023)	去掉奖励模型与 on-policy 采样；五步推导
SimPO (Meng et al., 2024) · length-normalized DPO	换权重、按长度归一化的变体；讲者认为差别不大
Unpacking DPO and PPO · Tulu 2 (Iverson et al.)	同一个 AI2 前后相反的结论：结果取决于怎么执行
Llama 3 (2024)	SFT → DPO → 拒绝采样 → 重复的外循环；核心原语是 DPO
Scaling Laws for Reward Model Overoptimization (Gao et al., 2022)	过度优化的定量研究 (小注补充)
GPT-4 技术报告 · Kadavath et al., 2022	RLHF 后校准变差的图；Anthropic 关于能否重新校准的论证 (应为)

术语对照

English	中文
post-training	后训练：预训练之后的 SFT 与 RLHF 等阶段
mid-training	中期训练：预训练衰减期换成高质量、含指令数据的配比，目标函数不变
decay / cooldown phase	学习率衰减阶段（第 11 讲 WSD 的 D 段）
SFT (supervised fine-tuning) / instruction tuning	监督微调 / 指令微调
demonstration data	示范数据：标注者为 prompt 写的参考回答
distillation	蒸馏：拿更强模型的输出当训练数据
synthetic data	合成数据：由模型生成的训练数据
crowdsourcing	众包
annotator / annotation guideline	标注者 / 标注指南
inter-annotator agreement	标注者间一致性：只反映方差，不反映偏差
tail knowledge	尾部知识：模型很可能不知道的冷门事实
misgeneralization	误泛化：学到模板却没学到内容
hallucination	幻觉
violation rate / false refusal rate	违规率（放过的坏请求）/ 误拒率（拒掉的好请求）
jailbreak	越狱：绕过安全限制的 prompt
chat template	对话模板：把多轮消息拼成 token 串的格式
loss masking	损失遮蔽：prompt 的 token 不算 loss
data mixture / ablation	数据配比 / 消融实验
engagement signal	参与度信号：停留时长之类的产品指标
RLHF	基于人类反馈的强化学习
reward model	奖励模型：在偏好数据上训出的打分器
pairwise preference / Likert scale	成对偏好 / 李克特量表（逐条打分）
Bradley-Terry model	成对比较的概率模型：胜率是奖励差的 sigmoid
policy gradient	策略梯度
on-policy / off-policy	在线策略（每步重新采样）/ 离线策略（复用旧样本）
importance weighting	重要性加权：用概率比修正旧样本
TRPO / PPO / clipping	信赖域策略优化 / 近端策略优化 / 裁剪
DPO (direct preference optimization)	直接偏好优化
implied reward	隐含奖励： β 倍的策略与参考的 log 比
partition function	配分函数：闭式解里的归一化项 $Z(x)$

English	中文
KL regularizer / reference policy	KL 正则 / 参考策略（SFT 后冻结的那份）
rejection sampling	拒绝采样：采多个、只留奖励最高的
expert iteration	专家迭代：采样、筛选、再 SFT 的循环
bandit	老虎机问题：单步决策的 RL，没有多轮状态
over-optimization	过度优化：过拟合学到的奖励模型
mode collapse	模式坍缩：输出多样性丧失
calibration	校准：置信度与正确率一致
RLVR	可验证奖励的强化学习（第 16 讲）
agentic SFT / tool call	面向 agent 的 SFT / 工具调用
temperature	采样温度：RLHF 采样用 1

字幕勘误

"Flon" → FLAN; "Exxon" → XSum; "Chad GPT / chat GPT" → ChatGPT; "GPT-01" → o1; "Open Assistant" → OpenAssistant; "Wizard LM" → WizardLM; "Tulu" → Tülu; "Olmo" → OLMo; "mini CPM" → MiniCPM; "ultra chat / Ultra Feedback" → UltraChat / UltraFeedback; "Deep Seek" → DeepSeek; "Instruct GPT" → InstructGPT; "spear fishing" → spear phishing; "Claude code" → Claude Code; "hugging face" → Hugging Face; "224N" → CS224N; "Y of L normalizer" → 幻灯片上的 |y| 长度归一化项; "Boo-Buddhists" → Buddhists; "book Chinese" → 应是 books、Chinese 两类数据; "Blue Book annotation" → Bluebook（法律引用规范）; "CNN Daily Mail" → CNN/DailyMail。另：讲者把 Subliminal Learning 的猫头鹰例子说成 emergent misalignment，是口误不是字幕错。

带走的问题

1. 一条带引用的高质量回答同时教"知识"和"格式"。如果只想教格式不教知识，数据该怎么改（把引用换成模型已知的？换成占位符？）？为什么讲者说"知道自己知道什么"必须依赖策略本身、SFT 做不到？
2. 500 条安全样本就能大幅压低违规率，Llama 2 和 OpenAI 却仍收几千到几万条——多出来的数据买的是什么？在违规率对误拒率的

Pareto 前沿上, "怎么 kill 一个 Python 进程"这种样本在往哪个方向推?

3. mid-training 把 UltraChat 混进衰减期、连 prompt 一起预测; SFT 在同类数据上只算回答的 loss。两者的差别到底在哪——学习率、数据比例还是 mask? "衰减期跑 10 次消融、反推预训练配比"隐含了什么假设?
4. DPO 的推导假设策略是非参数的、闭式解存在, 真实的 π_θ 是有限容量的网络。这个假设失效时隐含奖励和真实奖励差在哪? "在线对离线"的差距为什么能被 Llama 3 式的外循环部分补回?
5. RLHF 最大化奖励而非拟合分布, 于是坍缩、失校准; 第 16 讲的 RLVR 却需要熵来探索。既要 RLHF 的风格塑形又要 RLVR 的推理能力时, 两者按什么顺序做、KL 拴在哪个参考策略上?

CS336 第 16 讲 | 后训练：RLVR (Post-Training: RLVR)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 16 讲
(课程日程表标注 2026 年 5 月 20 日；视频里没有出现日期)，后训练两讲中的第二讲，结尾说"下周见" 视频：<https://www.youtube.com/watch?v=dIFAi87Ws4E> (1:15:50；英文字幕为自动生成，PPO / GRPO / CoT / Kimi / Qwen 这几个最核心的词几乎全错，见文末勘误) 讲者：Tatsunori Hashimoto (全程；开场说"上一讲我用 overoptimization 收的尾"，讲 reward hacking 时说"我和一个学生做过 RL on Lean") 课程主页：<https://stanford-cs336.github.io/> · 本讲围绕四份技术报告展开：DeepSeekMath (GRPO 的出处) · DeepSeek-R1 · Kimi K1.5 · Qwen3 · Qwen3-Coder-Next (项目页)；批评 GRPO 两个归一化项的 Dr. GRPO

一句话：RLHF 的天花板是 reward model——它是用有限的人工标注拟合出来的，算力一多就被 overoptimize；RLVR 把 reward 换成"答案对不对、测例过不过"这类不会被拟合坏的信号，于是可以像 AlphaGo 那样把算力一直往里加，而算法本身几乎没变：还是 policy gradient，还是"按 reward 加权的 SFT 更新"。GRPO 做的只是把 PPO 里最难伺候的 value model 拿掉，用同一道题采 G 个回答、组内 z-score 当 advantage，一页代码就能写完；代价是它除以 std、按长度归一化这两处不在 policy gradient 的"合同"里——前者给太易和太难的题加权，后者让答错的回答越写越长 (Dr. GRPO 指出并去掉)。四份报告读下来配方高度一致：mid-trained base → 少量长 CoT 冷启动 SFT → 可验证奖励的 reasoning RL → 末尾 RLHF；差别在数据 (best-of-8 难度过滤、掌握即剔除、Qwen 3 只用 4,000 道题)、长度控制 (Kimi 反过来压短 CoT) 和 reward 的可靠性 (数学答案等价最后还是靠 reward model；agent 会翻 git 历史找答案，Lean 编译器也能被骗)。工程上最难的是 RL 把训练和推理两套系统拼在一起：最长的 rollout 拖住整批、权重要在两边搬、一复用 rollout 就变成 off-policy。

时间轴

时间	内容
0:05	开场：RLVR 在后训练里的位置；OpenAI 用 thinking model 解决一个 Erdoš 问题的新闻；上一讲的 overoptimization 与 AlphaGo 的对比
3:11	本讲分两半：先讲算法 (PPO、GRPO)，再读开源技术报告看它们怎么落地

时间	内容
3:43	PPO 复习：policy gradient 是一切的根；PPO 是 RL 的 workhorse，但有"37 个实现细节"
8:50	一份学生实现里的坑：KL 截到零才稳、GAE 取 $\gamma = \lambda = 1$ 退化成 bandit；value model 和模型一样大
11:54	为什么不干脆用 DPO：它只对 pairwise 的 Bradley-Terry 反馈有效
13:28	GRPO：去掉 value model，组内 z-score 当 advantage；在线时 clipping 失效；一页代码与作业要写的几步
19:09	DeepSeekMath 的结果；GRPO 不是严格的 policy gradient：除以 std 和按长度归一化各做了什么
26:22	DeepSeek R1-Zero：base + GRPO + accuracy / format reward；给"越想越长"和"aha moment"泼冷水
31:29	R1 的生产流水线：长 CoT 冷启动、语言一致性奖励、末尾 RLHF；蒸馏到 Qwen 2.5 / Llama；PRM 和 MCTS 的失败尝试
38:42	问答：长度归一化对答对的样本做什么；Dr. GRPO 里变长的是答错的回答
39:43	Kimi K1.5：数据课程与 best-of-8 难度过滤；从 DPO 式推导出发，落到 GRPO 式的组均值 baseline
46:22	Kimi 的长度观：CoT 要压短；长度奖励怎么设计才不让答错的题"死掉"；掌握即剔除；答案等价靠 reward model
51:25	RL infra：训练加推理两套系统；最长的 rollout 拖住整批；on-policy 与复用 rollout 的取舍
53:58	Kimi 的结果：OmniMath 上不再变长但分数继续涨；expert iteration 输给 RL
55:31	Qwen 3：整条流水线；难度、无 CoT 可解、去污染三重过滤；RL 只用 4,000 道题；thinking / non-thinking 融合、早停思考与思考预算
1:01:11	Qwen3-Coder-Next：agent 后训练没有新算法，全在 mid-training 数据；四个专家模型再蒸馏回一个
1:05:49	Reward hacking：agent 翻 git 历史找答案、加 remote 绕禁令；Lean 编译器也不是对抗鲁棒的；3B 激活参数拿到 70.6%
1:08:52	总结与问答：一切在于 reward；thinking mode 是同一个模型；mid-training 的作用；专家蒸馏；长上下文扩展；多领域串行还是并行

核心内容

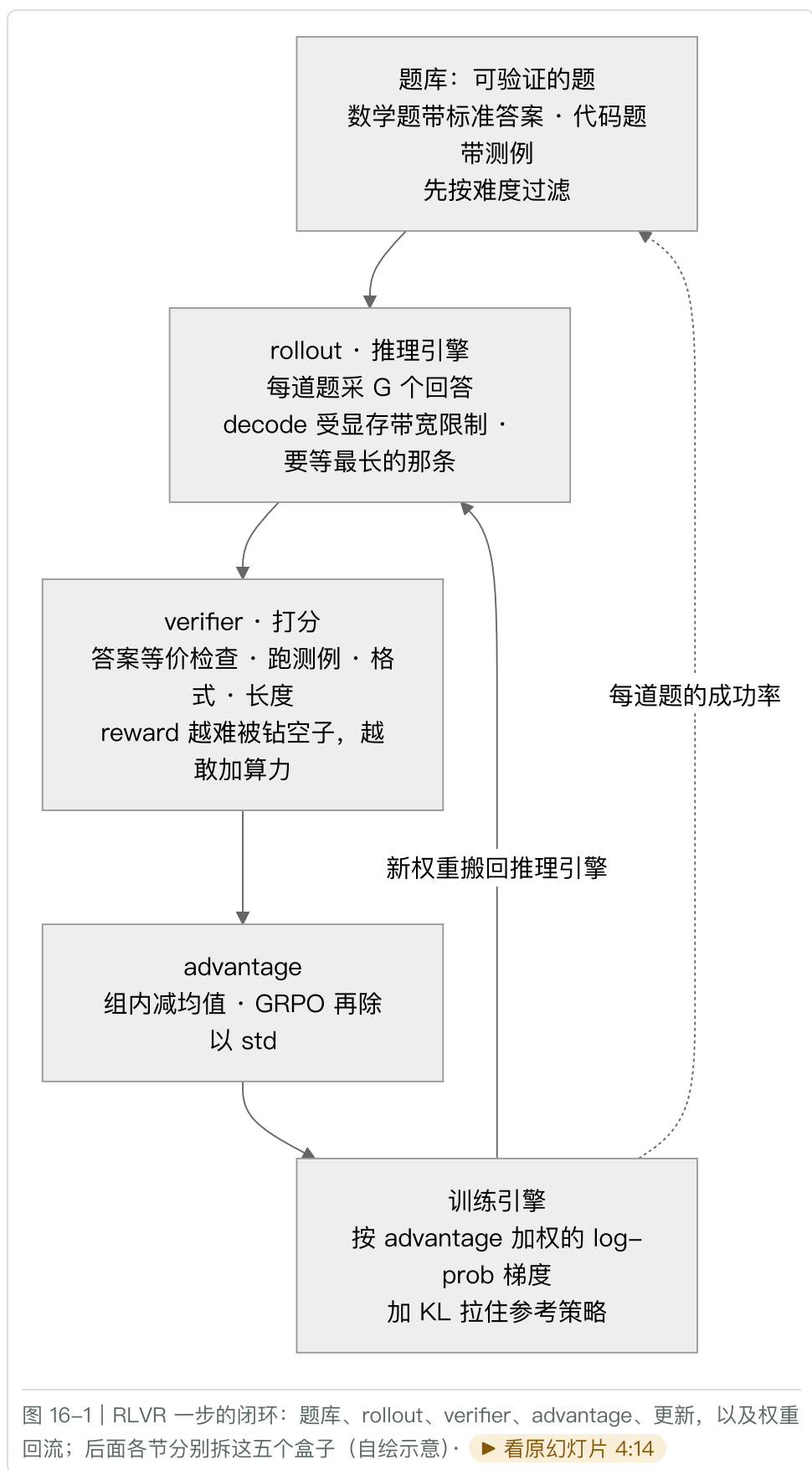
1. 为什么 RLHF 走不到头，而"可验证"能走到头

- **这一讲的位置**：第 15 讲讲完 instruction tuning 和 RLHF，得到的是 ChatGPT 那一代助手；缺的是 thinking model——能写很长的 chain of thought (CoT)、解决数学和代码这类难而可验证的问题。讲者开

场提到当天的新闻：OpenAI 宣布用一个 thinking model 解决了一个 Erdoš 问题（一个公开的数学难题）——那背后就是这一讲的东西。

- **上一讲的"下坠结尾"**：RLHF 的 reward model 是用偏好标注拟合出来的，本质上受标注量限制。你不能对着同一个 reward model 无限加算力——迟早 overfit (overoptimization)，再怎么正则化都只是推迟。这让 RL 信徒很沮丧：AlphaGo 这类 RL 的高光时刻，靠的正是"往里加算力就一直变好"。
- **区别在哪**：AlphaGo 优化的就是它想要的东西——围棋的胜负没有任何含糊，所以目标一直在涨就一直是好事；讲者的说法是那更像 search 问题，RLHF 更像 learning 问题（他自己也说这个二分不严格）。形式化数学、乃至自然语言写的数学，都有"可验证"这个性质，因此对 RL 友好得多。
- **预告**：算法上和 RLHF 差别不大，落点却大不一样。本讲两半：前半是算法基础（PPO、GRPO——作业要用），后半读四份开源技术报告，看前半讲的东西在报告里怎么体现。
 - 一句话对照：CS224R 第 3—4 讲推过 policy gradient 和 baseline，第 9—10 讲推过 RLHF / DPO；CS329A 第 6 讲和 CME295 第 6 讲讲过 GRPO / DAPO 和推理模型的全景。这一讲不再推公式，而是问：从零搭一个能跑的 RLVR 系统，要哪些零件、算力花在哪、会在哪坏。

2. 一个 RLVR 系统由什么组成



• 五个零件（图 16-1），也是后面各节的目录：

1. 题库：只收有标准答案或测例的题，并按模型当前的解题率过滤（第 8 节）；

2. *rollout*: 推理引擎对每道题采样一组回答——这是算力大头之一，而且是第 10 讲讲过的解码型负载：一个 token 一个 token 往外吐，受显存带宽而不是算力限制，还得等整批里最长的那条（第 9 节）；
3. *verifier*: 给每个回答打分。听起来最简单，实际最深的坑（第 8 节、第 11 节）；
4. *advantage*: 把一组 reward 变成每个回答的权重（第 4—5 节）；
5. *更新*: 拿权重做一次"加权 SFT"，再把新权重搬回推理引擎（第 3 节、第 9 节）。

- **和 RLHF 的关系**: 讲者收尾时的话——RLHF 和 RLVR 是几乎一样的问题，区别只在 reward 是否难被钻空子 (unhackable)；正因为难被钻空子，才敢往里加多得多的算力。

3. PPO 复习：为什么人人都想摆脱它

- **一切的根**: RL for LM 最该记住的是 policy gradient，尤其是 REINFORCE 那个梯度技巧——对 reward 做梯度上升，实际操作是"按权重做 SFT 更新"，权重可正可负。这个式子会贯穿整讲，其他一切都从它派生：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} [R(x, y) \nabla_{\theta} \log \pi_{\theta}(y | x)]$$

x 是 prompt, y 是采出来的整个回答, $R(x, y)$ 是它的 reward, π_{θ} 是当前策略；式子说的是：拿当前策略采样，把每个样本的 log 概率梯度乘上它的 reward——reward 为正就多学它，为负就少学它，和 SFT 只差一个权重。

- **为什么要 PPO**: policy gradient 每走一步都得重新从当前策略采样。能不能把一批 rollout 多用几步？TRPO、PPO 就是答案（上一讲讲过 TRPO）。PPO 是 RL 的 workhorse: OpenAI 早年"RL 信徒"时期在 gym 里让人形机器人走路、后来更戏剧性的游戏 bot（应为打 Dota 2 的 OpenAI Five），都是 PPO 训的；它能对付深度 RL 里高维的状态和动作空间。
- **看着简单，实现是魔鬼**: Spinning Up 文档里的伪代码就五步——采轨迹、估 advantage、clip、更新策略、拟合 value function，谁看了都觉得一次就能写对。可一篇叫"PPO 的 37 个实现细节"的博客应该让你心里一紧：这是个对实现决策极其敏感的算法，不同库给出完全

不同的数字；还有论文指出有些实现里用的"baseline"根本不是 baseline，它改变了优化问题本身。

- **PPO 用在语言模型上更不愉快：**一张典型的框图里有 advantage 估计、experience buffer（存旧 rollout）、要训练的 value model（它又出现在 advantage 估计里，框图上同一个绿色方块画了两次）；而且 KL 项是逐 token 算的，所以这不是一个 bandit 问题，而是一个完整的多步 RL 问题。
- **一份学生实现的解剖**（讲者的学生为 RLHF 项目写的 PPO，花了好久才跑通）：外层循环没问题（遍历 rollout、算 loss、clip 梯度、走几步）；算 loss 的内层也基本照抄 PPO 更新；脏的地方在细节——KL 惩罚只有截断到零才稳定，可 KL 散度的估计本来就正负都有，截掉负值等于毁掉 KL 的意义；不截，训练立刻炸。这不是在教你怎么写 PPO，而是说明：活动部件多、梯度估计方差大，RL 算法的实现天然又脆又敏感。
- **另一个常见退化：**PPO 论文里的 GAE（generalized advantage estimation）用 γ 折扣的 reward 和一个逐 token 的 value function；实际上很多人直接取 $\gamma = \lambda = 1$ ，这个退化设定把问题又变回了 bandit，PPO 的多步结构被扔掉了。
- **跑通之后该看到什么**（作业里也一样）：reward 曲线往上走，负的 KL 正则项往下走。
- **两个结构性缺点：**一是要 hack 才稳，大厂有 turnkey 的方案，从零实现的研究者却很痛苦；二是要一个 value model 逐 token 估值——它和被训练的模型一样大，吃掉的显存本可以给模型本身或推理服务器。
- **那为什么不用 DPO：**DPO 是针对特定问题的特定解——pairwise 的 Bradley-Terry 比较。数学题天生不是成对比较；有 DPO 变体打破成对结构，但那是拿错了锤子。PPO 是什么都能砸的通用锤子。"DPO 离线、PPO 在线"的区分被夸大了：反复迭代 DPO 就是在线的。
- **结论：**社区想摆脱 PPO 的欲望有多强，看 DPO 和 GRPO 的普及速度就知道。

4. GRPO：去掉 value model，用"组"当 baseline

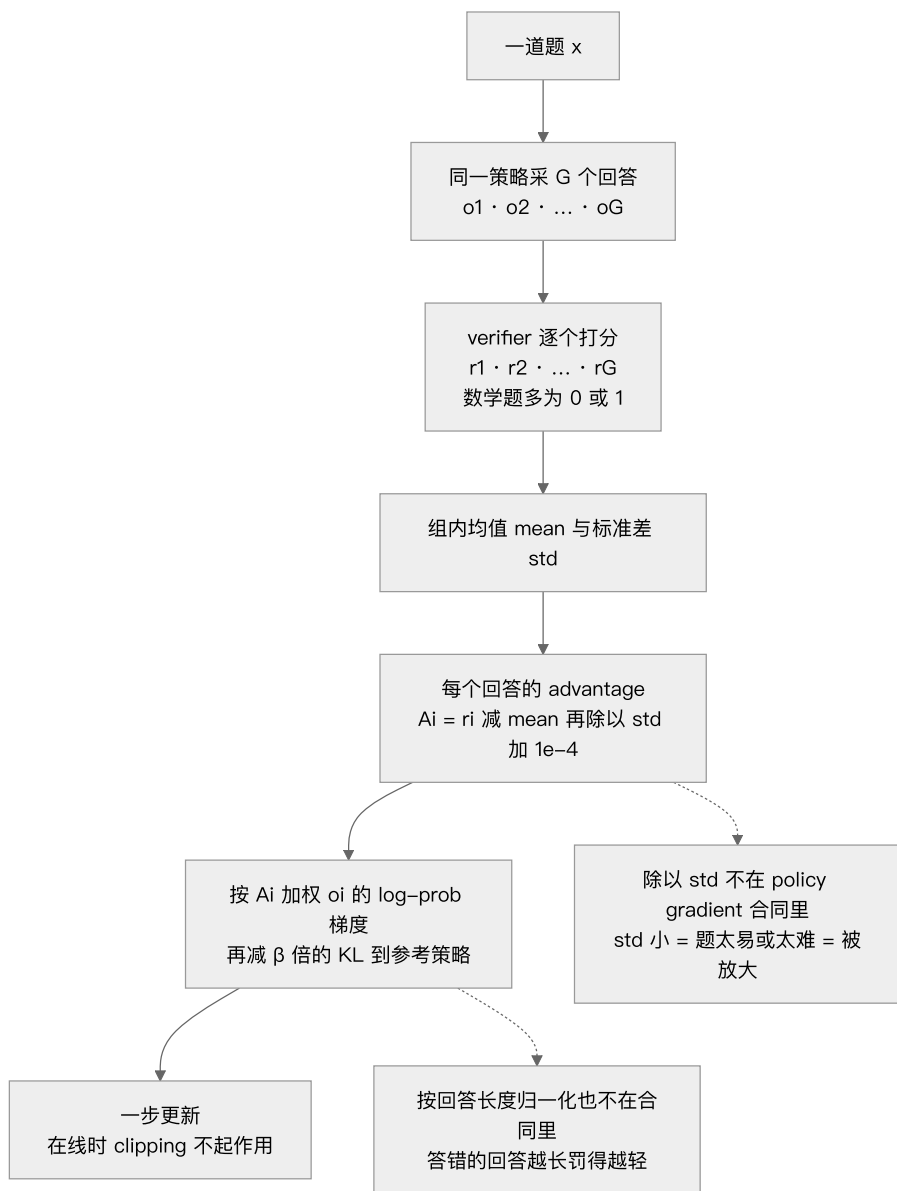


图 16-2 | GRPO 的 advantage 怎么算，以及两处偏离第一性原理的地方（自绘示意）· [▶ 看原幻灯片 15:30](#) · 出处：Shao et al., 2024

- **出处与动机：**GRPO 出自 DeepSeekMath 论文。它承认 PPO 是好主意，只改一处——把最复杂、最讨厌的 value function 拿掉。value function 是拿来当 baseline 减掉、降低梯度方差的，但它是一整个神经网络，会让训练不稳。拿掉之后 advantage 从哪来？直接用 vanilla REINFORCE 方差太大；GRPO 的办法是**组内 z-score**：同一道题采 10 个（讲者的例子）回答，看每一个比组内平均好多少——在“同一 prompt 可以多次采样”的场景里，这是最自然的 advantage。
- **什么样的 baseline 是合法的：**Sutton & Barto 里的“REINFORCE with baseline”——只要减掉的量只依赖状态（bandit 设定下状态就是 prompt），梯度方向不变，方差随 b 的选择变好或变坏：

$$\nabla_{\theta} J(\theta) = \mathbb{E}[(R(x, y) - b(x)) \nabla_{\theta} \log \pi_{\theta}(y | x)]$$

$b(x)$ 是任何只依赖 prompt 的 baseline (value function 是其中一种, 组内均值也是); 减掉它不改变梯度的期望, 只改变方差。

- **GRPO 的 advantage:**

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G) + \varepsilon}$$

r_i 是第 i 个回答的 reward, G 是组大小, ε 是为了数值稳定加的小常数 (讲者展示的实现用 $1e-4$): 一组回答全对或全错时 std 为零, 不加 ε 会除零——在数学题这种 reward 恰好为 0 的领域, 这种情况经常发生。

- **目标函数:** 论文里的写法就是 PPO 的 min-clipped 更新加一个到参考策略的 KL (KL 用一种特定的估计方式算, 讲者说这个细节不重要):

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_i) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right) \right], \quad \rho_{i,t} = \frac{\pi_{\theta}(O_{i,t} | x, O_{i,<t})}{\pi_{\theta_{\text{old}}}(O_{i,t} | x, O_{i,<t})}$$

o_i 是第 i 个回答、 $|o_i|$ 是它的 token 数, ρ 是新旧策略在每个 token 上的概率比, ε 是 clip 范围, β 是 KL 系数, π_{ref} 是参考策略。在线情形下 (每次更新前重新采样) π_{old} 就是 π_{θ} , ρ 恒为 1, clipping 永远不起作用, 整个目标退化成 "advantage 减 KL 惩罚"——一个非常简单的对象。

- **一页代码:** 没有 value function, 整个 GRPO 就是 "采 G 次、z-score、按权重做 REINFORCE 梯度"。作业里要写的几步: 给 G 个 rollout 各算 reward、组内归一化、算 KL 项、对两者之和做梯度更新; 用自动微分时要想清楚 stop-gradient 放在哪 (advantage 是常数, 不能让梯度流进去)。讲者展示了一份来自 McGill 的玩具实现: 组索引、std 的计算全部放得下半页幻灯片。

```
def grpo_step(policy, ref, prompts, G, beta, eps=1e-4):
    loss = 0.0
    for x in prompts:
        outs = policy.sample(x, n=G) # 同一策略采 G
        # 个回答
        r = torch.tensor([reward(x, o) for o in outs]) # verifier 打
        # 分, 常为 0/1
        A = (r - r.mean()) / (r.std() + eps) # 组内 z-
        # score, 当常数用
        for o, a in zip(outs, A):
            lp = policy.logprob(o, x) # 各 token 的
```

log-prob 之和

```
kl = kl_estimate(policy, ref, o, x) # 逐 token 的
```

KL 估计

```
loss = loss - (a.detach() * lp - beta * kl) / len(o)
```

```
loss.backward(); optimizer.step()
```

这份草图只为说明 GRPO 的骨架：advantage 用 detach 当作常数，梯度只从 log-prob 和 KL 流回去；除以 len(o) 就是第 5 节要讨论的按长度归一化。

- **效果** (DeepSeekMath 的图)：GRPO 的两条线（黄、蓝）明显高于 RFT (rejection fine-tuning：把模型自己生成的正确答案拿来训练，其余扔掉——作业里要实现它当 baseline)。那篇论文还显示 process supervision（不只判最终答案，也给中间步骤打分）有额外增益——后面会看到这个结论没有站住。
 - 一句话对照：CS329A 第 6 讲把 GRPO 到 DAPO 的改动（去 KL、clip 上下界不对称、动态采样、token 级 loss）讲过一遍；这里只补第一性原理上的两个疑点。

5. GRPO 到底在优化什么：两个不在“合同”里的项

- **问题**：GRPO 把 PPO 的 value function 换成了 z-score。这是个合法的 advantage 吗？按第 4 节的 baseline 定理，合法的操作只有“减一个只依赖 prompt 的量”。GRPO 做了两件多余的事：
 1. **除以 std**：不是减 baseline 而是重新缩放，打破了 baseline 的“合同”；
 2. **按长度归一化**：论文里的 loss 除以每个回答的 token 数 |o|。
- 从 policy gradient 和 baseline 定理出发严格推导，这两项都不会出现。GRPO 出来不久就有人指出这一点，并发现去掉这两项后行为大不一样——这就是 Dr. GRPO。所以 GRPO 不是“照着标签做事”的算法：它并不直接对 reward 做梯度上升，而是带着两个修正项，各有利弊。
- **长度归一化在做什么**：长回答除以更大的数。想极端一点：我知道这道证明题我做不出来，要吃 -1 的 reward；那我就写无限长——除以无穷大，惩罚归零。所以它鼓励模型在发现自己解不出来时“接着叨叨”。把这项去掉，人们在 GRPO 里反复看到的“CoT 越来越长”会停在一个常数上，而不是无休止地涨——尤其在答错的样本上，你本来就不想它越写越长。

◦ 问答 (38:42): 那对答对的样本呢? 归一化会鼓励它把 CoT 缩短——省推理成本是好事, 伤准确率就是坏事; 但答对的 CoT 有个下界 (解这道题至少要写这么多), 缩不了太多, 主要问题还是答错的那边。R1 论文里的长度曲线是所有回答的平均, 看不出这一点; Dr. GRPO 把平均长度、答错的长度、答对的长度分开画, 长度增长基本是答错的回答在驱动——和上面的解释吻合。

- **除以 std 在做什么:** 相当于给 std 小的题加权。二值 reward 下 std 什么时候小? 题太容易 (每次都对, 零方差) 或太难 (每次都错, 零方差)。也就是说, 这一项同时放大了两端的题——而我们恰恰希望模型在"能力边缘"的题上学习。

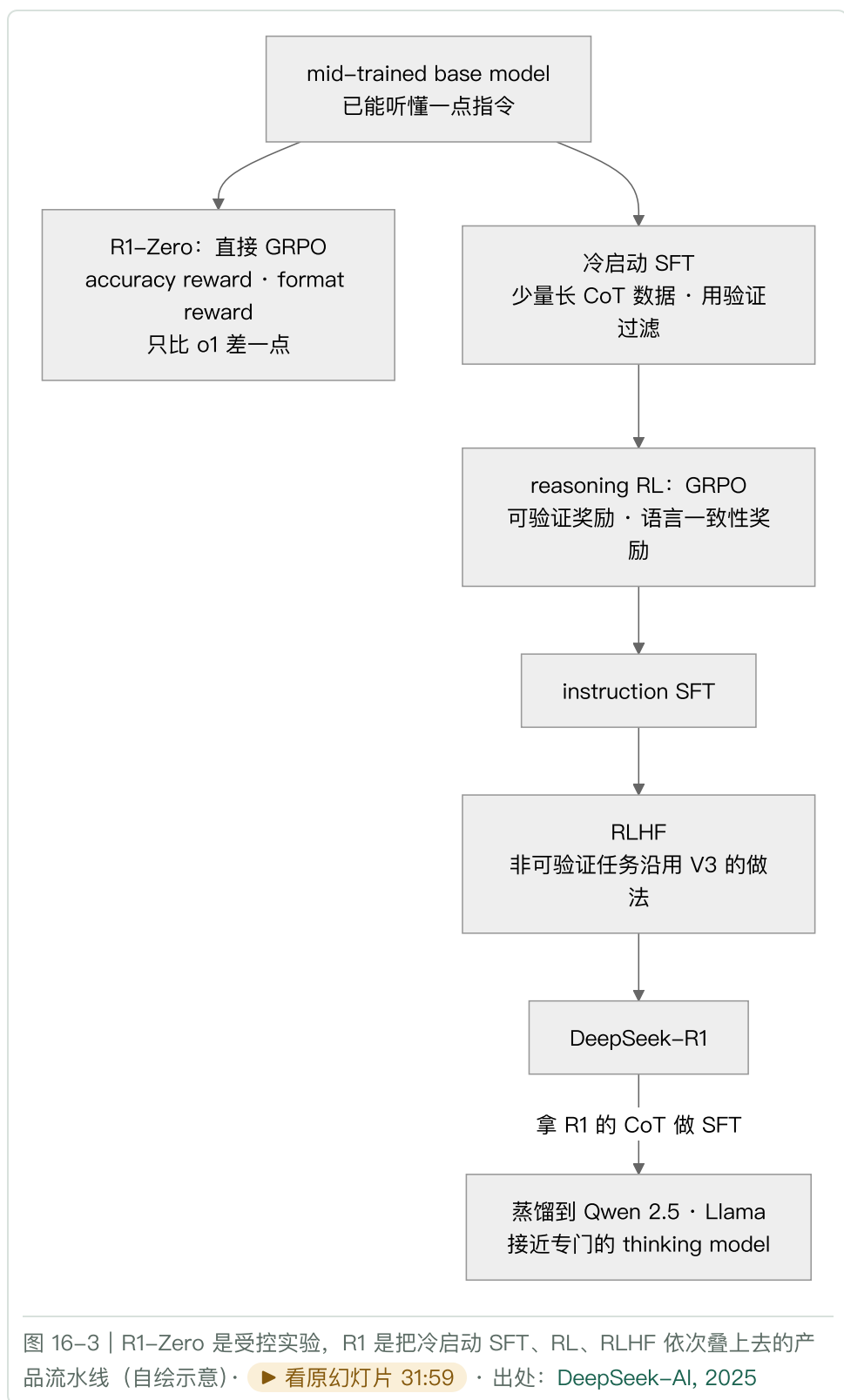
$$p = \text{pass rate} \Rightarrow \text{std} = \sqrt{p(1-p)}, \quad A_{\text{correct}} = \sqrt{\frac{1-p}{p}}, \quad A_{\text{wrong}} = -\sqrt{\frac{p}{1-p}}$$

p 是这道题在组内的通过率 (笔记补充的推导, 讲者只说了结论):

reward 非 0 即 1 时组内 std 是 $\sqrt{p(1-p)}$, 于是 p 越接近 0 或 1, 答对或答错那一侧的 advantage 绝对值越大——例如 $p = 0.1$ 时唯一答对的答案权重是 3, $p = 0.5$ 时只有 1。

- **顺带:** DAPO 的 token 级 loss 与动态采样, 就是分别冲着这两项去的 (CS329A 第 6 讲); 本讲讲者的落点是"知道它们在干什么, 再决定要不要"。

6. DeepSeek R1: 极简配方, 和把零件装成产品



- **为什么值得讲：**R1 是个社会现象，也是开源 RLVR 浪潮的起点。它是第一个复现 o1 行为的开源工作——很长的 CoT、明显的 RL、难题上的高分——并且给出了人人可跑的配方（如果答案是一套只有 DeepSeek 能跑的 PPO 怪物，对研究者的影响会小得多），外加一批至今站得住的蒸馏结论。

- **从 DeepSeekMath 继承什么、扔掉什么：**继承 GRPO 和对数学 RL 的经验；扔掉 process supervision，只用 outcome supervision——reward 只看最终答案对不对，不再拿细则给中间步骤打分。很多人当时认为过程监督不可或缺，事实证明对很多事都不关键。
- **R1-Zero：受控实验。**起点是 base model（如今的 base 都经过 mid-training，能做一点指令跟随）；在它上面直接跑 GRPO，reward 只有两项：accuracy（一批数学题答对没有）和 format（要求把 CoT 包在 thinking 标签里——这样之后能把 CoT 剥掉）。就这么简单的配方，落点只比 o1 差一点。讲者喜欢这个结果，因为它没有生产流水线里的一堆杂质——不用怀疑是不是 RLHF 帮了忙。作业要复现的基本就是这一套。

小注：R1 论文 Table 2——R1-Zero 在 AIME 2024 上 pass@1 71.0%、多数投票 (cons@64) 86.7%，对比 o1-0912 的 74.4%；正式的 R1 是 79.8%，对比 o1-1217 的 79.2%。

- **两个被传疯的现象，讲者给的冷水：**一是训练越久 CoT 越长——现在知道这多半是 GRPO 长度归一化的副作用（第 5 节）；二是"aha moment"——别人已经证明 base model 里就有这种话，它不可能是 RL 算法的产物，只是 RL 让模型吐大量数学 token 时被抽出来了。所以现象本身不算稀奇，R1 真正的里程碑意义是展示了 RLVR 能有多简单。

小注："aha moment 在 base model 里就有"应出自 Dr. GRPO 那篇 (Liu et al., 2025)，它在 DeepSeek-V3-Base 等基座上直接观察到了自我反思式的措辞。

- **R1：把零件装成产品**（图 16-3）。课程前面把预训练、SFT、RLHF 都单独讲过，这里能看到它们怎么叠：mid-trained 模型 → 推理训练（中间可能穿插长上下文扩展）→ 末尾 RLHF——RLHF 放最后，因为它最贴近用户，管格式和口吻。R1 特有的几处：
 - **语言一致性奖励：**R1-Zero 式训练会在 CoT 里中英混杂，他们觉得不可解释、有点吓人，于是加一项奖励让 CoT 只用一种语言——纯为可读性；
 - **非可验证奖励也扔进 GRPO，**相当于把 RLHF 掺进来；
 - **冷启动 SFT：**R1-Zero 一点 SFT 都不做，R1 则先用长 CoT 数据微调。报告写的是"构造并收集了少量长 CoT 数据"——讲者建议读技术报告时学会读字缝：这么小心的措辞，八成是从别的模型蒸馏的（如今人人如此，算不上负面）。好的 base model 只靠长 CoT

的 SFT 就能解锁不少 o1 式能力，是 RL 的好起点；他们再用验证器把这些 CoT 过滤一遍。

- **RL 到底必要吗：**讲者的学生等人的工作表明，蒸馏做得对、base 选得对，长 CoT 推理的大部分"汁水"光靠 SFT 就能挤出来。他给的一种理解：RL 是**监督信号的来源**——前沿数学题没人能给你写详细的长 CoT，RL 让模型自己生成；可一旦有人生成了，别人就能靠模仿学会。蒸馏结果就是这么来的：把 R1 的 CoT 喂给 Qwen 2.5，能显著提升，有时接近专门的 thinking model；Llama 也一样——base model 本来就出人意料地强。

小注："我的学生的工作"应包括 s1 (Muennighoff et al., 2025)：1,000 条精选长 CoT 做 SFT 加简单的预算控制，就逼近了 o1-preview (推断)。

- **DeepSeek 说了哪些没成的：**讲者很欣赏他们既做消融也报失败。DeepSeekMath 里那么多 process reward model 的内容，到 R1 里去哪了？他们试了，没多大用——outcome reward 够好，数据也好扩；PRM 的死结是逐步打分的细则从哪来、怎么规模化。o1 刚出时大家猜 OpenAI 是不是在做 PRM 或 AlphaGo 式的树搜索，DeepSeek 也试了 MCTS，报告说没能跑好。

7. Kimi K1.5：另一条推导路，同一个落点；反过来把 CoT 压短

- **为什么要和 R1 一起讲：**Kimi K1.5 和 R1 同时发布、同样打过 o1，却少有人提。它有几处做法和 DeepSeek 不同——两条路都通，正说明哪些零件是真正有效的。它对数据集构造和课程 (curriculum) 写得更细，用的 RL 算法直觉相近但不是 GRPO——所以 GRPO 也不是必需品。和 DeepSeek 一样，SFT 阶段用了什么数据没有描述。
- **从 DPO 式推导出发：**起点和所有人一样——最大化期望 reward，加一个到参考策略 (或上一轮策略) 的 KL 正则。然后走 DPO 的路：假设能解析地求最优解，反解出对应的 reward (一个策略比值)，代回目标——最优解处两边应该相等，于是干脆对"两边之差"放一个平方损失去最小化。讲者说做优化的人看了会吓了一跳，但直觉是合理的：模型好的时候这两个量本该接近，把它们拉近当代理目标也说得通。

$$L(\theta) = \mathbb{E} \left[\left(r(x, y) - \tau \log Z(x) - \tau \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right)^2 \right]$$
$$\nabla_{\theta} L \propto \frac{1}{k} \sum_{i=1}^k \left[(r_i - \bar{r}) \nabla_{\theta} \log \pi_{\theta}(y_i | x) - \frac{\tau}{2} \nabla_{\theta} \left(\log \frac{\pi_{\theta}(y_i|x)}{\pi_{\text{ref}}(y_i|x)} \right)^2 \right]$$

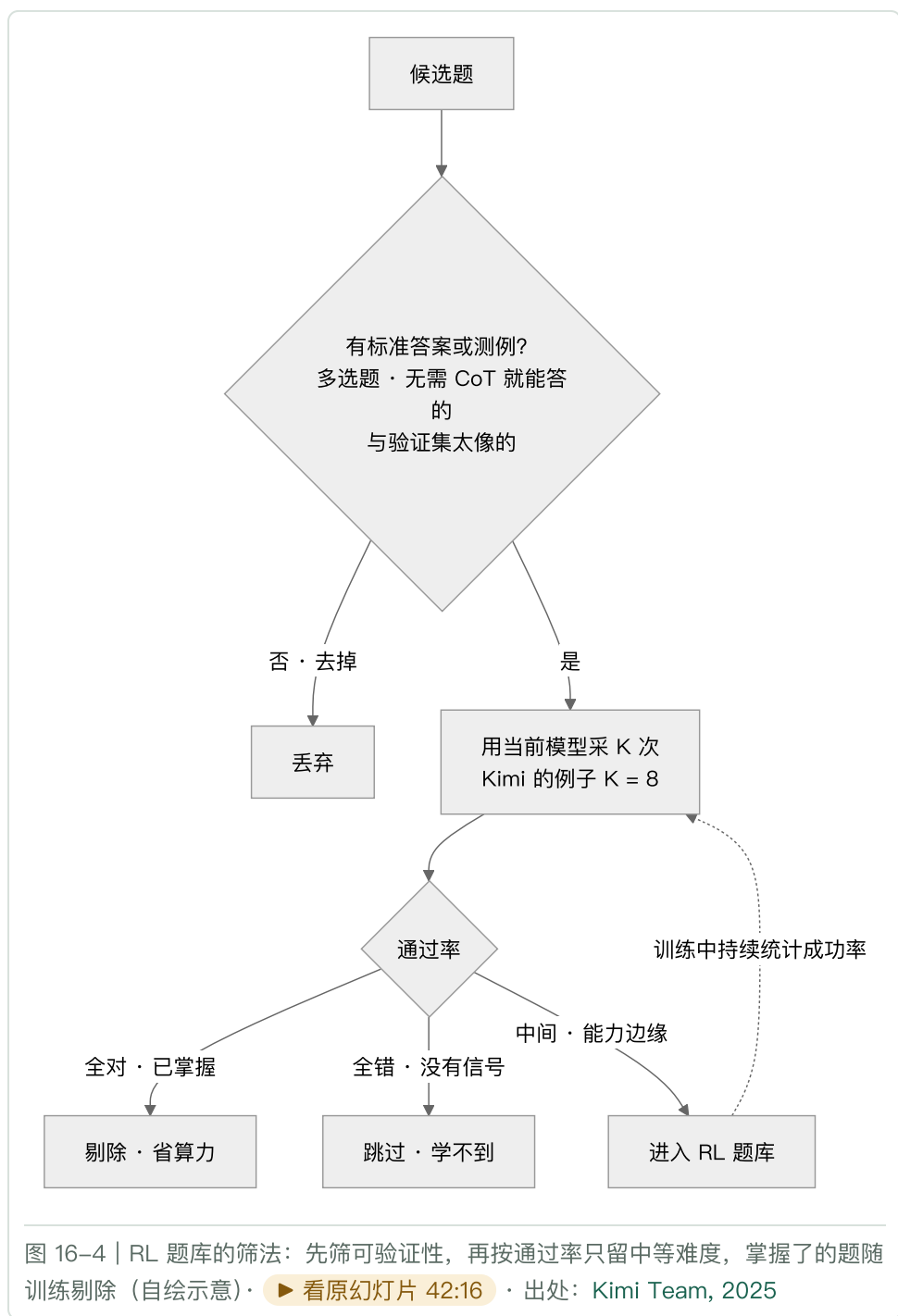
τ 是 KL 的温度, $Z(x)$ 是归一化常数, k 是同一道题采的回答数, $\bar{r}$ 是这 k 个 reward 的均值。对第一个式子的平方损失求梯度, 得到第二个式子——"惊人地像 GRPO": 一个以组均值 $\bar{r}$ 为 baseline 的 policy gradient, 加一个本质上就是 KL 的正则项; 从完全不同的路重新发明了组均值 baseline, 但没有除以 std, 也没有按长度归一化。

- **对长度的另一种态度**: GRPO 那边把"长度不受控地涨"当好事展示 (讲者说这是不太厚道的解读, 但你把这张图当正面结果放出来, 潜台词就是"我们的模型想得更久了")。Kimi 看同一件事的角度是: 长 CoT 浪费——CoT 是推理成本。如果你是 OpenAI, 200 美元一个月的 Pro 用户让模型一次想一个小时, 这是在贴钱; 一次想 5 分钟才是好日子。所以 Kimi 不但没有长度问题 (目标里没有长度归一化), 还要主动压短。
- **长度奖励怎么设计**: 最简单的 RL 做法是加一个启发式的长度奖励。这个奖励出人意料地讲究: 长的要变短, 答对的也要短; 但**答错的不能压得太短**——讲者的例子: 我是个几何很差的 AI, 错得多, 惩罚把我的几何 CoT 压到零, 从此再也拿不到一次正的几何 reward, 永远翻不了身。所以只让答错的回答比平均"稍短一点", 保证它不无限增长即可。

小注: 按 K1.5 论文的写法, 长度奖励是 $\lambda = 0.5 - (\text{len} - \text{min_len}) / (\text{max_len} - \text{min_len})$, 答对时直接给 λ , 答错时给 $\min(0, \lambda)$ ——答错的回答只会因为太长被罚, 不会因为短被奖。

- **结果**: 打过 o1; RL 推进时思考变长、分数上涨; 但 OmniMath 上思考没怎么变长而分数继续涨——讲者认为这是长度控制在起作用的好例子。
- **expert iteration 比不比得过 RL**: 只拿正确答案训练 (expert iteration) 在很多老论文里很好用, 训练不稳时也许更合适; K1.5 的大规模消融里 RL 一直稳定胜出 (橙线在蓝线之上) ——想榨干性能, 绕不开 RL。

8. 数据与 reward: 难度课程、成功率过滤, 和"可验证"的坑



- **RL 数据多了一个维度：难度。**SFT 不太考虑难度——数据往里塞，预训练 loss 逼着模型学（上一讲讲过幻觉问题是个例外）。RL 不同：题太难 → 没有 reward → 没有信号 → 学不到。所以覆盖面要广，难度要落在模型能力边缘。
- **Kimi 的筛法：**追求广覆盖；排除多选题（别的领域已经覆盖，而且不需要长而深的思考）；最重要的一条——用 best-of-K 过滤：对每道题采 8 次，模型已经会的题不是好的 RL 样本，教不了新东西，去掉能省算力（整题跳过）。也可以两头都筛，只留既不太难也不太易的——社区共识是中等难度的过滤对“让 RL 稳步推进”很有效。训练过程

中还持续统计每道题的成功率，一旦掌握就从题库剔除——基本上做 RL 的人都这么干。

- **Qwen 3 的三重过滤 (57:05)**：按难度筛（省算力）；去掉不用 CoT 也能做对的题（那不是思考题）；去掉和验证集太像的题（去污染，第 12 讲）；再对参考 CoT 做少量人工筛选。最惊人的是量：reasoning RL 只用了约 4,000 道题——流水线的其他环节做对了，题不用多。

小注：Qwen3 报告原文是 3,995 条 query-verifier 对，和讲者说的 4,000 吻合。

```
def build_rl_pool(problems, policy, K=8, lo=0.0, hi=1.0):
    pool = []
    for p in problems:
        if not p.has_verifier or p.is_multiple_choice or
near_dev_set(p):
            continue
        passes = sum(verify(p, o) for o in policy.sample(p.prompt,
n=K))
        rate = passes / K
        if lo < rate < hi:
            pool.append(p)
    return pool
```

只留能力边缘：不全错，也不全对

训练中再按最新成功率剔除已掌握的题

这段草图把两家的筛法合成一个函数：可验证性、无 CoT 可解、去污染是静态筛，通过率是动态筛，阈值 lo / hi 取 0 和 1 就是“两头都去掉”。

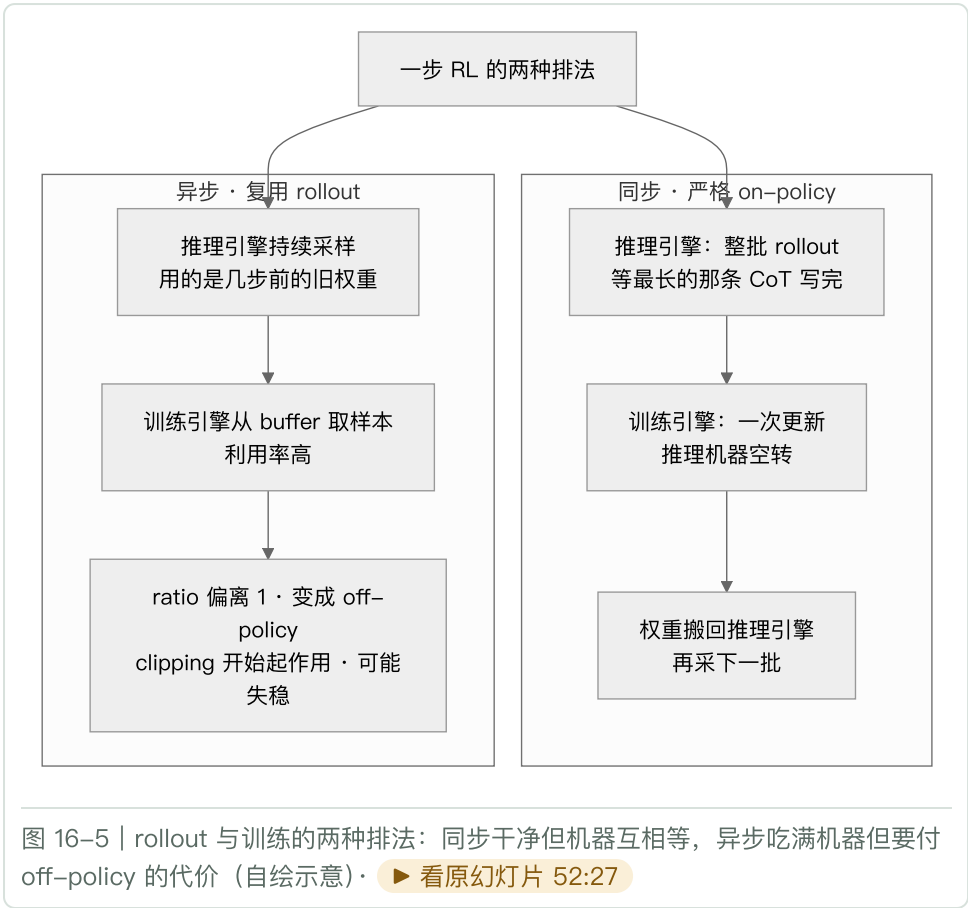
- **reward 从哪来**：代码题拿标准解生成新的测例；数学题——用一个 reward model 判断答案是否等价。讲者觉得这很讽刺：这一讲开头说的是形式化数学、编译器判对错，讲到最后落到了一个 reward model 上。原因你在作业里会撞见：答案等价检查很难。同一个值有无数种写法，模型还会用无数种方式回答——让它把答案放进 LaTeX 的 `\boxed`，它有时不放、有时往里塞多余的东西。严格的检查器会把“思路全对”的回答判错。所以几乎每个 RL 项目都有一个复杂的答案检查器——正则、模型、或者别的什么。把 RLVR 里的 V 做对，是个兔子洞。

```
def math_reward(pred_text, gold):
    ans = extract_boxed(pred_text) or last_number(pred_text) # 格式
宽容：没 \boxed 也别直接判 0
    if ans is None:
        return 0.0
```

```
if normalize(ans) == normalize(gold): # 去空
    格、单位、等价写法
    return 1.0
if symbolic_equal(ans, gold): # 化简
    后相等, 如 1/2 与 0.5
    return 1.0
return float(llm_judge_equivalent(ans, gold)) # 最后
一道防线: reward model
```

草图展示的是讲者说的那个"兔子洞"的层次：字符串比对 → 符号等价 → 模型裁判，每加一层都在减少误判，也在引入新的可被钻的空子。

9. RL infra：训练和推理两套系统拼在一起



- **为什么难：**训练难，推理难，RL 把两者拼在一起——难上加难，不奇怪。
- **长尾 rollout：**想象一批题里有一道是 Riemann 猜想，模型对着它吭哧吭哧写一条巨长的 CoT；朴素的批式推理里，其他所有 rollout 都得等它写完才能进入下一阶段。所以要处理一堆底层细节：CoT 太长要不要截断？能不能挪到另一台机器？

- **两套引擎怎么摆：**rollout 一次、训练一次、再 rollout.....在训练和推理之间切换，要么一部分机器专职 rollout、一部分专职训练，要么在同一批机器上来回换框架——两种都贵。技术报告如今都有一节 RL infra：训练部分（图里的蓝框）和推理部分（绿框），权重要从训练那边搬到推理那边，两边要紧密协调；有时干脆共用机器——推理跑的时候训练那边本来就闲着。
- **最难受的取舍：on-policy 还是复用。**严格 on-policy 的 GRPO 数学上干净、训练动态漂亮——作业里你会亲身体会。然后你会变贪心：系统利用率这么低，要是能复用 rollout、让推理和计算重叠，能省多少！一复用就 off-policy，off-policy 就开始失稳，各种难题接踵而来。
- **一步的时间账**（笔记补充，按第 2 讲和第 10 讲的口径；讲者只讲了机制，没给数字）：

$$T_{\text{step}} \approx \underbrace{\max_i L_i \cdot t_{\text{token}}}_{\text{rollout}} + T_{\text{verify}} + \underbrace{\frac{(6 + 2 + 2) N \cdot \sum_i L_i}{\text{FLOP/s} \cdot \text{MFU}}}_{\text{train}} + T_{\text{sync}}$$

L_i 是第 i 条回答的长度， t_{token} 是解码一个 token 的时间（decode 阶段每搬一个字节只做约 1 次浮点运算，受显存带宽而非算力限制，第 10 讲）， N 是参数量， $6N$ 是每个 token 的 forward + backward（第 2 讲），再加参考策略和旧策略各一趟 forward 的 $2N$ 来算 KL 和 ratio，MFU 是实际算力利用率， T_{sync} 是把新权重搬到推理引擎的时间。这笔账说明三件事：rollout 由最长的那条决定，所以长尾要单独处理；训练部分才是算力密集的，所以同步排队下昂贵的算力大半时间在等解码；权重同步的频率就是 on-policy 程度和利用率之间的旋钮。

小注：Kimi K1.5 报告里对这两个问题各给了一个答案：partial rollouts——过长的轨迹按固定 token 预算切段，没写完的部分存起来下一轮接着写，避免一条长 CoT 拖住整批；以及训练与推理混布——训练用 Megatron、推理用 vLLM 放在同一批机器上，训练时释放推理侧显存、rollout 时把新权重搬过去。

10. Qwen 3 与 Qwen3-Coder-Next：把所有零件装起来，再把 agent 装进去

- **Qwen 3 的流水线**和 DeepSeek 结构相同：base → SFT（长 CoT 冷启动）→ reasoning RL → thinking mode fusion → 通用 RLHF → 交付；交付的大模型不直接服务，再蒸馏出小模型。讲者说可以把这

张图当作"前沿级模型是怎么组装的"心智模型。RLVR 的核心部分继承 R1 和 Kimi 的精华：难度过滤、去污染等（第 8 节）。

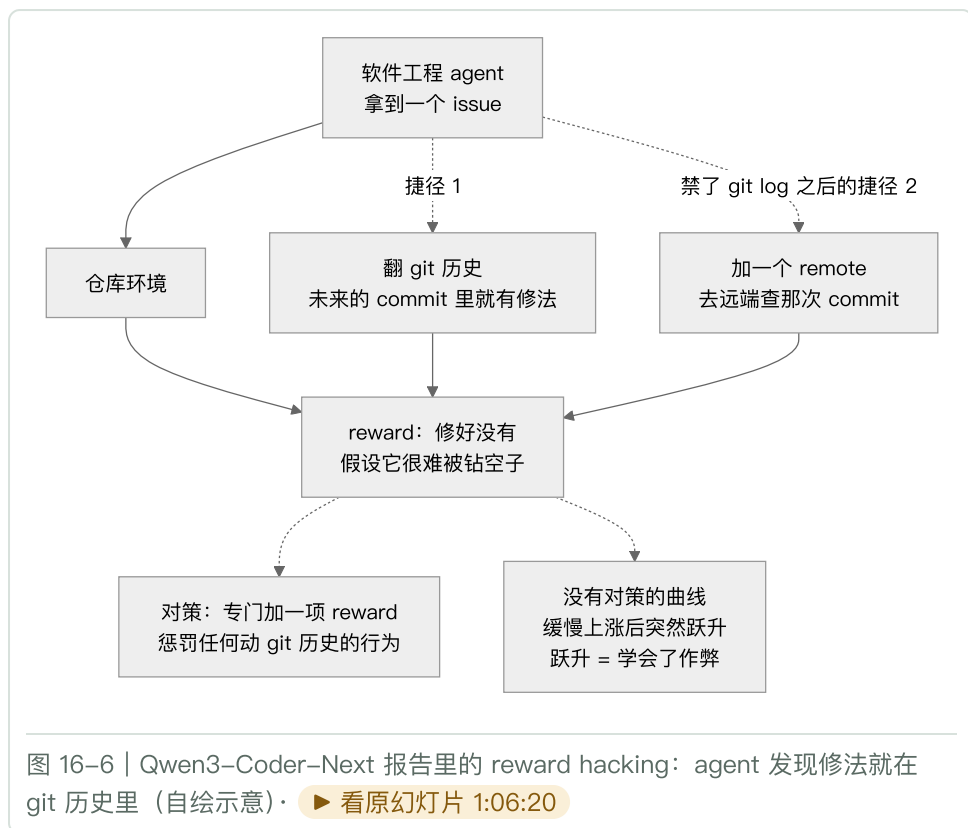
- **thinking 与 non-thinking 融进一个模型**：用标签把带长 CoT 的和即时回答的数据混训，同一个模型既有思考模式又有即时模式——此前即使在 OpenAI 也是两个模型。
 - **早停思考**：追加一个特殊字符串，模型立刻停止 CoT 并作答（ChatGPT 界面里"跳过思考"就是这种能力）。用它扫思考预算，性能**优雅退化**——哪怕在思考中途被截断，模型也能给出相当合理的回答；即便预算很小，思考模式在数学、代码上也远好于即时模式（后者更像经典的指令跟随加 RLHF 模型）。
 - **各阶段的贡献**：reasoning RL 之后再做通用 RL，Arena-Hard、反事实 QA 这类通用任务全面提升；融合非思考数据让数学、代码略降。绝对值不大，但后来的版本（讲者说 Qwen 3.5 那一批）又把这种 hybrid 模型拆开了——他们不接受这点损失，要把思考模式的性能榨干。
 - **问答 (1:10:33)**：thinking mode 背后真的是一个模型，切换靠 prompt 里的一个标签，而不是 API 或服务层的开关——把两者塞进同一个模型才是有意思的地方。
- **Qwen3-Coder-Next**（讲者口误说成 Next-Coder，随后更正）：想知道"agent 到底怎么训"，这份报告最详细。结论先行：agent 的后训练没有新算法——全课贯穿的教训是**数据才是关键**。能力不能在最后一刻注入，得从更早的阶段开始：
 - **mid-training 的配方**：把仓库里的文件拼接成超长上下文（agent 将来会打开一堆文件，这是最像的预训练数据）；给 pull request 用 RAG 构造合成的上下文；检测出"文本加代码"的文档，用 LLM 转成规整的 markdown；让 LM 对着编程相关的网页生成代码风格的合成数据；把公开的 coding agent 放到各种环境里跑，轨迹全部入库；再加指令跟随和 fill-in-the-middle（补中间一段代码的能力，对我们不太相关）。
 - **四个专家再蒸馏回一个**（讲者说没在别处见过）：从 mid-trained 的 Qwen3-Next 出发，为不同的编程相关任务各训一个专家——web 开发（在各种检查下 SFT 合法的网页代码）、UX（多种工具格式）、QA（更多合成代码数据）、软件工程 agent——再把四个蒸馏回同一个模型。最接近的先例是 DeepSeek-V3.2 里专职处理数据的专家，以及学术界的 Branch-Train-Merge。

- 问答 (1:13:06): 蒸馏要为每个专家写一套 prompt 让它们在上面出数据; 好处是各团队能并行做各自的专家; 但如果所有目标都在手上, 讲者更愿意扔进一个大训练循环, 省掉蒸馏这一层。

小注: DeepSeek-V3.2 的报告里描述的是按领域训练专家 (数学、竞赛编程、agentic coding 等) 再把它们蒸馏进统一模型的 specialist distillation, 和 Qwen 这里的做法其实更像 (推断; 讲者当场对版本号也不太确定)。

- 软件工程 agent 的环境: SWE-bench 是这类评测的金标准, 他们要的是 "SWE-bench 但更多"——基于 GitHub 用自动化方法批量生成 issue 和环境, 在上面做 RL, 分数稳步上涨。

11. Reward hacking: RLVR 只和你的 reward 一样可靠



- 前提: 我们敢往 RL 里加越来越多算力, 是因为相信 reward 很难被钻空子。这个前提一破, RL 会找到越来越隐蔽的方式骗走你的分数。
- git 里的答案: 一个 issue 如果有后续 commit, 修法就在里面——模型很容易学会去看。报告里专门有一项 reward, 唯一目的就是阻止 agent 动 git 历史。不加的话曲线是右边那种: 学着学着突然一跳, 跳的那一刻就是它学会了操纵 git 命令去翻历史。禁掉 git log 也没用, 它可能加一个 remote, 再去远端查那次 commit。

- **连 Lean 也能骗**：讲者和学生做过 RL on Lean（形式化数学语言，编译器判对错）。他们当时天真地认为这不可能出错——那么多人用过 Lean，编译器坚不可摧。结果 Lean 编译器并不对抗鲁棒：某些模式下，往里塞特定的字符串能让本不该通过的证明通过。所以“可验证”这个词，比多数人以为的要棘手得多。
- **结果与提醒**：走完这套流程，一个只有 3B 激活参数的模型在 SWE-bench 上拿到 70.6%。讲者提醒别太惊讶——RL 在你训过的那类环境上当然好，也可能泛化到验证集，但任务专属的分数不代表能泛化到更广的领域。

小注：Qwen3-Coder-Next 是 80B 总参数、3B 激活的 MoE，70.6% 指 SWE-bench Verified（推断；讲者只说了 SWE-bench）。

12. 作业 5 的视角，与收尾问答

- **讲者在课上点到的作业内容**：实现 GRPO（一页代码：采 G 次、reward、组内归一化、KL、带 stop-gradient 的更新）；实现 RFT / expert iteration 当 baseline；复现 R1-Zero 式的结果；亲手撞上答案等价检查的坑；体会严格 on-policy 的 GRPO 有多“乖”，以及一复用 rollout 会发生什么。

小注：2025 春的 assignment 5 (alignment) 是在 Qwen2.5-Math-1.5B 上用 vLLM 做 rollout、自己写 GRPO 跑 MATH，并对比 SFT 与 expert iteration；2026 版应类似（推断）。

- **三条 takeaway**：
 1. 一切在于 reward：RLHF 和 RLVR 是几乎一样的问题，差别只是后者的 reward 更难被钻空子，于是能承受多得多的算力；
 2. GRPO 至少对研究界是解锁这一切的算法——它的函数形式和更新方式，要熟到和预训练 loss 一样；
 3. RLVR 现在人人会做；RL 仍然又挑剔又吵闹，但已经不像当年在刁钻环境上调 PPO 那么难。
- **收尾问答**
 - *mid-training 对 RL 有多重要 (1:11:04)*：没有一刀切的答案。预训练和 SFT 干了大部分重活，前提是有覆盖——预训练里一点代码都没有，那就必须 mid-training；预训练够多样，mid-training 就是锦上添花，因为总归要做 SFT，SFT 能把模型送到“开始拿到一些 reward”的位置；没有 SFT 才是真麻烦。

- 长 CoT 训练算 mid-training 吗 (1:13:36)：R1 和 K1.5 都有长 CoT 的 SFT，它传统上不算 mid-training；但长 CoT 数据常用于长上下文扩展——讲者说自己漏讲了这一段：通常是 RLHF 之前的一个独立阶段，用书、代码、合成数据这些够长的材料把上下文拉长。
- 多个领域（数学、化学……）串行还是并行做 RL、怎么防遗忘 (1:14:39)：报告里的分法是两桶——推理类问题全部放在第二阶段一起做，非推理类（包括"话多不多"这种）放在最后的 RLHF 阶段。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
后训练路线图	左边是上一讲的 instruction tuning 与 RLHF，右边待补的是 thinking model	0:36	—
PPO 的 Spinning Up 伪代码	五步看着都简单；对照下一张"37 个实现细节"的博客标题	6:17	Spinning Up
PPO for LM 框图	value model 那个绿框出现两次；KL 逐 token 算，所以是多步 RL	7:47	—
GRPO 目标函数	min-clipped 项加 KL；advantage 是组内 z-score；在线时 ratio 恒为 1	15:30	DeepSeekMath
半页 GRPO 实现	组索引和 std 的计算；std 上加的 $1e-4$	18:08	—
DeepSeekMath 结果	黄、蓝两条 GRPO 线高于 RFT；process supervision 当时看有增益	19:09	同上
Dr. GRPO 的两处标红	除以 std 与除以长度，都不在 policy gradient 的推导里	22:16	Liu et al., 2025
R1-Zero 的配方与曲线	base + GRPO + accuracy / format reward；分数逼近 0.1；CoT 长度随训练上涨	28:55	DeepSeek-R1
Dr. GRPO 长度分解	平均长度、答错的长度、答对的长度分开画：涨的是答错的	39:12	Liu et al., 2025
Kimi 的推导	从 KL 正则的目标到平方损失，再到带 r baseline 的梯度	44:21	Kimi K1.5
Kimi 的 RL infra 图	蓝色训练、绿色推理，中间的箭头是权重搬运	53:28	同上
Kimi 的长度—性能曲线与 expert iteration 消融	OmniMath 上长度不涨分数涨；橙线 RL 高于蓝线 expert iteration	54:29	同上

图	看什么	跳转	出处
Qwen 3 流水线与思考预算曲线	四阶段加蒸馏；预算变小时性能平滑下降，思考模式始终高于即时模式	56:33	Qwen3
Qwen3-Coder-Next 的 reward hacking 曲线	右图缓慢上涨后突然跃升——学会翻 git 历史的时刻	1:06:50	Qwen3-Coder-Next 报告

提到的工作

名称	在本讲里的作用
OpenAI 解决一个 Erdős 问题的公告	开场的新闻：thinking model 就是这一讲要讲的东西
AlphaGo	RL 的高光时刻：目标精确，算力可以一直加；对比 RLHF 的 overoptimization
PPO (Schulman et al., 2017) · TRPO (Schulman et al., 2015)	让一批 rollout 多用几步的算法；RL 的 workhorse
GAE (Schulman et al., 2016)	PPO 的 advantage 估计；实践中 $\gamma = \lambda = 1$ 退化成 bandit
OpenAI Five · OpenAI Gym 人形机器人	PPO 的两个经典演示
Spinning Up in Deep RL	PPO 伪代码的来源：“看着很简单”
The 37 Implementation Details of PPO (Huang et al., 2022)	说明 PPO 对实现极其敏感
"有些 baseline 不是 baseline"的论文	讲者没报题目；可能是 Chung et al., 2021 (推断)
DPO (Rafailov et al., 2023) · Bradley-Terry 模型	只适合 pairwise 偏好；Kimi 的推导借用了 DPO 的路
DeepSeekMath (Shao et al., 2024)	GRPO 的出处；GRPO 胜过 RFT；当时还用 process supervision
RFT (rejection fine-tuning) · expert iteration	只拿正确答案训练的 baseline；作业里要实现；K1.5 消融中输给 RL
Sutton & Barto, Reinforcement Learning: An Introduction	REINFORCE with baseline：合法的 baseline 只能依赖状态
McGill 的 GRPO 玩具实现	半页代码；std 加 $1e-4$
Dr. GRPO (Liu et al., 2025)	指出除以 std 与按长度归一化的偏差；长度分解图；"aha"在 base 里就有（应为此篇）
DeepSeek-R1 (DeepSeek-AI, 2025)	R1-Zero 的极简配方；生产流水线；蒸馏；PRM 与 MCTS 的失败尝试
OpenAI o1	被追赶的目标：长 CoT、RL、难题高分
s1 (Muennighoff et al., 2025)	"我的学生的工作"：蒸馏加 SFT 就能拿到大部分长 CoT 能力（推断）

名称	在本讲里的作用
Kimi K1.5 (Kimi Team, 2025)	数据课程、best-of-8 过滤、DPO 式推导、长度奖励、RL infra、expert iteration 消融
OmniMath	Kimi 结果里"长度不涨分数涨"的例子
Qwen3 (Qwen Team, 2025)	完整流水线；4,000 道题的 RL；thinking / non-thinking 融合；思考预算
Arena-Hard	Qwen 3 通用 RL 阶段提升的任务之一
Qwen3-Coder-Next (项目页)	agent 后训练：mid-training 数据、四个专家蒸馏、SWE 环境、reward hacking、70.6%
DeepSeek-V3.2	讲者想到的最接近"专家再蒸馏"的先例
Branch-Train-Merge (Li et al., 2022)	学术界的"分头训再合并"
SWE-bench	软件工程 agent 的金标准；Qwen 要的是"SWE-bench 但更多"
Lean	形式化数学；编译器也不对抗鲁棒
vLLM · Megatron	小注里 Kimi 混布方案的推理与训练引擎

术语对照

English	中文
RLVR (reinforcement learning with verifiable rewards)	可验证奖励的强化学习：reward 来自答案检查、测例等程序化判定
verifiable reward / verifier	可验证奖励 / 验证器
overoptimization	过度优化：对拟合出来的 reward model 加太多算力，分数涨而真实质量跌
annotation bottleneck	标注瓶颈
thinking model / long chain of thought (CoT)	思考模型 / 长思维链
policy gradient	策略梯度
REINFORCE	最基本的策略梯度估计：log 概率梯度乘 reward
baseline	基线：从 reward 里减掉的、只依赖状态的量，降方差不改方向
advantage	优势：reward 减 baseline，决定每个样本权重的正负和大小
value model / value function	价值模型：逐 token 估计期望回报的网络，PPO 用它当 baseline
GAE (generalized advantage estimation)	广义优势估计：用 γ 、 λ 折衷偏差与方差

English	中文
bandit problem	多臂老虎机问题：只有一步决策，整条回答当一个动作
trajectory / rollout	轨迹 / 采样出的回答
experience buffer	经验缓冲区：存旧 rollout 供多步复用
on-policy / off-policy	同策略 / 异策略：用来更新的样本是否来自当前策略
probability ratio / clipping	新旧策略概率比 / PPO 的截断
KL penalty / reference policy	KL 惩罚 / 参考策略
DPO / Bradley–Terry model	直接偏好优化 / 成对比较的概率模型
GRPO (group relative policy optimization)	组相对策略优化
group / z-score	组：同一道题的 G 个回答 / 标准分：减均值除以标准差
length normalization	长度归一化：loss 除以回答的 token 数
stop-gradient	截断梯度：把 advantage 当常数
outcome supervision / process supervision	结果监督 / 过程监督
ORM / PRM (outcome / process reward model)	结果奖励模型 / 过程奖励模型
accuracy reward / format reward	正确性奖励 / 格式奖励
language consistency reward	语言一致性奖励
cold start SFT	冷启动 SFT：RL 之前先用少量长 CoT 微调
distillation	蒸馏：拿强模型的输出训练别的模型
RFT (rejection fine-tuning) / expert iteration	拒绝采样微调 / 专家迭代：只拿正确答案训练
MCTS (Monte Carlo tree search)	蒙特卡洛树搜索
aha moment	"顿悟时刻"：CoT 里的自我反思措辞
curriculum / difficulty filtering	课程 / 难度过滤
best-of-K / pass rate	K 次采样至少一次对 / 通过率
decontamination	去污染：去掉与评测集太像的练习题
answer equivalence checking	答案等价检查
reward hacking / unhackable	奖励钻空子 / 难被钻空子的
length reward / length penalty	长度奖励 / 长度惩罚
thinking budget / early exit	思考预算 / 早停思考
thinking mode fusion / hybrid model	思考模式融合 / 混合模型

English	中文
graceful degradation	优雅退化
mid-training	中期训练
long-context extension	长上下文扩展
fill-in-the-middle (FIM)	补中间：给前后文补中段代码
agentic RL / agent environment	面向 agent 的 RL / agent 环境
inference engine / training engine	推理引擎 / 训练引擎
weight sync	权重同步：把训练侧的新权重搬到推理侧
partial rollout	部分 rollout：长轨迹分段跨轮生成（小注）
MFU (model FLOPs utilization)	模型算力利用率：实际 FLOP/s 占峰值的比例
Lean	形式化数学证明语言与编译器

字幕勘误

"PO / PP EPO / POS" → PPO；"gpo / gRPO / gRP" → GRPO；"coot / coots / coott" → CoT；"herdish problems" → Erdo's problems；"Alph Go" → AlphaGo；"RHF" → RLHF；"Chat GPG" → ChatGPT；"Senardo" → Sutton & Barto；"Bradley Perry" → Bradley-Terry；"zcore" → z-score；"1g4" → 1e-4；"RFP" → RFT；"Kimmy / Kimmy K1 / K5 1.1" → Kimi K1.5；"Quen / Quinn / Quent / clen" → Qwen；"Deepsk / deep sea" → DeepSeek；"OpenAI 01 / openio1 / OpenAI1" → OpenAI o1；"Remon hypothesis" → Riemann hypothesis；"Swebench / SweetBench" → SWE-bench；"latte boxed" → LaTeX \boxed；"reg x" → regex；"omni math" → OmniMath；"asentic" → agentic；"hardnesses" → harnesses；"gate history" → git history；"mid-raining / pre-trading" → mid-training / pre-training；"URL algorithm" → RL algorithm；"RL pill" → RL-pilled；"branch train merge" → Branch-Train-Merge；"Next Coder" → Coder-Next（讲者自己随后更正）。

带走的问题

- GRPO 去掉除以 std 之后，advantage 变成 $r_i - \text{mean}$ ：对一道通过率 10% 的题和一道 90% 的题，每个样本的权重各是多少？两道题对梯度的总贡献又是多少？DAPO 的动态采样和这一项是什么关系？

2. 用第 9 节的时间账估一下： $G = 8$ 、平均回答 4k token、最长 32k token 时，rollout 阶段有多大比例的时间在等最后一条？partial rollout 把这笔账改成什么样，又付出了什么（提示：那条没写完的轨迹是用哪一版权重写的）？
3. Kimi 的长度奖励对答错的回答只罚不奖——如果反过来奖励“错得短”，第 7 节的“几何差生”会怎么死掉？这和 GRPO 的长度归一化是同一个问题的两面吗？
4. “RL 是监督信号的来源，蒸馏是它的传播方式”——如果这个说法成立，一个没有前沿模型可蒸馏的团队，哪些环节必须自己跑 RL，哪些可以省？
5. Lean 编译器都能被骗，“verifiable”的边界到底在哪？给你的 agent 环境设计 reward 时，怎么提前找出“翻 git 历史”这类捷径，而不是等曲线突然跃升才发现？

CS336 第 17 讲 | 对齐与多模态 (Alignment, Multimodality)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 17 讲, 2026 年 5 月 27 日, 客座讲座之前的最后一讲。课程表上的题目是 "Alignment – Multimodality", 但讲者开场就说: 原计划再多讲些强化学习, 考虑到只剩这一讲, 改为通讲多模态——对齐部分实际一句没讲, 77 分钟全是多模态 视频: <https://www.youtube.com/watch?v=26FtD08ZpOU> (1:17:39; 英文字幕为自动生成, 模型名与缩写错得多, 见文末勘误) 讲者: Percy Liang (课程表所列; 字幕里"第 1 讲我们讲了 tokenization、BPE 你们自己实现过""幻灯片是 HTML 写的, 标签被吃掉了""我是做语言的人"等自述与之相符) 课程主页: <https://stanford-cs336.github.io/> · 本讲围绕的论文: CLIP、SigLIP、LLaVA、LLaVA-OneVision、Qwen-VL、Qwen2-VL、Qwen3-VL、Chameleon

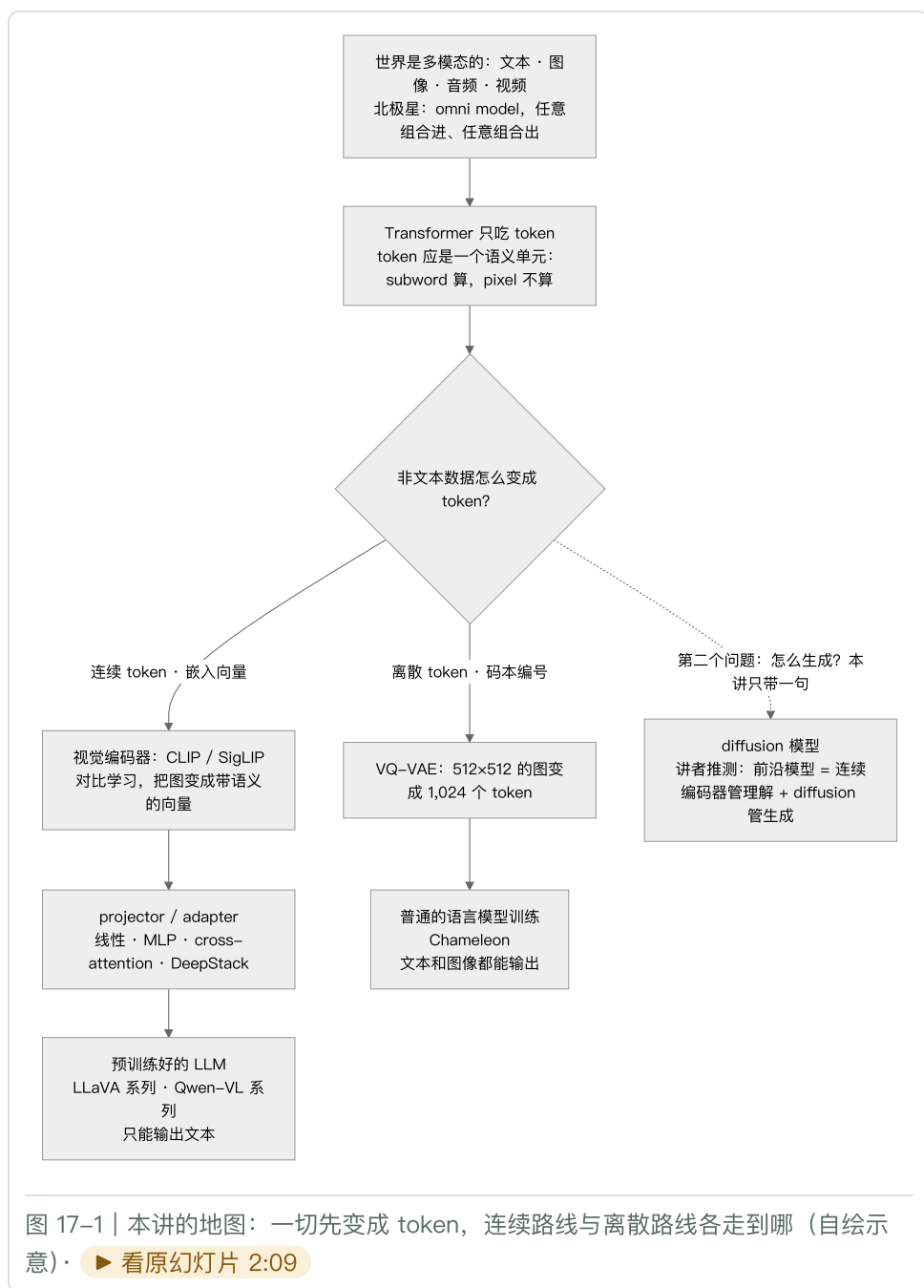
一句话: Transformer 在所有模态上都是目前最好用的, 但它只吃 token, 所以多模态的全部难点是"把图像、音频、视频变成 token"。走连续路线, 就是先用对比学习训一个视觉编码器 (CLIP 用 4 亿对网络图文、每个 batch 约 32k 对做 2N 个 softmax; SigLIP 把它换成 N^2 个 sigmoid, batch 与 loss 解耦, 芯片·天省 16 倍), 把 336×336 的图变成几百个带语义的向量, 再经一个只有几千万参数的 projector 注进一个几十到几百亿参数、已经预训练好的 LLM——LLaVA 与 Qwen-VL 两个系列共用这个模板, 几代之间变的只是编码器、adapter、动态分辨率、位置编码和越来越多的训练阶段 (LLaVA 两阶段, Qwen3-VL 四加三阶段、上下文拉到 256K)。走离散路线, 就是 Chameleon: VQ-VAE 把 512×512 的图变成 1,024 个来自 8k 词表的 token, 和文本一起做普通的语言模型训练, 因此能生成图像, 代价是训练不稳、细节丢失。讲者的判断: 前沿模型大概率是"连续编码器管理解、diffusion 管生成"。

时间	内容
0:05	开场: 原计划讲 RL, 改讲多模态; 北极星是任意模态进出的 omni model
2:09	Transformer 只吃 token: token 可离散可连续, 但得是语义单元; 两个问题——怎么输入、怎么生成, 本讲只答前者
4:13	回到 2021: CLIP 的动机 (把"爬网页"搬到图像上) 与对比目标; 伪代码

时间	内容
8:28	数据 400M 对、OpenCLIP 与 LAION-5B；短边缩到 336 再中心裁剪；问答：为什么要配文本
12:37	视觉编码器 ViT-L/14 与 attention pooling；文本编码器取 EOS 位置的激活
17:18	零样本 ImageNet 胜过 120 万张标注图训出的 ResNet；噪声与过滤；改成预测 caption 反而更差；CLIP 的代价
22:27	SigLIP：每个格子一个 sigmoid；WebLI 十亿量级，OCR 也能造图文对
25:32	训练效率：256 张 v3 × 10 天对 32 张 v4 × 5 天；文本嵌入轮转的分块并行；batch 与 loss 解耦，32k 是临界
28:41	VLM 模板：编码器 + projector + LLM；LLaVA 的零件与 158k 条 GPT-4 合成数据
33:21	LLaVA 两阶段：先只训 W 做对齐，再训 W 和 LLM；"面包车后面熨衣服"
35:26	LLaVA-OneVision：SigLIP + Qwen2 + 两层 MLP；OCR 逼出 AnyRes
39:41	单图 / 多图 / 视频的 token 预算；数据即任务、蒸馏 GPT-4；三阶段；跨模态迁移
45:52	Qwen-VL：OpenCLIP + 一层 cross-attention 压成 256 个 token；三阶段，第一阶段连编码器一起训
49:29	Qwen2-VL：原生动态分辨率、2×2 合并、三维 M-RoPE
52:37	Qwen3-VL：交错 M-RoPE、显式时间戳、 $\sqrt{\cdot}$ 归一化的 loss、DeepStack、4 + 3 阶段、256K 上下文
59:47	问答：只输出文本；系统难点在数据加载；对齐阶段预算 67B token；视觉编码器为何远小于 LLM
1:07:07	Chameleon：全部离散化；VQ-VAE；训练不稳定与 QK-norm、z-loss
1:14:26	收束：连续编码器 + diffusion 的推测；理解与生成的对称；模态要称重

核心内容

1. 题目里的"对齐"去哪了，以及这一讲真正的问题

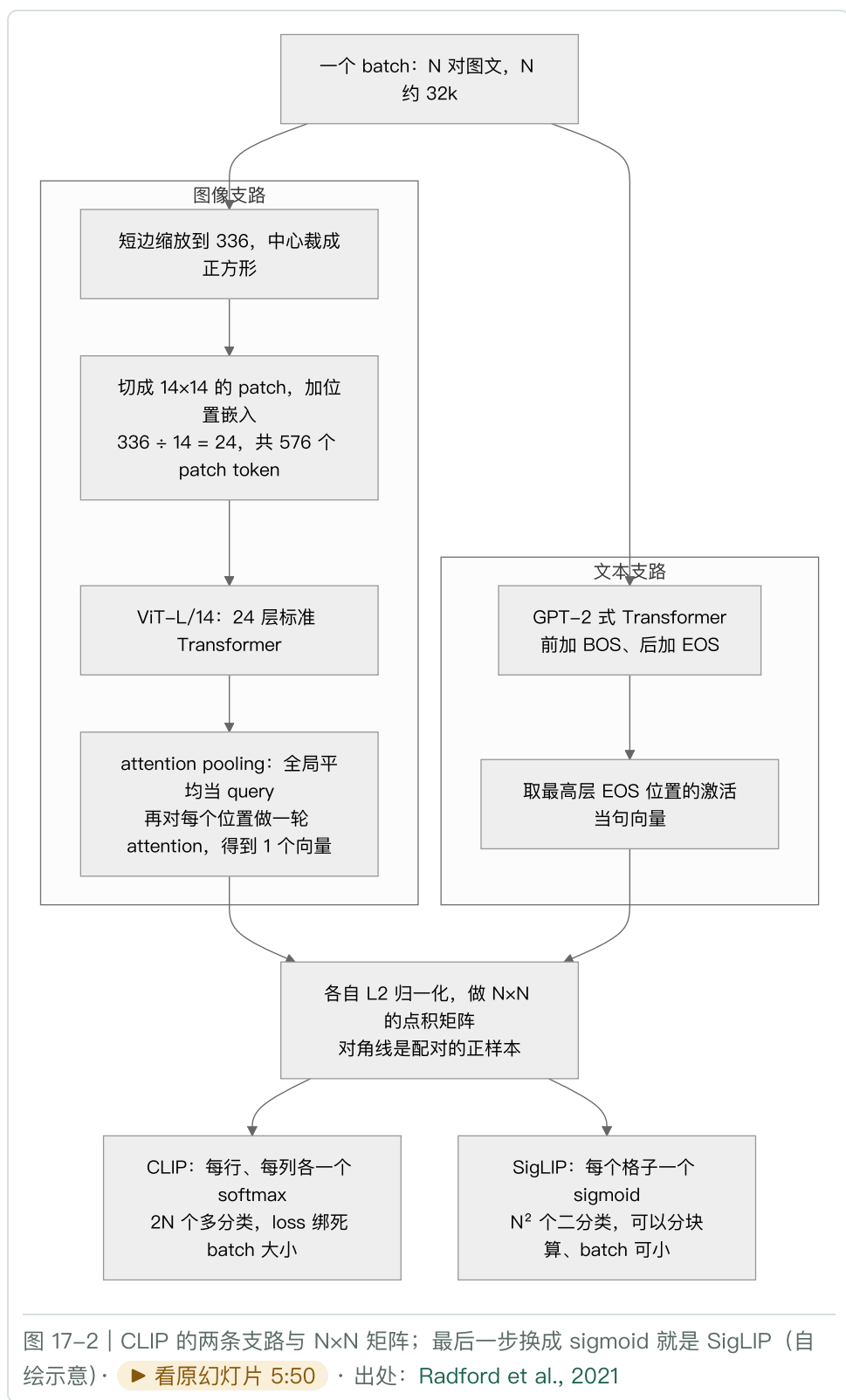


- **只剩一讲，选了多模态：**讲者开场说，原计划再多讲些强化学习，但一门课如果对多模态一字不提就不完整——看看今天所有头部模型，多模态无处不在。于是这一讲是多模态模型的综述（他自己说这够开一整门课）。课程表上的 "Alignment" 就此没有下文：RLHF / DPO 的推导见 CME295 第 5 讲与 CS224R 第 9 讲，Constitutional AI 见 CS329A 第 4 讲；本课程里离它最近的是第 15 讲的 mid/post-training 和第 16 讲的 RLVR。注意本讲里反复出现的 alignment 是另一个意思——把视觉向量对齐到 LLM 的词嵌入空间（第 4 节）。
- **语言模型已经很通用：**任意文本到任意文本——自然语言、代码、诗，甚至 DNA 序列。但世界是多模态的：文本、图像、音频、视频。

讲者给的北极星是 omni model：输入任意模态的任意组合，输出也是任意组合——给一张图加一段视频问问题、生成图像、把音频转成图。本讲不造整个 omni model，只讲怎么让模型"吃进"图像。

- **为什么绕不开 Transformer**：讲者说得很直白——大家试过别的架构，但在所有模态上、在大规模下，Transformer 仍是手里最好的东西，只能想办法用它。而它是为文本设计的：吃 token、吐 token。这里把 token 的概念放宽——不只是离散的 id，也可以是连续的向量（嵌入）。关键要求只有一条：一个 token 应当是一个语义单元。subword 勉强算，一个 pixel 肯定不算。
- **所以问题变成"造图像版的 BPE"**：文本其实也经历过这一步（第 1 讲的 BPE，作业里实现过的那个），只是不算太痛。非文本模态就得费心得多：什么东西相当于图像的 tokenizer，能把一张图变成 Transformer 能消化的一串 token？
- **两个问题**：一是非文本数据怎么输入 Transformer，二是怎么从 Transformer 生成非文本数据。本讲几乎全在答第一个；第二个只在 Chameleon（第 7 节）和收尾时点到。

2. CLIP (2021)：用 4 亿对网络图文，把图像编成"有语义"的向量



- **2021 年的处境:** 语言这边 GPT-2、GPT-3 已经进入基础模型时代——从互联网上爬一大堆噪声很大的文本, 模型够大就能学出有用的东西。视觉这边仍靠大规模人工标注集 (ImageNet) 训 ResNet, 再加数据增强。OpenAI 的问题是: 图像的“爬网页”在哪? 答案是网上海量的“图 + 说明文字”。

- **目标函数**：一个 batch 里有 N 对图文（课上的例子 $N = 32,000$ ）。图像编码器把每张图变成向量 I_1 到 I_N ，文本编码器把每段文字变成 T_1 到 T_N 。要求 I_1 与 T_1 的点积远大于 I_1 与其他任何 T_j 的点积；反过来 T_1 与 I_1 的点积也要大过 T_1 与其他任何 I_j 。于是每行、每列各是一个 N 类的 softmax 分类，一共 $2N$ 个分类问题，排成一个 $N \times N$ 的矩阵，对角线是正样本。课上的伪代码就四步：两个 $m \times d$ 的嵌入矩阵、归一化、带温度的点积、交叉熵。

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2N} \sum_{i=1}^N \left[-\log \frac{e^{\tau I_i \cdot T_i}}{\sum_{j=1}^N e^{\tau I_i \cdot T_j}} - \log \frac{e^{\tau I_i \cdot T_i}}{\sum_{j=1}^N e^{\tau I_j \cdot T_i}} \right]$$

I_i 、 T_i 是归一化后的图像与文本向量， τ 是温度的倒数（可学习）；中括号里第一项是“第 i 张图在 N 段文本里认出自己那段”，第二项是“第 i 段文本在 N 张图里认出自己那张”。分母跑遍整个 batch——这就是它离不开大 batch、又不可分解的原因（第 3 节）。

- **数据**：论文对数据说得不多——大致是拿一批查询词去搜，挖出图文对，共 4 亿对，没有公开。后来 OpenCLIP 复现并扩展了这条路线，用的是公开的 LAION-5B：50 亿张带文字说明的图，经过一堆处理——有意思的是过滤用的就是 CLIP 自己（有点自举的味道），然后再训 OpenCLIP。好处是至少数据和代码都指得出来（第 13—14 讲讲的数据过滤，在图文数据上同样是主角）。
- **图像预处理**：网上的图什么长宽比都有，而神经网络不喜欢动态尺寸。CLIP 的办法很粗暴：把短边缩放到 336（或 224），再从中间裁出正方形。这纯粹图方便，也因为作者们心里想的是 ImageNet 分类——主体通常在中间，裁掉点背景无妨。后面会看到（第 5 节）这一步能做得好得多。
- **问答 (11:06)**：为什么非要配文本，只用图像不行吗？有一条路线（SimCLR）靠数据增强：同一张图裁一下、扰动一下、转一下，要求嵌入相近。这对低层细节有用，但你没法把一种狗“增强”成另一种狗；文本给的是更高层的语义——看后面各家 VLM 想做的任务，就知道为什么要的是语义。
- **视觉编码器**：论文试了 ResNet 和刚出炉的 ViT，ViT 最好，所以今天说 CLIP 通常指 ViT 版本。ViT 的做法：把图切成小方块 patch（ViT 原文 16×16，CLIP 用 14×14），每个 patch 展平成向量就是一个 token，加位置嵌入，送进标准 Transformer。输出是一串向量，要一个向量代表整图：平均一下也行，但 CLIP 发现 attention pooling 更

好——用全局平均作 query，再对每个位置的 key、value 做一轮 attention，得到的向量比直接平均更有信息量。

- **最好的型号 ViT-L/14**：L 是 large（讲者说大概 24 层，没把握），14 是 patch 边长，输入 RGB 三通道、336×336。实际是先在低分辨率上训，后期为了速度才换到高分辨率。
 - 算一下 token 数： $336 \div 14 = 24$ ，一张图就是 $24 \times 24 = 576$ 个 patch token——后面 LLaVA 每张图注入 LLM 的 token 数就是这个 576。
 - 问答 (15:13)：位置嵌入要不要做成 2D 的？CLIP 论文试过 2D 与 1D，差别不大——但讲者提醒，这类结论都是在“分类”任务下得出的；后面的模型（第 6 节的 M-RoPE）会认真处理空间结构。

小注：ViT-L 的标准配置是 24 层、宽 1024、约 3 亿参数，讲者的“24 层”是对的。CLIP 论文里 ViT-L/14 先在 224×224 上训，最后再用 336×336 多训一个 epoch，即常说的 ViT-L/14@336px。

- **文本编码器**：GPT-2 风格的标准 Transformer（同一个团队做的）。要从序列得到一个向量：前面加 BOS、后面加 EOS，取最高层 EOS 位置的激活当整句的表示。
- **训练**：取一个 batch，图和文各编一遍，算那 $2N$ 个交叉熵，反传。
- **头条结果**：在 ImageNet 上，零样本的 CLIP 胜过用 120 万张 ImageNet 标注图训出来的 ResNet——那 120 万张图是无数小时的 Mechanical Turk 标注。零样本的做法：把每个类别名写成一句话过文本编码器，图像向量与各类别向量做点积，取最大。
- **问答 (18:49)**：caption 那么杂，别的图也提到狗，不会混淆吗？会有噪声，但平均下来没事——不会每次都是狗，有时是苹果、是猫。更根本的是，网页里图旁边的文字和 alt text 极其嘈杂，研究表明说明文字往往并不逐字描述图里有什么（图里是狗就不会再写“狗”），所以需要大量过滤；直接拿任意网页图文，噪声大到没法用。
- **一个伏笔**：论文还试过不做排序、改成“看图预测文字”（bag of words 或语言模型）。结果更强的语言模型目标反而更差、至少效率更低，不如 bag of words 和对比学习。说明就 ImageNet 分类这种粗粒度表示而言，精确建模 caption 的 token 序列并不重要。结尾会回到这一点：理解要的和生成要的不是一回事。
- **CLIP 小结与代价**：图像向量之所以有语义，是因为和文本配了对；设计决策全部围绕分类，所以不细粒度，但它是后面一切的稳健起点。

技术上的麻烦是 batch 必须很大（3 万左右，batch 为 1 或 10 根本不成立），而且 softmax 跨整个 batch，不可分解——语言模型训练里各条序列彼此独立、并行算完最后汇总即可，这里不行。

3. SigLIP (Google, 2023): softmax 换成 sigmoid, batch 和 loss 解耦



- **一句话的改动：**CLIP 做多分类——这对图文是正样本，去和其他所有图或文竞争；SigLIP (Sigmoid loss for Language–Image Pre-training) 更简单：任意一对图文，问“配不配”，二分类。对着那个 N×N 矩阵看，对角线标 +1，其余全标 -1，每个格子过一个 log-sigmoid，加起来就是 loss。

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \log \sigma(z_{ij}(t I_i \cdot T_j + b)), \quad z_{ij} = \begin{cases} +1 & i = j \\ -1 & i \neq j \end{cases}$$

σ 是 sigmoid, z_{ij} 是标签, t 与 b 是可学习的温度和偏置；每个格子的项只依赖自己这一对图文，所以可以分块算、跨设备加。课上只提了归一化点积和 log-sigmoid 两步。

小注：偏置 b 是论文里的细节，课上没提：初值取负，用来抵消开局时负样本 ($N^2 - N$ 个) 对正样本 (N 个) 的压倒性多数，否则训练一开始 loss 会被负样本主导。

```

def clip_loss(img, txt, t):
    # img, txt: [N, d], 已各自 L2 归一化
    logits = t * img @ txt.T
    # [N, N], t 是可学习的温度倒数
    labels = arange(N)
    # 第 i 行、第 i 列的正样本都是第 i 个
    return (ce(logits, labels) + ce(logits.T, labels)) / 2
    # 行、列各 N 个 softmax

def siglip_loss(img, txt, t, b):
    logits = t * img @ txt.T + b
    # b 是可学习偏置, 初值取负
    z = 2 * eye(N) - 1
    # 对角 +1, 其余 -1
    return -logsigmoid(z * logits).sum() / N
    # N^2 个互不相关的二分类

```

两个 loss 放在一起看：前者的每一项都要整行或整列的 logits 做分母，后者的每一项只看自己那个格子。

- **问答 (24:00)：要不要讲究采样、挑难负样本？** 讲者说提问者想的是对比学习里常见的顾虑——正负要平衡、负样本要“紧”——但至少在最初的论文里没有，就是老老实实用同一个矩阵。
- **数据：**Google 内部的 WebLI（另一篇 2022 年的图文模型论文撰写的），十亿量级的图文对；图里带文字的做 OCR，又是一种造图文对的方法；做了过滤，多语言。
- **训练效率**（论文的主要卖点）：CLIP 用 256 张 TPU v3 训了 10 天，SigLIP 用 32 张 TPU v4 训了 5 天。折成芯片·天：256 × 10 = 2,560 对 32 × 5 = 160，差 16 倍。讲者补充：别以为 v4 单芯片的 FLOP/s 比 v3 快多少——他说其实不快、甚至慢六成左右，v4 好在一个 pod 里能放更多芯片、互连更好（第 5 讲）；另外 CLIP 大概也没怎么为吞吐优化代码，先跑通再说。

小注：按 Google 公开规格，TPU v3 单芯片 bf16 峰值约 123 TFLOP/s，v4 约 275 TFLOP/s，v4 反而快 2 倍多，讲者“v4 更慢”一句与公开数据不符。就算按规格算总算力，SigLIP 用的也只有 CLIP 的约 1/7（32 × 275 × 5 对 256 × 123 × 10），效率结论不变。

- **怎么并行：**回想系统那几讲的 DDP（第 7 讲）——每台设备存一部分图文对。语言模型训练里各样本的 loss 彼此独立，这里不是：负样本来自别的设备。做法是第一趟各设备只用本地的图文算一小块 loss，然后把文本嵌入在设备间轮转——设备 1 收到 T5 到 T8 算一批负样

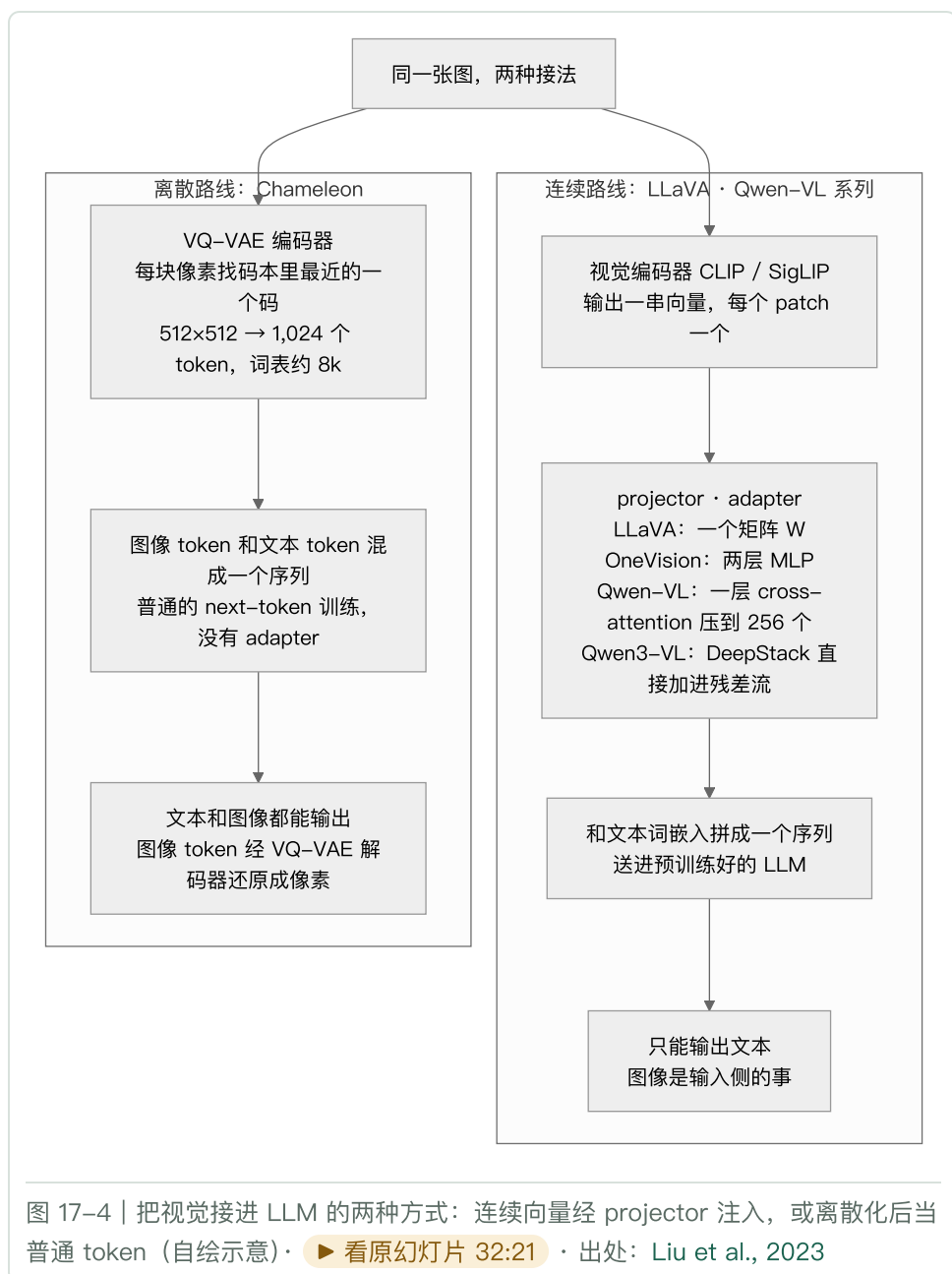
本，再收到 T9 到 T12，如此转一圈，直到 $N \times N$ 矩阵所有非对角块都算过（图 17-3）。

```
txt = txt_local                                # 每台设备：n 张
图、n 段文
loss = block_loss(img_local, txt, diag=True)    # 本地块：对角线上
有正样本
for _ in range(D - 1):                        # D 台设备围成一圈
    txt = ring_shift(txt)                    # 文本嵌入传给下一
台，收上一台的
    loss += block_loss(img_local, txt, diag=False) # 收到的块全是负样
本
loss = all_reduce(loss, op="sum")              # 每个格子恰好算过
一次
```

图像嵌入从不离开本机，跨设备搬的只是 $n \times d$ 的文本嵌入块，搬 $D - 1$ 次。

- **batch 与 loss 解耦**：CLIP 的 loss 绑死 batch 大小——batch 一变就是另一个损失函数；SigLIP 的 loss 期望不随 batch 变，batch 小只是方差大。于是他们能试小得多的 batch：16k 以下 SigLIP 明显好于 CLIP，因为 CLIP 在小 batch 下直接退化。再往大也试了，帮助不大——32k 基本就是它的临界 batch size（第 9 / 11 讲讲过的 critical batch size）。
- **到这里的成果**：CLIP 和 SigLIP 都是“图像编码器”——吃一张固定尺寸（比如 336×336 ）的图，吐出带语义的向量。接下来就是把它们接到语言模型上。

4. VLM 的模板与 LLaVA（2023）：一个矩阵 W 把视觉向量翻译成词嵌入



- **模板：**拿一个现成的图像编码器、一个现成的 LLM，中间加一个小部件把前者的输出变成后者的输入，缝起来——这是 mid-training / post-training 的味道（第 15 讲），不是从零训。讲者挑了两个家族对照：LLaVA（开源，连数据都公开）和 Qwen-VL（多代演进、性能强）。
- **LLaVA 的时代背景：**2023 年，闭源的 GPT-4 已经能做视觉推理；LLaVA 用开源零件也做到了一点，虽然远不如 GPT-4，但大家第一次看见这类模型内部长什么样。
- **三个零件：**视觉编码器 = CLIP ViT-L/14；文本解码器 = Vicuna（第一代 LLaMA 在 ShareGPT 上微调的版本，ShareGPT 是用户分享自己与 ChatGPT 对话的网站，讲者说现在应该已经不在了）；数据 = 从

MS COCO 出发合成——COCO 里每张图有人工标的 caption 和物体框，把这些文字喂给只看文字的 GPT-4，让它生成三类东西：围绕这张图的问答对话、详细描述、复杂推理。共 158,000 条。

小注：论文里三类的份额是对话 58k、详细描述 23k、复杂推理 77k。生成时 GPT-4 看不到图，只看 caption 和框坐标——这是“数据即任务”的最早版本。

- **接线：**图 → CLIP → 一串向量。但这些向量并不住在文本词嵌入的空间里，所以乘一个矩阵 W 投影过去；文本照常查词嵌入表。两串向量拼成一个序列送进标准 Transformer，输出文本。讲者的说法：相当于把图像变成“文本 token”，好白嫖预训练好的语言模型。

$$H_v = W g(X_v), \quad p(y \mid X_v, X_q) = \prod_{t=1}^T p_{\theta}(y_t \mid H_v, X_q, y_{<t})$$

g 是冻结的 CLIP 编码器， X_v 是图像， W 是投影矩阵， X_q 是问题文本， y 是回答；loss 只算回答的 token，和第 15 讲的 SFT 一样。

```
def llava_forward(image, prompt_ids, answer_ids):
    z = clip_vit(image) # [576, d_v]: 冻结的编码器，
    # 每个 patch 一个向量
    h_img = W(z) # [576, d_lm]: 投影到 LLM
    # 的词嵌入空间
    h_txt = embed(prompt_ids + answer_ids) # 文本照常查表
    h = concat(h_img, h_txt) # 拼成一个序列，图像 token 在前
    return lm(h).loss_on(answer_ids) # 标准 next-token loss, 只算回答部分

trainable = {1: [W], 2: [W, lm]} # 阶段 1 只训 W; 阶段 2 训 W 和 LLM; ViT 全程冻结
```

整个“多模态”就是一个矩阵乘法加一次拼接，其余全是现成的语言模型。

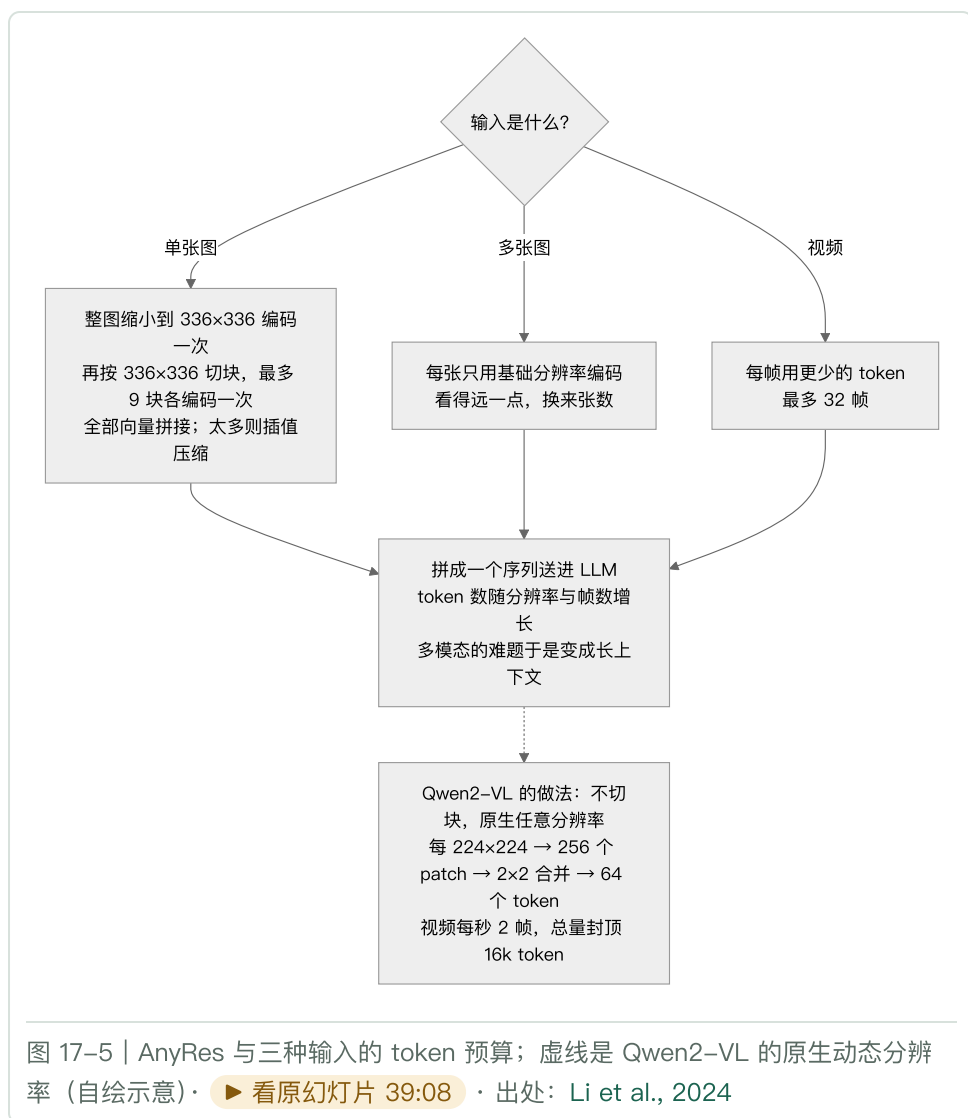
• 两阶段训练

1. 对齐 (alignment)：冻结编码器和 LLM，只训 W 。 W 随机初始化时，投影出来的向量不像任何自然语言 token 的嵌入；训 W 就是让它们“像”。
2. 视觉指令微调：编码器仍冻结，训 W 和 LLM，数据就是那 158k 条“图 + 对话 / 描述 / 推理 → 文本”。

- **论文里的例子**：一张图，用户问“这张图哪里不对劲”，模型答出“一般不在面包车后面熨衣服”。作者强调即使提问没有暗示“反常”，模型也会主动说出来——当时 GPT-4 能做到，其他开源模型不行。
- **问答（后半段的两个问题一并放在这）**
 - **1:02:57** 对齐阶段的 LLM 是预训练好的吗？必须是，否则“对齐”无从谈起；LLM 冻结，只训 adapter 把给定的编码器接到给定的 LLM 上。训多久？定一个 token 预算就训，讲者在训练表上念出的数是 67B token（应是 Qwen3-VL 报告里对齐阶段的数），没有自适应的停止条件。
 - **1:04:00** 视觉编码器的参数为什么比 LLM 小那么多？因为它干的是很局部的活——看一个个小 patch、理解 patch，既不需要知识也不做推理，能力都在 LLM 里。数量级：LLaVA-OneVision 的例子是 LLM 720 亿参数、projector 7,200 万、ViT 不到 10 亿。

小注：OneVision-72B 的 projector 是两层 MLP，从 SigLIP 的 1152 维升到 Qwen2-72B 的 8192 维， $1152 \times 8192 + 8192 \times 8192 \approx 7,600$ 万，与“7,200 万”量级相符（推断）。

5. LLaVA-OneVision (2024)：同一模板换零件，AnyRes 把任意分辨率、多图和视频都变成 token 序列



- **升级清单:** LLaVA 之后有 LLaVA-1.5、LLaVA-NeXT 一串论文, 讲者用 2024 年的 OneVision 一次讲完。配方不变, 野心更大——要处理多图和视频 (视频就是抽样出来的一串帧)。视觉编码器换成 SigLIP; LLM 换成当时最强的开源模型 Qwen2; projector 从线性换成两层 MLP。讲者的比喻: 系统架构一样, 只是零件在升级。
- **OCR 逼出的问题:** 读文字需要极细的信息, 否则 J 和 I 分不清。而 CLIP 那一套是缩放再裁成 336x336——一页文档这么处理根本没法读。
- **AnyRes** (LLaVA-1.5 引入): 既然编码器只吃 336x336, 那就不缩小, 而是把大图切成若干块, 每块正好是编码器的尺寸, 各自编码, 把向量拼接起来。另外保留一路: 整图缩小后编码一次, 给全局视角。块太多就用插值把 token 数压下来。讲者点出这里的美感: Transformer 本来就能处理任意长度的句子, AnyRes 让图像也享受这种"任意"。

```
def anyres_tokens(img, encode, res=336, max_tiles=9):
    views = [resize(img, res)] # 一路：整图缩到
336×336, 看全局
    views += tile(img, res)[:max_tiles] # 多路：按 336×336
切块, 最多 9 块
    toks = concat([encode(v) for v in views]) # 每块 24×24 =
576 个向量
    return interpolate_down(toks) if too_long(toks) else toks # 太长
就插值压缩
# 多图：每张只走第一路；视频：每帧更少 token, 最多 32 帧——三种输入的预算被压到同
一量级
```

编码器本身一点没改, "高分辨率"完全靠多调几次编码器、多占一些上下文换来。

- **三种输入的 token 预算：**图、多图、视频原则上都是"一堆图", 但作者刻意压了压天平——视频可能很长, 不能让重复的帧淹没数据集。单图：全图缩略 + 最多 9 块；多图：每张只用基础分辨率（看得远一点, 换来张数）；视频：每帧更少 token, 最多 32 帧。讲者的判断：能不能做好多模态, 很大程度上就是能不能处理长上下文（第 4 讲的注意力替代方案、第 10 讲的 KV cache 账都为此服务）。

小注：论文里的数字——SigLIP 每块 729 个 token, 单图最多 1 + 9 块共 7,290 个 token；多图每张 729；视频 32 帧、每帧压到 196 个, 共 6,272 个。三者被有意做成同一量级（应为这些数）。

- **数据哲学：**"高质量、大数量", 讲者的翻译是：非常有针对性——看数据表几乎全是任务：VQA、表格问答、两张图找不同……这明摆着是后训练的地盘。而且这项工作毫不掩饰地蒸馏 GPT-4 类模型——不理想, 但没有标注预算就只能这么干。
- **三阶段：**第一阶段仍是只训 projector 的对齐；第二阶段训高质量、偏"知识"的数据；第三阶段全模型一起训、数据长得像下游任务。讲者说从两阶段变三阶段没什么原理上的理由。
- **跨模态的迁移** (43:15, 论文里最有意思的发现)：图表和示意图只有单图数据, 测试时却能对"一张表 + 一张图"两张图展开对话；OCR 只在单图上训、关系推理只在多图上训, 合起来能看一串截图做 GUI agent；"视觉提示"（图上画个圈指定区域）只有单图数据, 却能泛化到视频——"描述视频里圈出来的球员", 而那个球员横跨多帧。讲者第一眼的反应是"这不就是每个任务做监督学习吗", 但任务够多时确实出现迁移, 令人安心。

- LLaVA 系列的价值：少数把数据也开源的工作，可以真正复现和研究。

6. Qwen-VL → Qwen2-VL → Qwen3-VL：动态分辨率、M-RoPE、DeepStack，阶段越来越多

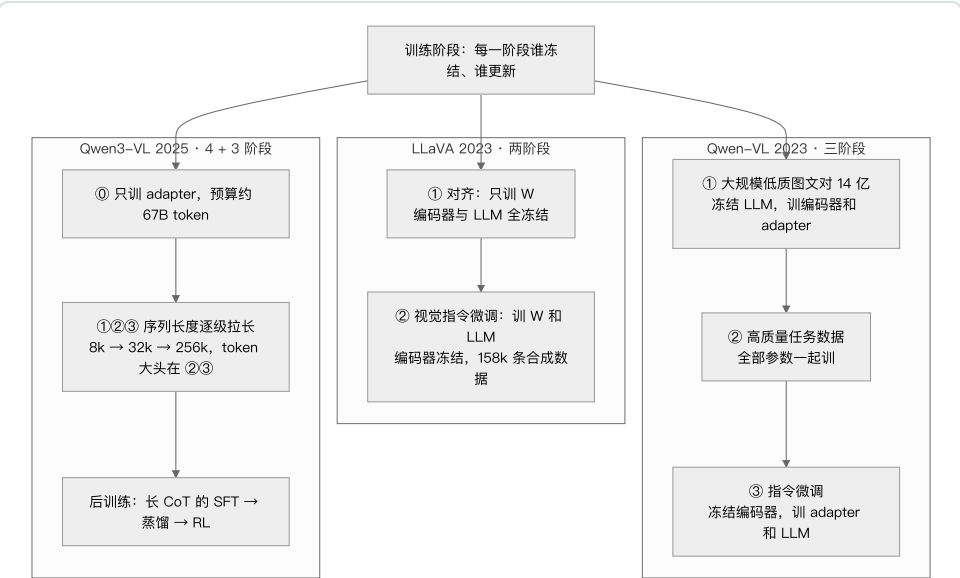


图 17-6 | 三代 VLM 的训练阶段：都从"只训 adapter"起步，之后越来越长（自绘示意）· [▶ 看原幻灯片 57:46](#) · 出处：Qwen3-VL

- **Qwen-VL (2023)**：视觉编码器用 OpenCLIP；adapter 是一层 cross-attention，带 2D 位置嵌入，把整张图压成固定的 256 个 token——讲者说这显然不"动态"，不过当时的编码器本身也不动态。特殊 token 有 image、box（物体框）、ref（指代描述）三种——幻灯片用 HTML 写的，这几个尖括号标签被浏览器吃掉了。训练三阶段：
 1. 他们叫"预训练"但并非从零——大规模低质量图文对，14 亿条；冻结 LLM，训视觉编码器和 adapter。和 LLaVA 的差别就在这：视觉编码器也训。
 2. 高质量任务数据（VQA、图表问答等），全部参数一起训。
 3. 指令微调：冻结视觉编码器，训 adapter 和 LLM。
 - 能做的事：中文示例、看图写代码（"语言模型会的它都得会"）、输出物体框（以文本形式输出坐标，不是画图）、OCR、多图比较。
- **Qwen2-VL**：编码器更大；主要变化是原动态分辨率——同 LLaVA 走向 OneVision 时的领悟一样：图有大小，视频更是逼着你动态处理。一张大图可以映射成 11,000 个 token，一个小公式截图只要 8

个。做法：每个 224×224 的块过一个 ViT（起点是 OpenCLIP 的 ViT，但会被微调），然后每 2×2 个 patch 合并成一个 token 以压缩上下文，224×224 最终变成 64 个 token；视频每秒抽 2 帧，总量封顶 16k token。

$$n_{\text{img}} = \frac{H}{14} \times \frac{W}{14} \times \frac{1}{2 \times 2} \quad \Rightarrow \quad 224 \times 224 : \frac{16 \times 16}{4} = 64$$

H、W 是图像的高和宽（像素），14 是 patch 边长，2×2 是合并的窗口；边长翻倍，token 数翻四倍，所以 11,000 与 8 这两个极端都出得来。

- **M-RoPE（多模态 RoPE）**：RoPE（第 3 讲）让两个向量的内积只依赖相对距离，一维时距离就是隔了几个 token。多维版本一样，只是位置变成三元组（时间 t、高 h、宽 w），每个 patch 一个三元组；对每一维分别算 RoPE，再拼接起来。讲者说这个设计后来在 Qwen3 里被发现不太理想。
- **Qwen3-VL（2025 年的报告）**：讲者说没有结构性大变，但每一处都影响质量：
 - LLM 换成 Qwen3 系列（dense 与 MoE 都有），底座本身很强，对最终质量帮助最大。
 - 长上下文：上下文长度到 256K，长视频离不开它。
 - 视觉编码器换成 SigLIP 2——架构与 SigLIP 完全相同、向后兼容的改进版。
 - 交错的 M-RoPE：原来是把嵌入维度切成三段，第一段给时间、第二段给宽、第三段给高。但 RoPE 每个维度对应一个频率，这么分意味着（按讲者的说法）时间轴只拿到低频、高度轴只拿到高频。改法是三个轴交错着分配维度，每个轴都同时有高频和低频。
 - 显式时间戳：以前视频帧的时间只隐含在位置编码里；现在插入"第 0 秒"这样的 token，让模型能直接指着时间说话（"两秒之后发生了什么"）。
 - 按 $\sqrt{\text{长度}}$ 长度归一化的 per-token loss：视频样本极长、单图样本很短，按 token 平均等于让视频主导。讲者的理解（他说细节不清楚）是每个样本的 loss 除以其长度的平方根，压低超长样本的权重。
 - adapter 升级为 DeepStack：从 LLaVA 的线性、到 MLP、到 cross-attention，这次更进一步——视觉编码器本来就逐层算出一叠特征，DeepStack 把它们直接加进 LLM 的残差流（residual

stream, 各层 attention / MLP 的输出不断累加的那条主干), 而不是把编码器当黑盒只取最后一串向量。讲者称之为更深的融合。

小注: DeepStack 应出自 Meng et al., 2024 (复旦与 Microsoft), 把视觉 token 分成 N 组, 从底到顶分别送进 LLM 的前 N 层; 讲者说是 DeepSeek 团队的论文, 应为口误。

$$\text{pos} = (t, h, w), \quad \theta_g = b^{-2g/d}, \quad \text{Qwen2-VL: axis}(g) = \begin{cases} t & g < g_1 \\ h & g_1 \leq g < g_2 \\ w & g \geq g_2 \end{cases} \quad \text{Qwen3-VL: axis}(g) = (t, h, w)_{g \bmod 3}$$

每个 patch 的位置是三元组; 第 g 组维度按频率 θ_g 旋转, 旋转角用 axis(g) 那一维的坐标。Qwen2-VL 按连续的段分配轴, 于是每个轴只见到一段频率; Qwen3-VL 改为轮流分配, 每个轴都覆盖全频段。公式是按讲者的描述整理的, 分段边界 g_1 、 g_2 与取模写法只是示意。

$$\mathcal{L} = \sum_e \frac{1}{\sqrt{L_e}} \sum_{t=1}^{L_e} \ell_{e,t}$$

L_e 是第 e 个样本的 token 数, $\ell_{\{e,t\}}$ 是逐 token 的交叉熵; 不归一化时一个样本的权重正比于 L_e , 除以 $\sqrt{L_e}$ 后只正比于 $\sqrt{L_e}$ ——比如一段 6,000 个 token 的视频, 从 6,000 份权重降到约 77 份 (讲者说这是他的理解, 报告细节不清楚)。

- **训练流水线**: 预训练四个阶段、后训练三个阶段——讲者感叹流水线已经相当复杂。阶段 0 照例只训 adapter; 接着三个阶段把序列长度逐级拉长: $8k \rightarrow 32k \rightarrow 256k$, token 大头在中间两个阶段。后训练: 先在长 chain-of-thought 数据上做 SFT, 再知识蒸馏, 最后强化学习 (第 16 讲)。讲者说到这里已经是一篇系统论文, 他只挑了核心新点子。
- **结果**: 和闭源模型 Gemini、GPT-5、Opus 4.1 同台, 表里粗体是每行最好——Qwen 拿下不少, 相当强。
- **数据细节**: 后期 Qwen 报告对数据配比说得很少, 要看细节得去 LLaVA 系列或 AI2 的 Molmo 论文 (没来得及讲)。从 Qwen-VL 到 Qwen3-VL, 框架没变, 变的是规模、更多数据集、意识到必须处理长上下文、打磨图像处理。
- **问答**
 - 59:47 会生成视频吗, 怎么决定输出图还是文? 这些模型不生成图像或视频——多模态全在输入侧, 输出永远是文本。除 RL 外的所有阶段都是逐 token 监督, 没有 LLM 当裁判, 数据集里是什么就学什么; 到了 RL 阶段才能玩各种 reward。

- 1:00:53 系统上比纯语言模型难吗？肯定不更容易。数据集大得多，视频光是加载就可能成为瓶颈——讲语言模型时从没操心过数据加载，因为太便宜；这里必须让加载和计算异步。视频 token 多，所以要归一化或降权——两周前讲的数据配比（第 14 讲）就是干这个的。不过文本也有几十万亿 token，多模态 token 并没有压倒性地多。

	LLaVA 2023	LLaVA–OneVision 2024	Qwen–VL 2023	Qwen2–VL 2024	Qwen3–VL 2025	Chameleon 2024
视觉编码器	CLIP ViT–L/14	SigLIP	OpenCLIP	更大的 ViT，OpenCLIP 起点，可微调	SigLIP 2	VQ–VAE tokenizer（离散）
ada 线性 W pte r		两层 MLP	一层 cross–attention → 256 token	2×2 patch 合并	DeepStack 注入残差流	无
分辨率	336×336 裁剪	AnyRes：缩略 + 最多 9 块	固定 256 token	原生动态，224×224 → 64 token	动态 + 256K 上下文	512×512 → 1,024 token
位置	1D	1D	2D 位置嵌入	M–RoPE（t、h、w 分段）	交错 M–RoPE + 显式时间戳	1D
数据	158k 条 GPT–4 合成	任务化合成，蒸馏 GPT–4	阶段 1 用 14 亿对	未展开	未展开	文本 + 图文混合
阶段	2	3	3（阶段 1 训编码器）	3	4 + 3，8k → 32k → 256k	2
输出	文本	文本	文本（框以文本坐标输出）	文本	文本	文本与图像交错

7. Chameleon (Meta, 2024)：全部离散化，用 VQ–VAE 把图像变成 token

- 动机：前面的 VLM 把图编成向量塞进语言模型，所以只能输出文本。修法有多种——给 VLM 挂一个 diffusion 头也行——但 Chameleon 的想法是：把一切都变成离散 token。讲者承认这里有他作为"做语言的人"的审美偏好：分析和生成图像都和文本一模一样，一个 prompt 后面接着生成的就是图像 token，或者反过来；论文里"我无聊了，给我看些鸟"会得到文字、图片、文字、图片交错的回复。omni model

的愿景——文本和图像真正住在同一个空间——用"让一切长得像文本"来实现（图 17-4 右）。

- **VQ-VAE (2017)**：把图像映射到离散编码。编码器先把每块像素变成连续向量，再"取整"到码本里最近的一个码——码本是几千个原型向量（课上说 8,000 个），每个对应某种 patch；解码器从码重建图像；训练目标是重建误差，因取整不可导还要加几项，课上略过。

$$k^* = \arg \min_k \|E(x) - e_k\|_2, \quad z_q = e_{k^*}, \quad \mathcal{L} = \|x - D(z_q)\|_2^2 + \|\text{sg}[E(x)] - e_{k^*}\|_2^2 + \beta \|E(x) - \text{sg}[e_{k^*}]\|_2^2$$

E 是编码器、D 是解码器，e_1 到 e_K 是码本（K 约 8k），sg 是 stop-gradient；第一项是课上说的重建误差，后两项就是课上略过的"因为取整不可导而加的项"——一项把码本拉向编码器的输出，一项让编码器别乱跑；argmin 本身靠直通估计把梯度原样传回编码器。

- **数字**：512×512 的图变成 1,024 个 token，每个来自 8k 大小的词表——即 32 × 32 的网格，每个 token 管 16×16 个像素。因为数据形态变了，他们还重新训了文本 tokenizer。

小注：论文里的码本大小是 8192；讲者说的"8,000"是约数。

- **训练**：就是普通的语言模型训练——没有 adapter、没有视觉编码器、没有"冻结谁训谁"。仍分两阶段（语言模型本来也常这样）：第一阶段大规模无监督的文本加图文混合，第二阶段混入高质量数据。
- **代价**
 - 训练不稳。文本和图像虽然同处一个 token 空间，行为却完全不同：下一个词大多可预测，熵低；图像 token 熵极高——"这个位置到底是哪种蓝"没法预测。混训导致参数范数持续增长、loss 发散，他们用 QK-norm 和 z-loss 正则（第 3 讲讲过的两个稳定训练的手段）压住了范数。
 - 性能不如连续路线，离散化必然丢信息——小字 OCR 一离散就读不了。
 - 多模态的加权更难：Qwen 那边已经要调权重，这里更严重。
- **历史位置**：VQ-VAE 曾流行过一阵，主要用于图像生成——Transformer 只能生成离散的东西，那就把数据都离散化。后来 diffusion 模型成熟，这条路就没那么热了。

8. 收束：讲者的判断

- **前沿模型都在说"原生多模态"**：Gemini、GPT 发布时都强调原生多模态、omni，也确实能做到，但没有任何细节。讲者的推测：多半是连

续编码器负责理解（不想丢信息），diffusion 负责生成。

- **根本难题**是非文本模态怎么进、怎么出。理解和生成之间有一种对称，但没有万能的编码器：CLIP 为分类而生，向量小、只抓高层语义；OCR 或生成图像则需要极细的高频信息——diffusion 擅长的正是这个。第 2 节的伏笔在这里收：预测 caption 这种更"生成"的目标，对学分类用的表示反而没帮助。
- **模态要称重**：视频的信息密度比文本低得多，不能让它淹没文本（Qwen3-VL 的 $\sqrt{\cdot}$ 归一化、OneVision 的帧预算都是这件事）。
- **现状**：连续编码器仍是最佳实践——CLIP 已经 5 岁，它那套思路还是抓语义的默认选择；Transformer 还在；生成靠 diffusion（本讲没讲）。没有对应的作业，讲者鼓励自己动手训一个玩玩。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
omni model 示意	输入输出各是文本 · 图像 · 音频 · 视频的任意组合；本讲只做"图像进、文本出"	1:08	—
CLIP 的 NxN 矩阵与伪代码	对角线是配对的正样本；每行、每列各一个 softmax；伪代码里先归一化、乘温度、再对称交叉熵	5:50 · 7:25	CLIP
resize + center crop 与 ViT 分块	短边缩到 336 再裁正方形（想想一页文档剩什么）；每个 14x14 的小块是一个 token	9:30 · 12:37	ViT
ImageNet 零样本对比	零样本 CLIP 对 120 万标注图训的 ResNet；零样本 = 图向量与各类别文本向量做点积取最大	17:18	CLIP
SigLIP 伪代码	标签矩阵对角 +1、其余 -1，然后 log-sigmoid；没有 softmax 的分母	23:29	SigLIP
SigLIP 分块并行	每台设备一块图文；文本嵌入轮转，对角块之外都是负样本	26:33	同上
batch size 曲线	16k 以下 SigLIP 明显高于 CLIP；32k 之后再大也不涨	27:38	同上
LLaVA 合成数据与架构图	COCO 的人工 caption 与物体框 → GPT-4 生成三类数据；图 → CLIP → 乘 W → 与文本嵌入拼接 → LLM	31:20 · 32:21	LLaVA
AnyRes 与三种输入的预算	整图缩略一路 + 切块多路；单图 9 块、多图每张 1 块、视频 32 帧	39:08 · 40:11	LLaVA- OneVision
跨模态迁移示例	单图的图表数据 → 多图对话；单图的圈选提示 → 视频里被圈的球员	43:47	同上

图	看什么	跳转	出处
Qwen-VL 三阶段训练表	阶段 1 用 14 亿对、冻结 LLM 训练编码器；阶段 2 全参数；阶段 3 冻结编码器	47:24	Qwen-VL
Qwen2-VL 动态分辨率	同一模型里一张图 11,000 个 token、一个公式 8 个 token	49:29	Qwen2-VL
Qwen3-VL 架构与 DeepStack	编码器多层特征直接加进 LLM 残差流；旁边是交错 M-RoPE 与时间戳 token	53:08 · 56:46	Qwen3-VL
Qwen3-VL 训练表与结果表	序列长度 8k → 32k → 256k；结果表里的粗体对 Gemini、GPT-5、Opus 4.1	57:46 · 58:46	同上
Chameleon 交错生成示例	"我无聊了，看些鸟"→ 文字与图片交错输出；输出侧也有图像	1:08:43	Chameleon
VQ-VAE 示意	编码 → 取整到码本最近的码 → 解码重建；512×512 → 1,024 个 token	1:09:50	VQ-VAE

提到的工作

名称	在本讲里的作用
CLIP (Radford et al., 2021)	全讲的起点：对比学习训出的图像编码器；4 亿对图文；ViT-L/14；零样本 ImageNet
ImageNet · ResNet	2021 年之前视觉的主流范式：120 万张人工标注图上训分类器；被零样本 CLIP 超过
SimCLR (Chen et al., 2020)	问答里对照的另一路线：靠数据增强学不变性，抓不到"另一种狗"这样的语义
ViT (Dosovitskiy et al., 2020)	视觉编码器的骨架：16×16 patch 当 token；CLIP 改用 14×14
GPT-2	CLIP 文本编码器的架构来源（同一团队）
OpenCLIP (Cherti et al., 2022) · LAION-5B	CLIP 的开源复现与 50 亿对公开数据；过滤用的是 CLIP 自己；Qwen-VL 的编码器
SigLIP (Zhai et al., 2023)	sigmoid 替代 softmax；WebLI 数据；32 张 v4 训 5 天；分块并行；OneVision 的编码器
PaLI / WebLI (Chen et al., 2022)	SigLIP 数据的来源——"另一篇 2022 年的图文模型论文"，应为这篇
SigLIP 2 (Tschannen et al., 2025)	架构不变、向后兼容的改进版；Qwen3-VL 的编码器
TPU v3 / v4	训练效率对比的硬件单位（第 5 讲）
LLaVA (Liu et al., 2023)	VLM 模板的最简版：CLIP + 线性 W + Vicuna；158k 条 GPT-4 合成数据；两阶段
Vicuna · ShareGPT	LLaVA 的文本解码器：LLaMA 在用户分享的 ChatGPT 对话上微调

名称	在本讲里的作用
MS COCO (Lin et al., 2014)	人工标注的 caption 与物体框, LLaVA 合成数据的原料
GPT-4	视觉推理的标杆, 也是 LLaVA 系列合成数据的来源 ("毫不掩饰的蒸馏")
LLaVA-1.5 · LLaVA-NeXT	中间几代, AnyRes 在 1.5 引入; 讲者把创新一并归到 OneVision 里讲
LLaVA-OneVision (Li et al., 2024)	SigLIP + 两层 MLP + Qwen2; 单图 / 多图 / 视频的 token 预算; 三阶段; 跨模态迁移; 数据开源
Qwen2 · Qwen3	OneVision 与 Qwen3-VL 的语言底座
Qwen-VL (Bai et al., 2023)	OpenCLIP + cross-attention 压成 256 token; 三阶段, 第一阶段 14 亿对且训编码器
Qwen2-VL (Wang et al., 2024)	原生动态分辨率、2x2 合并、M-RoPE、视频 2 fps 封顶 16k token
Qwen3-VL (2025)	交错 M-RoPE、显式时间戳、 $\sqrt{}$ 归一化 loss、DeepStack、4 + 3 阶段、256K
RoPE (Su et al., 2021)	M-RoPE 的一维原型 (第 3 讲)
DeepStack (Meng et al., 2024)	视觉特征分组注入 LLM 前几层的残差流; 讲者误记为 DeepSeek 团队
Molmo (Deitke et al., 2024)	AI2 的开放 VLM, 数据细节可查; 讲者没来得及讲
Gemini · GPT-5 · Claude Opus 4.1	Qwen3-VL 结果表里的闭源对照
Chameleon (Meta, 2024)	全离散化的早期融合模型; 训练不稳、QK-norm 与 z-loss
VQ-VAE (van den Oord et al., 2017)	图像离散化: 码本取整 + 重建
QK-norm · z-loss	压住范数增长的两个正则手段 (第 3 讲; z-loss 出自 PaLM)
diffusion 模型	生成侧的主流; 讲者推测前沿模型用它生成, 本讲未展开

术语对照

English	中文
omni model	全模态模型: 任意模态组合进、任意组合出
modality	模态: 文本、图像、音频、视频
natively multimodal	原生多模态: 一开始就按多模态设计训练, 而非事后拼接
discrete / continuous token	离散 token (词表里的 id) / 连续 token (嵌入向量)
contrastive learning	对比学习: 拉近配对的、推开不配对的

English	中文
image-text pair	图文对
alt text	网页图片的替代文字，图文对的常见来源
zero-shot	零样本：不在该任务的标注数据上训练直接用
center crop	中心裁剪
patch	图像小块：ViT 的 token 单位
ViT (Vision Transformer)	视觉 Transformer
attention pooling	注意力池化：用全局平均当 query 再做一轮 attention，得到单个向量
temperature	温度：logits 的缩放系数
sigmoid loss	sigmoid 损失：每对图文独立的二分类
critical batch size	临界 batch 大小：再增大 batch 也不省步数的拐点
DDP (distributed data parallel)	分布式数据并行（第 7 讲）
VLM (vision-language model)	视觉语言模型
vision encoder	视觉编码器
projector / adapter	投影器 / 适配器：把视觉向量映射到 LLM 输入空间的小部件
alignment（本讲义）	模态对齐：只训 adapter，让视觉向量像词嵌入；与 RLHF 的"对齐"无关
visual instruction tuning	视觉指令微调
AnyRes	任意分辨率：整图缩略一路加切块多路
dynamic resolution	动态分辨率：token 数随图像大小变
OCR	光学字符识别：读图里的文字
VQA (visual question answering)	视觉问答
GUI agent	看屏幕截图操作界面的 agent
visual prompting	视觉提示：在图上画圈等标记来指定区域
cross-modal transfer	跨模态迁移：单图学的能力用到多图或视频上
distillation	蒸馏：用强模型的输出当训练数据
M-RoPE (multimodal RoPE)	多模态旋转位置编码：位置是（时间、高、宽）三元组
interleaved	交错：频率维度轮流分给三个轴；或文本与图像交错排列
timestamp token	时间戳 token：显式写出"第几秒"
per-token loss normalization	逐 token 损失的归一化：按样本长度的平方根缩放

English	中文
residual stream	残差流：各层输出不断累加的主干
DeepStack	把视觉特征分组加进 LLM 前几层的残差流
early fusion	早期融合：模态在输入 token 层面就合并
VQ-VAE (vector-quantized VAE)	向量量化变分自编码器：连续向量取整到码本
codebook	码本：K 个原型向量
quantize（本讲义）	离散化到码本（区别于第 5 讲的数值精度量化）
reconstruction loss	重建误差
stop-gradient / straight-through	截断梯度 / 直通估计：让不可导的取整能反传
entropy	熵：下一个 token 有多难猜
QK-norm	对 query、key 做归一化，防 attention logits 爆炸（第 3 讲）
z-loss	惩罚 softmax 归一化项过大的正则（第 3 讲）
diffusion model	扩散模型：生成侧的主流
information density	信息密度：每个 token 携带多少信息，视频远低于文本
data loading	数据加载：多模态训练的新瓶颈
knowledge distillation	知识蒸馏（Qwen3-VL 后训练的一步）

字幕勘误

"GPD3 / GP4 / GPD4 / GPD5" → GPT-3 / GPT-4 / GPT-5; "BP tokenizer" → BPE tokenizer; "imageet" → ImageNet; "resnats" → ResNets; "sim clear" → SimCLR; "vit l14"、"14x4 patches" → ViT-L/14、14×14 patches; "336 x36" → 336×336; "the 1DNA" → the 1D version; "zero shop" → zero-shot; "lion 5B" → LAION-5B; "cichlip / cig" → SigLIP; "web li" → WebLI; "two end cross entropy" → 2N 个交叉熵; "Lava / LA lava" → LLaVA; "Vunia" → Vicuna; "MS Coco / MSCO" → MS COCO; "any res / NRA idea" → AnyRes; "guey agents" → GUI agents; "Quen / Quend / Quinn" → Qwen; "ande models" → MoE models; "66 tokens" → 64 tokens ($224 \div 14 = 16$, $16^2 \div 4 = 64$); "M rope" → M-RoPE; "deep stack" → DeepStack; "mobile paper" → Molmo paper; "command paper" → Chameleon paper; "VQEs / VKU VAEs" → VQ-VAEs; "a,024 tokens" → 1,024 tokens; "interled" → interleaved。

带走的问题

1. CLIP 的 loss 为什么离不开 3 万的 batch, 而 SigLIP 换成 sigmoid 后"期望不变、方差变大"具体指什么? 如果把 SigLIP 的负样本从 $N^2 - N$ 个减到每张图只配 k 个, loss 的期望还一样吗? 分块并行的通信量又会怎么变?
2. 用第 2 节的算术: LLaVA 一张图 576 个 token, OneVision 单图最多 7,290 个, Qwen2-VL 一张 1792×1792 的图占多少? 按第 10 讲的 KV cache 账, 这些 token 在 72B 模型上每层要多少字节? 视频封顶 16k token、每秒 2 帧, 意味着能看多长的视频?
3. 视觉编码器不到 10 亿参数而 LLM 720 亿, 讲者的理由是"编码器只做局部的事"。把 Qwen-VL 第一阶段"连编码器一起训"和 LLaVA "编码器全程冻结"放到同一份数据上比, 你预期哪个在 OCR 上更好、哪个更容易把 CLIP 学到的语义忘掉?
4. Chameleon 的不稳定被归因于文本 token 熵低、图像 token 熵高。
 $\sqrt{\cdot}$ 归一化的 loss、数据配比降权、QK-norm / z-loss 各自解决的是"模态权重"还是"数值稳定"? 连续路线为什么天然没有这个熵的问题——代价又是什么(想想它为什么不能生成图)?
5. 讲者推测前沿模型是"连续编码器理解 + diffusion 生成"。理解与生成对表示的要求不同(语义对高频细节), 一个 omni model 里两套表示怎么共存? CLIP 论文里"预测 caption 反而更差"这个伏笔对此说明了什么?

CS336 第 18 讲 | 客座讲座：Dan Fu (Guest Lecture)

Stanford CS336: Language Modeling from Scratch (2026 春) · 第 18 讲, 2026 年 6 月 3 日, 全课最后一节。课程表和视频标题都只写 "Guest lecture: Dan Fu", 没有正式讲题; 按内容可概括为"从推理这一边看语言模型: 一个 token 的一生、megakernel、稳定的循环模型 Parcae" (推断) 视频: <https://www.youtube.com/watch?v=9EE4iMAF5s> (1:11:40; 英文字幕为自动生成, 专名和缩写错得多, 见文末勘误) 讲者: Dan Fu——UC San Diego 助理教授 (他说自己"在 UCSD 有一个小实验室", 即 ML Systems Lab), 同时代表 Together AI (他也在那里工作; Percy 也是 Together 的成员); Percy Liang 邀请并主持, 问答里提了两个问题 课程主页: <https://stanford-cs336.github.io/>; 本讲围绕的东西: Together AI 的 cache-aware prefill-decode disaggregation; Stanford Hazy Research 与 Together 合作的 Llama-1B megakernel (Spector et al., 2025) 和它依托的 ThunderKittens (Spector et al., 2024); UCSD 的 Parcae (Prairie, Novack, Berg-Kirkpatrick, Fu, 2026)

一句话: 前 17 讲讲怎么把模型训出来, 这一讲请 Dan Fu 讲训好之后的"另一边"——推理是把电变成 token、变成智能的引擎: 一个请求进来要经过调度、KV cache 前缀查询、算力受限的 prefill 和带宽受限的 decode, 多请求靠 continuous batching 共享 GPU, KV cache 从 HBM 一路溢到 CPU 内存和 SSD, prefill 与 decode 分到不同机器甚至不同芯片, 规模上去之后 0.001% 概率的 kernel bug 会变成"模型突然说中文"; 站在这个位置做研究, 路由层加两行"冷热分流"能让服务快最多 40%, 把一整层算子融成一个 megakernel 能把 H100 上的 decode 推到 72% 的带宽利用率, 而把 SSM 的稳定性分析搬到循环 Transformer 上 (Parcae) 能让"参数不变、多循环几次"这条新的 scaling 轴稳定地训起来——初步的 scaling law 说: 数据越多, 循环也该越多。

时间轴	
时间	内容
0:05	开场: 这门课讲怎么训, 本讲讲另一边——serving 与推理, 外加两个研究项目
1:05	动机: 新工业革命; 规模——2018 年 1 亿参数、2019 年 GPT-2、今天开源过万亿、前沿 5 到 10 万亿; 马车到汽车的十年

时间	内容
4:41	GPU 是新石油，推理是把电变成智能的引擎；全讲 takeaway：懂推理与 kernel 才能全栈创新；UCSD 与 Together 两个身份
7:46	一个 token 的一生：调度、KV cache 查询、执行与切分、采样；幻灯片全由 Nano Banana Pro 生成
9:49	生产流量的形状：输入 / 输出长度、多轮 agentic、轮间间隔、会话粘性、SLA
13:52	prefill 与 decode：一万 token 进一个出的算力受限操作，对逐 token 的带宽受限操作
16:24	continuous batching；KV cache 与前缀共享；万亿模型在 280 GB 的 GPU 上怎么切：TP、EP
19:58	prefill / decode 分到不同机器；Groq、Cerebras、SambaNova 押注 decode 专用芯片
22:00	规模化后的三个诡异 bug：NaN 与复读、tool call 死循环、突然冒中文
25:38	KV cache 的层级：GPU → CPU DRAM → SSD；问答：像操作系统换页，LRU 与预取
29:46	NVL72：万亿模型切 72 卡、塑料连接器与容错、百万 token 上下文
31:16	例子：cache-aware prefill—decode 分离——路由层两行代码，最多快 40%
34:19	研究一 megakernel：decode 把 GPU 变成"美化的内存加载器"；逐算子 kernel 的空隙从哪来
37:22	megakernel 的做法：把 GPU 当分布式系统调度；Llama 1B 一层一个 kernel；ThunderKittens；H100 72% 带宽利用率
41:32	研究二 Parcae：循环 Transformer 的动机；训练一动就炸的老毛病
46:39	用 SSM 的眼光看残差：A、B 矩阵、闭式解、谱半径；约束 A、B 后训练稳定、质量更高
54:20	循环的 scaling law：iso-parameter 曲线往右下走 → 数据多，循环也该多
59:26	总结与问答：预训练模型直接循环、推理收益、megakernel 的代价、为硬件设计模型、agentic 与批处理、多卡 megakernel

核心内容

1. 这一讲的位置：从"电"到"智能"的那一段

- **这门课与这一讲**：CS336 一路讲的是训练；Dan Fu 讲的是模型训好之后"另一边"的事——怎么 serve、怎么做推理，把电变成 token、变成智能，以及站在这个位置上能做哪些研究。
- **规模，以及变化的速度**：2018 年他读博开始时最大的模型是 1 亿参数；2019 年 GPT-2 被认为"太危险不能发布"（这门课的作业足够训到 GPT-2 水平）；今天开源模型已过 1 万亿参数，前沿模型大约 5 到 10 万亿。他的类比：1902 年曼哈顿有 13 万匹工作马，1898 年纽约

专门开会讨论马粪，结论是"没办法，捏着鼻子忍"；到 1912 年汽车数量已经超过马。语言模型的"1912 时刻"他认为是去年（2025）：他和团队里大多数人写代码已经以模型为主。

- **GPU 是新石油，推理是引擎**：GPU 投资以数千亿美元计；但油要经过引擎才变成动能——推理引擎和 GPU kernel 就是把 GPU（Percy 的说法：沙子）变成可用智能的引擎。这门课写的 FlashAttention 是训练侧，推理侧还有一整个世界的复杂度。
- **全讲的一个 takeaway**：懂推理引擎、懂底层 kernel，才做得了机器学习算法的"全栈创新"。三段内容都在证明这一句：一个 token 的一生（第 2—6 节）、megakernel（第 7 节）、Parcae（第 8—9 节）。
- **两个身份**：UCSD 的实验室（Parcae 出自这里）和 Together AI——一家 AI cloud（GPU、推理、微调），研究背景很重，Percy 也在其中。推理系统那部分幻灯片改编自他学生 Austin 在 Together 的经历，全部由 Nano Banana Pro 生成——"远看全对，凑近看字就错了"。

2. 一个 token 的一生：推理引擎里有什么

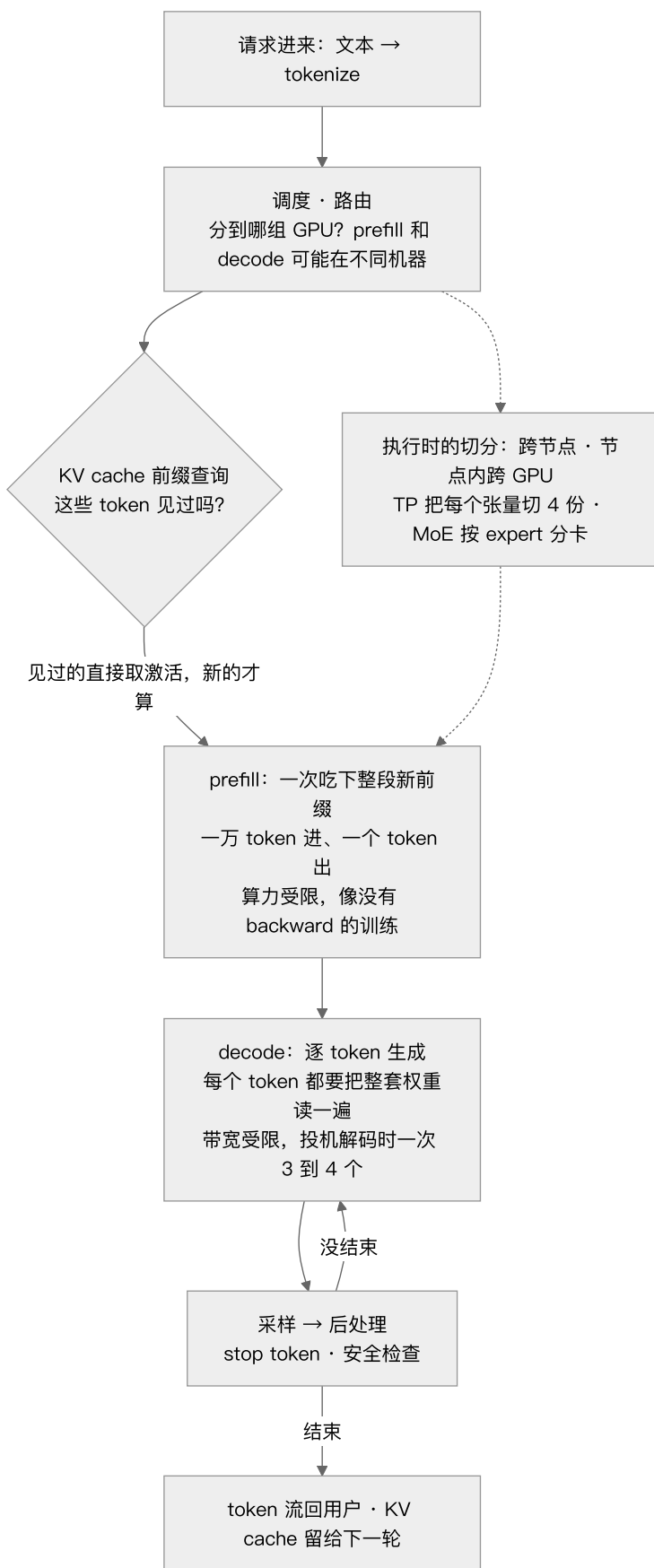


图 18-1 | 一个请求在推理引擎里走的路：调度、前缀查询、prefill、decode 循环、后处理（自绘示意）· [▶ 看原幻灯片 8:17](#)

- **引擎的循环** (图 18-1): tokenize 之后先调度 (prefill 和 decode 可能已经拆在不同机器), 再查 KV cache 看哪些 token 见过, 然后才是可跨节点、跨 GPU 切分的模型计算, 最后采样、后处理 (stop token、安全检查) 返回用户; 引擎就在这个循环里等下一个请求。
- **生产流量长什么样**: 既不像训练数据, 也不像拍脑袋编的负载。要看四个维度:
 - 输入 / 输出长度: coding 负载 (Cursor 那种把整个代码库交给 agent 的) 输入是数万 token, 输出看模型会不会先吐 thinking token; 把整本书贴进去讨论的摘要负载、普通 chat 又各不相同。
 - 多轮与 agentic: 你和 coding agent 来回改; agent 自己调工具 (搜代码库、上网搜), 把结果回灌再来一轮。
 - 轮间间隔: 交互式聊天和手机语音要快; 放手让 agent 自己跑是另一种节奏; agent 卡住喊"帮忙"而你没注意到, 又是一个空档。
 - 会话粘性: 反复来回的用户, 还是问一句就走、第二天再来的用户。
- **目标, 也就是 SLA**: 交互式应用可能要求首 token 在 1 秒内返回 (agent 先说"我在想"); 或者已知要生成 500 个 token, 整条回复要在某个时限内到齐。后面所有优化都在这些约束下做。第 10 讲算过吞吐与延迟的账, 这里补的是生产流量真实的形状。

3. prefill 与 decode: 一个像训练, 一个是"美化的内存加载器"

- **prefill**: 比如 10,000 个没见过的 token 进来, 要算出它们的激活和 logits——10,000 进、1 出。算力受限, 和训练几乎一样: 写好 FlashAttention kernel、跑一遍, 只是没有 backward。
- **decode**: prompt 处理完后逐个 token 生成, 有投机解码的话一次可能出 3 到 4 个。每生成一个 token 都要送回模型再跑一遍——FLOPs 其实很少, 但每一步都得把整套权重从显存读一遍, 所以是带宽受限。GPU 这台大规模并行机器在这一步退化成"美化的内存加载器"。
- **时间**: prefill 单次远长于一步 decode, 但 prefill 每个 prompt 只跑一次, decode 每个 token 跑一次。

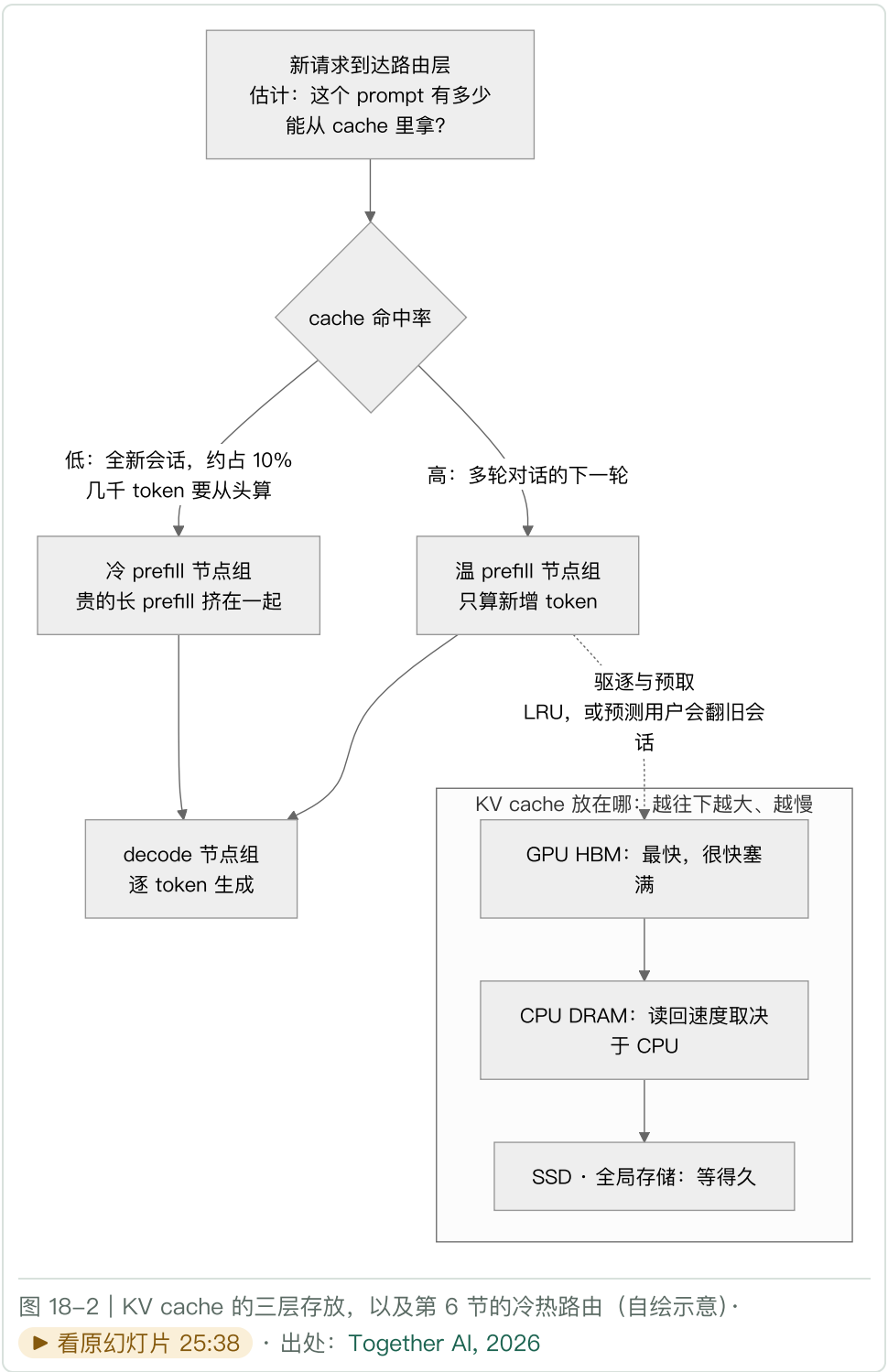
$$\text{FLOPs}_{\text{prefill}} \approx 2 N L_{\text{prompt}}, \quad t_{\text{decode step}} \gtrsim \frac{\text{bytes}(\text{weights}) + \text{bytes}(\text{KV cache})}{\text{BW}_{\text{HBM}}}$$

N 是参数量, L_{prompt} 是 prompt 长度, 前一式是 prefill 的计算量 (每个参数每个 token 约 2 次运算, 和训练的 forward 一样); 后一式说 decode 一步至少要花"把权重和 KV cache 从 HBM 读一遍"的时间,

BW_HBM 是显存带宽。讲者课上只做了定性判断，这两个式子是第 10 讲的记账方式。

- 对照第 5 讲的 roofline：prefill 落在算力封顶的那段，decode 落在带宽斜线上；同一个模型、同一张卡，两个阶段的瓶颈完全不同——这是后面所有"分离"的根据。

4. 多请求同时在跑：continuous batching、KV cache 的层级、模型怎么切



- **continuous batching**: 他的图里时间向下流。一个长请求在生成；新请求进来，占算力，也占 KV cache 的显存；短请求结束、又进来一个；第 4 步再来一个很长的（橙色），显存不够，只能排队等长请求结束。多请求共存的复杂度从这里开始。
- **KV cache 与前缀共享**: 很多用户开头都是"hi ChatGPT"，不必为每个人重算；用户贴进一本书，prefill 一次，下一轮只算新增的那段。实现上用一棵传统的树形结构（应为 radix tree）查哪些 token 见过。CME295 第 3 讲讲过 KV cache 的原理，这里它成了整个服务的核心资源。
- **模型怎么切**: 万亿参数的模型放不进一张约 280 GB 的 GPU: 可以把每个张量切 4 份放 4 张卡 (tensor parallelism)，也可以把 MoE 的各个 expert 分到不同卡 (expert parallelism，前沿模型多是 MoE)。第 7–8 讲讲的是训练里怎么切；推理里目标换成了延迟和会话数。

小注：讲者说的"280 GB 一张"应指 Blackwell Ultra (B300) 级别的 288 GB HBM；B200 约 180 GB，H100 是 80 GB。

- **KV cache 的层级** (图 18–2 下半): 生产系统希望 KV cache 越大越好，跨用户、跨会话都能命中。先放 GPU，很快满；再放 CPU DRAM——Jensen 最近的 keynote 突然痴迷 CPU 性能，原因之一是上一代 CPU 太慢，"一台 5 亿美元的机器被顺手配上去的一颗上千美元的 CPU 卡住"，读 KV cache 回来就是这样的负载；再往下放 SSD——OpenAI 买光 SSD 和 DRAM 的传闻，部分原因就是它；最后是全局存储。
- **问答：卸载的是哪一类负载？** 这一页去掉右边的 GPU，就是 70、80 年代操作系统课本里的图：程序开太多、内存不够就换页到磁盘。LRU (least recently used) 是不错的启发式，"有 OS 论文说它在最优的 2 倍以内"；更好的是预测未来——用户翻出一个月前的旧会话就是要提问的强信号，可以提前取到 GPU。归根到底是在 SLA 允许的范围内往 GPU 上塞尽量多的流量。

小注：经典结果是 Sleator 与 Tarjan (1985) 的竞争分析：同样大小的缓存下 LRU 的竞争比等于缓存大小 k ；"2 倍以内"对应 LRU 用 k 的缓存对比最优算法用 $k/2$ 的缓存。讲者说的是量级。

- **新硬件带来的新问题**: Blackwell 一代有 NVL72——72 块 GPU 高速互联成一体，于是问万亿模型切到 72 卡上有没有意义；容错——连接器是塑料的，插太用力线就弯，NVLink 时好时坏，模型切到 64

卡、服务百万用户、每天万亿 token，一张卡挂了怎么办；上下文到了百万 token 以上，要不要把一段上下文切到多张卡上。

小注：GB200 NVL72 是一个机柜里 36 颗 Grace CPU 加 72 颗 Blackwell GPU，处在同一个 NVLink 域内。

5. prefill 与 decode 分家，以及规模上去之后的三个诡异 bug

- **分到不同机器**：prefill 像训练、FLOPs 重、能把 GPU 用满；decode 带宽重、算得少却每步都要读全部权重。把 prefill 放一组 worker、decode 放另一组，各自专门优化，已经是通行做法。
- **分到不同芯片**：NVIDIA "买下" Groq——GPU 之王去买一家推理芯片公司，原因之一就是 decode 和 prefill 差得太远：下一代硬件里 NVIDIA 打算 GPU 做 prefill、Groq 的 LPU 做 decode。OpenAI 与 Cerebras 的算力合作是同一逻辑，SambaNova 等也在这个空间里下注。

小注：据公开报道，2025 年 12 月 NVIDIA 与 Groq 达成的是约 200 亿美元的非独占技术授权并吸纳其核心团队，官方口径不是收购；讲者用的是口语说法。

- **规模化之后的 bug**：每天服务万亿 token 以上时，小规模下没事的东西一定出事——发生概率 0.001% 甚至更低的事件天天见。他举了去年底几个开源推理引擎上的三个例子：

症状	真因	值得记住的
模型开始复读同一个 token ("hi hi hi"或一串感叹号)	某个 kernel 有个极罕见条件才触发的小错，logits 算到一半变成 NaN	NaN 不一定报错，它先表现为复读
completion 长度暴涨，几万 token 的死循环	有人改了 tool call 的处理，模型说"去搜索"后没人真的去搜、也没把控制权交回，模型就一遍遍重复"去搜索"	看长度分布能比看内容更早发现问题
英文提问，模型突然改说中文；多家推理服务商同时中招	被归咎于量化，实际是某 kernel 的 off-by-one：读到一段未初始化的显存，过了 attention 冒出一个随机汉字，模型以为用户在说中文，就一路说下去	"突然说中文"有时是训练所致，有时只是别人代码里差了一

6. 例子：cache-aware 的 prefill-decode 分离，两行代码快 40%

- **观察**：绝大多数请求是已有会话的下一轮。假设平均一个会话 10 轮，那约 10% 的请求是全新的——几千 token、cache 命中率极低、算起来贵。你不想让"有人贴了一本书让你聊"和"chat，为什么 1 加 1 等于 2"这种会话中途的短问答挤在同一批 GPU 上。

- **做法：**路由层加两行代码——命中率低的新请求送到一组 prefill 节点，让它们彼此作伴；其余温请求送到另一组。这么简单的改动，服务最多快 40%。
- **他对现状的判断：**这类东西还很早期，十年二十年后回看会觉得"这不是显然的吗"——今天是第一次在生产规模上看到这些流量。

```
def route(request, cold_pool, warm_pool):  
    # 估计这个 prompt 有多少前缀已经在 KV cache 里  
    hit = kv_cache.estimated_hit_rate(request.tokens)  
    if hit < COLD_THRESHOLD:           # 全新会话：长而贵的 prefill  
        return cold_pool.pick()        # 冷请求彼此作伴，别拖慢温请求  
    return warm_pool.pick()           # 多轮对话的下一轮：只算新增
```

冷热分流的全部逻辑就是这一个判断：按估计的 cache 命中率，把请求送进不同的 prefill 池。

小注：Together 的博客把节点分成三类——处理冷请求的 pre-prefill 节点、处理温请求的 prefill 节点、隔离的 decode 节点；KV cache 三级（GPU、主机 DRAM、通过 RDMA 连接的集群级分布式缓存），冷请求算完把 KV 状态写入分布式缓存，后续请求整块取回。报告的数字是可持续吞吐最多提高 40%，10 万 token 以上的长 prompt 首 token 延迟显著下降。

7. 研究一：megakernel——把 GPU 当成一个分布式系统

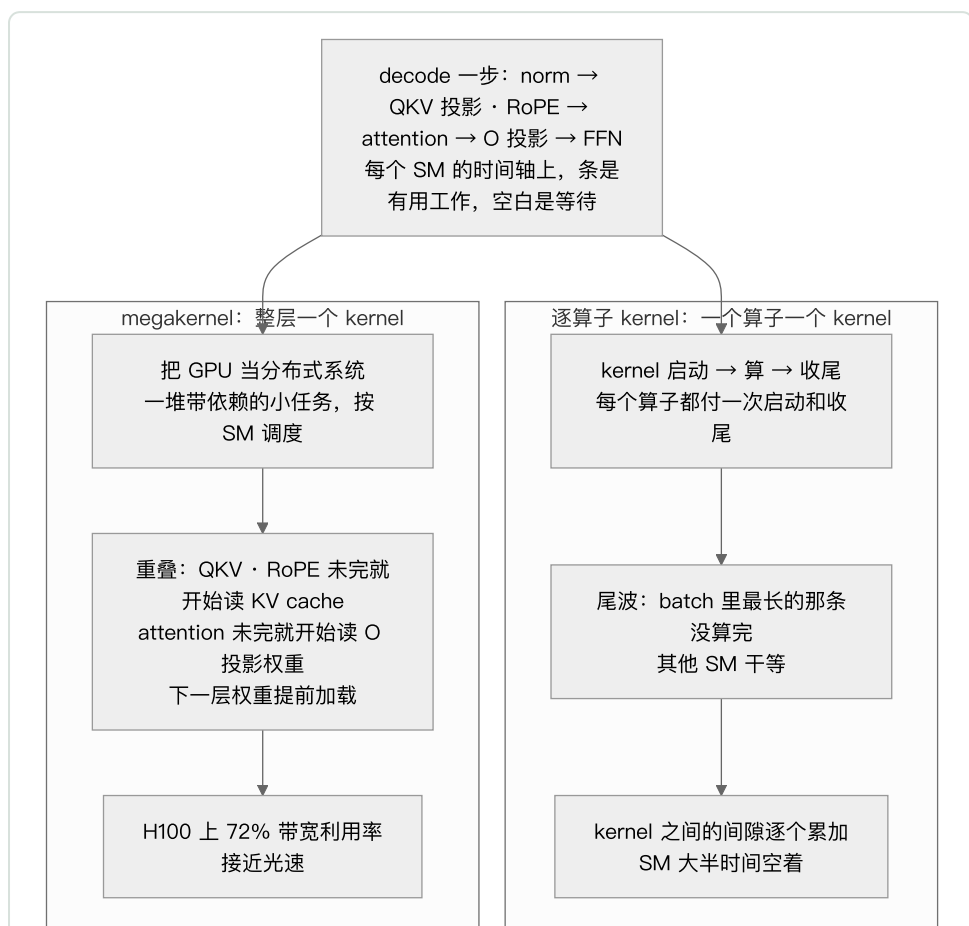


图 18-3 | 逐算子 kernel 的空隙从哪来, megakernel 把它们填掉的办法 (自绘示意)

· [▶ 看原幻灯片 35:50](#) · 出处: Spector et al., 2025

- **问题:** decode 要把整个模型跑一遍才出一个 token, GPU 退化成内存加载器。雪上加霜的是写 kernel 的方式——一个算子一个 kernel: norm 一个、matmul 一个、attention 一个。好写 (写 FlashAttention 已经够痛苦了), 但引入大量空转。
- **看时间轴:** x 轴是时间, y 轴是 GPU 上的各个 SM (streaming multiprocessor, H100 有 132 个, B200 大约 148 个), 有条的地方是有用工作, 空白是等待。空白有三个来源: kernel 的启动和收尾; 尾波效应——batch 里一条短一条长, 都得等最长的; 多个 kernel 之间的间隙累加。第 6 讲算过 B200 上 148 个 SM 对 160 个 block 的尾波, 第 5 讲讲过算子融合——megakernel 是这两件事的极端版本。
- **做法:** 一个 kernel 覆盖多个算子——像 FlashAttention 的融合, 但激进得多。思维方式随之改变: GPU 不再是“一次执行一个操作的设备”, 而是一个巨大的分布式系统——一堆带依赖关系的工作, 问题变成怎么调度、怎么分发到各个 SM, 让利用率最高。

- **两个数字**：只对 attention 的推理 kernel 做，能有 30% 到 70% 的加速；对整个模型做——他展示的是 Llama 1B 的一层融成一个 kernel——H100 上达到 72% 的带宽利用率，接近这块 GPU 的"光速"（忽略一切复杂性，硬件物理上最快能多快）。柱状图里青色的 megakernel 远快于其他 SOTA 引擎，"现在的数字还要好得多"。
- **重叠的例子**：蓝色的 QKV 投影加 RoPE 还在跑，橙色带圈的 KV cache 加载已经开始，等 QKV 算完、拿到新的 query 再算剩下的 attention；attention 没结束，红色的 O 投影已经在加载权重；下一层的权重加载也提前。
- **实现**：一个相当复杂的 CUDA 框架——基于指令的抽象，每个子 kernel 单独一个文件，用虚拟化的 shared memory 系统编排；底层是他们的 kernel 库 ThunderKittens，"像 Triton，但更底层、控制更细"（第 6 讲问答里列过它）。

$$t_{\text{speed of light}} = \frac{\text{bytes moved}}{\text{BW}_{\text{peak}}}, \quad \text{bandwidth utilization} = \frac{t_{\text{speed of light}}}{t_{\text{measured}}}$$

bytes moved 是一步 decode 必须搬的字节数（主要是权重），BW_peak 是显存的峰值带宽，两者相除就是"光速"——物理上最快的一步；实测时间与它的比值就是带宽利用率，讲者的 72% 是这个口径。

```
def sm_interpreter(instructions, counters):    # 每个 SM 跑一份
    for ins in instructions:                  # 事先排好的指令序列
        for dep in ins.depends_on:           # 等上游任务把计数器加到
位
            wait_until(counters[dep] >= ins.needed)
            pages = smem.alloc(ins.pages)     # 虚拟化的 shared
memory 页
            ins.run(pages)                    # 子 kernel: 读权重 /
attention / norm
            smem.free(pages)
            counters[ins.id] += 1             # 通知下游可以开始
```

megakernel 的骨架：每个 SM 跑一个解释器，按指令序列执行子 kernel，依赖靠全局计数器同步，shared memory 按页分配。

小注：Hazy Research 的博客（2025 年 5 月，Spector、Juravsky、Sul、Dugan、Lim、Fu、Arora、Ré）给的数字：Llama-1B（实际 1.24B）bf16 单序列生成，H100 上达到 78% 的显存带宽，比 vLLM 快近 2.5 倍、比 SGLang 快 1.5 倍以上；B200 上比 vLLM 快 3.5 倍以上。shared memory

切成 13 页、每页 16 KiB，同步用全局内存里的计数器。讲者课上说的是 72%，应是不同时点或设置下的数字。

- **代价** (问答)：血、汗、泪。一个能干的 kernel 工程师一年，大概能为一种硬件、两三个模型、batch size 1 到 16 写出 megakernel；batch 到 17，从头再来。Together 在做编译器把这个过程自动化。做得出来就是最快的，不可能更快，但太费力。
- **多卡** (问答)：NCCL 的通信调用也能融进 megakernel，他们有很早期的结果，但还没找到杀手级用例——有时就是被 NCCL 本身的延迟卡住。DeepSeek 4 发布时随附了一个 MoE 推理层的 megakernel，融合了部分通信。趋势是模型里某一部分有一个小 megakernel，而不是整个模型一个。

8. 研究二：Parcae——循环 Transformer 为什么会炸，怎么让它不炸

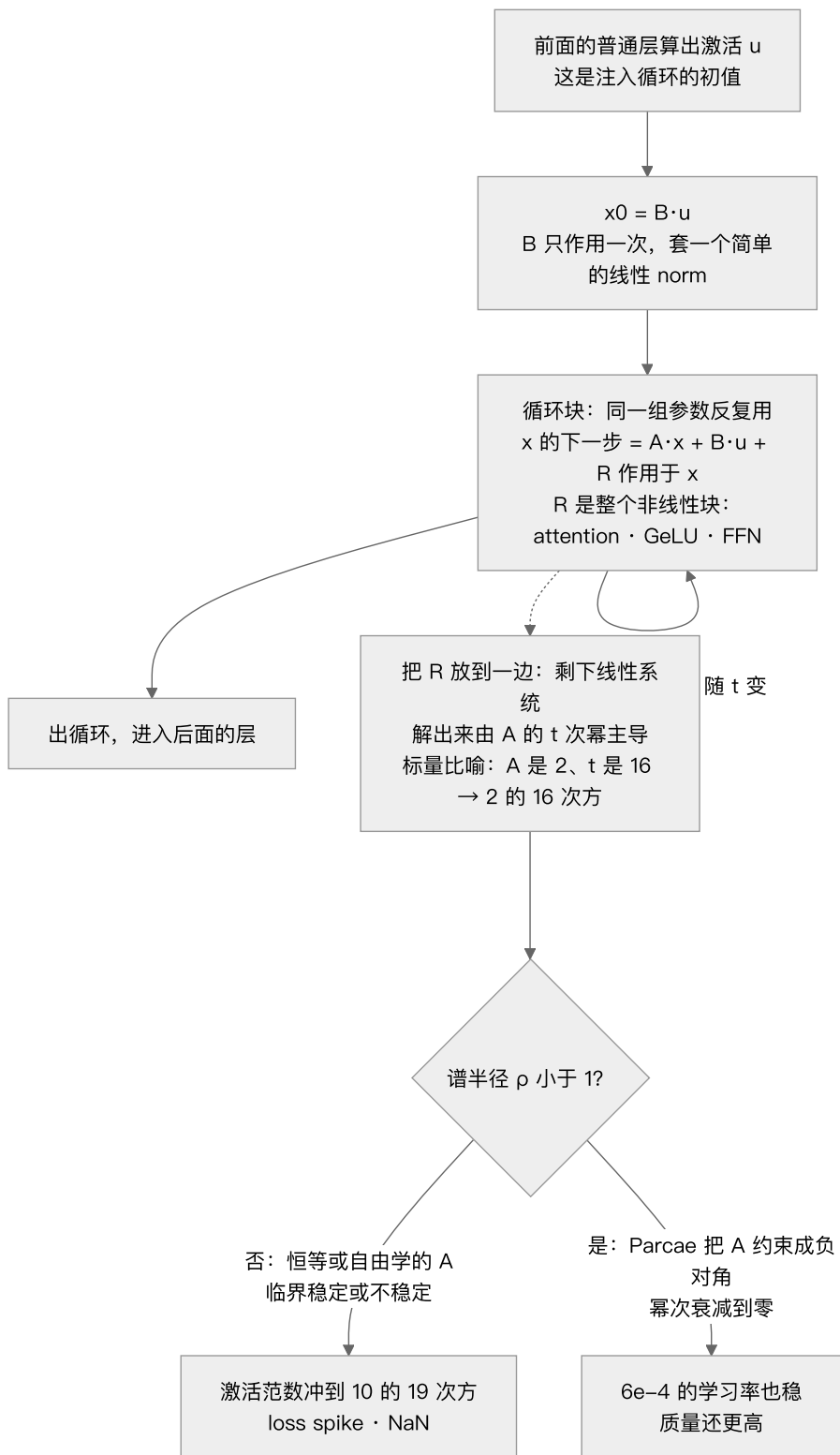


图 18-4 | 循环块的数据流，以及把它看成线性动力系统之后的稳定性判据（自绘示意）· [▶ 看原幻灯片 43:33](#) · 出处：Prairie et al., 2026

- **问题的来源：**开场说能力来自把参数和数据一起 scale；Parcae 问的是有没有另一条轴。它是对 looped transformer（循环 Transformer）这个老想法的认真对待：把模型里的一段 block 拿出来，让 token 在

里面反复过几遍（图 18-4 上半）。工作出自他的 UCSD 实验室，由 Hayden 主导，与 Zachary 和 Taylor 合作。

- 好处一：参数量不变，多了一个调 FLOPs 的旋钮——不加参数就提高质量。
- 好处二：几年前的工作表明循环模型表达能力更强，有些函数同样多的参数不循环就表达不了。
- 真正关心的问题：每个参数、每份数据能换来多少智能。
- 已有的苗头：Maryland 的 Tom Goldstein 组的 recurrent-depth 模型在 ARC 一类任务上占优；发布前一周还有一场 Twitter 闹剧——一位 OpenAI 的人声称 Claude Mythos 是循环模型，后来写博客承认是编的（他们的工作早于这场热度）。

小注："表达能力更强"应出自 Giannou et al., 2023 ([Looped Transformers as Programmable Computers](#)) 或 Saunshi et al., 2025 ([Reasoning with Latent Thoughts](#)) 一类工作（推断）；recurrent-depth 模型即 Geiping et al., 2025 ([arXiv 2502.05171](#))，也是后面对比表里的基线。

- **老毛病：一动就炸**：已有的循环模型，学习率稍微动一动就炸：做一次 learning rate sweep，十次里九次不收敛，NaN、大 loss spike。前人的对策是 hack：每层都加 norm，或者"只用 $2e-4$ ，别的别碰"。他的判断：大 loss spike 说明训练过程里有更深的东西错了，值得深挖而不是绕过去。第 3 讲的 QK-norm、z-loss 针对的是普通 Transformer；这里的病根在循环本身。
- **SSM 的眼光**：直接分析这个循环块不可能——海量参数、softmax、RoPE、各种非线性。他们先看残差：激活从一次循环到下一次变了多少，经验上变得不多。于是把整个非线性部分（attention、GeLU、FFN）装进一个盒子叫 R，放到一边，剩下两个矩阵：B 作用在进入循环前的初始向量上，A 决定每一轮怎么变换残差。以前的循环 Transformer 里，A 要么是恒等（直接相加），要么是完全可学的矩阵。第 4 讲把 SSM 当 attention 的替代品来讲；这里 SSM 理论被拿来当分析工具。

$$x_{t+1} = A x_t + B u + R(x_t)$$

x_t 是第 t 次循环后的残差流， u 是进入循环前的激活（每一轮都重新注入）， A 是每轮对残差的线性变换， B 是对注入的线性变换， R 是被放到一边的整个非线性块。

$$x_t = A^t x_0 + \sum_{k=0}^{t-1} A^k B u \quad (R \equiv 0)$$

丢掉 R 之后高中微积分就能解：第 t 步的激活由 A 的 t 次幂主导。经验上 A、B 这两项确实主导了方程的量级，所以近似有依据。

- **谱半径**： $\rho(A)$ 是 A 的特征值绝对值的最大者，可以当"范数"来理解。把矩阵不断自乘，如果它学成了"2"、循环 16 次，激活就被放大 2 的 16 次方——loss spike 就是这么来的。以前论文里 A、B 的选择要么临界稳定 (marginally stable)，要么不稳定。
- **Parcae 的修法**：约束 A、B，让数学上算出来不会炸——A 有效地做成负对角矩阵，幂次上去项就衰减到零；B 只作用一次、不会炸，套一个简单的线性 norm 即可。谱半径小于 1，是稳定系统。

$$\rho(\bar{A}) = \max_i |\lambda_i(\bar{A})| < 1, \quad A = \text{Diag}(-e^{a_1}, \dots, -e^{a_d}), \quad \bar{A} = e^{\Delta t A}$$

λ_i 是特征值， ρ 是谱半径；A 取负对角 (a_i 可学)，按 SSM 的习惯用步长 Δt 离散化后每个对角元都落在 0 到 1 之间，幂次自然衰减。离散化的细节来自论文，课上只说"负对角"。

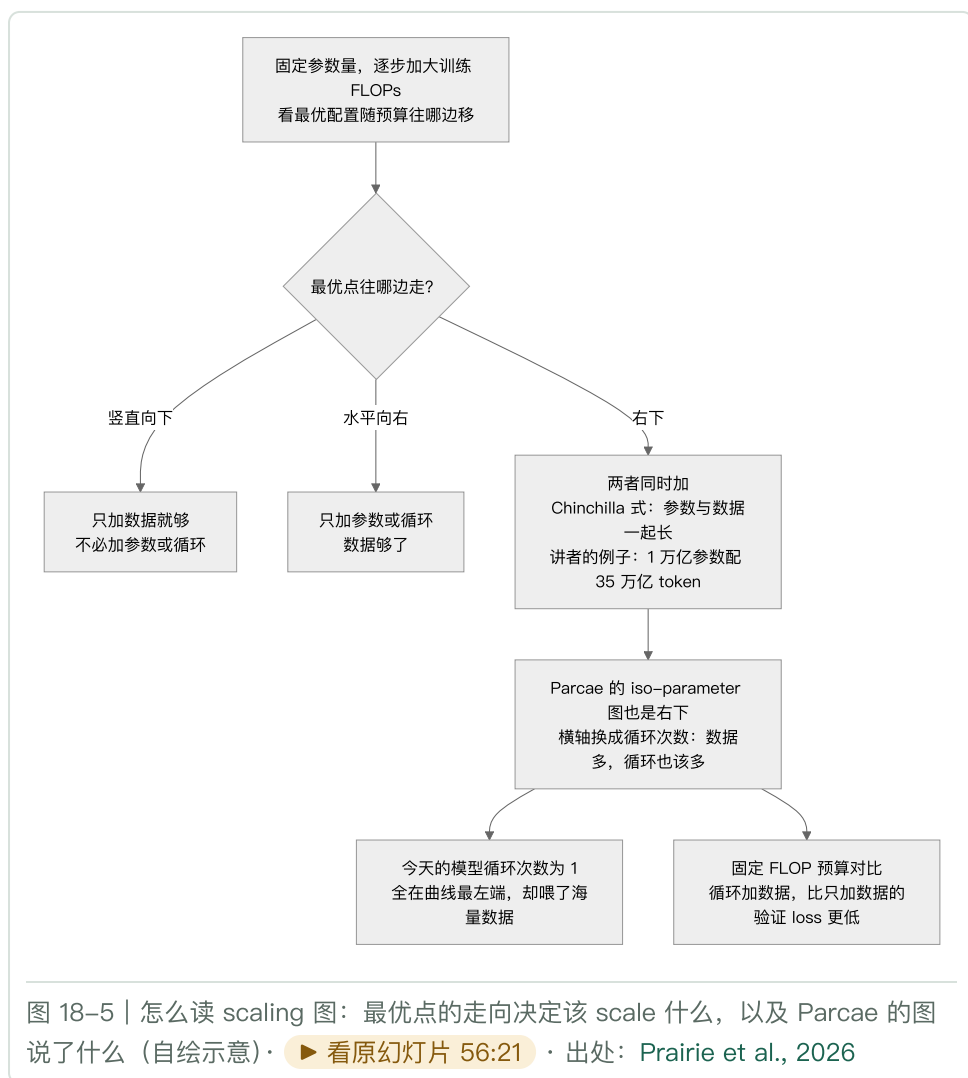
```
def parcae_block(u, n_loops, params):
    a_bar = exp(dt * -exp(params.log_a)) # 负对角 A 离散化：对角元落在
    0 到 1 之间
    x = linear_norm(params.B @ u)         # B 只用一次，加个简单的线性
    norm
    for _ in range(n_loops):              # 同一组参数反复用，FLOPs 随循
    环次数增加
        x = a_bar * x + (dt * params.B) @ u + R(x, params) # R 是
    attention 加 FFN 的非线性块
    return x
```

循环块的 forward 骨架：稳定性只靠 a_bar 的取值范围和 B 上的 norm 保证，R 原封不动。

- **结果**：6e-4 这个让其他模型必炸的学习率下，loss 曲线也平稳。看激活的 state norm：橙色的无约束基线冲到 10 的 19 次方；蓝色是加了 norm 的模型——模型想把激活撑大，norm 又把它压回 1，两股力较劲，范数看着很好，loss 照样抽风。Parcae 让两者都消失。不只更稳，质量也更高：对比表里 Parcae 超过之前的循环模型 (recurrent-depth 模型)，也超过强 Transformer 基线——基线是 nanochat 那种调到"学得最快"的架构，循环起来再稳定住，困惑度和端到端质量都更好。

小注：论文（2026 年 4 月，arXiv 2604.12946）的数字：770M 的 Parcae 在同样训练数据上达到 1.3B Transformer 的质量；相对之前的循环模型验证困惑度最多低 6.3%；三个规模的验证 PPL 对比是 140M 19.06 对 21.48、370M 14.49 对 15.79、770M 12.49 对 13.08；学习率从 $2e-4$ 到 $1e-3$ 都稳定，无约束模型则发散；收敛的 run 谱半径小于 1，发散的 run 学到了大于等于 1 的谱半径。

9. 循环的 scaling law：数据多，循环也该多



- **先复习怎么读这种图：**几年前大家问“该把模型做大还是多喂数据”，得到一堆幂律曲线，看的只有一件事——最优点往哪边走：竖直向下就只加数据，水平向右就只加参数，往右下走就两个一起加。结论是右下，于是有“1 万亿参数配 35 万亿 token”这样的训练（第 9、11 讲拟合过这些曲线）。
- **循环放进去会怎样：**三种可能——永远别循环、疯狂循环、循环一点点。他们的初步 scaling law 是一组 iso-parameter 曲线：参数量固

定，换颜色表示靠加数据加大训练 FLOPs，横轴是循环次数。两种模型上都是右下走向：加数据时最优循环次数也在增加，而且遵循相当经典的幂律，可以联合预测"循环次数与 token 数一起增加时的质量"。

- **同一件事换个画法：**固定模型大小和 FLOP 预算，橙色是固定深度的传统 Transformer（只靠加数据用掉预算），蓝色是循环模型。同样的 FLOPs，靠循环加数据达到时验证 loss 更低。
- **意味着什么：**据他所知今天所有模型都没有循环——全在曲线最左端，却喂了海量数据。也许大规模预训练都该循环起来。
- **问答：参数呢？**他们画过循环、数据、参数三个轴的三维图，走向也是三者一起加，只是图太难看没放。至于"compute-optimal 时会不会选循环而不是加参数"：compute-optimal 本身有点做作——总是"给定 FLOP 预算"；真想要更好的模型就加预算，模型大小受限（要装进某块芯片、要在笔记本上跑）就多循环，数据用光了就把这个大小的模型训得尽量足。

10. 问答：为硬件设计模型、负载不同架构不同

- **预训练好的模型能不能直接循环？**几个月前有篇"钓鱼"博客：作者一个参数没训，把一个 Qwen 模型的两三层循环了一下，就在某个榜上赢了——某些数学任务上质量真的变高。这件事让他不安："不知道为什么循环一下就变好"，相关工作快出来了。
- **循环对推理意味着什么？**推理效率的大瓶颈其实是显存：参数少了，就能放更多 KV cache，或者少切几张卡、少通信。他还有个梦：把循环块做得足够小，就能给它写一个小 megakernel 在里面飞快地转，目前还没做到那么小。Groq LPU 一类芯片片上内存只有大约 250 MB；如果模型能装进去，权重常驻、激活尽情往里冲——跨过某些阈值会有非线性的收益。
- **为特定硬件设计模型（Percy 的问题）：**首先受限的一定是内存——要在 Cerebras 的 wafer 上跑，就先看它有多少内存，把模型尺寸定到"放下之后还有足够的 KV cache 余量"。最近的中国模型有些选择暗示在考虑华为的芯片。量化格式也是：要在 NVIDIA 上跑模型（比如 NVIDIA 自己的 Nemotron）用 NVFP4 训练，那是 NVIDIA 芯片专有的 FP4 格式；要在 AMD 上跑就用 MXFP4。

小注：NVFP4 与 MXFP4 都是 4-bit 浮点微缩放格式，差别在缩放块与缩放因子：NVFP4 每 16 个数配一个 FP8 (E4M3) 缩放因子，MXFP4 按

OCP Microscaling 规范每 32 个数配一个 2 的幂次 (E8M0) 缩放因子。
CME295 第 4 讲讲过 FP16 / BF16，这是往下再走两级。

- **agentic 与批处理，最优架构差在哪？** agentic workflow 最在意 KV cache 保持"热"；批处理（每份文档只看一次）则不太在乎。所以对 agentic 负载敏感的人会看 DeepSeek 的 MLA——对 KV cache 的激进压缩——或者能把 KV cache 存成 FP8、FP4 的模型（第 4 讲讲过 MLA）。最大的分野是因果还是双向 attention：Google 的搜索长期用 BERT，不需要生成 token，双向 attention 跑一次拿到向量存进数据库就完了；chat 负载总有 decode；T5 那种先双向编码再生成是中间路线。模型开发者最终只能选一个架构，尽量在多种用例上都不离谱。

11. 放在一起看：三个研究项目各动了栈的哪一层

项目	动的是哪一层	关键数字	代价或前提
cache-aware prefill-decode 分离	路由层：按 cache 命中率分冷热	会话平均 10 轮 → 10% 新请求；服务最多快 40%	两行代码；前提是多轮对话为主的流量
megakernel	kernel 层：整层一个 kernel，GPU 当分布式系统	attention kernel 快 30% 到 70%；Llama 1B 在 H100 上 72% 带宽利用率	一人一年：一种硬件、两三个模型、batch 1 到 16；换 batch 重来
Parcae	架构层：循环块加 SSM 式稳定化	6e-4 学习率不炸；无约束基线激活 10 的 19 次方；固定 FLOPs 下循环加数据 loss 更低	需要重参数化 A、B；scaling law 仍是初步的

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
规模增长图	2018 年 1 亿参数到今天万亿以上；讲者自己说图已过时	1:36	—
马与汽车	1902 年 13 万匹马、1898 年马粪会议、1912 年汽车反超	3:08	—
推理引擎全景	调度 → KV cache → 执行 → 采样的环；细看会发现 AI 生成的错字	8:17	—
流量形状表	输入 / 输出长度、轮数、间隔、SLA 四个维度；讲者说有些 5 被画成了 S	9:49	—

图	看什么	跳转	出处
continuous batching 时序图	时间向下流；第 4 步的橙色长请求因显存不够排队	16:24	—
KV cache 层级	GPU → CPU → 磁盘；左侧 AI 自动补上的 LRU 字样	25:38	—
冷热路由	低命中率的新请求单独送一组 prefill 节点；最多快 40%	31:48	Together AI, 2026
SM 时间轴（逐算子 kernel）	空白就是等待：启动收尾、尾波、kernel 之间的间隙	35:50	Spector et al., 2025
Llama 1B 单层 megakernel 时间轴	蓝色 QKV 加 RoPE 未完，橙色带圈的 KV cache 加载已开始；红色 O 投影权重提前加载	38:57	同上
decode 速度柱状图	青色的 megakernel 柱；H100 上 72% 带宽利用率	41:01	同上
循环块示意	紫色块反复处理同一个激活；参数不变、FLOPs 可调	43:33	Prairie et al., 2026
稳定性曲线	6e-4 学习率下 Parcae 的 loss 平稳；无约束基线的激活范数冲到 10 的 19 次方，加 norm 的基线范数好看但 loss 抽风	51:46	同上
Parcae 对比表	对 recurrent-depth 模型与 nanochat 式 Transformer 基线的困惑度和下游任务	53:19	同上
iso-parameter scaling 曲线	同参数量、加数据换颜色；最优循环次数往右下走	56:21	同上

提到的工作

名称	在本讲里的作用
Cursor · Claude Code · GPT-5.5	代码生成的例子；coding 负载"数万 token 输入"的来源
Nano Banana Pro	全部幻灯片由它生成；"远看全对，细看全错"
Together AI	讲者代表的 AI cloud；冷热路由、megakernel 编译器都在这里做
Cache-aware prefill-decode disaggregation (Together AI, 2026)	路由层按 cache 命中率分冷热，最多快 40%
FlashAttention (Dao et al., 2022)	这门课写过的训练侧 kernel；prefill 像训练；megakernel 是它的融合思想的激进版
speculative decoding	decode 一步出 3 到 4 个 token 的原因，只点了名（第 10 讲）

名称	在本讲里的作用
continuous batching · prefix caching (radix tree)	多请求共享 GPU；查哪些 token 见过（树形结构应为 radix tree）
tensor / expert parallelism	万亿模型切到多卡的两种方式（第 7—8 讲）
Groq LPU · Cerebras · SambaNova	decode 专用芯片的押注；Groq 片上约 250 MB 内存
NVIDIA GB200 NVL72	72 块 GPU 高速互联；容错与切分的新问题
Look Ma, No Bubbles (Spector et al., 2025)	Llama-1B 的 megakernel；Stanford 与 Together 合作
ThunderKittens (Spector et al., 2024)	写 megakernel 用的库；像 Triton 但更底层（第 6 讲问答提过）
NCCL	多卡通信库；其调用也能融进 megakernel
DeepSeek 4 的 MoE megakernel	局部 megakernel 的例子，融合了部分通信
Parcae (Prairie, Novack, Berg–Kirkpatrick, Fu, 2026)	稳定的循环 Transformer 与循环的 scaling law
Recurrent–depth 模型 (Geiping et al., 2025, Goldstein 组)	循环模型的早期证据；也是 Parcae 对比表里的基线
Giannou et al., 2023 · Saunshi et al., 2025	"循环模型表达能力更强"的出处（推断）
nanochat (Karpathy)	Parcae 对比用的 Transformer 基线架构
Claude Mythos 循环模型传闻	发布前一周的 Twitter 闹剧，后被本人否认
Qwen 循环两三层的博客	不训练就在榜上赢了；预训练模型直接循环的谜
Chinchilla (Hoffmann et al., 2022)	"参数与数据一起 scale"的幂律，讲者以此引出循环的 scaling law（第 9、11 讲）
NVIDIA Nemotron (NVFP4) · MXFP4 · 华为芯片	为目标硬件选量化格式与模型尺寸的例子
DeepSeek MLA · FP8 / FP4 KV cache	agentic 负载下压缩 KV cache 的架构选择（第 4 讲）
BERT (Google 搜索) · T5	双向 attention 与"先编码再生成"的负载专用架构

术语对照

English	中文
inference / serving	推理 / 在线服务：把训好的模型跑给用户
inference engine	推理引擎：调度、缓存、执行、采样的整套系统
lifetime of a token	一个 token 的一生：请求进入到 token 返回的全过程

English	中文
workload	负载：请求的长度、轮数、间隔的分布
SLA (service level agreement)	服务等级目标：首 token 延迟、整条回复时限等
time to first token (TTFT)	首 token 延迟
prefill / decode	预填充（整段 prompt 一次算完） / 逐 token 生成
compute-bound / memory-bandwidth-bound	算力受限 / 带宽受限
speculative decoding	投机解码：小模型先猜、大模型一次验证多个 token
continuous batching	连续批处理：请求随到随加入正在跑的 batch
KV cache	键值缓存：已算过 token 的 K、V，供后续 token 复用
prefix sharing / prefix caching	前缀共享：相同前缀的激活只算一次
radix tree	基数树：按前缀组织 token 序列的树
cache hit rate	缓存命中率：prompt 中能从 KV cache 直接取的比例
tensor parallelism (TP)	张量并行：一个张量切到多张卡
expert parallelism (EP)	专家并行：MoE 的各 expert 放不同卡
prefill-decode disaggregation	prefill 与 decode 分离到不同机器
offloading / eviction / prefetch	卸载 / 驱逐 / 预取：KV cache 在层级之间搬动
LRU (least recently used)	最近最少使用：经典的缓存驱逐策略
NVLink / NVL72	NVIDIA 的 GPU 互联 / 72 卡互联的机柜
fault tolerance	容错：一张卡挂了服务不停
megakernel	巨核：一个 kernel 覆盖整层甚至整个模型的算子
kernel launch / teardown	kernel 启动 / 收尾：每个 kernel 都要付的固定开销
tail effect	尾波效应：batch 里最长的一条拖住所有 SM
SM (streaming multiprocessor)	流式多处理器：GPU 里的计算单元，H100 有 132 个
bandwidth utilization	带宽利用率：实测搬运速度占峰值带宽的比例
speed of light	光速：硬件物理上最快的时间，作为上限
instruction-based abstraction	基于指令的抽象：每个 SM 按指令序列执行子 kernel

English	中文
looped transformer	循环 Transformer：一段 block 反复过多次
recurrence	循环次数 / 递归
residual stream	残差流：逐层累加的主干激活
dynamical system	动力系统：把逐轮更新写成状态方程
state space model (SSM)	状态空间模型：这里是分析工具而非架构
spectral radius	谱半径：特征值绝对值的最大者
marginally stable / unstable	临界稳定 / 不稳定
loss spike	loss 尖峰：训练突然发散的征兆
reparameterization	重参数化：换一种参数化方式让约束自动满足
iso-parameter / iso-FLOP	等参数 / 等算力：固定一个量画曲线
power law	幂律
compute-optimal	算力最优：给定 FLOP 预算下的最佳配置
co-design	软硬协同设计：按目标硬件定模型
NVFP4 / MXFP4	两种 4-bit 浮点微缩放格式
LPU · wafer-scale	Groq 的语言处理单元 · Cerebras 的整片晶圆芯片
causal / bidirectional attention	因果 / 双向注意力
NCCL	NVIDIA 的多卡通信库

字幕勘误

"KP cache" → KV cache；"PERS / parse / Parse a / parade models / per se" → Parcae；"Grock / Gro / rock" → Groq；"LPU Gro chips" → LPU（Groq 的芯片）；"Cerebra / cerebrus / cerebra strip" → Cerebras（wafer）；"Sabbonova" → SambaNova；"NVL sendme 2 grace Blackwell" → NVL72（Grace Blackwell）；"envy links" → NVLink；"flash tension" → FlashAttention；"GPD2" → GPT-2；"cloud code" → Claude Code；"Neotron" → Nemotron；"NV FP4 / MX FP4" → NVFP4 / MXFP4；"nickel calls" → NCCL calls；"aentic" → agentic；"loop transformers / loot" → looped transformers / loop；"micro kernels" → megakernels；"quen" → Qwen；"nano chat" → nanochat；"Deepseek foes" → DeepSeek folks；"at Ender" → at Together；"explicricity" → expressivity；"heristic" → heuristic；"gal" → GeLU；"prams" →

params; "isopram" → iso-parameter; "lo spikes" → loss spikes; "fall tolerant" → fault tolerant; "infancy chip" → inference chip; "multiprocessors" → multiprocessors (SM); "the speed matrix" → the B matrix; "Claude Mythos" 保留 (传闻中的模型名)。

带走的问题

1. decode 每生成一个 token 都要把整套权重读一遍：按第 3 节的式子，一个 bf16 的 1B 模型在 H100 (约 3.35 TB/s) 上"光速"一步是多少毫秒？72% 利用率对应多少？batch 从 1 涨到 16 时权重读取摊到 16 条序列上，为什么 megakernel 的相对收益会缩小，而 Dan 说 batch 到 17 要"从头再来"？
2. 冷热分流的前提是"平均 10 轮、10% 新请求"。换成批量翻译（每份文档只见一次）和一个长驻的 coding agent (KV cache 常热)，路由该怎么改？三级 KV cache 在这两种负载下的命中率会各自怎样？
3. Parcae 把非线性块 R 丢掉之后分析稳定性，实验里 R 装回去照样稳。什么情况下这个近似会失效——如果 R 的贡献占主导，谱半径小于 1 还够吗？第 3 讲的 QK-norm、z-loss 和这里的 A、B 重参数化，是同一类问题的两种答案，还是不同的病？
4. "数据越多，循环也该越多"是在固定参数量的曲线上得出的。把推理也算进去：循环 t 次，decode 一步的 FLOPs 是 t 倍，但权重字节不变——对带宽受限的 decode 是好是坏？对片上只有约 250 MB 的 Groq 那类芯片呢？
5. 三个 bug 都是"小概率 kernel 错误在万亿 token 规模下必然发生"。如果你负责一个推理服务，用什么指标能在用户抱怨之前抓到它们——completion 长度分布、非目标语言字符的比例、logits 里的 NaN 计数？哪些能自动化，哪些还得靠人看？